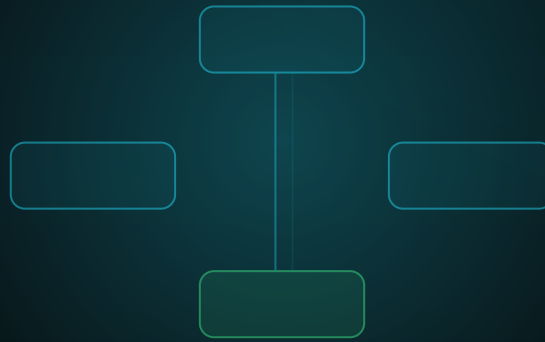


FROM SCHEMA TO SYSTEM

The Complete Data Backbone of the NEXUS-1 Digital Twin



TECHNICAL REFERENCE COMPANION

One database - seventeen sectors - fully keyed

SQL Server / EF Core friendly schema atlas for the NEXUS-1 companion system.

From Schema to System

The Complete Data Backbone of the NEXUS-1 Digital Twin SQL Server, Entity Framework Core, lookup discipline, foreign-key passports, auditability, and the seventeen-sector platform schema.

Grigorios Agathangelidis

A NEXUS-1 technical reference companion - demonstration build NX1-094C79E0A366D34F

This book describes a software schema for a browser-based demonstration and companion literature. It is not a licensed plant information model, not safety-class software, and not connected to any live facility.

Preface

Why this atlas exists

The earlier NEXUS-1 books explained engines: a plant model, an alarm-flood collapse method, a readable reinforcement-learning policy, and the act of turning tables into a digital twin. This book writes down the full data backbone those engines imply.

From Table to Twin taught the design journey: why a schema matters, why a bounded context can live as a SQL Server schema, why lookup tables should be real tables, and why audit is not decoration. *From Schema to System* is the reference volume that follows from that argument. It is less narrative and more atlas: every sector is named, every table earns its place, and every important foreign key is shown as a passport between contexts.

The book is deliberately practical. The DDL is SQL Server friendly. The model is EF Core friendly. The keys are named. The lookup tables are typed. The event records are append-only where they should be. The mutable tables carry lifecycle columns where they should. The audit trail is not a final chapter added because auditors exist; it is the closure that lets the rest of the platform be trusted.

One schema becomes a system only when its boundaries are named, its keys are stable, and its history can be replayed.

This is still NEXUS-1: a demonstrator, a teaching platform, and a software architecture exercise. Nothing here claims plant authority. The value is architectural: to show how a serious digital-twin database can be divided, keyed, queried, audited, and explained.

How to read this book

Read the first chapter, C.0, before jumping to any schema. It defines the naming rules, key conventions, lookup pattern, foreign-key discipline, and audit stamping used everywhere else.

For database readers

Follow the DDL, constraints, indexes and verification queries. The model is normalized enough to be serious without hiding behind abstractions.

For .NET / EF Core readers

Notice the table shapes that scaffold cleanly: explicit keys, named FKs, concrete lookup tables, and RowVersion columns for concurrency.

For architecture readers

Read the ER diagrams and passport maps. The central lesson is ownership: which schema owns which truth, and how the other schemas reference it.

For NEXUS-1 readers

Use the book as the backing atlas for the console screens: fleet, instrumentation, digital twin, alarms, RCA, RL, robotics, radiation, compliance, reporting and audit.

The chapters retain their C.0-C.17 numbering because the book also functions as the complete Appendix C reference for *From Table to Twin*. In this standalone volume, those appendix sections become the atlas chapters.

Contents

C.0	Schema-wide Conventions and Master Map	6
C.1	CorePlatform Schema	14
C.2	Security Schema	28
C.3	Organization Schema	53

C.4	ReactorFleet Schema	77
C.5	Instrumentation Schema	104
C.6	DigitalTwin Schema	128
C.7	AlarmManagement Schema	152
C.8	EventManager Schema	174
C.9	Maintenance Schema	195
C.10	RootCause Schema	216
C.11	ReinforcementLearning Schema	236
C.12	Robotics Schema	254
C.13	RadiationMonitoring Schema	273
C.14	EmergencyPreparedness Schema	294
C.15	Compliance Schema	310
C.16	Reporting Schema	326
C.17	Audit Schema	339
E	Epilogue - The schema holds the twin	351
R	References used to write this book	352
I	Indexes	353

The seventeen-sector map

The book follows dependency order. Core reference data comes first. The physical fleet appears before instrumentation. Instrumentation comes before the digital twin. Alarms and events come before maintenance, RCA and compliance. Audit closes the loop.

C.0 Schema-wide Conventions and Master Map	C.1 CorePlatform Schema	C.2 Security Schema
C.3 Organization Schema	C.4 ReactorFleet Schema	C.5 Instrumentation Schema
C.6 DigitalTwin Schema	C.7 AlarmManagement Schema	C.8 EventManager Schema
C.9 Maintenance Schema	C.10 RootCause Schema	C.11 ReinforcementLearning Schema
C.12 Robotics Schema	C.13 RadiationMonitoring Schema	C.14 EmergencyPreparedness Schema

C.15
Compliance Schema

C.16
Reporting Schema

C.17
Audit Schema

Appendix C.0

Schema-wide Conventions and Master Map

This appendix gathers the foundation rules for the full SQL schema of the NEXUS-1 data backbone. It is written for SQL Server and for clean reverse-engineering into Entity Framework Core.

The goal is not to minimise the table count. The goal is to make every table explicit, every key named, every cross-sector dependency visible, and every lookup enforceable by a real foreign key.

*One sector, one SQL schema.
One meaning per key.
One source of truth.
Foreign keys are the passports between sectors.*

The schema is demonstrator-grade and educational. It supports the NEXUS-1 companion console and the books in the series. It is not a licensed plant information model, not a safety-class database, and not a claim about any real facility.

C.0.1
SQL Server baseline

C.0.2
Sector schemas

C.0.3
Sector order

C.0.4
Standard mutable columns

C.0.5
Lookup table shape

C.0.6-C.0.9
Keys, FKs, indexes, ER diagram conventions

C.0.10
Master dependency map

C.0.11
Standing boundary

C.0.1 SQL Server baseline

The DDL in this appendix assumes Microsoft SQL Server and uses SQL Server-native conventions.

CONCERN	CONVENTION
Schema namespaces	One SQL schema per NEXUS-1 sector
Normal entity key	<code>INT IDENTITY(1,1)</code>
High-volume fact key	<code>BIGINT IDENTITY(1,1)</code>
Timestamps	<code>DATETIME2(7)</code> in UTC
Optimistic concurrency	<code>ROWVERSION</code> on mutable operational tables
Boolean values	<code>BIT NOT NULL DEFAULT 0/1</code>
Text values	<code>NVARCHAR</code> , never <code>VARCHAR</code>
Decimal engineering values	<code>DECIMAL(p,s)</code> where precision matters
Fast telemetry values	<code>FLOAT</code> only where physical approximation is acceptable
Delete behaviour	<code>NO ACTION</code> unless cascade is explicitly justified
Soft delete	<code>IsDeleted BIT NOT NULL DEFAULT 0</code> on mutable business tables
Audit stamping	<code>CreatedAtUtc</code> , <code>CreatedBy</code> , <code>ModifiedAtUtc</code> , <code>ModifiedBy</code>
Immutable event tables	Append-only, usually no soft delete
Lookup tables	One typed table per lookup category
Index names	<code>IX_<Schema>_<Table>_<ColumnList></code>
PK names	<code>PK_<Schema>_<Table></code>
FK names	<code>FK_<Schema>_<Table>_<PrincipalTable></code>
Unique constraints	<code>UQ_<Schema>_<Table>_<ColumnList></code>
Check constraints	<code>CK_<Schema>_<Table>_<RuleName></code>

C.0.2 Sector schemas

The database is divided into seventeen schemas. Each schema owns its own vocabulary. The schema name is therefore not decoration; it is part of the meaning.

```
CREATE SCHEMA CorePlatform;  
CREATE SCHEMA Security;  
CREATE SCHEMA Organization;  
CREATE SCHEMA ReactorFleet;  
CREATE SCHEMA DigitalTwin;  
CREATE SCHEMA Instrumentation;  
CREATE SCHEMA AlarmManagement;  
CREATE SCHEMA EventManagement;  
CREATE SCHEMA Maintenance;  
CREATE SCHEMA RootCause;  
CREATE SCHEMA ReinforcementLearning;  
CREATE SCHEMA Robotics;  
CREATE SCHEMA RadiationMonitoring;  
CREATE SCHEMA EmergencyPreparedness;  
CREATE SCHEMA Compliance;  
CREATE SCHEMA Reporting;  
CREATE SCHEMA Audit;
```

A Unit in ReactorFleet means a physical reactor unit. A unit of measure belongs in CorePlatform. An organisational unit belongs in Organization. The dot before the table name is part of the model.

C.0.3 Sector order

The appendix is built in dependency order, not alphabetically.

APPENDIX SECTION	SCHEMA	ROLE
C.1	CorePlatform	Shared platform reference data: units, settings, language, versioning
C.2	Security	Identity, roles, permissions, policies
C.3	Organization	Sites, plants, departments, teams, people
C.4	ReactorFleet	Fleet, units, reactors, core, rods, steam generators, plant equipment
C.5	Instrumentation	Tags, signals, sensors, historian readings, signal quality
C.6	DigitalTwin	Twin models, variables, bindings, snapshots, divergences
C.7	AlarmManagement	Alarm definitions, alarm events, floods, acknowledgements
C.8	EventManager	Incidents, operational events, near misses
C.9	Maintenance	Assets, inspections, degradation, work orders
C.10	RootCause	Causal graph, hypotheses, evidence, rejected candidates
C.11	ReinforcementLearning	Q-table, states, actions, training runs, episode logs
C.12	Robotics	Robots, vehicles, missions, capabilities, readiness
C.13	RadiationMonitoring	Radiation zones, monitors, dose readings, dosimeters
C.14	EmergencyPreparedness	Emergency plans, drills, exercises, evacuation routes
C.15	Compliance	Regulations, controls, audits, findings, evidence
C.16	Reporting	Report definitions, dashboards, exports, generated snapshots
C.17	Audit	Change log, history, lineage, append-only audit records

C.0.4 Standard mutable table columns

Most mutable business tables use the following lifecycle columns.

```

CreatedAtUtc    DATETIME2(7) NOT NULL
CONSTRAINT DF_<Table>_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
CreatedBy      NVARCHAR(100) NOT NULL,
ModifiedAtUtc   DATETIME2(7) NOT NULL
CONSTRAINT DF_<Table>_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedBy      NVARCHAR(100) NOT NULL,
IsDeleted       BIT NOT NULL
CONSTRAINT DF_<Table>_IsDeleted DEFAULT 0,
RowVersion      ROWVERSION NOT NULL

```

These columns are not repeated in every explanatory paragraph, but they are included in the DDL where appropriate.

IsDeleted is not used for every table. It belongs on mutable operational records where the row should disappear from normal application queries without being physically removed. It does not belong on immutable facts such as telemetry samples, alarm events, audit entries, or historical snapshots.

C.0.5 Standard lookup table shape

Lookup tables are typed. There is no generic **Lookup** table. Each category receives its own table so that foreign keys remain enforceable.

```
CREATE TABLE <Schema>.<LookupName> (  
  <LookupName>Id INT IDENTITY(1,1) NOT NULL,  
  Code NVARCHAR(50) NOT NULL,  
  Name NVARCHAR(150) NOT NULL,  
  Description NVARCHAR(500) NULL,  
  DisplayOrder INT NOT NULL  
    CONSTRAINT DF_<LookupName>_DisplayOrder DEFAULT 0,  
  IsActive BIT NOT NULL  
    CONSTRAINT DF_<LookupName>_IsActive DEFAULT 1,  
  CreatedAtUtc DATETIME2(7) NOT NULL  
    CONSTRAINT DF_<LookupName>_CreatedAtUtc DEFAULT SYSUTCDATETIME(),  
  ModifiedAtUtc DATETIME2(7) NOT NULL  
    CONSTRAINT DF_<LookupName>_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),  
  RowVersion ROWVERSION NOT NULL,  
  
  CONSTRAINT PK_<Schema>_<LookupName>  
    PRIMARY KEY (<LookupName>Id),  
  
  CONSTRAINT UQ_<Schema>_<LookupName>_Code  
    UNIQUE (Code)  
);
```

Examples

```
ReactorFleet.UnitStatus  
ReactorFleet.ReactorType  
Instrumentation.SignalType  
Instrumentation.SignalQuality  
AlarmManagement.AlarmSeverity  
AlarmManagement.AlarmState  
RootCause.CausalEdgeType  
RootCause.HypothesisStatus  
ReinforcementLearning.TrainingRunStatus  
Robotics.AssetStatus  
RadiationMonitoring.DoseUnitType  
Compliance.FindingSeverity
```

A two-value fact that will never be joined to, such as true/false, remains a **BIT**. A value that is displayed, ordered, translated, joined to, seeded, or audited becomes a lookup table.

C.0.6 Primary key conventions

Most entity tables use a surrogate integer key. High-volume append-only fact tables use `BIGINT IDENTITY`. Join tables use a composite primary key when the pair is the identity of the row.

```
-- Normal entity key
UnitId INT IDENTITY(1,1) NOT NULL

-- High-volume append-only fact key
MeasurementId BIGINT IDENTITY(1,1) NOT NULL
AlarmEventId BIGINT IDENTITY(1,1) NOT NULL
AuditEntryId BIGINT IDENTITY(1,1) NOT NULL

-- Composite key for a pure join table
CONSTRAINT PK_Security_RolePermission
PRIMARY KEY (RoleId, PermissionId)
```

Join tables receive their own identity key only when the relationship carries a lifecycle, status, history, comments, or audit trail of its own.

C.0.7 Foreign key conventions

Foreign keys are always explicit and named.

```
CONSTRAINT FK_Instrumentation_Signal_EngineeringUnit
FOREIGN KEY (EngineeringUnitId)
REFERENCES CorePlatform.EngineeringUnit(EngineeringUnitId)
```

Cross-schema foreign keys are not avoided. They are the controlled way one sector references another. The dependency must be visible in the DDL, visible in the ER diagram, and listed in the foreign-key mapping table for the sector.

The default delete behaviour is `NO ACTION`. Cascade delete is reserved for dependent details whose existence has no meaning without the parent.

```
TrainingRun -> Episode
ReportExport -> ReportExportFile
AuditBatch -> AuditEntry
```

Business records, measurements, alarms, incidents, and evidence are not cascade-deleted by default.

C.0.8 Index conventions

Every foreign key receives an index unless it is already covered by a primary key or unique constraint. Common lookup tables receive a unique index or unique constraint on `Code`.

```
-- Every foreign key receives an index unless already covered.
CREATE INDEX IX_Instrumentation_Signal_UnitId
ON Instrumentation.Signal(UnitId);

-- High-volume fact table query pattern.
CREATE INDEX IX_Instrumentation_Measurement_SignalId_TimestampUtc
ON Instrumentation.Measurement(SignalId, TimestampUtc DESC);
```

Telemetry and audit tables may use columnstore indexes later in the performance chapter, but the base schema remains readable and normal first.

C.0.9 ER diagram convention

Each sector includes two diagrams. The first is the sector ER diagram. It shows the principal tables inside the schema and the most important relationships. The second is the foreign-key passport map. It shows cross-schema dependencies.

Sector ER Diagram
inside one schema

Foreign-Key Passport Map
cross-schema
dependencies

FK Mapping Table
auditable relationship list

Every sector section follows the same evidence pattern: DDL, diagram, and mapping table.

solid arrow	same-schema relationship
dashed arrow	cross-schema foreign key
PK	primary key
FK	foreign key
LK	lookup table
FACT	append-only high-volume fact table
HIST	history or snapshot table

C.0.10 Master dependency map

At platform level, the strongest dependencies are shown below. The map is not a call graph. It is a referential-integrity map: the direction points from the table holding the foreign key to the table it depends on.

SECTOR	PRIMARY REFERENTIAL DEPENDENCIES
Security	CorePlatform
Organization	CorePlatform
ReactorFleet	Organization, CorePlatform
Instrumentation	ReactorFleet, CorePlatform
DigitalTwin	ReactorFleet, Instrumentation, CorePlatform
AlarmManagement	ReactorFleet, Instrumentation, Security
EventManagerment	ReactorFleet, AlarmManagement, Security
Maintenance	ReactorFleet, Organization, Security
RootCause	ReactorFleet, Instrumentation, AlarmManagement, EventManagement
ReinforcementLearning	ReactorFleet, DigitalTwin
Robotics	ReactorFleet, RadiationMonitoring
RadiationMonitoring	ReactorFleet, Organization, CorePlatform
EmergencyPreparedness	ReactorFleet, Organization, RadiationMonitoring
Compliance	Organization, ReactorFleet, EventManagement, Audit
Reporting	All read-side sectors
Audit	All write-side sectors

C.0.11 Standing boundary

This schema is a software model for the NEXUS-1 companion system. It is designed to support a browser-based demonstrator, explanatory books, SQL Server implementation, EF Core reverse engineering, and a serious discussion of digital-twin data modelling.

- It is not a licensed nuclear plant schema.
- It is not safety-class software.
- It is not connected to any live plant.
- It does not claim to describe any real facility.

The engineering value is the structure: explicit ownership, typed lookups, visible foreign keys, auditable history, reproducible seeds, and one database backbone for the plant, the alarm flood, the root-cause engine, the Q-table, and the digital twin.

Appendix C.1

CorePlatform Schema

The CorePlatform sector is the shared infrastructure of the NEXUS-1 data backbone. It owns facts that would otherwise be duplicated everywhere: configuration, feature flags, languages, translated resources, countries, regions, time zones, calendars, engineering units, and deployable version metadata.

This section expands the Core Platform room of the schema into a runnable SQL Server script. It follows the conventions of Appendix C.0: named constraints, explicit foreign keys, UTC timestamps, optimistic concurrency where appropriate, typed reference tables, and indexes shaped for EF Core and direct SQL use.

The Core Platform is deliberately modest. Its value is that every other sector can stop hardcoding the same facts.

C.1.1

Sector purpose

C.1.2

Full table list

C.1.3

Lookup and reference tables

C.1.4

SQL DDL

C.1.5

Indexes and seed examples

C.1.6

ER diagram

C.1.7

Foreign-key mapping

C.1.8

Boundary and next sector

C.1.1 Sector purpose

CorePlatform is a support sector, not a reactor sector. It does not describe pumps, valves, rods, alarms, incidents, or twin snapshots. It describes the shared vocabulary those sectors need before they can be consistent.

- Configuration values live here instead of inside application constants.
- Languages and resource strings live here instead of inside hardcoded UI labels.
- Countries, regions, time zones, and calendars live here instead of being repeated in Organization, Security, Reporting, and Emergency Preparedness.
- Engineering units live here so instrumentation, alarm thresholds, model variables, and reports all agree on symbols such as **MW**, **bar**, **pcm**, and **mSv/h**.

- Version rows live here so a support engineer can query what schema, console, seed data, and services are deployed.

C.1.2 Full table list

The CorePlatform sector contains eleven tables. Three have internal foreign keys; the remaining eight are independent reference or configuration roots.

CONFIGURATION

CorePlatform.AppSetting

PK AppSettingId
FK None

Mutable key-value setting. Runtime values that can be changed without redeploying the application.

CONFIGURATION

CorePlatform.SystemConfiguration

PK SystemConfigurationId
FK None

Versioned configuration contract. Structured JSON grouped by module and schema version.

CONFIGURATION

CorePlatform.FeatureFlag

PK FeatureFlagId
FK None

Environment feature gate. Switches capabilities on or off per environment, optionally with expiry.

LOCALE

CorePlatform.Language

PK LanguageId
FK None

Reference table. Supported BCP 47 languages for user interface and text resources.

LOCALE

CorePlatform.Localization

PK LocalizationId
FK LanguageId

Translated resource text. One value for each resource key and language.

GEOGRAPHY

CorePlatform.Country

PK CountryId
FK None

Reference table. ISO country catalogue used by organization, security, and compliance.

GEOGRAPHY

CorePlatform.Region

PK RegionId
FK CountryId

Reference table. Sub-national regions, provinces, states, prefectures, or administrative areas.

TIME

CorePlatform.TimeZone

PK TimeZoneId
FK None

Reference table. Canonical IANA time zones used for display and local scheduling.

TIME

CorePlatform.Calendar

PK CalendarId

FK TimeZoneId

Reference table. Named plant, shift, maintenance, and reporting calendars.

ENGINEERING

CorePlatform.EngineeringUnit

PK EngineeringUnitId

FK None

Reference table. Authoritative unit of measure catalogue for measurements and thresholds.

VERSIONING

CorePlatform.Version

PK VersionId

FK None

Deployment metadata for the console, schema, seed data, and backend services.

C.1.3 Lookup and reference tables

Not every CorePlatform table is a standard lookup table, but six of them behave as platform reference tables. They are small, seeded, joined frequently, and reused across sectors. They therefore receive real keys and unique natural codes.

TABLE	REFERENCE TABLE?	NATURAL KEY	TYPICAL VALUES	USED BY
Language	Yes	Code	en-GB , el-GR , fr-FR	Used by localization, user preferences, alarm texts, and reports.
Country	Yes	Iso2Code	GR , FR , US	Used by sites, jurisdictions, and compliance mapping.
Region	Yes	CountryId + Code	GR-C , FR-IDF	Dependent geographic subdivision.
TimeZone	Yes	IanaName	Europe/Athens	Used for local display and schedules; timestamps remain UTC.
Calendar	Yes	Code	OPS-24X7 , MAINT-WEEKDAY	Reference table for scheduling rules.
EngineeringUnit	Yes	Symbol	MW , bar , pcm	The unit catalogue consumed by instrumentation and alarm thresholds.

Why no generic lookup table? A single generic table would make `LanguageId`, `CountryId`, and `EngineeringUnitId` indistinguishable to the database. This design keeps each category typed, enforceable, and readable in EF Core. `AppSetting.ValueType` remains a check-constrained column because it has only four parser modes.

C.1.4 SQL DDL

The script below can be run after Appendix C.0 or independently against an empty database. It creates the schema if missing and then creates the eleven CorePlatform tables.

C.1.4.1 Schema

```
IF SCHEMA_ID(N'CorePlatform') IS NULL
    EXEC(N'CREATE SCHEMA CorePlatform');
GO
```

C.1.4.2 AppSetting

`AppSetting` holds runtime key-value values. It is intentionally flexible, but not untyped: the `ValueType` check tells the application how to parse the stored text.

```
CREATE TABLE CorePlatform.AppSetting (
    AppSettingId INT IDENTITY(1,1) NOT NULL,
    [Key] NVARCHAR(200) NOT NULL,
    [Value] NVARCHAR(2000) NOT NULL,
    ValueType NVARCHAR(20) NOT NULL,
    IsEncrypted BIT NOT NULL
    CONSTRAINT DF_CorePlatform_AppSetting_IsEncrypted DEFAULT 0,
    Description NVARCHAR(500) NULL,
    IsSystem BIT NOT NULL
    CONSTRAINT DF_CorePlatform_AppSetting_IsSystem DEFAULT 0,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_CorePlatform_AppSetting_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_CorePlatform_AppSetting_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL
    CONSTRAINT DF_CorePlatform_AppSetting_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_CorePlatform_AppSetting
        PRIMARY KEY (AppSettingId),

    CONSTRAINT UQ_CorePlatform_AppSetting_Key
        UNIQUE ([Key]),

    CONSTRAINT CK_CorePlatform_AppSetting_ValueType
        CHECK (ValueType IN (N'String', N'Int', N'Bool', N'Json'))
);
GO
```

C.1.4.3 SystemConfiguration

`SystemConfiguration` is the structured sibling of `AppSetting`. It stores versioned JSON contracts by module so the application can reject an incompatible configuration instead of silently misreading it.

```

CREATE TABLE CorePlatform.SystemConfiguration (
  SystemConfigurationId INT IDENTITY(1,1) NOT NULL,
  ModuleName            NVARCHAR(100) NOT NULL,
  ConfigurationKey       NVARCHAR(150) NOT NULL,
  ConfigurationJson      NVARCHAR(MAX) NOT NULL,
  SchemaVersion          INT NOT NULL,
  IsActive              BIT NOT NULL
  CONSTRAINT DF_CorePlatform_SystemConfiguration_IsActive DEFAULT 1,
  EffectiveFromUtc       DATETIME2(7) NOT NULL
  CONSTRAINT DF_CorePlatform_SystemConfiguration_EffectiveFromUtc DEFAULT SYSUTCDATETIME(),
  EffectiveToUtc         DATETIME2(7) NULL,
  Description            NVARCHAR(500) NULL,
  CreatedAtUtc           DATETIME2(7) NOT NULL
  CONSTRAINT DF_CorePlatform_SystemConfiguration_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  CreatedBy              NVARCHAR(100) NOT NULL,
  ModifiedAtUtc          DATETIME2(7) NOT NULL
  CONSTRAINT DF_CorePlatform_SystemConfiguration_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedBy             NVARCHAR(100) NOT NULL,
  IsDeleted              BIT NOT NULL
  CONSTRAINT DF_CorePlatform_SystemConfiguration_IsDeleted DEFAULT 0,
  RowVersion             ROWVERSION NOT NULL,

  CONSTRAINT PK_CorePlatform_SystemConfiguration
    PRIMARY KEY (SystemConfigurationId),

  CONSTRAINT UQ_CorePlatform_SystemConfiguration_Module_Key_Version
    UNIQUE (ModuleName, ConfigurationKey, SchemaVersion),

  CONSTRAINT CK_CorePlatform_SystemConfiguration_SchemaVersion_Positive
    CHECK (SchemaVersion > 0),

  CONSTRAINT CK_CorePlatform_SystemConfiguration_EffectiveWindow
    CHECK (EffectiveToUtc IS NULL OR EffectiveToUtc > EffectiveFromUtc),

  CONSTRAINT CK_CorePlatform_SystemConfiguration_Json
    CHECK (ISJSON(ConfigurationJson) = 1)
);
GO

```

C.1.4.4 FeatureFlag

FeatureFlag gates capabilities per environment. The optional expiry protects trial flags from accidentally becoming permanent.

```

CREATE TABLE CorePlatform.FeatureFlag (
  FeatureFlagId INT IDENTITY(1,1) NOT NULL,
  Code          NVARCHAR(80) NOT NULL,
  Name          NVARCHAR(150) NOT NULL,
  EnvironmentName NVARCHAR(50) NOT NULL
  CONSTRAINT DF_CorePlatform_FeatureFlag_EnvironmentName DEFAULT N'All',
  Description    NVARCHAR(500) NULL,
  IsEnabled      BIT NOT NULL
  CONSTRAINT DF_CorePlatform_FeatureFlag_IsEnabled DEFAULT 0,
  EnabledFromUtc DATETIME2(7) NULL,
  ExpiresAtUtc   DATETIME2(7) NULL,
  Owner          NVARCHAR(150) NULL,
  CreatedAtUtc   DATETIME2(7) NOT NULL
  CONSTRAINT DF_CorePlatform_FeatureFlag_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  CreatedBy      NVARCHAR(100) NOT NULL,
  ModifiedAtUtc  DATETIME2(7) NOT NULL
  CONSTRAINT DF_CorePlatform_FeatureFlag_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedBy     NVARCHAR(100) NOT NULL,
  IsDeleted      BIT NOT NULL
  CONSTRAINT DF_CorePlatform_FeatureFlag_IsDeleted DEFAULT 0,
  RowVersion     ROWVERSION NOT NULL,

  CONSTRAINT PK_CorePlatform_FeatureFlag
    PRIMARY KEY (FeatureFlagId),

  CONSTRAINT UQ_CorePlatform_FeatureFlag_Code_Environment
    UNIQUE (Code, EnvironmentName),

  CONSTRAINT CK_CorePlatform_FeatureFlag_ExpiryWindow

```

```

);
GO
CHECK (ExpiresAtUtc IS NULL OR EnabledFromUtc IS NULL OR ExpiresAtUtc > EnabledFromUtc)

```

C.1.4.5 Language and Localization

Language is the catalogue of supported languages. **Localization** stores one translated value per resource key and language.

```

CREATE TABLE CorePlatform.Language (
    LanguageId      INT IDENTITY(1,1) NOT NULL,
    Code            NVARCHAR(10) NOT NULL,
    Name            NVARCHAR(100) NOT NULL,
    NativeName      NVARCHAR(100) NOT NULL,
    IsRightToLeft   BIT NOT NULL
        CONSTRAINT DF_CorePlatform_Language_IsRightToLeft DEFAULT 0,
    IsDefault       BIT NOT NULL
        CONSTRAINT DF_CorePlatform_Language_IsDefault DEFAULT 0,
    IsActive       BIT NOT NULL
        CONSTRAINT DF_CorePlatform_Language_IsActive DEFAULT 1,
    DisplayOrder    INT NOT NULL
        CONSTRAINT DF_CorePlatform_Language_DisplayOrder DEFAULT 0,
    CreatedAtUtc    DATETIME2(7) NOT NULL
        CONSTRAINT DF_CorePlatform_Language_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc   DATETIME2(7) NOT NULL
        CONSTRAINT DF_CorePlatform_Language_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion      ROWVERSION NOT NULL,

    CONSTRAINT PK_CorePlatform_Language
        PRIMARY KEY (LanguageId),

    CONSTRAINT UQ_CorePlatform_Language_Code
        UNIQUE (Code)
);
GO

```

```

CREATE TABLE CorePlatform.Localization (
    LocalizationId  INT IDENTITY(1,1) NOT NULL,
    ResourceKey     NVARCHAR(300) NOT NULL,
    LanguageId     INT NOT NULL,
    [Value]        NVARCHAR(MAX) NOT NULL,
    SourceText     NVARCHAR(MAX) NULL,
    IsMachineTranslated BIT NOT NULL
        CONSTRAINT DF_CorePlatform_Localization_IsMachineTranslated DEFAULT 0,
    LastReviewedAtUtc DATETIME2(7) NULL,
    CreatedAtUtc    DATETIME2(7) NOT NULL
        CONSTRAINT DF_CorePlatform_Localization_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy      NVARCHAR(100) NOT NULL,
    ModifiedAtUtc   DATETIME2(7) NOT NULL
        CONSTRAINT DF_CorePlatform_Localization_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy     NVARCHAR(100) NOT NULL,
    IsDeleted      BIT NOT NULL
        CONSTRAINT DF_CorePlatform_Localization_IsDeleted DEFAULT 0,
    RowVersion      ROWVERSION NOT NULL,

    CONSTRAINT PK_CorePlatform_Localization
        PRIMARY KEY (LocalizationId),

    CONSTRAINT FK_CorePlatform_Localization_Language
        FOREIGN KEY (LanguageId)
        REFERENCES CorePlatform.Language(LanguageId),

    CONSTRAINT UQ_CorePlatform_Localization_ResourceKey_Language
        UNIQUE (ResourceKey, LanguageId)
);
GO

```

C.1.4.6 Country and Region

Country and **Region** provide the geographic reference used by Organization, Security, Compliance, Emergency Preparedness, and Reporting.

```
CREATE TABLE CorePlatform.Country (
    CountryId          INT IDENTITY(1,1) NOT NULL,
    Iso2Code           NCHAR(2) NOT NULL,
    Iso3Code           NCHAR(3) NOT NULL,
    NumericCode        NCHAR(3) NULL,
    Name               NVARCHAR(150) NOT NULL,
    OfficialName       NVARCHAR(250) NULL,
    IsActive           BIT NOT NULL
        CONSTRAINT DF_CorePlatform_Country_IsActive DEFAULT 1,
    DisplayOrder       INT NOT NULL
        CONSTRAINT DF_CorePlatform_Country_DisplayOrder DEFAULT 0,
    CreatedAtUtc       DATETIME2(7) NOT NULL
        CONSTRAINT DF_CorePlatform_Country_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc      DATETIME2(7) NOT NULL
        CONSTRAINT DF_CorePlatform_Country_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion         ROWVERSION NOT NULL,

    CONSTRAINT PK_CorePlatform_Country
        PRIMARY KEY (CountryId),

    CONSTRAINT UQ_CorePlatform_Country_Iso2Code
        UNIQUE (Iso2Code),

    CONSTRAINT UQ_CorePlatform_Country_Iso3Code
        UNIQUE (Iso3Code)
);
GO
```

```
CREATE TABLE CorePlatform.Region (
    RegionId          INT IDENTITY(1,1) NOT NULL,
    CountryId         INT NOT NULL,
    Code              NVARCHAR(30) NOT NULL,
    Name              NVARCHAR(150) NOT NULL,
    RegionType        NVARCHAR(50) NULL,
    IsActive          BIT NOT NULL
        CONSTRAINT DF_CorePlatform_Region_IsActive DEFAULT 1,
    DisplayOrder       INT NOT NULL
        CONSTRAINT DF_CorePlatform_Region_DisplayOrder DEFAULT 0,
    CreatedAtUtc       DATETIME2(7) NOT NULL
        CONSTRAINT DF_CorePlatform_Region_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc      DATETIME2(7) NOT NULL
        CONSTRAINT DF_CorePlatform_Region_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion         ROWVERSION NOT NULL,

    CONSTRAINT PK_CorePlatform_Region
        PRIMARY KEY (RegionId),

    CONSTRAINT FK_CorePlatform_Region_Country
        FOREIGN KEY (CountryId)
        REFERENCES CorePlatform.Country(CountryId),

    CONSTRAINT UQ_CorePlatform_Region_Country_Code
        UNIQUE (CountryId, Code)
);
GO
```

C.1.4.7 TimeZone and Calendar

TimeZone stores the canonical IANA name used for display and scheduling. **Calendar** defines named schedule rules such as 24x7 operations, weekday maintenance, or regulatory reporting calendars.


```

CREATE TABLE CorePlatform.TimeZone (
    TimeZoneId          INT IDENTITY(1,1) NOT NULL,
    IanaName             NVARCHAR(100) NOT NULL,
    DisplayName         NVARCHAR(150) NOT NULL,
    WindowsName         NVARCHAR(100) NULL,
    CurrentUtcOffsetMinutes SMALLINT NOT NULL,
    ObservesDst         BIT NOT NULL
    CONSTRAINT DF_CorePlatform_TimeZone_ObservesDst DEFAULT 0,
    IsActive           BIT NOT NULL
    CONSTRAINT DF_CorePlatform_TimeZone_IsActive DEFAULT 1,
    DisplayOrder       INT NOT NULL
    CONSTRAINT DF_CorePlatform_TimeZone_DisplayOrder DEFAULT 0,
    CreatedAtUtc       DATETIME2(7) NOT NULL
    CONSTRAINT DF_CorePlatform_TimeZone_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc      DATETIME2(7) NOT NULL
    CONSTRAINT DF_CorePlatform_TimeZone_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion         ROWVERSION NOT NULL,

    CONSTRAINT PK_CorePlatform_TimeZone
        PRIMARY KEY (TimeZoneId),

    CONSTRAINT UQ_CorePlatform_TimeZone_IanaName
        UNIQUE (IanaName),

    CONSTRAINT CK_CorePlatform_TimeZone_OffsetRange
        CHECK (CurrentUtcOffsetMinutes BETWEEN -840 AND 840)
);
GO

```

```

CREATE TABLE CorePlatform.Calendar (
    CalendarId          INT IDENTITY(1,1) NOT NULL,
    Code               NVARCHAR(50) NOT NULL,
    Name              NVARCHAR(150) NOT NULL,
    TimeZoneId        INT NOT NULL,
    CalendarType       NVARCHAR(30) NOT NULL,
    WorkingDaysMask    TINYINT NOT NULL
    CONSTRAINT DF_CorePlatform_Calendar_WorkingDaysMask DEFAULT 127,
    WorkingDayStart    TIME(0) NULL,
    WorkingDayEnd      TIME(0) NULL,
    Is24x7            BIT NOT NULL
    CONSTRAINT DF_CorePlatform_Calendar_Is24x7 DEFAULT 0,
    Description        NVARCHAR(500) NULL,
    IsActive          BIT NOT NULL
    CONSTRAINT DF_CorePlatform_Calendar_IsActive DEFAULT 1,
    CreatedAtUtc       DATETIME2(7) NOT NULL
    CONSTRAINT DF_CorePlatform_Calendar_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy         NVARCHAR(100) NOT NULL,
    ModifiedAtUtc      DATETIME2(7) NOT NULL
    CONSTRAINT DF_CorePlatform_Calendar_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy        NVARCHAR(100) NOT NULL,
    IsDeleted         BIT NOT NULL
    CONSTRAINT DF_CorePlatform_Calendar_IsDeleted DEFAULT 0,
    RowVersion         ROWVERSION NOT NULL,

    CONSTRAINT PK_CorePlatform_Calendar
        PRIMARY KEY (CalendarId),

    CONSTRAINT FK_CorePlatform_Calendar_TimeZone
        FOREIGN KEY (TimeZoneId)
        REFERENCES CorePlatform.TimeZone(TimeZoneId),

    CONSTRAINT UQ_CorePlatform_Calendar_Code
        UNIQUE (Code),

    CONSTRAINT CK_CorePlatform_Calendar_Type
        CHECK (CalendarType IN (N'General', N'Shift', N'Maintenance', N'Reporting', N'Regulatory')),

    CONSTRAINT CK_CorePlatform_Calendar_WorkingDaysMask
        CHECK (WorkingDaysMask BETWEEN 0 AND 127),

    CONSTRAINT CK_CorePlatform_Calendar_WorkingHours
        CHECK (Is24x7 = 1 OR (WorkingDayStart IS NOT NULL AND WorkingDayEnd IS NOT NULL AND WorkingDayEnd >

```

```
WorkingDayStart))
);
GO
```

C.1.4.8 EngineeringUnit

EngineeringUnit is the unit-of-measure passport for the whole platform. Instrumentation signals, alarm thresholds, model variables, and reports reference this table instead of storing free-text symbols.

```
CREATE TABLE CorePlatform.EngineeringUnit (
  EngineeringUnitId INT IDENTITY(1,1) NOT NULL,
  Symbol NVARCHAR(20) NOT NULL,
  Name NVARCHAR(100) NOT NULL,
  QuantityType NVARCHAR(50) NOT NULL,
  SiConversionFactor DECIMAL(28,12) NULL,
  SiConversionOffset DECIMAL(28,12) NULL,
  IsDimensionless BIT NOT NULL
  CONSTRAINT DF_CorePlatform_EngineeringUnit_IsDimensionless DEFAULT 0,
  IsActive BIT NOT NULL
  CONSTRAINT DF_CorePlatform_EngineeringUnit_IsActive DEFAULT 1,
  DisplayOrder INT NOT NULL
  CONSTRAINT DF_CorePlatform_EngineeringUnit_DisplayOrder DEFAULT 0,
  CreatedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_CorePlatform_EngineeringUnit_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_CorePlatform_EngineeringUnit_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  RowVersion ROWVERSION NOT NULL,

  CONSTRAINT PK_CorePlatform_EngineeringUnit
    PRIMARY KEY (EngineeringUnitId),

  CONSTRAINT UQ_CorePlatform_EngineeringUnit_Symbol
    UNIQUE (Symbol),

  CONSTRAINT CK_CorePlatform_EngineeringUnit_QuantityType
    CHECK (QuantityType IN (
      N'Power', N'ThermalPower', N'Temperature', N'Pressure', N'Reactivity',
      N'Flow', N'Frequency', N'Percentage', N'RadiationDoseRate',
      N'RadiationDose', N'CountRate', N'Mass', N'Time', N'Other'))
);
GO
```

C.1.4.9 Version

Version records deployed components: the console, database schema, seed data, API services, workers, and documentation bundles.

```
CREATE TABLE CorePlatform.Version (
  VersionId INT IDENTITY(1,1) NOT NULL,
  ComponentName NVARCHAR(120) NOT NULL,
  ComponentType NVARCHAR(50) NOT NULL,
  VersionNumber NVARCHAR(50) NOT NULL,
  BuildSignature NVARCHAR(100) NULL,
  GitCommit CHAR(40) NULL,
  SchemaMigration NVARCHAR(150) NULL,
  ReleaseDateUtc DATETIME2(7) NULL,
  ChangelogSummary NVARCHAR(1000) NULL,
  IsCurrent BIT NOT NULL
  CONSTRAINT DF_CorePlatform_Version_IsCurrent DEFAULT 0,
  CreatedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_CorePlatform_Version_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  CreatedBy NVARCHAR(100) NOT NULL,
  RowVersion ROWVERSION NOT NULL,

  CONSTRAINT PK_CorePlatform_Version
    PRIMARY KEY (VersionId),

  CONSTRAINT UQ_CorePlatform_Version_Component_Version
```

```

        UNIQUE (ComponentName, VersionNumber),

        CONSTRAINT CK_CorePlatform_Version_ComponentType
        CHECK (ComponentType IN (N'Console', N'Schema', N'SeedData', N'ApiService', N'Worker', N'Documentation'))
    );
GO

```

C.1.5 Indexes and seed examples

Every internal foreign key receives an index. Two filtered unique indexes enforce a single default language and a single current version per component.

```

CREATE INDEX IX_CorePlatform_Localization_LanguageId
ON CorePlatform.Localization(LanguageId);
GO

CREATE INDEX IX_CorePlatform_Localization_ResourceKey
ON CorePlatform.Localization(ResourceKey);
GO

CREATE INDEX IX_CorePlatform_Region_CountryId
ON CorePlatform.Region(CountryId);
GO

CREATE INDEX IX_CorePlatform_Calendar_TimeZoneId
ON CorePlatform.Calendar(TimeZoneId);
GO

CREATE INDEX IX_CorePlatform_EngineeringUnit_QuantityType
ON CorePlatform.EngineeringUnit(QuantityType);
GO

CREATE UNIQUE INDEX UX_CorePlatform_Language_Default
ON CorePlatform.Language(IsDefault)
WHERE IsDefault = 1;
GO

CREATE UNIQUE INDEX UX_CorePlatform_Version_Current_Component
ON CorePlatform.Version(ComponentName)
WHERE IsCurrent = 1;
GO

CREATE INDEX IX_CorePlatform_FeatureFlag_Enabled
ON CorePlatform.FeatureFlag(EnvironmentName, IsEnabled, ExpiresAtUtc)
WHERE IsDeleted = 0;
GO

```

C.1.5.1 Minimal seed examples

The seeds below are intentionally small. The complete seed-data catalogue belongs in Appendix H; this section only shows the shape.

```

-- Minimal demonstrator seeds. Full seed data belongs in Appendix H.
INSERT INTO CorePlatform.Language (Code, Name, NativeName, IsDefault, DisplayOrder)
VALUES (N'en-GB', N'English (United Kingdom)', N'English', 1, 10),
       (N'el-GR', N'Greek', N'Ελληνικά', 0, 20),
       (N'fr-FR', N'French', N'Français', 0, 30);

INSERT INTO CorePlatform.TimeZone (IanaName, DisplayName, WindowsName, CurrentUtcOffsetMinutes, ObservesDst)
VALUES (N'UTC', N'Coordinated Universal Time', N'UTC', 0, 0),
       (N'Europe/Athens', N'Athens', N'GTB Standard Time', 120, 1);

INSERT INTO CorePlatform.EngineeringUnit
(Symbol, Name, QuantityType, SiConversionFactor, SiConversionOffset, IsDimensionless, DisplayOrder)
VALUES (N'MW', N'megawatt electric', N'Power', 1000000.0, 0.0, 0, 10),
       (N'MWt', N'megawatt thermal', N'ThermalPower', 1000000.0, 0.0, 0, 20),
       (N'bar', N'bar', N'Pressure', 100000.0, 0.0, 0, 30),
       (N'°C', N'degree Celsius', N'Temperature', 1.0, 273.15, 0, 40),

```

```
(N'K', N'kelvin', N'Temperature', 1.0, 0.0, 0, 50),
(N'pcm', N'per cent mille', N'Reactivity', NULL, NULL, 1, 60),
(N'%' , N'percent', N'Percentage', 0.01, 0.0, 1, 70),
(N'Hz', N'hertz', N'Frequency', 1.0, 0.0, 0, 80),
(N'mSv/h', N'millisievert per hour', N'RadiationDoseRate', NULL, NULL, 0, 90);
```

GO

C.1.6 ER diagram

The CorePlatform ER diagram has three internal relationships: **Localization** depends on **Language**, **Region** depends on **Country**, and **Calendar** depends on **TimeZone**. The other tables are roots consumed by other sectors.

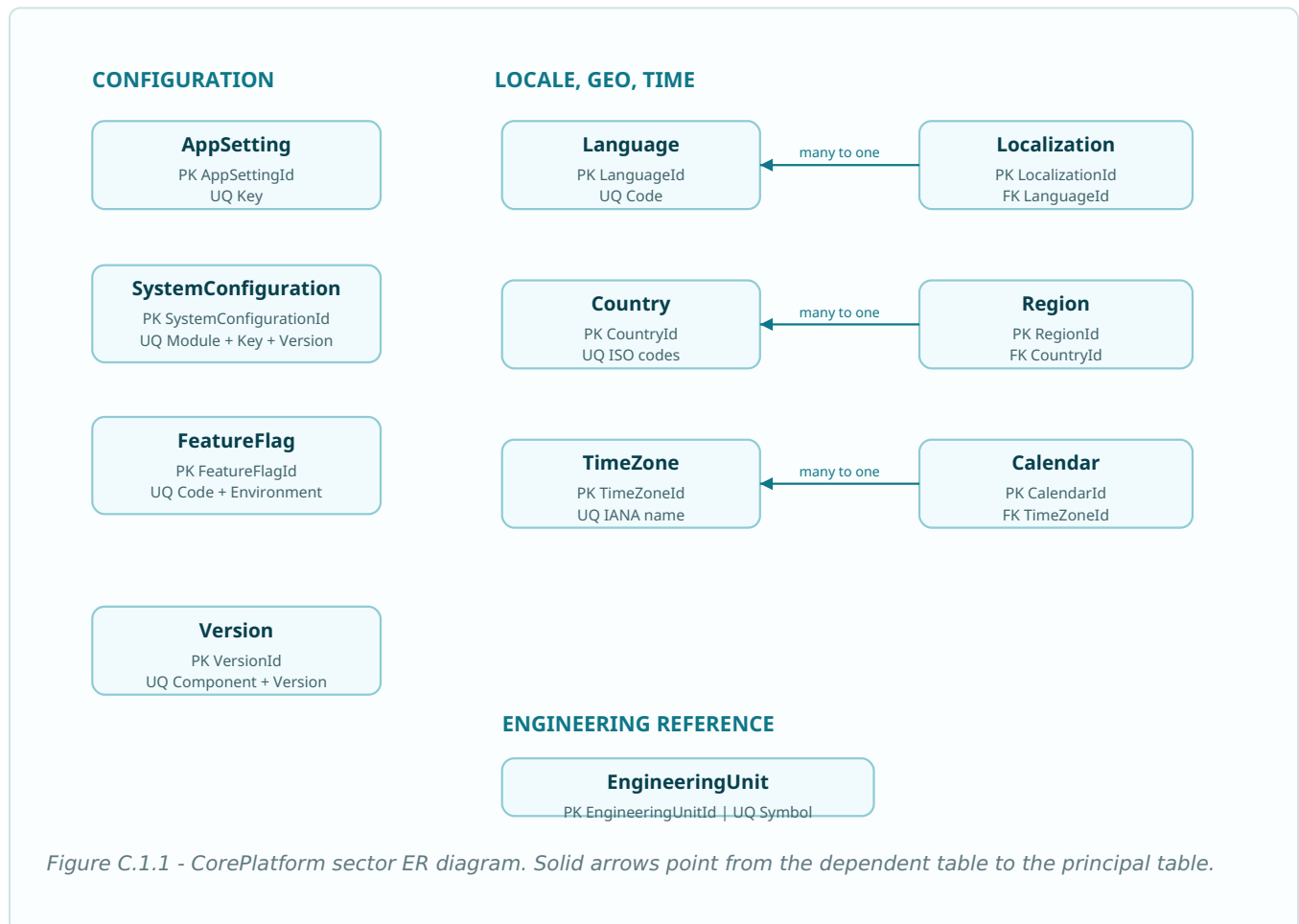


Figure C.1.1 - CorePlatform sector ER diagram. Solid arrows point from the dependent table to the principal table.

C.1.7 Foreign-key mapping

C.1.7.1 Internal CorePlatform foreign keys

FROM TABLE	FK COLUMN	TO TABLE	PK COLUMN	RELATIONSHIP
CorePlatform.Localization	LanguageId	CorePlatform.Language	LanguageId	Many localized resources per language.
CorePlatform.Region	CountryId	CorePlatform.Country	CountryId	Many regions per country.
CorePlatform.Calendar	TimeZonedId	CorePlatform.TimeZone	TimeZonedId	Many calendars may use the same time zone.

C.1.7.2 Incoming passports from later sectors

The following rows are not created by the CorePlatform script. They are the expected cross-schema foreign keys that later sectors declare when they consume CorePlatform reference data.

Security.UserPreference

FK LanguageId
TO CorePlatform.Language
User interface language.

Security.UserPreference

FK TimeZonedId
TO CorePlatform.TimeZone
User local display time zone.

Organization.Site

FK CountryId / RegionId / TimeZonedId
TO Country / Region / TimeZone
Physical site location and local operating clock.

Organization.Plant

FK CalendarId
TO CorePlatform.Calendar
Default operating calendar for a plant.

Instrumentation.Signal

FK EngineeringUnitId
TO CorePlatform.EngineeringUnit
Unit of measure for a signal value.

AlarmManagement.AlarmDefinitionText

FK LanguageId
TO CorePlatform.Language
Translated alarm text and operator instructions.

Reporting.ReportSchedule

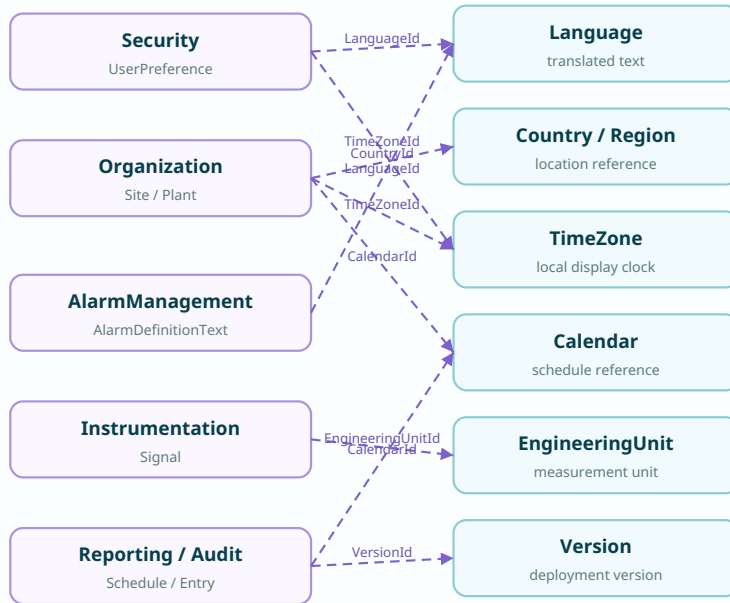
FK CalendarId / TimeZonedId
TO Calendar / TimeZone
Report timing and local wall-clock execution.

Audit.AuditEntry

FK VersionId
TO CorePlatform.Version
Optional link to deployed schema or application version.

CONSUMING SECTORS

CORE PLATFORM



Dashed arrows are incoming passports. They are declared in later sectors, but terminate in CorePlatform.

Figure C.1.2 - Foreign-key passport map. Dashed arrows show later sectors depending on CorePlatform reference rows.

C.1.8 Useful verification queries

After running the script and minimal seeds, these queries confirm that the sector behaves like a shared reference backbone rather than a collection of isolated tables.

```
-- All active units of measure used for instrumentation.
SELECT EngineeringUnitId, Symbol, Name, QuantityType
FROM CorePlatform.EngineeringUnit
WHERE IsActive = 1
ORDER BY QuantityType, DisplayOrder, Symbol;

-- Resolve a localized UI label with an English fallback.
DECLARE @ResourceKey NVARCHAR(300) = N'nav.digitalTwin';
DECLARE @LanguageCode NVARCHAR(10) = N'el-GR';

SELECT COALESCE(target.[Value], fallback.[Value]) AS DisplayText
FROM (SELECT 1 AS Dummy) AS d
LEFT JOIN CorePlatform.Language AS targetLang ON targetLang.Code = @LanguageCode
LEFT JOIN CorePlatform.Localization AS target
    ON target.ResourceKey = @ResourceKey AND target.LanguageId = targetLang.LanguageId
LEFT JOIN CorePlatform.Language AS fallbackLang ON fallbackLang.Code = N'en-GB'
LEFT JOIN CorePlatform.Localization AS fallback
    ON fallback.ResourceKey = @ResourceKey AND fallback.LanguageId = fallbackLang.LanguageId;

-- Which platform components are currently deployed?
SELECT ComponentName, ComponentType, VersionNumber, BuildSignature, ReleaseDateUtc
FROM CorePlatform.Version
WHERE IsCurrent = 1
ORDER BY ComponentType, ComponentName;
```

C.1.9 Boundary and next sector

Honest boundary. The CorePlatform sector is intentionally generic. It does not try to model plant physics, operator permissions, causal graphs, or alarm floods. Its job is smaller and stricter: provide the reusable reference facts that prevent the rest of the database from hardcoding strings, duplicating codes, or inventing parallel meanings for the same concept.

The next sector, **Security**, will be the first consumer of these passports. It will use platform languages, time zones, configuration, feature flags, and audit conventions while adding identity, roles, permissions, policies, and access decisions.

Appendix C.2

Security Schema

The Security sector owns accounts, roles, permissions, policies, sessions, tokens, API clients, and access-review records. It is deliberately separate from Organization: a login account is not the same thing as an employee, contractor, department, team, or site.

This section expands the second bounded context of the NEXUS-1 database. It is written as SQL Server DDL that reverse-engineers cleanly into EF Core and can be mapped to ASP.NET Core Identity with integer keys.

Security answers who can authenticate, what they can do, what policy allowed it, and which later records can prove who acted.

C.2.1 Sector purpose	C.2.2 Full table list
C.2.3 Lookup tables	C.2.4 SQL DDL
C.2.5 Indexes and seed examples	C.2.6 ER diagram
C.2.7 Foreign-key mapping	C.2.8 Boundary and next sector

C.2.1 Sector purpose

Security sits immediately after **CorePlatform** in the dependency order. It consumes platform reference data for user preferences, and then becomes a passport provider to almost every later sector.

- It owns authentication-facing users, roles, claims, external logins, sessions, login attempts, tokens, and API clients.
- It owns authorization-facing permissions, policies, role assignments, role policies, policy permissions, and user overrides.
- It owns governance-facing access reviews, so high-risk access can be periodically checked and recorded.

- It does not own departments, teams, sites, shifts, or employment details; those belong to **Organization**.

Design choice. The schema keeps ASP.NET Identity concepts, but names them as domain tables. **ApplicationUser** and **ApplicationRole** are still recognisable to .NET developers, while the permission and policy tables make authorization explicit instead of hiding it in code constants.

C.2.2 Full table list

The Security sector contains twenty-nine tables: eight lookup tables and twenty-one substantive tables. Several later sectors will depend on **Security.ApplicationUser** to record who acknowledged, created, approved, exported, closed, or changed something.

LOOKUP

Security.UserStatus

PK **UserStatusId**

FK None

Typed status catalogue for application users: active, locked, disabled, pending, retired.

LOOKUP

Security.RoleType

PK **RoleTypeId**

FK None

Classifies roles: operator, administrator, security, auditor, service account, emergency.

LOOKUP

Security.PermissionCategory

PK **PermissionCategoryId**

FK None

Groups permissions by domain: platform, alarm, twin, reporting, audit, security.

LOOKUP

Security.PolicyType

PK **PolicyTypeId**

FK None

Classifies policies: role policy, data-access policy, unit-scope policy, emergency policy.

LOOKUP

Security.SessionStatus

PK **SessionStatusId**

FK None

Session lifecycle values: active, expired, revoked, signed out.

LOOKUP

Security.LoginResult

PK **LoginResultId**

FK None

Login outcome catalogue used by append-only login attempts.

LOOKUP**Security.TokenType**

PK TokenTypeId

FK None

Token catalogue: refresh, password reset, email confirmation, MFA, API client.

LOOKUP**Security.AccessReviewStatus**

PK AccessReviewStatusId

FK None

Lifecycle of periodic access reviews.

IDENTITY**Security.ApplicationUser**

PK ApplicationUserId

FK UserStatusId

Login-capable account. Compatible with ASP.NET Core Identity using integer keys.

IDENTITY**Security.ApplicationRole**

PK ApplicationRoleId

FK RoleTypeId , ParentRoleId

Named role, optionally hierarchical, used to group permissions and policies.

IDENTITY**Security.UserRole**

PK ApplicationUserId + ApplicationRoleId

FK ApplicationUserId , ApplicationRoleId

Many-to-many assignment of users to roles, with expiry and assignment audit.

IDENTITY**Security.UserClaim**

PK UserClaimId

FK ApplicationUserId

ASP.NET Identity-style user claim extension point.

IDENTITY**Security.RoleClaim**

PK RoleClaimId

FK ApplicationRoleId

ASP.NET Identity-style role claim extension point.

IDENTITY**Security.ExternalLogin**

PK ExternalLoginId

FK ApplicationUserId

Optional external identity-provider login binding.

AUTHORIZATION**Security.Permission**

PK PermissionId

FK PermissionCategoryId

Atomic permission such as alarm acknowledgement, report export, or security administration.

AUTHORIZATION**Security.Policy**

PK PolicyId

FK PolicyTypeId

Named policy with optional JSON conditions and effective window.

AUTHORIZATION**Security.RolePermission**

PK ApplicationRoleId + PermissionId
FK ApplicationRoleId , PermissionId

Grants or denies a permission through a role.

AUTHORIZATION**Security.PolicyPermission**

PK PolicyId + PermissionId
FK PolicyId , PermissionId

Declares which permissions are required or enabled by a policy.

AUTHORIZATION**Security.RolePolicy**

PK ApplicationRoleId + PolicyId
FK ApplicationRoleId , PolicyId

Attaches a policy to a role.

AUTHORIZATION**Security.UserPolicyAssignment**

PK ApplicationUserId + PolicyId
FK ApplicationUserId , PolicyId

Direct policy assignment to a user, used for exceptions and temporary access.

AUTHORIZATION**Security.UserPermissionOverride**

PK UserPermissionOverrideId
FK ApplicationUserId , PermissionId

Explicit grant or deny override, optionally scoped to a resource key.

PREFERENCE**Security.UserPreference**

PK ApplicationUserId
FK LanguageId , TimeZoneId

One-to-one user preference row consuming CorePlatform language and time-zone passports.

RUNTIME**Security.UserSession**

PK UserSessionId
FK ApplicationUserId , SessionStatusId

Mutable session record for sign-in state, expiry, revocation and last-seen timestamp.

RUNTIME**Security.LoginAttempt**

PK LoginAttemptId
FK ApplicationUserId , LoginResultId

Append-only authentication attempt log.

SERVICE**Security.ApiClient**

PK ApiClientId
FK OwnerUserId

Machine or integration client that can request tokens.

SERVICE**Security.ApiClientSecret**

PK ApiClientSecretId
FK ApiClientId

Hashed client secrets with validity window and revocation.

TOKEN

Security.SecurityToken

PK SecurityTokenId

FK TokenTypeId , ApplicationUserId ,
ApiClientId

Hashed refresh, reset, confirmation, MFA, or API tokens.

GOVERNANCE

Security.AccessReview

PK AccessReviewId

FK AccessReviewStatusId , user FKs

Periodic review campaign for users, roles, policies, and high-risk permissions.

GOVERNANCE

Security.AccessReviewItem

PK AccessReviewItemId

FK AccessReviewId , user/role/permission FKs

One reviewed entitlement inside an access-review campaign.

C.2.3 Lookup tables

The lookup tables use the standard shape defined in Appendix C.0: integer key, code, name, description, display order, active flag, timestamps, and row version. They are typed tables, not rows in one generic lookup table.

LOOKUP TABLE	TYPICAL SEED CODES	MAIN CONSUMER
UserStatus	ACTIVE , LOCKED , DISABLED , PENDING	Referenced by ApplicationUser .
RoleType	OPERATOR , ADMIN , AUDITOR , SERVICE	Referenced by ApplicationRole .
PermissionCategory	ALARM , TWIN , SECURITY , REPORTING	Referenced by Permission .
PolicyType	ROLE , DATA_ACCESS , UNIT_SCOPE , EMERGENCY	Referenced by Policy .
SessionStatus	ACTIVE , EXPIRED , REVOKED , SIGNED_OUT	Referenced by UserSession .
LoginResult	SUCCESS , BAD_PASSWORD , LOCKED_OUT , MFA_REQUIRED	Referenced by LoginAttempt .
TokenType	REFRESH , PASSWORD_RESET , MFA , API_CLIENT	Referenced by SecurityToken .
AccessReviewStatus	DRAFT , OPEN , COMPLETED , CANCELLED	Referenced by AccessReview .

C.2.4 SQL DDL

C.2.4.1 Schema

```
IF SCHEMA_ID(N'Security') IS NULL
    EXEC(N'CREATE SCHEMA Security');
GO
```

C.2.4.2 Security lookup tables

```
CREATE TABLE Security.UserStatus (
    UserStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Security_UserStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Security_UserStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_UserStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_UserStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Security_UserStatus
        PRIMARY KEY (UserStatusId),

    CONSTRAINT UQ_Security_UserStatus_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Security.RoleType (
    RoleTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Security_RoleType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Security_RoleType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_RoleType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_RoleType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Security_RoleType
        PRIMARY KEY (RoleTypeId),

    CONSTRAINT UQ_Security_RoleType_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Security.PermissionCategory (
    PermissionCategoryId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Security_PermissionCategory_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Security_PermissionCategory_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_PermissionCategory_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_PermissionCategory_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Security_PermissionCategory
```

```

        PRIMARY KEY (PermissionCategoryId),

        CONSTRAINT UQ_Security_PermissionCategory_Code
            UNIQUE (Code)
    );
GO

CREATE TABLE Security.PolicyType (
    PolicyTypeId          INT IDENTITY(1,1) NOT NULL,
    Code                  NVARCHAR(50) NOT NULL,
    Name                  NVARCHAR(150) NOT NULL,
    Description            NVARCHAR(500) NULL,
    DisplayOrder          INT NOT NULL
        CONSTRAINT DF_Security_PolicyType_DisplayOrder DEFAULT 0,
    IsActive              BIT NOT NULL
        CONSTRAINT DF_Security_PolicyType_IsActive DEFAULT 1,
    CreatedAtUtc           DATETIME2(7) NOT NULL
        CONSTRAINT DF_Security_PolicyType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc           DATETIME2(7) NOT NULL
        CONSTRAINT DF_Security_PolicyType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion             ROWVERSION NOT NULL,

    CONSTRAINT PK_Security_PolicyType
        PRIMARY KEY (PolicyTypeId),

    CONSTRAINT UQ_Security_PolicyType_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Security.SessionStatus (
    SessionStatusId       INT IDENTITY(1,1) NOT NULL,
    Code                  NVARCHAR(50) NOT NULL,
    Name                  NVARCHAR(150) NOT NULL,
    Description            NVARCHAR(500) NULL,
    DisplayOrder          INT NOT NULL
        CONSTRAINT DF_Security_SessionStatus_DisplayOrder DEFAULT 0,
    IsActive              BIT NOT NULL
        CONSTRAINT DF_Security_SessionStatus_IsActive DEFAULT 1,
    CreatedAtUtc           DATETIME2(7) NOT NULL
        CONSTRAINT DF_Security_SessionStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc           DATETIME2(7) NOT NULL
        CONSTRAINT DF_Security_SessionStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion             ROWVERSION NOT NULL,

    CONSTRAINT PK_Security_SessionStatus
        PRIMARY KEY (SessionStatusId),

    CONSTRAINT UQ_Security_SessionStatus_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Security.LoginResult (
    LoginResultId         INT IDENTITY(1,1) NOT NULL,
    Code                  NVARCHAR(50) NOT NULL,
    Name                  NVARCHAR(150) NOT NULL,
    Description            NVARCHAR(500) NULL,
    DisplayOrder          INT NOT NULL
        CONSTRAINT DF_Security_LoginResult_DisplayOrder DEFAULT 0,
    IsActive              BIT NOT NULL
        CONSTRAINT DF_Security_LoginResult_IsActive DEFAULT 1,
    CreatedAtUtc           DATETIME2(7) NOT NULL
        CONSTRAINT DF_Security_LoginResult_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc           DATETIME2(7) NOT NULL
        CONSTRAINT DF_Security_LoginResult_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion             ROWVERSION NOT NULL,

    CONSTRAINT PK_Security_LoginResult
        PRIMARY KEY (LoginResultId),

    CONSTRAINT UQ_Security_LoginResult_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Security.TokenType (
    TokenTypeId           INT IDENTITY(1,1) NOT NULL,
    Code                  NVARCHAR(50) NOT NULL,
    Name                  NVARCHAR(150) NOT NULL,

```

```

Description    NVARCHAR(500) NULL,
DisplayOrder   INT NOT NULL
    CONSTRAINT DF_Security_TokenType_DisplayOrder DEFAULT 0,
IsActive       BIT NOT NULL
    CONSTRAINT DF_Security_TokenType_IsActive DEFAULT 1,
CreatedAtUtc   DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_TokenType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc  DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_TokenType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion     ROWVERSION NOT NULL,

CONSTRAINT PK_Security_TokenType
    PRIMARY KEY (TokenTypeId),

CONSTRAINT UQ_Security_TokenType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Security.AccessReviewStatus (
    AccessReviewStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Security_AccessReviewStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Security_AccessReviewStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_AccessReviewStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_AccessReviewStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_Security_AccessReviewStatus
    PRIMARY KEY (AccessReviewStatusId),

CONSTRAINT UQ_Security_AccessReviewStatus_Code
    UNIQUE (Code)
);
GO

```

C.2.4.3 ApplicationUser

`ApplicationUser` is the login-capable account. It carries the familiar ASP.NET Identity fields but keeps the NEXUS-1 audit, soft-delete, and row-version conventions.

```

CREATE TABLE Security.ApplicationUser (
    ApplicationUserId INT IDENTITY(1,1) NOT NULL,
    UserStatusId INT NOT NULL,
    UserName NVARCHAR(256) NOT NULL,
    NormalizedUserName NVARCHAR(256) NOT NULL,
    Email NVARCHAR(256) NULL,
    NormalizedEmail NVARCHAR(256) NULL,
    EmailConfirmed BIT NOT NULL
    CONSTRAINT DF_Security_ApplicationUser_EmailConfirmed DEFAULT 0,
    PasswordHash NVARCHAR(MAX) NULL,
    SecurityStamp NVARCHAR(100) NULL,
    ConcurrencyStamp NVARCHAR(100) NULL,
    PhoneNumber NVARCHAR(50) NULL,
    PhoneNumberConfirmed BIT NOT NULL
    CONSTRAINT DF_Security_ApplicationUser_PhoneConfirmed DEFAULT 0,
    TwoFactorEnabled BIT NOT NULL
    CONSTRAINT DF_Security_ApplicationUser_TwoFactorEnabled DEFAULT 0,
    LockoutEndUtc DATETIME2(7) NULL,
    LockoutEnabled BIT NOT NULL
    CONSTRAINT DF_Security_ApplicationUser_LockoutEnabled DEFAULT 1,
    AccessFailedCount INT NOT NULL
    CONSTRAINT DF_Security_ApplicationUser_AccessFailedCount DEFAULT 0,
    DisplayName NVARCHAR(200) NOT NULL,
    GivenName NVARCHAR(100) NULL,
    FamilyName NVARCHAR(100) NULL,
    IsServiceAccount BIT NOT NULL
    CONSTRAINT DF_Security_ApplicationUser_IsServiceAccount DEFAULT 0,
    LastLoginAtUtc DATETIME2(7) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_ApplicationUser_CreatedAtUtc DEFAULT SYSUTCDATETIME(),

```

```

    CreatedBy          NVARCHAR(100) NOT NULL,
    ModifiedAtUtc      DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_ApplicationUser_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy          NVARCHAR(100) NOT NULL,
    IsDeleted           BIT NOT NULL
    CONSTRAINT DF_Security_ApplicationUser_IsDeleted DEFAULT 0,
    RowVersion          ROWVERSION NOT NULL,

    CONSTRAINT PK_Security_ApplicationUser
    PRIMARY KEY (ApplicationUserId),

    CONSTRAINT FK_Security_ApplicationUser_UserStatus
    FOREIGN KEY (UserStatusId)
    REFERENCES Security.UserStatus(UserStatusId),

    CONSTRAINT UQ_Security_ApplicationUser_NormalizedUserName
    UNIQUE (NormalizedUserName),

    CONSTRAINT CK_Security_ApplicationUser_AccessFailedCount
    CHECK (AccessFailedCount >= 0)
);
GO

```

C.2.4.4 ApplicationRole

ApplicationRole supports role hierarchy through **ParentRoleId**. That allows a broad operator role to parent more specific roles without merging their meanings.

```

CREATE TABLE Security.ApplicationRole (
    ApplicationRoleId INT IDENTITY(1,1) NOT NULL,
    RoleTypeId        INT NOT NULL,
    ParentRoleId       INT NULL,
    Name               NVARCHAR(256) NOT NULL,
    NormalizedName     NVARCHAR(256) NOT NULL,
    Description         NVARCHAR(500) NULL,
    IsBuiltIn           BIT NOT NULL
    CONSTRAINT DF_Security_ApplicationRole_IsBuiltIn DEFAULT 0,
    IsActive            BIT NOT NULL
    CONSTRAINT DF_Security_ApplicationRole_IsActive DEFAULT 1,
    CreatedAtUtc        DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_ApplicationRole_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy           NVARCHAR(100) NOT NULL,
    ModifiedAtUtc       DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_ApplicationRole_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy          NVARCHAR(100) NOT NULL,
    IsDeleted           BIT NOT NULL
    CONSTRAINT DF_Security_ApplicationRole_IsDeleted DEFAULT 0,
    RowVersion          ROWVERSION NOT NULL,

    CONSTRAINT PK_Security_ApplicationRole
    PRIMARY KEY (ApplicationRoleId),

    CONSTRAINT FK_Security_ApplicationRole_RoleType
    FOREIGN KEY (RoleTypeId)
    REFERENCES Security.RoleType(RoleTypeId),

    CONSTRAINT FK_Security_ApplicationRole_ParentRole
    FOREIGN KEY (ParentRoleId)
    REFERENCES Security.ApplicationRole(ApplicationRoleId),

    CONSTRAINT UQ_Security_ApplicationRole_NormalizedName
    UNIQUE (NormalizedName)
);
GO

```

C.2.4.5 Permission and Policy

Permission is atomic. **Policy** is a named rule or condition set that can group permissions and carry optional JSON conditions.

```

CREATE TABLE Security.Permission (
    PermissionId        INT IDENTITY(1,1) NOT NULL,
    PermissionCategoryId INT NOT NULL,

```



```

Code                NVARCHAR(120) NOT NULL,
Name                NVARCHAR(180) NOT NULL,
Description          NVARCHAR(700) NULL,
ResourceType        NVARCHAR(80) NULL,
ActionName           NVARCHAR(80) NOT NULL,
IsSafetyRelevant     BIT NOT NULL
    CONSTRAINT DF_Security_Permission_IsSafetyRelevant DEFAULT 0,
IsActive            BIT NOT NULL
    CONSTRAINT DF_Security_Permission_IsActive DEFAULT 1,
CreatedAtUtc         DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_Permission_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
CreatedBy            NVARCHAR(100) NOT NULL,
ModifiedAtUtc        DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_Permission_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedBy           NVARCHAR(100) NOT NULL,
IsDeleted            BIT NOT NULL
    CONSTRAINT DF_Security_Permission_IsDeleted DEFAULT 0,
RowVersion           ROWVERSION NOT NULL,

CONSTRAINT PK_Security_Permission
    PRIMARY KEY (PermissionId),

CONSTRAINT FK_Security_Permission_PermissionCategory
    FOREIGN KEY (PermissionCategoryId)
    REFERENCES Security.PermissionCategory(PermissionCategoryId),

CONSTRAINT UQ_Security_Permission_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Security.Policy (
    PolicyId          INT IDENTITY(1,1) NOT NULL,
    PolicyTypeId       INT NOT NULL,
    Code               NVARCHAR(100) NOT NULL,
    Name               NVARCHAR(180) NOT NULL,
    Description         NVARCHAR(700) NULL,
    ConditionJson       NVARCHAR(MAX) NULL,
    Priority            INT NOT NULL
    CONSTRAINT DF_Security_Policy_Priority DEFAULT 100,
    IsEnabled          BIT NOT NULL
    CONSTRAINT DF_Security_Policy_IsEnabled DEFAULT 1,
    EffectiveFromUtc    DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_Policy_EffectiveFromUtc DEFAULT SYSUTCDATETIME(),
    EffectiveToUtc      DATETIME2(7) NULL,
    CreatedAtUtc        DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_Policy_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy           NVARCHAR(100) NOT NULL,
    ModifiedAtUtc        DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_Policy_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy          NVARCHAR(100) NOT NULL,
    IsDeleted           BIT NOT NULL
    CONSTRAINT DF_Security_Policy_IsDeleted DEFAULT 0,
    RowVersion          ROWVERSION NOT NULL,

CONSTRAINT PK_Security_Policy
    PRIMARY KEY (PolicyId),

CONSTRAINT FK_Security_Policy_PolicyType
    FOREIGN KEY (PolicyTypeId)
    REFERENCES Security.PolicyType(PolicyTypeId),

CONSTRAINT UQ_Security_Policy_Code
    UNIQUE (Code),

CONSTRAINT CK_Security_Policy_EffectiveWindow
    CHECK (EffectiveToUtc IS NULL OR EffectiveToUtc > EffectiveFromUtc),

CONSTRAINT CK_Security_Policy_ConditionJson
    CHECK (ConditionJson IS NULL OR ISJSON(ConditionJson) = 1)
);
GO

```

C.2.4.6 Identity extension tables

Claims and external login rows are kept explicit for compatibility with .NET identity workflows and future federation.

```

CREATE TABLE Security.UserClaim (
    UserClaimId INT IDENTITY(1,1) NOT NULL,
    ApplicationUserId INT NOT NULL,
    ClaimType NVARCHAR(250) NOT NULL,
    ClaimValue NVARCHAR(1000) NOT NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_UserClaim_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,

    CONSTRAINT PK_Security_UserClaim
        PRIMARY KEY (UserClaimId),

    CONSTRAINT FK_Security_UserClaim_ApplicationUser
        FOREIGN KEY (ApplicationUserId)
        REFERENCES Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE Security.RoleClaim (
    RoleClaimId INT IDENTITY(1,1) NOT NULL,
    ApplicationRoleId INT NOT NULL,
    ClaimType NVARCHAR(250) NOT NULL,
    ClaimValue NVARCHAR(1000) NOT NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_RoleClaim_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,

    CONSTRAINT PK_Security_RoleClaim
        PRIMARY KEY (RoleClaimId),

    CONSTRAINT FK_Security_RoleClaim_ApplicationRole
        FOREIGN KEY (ApplicationRoleId)
        REFERENCES Security.ApplicationRole(ApplicationRoleId)
);
GO

CREATE TABLE Security.ExternalLogin (
    ExternalLoginId INT IDENTITY(1,1) NOT NULL,
    ApplicationUserId INT NOT NULL,
    LoginProvider NVARCHAR(128) NOT NULL,
    ProviderKey NVARCHAR(256) NOT NULL,
    ProviderDisplayName NVARCHAR(256) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_ExternalLogin_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    LastUsedAtUtc DATETIME2(7) NULL,

    CONSTRAINT PK_Security_ExternalLogin
        PRIMARY KEY (ExternalLoginId),

    CONSTRAINT FK_Security_ExternalLogin_ApplicationUser
        FOREIGN KEY (ApplicationUserId)
        REFERENCES Security.ApplicationUser(ApplicationUserId),

    CONSTRAINT UQ_Security_ExternalLogin_Provider_Key
        UNIQUE (LoginProvider, ProviderKey)
);
GO

```

C.2.4.7 Role, permission, and policy joins

Join tables use composite primary keys where the relationship itself is the identity of the row. Expiry columns make temporary access possible without deleting history.

```

CREATE TABLE Security.UserRole (
    ApplicationUserId INT NOT NULL,
    ApplicationRoleId INT NOT NULL,
    AssignedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_UserRole_AssignedAtUtc DEFAULT SYSUTCDATETIME(),
    AssignedByUserId INT NULL,
    ExpiresAtUtc DATETIME2(7) NULL,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Security_UserRole_IsActive DEFAULT 1,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Security_UserRole
        PRIMARY KEY (ApplicationUserId, ApplicationRoleId),

```

```

CONSTRAINT FK_Security_UserRole_ApplicationUser
    FOREIGN KEY (ApplicationUserId)
    REFERENCES Security.ApplicationUser(ApplicationUserId),

CONSTRAINT FK_Security_UserRole_ApplicationRole
    FOREIGN KEY (ApplicationRoleId)
    REFERENCES Security.ApplicationRole(ApplicationRoleId),

CONSTRAINT FK_Security_UserRole_AssignedByUser
    FOREIGN KEY (AssignedByUserId)
    REFERENCES Security.ApplicationUser(ApplicationUserId),

CONSTRAINT CK_Security_UserRole_Expiry
    CHECK (ExpiresAtUtc IS NULL OR ExpiresAtUtc > AssignedAtUtc)
);
GO

CREATE TABLE Security.RolePermission (
    ApplicationRoleId INT NOT NULL,
    PermissionId      INT NOT NULL,
    IsGranted         BIT NOT NULL
    CONSTRAINT DF_Security_RolePermission_IsGranted DEFAULT 1,
    GrantedAtUtc      DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_RolePermission_GrantedAtUtc DEFAULT SYSUTCDATETIME(),
    GrantedByUserId   INT NULL,
    ExpiresAtUtc      DATETIME2(7) NULL,
    RowVersion        ROWVERSION NOT NULL,

    CONSTRAINT PK_Security_RolePermission
        PRIMARY KEY (ApplicationRoleId, PermissionId),

    CONSTRAINT FK_Security_RolePermission_ApplicationRole
        FOREIGN KEY (ApplicationRoleId)
        REFERENCES Security.ApplicationRole(ApplicationRoleId),

    CONSTRAINT FK_Security_RolePermission_Permission
        FOREIGN KEY (PermissionId)
        REFERENCES Security.Permission(PermissionId),

    CONSTRAINT FK_Security_RolePermission_GrantedByUser
        FOREIGN KEY (GrantedByUserId)
        REFERENCES Security.ApplicationUser(ApplicationUserId),

    CONSTRAINT CK_Security_RolePermission_Expiry
        CHECK (ExpiresAtUtc IS NULL OR ExpiresAtUtc > GrantedAtUtc)
);
GO

CREATE TABLE Security.PolicyPermission (
    PolicyId          INT NOT NULL,
    PermissionId      INT NOT NULL,
    IsRequired        BIT NOT NULL
    CONSTRAINT DF_Security_PolicyPermission_IsRequired DEFAULT 1,
    CreatedAtUtc      DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_PolicyPermission_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy         NVARCHAR(100) NOT NULL,
    RowVersion        ROWVERSION NOT NULL,

    CONSTRAINT PK_Security_PolicyPermission
        PRIMARY KEY (PolicyId, PermissionId),

    CONSTRAINT FK_Security_PolicyPermission_Policy
        FOREIGN KEY (PolicyId)
        REFERENCES Security.Policy(PolicyId),

    CONSTRAINT FK_Security_PolicyPermission_Permission
        FOREIGN KEY (PermissionId)
        REFERENCES Security.Permission(PermissionId)
);
GO

CREATE TABLE Security.RolePolicy (
    ApplicationRoleId INT NOT NULL,
    PolicyId          INT NOT NULL,
    AssignedAtUtc     DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_RolePolicy_AssignedAtUtc DEFAULT SYSUTCDATETIME(),
    AssignedByUserId  INT NULL,
    IsActive          BIT NOT NULL
    CONSTRAINT DF_Security_RolePolicy_IsActive DEFAULT 1,
    RowVersion        ROWVERSION NOT NULL,

    CONSTRAINT PK_Security_RolePolicy
        PRIMARY KEY (ApplicationRoleId, PolicyId),

```

```

CONSTRAINT FK_Security_RolePolicy_ApplicationRole
    FOREIGN KEY (ApplicationRoleId)
    REFERENCES Security.ApplicationRole(ApplicationRoleId),

CONSTRAINT FK_Security_RolePolicy_Policy
    FOREIGN KEY (PolicyId)
    REFERENCES Security.Policy(PolicyId),

CONSTRAINT FK_Security_RolePolicy_AssignedByUser
    FOREIGN KEY (AssignedById)
    REFERENCES Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE Security.UserPolicyAssignment (
    ApplicationUserId INT NOT NULL,
    PolicyId          INT NOT NULL,
    AssignedAtUtc     DATETIME2(7) NOT NULL,
    CONSTRAINT DF_Security_UserPolicyAssignment_AssignedAtUtc DEFAULT SYSUTCDATETIME(),
    AssignedByUserId  INT NULL,
    ExpiresAtUtc      DATETIME2(7) NULL,
    IsActive          BIT NOT NULL,
    CONSTRAINT DF_Security_UserPolicyAssignment_IsActive DEFAULT 1,
    RowVersion        ROWVERSION NOT NULL,

    CONSTRAINT PK_Security_UserPolicyAssignment
        PRIMARY KEY (ApplicationUserId, PolicyId),

    CONSTRAINT FK_Security_UserPolicyAssignment_ApplicationUser
        FOREIGN KEY (ApplicationUserId)
        REFERENCES Security.ApplicationUser(ApplicationUserId),

    CONSTRAINT FK_Security_UserPolicyAssignment_Policy
        FOREIGN KEY (PolicyId)
        REFERENCES Security.Policy(PolicyId),

    CONSTRAINT FK_Security_UserPolicyAssignment_AssignedByUser
        FOREIGN KEY (AssignedById)
        REFERENCES Security.ApplicationUser(ApplicationUserId),

    CONSTRAINT CK_Security_UserPolicyAssignment_Expiry
        CHECK (ExpiresAtUtc IS NULL OR ExpiresAtUtc > AssignedAtUtc)
);
GO

```

C.2.4.8 User overrides and preferences

UserPermissionOverride records explicit grants or denials outside normal role membership. **UserPreference** is the first cross-schema consumer of CorePlatform: it references language and time zone.

```

CREATE TABLE Security.UserPermissionOverride (
    UserPermissionOverrideId INT IDENTITY(1,1) NOT NULL,
    ApplicationUserId        INT NOT NULL,
    PermissionId             INT NOT NULL,
    IsGranted                BIT NOT NULL,
    ResourceType             NVARCHAR(80) NULL,
    ResourceKey              NVARCHAR(150) NULL,
    Reason                   NVARCHAR(700) NOT NULL,
    EffectiveFromUtc         DATETIME2(7) NOT NULL,
    CONSTRAINT DF_Security_UserPermissionOverride_EffectiveFromUtc DEFAULT SYSUTCDATETIME(),
    EffectiveToUtc           DATETIME2(7) NULL,
    ApprovedByUserId        INT NULL,
    CreatedAtUtc             DATETIME2(7) NOT NULL,
    CONSTRAINT DF_Security_UserPermissionOverride_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy                NVARCHAR(100) NOT NULL,
    ModifiedAtUtc            DATETIME2(7) NOT NULL,
    CONSTRAINT DF_Security_UserPermissionOverride_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy               NVARCHAR(100) NOT NULL,
    IsDeleted                BIT NOT NULL,
    CONSTRAINT DF_Security_UserPermissionOverride_IsDeleted DEFAULT 0,
    RowVersion               ROWVERSION NOT NULL,

    CONSTRAINT PK_Security_UserPermissionOverride
        PRIMARY KEY (UserPermissionOverrideId),

    CONSTRAINT FK_Security_UserPermissionOverride_ApplicationUser

```

```

        FOREIGN KEY (ApplicationUserId)
        REFERENCES Security.ApplicationUser(ApplicationUserId),

CONSTRAINT FK_Security_UserPermissionOverride_Permission
    FOREIGN KEY (PermissionId)
    REFERENCES Security.Permission(PermissionId),

CONSTRAINT FK_Security_UserPermissionOverride_ApprovedByUser
    FOREIGN KEY (ApprovedByUserId)
    REFERENCES Security.ApplicationUser(ApplicationUserId),

CONSTRAINT CK_Security_UserPermissionOverride_EffectiveWindow
    CHECK (EffectiveToUtc IS NULL OR EffectiveToUtc > EffectiveFromUtc)
);
GO

CREATE TABLE Security.UserPreference (
    ApplicationUserId INT NOT NULL,
    LanguageId        INT NOT NULL,
    TimeZoneId        INT NOT NULL,
    Theme              NVARCHAR(30) NOT NULL
        CONSTRAINT DF_Security_UserPreference_Theme DEFAULT N'System',
    DateFormat         NVARCHAR(30) NULL,
    NumberFormat       NVARCHAR(30) NULL,
    ReceiveEmailAlerts BIT NOT NULL
        CONSTRAINT DF_Security_UserPreference_ReceiveEmailAlerts DEFAULT 1,
    ReceiveInAppAlerts BIT NOT NULL
        CONSTRAINT DF_Security_UserPreference_ReceiveInAppAlerts DEFAULT 1,
    CreatedAtUtc       DATETIME2(7) NOT NULL
        CONSTRAINT DF_Security_UserPreference_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc      DATETIME2(7) NOT NULL
        CONSTRAINT DF_Security_UserPreference_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion         ROWVERSION NOT NULL,

CONSTRAINT PK_Security_UserPreference
    PRIMARY KEY (ApplicationUserId),

CONSTRAINT FK_Security_UserPreference_ApplicationUser
    FOREIGN KEY (ApplicationUserId)
    REFERENCES Security.ApplicationUser(ApplicationUserId),

CONSTRAINT FK_Security_UserPreference_Language
    FOREIGN KEY (LanguageId)
    REFERENCES CorePlatform.Language(LanguageId),

CONSTRAINT FK_Security_UserPreference_TimeZone
    FOREIGN KEY (TimeZoneId)
    REFERENCES CorePlatform.TimeZone(TimeZoneId),

CONSTRAINT CK_Security_UserPreference_Theme
    CHECK (Theme IN (N'Light', N'Dark', N'System'))
);
GO

```

C.2.4.9 Sessions, login attempts, API clients, secrets, and tokens

Session and token tables are mutable because sessions can be revoked and tokens can be consumed. Login attempts are append-only facts.

```

CREATE TABLE Security.UserSession (
    UserSessionId    BIGINT IDENTITY(1,1) NOT NULL,
    ApplicationUserId INT NOT NULL,
    SessionStatusId  INT NOT NULL,
    SessionKey        UNIQUEIDENTIFIER NOT NULL
        CONSTRAINT DF_Security_UserSession_SessionKey DEFAULT NEWID(),
    IpAddress         NVARCHAR(64) NULL,
    UserAgent         NVARCHAR(512) NULL,
    StartedAtUtc      DATETIME2(7) NOT NULL
        CONSTRAINT DF_Security_UserSession_StartedAtUtc DEFAULT SYSUTCDATETIME(),
    LastSeenAtUtc     DATETIME2(7) NULL,
    EndedAtUtc        DATETIME2(7) NULL,
    ExpiresAtUtc      DATETIME2(7) NOT NULL,
    RevokedByUserId   INT NULL,
    RevokeReason      NVARCHAR(500) NULL,
    RowVersion        ROWVERSION NOT NULL,

CONSTRAINT PK_Security_UserSession
    PRIMARY KEY (UserSessionId),

```

```

CONSTRAINT FK_Security_UserSession_ApplicationUser
    FOREIGN KEY (ApplicationUserId)
    REFERENCES Security.ApplicationUser(ApplicationUserId),

CONSTRAINT FK_Security_UserSession_SessionStatus
    FOREIGN KEY (SessionStatusId)
    REFERENCES Security.SessionStatus(SessionStatusId),

CONSTRAINT FK_Security_UserSession_RevokedByUser
    FOREIGN KEY (RevokedById)
    REFERENCES Security.ApplicationUser(ApplicationUserId),

CONSTRAINT UQ_Security_UserSession_SessionKey
    UNIQUE (SessionKey),

CONSTRAINT CK_Security_UserSession_Expiry
    CHECK (ExpiresAtUtc > StartedAtUtc)
);
GO

CREATE TABLE Security.LoginAttempt (
    LoginAttemptId BIGINT IDENTITY(1,1) NOT NULL,
    ApplicationUserId INT NULL,
    LoginResultId INT NOT NULL,
    UserName NVARCHAR(256) NULL,
    NormalizedUserName NVARCHAR(256) NULL,
    IPAddress NVARCHAR(64) NULL,
    UserAgent NVARCHAR(512) NULL,
    AttemptedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_LoginAttempt_AttemptedAtUtc DEFAULT SYSUTCDATETIME(),
    FailureReason NVARCHAR(500) NULL,
    CorrelationId UNIQUEIDENTIFIER NULL,

    CONSTRAINT PK_Security_LoginAttempt
        PRIMARY KEY (LoginAttemptId),

    CONSTRAINT FK_Security_LoginAttempt_ApplicationUser
        FOREIGN KEY (ApplicationUserId)
        REFERENCES Security.ApplicationUser(ApplicationUserId),

    CONSTRAINT FK_Security_LoginAttempt_LoginResult
        FOREIGN KEY (LoginResultId)
        REFERENCES Security.LoginResult(LoginResultId)
);
GO

CREATE TABLE Security.ApiClient (
    ApiClientId INT IDENTITY(1,1) NOT NULL,
    OwnerUserId INT NULL,
    ClientId NVARCHAR(100) NOT NULL,
    Name NVARCHAR(180) NOT NULL,
    Description NVARCHAR(700) NULL,
    IsConfidential BIT NOT NULL
    CONSTRAINT DF_Security_ApiClient_IsConfidential DEFAULT 1,
    IsEnabled BIT NOT NULL
    CONSTRAINT DF_Security_ApiClient_IsEnabled DEFAULT 1,
    AllowedScopesJson NVARCHAR(MAX) NULL,
    RedirectUriJson NVARCHAR(MAX) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_ApiClient_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_ApiClient_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL
    CONSTRAINT DF_Security_ApiClient_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Security_ApiClient
        PRIMARY KEY (ApiClientId),

    CONSTRAINT FK_Security_ApiClient_OwnerUser
        FOREIGN KEY (OwnerUserId)
        REFERENCES Security.ApplicationUser(ApplicationUserId),

    CONSTRAINT UQ_Security_ApiClient_ClientId
        UNIQUE (ClientId),

    CONSTRAINT CK_Security_ApiClient_AllowedScopesJson
        CHECK (AllowedScopesJson IS NULL OR ISJSON(AllowedScopesJson) = 1),

    CONSTRAINT CK_Security_ApiClient_RedirectUriJson
        CHECK (RedirectUriJson IS NULL OR ISJSON(RedirectUriJson) = 1)
);
GO

```

```

);
GO

CREATE TABLE Security.ApiClientSecret (
    ApiClientSecretId INT IDENTITY(1,1) NOT NULL,
    ApiClientId INT NOT NULL,
    SecretHash NVARCHAR(512) NOT NULL,
    SecretHint NVARCHAR(80) NULL,
    ValidFromUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_ApiClientSecret_ValidFromUtc DEFAULT SYSUTCDATETIME(),
    ValidToUtc DATETIME2(7) NULL,
    IsRevoked BIT NOT NULL
    CONSTRAINT DF_Security_ApiClientSecret_IsRevoked DEFAULT 0,
    RevokedAtUtc DATETIME2(7) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_ApiClientSecret_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Security_ApiClientSecret
        PRIMARY KEY (ApiClientSecretId),

    CONSTRAINT FK_Security_ApiClientSecret_ApiClient
        FOREIGN KEY (ApiClientId)
        REFERENCES Security.ApiClient(ApiClientId),

    CONSTRAINT CK_Security_ApiClientSecret_Validity
        CHECK (ValidToUtc IS NULL OR ValidToUtc > ValidFromUtc)
);
GO

CREATE TABLE Security.SecurityToken (
    SecurityTokenId BIGINT IDENTITY(1,1) NOT NULL,
    TokenTypeId INT NOT NULL,
    ApplicationUserId INT NULL,
    ApiClientId INT NULL,
    TokenHash NVARCHAR(512) NOT NULL,
    Purpose NVARCHAR(120) NULL,
    IssuedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_SecurityToken_IssuedAtUtc DEFAULT SYSUTCDATETIME(),
    ExpiresAtUtc DATETIME2(7) NOT NULL,
    ConsumedAtUtc DATETIME2(7) NULL,
    IsRevoked BIT NOT NULL
    CONSTRAINT DF_Security_SecurityToken_IsRevoked DEFAULT 0,
    RevokedAtUtc DATETIME2(7) NULL,
    CreatedFromIp NVARCHAR(64) NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Security_SecurityToken
        PRIMARY KEY (SecurityTokenId),

    CONSTRAINT FK_Security_SecurityToken_TokenType
        FOREIGN KEY (TokenTypeId)
        REFERENCES Security.TokenType(TokenTypeId),

    CONSTRAINT FK_Security_SecurityToken_ApplicationUser
        FOREIGN KEY (ApplicationUserId)
        REFERENCES Security.ApplicationUser(ApplicationUserId),

    CONSTRAINT FK_Security_SecurityToken_ApiClient
        FOREIGN KEY (ApiClientId)
        REFERENCES Security.ApiClient(ApiClientId),

    CONSTRAINT UQ_Security_SecurityToken_TokenHash
        UNIQUE (TokenHash),

    CONSTRAINT CK_Security_SecurityToken_Principal
        CHECK (ApplicationUserId IS NOT NULL OR ApiClientId IS NOT NULL),

    CONSTRAINT CK_Security_SecurityToken_Expiry
        CHECK (ExpiresAtUtc > IssuedAtUtc)
);
GO

```

C.2.4.10 Access review governance

`AccessReview` and `AccessReviewItem` answer the governance question: who reviewed which access, what decision was made, and when?

```

CREATE TABLE Security.AccessReview (
    AccessReviewId INT IDENTITY(1,1) NOT NULL,
    AccessReviewStatusId INT NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(180) NOT NULL,
    ScopeDescription NVARCHAR(700) NULL,
    StartedAtUtc DATETIME2(7) NULL,
    DueAtUtc DATETIME2(7) NULL,
    CompletedAtUtc DATETIME2(7) NULL,
    RequestedByUserId INT NOT NULL,
    ApprovedByUserId INT NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_AccessReview_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_AccessReview_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL
    CONSTRAINT DF_Security_AccessReview_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Security_AccessReview
    PRIMARY KEY (AccessReviewId),

    CONSTRAINT FK_Security_AccessReview_Status
    FOREIGN KEY (AccessReviewStatusId)
    REFERENCES Security.AccessReviewStatus(AccessReviewStatusId),

    CONSTRAINT FK_Security_AccessReview_RequestedByUser
    FOREIGN KEY (RequestedByUserId)
    REFERENCES Security.ApplicationUser(ApplicationUserId),

    CONSTRAINT FK_Security_AccessReview_ApprovedByUser
    FOREIGN KEY (ApprovedByUserId)
    REFERENCES Security.ApplicationUser(ApplicationUserId),

    CONSTRAINT UQ_Security_AccessReview_Code
    UNIQUE (Code),

    CONSTRAINT CK_Security_AccessReview_Window
    CHECK (DueAtUtc IS NULL OR StartedAtUtc IS NULL OR DueAtUtc > StartedAtUtc)
);
GO

CREATE TABLE Security.AccessReviewItem (
    AccessReviewItemId INT IDENTITY(1,1) NOT NULL,
    AccessReviewId INT NOT NULL,
    ApplicationUserId INT NOT NULL,
    ApplicationRoleId INT NULL,
    PermissionId INT NULL,
    ReviewDecision NVARCHAR(30) NOT NULL
    CONSTRAINT DF_Security_AccessReviewItem_ReviewDecision DEFAULT N'Pending',
    DecisionReason NVARCHAR(700) NULL,
    ReviewedByUserId INT NULL,
    ReviewedAtUtc DATETIME2(7) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Security_AccessReviewItem_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Security_AccessReviewItem
    PRIMARY KEY (AccessReviewItemId),

    CONSTRAINT FK_Security_AccessReviewItem_AccessReview
    FOREIGN KEY (AccessReviewId)
    REFERENCES Security.AccessReview(AccessReviewId),

    CONSTRAINT FK_Security_AccessReviewItem_ApplicationUser
    FOREIGN KEY (ApplicationUserId)
    REFERENCES Security.ApplicationUser(ApplicationUserId),

    CONSTRAINT FK_Security_AccessReviewItem_ApplicationRole
    FOREIGN KEY (ApplicationRoleId)
    REFERENCES Security.ApplicationRole(ApplicationRoleId),

    CONSTRAINT FK_Security_AccessReviewItem_Permission
    FOREIGN KEY (PermissionId)
    REFERENCES Security.Permission(PermissionId),

    CONSTRAINT FK_Security_AccessReviewItem_ReviewedByUser
    FOREIGN KEY (ReviewedByUserId)
    REFERENCES Security.ApplicationUser(ApplicationUserId),

```



```

CONSTRAINT CK_Security_AccessReviewItem_Decision
CHECK (ReviewDecision IN (N'Pending', N'Keep', N'Remove', N'Modify'))
);
GO

```

C.2.5 Indexes and seed examples

Every foreign key receives an index unless covered by a primary key. High-volume runtime tables are indexed by user and descending UTC time so recent activity can be found quickly.

```

CREATE UNIQUE INDEX UX_Security_ApplicationUser_NormalizedEmail
ON Security.ApplicationUser(NormalizedEmail)
WHERE NormalizedEmail IS NOT NULL AND IsDeleted = 0;
GO

CREATE INDEX IX_Security_ApplicationUser_UserStatusId
ON Security.ApplicationUser(UserStatusId);
GO

CREATE INDEX IX_Security_ApplicationRole_RoleTypeId
ON Security.ApplicationRole(RoleTypeId);
GO

CREATE INDEX IX_Security_ApplicationRole_ParentRoleId
ON Security.ApplicationRole(ParentRoleId);
GO

CREATE INDEX IX_Security_UserRole_ApplicationRoleId
ON Security.UserRole(ApplicationRoleId);
GO

CREATE INDEX IX_Security_Permission_Category
ON Security.Permission(PermissionCategoryId, Code);
GO

CREATE INDEX IX_Security_RolePermission_PermissionId
ON Security.RolePermission(PermissionId);
GO

CREATE INDEX IX_Security_PolicyPermission_PermissionId
ON Security.PolicyPermission(PermissionId);
GO

CREATE INDEX IX_Security_RolePolicy_PolicyId
ON Security.RolePolicy(PolicyId);
GO

CREATE INDEX IX_Security_UserPolicyAssignment_PolicyId
ON Security.UserPolicyAssignment(PolicyId);
GO

CREATE INDEX IX_Security_UserPermissionOverride_User_Permission
ON Security.UserPermissionOverride(ApplicationUserId, PermissionId, ResourceType, ResourceKey)
WHERE IsDeleted = 0;
GO

CREATE INDEX IX_Security_UserPreference_LanguageId
ON Security.UserPreference(LanguageId);
GO

CREATE INDEX IX_Security_UserPreference_TimeZoneId
ON Security.UserPreference(TimeZoneId);
GO

CREATE INDEX IX_Security_UserSession_User_Status
ON Security.UserSession(ApplicationUserId, SessionStatusId, ExpiresAtUtc DESC);
GO

CREATE INDEX IX_Security_LoginAttempt_User_Time
ON Security.LoginAttempt(ApplicationUserId, AttemptedAtUtc DESC);
GO

CREATE INDEX IX_Security_LoginAttempt_NormalizedUser_Time
ON Security.LoginAttempt(NormalizedUserName, AttemptedAtUtc DESC);
GO

```

```

CREATE INDEX IX_Security_ApiClient_OwnerUserId
ON Security.ApiClient(OwnerUserId);
GO

CREATE INDEX IX_Security_ApiClientSecret_ApiClientId
ON Security.ApiClientSecret(ApiClientId);
GO

CREATE INDEX IX_Security_SecurityToken_User_Type_Expiry
ON Security.SecurityToken(ApplicationUserId, TokenTypeId, ExpiresAtUtc DESC);
GO

CREATE INDEX IX_Security_SecurityToken_ApiClient_Type_Expiry
ON Security.SecurityToken(ApiClientId, TokenTypeId, ExpiresAtUtc DESC);
GO

CREATE INDEX IX_Security_AccessReview_Status
ON Security.AccessReview(AccessReviewStatusId, DueAtUtc);
GO

CREATE INDEX IX_Security_AccessReviewItem_Review_User
ON Security.AccessReviewItem(AccessReviewId, ApplicationUserId);
GO

```

C.2.5.1 Minimal seed examples

```

-- Minimal demonstrator seeds. Complete seed data belongs in Appendix H.
INSERT INTO Security.UserStatus (Code, Name, DisplayOrder)
VALUES (N'ACTIVE', N'Active', 10),
       (N'LOCKED', N'Locked', 20),
       (N'DISABLED', N'Disabled', 30),
       (N'PENDING', N'Pending activation', 40),
       (N'RETIRED', N'Retired', 50);

INSERT INTO Security.RoleType (Code, Name, DisplayOrder)
VALUES (N'OPERATOR', N'Operator role', 10),
       (N'ADMIN', N'Administrator role', 20),
       (N'AUDITOR', N'Auditor role', 30),
       (N'SERVICE', N'Service account role', 40),
       (N'EMERGENCY', N'Emergency response role', 50);

INSERT INTO Security.PermissionCategory (Code, Name, DisplayOrder)
VALUES (N'PLATFORM', N'Platform', 10),
       (N'ALARM', N'Alarm management', 20),
       (N'TWIN', N'Digital twin', 30),
       (N'REPORTING', N'Reporting', 40),
       (N'AUDIT', N'Audit', 50),
       (N'SECURITY', N'Security administration', 60);

INSERT INTO Security.PolicyType (Code, Name, DisplayOrder)
VALUES (N'ROLE', N'Role policy', 10),
       (N'DATA_ACCESS', N'Data access policy', 20),
       (N'UNIT_SCOPE', N'Unit scope policy', 30),
       (N'EMERGENCY', N'Emergency policy', 40);

INSERT INTO Security.SessionStatus (Code, Name, DisplayOrder)
VALUES (N'ACTIVE', N'Active', 10),
       (N'EXPIRED', N'Expired', 20),
       (N'REVOKED', N'Revoked', 30),
       (N'SIGNED_OUT', N'Signed out', 40);

INSERT INTO Security.LoginResult (Code, Name, DisplayOrder)
VALUES (N'SUCCESS', N'Success', 10),
       (N'BAD_PASSWORD', N'Bad password', 20),
       (N'LOCKED_OUT', N'Locked out', 30),
       (N'DISABLED', N'Disabled account', 40),
       (N'MFA_REQUIRED', N'MFA required', 50);

INSERT INTO Security.TokenType (Code, Name, DisplayOrder)
VALUES (N'REFRESH', N'Refresh token', 10),
       (N'PASSWORD_RESET', N'Password reset token', 20),
       (N'EMAIL_CONFIRMATION', N'Email confirmation token', 30),
       (N'MFA', N'MFA token', 40),
       (N'API_CLIENT', N'API client token', 50);

INSERT INTO Security.AccessReviewStatus (Code, Name, DisplayOrder)
VALUES (N'DRAFT', N'Draft', 10),
       (N'OPEN', N'Open', 20),

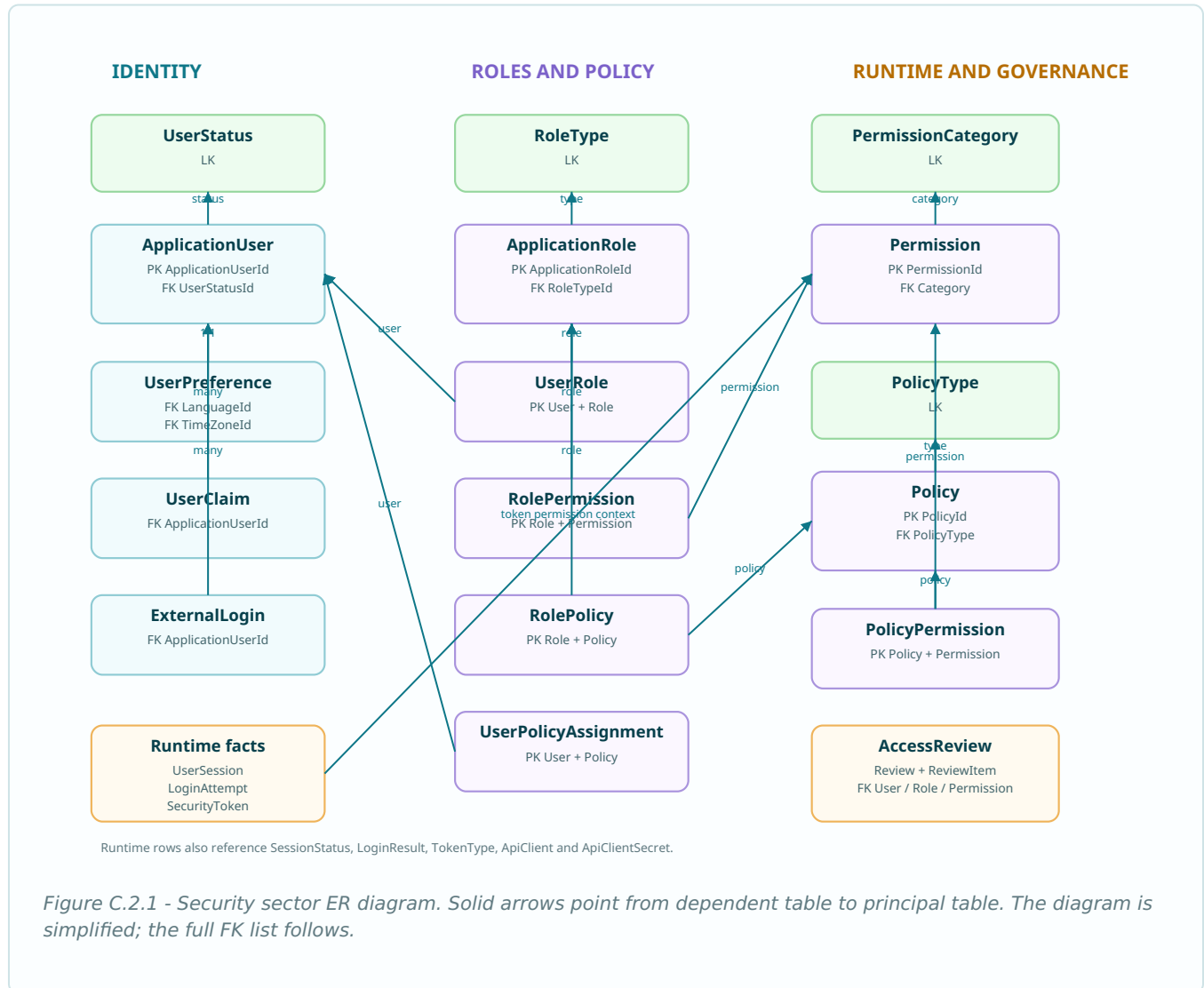
```

```
(N'COMPLETED', N'Completed', 30),
(N'CANCELLED', N'Cancelled', 40);
```

G0

C.2.6 ER diagram

The diagram groups the Security sector into identity, authorization, runtime, and governance. Lookup tables are shown where they control the lifecycle or type of a principal object.



C.2.7 Foreign-key mapping

C.2.7.1 Internal Security foreign keys

FROM TABLE	FK COLUMN	TO TABLE	PK COLUMN	RELATIONSHIP
Security.ApplicationUser	UserStatusId	Security.UserStatus	UserStatusId	Many users per status.
Security.ApplicationRole	RoleTypeId	Security.RoleType	RoleTypeId	Many roles per role type.
Security.ApplicationRole	ParentRoleId	Security.ApplicationRole	ApplicationRoleId	Optional role hierarchy.
Security.UserRole	ApplicationUserId	Security.ApplicationUser	ApplicationUserId	Many roles per user.
Security.UserRole	ApplicationRoleId	Security.ApplicationRole	ApplicationRoleId	Many users per role.
Security.Permission	PermissionCategoryId	Security.PermissionCategory	PermissionCategoryId	Many permissions per category.
Security.Policy	PolicyTypeId	Security.PolicyType	PolicyTypeId	Many policies per policy type.
Security.RolePermission	ApplicationRoleId	Security.ApplicationRole	ApplicationRoleId	Role grants or denies permissions.
Security.RolePermission	PermissionId	Security.Permission	PermissionId	Permission included in a role.
Security.PolicyPermission	PolicyId	Security.Policy	PolicyId	Policy contains permissions.
Security.PolicyPermission	PermissionId	Security.Permission	PermissionId	Permission required by policy.
Security.RolePolicy	ApplicationRoleId	Security.ApplicationRole	ApplicationRoleId	Policy assigned to role.
Security.RolePolicy	PolicyId	Security.Policy	PolicyId	Role receives policy.
Security.UserPolicyAssignment	ApplicationUserId	Security.ApplicationUser	ApplicationUserId	Policy assigned directly to user.
Security.UserPermissionOverride	ApplicationUserId	Security.ApplicationUser	ApplicationUserId	Explicit user exception.
Security.UserPermissionOverride	PermissionId	Security.Permission	PermissionId	Permission overridden.
Security.UserPreference	ApplicationUserId	Security.ApplicationUser	ApplicationUserId	One preference row per user.
Security.UserSession	ApplicationUserId	Security.ApplicationUser	ApplicationUserId	Many sessions per user.
Security.UserSession	SessionStatusId	Security.SessionStatus	SessionStatusId	Session lifecycle.

FROM TABLE	FK COLUMN	TO TABLE	PK COLUMN	RELATIONSHIP
Security.LoginAttempt	ApplicationUserId	Security.ApplicationUser	ApplicationUserId	Nullable: unknown usernames still logged.
Security.LoginAttempt	LoginResultId	Security.LoginResult	LoginResultId	Attempt outcome.
Security.ApiClient	OwnerUserId	Security.ApplicationUser	ApplicationUserId	Optional accountable owner.
Security.ApiClientSecret	ApiClientId	Security.ApiClient	ApiClientId	Many rotating secrets per client.
Security.SecurityToken	TokenTypeId	Security.TokenType	TokenTypeId	Token category.
Security.SecurityToken	ApplicationUserId	Security.ApplicationUser	ApplicationUserId	Optional user principal.
Security.SecurityToken	ApiClientId	Security.ApiClient	ApiClientId	Optional API client principal.
Security.AccessReview	AccessReviewStatusId	Security.AccessReviewStatus	AccessReviewStatusId	Review lifecycle.
Security.AccessReviewItem	AccessReviewId	Security.AccessReview	AccessReviewId	Many items per review.
Security.AccessReviewItem	ApplicationUserId	Security.ApplicationUser	ApplicationUserId	User being reviewed.
Security.AccessReviewItem	ApplicationRoleId	Security.ApplicationRole	ApplicationRoleId	Optional role under review.
Security.AccessReviewItem	PermissionId	Security.Permission	PermissionId	Optional permission under review.

C.2.7.2 CorePlatform passports consumed by Security

FROM TABLE	FK COLUMN	TO TABLE	PK COLUMN	RELATIONSHIP
Security.UserPreference	LanguageId	CorePlatform.Language	LanguageId	Preferred UI language.
Security.UserPreference	TimeZoneId	CorePlatform.TimeZone	TimeZoneId	Preferred display and scheduling time zone.

C.2.7.3 Incoming passports from later sectors

The following relationships are declared in later sectors. They are listed here because they explain why `ApplicationUser` is one of the most important passport tables in the whole database.

Organization.Person

FK ApplicationUserId
TO Security.ApplicationUser

Optional login account for a person, employee, or contractor.

AlarmManagement.AlarmAcknowledgement

FK AcknowledgedByUserId
TO Security.ApplicationUser

Who acknowledged an alarm.

EventManager.Incident

FK OpenedByUserId / AssignedToUserId
TO Security.ApplicationUser

Who opened or owns an incident.

Maintenance.WorkOrder

FK RequestedByUserId / ApprovedByUserId
TO Security.ApplicationUser

Who requested, approved, or closed maintenance work.

Compliance.AuditFinding

FK OwnerUserId / ClosedByUserId
TO Security.ApplicationUser

Finding owner and closure authority.

Reporting.ReportDefinition

FK CreatedByUserId
TO Security.ApplicationUser

Report ownership and controlled export.

Audit.AuditEntry

FK ActorUserId
TO Security.ApplicationUser

Actor captured by immutable audit log.

CORE PLATFORM

SECURITY

LATER SECTORS

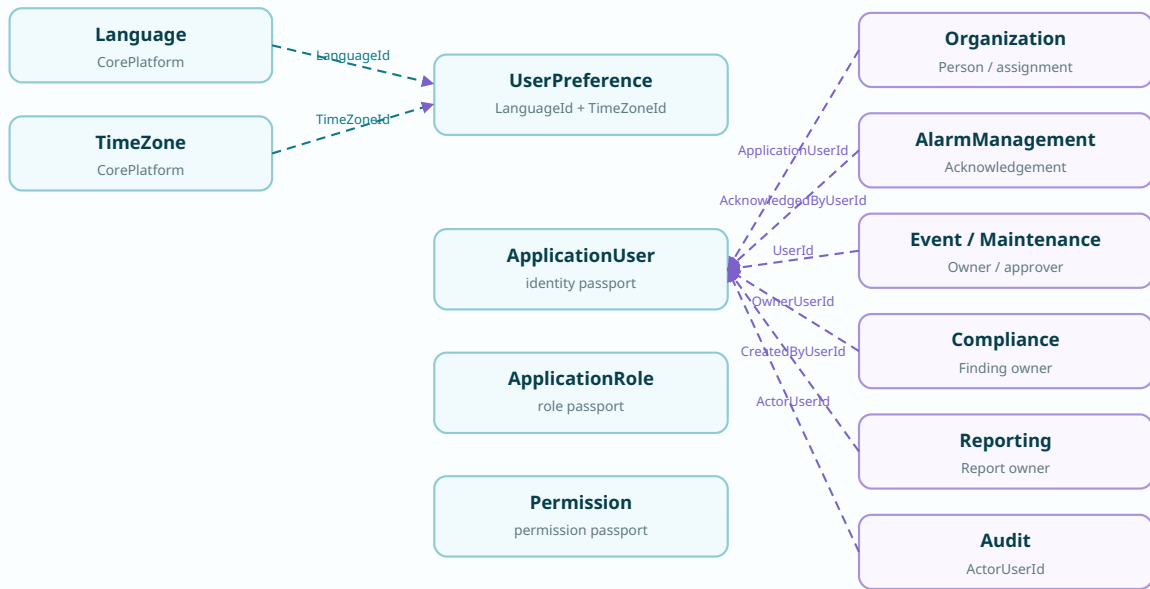


Figure C.2.2 - Security passport map. Security consumes CorePlatform for preferences and is then consumed by later sectors for actor, owner, approver, and acknowledgement references.

C.2.8 Useful verification queries

These queries verify that the sector behaves like an authorization backbone rather than a set of disconnected identity tables.

```
-- Effective permissions for one user from active roles.
DECLARE @UserId INT = 1;

SELECT DISTINCT p.Code, p.Name, pc.Code AS Category, rp.IsGranted
FROM Security.ApplicationUser AS u
JOIN Security.UserRole AS ur
  ON ur.ApplicationUserId = u.ApplicationUserId AND ur.IsActive = 1
JOIN Security.ApplicationRole AS r
  ON r.ApplicationRoleId = ur.ApplicationRoleId AND r.IsDeleted = 0
JOIN Security.RolePermission AS rp
  ON rp.ApplicationRoleId = r.ApplicationRoleId
JOIN Security.Permission AS p
  ON p.PermissionId = rp.PermissionId AND p.IsDeleted = 0
JOIN Security.PermissionCategory AS pc
  ON pc.PermissionCategoryId = p.PermissionCategoryId
WHERE u.ApplicationUserId = @UserId
  AND u.IsDeleted = 0
  AND (ur.ExpiresAtUtc IS NULL OR ur.ExpiresAtUtc > SYSUTCDATETIME())
  AND (rp.ExpiresAtUtc IS NULL OR rp.ExpiresAtUtc > SYSUTCDATETIME())
ORDER BY pc.Code, p.Code;

-- Recent failed logins for a normalized username.
SELECT TOP (20) la.AttemptedAtUtc, lr.Code AS Result, la.IpAddress, la.FailureReason
FROM Security.LoginAttempt AS la
JOIN Security.LoginResult AS lr ON lr.LoginResultId = la.LoginResultId
WHERE la.NormalizedUserName = N'OPERATOR1'
  AND lr.Code <> N'SUCCESS'
ORDER BY la.AttemptedAtUtc DESC;

-- Open access-review items that have not yet been decided.
```

```

SELECT ar.Code AS ReviewCode, u.UserName, r.Name AS RoleName, p.Code AS PermissionCode,
       ari.ReviewDecision
FROM Security.AccessReviewItem AS ari
JOIN Security.AccessReview AS ar ON ar.AccessReviewId = ari.AccessReviewId
JOIN Security.ApplicationUser AS u ON u.ApplicationUserId = ari.ApplicationUserId
LEFT JOIN Security.ApplicationRole AS r ON r.ApplicationRoleId = ari.ApplicationRoleId
LEFT JOIN Security.Permission AS p ON p.PermissionId = ari.PermissionId
WHERE ari.ReviewDecision = N'Pending'
ORDER BY ar.Code, u.UserName;

```

C.2.9 Boundary and next sector

Honest boundary. This schema is an enterprise demonstrator security model. It is not a complete production identity server. It does not include every possible authentication method, cryptographic detail, single sign-on protocol table, device-binding table, or compliance-specific access-control workflow. Passwords, secrets, and tokens are represented only as hashes; plaintext secret material never belongs in the database.

The next sector, **Organization**, will attach human and institutional meaning to these accounts: sites, plants, departments, teams, employees, contractors, shifts, and responsibility chains. That separation matters. A user can log in; a person can belong to a team; an operator can be assigned to a shift. They overlap, but they are not the same table.

Appendix C.3

Organization Schema

The Organization sector owns the institutional and human structure around the plant: legal entities, sites, plants, buildings, departments, teams, positions, people, shifts, qualifications, certifications, and staffing scenarios.

Security answers who can log in. Organization answers who the person is, where that person belongs, which team or shift they serve, what qualifications they hold, and which plant, site, department, or legal entity owns the work.

A login account is not an employee. A department is not a role. A shift assignment is not a permission. Organization keeps those facts separate and joins them by passports.

C.3.1

Sector purpose

C.3.2

Full table list

C.3.3

Lookup tables

C.3.4

SQL DDL

C.3.5

Indexes and seed examples

C.3.6

ER diagram

C.3.7

Foreign-key mapping

C.3.8

Boundary and next sector

C.3.1 Sector purpose

Organization follows **CorePlatform** and **Security** in the dependency order. It consumes countries, regions and time zones from CorePlatform, and it optionally links people to login accounts in Security.

- It owns the legal and physical structure: legal entities, sites, plants and buildings.
- It owns the human structure: departments, teams, positions, people and employment relationships.
- It owns rostering and competence: shifts, on-call windows, qualifications, certifications and staffing checks.
- It becomes one of the main passport providers for later sectors: ReactorFleet, Maintenance, RadiationMonitoring, EmergencyPreparedness, Compliance and Reporting.

Design choice. `Organization.Plant` is not a reactor unit. It is the organizational plant container under a physical site. `ReactorFleet.Unit` will later reference `Organization.Plant` through `PlantId`, preserving the distinction between institution, site, plant and machine.

C.3.2 Full table list

The Organization sector contains thirty-seven tables: ten lookup tables and twenty-seven substantive tables. It is intentionally larger than Security because it carries the real-world hierarchy used by many later sectors.

LOOKUP

Organization.LegalEntityType

PK `LegalEntityTypeId`

FK None

Classifies operators, vendors, regulators, emergency partners, universities, and service providers.

LOOKUP

Organization.SiteType

PK `SiteTypeId`

FK None

Classifies headquarters, plant sites, training centres, warehouses, and external facilities.

LOOKUP

Organization.PlantType

PK `PlantTypeId`

FK None

Classifies plant-level installations: nuclear site, simulator plant, training plant, support plant.

LOOKUP

Organization.DepartmentType

PK `DepartmentTypeId`

FK None

Classifies operations, maintenance, engineering, safety, radiation protection, security, and administration.

LOOKUP

Organization.TeamType

PK `TeamTypeId`

FK None

Classifies shift crew, maintenance crew, emergency team, robotics team, audit team, and contractor team.

LOOKUP

Organization.PersonType

PK `PersonTypeId`

FK None

Classifies employee, contractor, vendor representative, regulator, observer, and emergency contact.

LOOKUP

Organization.EmploymentStatus

PK `EmploymentStatusId`

FK None

Lifecycle of employment and engagements: active, inactive, suspended, retired, ended.

LOOKUP

Organization.ShiftType

PK `ShiftTypeId`

FK None

Classifies day, night, rotating, outage, emergency, training, and on-call shifts.

LOOKUP

Organization.QualificationStatus

PK QualificationStatusId

FK None

Status of qualification or certification rows: valid, expired, pending, revoked.

LOOKUP

Organization.ContactMethodType

PK ContactMethodTypeId

FK None

Classifies work email, mobile, radio, office phone, emergency phone, and external contact route.

STRUCTURE

Organization.LegalEntity

PK LegalEntityId

FK LegalEntityTypeId , ParentLegalEntityId , CountryId

The company, operator, vendor, regulator, or partner that owns sites, employs people, or provides services.

STRUCTURE

Organization.Site

PK SiteId

FK LegalEntityId , SiteTypeId , CountryId , RegionId , TimeZoneId

A physical site: plant campus, headquarters, training centre, or support facility.

STRUCTURE

Organization.Plant

PK PlantId

FK SiteId , PlantTypeId

A plant within a site. ReactorFleet.Unit will carry its passport to this table.

STRUCTURE

Organization.Building

PK BuildingId

FK SiteId

Building or facility structure used by access, radiation, maintenance, and emergency sectors.

STRUCTURE

Organization.Department

PK DepartmentId

FK LegalEntityId , ParentDepartmentId , DepartmentTypeId

Organisational unit: operations, engineering, safety, maintenance, security, compliance.

STRUCTURE

Organization.Team

PK TeamId

FK DepartmentId , TeamTypeId

Working team or crew, including shift crews and temporary mission teams.

PERSONNEL

Organization.Position

PK PositionId

FK DepartmentId

A job or responsibility position: reactor operator, shift supervisor, radiation technician, maintenance lead.

PERSONNEL

Organization.Person

PK PersonId

FK PersonTypeId , ApplicationUserId , LegalEntityId

A human person. Optional passport to Security.ApplicationUser only when the person can log in.

PERSONNEL**Organization.PersonContact**

PK PersonContactId
FK PersonId , ContactMethodTypeId

Contact routes for a person: email, mobile, radio, phone, or emergency contact.

PERSONNEL**Organization.PersonAddress**

PK PersonAddressId
FK PersonId , CountryId , RegionId

Work, mailing, or emergency-contact address. Kept outside the person row.

PERSONNEL**Organization.Employment**

PK EmploymentId
FK PersonId , LegalEntityId , PositionId ,
EmploymentStatusId

Employment relationship, manager chain, and current position.

PERSONNEL**Organization.ContractorEngagement**

PK ContractorEngagementId
FK PersonId , VendorLegalEntityId , SiteId ,
EmploymentStatusId

Contractor or vendor engagement scoped to a site and validity window.

ASSIGNMENT**Organization.DepartmentAssignment**

PK DepartmentAssignmentId
FK PersonId , DepartmentId , PositionId

Time-bounded assignment of a person to a department and position.

ASSIGNMENT**Organization.TeamMembership**

PK TeamMembershipId
FK PersonId , TeamId , PositionId

Time-bounded membership of a person in a team or shift crew.

SHIFT**Organization.ShiftPattern**

PK ShiftPatternId
FK ShiftTypeId

Reusable shift pattern: day, night, rotating, outage, emergency, or training.

SHIFT**Organization.Shift**

PK ShiftId
FK SiteId , ShiftPatternId

One scheduled shift occurrence with UTC start and end times.

SHIFT**Organization.ShiftAssignment**

PK ShiftAssignmentId
FK ShiftId , PersonId , TeamId , PositionId

Assigns a person, team, and position to one shift occurrence.

SHIFT**Organization.OnCallRoster**

PK OnCallRosterId
FK SiteId , PersonId

On-call coverage window and contact priority for a site.

COMPETENCE**Organization.Qualification**

PK QualificationId

FK None

Catalogue of skills or authorisations required for roles and staffing scenarios.

COMPETENCE**Organization.PersonQualification**

PK PersonQualificationId

FK PersonId, QualificationId,
QualificationStatusId

A person holding a qualification, with issue, expiry, and verification.

COMPETENCE**Organization.Certification**

PK CertificationId

FK None

External certification or regulatory credential.

COMPETENCE**Organization.PersonCertification**

PK PersonCertificationId

FK PersonId, CertificationId,
QualificationStatusId

A person holding a certification, with issue, expiry, and verification.

STAFFING**Organization.PersonnelRequirement**

PK PersonnelRequirementId

FK SiteId, PlantId, DepartmentId, PositionId

Baseline staffing rule: required count for a site, plant, department, or position.

STAFFING**Organization.StaffingScenario**

PK StaffingScenarioId

FK SiteId, CreatedByUserId

Named what-if or stress-test staffing scenario.

STAFFING**Organization.StaffingScenarioRequirement**

PK StaffingScenarioRequirementId

FK StaffingScenarioId, PositionId,
RequiredQualificationId

One required position or qualification inside a staffing scenario.

STAFFING**Organization.StaffingScenarioResult**

PK StaffingScenarioResultId

FK StaffingScenarioId, EvaluatedByUserId

A saved evaluation of a staffing scenario.

STAFFING**Organization.StaffingScenarioGap**

PK StaffingScenarioGapId

FK StaffingScenarioResultId, PositionId

Gap row showing required, available, and missing personnel counts.

C.3.3 Lookup tables

The lookup tables use the standard Appendix C.0 shape. They are typed tables, because each value category needs its own foreign key, seed list, display order and audit trail.

LOOKUP TABLE	TYPICAL SEED CODES	MAIN CONSUMER
LegalEntityType	OPERATOR, VENDOR, REGULATOR, EMERGENCY_PARTNER	Referenced by LegalEntity.
SiteType	PLANT_SITE, HQ, TRAINING_CENTER, WAREHOUSE	Referenced by Site.
PlantType	NUCLEAR_DEMO, SIMULATOR, TRAINING, SUPPORT	Referenced by Plant.
DepartmentType	OPS, MAINT, ENG, SAFETY, RAD_PROT	Referenced by Department.
TeamType	SHIFT_CREW, MAINT_CREW, EMERGENCY, ROBOTICS	Referenced by Team.
PersonType	EMPLOYEE, CONTRACTOR, VENDOR_REP, REGULATOR	Referenced by Person.
EmploymentStatus	ACTIVE, INACTIVE, SUSPENDED, ENDED	Referenced by Employment and ContractorEngagement.
ShiftType	DAY, NIGHT, ROTATING, OUTAGE, EMERGENCY	Referenced by ShiftPattern.
QualificationStatus	VALID, EXPIRED, PENDING, REVOKED	Referenced by person qualification/certification rows.
ContactMethodType	WORK_EMAIL, MOBILE, RADIO, OFFICE_PHONE	Referenced by PersonContact.

C.3.4 SQL DDL

C.3.4.1 Schema

```
IF SCHEMA ID(N'Organization') IS NULL
EXEC(N'CREATE SCHEMA Organization');
GO
```

C.3.4.2 Organization lookup tables

```
CREATE TABLE Organization.LegalEntityType (
  LegalEntityTypeId INT IDENTITY(1,1) NOT NULL,
  Code NVARCHAR(50) NOT NULL,
  Name NVARCHAR(150) NOT NULL,
  Description NVARCHAR(500) NULL,
  DisplayOrder INT NOT NULL
  CONSTRAINT DF_Organization_LegalEntityType_DisplayOrder DEFAULT 0,
  IsActive BIT NOT NULL
  CONSTRAINT DF_Organization_LegalEntityType_IsActive DEFAULT 1,
  CreatedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_Organization_LegalEntityType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_Organization_LegalEntityType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  RowVersion ROWVERSION NOT NULL,

  CONSTRAINT PK_Organization_LegalEntityType
    PRIMARY KEY (LegalEntityTypeId),
  CONSTRAINT UQ_Organization_LegalEntityType_Code
    UNIQUE (Code)
);
GO
```

```
CREATE TABLE Organization.SiteType (
  SiteTypeId INT IDENTITY(1,1) NOT NULL,
```

```

Code          NVARCHAR(50) NOT NULL,
Name          NVARCHAR(150) NOT NULL,
Description    NVARCHAR(500) NULL,
DisplayOrder  INT NOT NULL
    CONSTRAINT DF_Organization_SiteType_DisplayOrder DEFAULT 0,
IsActive      BIT NOT NULL
    CONSTRAINT DF_Organization_SiteType_IsActive DEFAULT 1,
CreatedAtUtc  DATETIME2(7) NOT NULL
    CONSTRAINT DF_Organization_SiteType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Organization_SiteType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion    ROWVERSION NOT NULL,

CONSTRAINT PK_Organization_SiteType
    PRIMARY KEY (SiteTypeId),
CONSTRAINT UQ_Organization_SiteType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Organization.PlantType (
    PlantTypeId INT IDENTITY(1,1) NOT NULL,
    Code        NVARCHAR(50) NOT NULL,
    Name        NVARCHAR(150) NOT NULL,
    Description  NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Organization_PlantType_DisplayOrder DEFAULT 0,
    IsActive    BIT NOT NULL
    CONSTRAINT DF_Organization_PlantType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Organization_PlantType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Organization_PlantType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion   ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_PlantType
        PRIMARY KEY (PlantTypeId),
    CONSTRAINT UQ_Organization_PlantType_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Organization.DepartmentType (
    DepartmentTypeId INT IDENTITY(1,1) NOT NULL,
    Code             NVARCHAR(50) NOT NULL,
    Name             NVARCHAR(150) NOT NULL,
    Description       NVARCHAR(500) NULL,
    DisplayOrder     INT NOT NULL
    CONSTRAINT DF_Organization_DepartmentType_DisplayOrder DEFAULT 0,
    IsActive         BIT NOT NULL
    CONSTRAINT DF_Organization_DepartmentType_IsActive DEFAULT 1,
    CreatedAtUtc     DATETIME2(7) NOT NULL
    CONSTRAINT DF_Organization_DepartmentType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc    DATETIME2(7) NOT NULL
    CONSTRAINT DF_Organization_DepartmentType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion       ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_DepartmentType
        PRIMARY KEY (DepartmentTypeId),
    CONSTRAINT UQ_Organization_DepartmentType_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Organization.TeamType (
    TeamTypeId INT IDENTITY(1,1) NOT NULL,
    Code       NVARCHAR(50) NOT NULL,
    Name       NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Organization_TeamType_DisplayOrder DEFAULT 0,
    IsActive    BIT NOT NULL
    CONSTRAINT DF_Organization_TeamType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Organization_TeamType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Organization_TeamType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion   ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_TeamType
        PRIMARY KEY (TeamTypeId),
    CONSTRAINT UQ_Organization_TeamType_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Organization.PersonType (
    PersonTypeId INT IDENTITY(1,1) NOT NULL,

```

```

Code          NVARCHAR(50) NOT NULL,
Name          NVARCHAR(150) NOT NULL,
Description   NVARCHAR(500) NULL,
DisplayOrder  INT NOT NULL
    CONSTRAINT DF_Organization_PersonType_DisplayOrder DEFAULT 0,
IsActive     BIT NOT NULL
    CONSTRAINT DF_Organization_PersonType_IsActive DEFAULT 1,
CreatedAtUtc  DATETIME2(7) NOT NULL
    CONSTRAINT DF_Organization_PersonType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Organization_PersonType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion   ROWVERSION NOT NULL,

CONSTRAINT PK_Organization_PersonType
    PRIMARY KEY (PersonTypeId),
CONSTRAINT UQ_Organization_PersonType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Organization.EmploymentStatus (
    EmploymentStatusId INT IDENTITY(1,1) NOT NULL,
    Code               NVARCHAR(50) NOT NULL,
    Name               NVARCHAR(150) NOT NULL,
    Description        NVARCHAR(500) NULL,
    DisplayOrder       INT NOT NULL
        CONSTRAINT DF_Organization_EmploymentStatus_DisplayOrder DEFAULT 0,
    IsActive          BIT NOT NULL
        CONSTRAINT DF_Organization_EmploymentStatus_IsActive DEFAULT 1,
    CreatedAtUtc       DATETIME2(7) NOT NULL
        CONSTRAINT DF_Organization_EmploymentStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc       DATETIME2(7) NOT NULL
        CONSTRAINT DF_Organization_EmploymentStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion         ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_EmploymentStatus
        PRIMARY KEY (EmploymentStatusId),
    CONSTRAINT UQ_Organization_EmploymentStatus_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Organization.ShiftType (
    ShiftTypeId       INT IDENTITY(1,1) NOT NULL,
    Code              NVARCHAR(50) NOT NULL,
    Name              NVARCHAR(150) NOT NULL,
    Description        NVARCHAR(500) NULL,
    DisplayOrder       INT NOT NULL
        CONSTRAINT DF_Organization_ShiftType_DisplayOrder DEFAULT 0,
    IsActive          BIT NOT NULL
        CONSTRAINT DF_Organization_ShiftType_IsActive DEFAULT 1,
    CreatedAtUtc       DATETIME2(7) NOT NULL
        CONSTRAINT DF_Organization_ShiftType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc       DATETIME2(7) NOT NULL
        CONSTRAINT DF_Organization_ShiftType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion         ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_ShiftType
        PRIMARY KEY (ShiftTypeId),
    CONSTRAINT UQ_Organization_ShiftType_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Organization.QualificationStatus (
    QualificationStatusId INT IDENTITY(1,1) NOT NULL,
    Code                 NVARCHAR(50) NOT NULL,
    Name                 NVARCHAR(150) NOT NULL,
    Description           NVARCHAR(500) NULL,
    DisplayOrder          INT NOT NULL
        CONSTRAINT DF_Organization_QualificationStatus_DisplayOrder DEFAULT 0,
    IsActive             BIT NOT NULL
        CONSTRAINT DF_Organization_QualificationStatus_IsActive DEFAULT 1,
    CreatedAtUtc          DATETIME2(7) NOT NULL
        CONSTRAINT DF_Organization_QualificationStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc          DATETIME2(7) NOT NULL
        CONSTRAINT DF_Organization_QualificationStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion            ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_QualificationStatus
        PRIMARY KEY (QualificationStatusId),
    CONSTRAINT UQ_Organization_QualificationStatus_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Organization.ContactMethodType (
    ContactMethodTypeId INT IDENTITY(1,1) NOT NULL,

```



```

Code          NVARCHAR(50) NOT NULL,
Name          NVARCHAR(150) NOT NULL,
Description    NVARCHAR(500) NULL,
DisplayOrder  INT NOT NULL
    CONSTRAINT DF_Organization_ContactMethodType_DisplayOrder DEFAULT 0,
IsActive      BIT NOT NULL
    CONSTRAINT DF_Organization_ContactMethodType_IsActive DEFAULT 1,
CreatedAtUtc  DATETIME2(7) NOT NULL
    CONSTRAINT DF_Organization_ContactMethodType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Organization_ContactMethodType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion    ROWVERSION NOT NULL,

CONSTRAINT PK_Organization_ContactMethodType
    PRIMARY KEY (ContactMethodTypeId),
CONSTRAINT UQ_Organization_ContactMethodType_Code
    UNIQUE (Code)
);
GO

```

C.3.4.3 Legal entity, site, plant and building

These tables are the physical and institutional anchor. Later `ReactorFleet.Unit` rows reference `Organization.Plant`; access, radiation and emergency sectors can reference `Site` and `Building`.

```

CREATE TABLE Organization.LegalEntity (
    LegalEntityId      INT IDENTITY(1,1) NOT NULL,
    LegalEntityTypeId  INT NOT NULL,
    ParentLegalEntityId INT NULL,
    CountryId          INT NULL,
    Code               NVARCHAR(50) NOT NULL,
    Name               NVARCHAR(250) NOT NULL,
    RegistrationNumber  NVARCHAR(100) NULL,
    TaxIdentifier       NVARCHAR(100) NULL,
    WebsiteUrl         NVARCHAR(300) NULL,
    IsOperator         BIT NOT NULL CONSTRAINT DF_Organization_LegalEntity_IsOperator DEFAULT 0,
    IsVendor           BIT NOT NULL CONSTRAINT DF_Organization_LegalEntity_IsVendor DEFAULT 0,
    CreatedAtUtc       DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_LegalEntity_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy          NVARCHAR(100) NOT NULL,
    ModifiedAtUtc      DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_LegalEntity_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy         NVARCHAR(100) NOT NULL,
    IsDeleted          BIT NOT NULL CONSTRAINT DF_Organization_LegalEntity_IsDeleted DEFAULT 0,
    RowVersion         ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_LegalEntity PRIMARY KEY (LegalEntityId),
    CONSTRAINT UQ_Organization_LegalEntity_Code UNIQUE (Code),
    CONSTRAINT FK_Organization_LegalEntity_LegalEntityType
        FOREIGN KEY (LegalEntityTypeId) REFERENCES Organization.LegalEntityType(LegalEntityTypeId),
    CONSTRAINT FK_Organization_LegalEntity_ParentLegalEntity
        FOREIGN KEY (ParentLegalEntityId) REFERENCES Organization.LegalEntity(LegalEntityId),
    CONSTRAINT FK_Organization_LegalEntity_Country
        FOREIGN KEY (CountryId) REFERENCES CorePlatform.Country(CountryId)
);
GO

CREATE TABLE Organization.Site (
    SiteId            INT IDENTITY(1,1) NOT NULL,
    LegalEntityId     INT NOT NULL,
    SiteTypeId        INT NOT NULL,
    CountryId         INT NOT NULL,
    RegionId          INT NULL,
    TimeZoneId        INT NOT NULL,
    Code              NVARCHAR(50) NOT NULL,
    Name              NVARCHAR(250) NOT NULL,
    AddressLine1      NVARCHAR(250) NULL,
    AddressLine2      NVARCHAR(250) NULL,
    City              NVARCHAR(150) NULL,
    PostalCode        NVARCHAR(50) NULL,
    Latitude           DECIMAL(10,7) NULL,
    Longitude         DECIMAL(10,7) NULL,
    IsOperational     BIT NOT NULL CONSTRAINT DF_Organization_Site_IsOperational DEFAULT 1,
    CreatedAtUtc      DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_Site_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy         NVARCHAR(100) NOT NULL,
    ModifiedAtUtc     DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_Site_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy        NVARCHAR(100) NOT NULL,
    IsDeleted         BIT NOT NULL CONSTRAINT DF_Organization_Site_IsDeleted DEFAULT 0,
    RowVersion        ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_Site PRIMARY KEY (SiteId),
    CONSTRAINT UQ_Organization_Site_Code UNIQUE (Code),
    CONSTRAINT FK_Organization_Site_LegalEntity
        FOREIGN KEY (LegalEntityId) REFERENCES Organization.LegalEntity(LegalEntityId),
    CONSTRAINT FK_Organization_Site_SiteType
        FOREIGN KEY (SiteTypeId) REFERENCES Organization.SiteType(SiteTypeId),

```

```

CONSTRAINT FK_Organization_Site_Country
    FOREIGN KEY (CountryId) REFERENCES CorePlatform.Country(CountryId),
CONSTRAINT FK_Organization_Site_Region
    FOREIGN KEY (RegionId) REFERENCES CorePlatform.Region(RegionId),
CONSTRAINT FK_Organization_Site_TimeZone
    FOREIGN KEY (TimeZoneId) REFERENCES CorePlatform.TimeZone(TimeZoneId),
CONSTRAINT CK_Organization_Site_Latitude CHECK (Latitude IS NULL OR Latitude BETWEEN -90 AND 90),
CONSTRAINT CK_Organization_Site_Longitude CHECK (Longitude IS NULL OR Longitude BETWEEN -180 AND 180)
);
GO

CREATE TABLE Organization.Plant (
    PlantId INT IDENTITY(1,1) NOT NULL,
    SiteId INT NOT NULL,
    PlantTypeId INT NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(250) NOT NULL,
    Description NVARCHAR(1000) NULL,
    OperationalStartDate DATE NULL,
    IsOperational BIT NOT NULL CONSTRAINT DF_Organization_Plant_IsOperational DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_Plant_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_Plant_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Organization_Plant_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_Plant PRIMARY KEY (PlantId),
    CONSTRAINT UQ_Organization_Plant_Code UNIQUE (Code),
    CONSTRAINT FK_Organization_Plant_Site
        FOREIGN KEY (SiteId) REFERENCES Organization.Site(SiteId),
    CONSTRAINT FK_Organization_Plant_PlantType
        FOREIGN KEY (PlantTypeId) REFERENCES Organization.PlantType(PlantTypeId)
);
GO

CREATE TABLE Organization.Building (
    BuildingId INT IDENTITY(1,1) NOT NULL,
    SiteId INT NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    BuildingUsage NVARCHAR(150) NULL,
    FloorCount INT NULL,
    IsControlledArea BIT NOT NULL CONSTRAINT DF_Organization_Building_IsControlledArea DEFAULT 0,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_Building_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_Building_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Organization_Building_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_Building PRIMARY KEY (BuildingId),
    CONSTRAINT UQ_Organization_Building_Site_Code UNIQUE (SiteId, Code),
    CONSTRAINT FK_Organization_Building_Site
        FOREIGN KEY (SiteId) REFERENCES Organization.Site(SiteId),
    CONSTRAINT CK_Organization_Building_FloorCount CHECK (FloorCount IS NULL OR FloorCount >= 0)
);
GO

```

C.3.4.4 Departments, teams and positions

Departments and teams form the organisational tree. Positions name the responsibilities that people, shift assignments, staffing requirements and stress tests refer to.

```

CREATE TABLE Organization.Department (
    DepartmentId INT IDENTITY(1,1) NOT NULL,
    LegalEntityId INT NOT NULL,
    ParentDepartmentId INT NULL,
    DepartmentTypeId INT NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    CostCentreCode NVARCHAR(50) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_Department_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_Department_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Organization_Department_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_Department PRIMARY KEY (DepartmentId),
    CONSTRAINT UQ_Organization_Department_LegalEntity_Code UNIQUE (LegalEntityId, Code),
    CONSTRAINT FK_Organization_Department_LegalEntity
        FOREIGN KEY (LegalEntityId) REFERENCES Organization.LegalEntity(LegalEntityId),
    CONSTRAINT FK_Organization_Department_ParentDepartment
        FOREIGN KEY (ParentDepartmentId) REFERENCES Organization.Department(DepartmentId),
    CONSTRAINT FK_Organization_Department_DepartmentType

```

```

        FOREIGN KEY (DepartmentTypeId) REFERENCES Organization.DepartmentType(DepartmentTypeId)
    );
GO

CREATE TABLE Organization.Team (
    TeamId          INT IDENTITY(1,1) NOT NULL,
    DepartmentId    INT NOT NULL,
    TeamTypeId      INT NOT NULL,
    Code            NVARCHAR(50) NOT NULL,
    Name            NVARCHAR(200) NOT NULL,
    IsShiftTeam     BIT NOT NULL CONSTRAINT DF_Organization_Team_IsShiftTeam DEFAULT 0,
    IsEmergencyTeam BIT NOT NULL CONSTRAINT DF_Organization_Team_IsEmergencyTeam DEFAULT 0,
    CreatedAtUtc    DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_Team_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy       NVARCHAR(100) NOT NULL,
    ModifiedAtUtc   DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_Team_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy      NVARCHAR(100) NOT NULL,
    IsDeleted       BIT NOT NULL CONSTRAINT DF_Organization_Team_IsDeleted DEFAULT 0,
    RowVersion      ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_Team PRIMARY KEY (TeamId),
    CONSTRAINT UQ_Organization_Team_Department_Code UNIQUE (DepartmentId, Code),
    CONSTRAINT FK_Organization_Team_Department
        FOREIGN KEY (DepartmentId) REFERENCES Organization.Department(DepartmentId),
    CONSTRAINT FK_Organization_Team_TeamType
        FOREIGN KEY (TeamTypeId) REFERENCES Organization.TeamType(TeamTypeId)
);
GO

CREATE TABLE Organization.Position (
    PositionId      INT IDENTITY(1,1) NOT NULL,
    DepartmentId    INT NULL,
    Code            NVARCHAR(50) NOT NULL,
    Title           NVARCHAR(200) NOT NULL,
    Description      NVARCHAR(1000) NULL,
    IsSafetyCritical BIT NOT NULL CONSTRAINT DF_Organization_Position_IsSafetyCritical DEFAULT 0,
    RequiresShiftWork BIT NOT NULL CONSTRAINT DF_Organization_Position_RequiresShiftWork DEFAULT 0,
    CreatedAtUtc    DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_Position_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy       NVARCHAR(100) NOT NULL,
    ModifiedAtUtc   DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_Position_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy      NVARCHAR(100) NOT NULL,
    IsDeleted       BIT NOT NULL CONSTRAINT DF_Organization_Position_IsDeleted DEFAULT 0,
    RowVersion      ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_Position PRIMARY KEY (PositionId),
    CONSTRAINT UQ_Organization_Position_Code UNIQUE (Code),
    CONSTRAINT FK_Organization_Position_Department
        FOREIGN KEY (DepartmentId) REFERENCES Organization.Department(DepartmentId)
);
GO

```

C.3.4.5 Person, contact and address

The person row is intentionally separate from the security user. A person may or may not have a login account, and a service account may have no person behind it.

```

CREATE TABLE Organization.Person (
    PersonId        INT IDENTITY(1,1) NOT NULL,
    PersonTypeId    INT NOT NULL,
    ApplicationUserId INT NULL,
    LegalEntityId   INT NULL,
    PersonnelNumber NVARCHAR(80) NULL,
    GivenName       NVARCHAR(100) NOT NULL,
    FamilyName      NVARCHAR(100) NOT NULL,
    DisplayName     NVARCHAR(200) NOT NULL,
    WorkEmail       NVARCHAR(256) NULL,
    WorkPhone       NVARCHAR(80) NULL,
    IsActive        BIT NOT NULL CONSTRAINT DF_Organization_Person_IsActive DEFAULT 1,
    CreatedAtUtc    DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_Person_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy       NVARCHAR(100) NOT NULL,
    ModifiedAtUtc   DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_Person_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy      NVARCHAR(100) NOT NULL,
    IsDeleted       BIT NOT NULL CONSTRAINT DF_Organization_Person_IsDeleted DEFAULT 0,
    RowVersion      ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_Person PRIMARY KEY (PersonId),
    CONSTRAINT UQ_Organization_Person_PersonnelNumber UNIQUE (PersonnelNumber),
    CONSTRAINT UQ_Organization_Person_ApplicationUserId UNIQUE (ApplicationUserId),
    CONSTRAINT FK_Organization_Person_PersonType
        FOREIGN KEY (PersonTypeId) REFERENCES Organization.PersonType(PersonTypeId),
    CONSTRAINT FK_Organization_Person_ApplicationUser
        FOREIGN KEY (ApplicationUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT FK_Organization_Person_LegalEntity
        FOREIGN KEY (LegalEntityId) REFERENCES Organization.LegalEntity(LegalEntityId)
);
GO

```

```

CREATE TABLE Organization.PersonContact (
    PersonContactId INT IDENTITY(1,1) NOT NULL,
    PersonId INT NOT NULL,
    ContactMethodTypeId INT NOT NULL,
    ContactValue NVARCHAR(300) NOT NULL,
    IsPrimary BIT NOT NULL CONSTRAINT DF_Organization_PersonContact_IsPrimary DEFAULT 0,
    IsEmergencyContact BIT NOT NULL CONSTRAINT DF_Organization_PersonContact_IsEmergencyContact DEFAULT 0,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_PersonContact_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_PersonContact_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Organization_PersonContact_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_PersonContact PRIMARY KEY (PersonContactId),
    CONSTRAINT FK_Organization_PersonContact_Person
        FOREIGN KEY (PersonId) REFERENCES Organization.Person(PersonId),
    CONSTRAINT FK_Organization_PersonContact_ContactMethodType
        FOREIGN KEY (ContactMethodTypeId) REFERENCES Organization.ContactMethodType(ContactMethodTypeId)
);
GO

CREATE TABLE Organization.PersonAddress (
    PersonAddressId INT IDENTITY(1,1) NOT NULL,
    PersonId INT NOT NULL,
    CountryId INT NOT NULL,
    RegionId INT NOT NULL,
    AddressType NVARCHAR(50) NOT NULL,
    AddressLine1 NVARCHAR(250) NOT NULL,
    AddressLine2 NVARCHAR(250) NULL,
    City NVARCHAR(150) NULL,
    PostalCode NVARCHAR(50) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_PersonAddress_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_PersonAddress_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Organization_PersonAddress_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_PersonAddress PRIMARY KEY (PersonAddressId),
    CONSTRAINT FK_Organization_PersonAddress_Person
        FOREIGN KEY (PersonId) REFERENCES Organization.Person(PersonId),
    CONSTRAINT FK_Organization_PersonAddress_Country
        FOREIGN KEY (CountryId) REFERENCES CorePlatform.Country(CountryId),
    CONSTRAINT FK_Organization_PersonAddress_Region
        FOREIGN KEY (RegionId) REFERENCES CorePlatform.Region(RegionId),
    CONSTRAINT CK_Organization_PersonAddress_AddressType CHECK (AddressType IN ('Work','Mailing','Emergency','Other'))
);
GO

```

C.3.4.6 Employment, contractor engagement and assignments

Employment, department assignment and team membership are time-bounded. This avoids overwriting history when a person changes department, position or crew.

```

CREATE TABLE Organization.Employment (
    EmploymentId INT IDENTITY(1,1) NOT NULL,
    PersonId INT NOT NULL,
    LegalEntityId INT NOT NULL,
    PositionId INT NOT NULL,
    EmploymentStatusId INT NOT NULL,
    ManagerPersonId INT NOT NULL,
    EmployeeNumber NVARCHAR(80) NULL,
    StartDate DATE NOT NULL,
    EndDate DATE NULL,
    FtePercent DECIMAL(5,2) NOT NULL CONSTRAINT DF_Organization_Employment_FtePercent DEFAULT 100.00,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_Employment_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_Employment_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Organization_Employment_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_Employment PRIMARY KEY (EmploymentId),
    CONSTRAINT UQ_Organization_Employment_EmployeeNumber UNIQUE (EmployeeNumber),
    CONSTRAINT FK_Organization_Employment_Person
        FOREIGN KEY (PersonId) REFERENCES Organization.Person(PersonId),
    CONSTRAINT FK_Organization_Employment_LegalEntity
        FOREIGN KEY (LegalEntityId) REFERENCES Organization.LegalEntity(LegalEntityId),
    CONSTRAINT FK_Organization_Employment_Position
        FOREIGN KEY (PositionId) REFERENCES Organization.Position(PositionId),
    CONSTRAINT FK_Organization_Employment_EmploymentStatus
        FOREIGN KEY (EmploymentStatusId) REFERENCES Organization.EmploymentStatus(EmploymentStatusId),
    CONSTRAINT FK_Organization_Employment_ManagerPerson
        FOREIGN KEY (ManagerPersonId) REFERENCES Organization.Person(PersonId),

```

```

CONSTRAINT CK_Organization_Employment_DateRange CHECK (EndDate IS NULL OR EndDate >= StartDate),
CONSTRAINT CK_Organization_Employment_FtePercent CHECK (FtePercent > 0 AND FtePercent <= 100)
);
GO

CREATE TABLE Organization.ContractorEngagement (
    ContractorEngagementId INT IDENTITY(1,1) NOT NULL,
    PersonId INT NOT NULL,
    VendorLegalEntityId INT NOT NULL,
    SiteId INT NOT NULL,
    EmploymentStatusId INT NOT NULL,
    ContractReference NVARCHAR(100) NULL,
    StartDate DATE NOT NULL,
    EndDate DATE NULL,
    SponsorPersonId INT NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_ContractorEngagement_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_ContractorEngagement_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Organization_ContractorEngagement_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_ContractorEngagement PRIMARY KEY (ContractorEngagementId),
    CONSTRAINT FK_Organization_ContractorEngagement_Person
        FOREIGN KEY (PersonId) REFERENCES Organization.Person(PersonId),
    CONSTRAINT FK_Organization_ContractorEngagement_VendorLegalEntity
        FOREIGN KEY (VendorLegalEntityId) REFERENCES Organization.LegalEntity(LegalEntityId),
    CONSTRAINT FK_Organization_ContractorEngagement_Site
        FOREIGN KEY (SiteId) REFERENCES Organization.Site(SiteId),
    CONSTRAINT FK_Organization_ContractorEngagement_EmploymentStatus
        FOREIGN KEY (EmploymentStatusId) REFERENCES Organization.EmploymentStatus(EmploymentStatusId),
    CONSTRAINT FK_Organization_ContractorEngagement_SponsorPerson
        FOREIGN KEY (SponsorPersonId) REFERENCES Organization.Person(PersonId),
    CONSTRAINT CK_Organization_ContractorEngagement_DateRange CHECK (EndDate IS NULL OR EndDate >= StartDate)
);
GO

CREATE TABLE Organization.DepartmentAssignment (
    DepartmentAssignmentId INT IDENTITY(1,1) NOT NULL,
    PersonId INT NOT NULL,
    DepartmentId INT NOT NULL,
    PositionId INT NULL,
    StartDate DATE NOT NULL,
    EndDate DATE NULL,
    IsPrimary BIT NOT NULL CONSTRAINT DF_Organization_DepartmentAssignment_IsPrimary DEFAULT 0,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_DepartmentAssignment_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_DepartmentAssignment_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Organization_DepartmentAssignment_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_DepartmentAssignment PRIMARY KEY (DepartmentAssignmentId),
    CONSTRAINT FK_Organization_DepartmentAssignment_Person
        FOREIGN KEY (PersonId) REFERENCES Organization.Person(PersonId),
    CONSTRAINT FK_Organization_DepartmentAssignment_Department
        FOREIGN KEY (DepartmentId) REFERENCES Organization.Department(DepartmentId),
    CONSTRAINT FK_Organization_DepartmentAssignment_Position
        FOREIGN KEY (PositionId) REFERENCES Organization.Position(PositionId),
    CONSTRAINT CK_Organization_DepartmentAssignment_DateRange CHECK (EndDate IS NULL OR EndDate >= StartDate)
);
GO

CREATE TABLE Organization.TeamMembership (
    TeamMembershipId INT IDENTITY(1,1) NOT NULL,
    PersonId INT NOT NULL,
    TeamId INT NOT NULL,
    PositionId INT NULL,
    StartDate DATE NOT NULL,
    EndDate DATE NULL,
    IsLead BIT NOT NULL CONSTRAINT DF_Organization_TeamMembership_IsLead DEFAULT 0,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_TeamMembership_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_TeamMembership_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Organization_TeamMembership_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_TeamMembership PRIMARY KEY (TeamMembershipId),
    CONSTRAINT FK_Organization_TeamMembership_Person
        FOREIGN KEY (PersonId) REFERENCES Organization.Person(PersonId),
    CONSTRAINT FK_Organization_TeamMembership_Team
        FOREIGN KEY (TeamId) REFERENCES Organization.Team(TeamId),
    CONSTRAINT FK_Organization_TeamMembership_Position
        FOREIGN KEY (PositionId) REFERENCES Organization.Position(PositionId),

```

```

        CONSTRAINT CK_Organization_TeamMembership_DateRange CHECK (EndDate IS NULL OR EndDate >= StartDate)
    );
GO

```

C.3.4.7 Shift and on-call roster

Shift rows are occurrences, not templates. A shift pattern says what kind of shift exists; a shift row says when it happened and who was assigned.

```

CREATE TABLE Organization.ShiftPattern (
    ShiftPatternId INT IDENTITY(1,1) NOT NULL,
    ShiftTypeId INT NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    StartTimeLocal TIME(0) NOT NULL,
    EndTimeLocal TIME(0) NOT NULL,
    CycleLengthDays INT NOT NULL CONSTRAINT DF_Organization_ShiftPattern_CycleLengthDays DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_ShiftPattern_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_ShiftPattern_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Organization_ShiftPattern_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_ShiftPattern PRIMARY KEY (ShiftPatternId),
    CONSTRAINT UQ_Organization_ShiftPattern_Code UNIQUE (Code),
    CONSTRAINT FK_Organization_ShiftPattern_ShiftType
        FOREIGN KEY (ShiftTypeId) REFERENCES Organization.ShiftType(ShiftTypeId),
    CONSTRAINT CK_Organization_ShiftPattern_CycleLengthDays CHECK (CycleLengthDays >= 1)
);
GO

CREATE TABLE Organization.Shift (
    ShiftId BIGINT IDENTITY(1,1) NOT NULL,
    SiteId INT NOT NULL,
    ShiftPatternId INT NOT NULL,
    ShiftCode NVARCHAR(80) NOT NULL,
    StartsAtUtc DATETIME2(7) NOT NULL,
    EndsAtUtc DATETIME2(7) NOT NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_Shift_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_Shift_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Organization_Shift_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_Shift PRIMARY KEY (ShiftId),
    CONSTRAINT UQ_Organization_Shift_Site_ShiftCode UNIQUE (SiteId, ShiftCode),
    CONSTRAINT FK_Organization_Shift_Site
        FOREIGN KEY (SiteId) REFERENCES Organization.Site(SiteId),
    CONSTRAINT FK_Organization_Shift_ShiftPattern
        FOREIGN KEY (ShiftPatternId) REFERENCES Organization.ShiftPattern(ShiftPatternId),
    CONSTRAINT CK_Organization_Shift_TimeRange CHECK (EndsAtUtc > StartsAtUtc)
);
GO

CREATE TABLE Organization.ShiftAssignment (
    ShiftAssignmentId BIGINT IDENTITY(1,1) NOT NULL,
    ShiftId BIGINT NOT NULL,
    PersonId INT NOT NULL,
    TeamId INT NULL,
    PositionId INT NULL,
    AssignmentRole NVARCHAR(120) NULL,
    IsConfirmed BIT NOT NULL CONSTRAINT DF_Organization_ShiftAssignment_IsConfirmed DEFAULT 0,
    ConfirmedAtUtc DATETIME2(7) NULL,
    ConfirmedByUserId INT NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_ShiftAssignment_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_ShiftAssignment_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Organization_ShiftAssignment_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_ShiftAssignment PRIMARY KEY (ShiftAssignmentId),
    CONSTRAINT UQ_Organization_ShiftAssignment_Shift_Person UNIQUE (ShiftId, PersonId),
    CONSTRAINT FK_Organization_ShiftAssignment_Shift
        FOREIGN KEY (ShiftId) REFERENCES Organization.Shift(ShiftId),
    CONSTRAINT FK_Organization_ShiftAssignment_Person
        FOREIGN KEY (PersonId) REFERENCES Organization.Person(PersonId),
    CONSTRAINT FK_Organization_ShiftAssignment_Team
        FOREIGN KEY (TeamId) REFERENCES Organization.Team(TeamId),
    CONSTRAINT FK_Organization_ShiftAssignment_Position
        FOREIGN KEY (PositionId) REFERENCES Organization.Position(PositionId),
    CONSTRAINT FK_Organization_ShiftAssignment_ConfirmedByUser

```



```

        FOREIGN KEY (ConfirmedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId)
    );
GO

CREATE TABLE Organization.OnCallRoster (
    OnCallRosterId BIGINT IDENTITY(1,1) NOT NULL,
    SiteId          INT NOT NULL,
    PersonId        INT NOT NULL,
    StartsAtUtc     DATETIME2(7) NOT NULL,
    EndsAtUtc       DATETIME2(7) NOT NULL,
    ContactPriority  INT NOT NULL CONSTRAINT DF_Organization_OnCallRoster_ContactPriority DEFAULT 1,
    Notes           NVARCHAR(500) NULL,
    CreatedAtUtc    DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_OnCallRoster_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy       NVARCHAR(100) NOT NULL,
    ModifiedAtUtc   DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_OnCallRoster_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy      NVARCHAR(100) NOT NULL,
    IsDeleted       BIT NOT NULL CONSTRAINT DF_Organization_OnCallRoster_IsDeleted DEFAULT 0,
    RowVersion      ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_OnCallRoster PRIMARY KEY (OnCallRosterId),
    CONSTRAINT FK_Organization_OnCallRoster_Site
        FOREIGN KEY (SiteId) REFERENCES Organization.Site(SiteId),
    CONSTRAINT FK_Organization_OnCallRoster_Person
        FOREIGN KEY (PersonId) REFERENCES Organization.Person(PersonId),
    CONSTRAINT CK_Organization_OnCallRoster_TimeRange CHECK (EndsAtUtc > StartsAtUtc),
    CONSTRAINT CK_Organization_OnCallRoster_ContactPriority CHECK (ContactPriority >= 1)
);
GO

```

C.3.4.8 Qualifications and certifications

Qualifications are internal skills or authorisations. Certifications are external or regulatory credentials. Both are linked to people through dated, verifiable rows.

```

CREATE TABLE Organization.Qualification (
    QualificationId INT IDENTITY(1,1) NOT NULL,
    Code            NVARCHAR(50) NOT NULL,
    Name            NVARCHAR(200) NOT NULL,
    Description      NVARCHAR(1000) NULL,
    Issuer          NVARCHAR(200) NULL,
    ValidityMonths  INT NULL,
    IsSafetyCritical BIT NOT NULL CONSTRAINT DF_Organization_Qualification_IsSafetyCritical DEFAULT 0,
    CreatedAtUtc    DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_Qualification_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy       NVARCHAR(100) NOT NULL,
    ModifiedAtUtc   DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_Qualification_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy      NVARCHAR(100) NOT NULL,
    IsDeleted       BIT NOT NULL CONSTRAINT DF_Organization_Qualification_IsDeleted DEFAULT 0,
    RowVersion      ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_Qualification PRIMARY KEY (QualificationId),
    CONSTRAINT UQ_Organization_Qualification_Code UNIQUE (Code),
    CONSTRAINT CK_Organization_Qualification_ValidityMonths CHECK (ValidityMonths IS NULL OR ValidityMonths > 0)
);
GO

CREATE TABLE Organization.PersonQualification (
    PersonQualificationId INT IDENTITY(1,1) NOT NULL,
    PersonId              INT NOT NULL,
    QualificationId        INT NOT NULL,
    QualificationStatusId  INT NOT NULL,
    IssuedAtUtc            DATETIME2(7) NULL,
    ExpiresAtUtc           DATETIME2(7) NULL,
    VerifiedAtUtc          DATETIME2(7) NULL,
    VerifiedByUserId       INT NULL,
    Notes                  NVARCHAR(500) NULL,
    CreatedAtUtc           DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_PersonQualification_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy              NVARCHAR(100) NOT NULL,
    ModifiedAtUtc          DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_PersonQualification_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
    ModifiedBy             NVARCHAR(100) NOT NULL,
    IsDeleted              BIT NOT NULL CONSTRAINT DF_Organization_PersonQualification_IsDeleted DEFAULT 0,
    RowVersion             ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_PersonQualification PRIMARY KEY (PersonQualificationId),
    CONSTRAINT FK_Organization_PersonQualification_Person
        FOREIGN KEY (PersonId) REFERENCES Organization.Person(PersonId),
    CONSTRAINT FK_Organization_PersonQualification_Qualification
        FOREIGN KEY (QualificationId) REFERENCES Organization.Qualification(QualificationId),
    CONSTRAINT FK_Organization_PersonQualification_QualificationStatus
        FOREIGN KEY (QualificationStatusId) REFERENCES Organization.QualificationStatus(QualificationStatusId),
    CONSTRAINT FK_Organization_PersonQualification_VerifiedByUser
        FOREIGN KEY (VerifiedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_Organization_PersonQualification_Expiry CHECK (ExpiresAtUtc IS NULL OR IssuedAtUtc IS NULL OR ExpiresAtUtc
> IssuedAtUtc)
);

```

GO

```
CREATE TABLE Organization.Certification (
  CertificationId INT IDENTITY(1,1) NOT NULL,
  Code NVARCHAR(50) NOT NULL,
  Name NVARCHAR(200) NOT NULL,
  RegulatoryBody NVARCHAR(200) NULL,
  Description NVARCHAR(1000) NULL,
  RequiredForSafetyCritical BIT NOT NULL CONSTRAINT DF_Organization_Certification_RequiredForSafetyCritical DEFAULT 0,
  CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_Certification_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  CreatedBy NVARCHAR(100) NOT NULL,
  ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_Certification_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedBy NVARCHAR(100) NOT NULL,
  IsDeleted BIT NOT NULL CONSTRAINT DF_Organization_Certification_IsDeleted DEFAULT 0,
  RowVersion ROWVERSION NOT NULL,

  CONSTRAINT PK_Organization_Certification PRIMARY KEY (CertificationId),
  CONSTRAINT UQ_Organization_Certification_Code UNIQUE (Code)
);
GO

CREATE TABLE Organization.PersonCertification (
  PersonCertificationId INT IDENTITY(1,1) NOT NULL,
  PersonId INT NOT NULL,
  CertificationId INT NOT NULL,
  QualificationStatusId INT NOT NULL,
  IssuedAtUtc DATETIME2(7) NULL,
  ExpiresAtUtc DATETIME2(7) NULL,
  VerifiedAtUtc DATETIME2(7) NULL,
  VerifiedByUserId INT NULL,
  CertificateNumber NVARCHAR(120) NULL,
  CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_PersonCertification_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
  CreatedBy NVARCHAR(100) NOT NULL,
  ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_PersonCertification_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
  ModifiedBy NVARCHAR(100) NOT NULL,
  IsDeleted BIT NOT NULL CONSTRAINT DF_Organization_PersonCertification_IsDeleted DEFAULT 0,
  RowVersion ROWVERSION NOT NULL,

  CONSTRAINT PK_Organization_PersonCertification PRIMARY KEY (PersonCertificationId),
  CONSTRAINT FK_Organization_PersonCertification_Person
    FOREIGN KEY (PersonId) REFERENCES Organization.Person(PersonId),
  CONSTRAINT FK_Organization_PersonCertification_Certification
    FOREIGN KEY (CertificationId) REFERENCES Organization.Certification(CertificationId),
  CONSTRAINT FK_Organization_PersonCertification_QualificationStatus
    FOREIGN KEY (QualificationStatusId) REFERENCES Organization.QualificationStatus(QualificationStatusId),
  CONSTRAINT FK_Organization_PersonCertification_VerifiedByUser
    FOREIGN KEY (VerifiedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
  CONSTRAINT CK_Organization_PersonCertification_Expiry CHECK (ExpiresAtUtc IS NULL OR IssuedAtUtc IS NULL OR ExpiresAtUtc
> IssuedAtUtc)
);
GO
```

C.3.4.9 Staffing requirements and stress-test scenarios

These tables support the console's personnel and stress-test idea: a scenario defines staffing needs, an evaluation records the result, and gap rows show shortages.

```
CREATE TABLE Organization.PersonnelRequirement (
  PersonnelRequirementId INT IDENTITY(1,1) NOT NULL,
  SiteId INT NOT NULL,
  PlantId INT NULL,
  DepartmentId INT NULL,
  PositionId INT NOT NULL,
  MinRequiredCount INT NOT NULL,
  RequiredQualificationId INT NULL,
  IsSafetyCritical BIT NOT NULL CONSTRAINT DF_Organization_PersonnelRequirement_IsSafetyCritical DEFAULT 0,
  ValidFromUtc DATETIME2(7) NOT NULL,
  ValidToUtc DATETIME2(7) NULL,
  CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_PersonnelRequirement_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
  CreatedBy NVARCHAR(100) NOT NULL,
  ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_PersonnelRequirement_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
  ModifiedBy NVARCHAR(100) NOT NULL,
  IsDeleted BIT NOT NULL CONSTRAINT DF_Organization_PersonnelRequirement_IsDeleted DEFAULT 0,
  RowVersion ROWVERSION NOT NULL,

  CONSTRAINT PK_Organization_PersonnelRequirement PRIMARY KEY (PersonnelRequirementId),
  CONSTRAINT FK_Organization_PersonnelRequirement_Site
    FOREIGN KEY (SiteId) REFERENCES Organization.Site(SiteId),
  CONSTRAINT FK_Organization_PersonnelRequirement_Plant
    FOREIGN KEY (PlantId) REFERENCES Organization.Plant(PlantId),
  CONSTRAINT FK_Organization_PersonnelRequirement_Department
    FOREIGN KEY (DepartmentId) REFERENCES Organization.Department(DepartmentId),
```



```

CONSTRAINT FK_Organization_PersonnelRequirement_Position
    FOREIGN KEY (PositionId) REFERENCES Organization.Position(PositionId),
CONSTRAINT FK_Organization_PersonnelRequirement_RequiredQualification
    FOREIGN KEY (RequiredQualificationId) REFERENCES Organization.Qualification(QualificationId),
CONSTRAINT CK_Organization_PersonnelRequirement_MinRequiredCount CHECK (MinRequiredCount >= 0),
CONSTRAINT CK_Organization_PersonnelRequirement_Validity CHECK (ValidToUtc IS NULL OR ValidToUtc > ValidFromUtc)
);
GO

CREATE TABLE Organization.StaffingScenario (
    StaffingScenarioId INT IDENTITY(1,1) NOT NULL,
    SiteId INT NOT NULL,
    ScenarioCode NVARCHAR(50) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    Description NVARCHAR(1000) NULL,
    CreatedByUserId INT NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_StaffingScenario_CreatedAtUtc DEFAULT
    SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_StaffingScenario_ModifiedAtUtc DEFAULT
    SYSUTCDATETIME(),
    IsDeleted BIT NOT NULL CONSTRAINT DF_Organization_StaffingScenario_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Organization_StaffingScenario PRIMARY KEY (StaffingScenarioId),
    CONSTRAINT UQ_Organization_StaffingScenario_Site_Code UNIQUE (SiteId, ScenarioCode),
    CONSTRAINT FK_Organization_StaffingScenario_Site
        FOREIGN KEY (SiteId) REFERENCES Organization.Site(SiteId),
    CONSTRAINT FK_Organization_StaffingScenario_CreatedByUser
        FOREIGN KEY (CreatedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE Organization.StaffingScenarioRequirement (
    StaffingScenarioRequirementId INT IDENTITY(1,1) NOT NULL,
    StaffingScenarioId INT NOT NULL,
    PositionId INT NOT NULL,
    RequiredQualificationId INT NULL,
    RequiredCount INT NOT NULL,
    Notes NVARCHAR(500) NULL,

    CONSTRAINT PK_Organization_StaffingScenarioRequirement PRIMARY KEY (StaffingScenarioRequirementId),
    CONSTRAINT FK_Organization_StaffingScenarioRequirement_StaffingScenario
        FOREIGN KEY (StaffingScenarioId) REFERENCES Organization.StaffingScenario(StaffingScenarioId),
    CONSTRAINT FK_Organization_StaffingScenarioRequirement_Position
        FOREIGN KEY (PositionId) REFERENCES Organization.Position(PositionId),
    CONSTRAINT FK_Organization_StaffingScenarioRequirement_RequiredQualification
        FOREIGN KEY (RequiredQualificationId) REFERENCES Organization.Qualification(QualificationId),
    CONSTRAINT CK_Organization_StaffingScenarioRequirement_RequiredCount CHECK (RequiredCount >= 0)
);
GO

CREATE TABLE Organization.StaffingScenarioResult (
    StaffingScenarioResultId INT IDENTITY(1,1) NOT NULL,
    StaffingScenarioId INT NOT NULL,
    EvaluatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Organization_StaffingScenarioResult_EvaluatedAtUtc DEFAULT
    SYSUTCDATETIME(),
    EvaluatedByUserId INT NULL,
    OverallStatus NVARCHAR(30) NOT NULL,
    Summary NVARCHAR(1000) NULL,

    CONSTRAINT PK_Organization_StaffingScenarioResult PRIMARY KEY (StaffingScenarioResultId),
    CONSTRAINT FK_Organization_StaffingScenarioResult_StaffingScenario
        FOREIGN KEY (StaffingScenarioId) REFERENCES Organization.StaffingScenario(StaffingScenarioId),
    CONSTRAINT FK_Organization_StaffingScenarioResult_EvaluatedByUser
        FOREIGN KEY (EvaluatedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_Organization_StaffingScenarioResult_OverallStatus CHECK (OverallStatus IN
('Pass', 'Warning', 'Fail', 'NotEvaluated'))
);
GO

CREATE TABLE Organization.StaffingScenarioGap (
    StaffingScenarioGapId INT IDENTITY(1,1) NOT NULL,
    StaffingScenarioResultId INT NOT NULL,
    PositionId INT NOT NULL,
    RequiredCount INT NOT NULL,
    AvailableCount INT NOT NULL,
    GapCount AS (CASE WHEN RequiredCount > AvailableCount THEN RequiredCount - AvailableCount ELSE 0 END)
    PERSISTED,
    Notes NVARCHAR(500) NULL,

    CONSTRAINT PK_Organization_StaffingScenarioGap PRIMARY KEY (StaffingScenarioGapId),
    CONSTRAINT FK_Organization_StaffingScenarioGap_StaffingScenarioResult
        FOREIGN KEY (StaffingScenarioResultId) REFERENCES Organization.StaffingScenarioResult(StaffingScenarioResultId),
    CONSTRAINT FK_Organization_StaffingScenarioGap_Position
        FOREIGN KEY (PositionId) REFERENCES Organization.Position(PositionId),
    CONSTRAINT CK_Organization_StaffingScenarioGap_Counts CHECK (RequiredCount >= 0 AND AvailableCount >= 0)
);
GO

```

C.3.5 Indexes and seed examples

Every foreign key receives an index unless it is covered by a primary key or unique constraint. Shift, roster and staffing tables are indexed by site and time because those are the operational access paths.

```
-- Structure indexes
CREATE INDEX IX_Organization_LegalEntity_LegalEntityTypeId ON Organization.LegalEntity(LegalEntityTypeId);
CREATE INDEX IX_Organization_LegalEntity_CountryId ON Organization.LegalEntity(CountryId);
CREATE INDEX IX_Organization_Site_LegalEntityId ON Organization.Site(LegalEntityId);
CREATE INDEX IX_Organization_Site_CountryId_RegionId ON Organization.Site(CountryId, RegionId);
CREATE INDEX IX_Organization_Site_TimeZoneId ON Organization.Site(TimeZoneId);
CREATE INDEX IX_Organization_Plant_SiteId ON Organization.Plant(SiteId);
CREATE INDEX IX_Organization_Building_SiteId ON Organization.Building(SiteId);

-- Org-tree indexes
CREATE INDEX IX_Organization_Department_LegalEntityId ON Organization.Department(LegalEntityId);
CREATE INDEX IX_Organization_Department_ParentDepartmentId ON Organization.Department(ParentDepartmentId);
CREATE INDEX IX_Organization_Team_DepartmentId ON Organization.Team(DepartmentId);
CREATE INDEX IX_Organization_Position_DepartmentId ON Organization.Position(DepartmentId);

-- Person and assignment indexes
CREATE INDEX IX_Organization_Person_ApplicationUserId ON Organization.Person(ApplicationUserId);
CREATE INDEX IX_Organization_Person_LegalEntityId ON Organization.Person(LegalEntityId);
CREATE INDEX IX_Organization_PersonContact_PersonId ON Organization.PersonContact(PersonId);
CREATE INDEX IX_Organization_PersonAddress_PersonId ON Organization.PersonAddress(PersonId);
CREATE INDEX IX_Organization_Employment_PersonId ON Organization.Employment(PersonId);
CREATE INDEX IX_Organization_Employment_ManagerPersonId ON Organization.Employment(ManagerPersonId);
CREATE INDEX IX_Organization_DepartmentAssignment_PersonId_Date ON Organization.DepartmentAssignment(PersonId, StartDate
DESC);
CREATE INDEX IX_Organization_TeamMembership_PersonId_Date ON Organization.TeamMembership(PersonId, StartDate DESC);

-- Shift and competence indexes
CREATE INDEX IX_Organization_Shift_Site_StartsAtUtc ON Organization.Shift(SiteId, StartsAtUtc DESC);
CREATE INDEX IX_Organization_ShiftAssignment_ShiftId ON Organization.ShiftAssignment(ShiftId);
CREATE INDEX IX_Organization_ShiftAssignment_PersonId ON Organization.ShiftAssignment(PersonId);
CREATE INDEX IX_Organization_OnCallRoster_Site_Time ON Organization.OnCallRoster(SiteId, StartsAtUtc, EndsAtUtc);
CREATE INDEX IX_Organization_PersonQualification_PersonId ON Organization.PersonQualification(PersonId);
CREATE INDEX IX_Organization_PersonCertification_PersonId ON Organization.PersonCertification(PersonId);

-- Staffing indexes
CREATE INDEX IX_Organization_PersonnelRequirement_Site_Plant_Position ON Organization.PersonnelRequirement(SiteId, PlantId,
PositionId);
CREATE INDEX IX_Organization_StaffingScenario_SiteId ON Organization.StaffingScenario(SiteId);
CREATE INDEX IX_Organization_StaffingScenarioRequirement_ScenarioId ON
Organization.StaffingScenarioRequirement(StaffingScenarioId);
CREATE INDEX IX_Organization_StaffingScenarioResult_Scenario_Evaluated ON
Organization.StaffingScenarioResult(StaffingScenarioId, EvaluatedAtUtc DESC);
CREATE INDEX IX_Organization_StaffingScenarioGap_ResultId ON Organization.StaffingScenarioGap(StaffingScenarioResultId);
GO
```

C.3.5.1 Minimal seed examples

```
INSERT INTO Organization.LegalEntityType (Code, Name, DisplayOrder) VALUES
('OPERATOR', 'Operating Organization', 10),
('VENDOR', 'Vendor or Supplier', 20),
('REGULATOR', 'Regulatory Body', 30),
('EMERGENCY_PARTNER', 'Emergency Partner', 40);

INSERT INTO Organization.SiteType (Code, Name, DisplayOrder) VALUES
('PLANT_SITE', 'Plant Site', 10),
('HQ', 'Headquarters', 20),
('TRAINING_CENTER', 'Training Centre', 30),
('WAREHOUSE', 'Warehouse or Stores', 40);

INSERT INTO Organization.DepartmentType (Code, Name, DisplayOrder) VALUES
('OPS', 'Operations', 10),
('MAINT', 'Maintenance', 20),
('ENG', 'Engineering', 30),
('SAFETY', 'Safety', 40),
('RAD_PROT', 'Radiation Protection', 50),
('SECURITY', 'Security', 60);

INSERT INTO Organization.PersonType (Code, Name, DisplayOrder) VALUES
('EMPLOYEE', 'Employee', 10),
('CONTRACTOR', 'Contractor', 20),
('VENDOR_REP', 'Vendor Representative', 30),
('REGULATOR', 'Regulator', 40);

INSERT INTO Organization.ShiftType (Code, Name, DisplayOrder) VALUES
('DAY', 'Day Shift', 10),
```

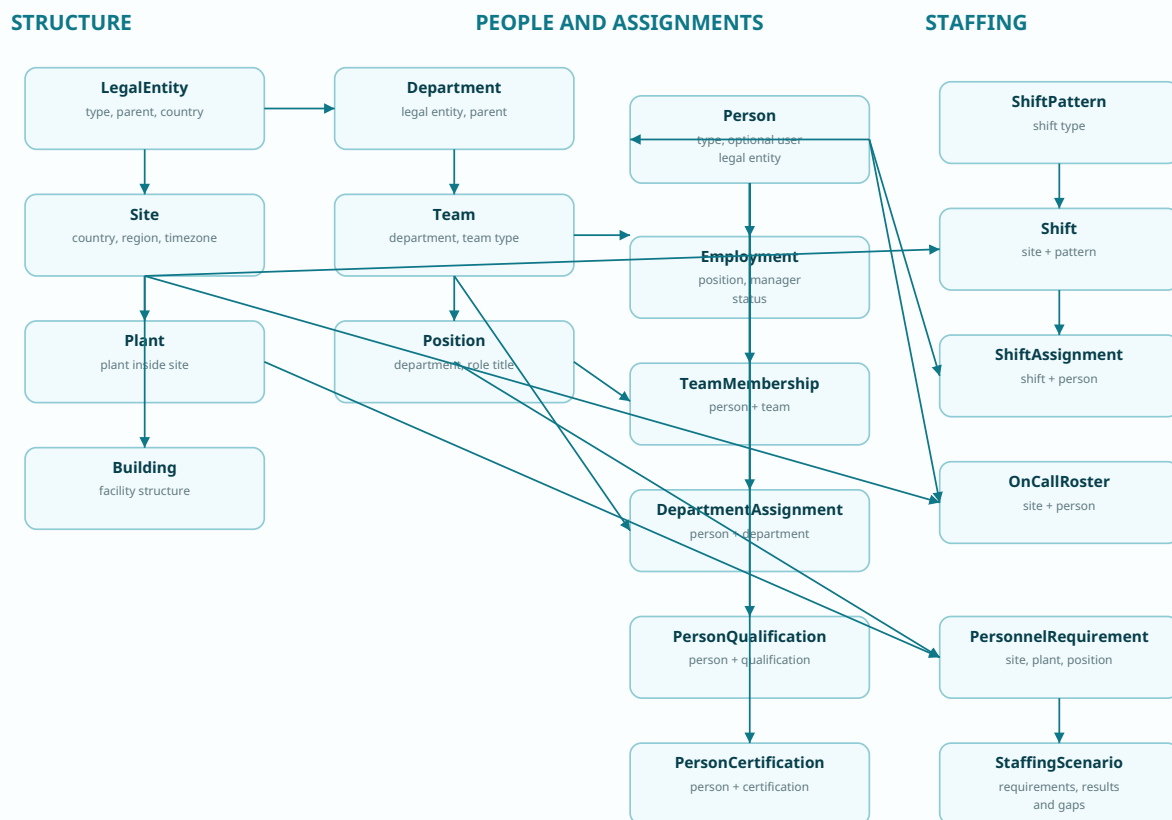
```

('NIGHT', 'Night Shift', 20),
('ROTATING', 'Rotating Shift', 30),
('EMERGENCY', 'Emergency Shift', 40);
GO

```

C.3.6 ER diagram

The ER diagram shows the sector in four clusters: institutional structure, teams and positions, people and assignments, and staffing/shift records.



Lookup catalogues are omitted from the main drawing for readability: legal entity type, site type, plant type, department type, team type, person type, employment status, shift type, qualification status, ai

Figure C.3.1 - Organization sector ER diagram. Lookup tables and some verification-user relationships are omitted from the drawing for readability; they are included in the full FK mapping.

C.3.7 Foreign-key mapping

C.3.7.1 Internal Organization foreign keys

FROM TABLE	FK COLUMN	TO TABLE	PK COLUMN	RELATIONSHIP
Organization.LegalEntity	LegalEntityTypeId	Organization.LegalEntityType	LegalEntityTypeId	Many legal entities per type.
Organization.LegalEntity	ParentLegalEntityId	Organization.LegalEntity	LegalEntityId	Optional legal-entity hierarchy.
Organization.Site	LegalEntityId	Organization.LegalEntity	LegalEntityId	Site owned or operated by legal entity.
Organization.Site	SiteTypeId	Organization.SiteType	SiteTypeId	Site classification.
Organization.Plant	SiteId	Organization.Site	SiteId	Many plants per site.
Organization.Plant	PlantTypeId	Organization.PlantType	PlantTypeId	Plant classification.
Organization.Building	SiteId	Organization.Site	SiteId	Many buildings per site.
Organization.Department	LegalEntityId	Organization.LegalEntity	LegalEntityId	Many departments per legal entity.
Organization.Department	ParentDepartmentId	Organization.Department	DepartmentId	Optional department hierarchy.
Organization.Department	DepartmentTypeId	Organization.DepartmentType	DepartmentTypeId	Department classification.
Organization.Team	DepartmentId	Organization.Department	DepartmentId	Many teams per department.
Organization.Team	TeamTypeId	Organization.TeamType	TeamTypeId	Team classification.
Organization.Position	DepartmentId	Organization.Department	DepartmentId	Optional owning department.
Organization.Person	PersonTypeId	Organization.PersonType	PersonTypeId	Person classification.
Organization.Person	LegalEntityId	Organization.LegalEntity	LegalEntityId	Primary affiliated entity.
Organization.PersonContact	PersonId	Organization.Person	PersonId	Many contact routes per person.
Organization.PersonContact	ContactMethodTypeId	Organization.ContactMethodType	ContactMethodTypeId	Contact route classification.
Organization.PersonAddress	PersonId	Organization.Person	PersonId	Many addresses per person.
Organization.Employment	PersonId	Organization.Person	PersonId	Many employments per person over time.

FROM TABLE	FK COLUMN	TO TABLE	PK COLUMN	RELATIONSHIP
Organization.Employment	LegalEntityId	Organization.LegalEntity	LegalEntityId	Employer entity.
Organization.Employment	PositionId	Organization.Position	PositionId	Employment position.
Organization.Employment	ManagerPersonId	Organization.Person	PersonId	Manager chain.
Organization.ContractorsEngagement	PersonId	Organization.Person	PersonId	Contractor person.
Organization.ContractorsEngagement	VendorLegalEntityId	Organization.LegalEntity	LegalEntityId	Vendor company.
Organization.DepartmentAssignment	PersonId	Organization.Person	PersonId	Person assigned to department.
Organization.TeamMembership	PersonId	Organization.Person	PersonId	Person belongs to team.
Organization.Shift	SiteId	Organization.Site	SiteId	Shift belongs to site.
Organization.Shift	ShiftPatternId	Organization.ShiftPattern	ShiftPatternId	Shift uses pattern.
Organization.ShiftAssignment	ShiftId	Organization.Shift	ShiftId	Person assigned to shift.
Organization.ShiftAssignment	PersonId	Organization.Person	PersonId	Person works shift.
Organization.PersonQualification	PersonId	Organization.Person	PersonId	Person holds qualification.
Organization.PersonQualification	QualificationId	Organization.Qualification	QualificationId	Qualification held.
Organization.PersonCertification	PersonId	Organization.Person	PersonId	Person holds certification.
Organization.PersonCertification	CertificationId	Organization.Certification	CertificationId	Certification held.
Organization.PersonnelRequirement	SiteId	Organization.Site	SiteId	Requirement scoped to site.
Organization.PersonnelRequirement	PlantId	Organization.Plant	PlantId	Optional plant scope.
Organization.StaffingScenarioRequirement	StaffingScenarioId	Organization.StaffingScenario	StaffingScenarioId	Requirements inside scenario.
Organization.StaffingScenarioResult	StaffingScenarioId	Organization.StaffingScenario	StaffingScenarioId	Results for scenario.
Organization.StaffingScenarioGap	StaffingScenarioResultId	Organization.StaffingScenarioResult	StaffingScenarioResultId	Gaps in result.

C.3.7.2 CorePlatform passports consumed by Organization

FROM TABLE	FK COLUMN	TO TABLE	PK COLUMN	RELATIONSHIP
Organization.LegalEntity	CountryId	CorePlatform.Country	CountryId	Country of registration or operation.
Organization.Site	CountryId	CorePlatform.Country	CountryId	Physical site country.
Organization.Site	RegionId	CorePlatform.Region	RegionId	Physical site region.
Organization.Site	TimeZoneId	CorePlatform.TimeZone	TimeZoneId	Local wall-clock and scheduling context.
Organization.PersonAddress	CountryId	CorePlatform.Country	CountryId	Address country.
Organization.PersonAddress	RegionId	CorePlatform.Region	RegionId	Address region.

C.3.7.3 Security passports consumed by Organization

FROM TABLE	FK COLUMN	TO TABLE	PK COLUMN	RELATIONSHIP
Organization.Person	ApplicationUserId	Security.ApplicationUser	ApplicationUserId	Optional login account for a person.
Organization.ShiftAssignment	ConfirmedByUserId	Security.ApplicationUser	ApplicationUserId	Who confirmed the shift assignment.
Organization.PersonQualification	VerifiedByUserId	Security.ApplicationUser	ApplicationUserId	Who verified a qualification.
Organization.PersonCertification	VerifiedByUserId	Security.ApplicationUser	ApplicationUserId	Who verified a certification.
Organization.StaffingScenario	CreatedByUserId	Security.ApplicationUser	ApplicationUserId	Who created the scenario.
Organization.StaffingScenarioResult	EvaluatedByUserId	Security.ApplicationUser	ApplicationUserId	Who evaluated the scenario.

C.3.7.4 Incoming passports from later sectors

The following relationships are declared in later sectors. They are listed here to show why Organization is an early dependency in the whole database.

ReactorFleet.Unit

FK PlantId
TO Organization.Plant
A reactor unit belongs to one plant.

Instrumentation.Signal

FK SiteId / PlantId
TO Organization.Site / Plant
Signals can be scoped to the organizational plant context.

RadiationMonitoring.RadiationZone

FK BuildingId / SiteId

TO Organization.Building / Site

Radiation zones are placed inside organizational structures.

Maintenance.WorkOrder

FK AssignedTeamId / RequestedByPersonId

TO Organization.Team / Person

Work is assigned to teams and people.

EmergencyPreparedness.DrillParticipant

FK PersonId / TeamId

TO Organization.Person / Team

Drills use real organizational rosters.

Compliance.AuditScope

FK LegalEntityId / SiteId / DepartmentId

TO Organization structure

Audits are scoped to organizational owners.

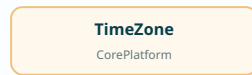
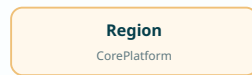
Reporting.ReportFilter

FK SiteId / DepartmentId / TeamId

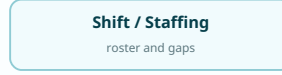
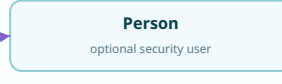
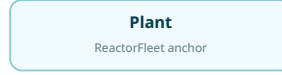
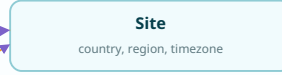
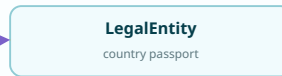
TO Organization structure

Reports are filtered by site, department, and team.

CORE PLATFORM



ORGANIZATION



LATER SECTORS

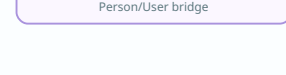
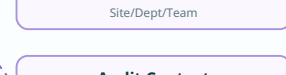
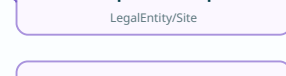
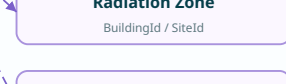
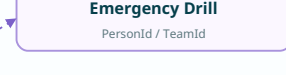
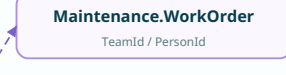


Figure C.3.2 - Organization passport map. Organization consumes CorePlatform and Security, then becomes a passport provider for the plant, maintenance, radiation, emergency, compliance and reporting sectors.

C.3.8 Useful verification queries

These queries validate that the sector joins identity, people, departments, teams, shifts and staffing gaps without duplicating keys.

```
-- 1. Find the plant hierarchy for a site.
SELECT s.Code AS SiteCode, p.Code AS PlantCode, p.Name AS PlantName
FROM Organization.Site AS s
JOIN Organization.Plant AS p ON p.SiteId = s.SiteId
WHERE s.Code = 'NX1-SITE';

-- 2. Resolve a login account to its person, department and team.
SELECT u.UserName, pr.DisplayName, d.Name AS Department, t.Name AS Team
FROM Security.ApplicationUser AS u
JOIN Organization.Person AS pr ON pr.ApplicationUserId = u.ApplicationUserId
LEFT JOIN Organization.DepartmentAssignment AS da ON da.PersonId = pr.PersonId AND da.EndDate IS NULL
LEFT JOIN Organization.Department AS d ON d.DepartmentId = da.DepartmentId
LEFT JOIN Organization.TeamMembership AS tm ON tm.PersonId = pr.PersonId AND tm.EndDate IS NULL
LEFT JOIN Organization.Team AS t ON t.TeamId = tm.TeamId
WHERE u.UserName = 'operator.u1';

-- 3. Check staffing gaps for the latest stress-test result.
SELECT ss.ScenarioCode, r.EvaluatedAtUtc, pos.Title,
       g.RequiredCount, g.AvailableCount, g.GapCount
FROM Organization.StaffingScenario AS ss
JOIN Organization.StaffingScenarioResult AS r ON r.StaffingScenarioId = ss.StaffingScenarioId
JOIN Organization.StaffingScenarioGap AS g ON g.StaffingScenarioResultId = r.StaffingScenarioResultId
JOIN Organization.Position AS pos ON pos.PositionId = g.PositionId
WHERE r.EvaluatedAtUtc = (
    SELECT MAX(r2.EvaluatedAtUtc)
    FROM Organization.StaffingScenarioResult AS r2
    WHERE r2.StaffingScenarioId = ss.StaffingScenarioId
);
```

C.3.9 Boundary and next sector

Honest boundary. This schema is an enterprise demonstrator organization model. It is not a human-resources system, payroll system, medical fitness system, or official personnel-licensing register. It deliberately avoids highly personal data that the NEXUS-1 demonstrator does not need. The point is operational structure: who belongs where, who can be assigned, who is qualified, and what organizational object later plant records can reference.

The next sector is `ReactorFleet`. That is where the institutional plant becomes the physical machine: fleet, unit, reactor, core, assemblies, rods, steam generators, pumps and component registry. Its first major passport is already prepared here:

`Organization.Plant.PlantId`.

Appendix C.4

ReactorFleet Schema

The ReactorFleet sector owns the physical plant model: fleet, plant unit, reactor, core, control rods, fuel assemblies, primary and secondary equipment, power conversion and grid interface.

Organization gave the platform its sites, plants and people. ReactorFleet turns one organizational plant into a machine with systems, equipment, components, loops, reactor internals and snapshots. It is the first major domain schema and one of the main passport providers for the rest of the database.

*A **UnitId** is not a screen filter. It is the spine of the database. Signals, alarms, twin models, causal nodes, maintenance assets and RL episodes all point back to the same unit.*

C.4.1

Sector purpose

C.4.2

Full table list

C.4.3

Lookup tables

C.4.4

SQL DDL

C.4.5

Indexes and seed examples

C.4.6

ER diagram

C.4.7

Foreign-key mapping

C.4.8

Boundary and next sector

C.4.1 Sector purpose

ReactorFleet follows Organization because a reactor unit must belong to an organizational plant before it can be modelled as a machine. It consumes Organization.Plant, Organization.Building, Organization.LegalEntity, CorePlatform.EngineeringUnit, CorePlatform.TimeZone, and Security.ApplicationUser.

- It owns the physical hierarchy from fleet to unit to reactor, core and equipment.
- It owns the component registry through ReactorFleet.Equipment.
- It owns system and equipment dependency edges used later by instrumentation, alarms, root cause and digital twin screens.
- It stores slow operational snapshots: unit power, fleet output, rod position and chemistry snapshots.

Design choice. Equipment is the common physical-component table. Specific tables such as SteamGenerator, ReactorCoolantPump, Generator and Pressurizer add domain-specific columns while keeping the generic component identity in one place.

C.4.2 Full table list

The ReactorFleet sector contains forty-eight tables: sixteen lookup tables and thirty-two substantive tables. It is intentionally large because it is the physical tree that later sectors depend on.

LOOKUP

ReactorFleet.UnitType

PK UnitTypeId
FK None

Classifies the unit as large PWR, compact PWR, SMR module, simulator unit, or support unit.

LOOKUP

ReactorFleet.UnitStatus

PK UnitStatusId
FK None

Lifecycle and operating status: planned, online, offline, outage, shutdown, decommissioning.

LOOKUP

ReactorFleet.OperationalMode

PK OperationalModeId
FK None

Console operating mode for the unit: power, load-follow, startup, shutdown, trip, maintenance, training.

LOOKUP

ReactorFleet.ReactorType

PK ReactorTypeId
FK None

Classifies the reactor model: PWR, compact PWR, SMR-PWR demonstrator, simulator.

LOOKUP

ReactorFleet.CoolantLoopType

PK CoolantLoopTypeId
FK None

Classifies primary loops and integral compact loops.

LOOKUP

ReactorFleet.SystemType

PK SystemTypeId
FK None

Classifies plant systems: primary, secondary, electrical, safety, support, I&C.

LOOKUP

ReactorFleet.EquipmentType

PK EquipmentTypeId
FK None

Classifies equipment: pump, valve, steam generator, pressurizer, turbine, generator, transformer.

LOOKUP

ReactorFleet.EquipmentStatus

PK EquipmentStatusId
FK None

Classifies equipment condition: available, running, standby, degraded, unavailable, retired.

LOOKUP

ReactorFleet.EquipmentCriticality

PK EquipmentCriticalityId

FK None

Classifies operational criticality for maintenance, RCA, and dashboards.

LOOKUP

ReactorFleet.FuelAssemblyType

PK FuelAssemblyTypeId

FK None

Classifies fuel assembly design families and lattice arrangements.

LOOKUP

ReactorFleet.FuelMaterialType

PK FuelMaterialTypeId

FK None

Classifies illustrative fuel material families used by the demonstration.

LOOKUP

ReactorFleet.ControlRodMaterial

PK ControlRodMaterialId

FK None

Classifies absorber materials such as B4C or Ag-In-Cd in the demonstration vocabulary.

LOOKUP

ReactorFleet.RodBankType

PK RodBankTypeId

FK None

Classifies regulating, shutdown, safety, and calibration rod banks.

LOOKUP

ReactorFleet.SteamGeneratorType

PK SteamGeneratorTypeId

FK None

Classifies U-tube, once-through, compact, or integral steam-generator representations.

LOOKUP

ReactorFleet.PumpType

PK PumpTypeId

FK None

Classifies reactor coolant, feedwater, condensate, auxiliary, and service-water pumps.

LOOKUP

ReactorFleet.GridConnectionStatus

PK GridConnectionStatusId

FK None

Classifies synchronized, isolated, trip, islanded, test, or disconnected grid state.

STRUCTURE

ReactorFleet.Fleet

PK FleetId

FK OperatorLegalEntityId , TimeZoneId

Top-level grouping of plant units for the fleet dashboard and aggregate views.

STRUCTURE

ReactorFleet.FleetPlant

PK FleetPlantId

FK FleetId , PlantId

Connects a fleet to the organization plants it contains without duplicating plant identity.

STRUCTURE**ReactorFleet.Unit**

PK UnitId
FK PlantId, UnitTypeId, UnitStatusId,
ReactorTypeId

One physical or simulated reactor unit; the most important passport provider for later sectors.

STRUCTURE**ReactorFleet.UnitDesignParameter**

PK UnitDesignParameterId
FK UnitId, EngineeringUnitId

Named design constants: thermal rating, pressure, loop count, fuel mass, design limits.

STRUCTURE**ReactorFleet.UnitOperationalState**

PK UnitOperationalStateId
FK UnitId, OperationalModeId,
RecordedByUserId

Append-style operating-state history for the unit.

SYSTEMS**ReactorFleet.PlantSystem**

PK PlantSystemId
FK UnitId, SystemTypeId, ParentPlantSystemId

Functional system tree: primary loop, feedwater, turbine train, electrical, safety, service.

SYSTEMS**ReactorFleet.SystemDependency**

PK SystemDependencyId
FK SourcePlantSystemId, TargetPlantSystemId

Directed dependency between systems, used by System Dependencies and RCA preparation.

EQUIPMENT**ReactorFleet.EquipmentLocation**

PK EquipmentLocationId
FK UnitId, BuildingId

Physical or logical location used by component registry, maintenance, radiation and access views.

EQUIPMENT**ReactorFleet.Equipment**

PK EquipmentId
FK UnitId, PlantSystemId, EquipmentTypeId,
EquipmentStatusId

The central component registry table. Later sectors reference equipment instead of inventing component IDs.

EQUIPMENT**ReactorFleet.EquipmentDependency**

PK EquipmentDependencyId
FK SourceEquipmentId, TargetEquipmentId

Directed component-to-component influence with delay and signed weight.

EQUIPMENT**ReactorFleet.EquipmentExternalReference**

PK EquipmentExternalReferenceId
FK EquipmentId

External tag, vendor, drawing, procedure, or document reference tied to one equipment row.

REACTOR**ReactorFleet.Reactor**

PK ReactorId
FK UnitId, ReactorTypeId,
ReactorVesselEquipmentId

The reactor vessel and reactor-level design anchor for a unit.

REACTOR

ReactorFleet.ReactorCore

PK ReactorCoreId

FK ReactorId

Core loading and neutronic design envelope for a reactor.

REACTOR

ReactorFleet.CoreRegion

PK CoreRegionId

FK ReactorCoreId

Named core regions used by fuel assembly and flux-map tables.

REACTOR

ReactorFleet.FuelAssembly

PK FuelAssemblyId

FK ReactorCoreId , CoreRegionId ,
FuelAssemblyTypeId

Fuel assembly identity, position and design information for the illustrative core.

REACTOR

ReactorFleet.ControlRodBank

PK ControlRodBankId

FK ReactorCoreId , RodBankTypeId ,
ControlRodMaterialId

Bank-level rod grouping used by operator controls, RL policy and snapshots.

REACTOR

ReactorFleet.ControlRod

PK ControlRodId

FK ControlRodBankId , EquipmentId

Individual rods inside a bank. Equipment link is optional but useful for maintenance.

REACTOR

ReactorFleet.RodPositionSnapshot

PK RodPositionSnapshotId

FK ControlRodBankId

Time-series snapshots of rod-bank position.

REACTOR

ReactorFleet.BoronChemistrySnapshot

PK BoronChemistrySnapshotId

FK ReactorCoreId

Time-series snapshots of soluble absorber and chemistry values.

REACTOR

ReactorFleet.NeutronicsParameterSet

PK NeutronicsParameterSetId

FK ReactorCoreId

Versioned point-kinetics parameter set for the demonstration solver.

PRIMARY/SECONDARY

ReactorFleet.PrimaryLoop

PK PrimaryLoopId

FK ReactorId , CoolantLoopTypeId

Primary loop or compact/integral loop representation.

PRIMARY/SECONDARY

ReactorFleet.ReactorCoolantPump

PK ReactorCoolantPumpId

FK PrimaryLoopId , EquipmentId , PumpTypeId

Reactor coolant pump linked to equipment registry and primary loop.

PRIMARY/SECONDARY

ReactorFleet.Pressurizer

PK PressurizerId

FK UnitId, EquipmentId

Pressurizer representation and design envelope.

PRIMARY/SECONDARY

ReactorFleet.SteamGenerator

PK SteamGeneratorId

FK PrimaryLoopId, EquipmentId,
SteamGeneratorTypeId

Steam-generator object bridging primary and secondary sides.

POWER CONVERSION

ReactorFleet.TurbineTrain

PK TurbineTrainId

FK UnitId, EquipmentId

Main turbine train for the unit.

POWER CONVERSION

ReactorFleet.TurbineStage

PK TurbineStageId

FK TurbineTrainId

HP, IP, LP or compact turbine stage.

POWER CONVERSION

ReactorFleet.Generator

PK GeneratorId

FK UnitId, EquipmentId

Electrical generator connected to the turbine train and grid interface.

POWER CONVERSION

ReactorFleet.Transformer

PK TransformerId

FK UnitId, EquipmentId

Generator step-up or auxiliary transformer.

POWER CONVERSION

ReactorFleet.SwitchyardBay

PK SwitchyardBayId

FK UnitId

Switchyard bay or electrical connection point.

POWER CONVERSION

ReactorFleet.GridConnection

PK GridConnectionId

FK UnitId, GeneratorId, TransformerId,
GridConnectionStatusId

Grid interface state and design values.

SNAPSHOTS

ReactorFleet.FleetSnapshot

PK FleetSnapshotId

FK FleetId

Fleet-level aggregate output, online count and demand snapshot.

SNAPSHOTS

ReactorFleet.UnitPowerSnapshot

PK UnitPowerSnapshotId

FK UnitId, OperationalModeId

Unit-level output, load, frequency and power-factor snapshot.

C.4.3 Lookup tables

Lookup tables use the standard Appendix C.0 shape. The values are demonstrator vocabulary, not a licensed plant taxonomy.

LOOKUP TABLE	TYPICAL SEED CODES	MAIN CONSUMER
UnitType	PWR_900 , PWR_1300 , SMR_170 , SIMULATOR	Referenced by Unit .
UnitStatus	ONLINE , OFFLINE , OUTAGE , TRIP , DECOMMISSIONING	Referenced by Unit .
OperationalMode	POWER , LOAD_FOLLOW , STARTUP , SHUTDOWN , TRAINING	Referenced by unit state and snapshots.
ReactorType	PWR , COMPACT_PWR , SMR_PWR , TRAINING_CORE	Referenced by Unit and Reactor .
CoolantLoopType	PRIMARY_LOOP , INTEGRAL_LOOP , COMPACT_LOOP	Referenced by PrimaryLoop .
SystemType	PRIMARY , SECONDARY , ELECTRICAL , SAFETY , SUPPORT	Referenced by PlantSystem .
EquipmentType	PUMP , VALVE , STEAM_GENERATOR , PRESSURIZER , TURBINE	Referenced by Equipment .
EquipmentStatus	AVAILABLE , RUNNING , STANDBY , DEGRADED , UNAVAILABLE	Referenced by Equipment .
EquipmentCriticality	LOW , MEDIUM , HIGH , CRITICAL	Referenced by Equipment .
FuelAssemblyType	STANDARD_17X17 , GUIDE_TUBE , INSTRUMENTED , DUMMY	Referenced by FuelAssembly .
FuelMaterialType	UO2 , MOX_DEMO , SIMULATED	Referenced by FuelAssembly .
ControlRodMaterial	B4C , AG_IN_CD , HAFNIUM , SIMULATED	Referenced by rod banks and rods.
RodBankType	REGULATING , SHUTDOWN , SAFETY , CALIBRATION	Referenced by ControlRodBank .
SteamGeneratorType	U_TUBE , ONCE_THROUGH , INTEGRAL , COMPACT	Referenced by SteamGenerator .
PumpType	RCP , FEEDWATER , CONDENSATE , AUXILIARY	Referenced by pump tables.
GridConnectionStatus	SYNCHRONIZED , ISLANDED , TRIPPED , TEST	Referenced by GridConnection .

C.4.4 SQL DDL

C.4.4.1 Schema

```
IF SCHEMA_ID(N'ReactorFleet') IS NULL
EXEC(N'CREATE SCHEMA ReactorFleet');
GO
```

C.4.4.2 ReactorFleet lookup tables

```
CREATE TABLE ReactorFleet.UnitType (
  UnitTypeId      INT IDENTITY(1,1) NOT NULL,
  Code            NVARCHAR(50) NOT NULL,
  Name            NVARCHAR(150) NOT NULL,
  Description      NVARCHAR(500) NULL,
  DisplayOrder    INT NOT NULL
  CONSTRAINT DF_ReactorFleet_UnitType_DisplayOrder DEFAULT 0,
  IsActive        BIT NOT NULL
  CONSTRAINT DF_ReactorFleet_UnitType_IsActive DEFAULT 1,
  CreatedAtUtc    DATETIME2(7) NOT NULL
  CONSTRAINT DF_ReactorFleet_UnitType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedAtUtc   DATETIME2(7) NOT NULL
  CONSTRAINT DF_ReactorFleet_UnitType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  RowVersion      ROWVERSION NOT NULL,

  CONSTRAINT PK_ReactorFleet_UnitType
    PRIMARY KEY (UnitTypeId),
  CONSTRAINT UQ_ReactorFleet_UnitType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE ReactorFleet.UnitStatus (
  UnitStatusId    INT IDENTITY(1,1) NOT NULL,
  Code            NVARCHAR(50) NOT NULL,
  Name            NVARCHAR(150) NOT NULL,
  Description      NVARCHAR(500) NULL,
  DisplayOrder    INT NOT NULL
  CONSTRAINT DF_ReactorFleet_UnitStatus_DisplayOrder DEFAULT 0,
  IsActive        BIT NOT NULL
  CONSTRAINT DF_ReactorFleet_UnitStatus_IsActive DEFAULT 1,
  CreatedAtUtc    DATETIME2(7) NOT NULL
  CONSTRAINT DF_ReactorFleet_UnitStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedAtUtc   DATETIME2(7) NOT NULL
  CONSTRAINT DF_ReactorFleet_UnitStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  RowVersion      ROWVERSION NOT NULL,

  CONSTRAINT PK_ReactorFleet_UnitStatus
    PRIMARY KEY (UnitStatusId),
  CONSTRAINT UQ_ReactorFleet_UnitStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE ReactorFleet.OperationalMode (
  OperationalModeId INT IDENTITY(1,1) NOT NULL,
  Code            NVARCHAR(50) NOT NULL,
  Name            NVARCHAR(150) NOT NULL,
  Description      NVARCHAR(500) NULL,
  DisplayOrder    INT NOT NULL
  CONSTRAINT DF_ReactorFleet_OperationalMode_DisplayOrder DEFAULT 0,
  IsActive        BIT NOT NULL
  CONSTRAINT DF_ReactorFleet_OperationalMode_IsActive DEFAULT 1,
  CreatedAtUtc    DATETIME2(7) NOT NULL
  CONSTRAINT DF_ReactorFleet_OperationalMode_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedAtUtc   DATETIME2(7) NOT NULL
  CONSTRAINT DF_ReactorFleet_OperationalMode_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  RowVersion      ROWVERSION NOT NULL,

  CONSTRAINT PK_ReactorFleet_OperationalMode
    PRIMARY KEY (OperationalModeId),
  CONSTRAINT UQ_ReactorFleet_OperationalMode_Code
    UNIQUE (Code)
);
GO

CREATE TABLE ReactorFleet.ReactorType (
  ReactorTypeId    INT IDENTITY(1,1) NOT NULL,
  Code            NVARCHAR(50) NOT NULL,
  Name            NVARCHAR(150) NOT NULL,
  Description      NVARCHAR(500) NULL,
  DisplayOrder    INT NOT NULL
  CONSTRAINT DF_ReactorFleet_ReactorType_DisplayOrder DEFAULT 0,
  IsActive        BIT NOT NULL
  CONSTRAINT DF_ReactorFleet_ReactorType_IsActive DEFAULT 1,
  CreatedAtUtc    DATETIME2(7) NOT NULL
  CONSTRAINT DF_ReactorFleet_ReactorType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedAtUtc   DATETIME2(7) NOT NULL
  CONSTRAINT DF_ReactorFleet_ReactorType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  RowVersion      ROWVERSION NOT NULL,

  CONSTRAINT PK_ReactorFleet_ReactorType
    PRIMARY KEY (ReactorTypeId),
  CONSTRAINT UQ_ReactorFleet_ReactorType_Code
    UNIQUE (Code)
);
GO
```



```

CREATE TABLE ReactorFleet.CoolantLoopType (
    CoolantLoopTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_ReactorFleet_CoolantLoopType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_ReactorFleet_CoolantLoopType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReactorFleet_CoolantLoopType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReactorFleet_CoolantLoopType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReactorFleet_CoolantLoopType
    PRIMARY KEY (CoolantLoopTypeId),
    CONSTRAINT UQ_ReactorFleet_CoolantLoopType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE ReactorFleet.SystemType (
    SystemTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_ReactorFleet_SystemType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_ReactorFleet_SystemType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReactorFleet_SystemType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReactorFleet_SystemType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReactorFleet_SystemType
    PRIMARY KEY (SystemTypeId),
    CONSTRAINT UQ_ReactorFleet_SystemType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE ReactorFleet.EquipmentType (
    EquipmentTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_ReactorFleet_EquipmentType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_ReactorFleet_EquipmentType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReactorFleet_EquipmentType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReactorFleet_EquipmentType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReactorFleet_EquipmentType
    PRIMARY KEY (EquipmentTypeId),
    CONSTRAINT UQ_ReactorFleet_EquipmentType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE ReactorFleet.EquipmentStatus (
    EquipmentStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_ReactorFleet_EquipmentStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_ReactorFleet_EquipmentStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReactorFleet_EquipmentStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReactorFleet_EquipmentStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReactorFleet_EquipmentStatus
    PRIMARY KEY (EquipmentStatusId),
    CONSTRAINT UQ_ReactorFleet_EquipmentStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE ReactorFleet.EquipmentCriticality (

```

```

EquipmentCriticalityId      INT IDENTITY(1,1) NOT NULL,
Code                        NVARCHAR(50) NOT NULL,
Name                       NVARCHAR(150) NOT NULL,
Description                 NVARCHAR(500) NULL,
DisplayOrder               INT NOT NULL
    CONSTRAINT DF_ReactorFleet_EquipmentCriticality_DisplayOrder DEFAULT 0,
IsActive                   BIT NOT NULL
    CONSTRAINT DF_ReactorFleet_EquipmentCriticality_IsActive DEFAULT 1,
CreatedAtUtc               DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReactorFleet_EquipmentCriticality_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc              DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReactorFleet_EquipmentCriticality_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion                 ROWVERSION NOT NULL,

CONSTRAINT PK_ReactorFleet_EquipmentCriticality
    PRIMARY KEY (EquipmentCriticalityId),
CONSTRAINT UQ_ReactorFleet_EquipmentCriticality_Code
    UNIQUE (Code)
);
GO

CREATE TABLE ReactorFleet.FuelAssemblyType (
    FuelAssemblyTypeId      INT IDENTITY(1,1) NOT NULL,
    Code                    NVARCHAR(50) NOT NULL,
    Name                    NVARCHAR(150) NOT NULL,
    Description              NVARCHAR(500) NULL,
    DisplayOrder            INT NOT NULL
        CONSTRAINT DF_ReactorFleet_FuelAssemblyType_DisplayOrder DEFAULT 0,
    IsActive                BIT NOT NULL
        CONSTRAINT DF_ReactorFleet_FuelAssemblyType_IsActive DEFAULT 1,
    CreatedAtUtc            DATETIME2(7) NOT NULL
        CONSTRAINT DF_ReactorFleet_FuelAssemblyType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc           DATETIME2(7) NOT NULL
        CONSTRAINT DF_ReactorFleet_FuelAssemblyType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion              ROWVERSION NOT NULL,

CONSTRAINT PK_ReactorFleet_FuelAssemblyType
    PRIMARY KEY (FuelAssemblyTypeId),
CONSTRAINT UQ_ReactorFleet_FuelAssemblyType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE ReactorFleet.FuelMaterialType (
    FuelMaterialTypeId      INT IDENTITY(1,1) NOT NULL,
    Code                    NVARCHAR(50) NOT NULL,
    Name                    NVARCHAR(150) NOT NULL,
    Description              NVARCHAR(500) NULL,
    DisplayOrder            INT NOT NULL
        CONSTRAINT DF_ReactorFleet_FuelMaterialType_DisplayOrder DEFAULT 0,
    IsActive                BIT NOT NULL
        CONSTRAINT DF_ReactorFleet_FuelMaterialType_IsActive DEFAULT 1,
    CreatedAtUtc            DATETIME2(7) NOT NULL
        CONSTRAINT DF_ReactorFleet_FuelMaterialType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc           DATETIME2(7) NOT NULL
        CONSTRAINT DF_ReactorFleet_FuelMaterialType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion              ROWVERSION NOT NULL,

CONSTRAINT PK_ReactorFleet_FuelMaterialType
    PRIMARY KEY (FuelMaterialTypeId),
CONSTRAINT UQ_ReactorFleet_FuelMaterialType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE ReactorFleet.ControlRodMaterial (
    ControlRodMaterialId    INT IDENTITY(1,1) NOT NULL,
    Code                    NVARCHAR(50) NOT NULL,
    Name                    NVARCHAR(150) NOT NULL,
    Description              NVARCHAR(500) NULL,
    DisplayOrder            INT NOT NULL
        CONSTRAINT DF_ReactorFleet_ControlRodMaterial_DisplayOrder DEFAULT 0,
    IsActive                BIT NOT NULL
        CONSTRAINT DF_ReactorFleet_ControlRodMaterial_IsActive DEFAULT 1,
    CreatedAtUtc            DATETIME2(7) NOT NULL
        CONSTRAINT DF_ReactorFleet_ControlRodMaterial_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc           DATETIME2(7) NOT NULL
        CONSTRAINT DF_ReactorFleet_ControlRodMaterial_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion              ROWVERSION NOT NULL,

CONSTRAINT PK_ReactorFleet_ControlRodMaterial
    PRIMARY KEY (ControlRodMaterialId),
CONSTRAINT UQ_ReactorFleet_ControlRodMaterial_Code
    UNIQUE (Code)
);
GO

CREATE TABLE ReactorFleet.RodBankType (
    RodBankTypeId           INT IDENTITY(1,1) NOT NULL,
    Code                    NVARCHAR(50) NOT NULL,
    Name                    NVARCHAR(150) NOT NULL,

```

```

Description    NVARCHAR(500) NULL,
DisplayOrder   INT NOT NULL
    CONSTRAINT DF_ReactorFleet_RodBankType_DisplayOrder DEFAULT 0,
IsActive       BIT NOT NULL
    CONSTRAINT DF_ReactorFleet_RodBankType_IsActive DEFAULT 1,
CreatedAtUtc   DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReactorFleet_RodBankType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc  DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReactorFleet_RodBankType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion     ROWVERSION NOT NULL,

CONSTRAINT PK_ReactorFleet_RodBankType
    PRIMARY KEY (RodBankTypeId),
CONSTRAINT UQ_ReactorFleet_RodBankType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE ReactorFleet.SteamGeneratorType (
    SteamGeneratorTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_ReactorFleet_SteamGeneratorType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_ReactorFleet_SteamGeneratorType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReactorFleet_SteamGeneratorType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReactorFleet_SteamGeneratorType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_ReactorFleet_SteamGeneratorType
    PRIMARY KEY (SteamGeneratorTypeId),
CONSTRAINT UQ_ReactorFleet_SteamGeneratorType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE ReactorFleet.PumpType (
    PumpTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_ReactorFleet_PumpType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_ReactorFleet_PumpType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReactorFleet_PumpType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReactorFleet_PumpType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_ReactorFleet_PumpType
    PRIMARY KEY (PumpTypeId),
CONSTRAINT UQ_ReactorFleet_PumpType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE ReactorFleet.GridConnectionStatus (
    GridConnectionStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_ReactorFleet_GridConnectionStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_ReactorFleet_GridConnectionStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReactorFleet_GridConnectionStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReactorFleet_GridConnectionStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_ReactorFleet_GridConnectionStatus
    PRIMARY KEY (GridConnectionStatusId),
CONSTRAINT UQ_ReactorFleet_GridConnectionStatus_Code
    UNIQUE (Code)
);
GO

```

C.4.4.3 Fleet, unit and unit state

```
CREATE TABLE ReactorFleet.Fleet (
  FleetId INT IDENTITY(1,1) NOT NULL,
  OperatorLegalEntityId INT NOT NULL,
  TimeZoneId INT NOT NULL,
  Code NVARCHAR(50) NOT NULL,
  Name NVARCHAR(250) NOT NULL,
  Description NVARCHAR(1000) NULL,
  CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_Fleet_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  CreatedBy NVARCHAR(100) NOT NULL,
  ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_Fleet_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedBy NVARCHAR(100) NOT NULL,
  IsDeleted BIT NOT NULL CONSTRAINT DF_ReactorFleet_Fleet_IsDeleted DEFAULT 0,
  RowVersion ROWVERSION NOT NULL,

  CONSTRAINT PK_ReactorFleet_Fleet PRIMARY KEY (FleetId),
  CONSTRAINT UQ_ReactorFleet_Fleet_Code UNIQUE (Code),
  CONSTRAINT FK_ReactorFleet_Fleet_OperatorLegalEntity
    FOREIGN KEY (OperatorLegalEntityId) REFERENCES Organization.LegalEntity(LegalEntityId),
  CONSTRAINT FK_ReactorFleet_Fleet_TimeZone
    FOREIGN KEY (TimeZoneId) REFERENCES CorePlatform.TimeZone(TimeZoneId)
);
GO

CREATE TABLE ReactorFleet.FleetPlant (
  FleetPlantId INT IDENTITY(1,1) NOT NULL,
  FleetId INT NOT NULL,
  PlantId INT NOT NULL,
  DisplayOrder INT NOT NULL CONSTRAINT DF_ReactorFleet_FleetPlant_DisplayOrder DEFAULT 0,
  IsActive BIT NOT NULL CONSTRAINT DF_ReactorFleet_FleetPlant_IsActive DEFAULT 1,
  CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_FleetPlant_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  CreatedBy NVARCHAR(100) NOT NULL,
  RowVersion ROWVERSION NOT NULL,

  CONSTRAINT PK_ReactorFleet_FleetPlant PRIMARY KEY (FleetPlantId),
  CONSTRAINT UQ_ReactorFleet_FleetPlant_Fleet_Plant UNIQUE (FleetId, PlantId),
  CONSTRAINT FK_ReactorFleet_FleetPlant_Fleet
    FOREIGN KEY (FleetId) REFERENCES ReactorFleet.Fleet(FleetId),
  CONSTRAINT FK_ReactorFleet_FleetPlant_Plant
    FOREIGN KEY (PlantId) REFERENCES Organization.Plant(PlantId)
);
GO

CREATE TABLE ReactorFleet.Unit (
  UnitId INT IDENTITY(1,1) NOT NULL,
  PlantId INT NOT NULL,
  UnitTypeId INT NOT NULL,
  UnitStatusId INT NOT NULL,
  ReactorTypeId INT NOT NULL,
  Code NVARCHAR(50) NOT NULL,
  Name NVARCHAR(250) NOT NULL,
  UnitNumber INT NOT NULL,
  ThermalMwt DECIMAL(12,3) NOT NULL,
  GrossMWe DECIMAL(12,3) NOT NULL,
  NetMWe DECIMAL(12,3) NOT NULL,
  DesignLifeYears INT NULL,
  CommissionedOn DATE NULL,
  RetiredOn DATE NULL,
  CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_Unit_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  CreatedBy NVARCHAR(100) NOT NULL,
  ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_Unit_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedBy NVARCHAR(100) NOT NULL,
  IsDeleted BIT NOT NULL CONSTRAINT DF_ReactorFleet_Unit_IsDeleted DEFAULT 0,
  RowVersion ROWVERSION NOT NULL,

  CONSTRAINT PK_ReactorFleet_Unit PRIMARY KEY (UnitId),
  CONSTRAINT UQ_ReactorFleet_Unit_Code UNIQUE (Code),
  CONSTRAINT UQ_ReactorFleet_Unit_Plant_UnitNumber UNIQUE (PlantId, UnitNumber),
  CONSTRAINT FK_ReactorFleet_Unit_Plant
    FOREIGN KEY (PlantId) REFERENCES Organization.Plant(PlantId),
  CONSTRAINT FK_ReactorFleet_Unit_UnitType
    FOREIGN KEY (UnitTypeId) REFERENCES ReactorFleet.UnitType(UnitTypeId),
  CONSTRAINT FK_ReactorFleet_Unit_UnitStatus
    FOREIGN KEY (UnitStatusId) REFERENCES ReactorFleet.UnitStatus(UnitStatusId),
  CONSTRAINT FK_ReactorFleet_Unit_ReactorType
    FOREIGN KEY (ReactorTypeId) REFERENCES ReactorFleet.ReactorType(ReactorTypeId),
  CONSTRAINT CK_ReactorFleet_Unit_Ratings CHECK (ThermalMwt > 0 AND GrossMWe > 0 AND NetMWe > 0)
);
GO

CREATE TABLE ReactorFleet.UnitDesignParameter (
  UnitDesignParameterId INT IDENTITY(1,1) NOT NULL,
  UnitId INT NOT NULL,
  EngineeringUnitId INT NULL,
  Code NVARCHAR(80) NOT NULL,
  Name NVARCHAR(200) NOT NULL,
  NumericValue DECIMAL(18,6) NULL,
  TextValue NVARCHAR(500) NULL,
  SourceReference NVARCHAR(300) NULL,
  CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_UnitDesignParameter_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
```

```

        CreatedBy          NVARCHAR(100) NOT NULL,
        ModifiedAtUtc      DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_UnitDesignParameter_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
        ModifiedBy          NVARCHAR(100) NOT NULL,
        RowVersion         ROWVERSION NOT NULL,

        CONSTRAINT PK_ReactorFleet_UnitDesignParameter PRIMARY KEY (UnitDesignParameterId),
        CONSTRAINT UQ_ReactorFleet_UnitDesignParameter_Unit_Code UNIQUE (UnitId, Code),
        CONSTRAINT FK_ReactorFleet_UnitDesignParameter_Unit
            FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
        CONSTRAINT FK_ReactorFleet_UnitDesignParameter_EngineeringUnit
            FOREIGN KEY (EngineeringUnitId) REFERENCES CorePlatform.EngineeringUnit(EngineeringUnitId)
    );
GO

CREATE TABLE ReactorFleet.UnitOperationalState (
    UnitOperationalStateId BIGINT IDENTITY(1,1) NOT NULL,
    UnitId                  INT NOT NULL,
    OperationalModeId       INT NOT NULL,
    RecordedAtUtc           DATETIME2(7) NOT NULL,
    RecordedByUserId        INT NULL,
    StateSummary            NVARCHAR(300) NULL,
    Reason                  NVARCHAR(500) NULL,

    CONSTRAINT PK_ReactorFleet_UnitOperationalState PRIMARY KEY (UnitOperationalStateId),
    CONSTRAINT FK_ReactorFleet_UnitOperationalState_Unit
        FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_ReactorFleet_UnitOperationalState_OperationalMode
        FOREIGN KEY (OperationalModeId) REFERENCES ReactorFleet.OperationalMode(OperationalModeId),
    CONSTRAINT FK_ReactorFleet_UnitOperationalState_RecordedByUser
        FOREIGN KEY (RecordedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId)
    );
GO

```

C.4.4.4 Plant systems, equipment registry and dependencies

```

CREATE TABLE ReactorFleet.PlantSystem (
    PlantSystemId          INT IDENTITY(1,1) NOT NULL,
    UnitId                  INT NOT NULL,
    SystemTypeId            INT NOT NULL,
    ParentPlantSystemId     INT NULL,
    Code                    NVARCHAR(60) NOT NULL,
    Name                    NVARCHAR(250) NOT NULL,
    SafetyRelated           BIT NOT NULL CONSTRAINT DF_ReactorFleet_PlantSystem_SafetyRelated DEFAULT 0,
    Description              NVARCHAR(1000) NULL,
    CreatedAtUtc            DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_PlantSystem_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy                NVARCHAR(100) NOT NULL,
    ModifiedAtUtc           DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_PlantSystem_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy              NVARCHAR(100) NOT NULL,
    IsDeleted               BIT NOT NULL CONSTRAINT DF_ReactorFleet_PlantSystem_IsDeleted DEFAULT 0,
    RowVersion              ROWVERSION NOT NULL,

    CONSTRAINT PK_ReactorFleet_PlantSystem PRIMARY KEY (PlantSystemId),
    CONSTRAINT UQ_ReactorFleet_PlantSystem_Unit_Code UNIQUE (UnitId, Code),
    CONSTRAINT FK_ReactorFleet_PlantSystem_Unit
        FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_ReactorFleet_PlantSystem_SystemType
        FOREIGN KEY (SystemTypeId) REFERENCES ReactorFleet.SystemType(SystemTypeId),
    CONSTRAINT FK_ReactorFleet_PlantSystem_Parent
        FOREIGN KEY (ParentPlantSystemId) REFERENCES ReactorFleet.PlantSystem(PlantSystemId)
    );
GO

CREATE TABLE ReactorFleet.SystemDependency (
    SystemDependencyId      INT IDENTITY(1,1) NOT NULL,
    SourcePlantSystemId     INT NOT NULL,
    TargetPlantSystemId     INT NOT NULL,
    DependencyCode           NVARCHAR(80) NOT NULL,
    InfluenceWeight          DECIMAL(9,6) NOT NULL CONSTRAINT DF_ReactorFleet_SystemDependency_InfluenceWeight DEFAULT 1,
    NominalDelaySeconds     DECIMAL(12,3) NULL,
    IsNegativeFeedback       BIT NOT NULL CONSTRAINT DF_ReactorFleet_SystemDependency_IsNegativeFeedback DEFAULT 0,
    Description              NVARCHAR(1000) NULL,
    CreatedAtUtc            DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_SystemDependency_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy                NVARCHAR(100) NOT NULL,
    ModifiedAtUtc           DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_SystemDependency_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
    ModifiedBy              NVARCHAR(100) NOT NULL,
    RowVersion              ROWVERSION NOT NULL,

    CONSTRAINT PK_ReactorFleet_SystemDependency PRIMARY KEY (SystemDependencyId),
    CONSTRAINT UQ_ReactorFleet_SystemDependency_Source_Target_Code UNIQUE (SourcePlantSystemId, TargetPlantSystemId,
DependencyCode),
    CONSTRAINT FK_ReactorFleet_SystemDependency_SourceSystem
        FOREIGN KEY (SourcePlantSystemId) REFERENCES ReactorFleet.PlantSystem(PlantSystemId),
    CONSTRAINT FK_ReactorFleet_SystemDependency_TargetSystem
        FOREIGN KEY (TargetPlantSystemId) REFERENCES ReactorFleet.PlantSystem(PlantSystemId),
    CONSTRAINT CK_ReactorFleet_SystemDependency_NoSelf CHECK (SourcePlantSystemId <> TargetPlantSystemId)
    );

```

GO

```
CREATE TABLE ReactorFleet.EquipmentLocation (
    EquipmentLocationId INT IDENTITY(1,1) NOT NULL,
    UnitId INT NOT NULL,
    BuildingId INT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(250) NOT NULL,
    FloorLevel NVARCHAR(50) NULL,
    RoomCode NVARCHAR(80) NULL,
    ZoneCode NVARCHAR(80) NULL,
    Description NVARCHAR(1000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_EquipmentLocation_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_EquipmentLocation_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_ReactorFleet_EquipmentLocation_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReactorFleet_EquipmentLocation PRIMARY KEY (EquipmentLocationId),
    CONSTRAINT UQ_ReactorFleet_EquipmentLocation_Unit_Code UNIQUE (UnitId, Code),
    CONSTRAINT FK_ReactorFleet_EquipmentLocation_Unit
        FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_ReactorFleet_EquipmentLocation_Building
        FOREIGN KEY (BuildingId) REFERENCES Organization.Building(BuildingId)
);
GO
```

```
CREATE TABLE ReactorFleet.Equipment (
    EquipmentId INT IDENTITY(1,1) NOT NULL,
    UnitId INT NOT NULL,
    PlantSystemId INT NULL,
    EquipmentLocationId INT NULL,
    EquipmentTypeId INT NOT NULL,
    EquipmentStatusId INT NOT NULL,
    EquipmentCriticalityId INT NOT NULL,
    ParentEquipmentId INT NULL,
    Tag NVARCHAR(80) NOT NULL,
    Name NVARCHAR(250) NOT NULL,
    Manufacturer NVARCHAR(200) NULL,
    Model NVARCHAR(200) NULL,
    SerialNumber NVARCHAR(120) NULL,
    InstalledOn DATE NULL,
    CommissionedOn DATE NULL,
    RetiredOn DATE NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_Equipment_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_Equipment_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_ReactorFleet_Equipment_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReactorFleet_Equipment PRIMARY KEY (EquipmentId),
    CONSTRAINT UQ_ReactorFleet_Equipment_Unit_Tag UNIQUE (UnitId, Tag),
    CONSTRAINT FK_ReactorFleet_Equipment_Unit
        FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_ReactorFleet_Equipment_PlantSystem
        FOREIGN KEY (PlantSystemId) REFERENCES ReactorFleet.PlantSystem(PlantSystemId),
    CONSTRAINT FK_ReactorFleet_Equipment_EquipmentLocation
        FOREIGN KEY (EquipmentLocationId) REFERENCES ReactorFleet.EquipmentLocation(EquipmentLocationId),
    CONSTRAINT FK_ReactorFleet_Equipment_EquipmentType
        FOREIGN KEY (EquipmentTypeId) REFERENCES ReactorFleet.EquipmentType(EquipmentTypeId),
    CONSTRAINT FK_ReactorFleet_Equipment_EquipmentStatus
        FOREIGN KEY (EquipmentStatusId) REFERENCES ReactorFleet.EquipmentStatus(EquipmentStatusId),
    CONSTRAINT FK_ReactorFleet_Equipment_EquipmentCriticality
        FOREIGN KEY (EquipmentCriticalityId) REFERENCES ReactorFleet.EquipmentCriticality(EquipmentCriticalityId),
    CONSTRAINT FK_ReactorFleet_Equipment_ParentEquipment
        FOREIGN KEY (ParentEquipmentId) REFERENCES ReactorFleet.Equipment(EquipmentId)
);
GO
```

```
CREATE TABLE ReactorFleet.EquipmentDependency (
    EquipmentDependencyId BIGINT IDENTITY(1,1) NOT NULL,
    SourceEquipmentId INT NOT NULL,
    TargetEquipmentId INT NOT NULL,
    DependencyCode NVARCHAR(80) NOT NULL,
    InfluenceWeight DECIMAL(9,6) NOT NULL CONSTRAINT DF_ReactorFleet_EquipmentDependency_InfluenceWeight DEFAULT 1,
    NominalDelaySeconds DECIMAL(12,3) NULL,
    Description NVARCHAR(1000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_EquipmentDependency_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReactorFleet_EquipmentDependency PRIMARY KEY (EquipmentDependencyId),
    CONSTRAINT UQ_ReactorFleet_EquipmentDependency_Source_Target_Code UNIQUE (SourceEquipmentId, TargetEquipmentId,
DependencyCode),
    CONSTRAINT FK_ReactorFleet_EquipmentDependency_SourceEquipment
        FOREIGN KEY (SourceEquipmentId) REFERENCES ReactorFleet.Equipment(EquipmentId),
    CONSTRAINT FK_ReactorFleet_EquipmentDependency_TargetEquipment
        FOREIGN KEY (TargetEquipmentId) REFERENCES ReactorFleet.Equipment(EquipmentId),
    CONSTRAINT CK_ReactorFleet_EquipmentDependency_NoSelf CHECK (SourceEquipmentId <> TargetEquipmentId)
);
```

GO

```
CREATE TABLE ReactorFleet.EquipmentExternalReference (
    EquipmentExternalReferenceId INT IDENTITY(1,1) NOT NULL,
    EquipmentId INT NOT NULL,
    ReferenceType NVARCHAR(50) NOT NULL,
    ReferenceCode NVARCHAR(120) NOT NULL,
    ReferenceTitle NVARCHAR(300) NULL,
    ReferenceUri NVARCHAR(1000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_EquipmentExternalReference_CreatedAtUtc
    DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReactorFleet_EquipmentExternalReference PRIMARY KEY (EquipmentExternalReferenceId),
    CONSTRAINT UQ_ReactorFleet_EquipmentExternalReference_Equipment_Type_Code UNIQUE (EquipmentId, ReferenceType,
    ReferenceCode),
    CONSTRAINT FK_ReactorFleet_EquipmentExternalReference_Equipment
        FOREIGN KEY (EquipmentId) REFERENCES ReactorFleet.Equipment(EquipmentId)
);
GO
```

C.4.4.5 Reactor, core, fuel and rods

```
CREATE TABLE ReactorFleet.Reactor (
    ReactorId INT IDENTITY(1,1) NOT NULL,
    UnitId INT NOT NULL,
    ReactorTypeId INT NOT NULL,
    ReactorVesselEquipmentId INT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(250) NOT NULL,
    DesignPressureBar DECIMAL(10,3) NULL,
    DesignInletTempC DECIMAL(10,3) NULL,
    DesignOutletTempC DECIMAL(10,3) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_Reactor_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_Reactor_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_ReactorFleet_Reactor_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReactorFleet_Reactor PRIMARY KEY (ReactorId),
    CONSTRAINT UQ_ReactorFleet_Reactor_Unit_Code UNIQUE (UnitId, Code),
    CONSTRAINT FK_ReactorFleet_Reactor_Unit
        FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_ReactorFleet_Reactor_ReactorType
        FOREIGN KEY (ReactorTypeId) REFERENCES ReactorFleet.ReactorType(ReactorTypeId),
    CONSTRAINT FK_ReactorFleet_Reactor_ReactorVesselEquipment
        FOREIGN KEY (ReactorVesselEquipmentId) REFERENCES ReactorFleet.Equipment(EquipmentId)
);
GO

CREATE TABLE ReactorFleet.ReactorCore (
    ReactorCoreId INT IDENTITY(1,1) NOT NULL,
    ReactorId INT NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(250) NOT NULL,
    CycleNumber INT NOT NULL CONSTRAINT DF_ReactorFleet_ReactorCore_CycleNumber DEFAULT 1,
    AssemblyCount INT NULL,
    EffectiveFullPowerDays DECIMAL(12,3) NULL,
    BurnupGwdPerTonne DECIMAL(12,4) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_ReactorCore_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_ReactorCore_ModifiedAtUtc DEFAULT
    SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_ReactorFleet_ReactorCore_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReactorFleet_ReactorCore PRIMARY KEY (ReactorCoreId),
    CONSTRAINT UQ_ReactorFleet_ReactorCore_Reactor_Code UNIQUE (ReactorId, Code),
    CONSTRAINT FK_ReactorFleet_ReactorCore_Reactor
        FOREIGN KEY (ReactorId) REFERENCES ReactorFleet.Reactor(ReactorId),
    CONSTRAINT CK_ReactorFleet_ReactorCore_CycleNumber CHECK (CycleNumber > 0)
);
GO

CREATE TABLE ReactorFleet.CoreRegion (
    CoreRegionId INT IDENTITY(1,1) NOT NULL,
    ReactorCoreId INT NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    RingNumber INT NULL,
    Description NVARCHAR(500) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_CoreRegion_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReactorFleet_CoreRegion PRIMARY KEY (CoreRegionId),
```



```

CONSTRAINT UQ_ReactorFleet_CoreRegion_Core_Code UNIQUE (ReactorCoreId, Code),
CONSTRAINT FK_ReactorFleet_CoreRegion_ReactorCore
    FOREIGN KEY (ReactorCoreId) REFERENCES ReactorFleet.ReactorCore(ReactorCoreId)
);
GO

CREATE TABLE ReactorFleet.FuelAssembly (
    FuelAssemblyId INT IDENTITY(1,1) NOT NULL,
    ReactorCoreId INT NOT NULL,
    CoreRegionId INT NULL,
    FuelAssemblyTypeId INT NOT NULL,
    FuelMaterialTypeId INT NOT NULL,
    AssemblySerial NVARCHAR(80) NOT NULL,
    CorePosition NVARCHAR(20) NOT NULL,
    InitialEnrichmentWt DECIMAL(8,5) NULL,
    BurnupGwdPerTonne DECIMAL(12,4) NULL,
    IsInstrumented BIT NOT NULL CONSTRAINT DF_ReactorFleet_FuelAssembly_IsInstrumented DEFAULT 0,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_FuelAssembly_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_FuelAssembly_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReactorFleet_FuelAssembly PRIMARY KEY (FuelAssemblyId),
    CONSTRAINT UQ_ReactorFleet_FuelAssembly_Core_Position UNIQUE (ReactorCoreId, CorePosition),
    CONSTRAINT FK_ReactorFleet_FuelAssembly_ReactorCore
        FOREIGN KEY (ReactorCoreId) REFERENCES ReactorFleet.ReactorCore(ReactorCoreId),
    CONSTRAINT FK_ReactorFleet_FuelAssembly_CoreRegion
        FOREIGN KEY (CoreRegionId) REFERENCES ReactorFleet.CoreRegion(CoreRegionId),
    CONSTRAINT FK_ReactorFleet_FuelAssembly_FuelAssemblyType
        FOREIGN KEY (FuelAssemblyTypeId) REFERENCES ReactorFleet.FuelAssemblyType(FuelAssemblyTypeId),
    CONSTRAINT FK_ReactorFleet_FuelAssembly_FuelMaterialType
        FOREIGN KEY (FuelMaterialTypeId) REFERENCES ReactorFleet.FuelMaterialType(FuelMaterialTypeId)
);
GO

CREATE TABLE ReactorFleet.ControlRodBank (
    ControlRodBankId INT IDENTITY(1,1) NOT NULL,
    ReactorCoreId INT NOT NULL,
    RodBankTypeId INT NOT NULL,
    ControlRodMaterialId INT NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    MinimumWorthPcm DECIMAL(12,4) NULL,
    MaximumWorthPcm DECIMAL(12,4) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_ControlRodBank_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_ControlRodBank_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReactorFleet_ControlRodBank PRIMARY KEY (ControlRodBankId),
    CONSTRAINT UQ_ReactorFleet_ControlRodBank_Core_Code UNIQUE (ReactorCoreId, Code),
    CONSTRAINT FK_ReactorFleet_ControlRodBank_ReactorCore
        FOREIGN KEY (ReactorCoreId) REFERENCES ReactorFleet.ReactorCore(ReactorCoreId),
    CONSTRAINT FK_ReactorFleet_ControlRodBank_RodBankType
        FOREIGN KEY (RodBankTypeId) REFERENCES ReactorFleet.RodBankType(RodBankTypeId),
    CONSTRAINT FK_ReactorFleet_ControlRodBank_ControlRodMaterial
        FOREIGN KEY (ControlRodMaterialId) REFERENCES ReactorFleet.ControlRodMaterial(ControlRodMaterialId)
);
GO

CREATE TABLE ReactorFleet.ControlRod (
    ControlRodId INT IDENTITY(1,1) NOT NULL,
    ControlRodBankId INT NOT NULL,
    EquipmentId INT NULL,
    ControlRodMaterialId INT NOT NULL,
    RodSerial NVARCHAR(80) NOT NULL,
    CorePosition NVARCHAR(20) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_ControlRod_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_ControlRod_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReactorFleet_ControlRod PRIMARY KEY (ControlRodId),
    CONSTRAINT UQ_ReactorFleet_ControlRod_Bank_Serial UNIQUE (ControlRodBankId, RodSerial),
    CONSTRAINT FK_ReactorFleet_ControlRod_ControlRodBank
        FOREIGN KEY (ControlRodBankId) REFERENCES ReactorFleet.ControlRodBank(ControlRodBankId),
    CONSTRAINT FK_ReactorFleet_ControlRod_Equipment
        FOREIGN KEY (EquipmentId) REFERENCES ReactorFleet.Equipment(EquipmentId),
    CONSTRAINT FK_ReactorFleet_ControlRod_ControlRodMaterial
        FOREIGN KEY (ControlRodMaterialId) REFERENCES ReactorFleet.ControlRodMaterial(ControlRodMaterialId)
);
GO

```


C.4.4.6 Rod, chemistry and neutronics snapshots

```
CREATE TABLE ReactorFleet.RodPositionSnapshot (
  RodPositionSnapshotId BIGINT IDENTITY(1,1) NOT NULL,
  ControlRodBankId      INT NOT NULL,
  RecordedAtUtc         DATETIME2(7) NOT NULL,
  PositionPercent       DECIMAL(9,6) NOT NULL,
  ReactivityWorthPcm    DECIMAL(12,4) NULL,
  Source                NVARCHAR(50) NOT NULL CONSTRAINT DF_ReactorFleet_RodPositionSnapshot_Source DEFAULT N'DEMONSTRATOR',

  CONSTRAINT PK_ReactorFleet_RodPositionSnapshot PRIMARY KEY (RodPositionSnapshotId),
  CONSTRAINT FK_ReactorFleet_RodPositionSnapshot_ControlRodBank
    FOREIGN KEY (ControlRodBankId) REFERENCES ReactorFleet.ControlRodBank(ControlRodBankId),
  CONSTRAINT CK_ReactorFleet_RodPositionSnapshot_Position CHECK (PositionPercent BETWEEN 0 AND 100)
);
GO

CREATE TABLE ReactorFleet.BoronChemistrySnapshot (
  BoronChemistrySnapshotId BIGINT IDENTITY(1,1) NOT NULL,
  ReactorCoreId            INT NOT NULL,
  RecordedAtUtc           DATETIME2(7) NOT NULL,
  BoronPpm                DECIMAL(12,4) NULL,
  LithiumPpm              DECIMAL(12,4) NULL,
  PH                       DECIMAL(6,3) NULL,
  TemperatureC            DECIMAL(10,3) NULL,
  Source                  NVARCHAR(50) NOT NULL CONSTRAINT DF_ReactorFleet_BoronChemistrySnapshot_Source DEFAULT
N'DEMONSTRATOR',

  CONSTRAINT PK_ReactorFleet_BoronChemistrySnapshot PRIMARY KEY (BoronChemistrySnapshotId),
  CONSTRAINT FK_ReactorFleet_BoronChemistrySnapshot_ReactorCore
    FOREIGN KEY (ReactorCoreId) REFERENCES ReactorFleet.ReactorCore(ReactorCoreId)
);
GO

CREATE TABLE ReactorFleet.NeutronicsParameterSet (
  NeutronicsParameterSetId INT IDENTITY(1,1) NOT NULL,
  ReactorCoreId            INT NOT NULL,
  Code                     NVARCHAR(80) NOT NULL,
  Name                     NVARCHAR(200) NOT NULL,
  EffectiveBeta            DECIMAL(18,10) NOT NULL,
  PromptNeutronLifetimeSec DECIMAL(18,12) NOT NULL,
  DopplerCoefficientPcmC   DECIMAL(18,8) NULL,
  ModeratorCoefficientPcmC DECIMAL(18,8) NULL,
  XenonWorthPcm           DECIMAL(18,8) NULL,
  IsActive                 BIT NOT NULL CONSTRAINT DF_ReactorFleet_NeutronicsParameterSet_IsActive DEFAULT 1,
  CreatedAtUtc             DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_NeutronicsParameterSet_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
  CreatedBy                NVARCHAR(100) NOT NULL,
  RowVersion               ROWVERSION NOT NULL,

  CONSTRAINT PK_ReactorFleet_NeutronicsParameterSet PRIMARY KEY (NeutronicsParameterSetId),
  CONSTRAINT UQ_ReactorFleet_NeutronicsParameterSet_Core_Code UNIQUE (ReactorCoreId, Code),
  CONSTRAINT FK_ReactorFleet_NeutronicsParameterSet_ReactorCore
    FOREIGN KEY (ReactorCoreId) REFERENCES ReactorFleet.ReactorCore(ReactorCoreId),
  CONSTRAINT CK_ReactorFleet_NeutronicsParameterSet_PositiveBeta CHECK (EffectiveBeta > 0),
  CONSTRAINT CK_ReactorFleet_NeutronicsParameterSet_PositiveLifetime CHECK (PromptNeutronLifetimeSec > 0)
);
GO
```

C.4.4.7 Primary loop and steam-generator side

```
CREATE TABLE ReactorFleet.PrimaryLoop (
  PrimaryLoopId INT IDENTITY(1,1) NOT NULL,
  ReactorId      INT NOT NULL,
  CoolantLoopTypeId INT NOT NULL,
  LoopCode       NVARCHAR(50) NOT NULL,
  Name           NVARCHAR(150) NOT NULL,
  DesignPressureBar DECIMAL(10,3) NULL,
  DesignFlowKgPerSec DECIMAL(12,3) NULL,
  CreatedAtUtc    DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_PrimaryLoop_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  CreatedBy       NVARCHAR(100) NOT NULL,
  ModifiedAtUtc   DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_PrimaryLoop_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedBy      NVARCHAR(100) NOT NULL,
  RowVersion      ROWVERSION NOT NULL,

  CONSTRAINT PK_ReactorFleet_PrimaryLoop PRIMARY KEY (PrimaryLoopId),
  CONSTRAINT UQ_ReactorFleet_PrimaryLoop_Reactor_LoopCode UNIQUE (ReactorId, LoopCode),
  CONSTRAINT FK_ReactorFleet_PrimaryLoop_Reactor
    FOREIGN KEY (ReactorId) REFERENCES ReactorFleet.Reactor(ReactorId),
  CONSTRAINT FK_ReactorFleet_PrimaryLoop_CoolantLoopType
    FOREIGN KEY (CoolantLoopTypeId) REFERENCES ReactorFleet.CoolantLoopType(CoolantLoopTypeId)
);
GO

CREATE TABLE ReactorFleet.ReactorCoolantPump (
  ReactorCoolantPumpId INT IDENTITY(1,1) NOT NULL,
  PrimaryLoopId        INT NOT NULL,
```

```

EquipmentId          INT NOT NULL,
PumpTypeId           INT NOT NULL,
DesignFlowKgPerSec   DECIMAL(12,3) NULL,
DesignHeadM          DECIMAL(12,3) NULL,
MotorPowerMW         DECIMAL(12,3) NULL,
CreatedAtUtc         DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_ReactorCoolantPump_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
CreatedBy            NVARCHAR(100) NOT NULL,
ModifiedAtUtc        DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_ReactorCoolantPump_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
ModifiedBy           NVARCHAR(100) NOT NULL,
RowVersion           ROWVERSION NOT NULL,

CONSTRAINT PK_ReactorFleet_ReactorCoolantPump PRIMARY KEY (ReactorCoolantPumpId),
CONSTRAINT UQ_ReactorFleet_ReactorCoolantPump_Equipment UNIQUE (EquipmentId),
CONSTRAINT FK_ReactorFleet_ReactorCoolantPump_PrimaryLoop
    FOREIGN KEY (PrimaryLoopId) REFERENCES ReactorFleet.PrimaryLoop(PrimaryLoopId),
CONSTRAINT FK_ReactorFleet_ReactorCoolantPump_Equipment
    FOREIGN KEY (EquipmentId) REFERENCES ReactorFleet.Equipment(EquipmentId),
CONSTRAINT FK_ReactorFleet_ReactorCoolantPump_PumpType
    FOREIGN KEY (PumpTypeId) REFERENCES ReactorFleet.PumpType(PumpTypeId)
);
GO

CREATE TABLE ReactorFleet.Pressurizer (
    PressurizerId      INT IDENTITY(1,1) NOT NULL,
    UnitId             INT NOT NULL,
    EquipmentId        INT NOT NULL,
    DesignPressureBar   DECIMAL(10,3) NULL,
    DesignVolumeM3      DECIMAL(12,3) NULL,
    CreatedAtUtc        DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_Pressurizer_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy           NVARCHAR(100) NOT NULL,
    ModifiedAtUtc        DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_Pressurizer_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy          NVARCHAR(100) NOT NULL,
    RowVersion          ROWVERSION NOT NULL,

CONSTRAINT PK_ReactorFleet_Pressurizer PRIMARY KEY (PressurizerId),
CONSTRAINT UQ_ReactorFleet_Pressurizer_Equipment UNIQUE (EquipmentId),
CONSTRAINT FK_ReactorFleet_Pressurizer_Unit
    FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
CONSTRAINT FK_ReactorFleet_Pressurizer_Equipment
    FOREIGN KEY (EquipmentId) REFERENCES ReactorFleet.Equipment(EquipmentId)
);
GO

CREATE TABLE ReactorFleet.SteamGenerator (
    SteamGeneratorId    INT IDENTITY(1,1) NOT NULL,
    PrimaryLoopId       INT NOT NULL,
    EquipmentId         INT NOT NULL,
    SteamGeneratorTypeId INT NOT NULL,
    Code                NVARCHAR(50) NOT NULL,
    TubeCount           INT NULL,
    DesignPrimaryTempC   DECIMAL(10,3) NULL,
    DesignSecondaryTempC DECIMAL(10,3) NULL,
    CreatedAtUtc         DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_SteamGenerator_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy           NVARCHAR(100) NOT NULL,
    ModifiedAtUtc        DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_SteamGenerator_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
    ModifiedBy          NVARCHAR(100) NOT NULL,
    RowVersion          ROWVERSION NOT NULL,

CONSTRAINT PK_ReactorFleet_SteamGenerator PRIMARY KEY (SteamGeneratorId),
CONSTRAINT UQ_ReactorFleet_SteamGenerator_Equipment UNIQUE (EquipmentId),
CONSTRAINT UQ_ReactorFleet_SteamGenerator_Loop_Code UNIQUE (PrimaryLoopId, Code),
CONSTRAINT FK_ReactorFleet_SteamGenerator_PrimaryLoop
    FOREIGN KEY (PrimaryLoopId) REFERENCES ReactorFleet.PrimaryLoop(PrimaryLoopId),
CONSTRAINT FK_ReactorFleet_SteamGenerator_Equipment
    FOREIGN KEY (EquipmentId) REFERENCES ReactorFleet.Equipment(EquipmentId),
CONSTRAINT FK_ReactorFleet_SteamGenerator_SteamGeneratorType
    FOREIGN KEY (SteamGeneratorTypeId) REFERENCES ReactorFleet.SteamGeneratorType(SteamGeneratorTypeId)
);
GO

```

C.4.4.8 Turbine, generator, transformer and grid connection

```

CREATE TABLE ReactorFleet.TurbineTrain (
    TurbineTrainId      INT IDENTITY(1,1) NOT NULL,
    UnitId              INT NOT NULL,
    EquipmentId         INT NOT NULL,
    Code                NVARCHAR(50) NOT NULL,
    RatedMWe            DECIMAL(12,3) NULL,
    CreatedAtUtc        DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_TurbineTrain_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy           NVARCHAR(100) NOT NULL,
    ModifiedAtUtc        DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_TurbineTrain_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy          NVARCHAR(100) NOT NULL,
    RowVersion          ROWVERSION NOT NULL,

CONSTRAINT PK_ReactorFleet_TurbineTrain PRIMARY KEY (TurbineTrainId),

```

```

CONSTRAINT UQ_ReactorFleet_TurbineTrain_Unit_Code UNIQUE (UnitId, Code),
CONSTRAINT UQ_ReactorFleet_TurbineTrain_Equipment UNIQUE (EquipmentId),
CONSTRAINT FK_ReactorFleet_TurbineTrain_Unit
    FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
CONSTRAINT FK_ReactorFleet_TurbineTrain_Equipment
    FOREIGN KEY (EquipmentId) REFERENCES ReactorFleet.Equipment(EquipmentId)
);
GO

CREATE TABLE ReactorFleet.TurbineStage (
    TurbineStageId INT IDENTITY(1,1) NOT NULL,
    TurbineTrainId INT NOT NULL,
    StageCode NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    DesignInletPressureBar DECIMAL(10,3) NULL,
    DesignOutletPressureBar DECIMAL(10,3) NULL,
    DisplayOrder INT NOT NULL CONSTRAINT DF_ReactorFleet_TurbineStage_DisplayOrder DEFAULT 0,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_TurbineStage_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReactorFleet_TurbineStage PRIMARY KEY (TurbineStageId),
    CONSTRAINT UQ_ReactorFleet_TurbineStage_Train_Code UNIQUE (TurbineTrainId, StageCode),
    CONSTRAINT FK_ReactorFleet_TurbineStage_TurbineTrain
        FOREIGN KEY (TurbineTrainId) REFERENCES ReactorFleet.TurbineTrain(TurbineTrainId)
);
GO

CREATE TABLE ReactorFleet.Generator (
    GeneratorId INT IDENTITY(1,1) NOT NULL,
    UnitId INT NOT NULL,
    EquipmentId INT NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    RatedMVA DECIMAL(12,3) NULL,
    RatedVoltageKV DECIMAL(12,3) NULL,
    RatedFrequencyHz DECIMAL(8,4) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_Generator_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_Generator_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReactorFleet_Generator PRIMARY KEY (GeneratorId),
    CONSTRAINT UQ_ReactorFleet_Generator_Unit_Code UNIQUE (UnitId, Code),
    CONSTRAINT UQ_ReactorFleet_Generator_Equipment UNIQUE (EquipmentId),
    CONSTRAINT FK_ReactorFleet_Generator_Unit
        FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_ReactorFleet_Generator_Equipment
        FOREIGN KEY (EquipmentId) REFERENCES ReactorFleet.Equipment(EquipmentId)
);
GO

CREATE TABLE ReactorFleet.Transformer (
    TransformerId INT IDENTITY(1,1) NOT NULL,
    UnitId INT NOT NULL,
    EquipmentId INT NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    RatedMVA DECIMAL(12,3) NULL,
    PrimaryVoltageKV DECIMAL(12,3) NULL,
    SecondaryVoltageKV DECIMAL(12,3) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_Transformer_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_Transformer_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReactorFleet_Transformer PRIMARY KEY (TransformerId),
    CONSTRAINT UQ_ReactorFleet_Transformer_Unit_Code UNIQUE (UnitId, Code),
    CONSTRAINT UQ_ReactorFleet_Transformer_Equipment UNIQUE (EquipmentId),
    CONSTRAINT FK_ReactorFleet_Transformer_Unit
        FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_ReactorFleet_Transformer_Equipment
        FOREIGN KEY (EquipmentId) REFERENCES ReactorFleet.Equipment(EquipmentId)
);
GO

CREATE TABLE ReactorFleet.SwitchyardBay (
    SwitchyardBayId INT IDENTITY(1,1) NOT NULL,
    UnitId INT NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    VoltageKV DECIMAL(12,3) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_SwitchyardBay_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_SwitchyardBay_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReactorFleet_SwitchyardBay PRIMARY KEY (SwitchyardBayId),
    CONSTRAINT UQ_ReactorFleet_SwitchyardBay_Unit_Code UNIQUE (UnitId, Code),
    CONSTRAINT FK_ReactorFleet_SwitchyardBay_Unit
        FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId)
);
GO

```

```

CREATE TABLE ReactorFleet.GridConnection (
    GridConnectionId INT IDENTITY(1,1) NOT NULL,
    UnitId INT NOT NULL,
    GeneratorId INT NULL,
    TransformerId INT NULL,
    SwitchyardBayId INT NULL,
    GridConnectionStatusId INT NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    GridName NVARCHAR(200) NOT NULL,
    NominalVoltageKV DECIMAL(12,3) NULL,
    NominalFrequencyHz DECIMAL(8,4) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_GridConnection_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_GridConnection_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReactorFleet_GridConnection PRIMARY KEY (GridConnectionId),
    CONSTRAINT UQ_ReactorFleet_GridConnection_Unit_Code UNIQUE (UnitId, Code),
    CONSTRAINT FK_ReactorFleet_GridConnection_Unit
        FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_ReactorFleet_GridConnection_Generator
        FOREIGN KEY (GeneratorId) REFERENCES ReactorFleet.Generator(GeneratorId),
    CONSTRAINT FK_ReactorFleet_GridConnection_Transformer
        FOREIGN KEY (TransformerId) REFERENCES ReactorFleet.Transformer(TransformerId),
    CONSTRAINT FK_ReactorFleet_GridConnection_SwitchyardBay
        FOREIGN KEY (SwitchyardBayId) REFERENCES ReactorFleet.SwitchyardBay(SwitchyardBayId),
    CONSTRAINT FK_ReactorFleet_GridConnection_GridConnectionStatus
        FOREIGN KEY (GridConnectionStatusId) REFERENCES ReactorFleet.GridConnectionStatus(GridConnectionStatusId)
);
GO

```

C.4.4.9 Fleet and unit power snapshots

```

CREATE TABLE ReactorFleet.FleetSnapshot (
    FleetSnapshotId BIGINT IDENTITY(1,1) NOT NULL,
    FleetId INT NOT NULL,
    RecordedAtUtc DATETIME2(7) NOT NULL,
    OnlineUnitCount INT NOT NULL,
    TotalGrossMWe DECIMAL(14,3) NOT NULL,
    TotalNetMWe DECIMAL(14,3) NOT NULL,
    GridDemandMWe DECIMAL(14,3) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReactorFleet_FleetSnapshot_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),

    CONSTRAINT PK_ReactorFleet_FleetSnapshot PRIMARY KEY (FleetSnapshotId),
    CONSTRAINT FK_ReactorFleet_FleetSnapshot_Fleet
        FOREIGN KEY (FleetId) REFERENCES ReactorFleet.Fleet(FleetId)
);
GO

CREATE TABLE ReactorFleet.UnitPowerSnapshot (
    UnitPowerSnapshotId BIGINT IDENTITY(1,1) NOT NULL,
    UnitId INT NOT NULL,
    OperationalModeId INT NULL,
    RecordedAtUtc DATETIME2(7) NOT NULL,
    ThermalPowerMWt DECIMAL(14,4) NULL,
    GrossPowerMWe DECIMAL(14,4) NULL,
    NetPowerMWe DECIMAL(14,4) NULL,
    PowerPercent DECIMAL(9,6) NULL,
    GridFrequencyHz DECIMAL(8,4) NULL,
    PowerFactor DECIMAL(8,6) NULL,
    Source NVARCHAR(50) NOT NULL CONSTRAINT DF_ReactorFleet_UnitPowerSnapshot_Source DEFAULT N'DEMONSTRATOR',

    CONSTRAINT PK_ReactorFleet_UnitPowerSnapshot PRIMARY KEY (UnitPowerSnapshotId),
    CONSTRAINT FK_ReactorFleet_UnitPowerSnapshot_Unit
        FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_ReactorFleet_UnitPowerSnapshot_OperationalMode
        FOREIGN KEY (OperationalModeId) REFERENCES ReactorFleet.OperationalMode(OperationalModeId),
    CONSTRAINT CK_ReactorFleet_UnitPowerSnapshot_PowerPercent CHECK (PowerPercent IS NULL OR PowerPercent BETWEEN 0 AND 200)
);
GO

```

C.4.5 Indexes and seed examples

Indexes follow the same rule as earlier sectors: every foreign key receives an index unless the primary key or unique constraint already covers it. Snapshot tables are indexed by parent key and timestamp descending.

```

CREATE INDEX IX_ReactorFleet_Fleet_OperatorLegalEntityId
ON ReactorFleet.Fleet(OperatorLegalEntityId);

CREATE INDEX IX_ReactorFleet_FleetPlant_PlantId
ON ReactorFleet.FleetPlant(PlantId);

CREATE INDEX IX_ReactorFleet_Unit_PlantId
ON ReactorFleet.Unit(PlantId);

CREATE INDEX IX_ReactorFleet_Unit_UnitStatusId
ON ReactorFleet.Unit(UnitStatusId);

CREATE INDEX IX_ReactorFleet_UnitOperationalState_UnitId_RecordedAtUtc
ON ReactorFleet.UnitOperationalState(UnitId, RecordedAtUtc DESC);

CREATE INDEX IX_ReactorFleet_PlantSystem_UnitId
ON ReactorFleet.PlantSystem(UnitId);

CREATE INDEX IX_ReactorFleet_Equipment_UnitId
ON ReactorFleet.Equipment(UnitId);

CREATE INDEX IX_ReactorFleet_Equipment_Tag
ON ReactorFleet.Equipment(Tag);

CREATE INDEX IX_ReactorFleet_Equipment_PlantSystemId
ON ReactorFleet.Equipment(PlantSystemId);

CREATE INDEX IX_ReactorFleet_EquipmentDependency_SourceEquipmentId
ON ReactorFleet.EquipmentDependency(SourceEquipmentId);

CREATE INDEX IX_ReactorFleet_EquipmentDependency_TargetEquipmentId
ON ReactorFleet.EquipmentDependency(TargetEquipmentId);

CREATE INDEX IX_ReactorFleet_Reactor_UnitId
ON ReactorFleet.Reactor(UnitId);

CREATE INDEX IX_ReactorFleet_ReactorCore_ReactorId
ON ReactorFleet.ReactorCore(ReactorId);

CREATE INDEX IX_ReactorFleet_FuelAssembly_ReactorCoreId
ON ReactorFleet.FuelAssembly(ReactorCoreId);

CREATE INDEX IX_ReactorFleet_RodPositionSnapshot_Bank_Time
ON ReactorFleet.RodPositionSnapshot(ControlRodBankId, RecordedAtUtc DESC);

CREATE INDEX IX_ReactorFleet_UnitPowerSnapshot_Unit_Time
ON ReactorFleet.UnitPowerSnapshot(UnitId, RecordedAtUtc DESC);

CREATE INDEX IX_ReactorFleet_FleetSnapshot_Fleet_Time
ON ReactorFleet.FleetSnapshot(FleetId, RecordedAtUtc DESC);
GO

```

C.4.5.1 Minimal seed examples

```

INSERT INTO ReactorFleet.UnitType (Code, Name, Description)
VALUES
(N'PWR_900', N'Large PWR 900 MWe', N'Illustrative large pressurized-water unit.'),
(N'PWR_1300', N'Large PWR 1300 MWe', N'Illustrative large pressurized-water unit.'),
(N'SMR_170', N'SMR 170 MWe Module', N'Illustrative compact modular unit.'),
(N'SIMULATOR', N'Training Simulator Unit', N'Non-physical training or demonstration unit.');
```

```

INSERT INTO ReactorFleet.UnitStatus (Code, Name, Description)
VALUES
(N'ONLINE', N'Online', N'Unit is synchronized or operating in normal demonstration mode.'),
(N'OFFLINE', N'Offline', N'Unit is not producing power.'),
(N'OUTAGE', N'Outage', N'Unit is in planned outage state.'),
(N'TRIP', N'Trip', N'Unit is in reactor-trip state.'),
(N'DECOMMISSIONING', N'Decommissioning', N'Unit is retired or in decommissioning workflow.');
```

```

INSERT INTO ReactorFleet.OperationalMode (Code, Name, Description)
VALUES
(N'POWER', N'Power operation', N'Normal power operation.'),
(N'LOAD_FOLLOW', N'Load-following', N'Power setpoint is following grid demand.'),
(N'STARTUP', N'Startup', N'Startup or approach to power.'),
(N'SHUTDOWN', N'Shutdown', N'Controlled shutdown or cooldown.'),
(N'TRAINING', N'Training', N'Guided drill or simulated scenario.');
```

```

INSERT INTO ReactorFleet.EquipmentStatus (Code, Name, Description)
VALUES
(N'AVAILABLE', N'Available', N'Available for service.'),
(N'RUNNING', N'Running', N'Currently running.'),
(N'STANDBY', N'Standby', N'Available but not currently running.'),
(N'DEGRADED', N'Degraded', N'Available with degraded condition.'),
(N'UNAVAILABLE', N'Unavailable', N'Not available for service.');
```

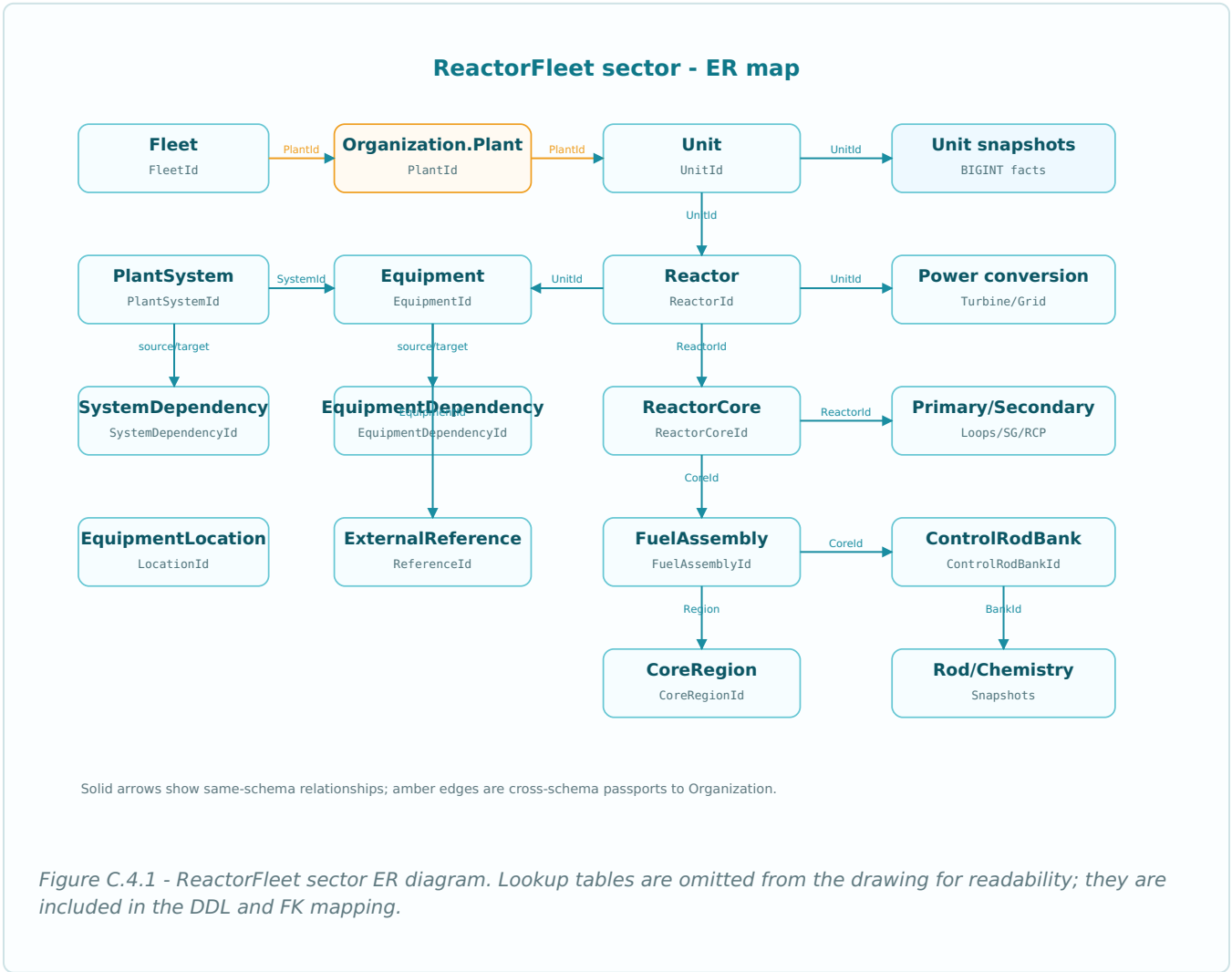
```

GO

```

C.4.6 ER diagram

The ER diagram shows ReactorFleet as a physical tree with several clusters: fleet/unit, systems/equipment, reactor/core, primary/secondary loop, and power conversion.



C.4.7 Foreign-key mapping

C.4.7.1 Internal ReactorFleet foreign keys

FROM TABLE	FK COLUMN	TO TABLE	PK COLUMN	RELATIONSHIP
FleetPlant	FleetId	Fleet	FleetId	many plants per fleet
Unit	UnitTypeId	UnitType	UnitTypeId	many units per unit type
Unit	UnitStatusId	UnitStatus	UnitStatusId	many units per status
Unit	ReactorTypeId	ReactorType	ReactorTypeId	many units per reactor type
UnitDesignParameter	UnitId	Unit	UnitId	many design parameters per unit
UnitOperationalState	UnitId	Unit	UnitId	many state records per unit
UnitOperationalState	OperationalModeId	OperationalMode	OperationalModeId	many state records per mode
PlantSystem	UnitId	Unit	UnitId	many systems per unit
PlantSystem	ParentPlantSystemId	PlantSystem	PlantSystemId	system hierarchy
SystemDependency	SourcePlantSystemId	PlantSystem	PlantSystemId	directed source system
SystemDependency	TargetPlantSystemId	PlantSystem	PlantSystemId	directed target system
EquipmentLocation	UnitId	Unit	UnitId	many locations per unit
Equipment	UnitId	Unit	UnitId	many equipment rows per unit
Equipment	PlantSystemId	PlantSystem	PlantSystemId	equipment belongs to a system
Equipment	EquipmentLocationId	EquipmentLocation	EquipmentLocationId	equipment has optional location
Equipment	ParentEquipmentId	Equipment	EquipmentId	equipment assembly hierarchy
EquipmentDependency	SourceEquipmentId	Equipment	EquipmentId	directed source equipment
EquipmentDependency	TargetEquipmentId	Equipment	EquipmentId	directed target equipment
Reactor	UnitId	Unit	UnitId	one or more reactor records per unit

FROM TABLE	FK COLUMN	TO TABLE	PK COLUMN	RELATIONSHIP
ReactorCore	ReactorId	Reactor	ReactorId	many core loadings per reactor
CoreRegion	ReactorCoreId	ReactorCore	ReactorCoreId	many regions per core
FuelAssembly	ReactorCoreId	ReactorCore	ReactorCoreId	many assemblies per core
FuelAssembly	CoreRegionId	CoreRegion	CoreRegionId	optional region assignment
ControlRodBank	ReactorCoreId	ReactorCore	ReactorCoreId	many banks per core
ControlRod	ControlRodBankId	ControlRodBank	ControlRodBankId	many rods per bank
RodPositionSnapshot	ControlRodBankId	ControlRodBank	ControlRodBankId	many snapshots per bank
PrimaryLoop	ReactorId	Reactor	ReactorId	many loops per reactor
ReactorCoolantPump	PrimaryLoopId	PrimaryLoop	PrimaryLoopId	many pumps per loop
SteamGenerator	PrimaryLoopId	PrimaryLoop	PrimaryLoopId	many steam generators per loop
TurbineTrain	UnitId	Unit	UnitId	turbine train belongs to unit
Generator	UnitId	Unit	UnitId	generator belongs to unit
GridConnection	UnitId	Unit	UnitId	grid connection belongs to unit

C.4.7.2 Passports consumed from previous sectors

FROM TABLE	FK COLUMN	TO TABLE	PK COLUMN	RELATIONSHIP
Fleet	OperatorLegalEntityId	Organization.LegalEntity	LegalEntityId	fleet operator or owner
Fleet	TimeZoneId	CorePlatform.TimeZone	TimeZoneId	fleet display/scheduling zone
FleetPlant	PlantId	Organization.Plant	PlantId	plant contained in fleet
Unit	PlantId	Organization.Plant	PlantId	unit belongs to organizational plant
UnitDesignParameter	EngineeringUnitId	CorePlatform.EngineeringUnit	EngineeringUnitId	typed design-value unit
UnitOperationalState	RecordedByUserId	Security.ApplicationUser	ApplicationUserId	operator or system account that recorded state
EquipmentLocation	BuildingId	Organization.Building	BuildingId	physical building location

C.4.7.3 Incoming passports from later sectors

The following relationships are declared in later sectors. They are listed here because ReactorFleet is one of the main providers of keys for the whole platform.

Instrumentation.Signal

FK UnitId, EquipmentId
TO ReactorFleet.Unit, ReactorFleet.Equipment
signals attach to the physical unit and optionally to exact equipment.

DigitalTwin.TwinModel

FK UnitId
TO ReactorFleet.Unit
a twin model mirrors one physical or simulated unit.

AlarmManagement.AlarmEvent

FK UnitId, EquipmentId
TO ReactorFleet.Unit, ReactorFleet.Equipment
alarms point back to the machine that raised them.

Maintenance.Asset

FK EquipmentId
TO ReactorFleet.Equipment
maintenance assets are passports over physical equipment.

RootCause.CausalNode

FK UnitId, EquipmentId
TO ReactorFleet.Unit, ReactorFleet.Equipment
causal graph nodes ground themselves in the component registry.

ReinforcementLearning.TrainingRun

FK UnitId
TO ReactorFleet.Unit
the policy table and episodes are scoped to a unit.

RadiationMonitoring.Zone

FK UnitId
TO ReactorFleet.Unit
radiation zones and dose records are scoped to plant areas and units.

Reporting.ReportSnapshot

FK UnitId
TO ReactorFleet.Unit
reporting snapshots aggregate unit, alarm and twin records.

ReactorFleet passport map

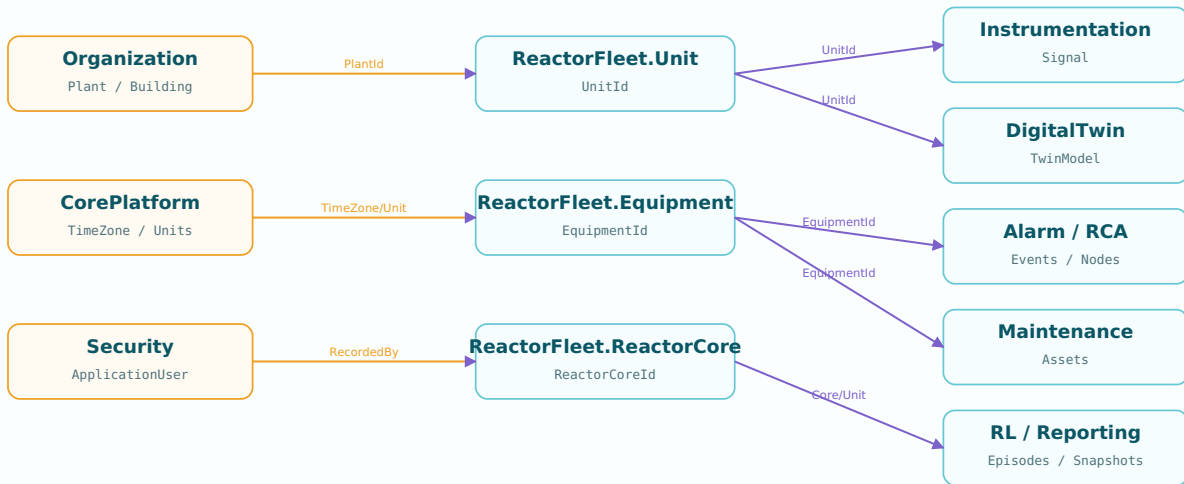


Figure C.4.2 - ReactorFleet passport map. ReactorFleet consumes Organization, CorePlatform and Security, then provides UnitId, EquipmentId and ReactorCoreId to almost every operational sector.

C.4.8 Useful verification queries

These queries validate that the physical tree can be joined from plant to unit, from unit to equipment, from unit to grid connection and from equipment to dependencies.

```

-- 1. One unit, its plant, reactor, grid connection and current status.
SELECT
  u.Code AS UnitCode,
  u.Name AS UnitName,
  p.Name AS PlantName,
  rt.Code AS ReactorType,
  us.Code AS UnitStatus,
  gc.GridName,
  gcs.Code AS GridState
FROM ReactorFleet.Unit AS u
JOIN Organization.Plant AS p ON p.PlantId = u.PlantId
JOIN ReactorFleet.ReactorType AS rt ON rt.ReactorTypeId = u.ReactorTypeId
JOIN ReactorFleet.UnitStatus AS us ON us.UnitStatusId = u.UnitStatusId
LEFT JOIN ReactorFleet.GridConnection AS gc ON gc.UnitId = u.UnitId
LEFT JOIN ReactorFleet.GridConnectionStatus AS gcs ON gcs.GridConnectionStatusId = gc.GridConnectionStatusId
WHERE u.Code = 'NX1-U1';

-- 2. Component registry for a unit, grouped by plant system.
SELECT
  u.Code AS UnitCode,
  ps.Code AS SystemCode,
  e.Tag,
  e.Name,
  et.Code AS EquipmentType,
  es.Code AS EquipmentStatus,
  ec.Code AS Criticality
FROM ReactorFleet.Equipment AS e
JOIN ReactorFleet.Unit AS u ON u.UnitId = e.UnitId
LEFT JOIN ReactorFleet.PlantSystem AS ps ON ps.PlantSystemId = e.PlantSystemId
JOIN ReactorFleet.EquipmentType AS et ON et.EquipmentTypeId = e.EquipmentTypeId
JOIN ReactorFleet.EquipmentStatus AS es ON es.EquipmentStatusId = e.EquipmentStatusId
JOIN ReactorFleet.EquipmentCriticality AS ec ON ec.EquipmentCriticalityId = e.EquipmentCriticalityId
WHERE u.Code = 'NX1-U1'
ORDER BY ps.Code, e.Tag;

-- 3. Latest unit power snapshot.
SELECT TOP (1)
  u.Code,
  s.RecordedAtUtc,
  s.PowerPercent,
  s.GrossPowerMWe,
  s.GridFrequencyHz,
  om.Code AS ModeCode

```

```

FROM ReactorFleet.UnitPowerSnapshot AS s
JOIN ReactorFleet.Unit AS u ON u.UnitId = s.UnitId
LEFT JOIN ReactorFleet.OperationalMode AS om ON om.OperationalModeId = s.OperationalModeId
WHERE u.Code = N'NX1-U1'
ORDER BY s.RecordedAtUtc DESC;

-- 4. Directed equipment dependencies for RCA preparation.
SELECT
    src.Tag AS SourceEquipment,
    dep.DependencyCode,
    dep.InfluenceWeight,
    dep.NominalDelaySeconds,
    tgt.Tag AS TargetEquipment
FROM ReactorFleet.EquipmentDependency AS dep
JOIN ReactorFleet.Equipment AS src ON src.EquipmentId = dep.SourceEquipmentId
JOIN ReactorFleet.Equipment AS tgt ON tgt.EquipmentId = dep.TargetEquipmentId
WHERE src.UnitId = (SELECT UnitId FROM ReactorFleet.Unit WHERE Code = N'NX1-U1')
ORDER BY src.Tag, tgt.Tag;
GO

```

C.4.9 Boundary and next sector

Honest boundary. This schema is an educational and demonstrator-grade physical plant model. It is not a licensed plant asset-management database, not a safety-class design database, and not a representation of any real nuclear facility. The values and names are deliberately illustrative. The important design claim is database structure: a unit has one identity, equipment has one registry, and later sectors refer to those rows rather than inventing their own copies.

The next sector is [Instrumentation](#). It will attach tags, sensors, signals, historian readings and signal quality to [ReactorFleet.Unit](#) and [ReactorFleet.Equipment](#). That is where the physical machine begins to speak in measurements.

Appendix C.5

Instrumentation Schema

The Instrumentation sector owns tags, sensors, channels, signal identity, raw measurements, aggregates, quality intervals, calibration records and acquisition mapping.

ReactorFleet gives the platform a physical unit and component registry. Instrumentation gives that machine a voice. A signal is not just a string on a screen: it is a row with a unit of measure, source, sampling mode, quality rules, acquisition path and historian contract.

A tag is the passport between reality and the rest of the platform. DigitalTwin binds it, AlarmManagement thresholds it, RootCause uses it as a witness, and Reporting reads it without translating its name.

C.5.1

Sector purpose

C.5.2

Full table list

C.5.3

Lookup tables

C.5.4

SQL DDL

C.5.5

Indexes and seed examples

C.5.6

ER diagram

C.5.7

Foreign-key mapping

C.5.8

Boundary and next sector

C.5.1 Sector purpose

Instrumentation consumes `ReactorFleet.Unit`, `ReactorFleet.Equipment`, `ReactorFleet.PlantSystem`, `CorePlatform.EngineeringUnit` and `Security.ApplicationUser`. It provides `SignalId` to later sectors.

- It owns the canonical signal registry through `Instrumentation.Signal`.
- It maps canonical tags to acquisition points and physical sensors.
- It stores high-volume historian measurements and lower-volume aggregates.
- It records signal quality, gaps and calibration evidence so a value can be trusted or rejected.

Design choice. The historian value table is intentionally narrow: `SignalId`, timestamp, value, quality and source. Rich metadata lives beside it in normalized tables so the hot path remains fast.

C.5.2 Full table list

The Instrumentation sector contains forty tables: fourteen lookup tables and twenty-six substantive tables. It is one of the largest sectors because it stands between the physical registry and every data-consuming engine.

LOOKUP

Instrumentation.SignalType

PK `SignalTypeId`
FK None

Classifies measured, calculated, commanded, state, alarm-derived, and simulation mirror signals.

LOOKUP

Instrumentation.SignalCategory

PK `SignalCategoryId`
FK None

Domain category such as power, pressure, temperature, flow, radiation, vibration, chemistry or electrical.

LOOKUP

Instrumentation.SignalRole

PK `SignalRoleId`
FK None

Role of the signal in the platform: telemetry, control indication, diagnostic, twin input, RCA witness or training signal.

LOOKUP

Instrumentation.SensorType

PK `SensorTypeId`
FK None

Physical sensor family: RTD, pressure transmitter, flow transmitter, neutron detector, vibration probe, radiation monitor.

LOOKUP

Instrumentation.SensorStatus

PK `SensorStatusId`
FK None

Sensor status: available, maintenance, suspect, failed, retired.

LOOKUP

Instrumentation.ChannelStatus

PK `ChannelStatusId`
FK None

Acquisition-channel status: online, stale, disabled, faulted, simulated.

LOOKUP

Instrumentation.SignalQuality

PK `SignalQualityId`
FK None

Quality state attached to measurements: good, uncertain, bad, stale, substituted, simulated.

LOOKUP

Instrumentation.SamplingMode

PK `SamplingModeId`
FK None

Sampling mode: periodic, event-driven, manual, derived, simulated.

LOOKUP**Instrumentation.MeasurementSource**

PK MeasurementSourceId

FK None

Origin of a value: historian, simulator, operator entry, import, twin backfill, test harness.

LOOKUP**Instrumentation.CalibrationStatus**

PK CalibrationStatusId

FK None

Calibration lifecycle: scheduled, due, overdue, passed, failed, waived.

LOOKUP**Instrumentation.HistorianRetentionClass**

PK HistorianRetentionClassId

FK None

Retention policy category for raw and aggregated measurements.

LOOKUP**Instrumentation.ThresholdType**

PK ThresholdTypeId

FK None

Threshold semantics: high, high-high, low, low-low, rate, deviation, deadband.

LOOKUP**Instrumentation.BackfillJobStatus**

PK BackfillJobStatusId

FK None

Backfill job status: queued, running, completed, failed, cancelled.

LOOKUP**Instrumentation.LineageRole**

PK LineageRoleId

FK None

Role of a source signal in a derived signal: primary, witness, fallback, correction, normalization.

ACQUISITION**Instrumentation.DataAcquisitionNode**

PK DataAcquisitionNodeId

FK UnitId, ChannelStatusId

Logical acquisition node, gateway, simulator feed or historian bridge for one unit.

ACQUISITION**Instrumentation.AcquisitionConnection**

PK AcquisitionConnectionId

FK DataAcquisitionNodeId, ChannelStatusId

Named protocol connection or local feed into an acquisition node.

ACQUISITION**Instrumentation.AcquisitionPoint**

PK AcquisitionPointId

FK AcquisitionConnectionId

Protocol address, register, topic, JSON path or synthetic point before it becomes a signal.

INSTRUMENTS**Instrumentation.Instrument**

PK InstrumentId

FK UnitId, EquipmentId, SensorStatusId

Installed instrument package or cabinet entry that may host one or more sensors.

INSTRUMENTS

Instrumentation.Sensor

PK **SensorId**
 FK **InstrumentId**, **SensorTypeId**,
EngineeringUnitId

Physical or simulated sensor with range, accuracy and status.

INSTRUMENTS

Instrumentation.SensorChannel

PK **SensorChannelId**
 FK **SensorId**, **ChannelStatusId**

A channel from a sensor; redundant channels become separate rows.

SIGNALS

Instrumentation.Signal

PK **SignalId**
 FK **UnitId**, **EquipmentId**, **EngineeringUnitId**

The canonical tag. Digital twin, alarms, root cause and historian all reference this table.

SIGNALS

Instrumentation.SignalAlias

PK **SignalAliasId**
 FK **SignalId**

Alternative tag, legacy name, historian name, screen key or vendor tag.

SIGNALS

Instrumentation.SignalGroup

PK **SignalGroupId**
 FK **UnitId**

Named signal set for screens, diagnostics, export packs and data-quality views.

SIGNALS

Instrumentation.SignalGroupMember

PK **SignalGroupMemberId**
 FK **SignalGroupId**, **SignalId**

Membership of signals in a named group.

SIGNALS

Instrumentation.SignalMapping

PK **SignalMappingId**
 FK **SignalId**, **AcquisitionPointId**

Maps a canonical signal to a raw acquisition point.

SIGNALS

Instrumentation.SignalDependency

PK **SignalDependencyId**
 FK **SourceSignalId**, **TargetSignalId**

Signal-level influence or derivation relationship with optional lag and weight.

SIGNALS

Instrumentation.SignalLineage

PK **SignalLineageId**
 FK **DerivedSignalId**, **SourceSignalId**

Explicit lineage for calculated and derived signals.

LIMITS

Instrumentation.SignalLimit

PK **SignalLimitId**
 FK **SignalId**, **ThresholdTypeId**

Engineering, display, alarm-preparation or model-boundary limit for a signal.

LIMITS

Instrumentation.SignalDeadband

PK SignalDeadbandId

FK SignalId

Deadband or noise band for display, alarm rationalisation and digital-twin divergence.

CALIBRATION

Instrumentation.CalibrationPlan

PK CalibrationPlanId

FK SignalId, SensorId, CalibrationStatusId

Calibration obligation and interval for a signal/sensor pair.

CALIBRATION

Instrumentation.CalibrationRecord

PK CalibrationRecordId

FK CalibrationPlanId, PerformedByUserId

Completed calibration result, as-found/as-left value and pass/fail state.

HISTORIAN

Instrumentation.Measurement

PK MeasurementId

FK SignalId, SignalQualityId,
MeasurementSourceId

High-volume raw time-series fact table.

HISTORIAN

Instrumentation.MeasurementAggregate

PK MeasurementAggregateId

FK SignalId, SignalQualityId

Aggregated windows for fast charts and reporting.

HISTORIAN

Instrumentation.MeasurementAnnotation

PK MeasurementAnnotationId

FK MeasurementId, CreatedByUserId

Human or system note attached to one measurement row.

HISTORIAN

Instrumentation.SignalQualityEvent

PK SignalQualityEventId

FK SignalId, SignalQualityId

Interval record explaining when a signal became stale, bad, substituted or simulated.

HISTORIAN

Instrumentation.DataGap

PK DataGapId

FK SignalId

Detected gap in historian coverage for one signal.

HISTORIAN

Instrumentation.HistorianRetentionPolicy

PK HistorianRetentionPolicyId

FK HistorianRetentionClassId

Retention, aggregation and purge contract for signals in one class.

HISTORIAN

Instrumentation.HistorianImportBatch

PK HistorianImportBatchId

FK MeasurementSourceId, CreatedByUserId

Imported block of measurements from simulator, historian or CSV feed.

HISTORIAN

Instrumentation.HistorianImportBatchItem

PK HistorianImportBatchItemId

FK HistorianImportBatchId, SignalId

Per-signal import counts and time range inside one batch.

HISTORIAN

Instrumentation.HistorianBackfillJob

PK HistorianBackfillJobId

FK SignalId, BackfillJobStatusId

Job used to backfill missing measurements or recompute derived signals.

C.5.3 Lookup tables

Lookup values are deliberately explicit because hardcoded signal states lead to disagreement between historian, alarms, twin and root-cause reasoning.

LOOKUP TABLE	TYPICAL SEED CODES	MAIN CONSUMER
SignalType	MEASURED , CALCULATED , COMMAND , STATE , SIMULATED	Referenced by <code>Signal</code> .
SignalCategory	POWER , TEMPERATURE , PRESSURE , FLOW , RADIATION , VIBRATION	Referenced by <code>Signal</code> .
SignalRole	TELEMETRY , TWIN_INPUT , RCA_WITNESS , TRAINING	Referenced by <code>Signal</code> .
SensorType	RTD , PRESSURE_TX , FLOW_TX , NEUTRON_DETECTOR , VIBRATION_PROBE	Referenced by <code>Sensor</code> .
SensorStatus	AVAILABLE , MAINTENANCE , SUSPECT , FAILED , RETIRED	Referenced by <code>Instrument</code> and <code>Sensor</code> .
ChannelStatus	ONLINE , STALE , DISABLED , FAULTED , SIMULATED	Referenced by acquisition nodes, connections and channels.
SignalQuality	GOOD , UNCERTAIN , BAD , STALE , SUBSTITUTED , SIMULATED	Referenced by measurements and quality events.
SamplingMode	PERIODIC , EVENT , MANUAL , DERIVED , SIMULATED	Referenced by <code>Signal</code> .
MeasurementSource	HISTORIAN , SIMULATOR , OPERATOR , IMPORT , TEST	Referenced by measurements and import batches.
CalibrationStatus	SCHEDULED , DUE , OVERDUE , PASSED , FAILED , WAIVED	Referenced by calibration plans.
HistorianRetentionClass	RAW_FAST , RAW_STANDARD , AGGREGATED , ARCHIVE	Referenced by <code>Signal</code> and retention policies.
ThresholdType	HIGH , HIGH_HIGH , LOW , LOW_LOW , RATE , DEVIATION	Referenced by <code>SignalLimit</code> .
BackfillJobStatus	QUEUED , RUNNING , COMPLETED , FAILED , CANCELLED	Referenced by <code>HistorianBackfillJob</code> .
LineageRole	PRIMARY , WITNESS , FALLBACK , CORRECTION , NORMALIZATION	Referenced by <code>SignalLineage</code> .

C.5.4 SQL DDL

C.5.4.1 Schema

```
IF SCHEMA_ID(N'Instrumentation') IS NULL
    EXEC(N'CREATE SCHEMA Instrumentation');
GO
```

C.5.4.2 Instrumentation lookup tables

```
CREATE TABLE Instrumentation.SignalType (
    SignalTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Instrumentation_SignalType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Instrumentation_SignalType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Instrumentation_SignalType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Instrumentation_SignalType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Instrumentation_SignalType
    PRIMARY KEY (SignalTypeId),
    CONSTRAINT UQ_Instrumentation_SignalType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Instrumentation.SignalCategory (
    SignalCategoryId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Instrumentation_SignalCategory_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Instrumentation_SignalCategory_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Instrumentation_SignalCategory_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Instrumentation_SignalCategory_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Instrumentation_SignalCategory
    PRIMARY KEY (SignalCategoryId),
    CONSTRAINT UQ_Instrumentation_SignalCategory_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Instrumentation.SignalRole (
    SignalRoleId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Instrumentation_SignalRole_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Instrumentation_SignalRole_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Instrumentation_SignalRole_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Instrumentation_SignalRole_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Instrumentation_SignalRole
    PRIMARY KEY (SignalRoleId),
    CONSTRAINT UQ_Instrumentation_SignalRole_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Instrumentation.SensorType (
    SensorTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Instrumentation_SensorType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Instrumentation_SensorType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Instrumentation_SensorType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Instrumentation_SensorType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Instrumentation_SensorType
    PRIMARY KEY (SensorTypeId),
    CONSTRAINT UQ_Instrumentation_SensorType_Code
    UNIQUE (Code)
);
GO
```

```

CREATE TABLE Instrumentation.SensorStatus (
    SensorStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Instrumentation_SensorStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Instrumentation_SensorStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Instrumentation_SensorStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Instrumentation_SensorStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Instrumentation_SensorStatus
    PRIMARY KEY (SensorStatusId),
    CONSTRAINT UQ_Instrumentation_SensorStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Instrumentation.ChannelStatus (
    ChannelStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Instrumentation_ChannelStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Instrumentation_ChannelStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Instrumentation_ChannelStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Instrumentation_ChannelStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Instrumentation_ChannelStatus
    PRIMARY KEY (ChannelStatusId),
    CONSTRAINT UQ_Instrumentation_ChannelStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Instrumentation.SignalQuality (
    SignalQualityId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Instrumentation_SignalQuality_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Instrumentation_SignalQuality_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Instrumentation_SignalQuality_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Instrumentation_SignalQuality_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Instrumentation_SignalQuality
    PRIMARY KEY (SignalQualityId),
    CONSTRAINT UQ_Instrumentation_SignalQuality_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Instrumentation.SamplingMode (
    SamplingModeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Instrumentation_SamplingMode_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Instrumentation_SamplingMode_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Instrumentation_SamplingMode_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Instrumentation_SamplingMode_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Instrumentation_SamplingMode
    PRIMARY KEY (SamplingModeId),
    CONSTRAINT UQ_Instrumentation_SamplingMode_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Instrumentation.MeasurementSource (

```

```

MeasurementSourceId      INT IDENTITY(1,1) NOT NULL,
Code                     NVARCHAR(50) NOT NULL,
Name                     NVARCHAR(150) NOT NULL,
Description               NVARCHAR(500) NULL,
DisplayOrder             INT NOT NULL
    CONSTRAINT DF_Instrumentation_MeasurementSource_DisplayOrder DEFAULT 0,
IsActive                 BIT NOT NULL
    CONSTRAINT DF_Instrumentation_MeasurementSource_IsActive DEFAULT 1,
CreatedAtUtc             DATETIME2(7) NOT NULL
    CONSTRAINT DF_Instrumentation_MeasurementSource_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc            DATETIME2(7) NOT NULL
    CONSTRAINT DF_Instrumentation_MeasurementSource_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion               ROWVERSION NOT NULL,

CONSTRAINT PK_Instrumentation_MeasurementSource
    PRIMARY KEY (MeasurementSourceId),
CONSTRAINT UQ_Instrumentation_MeasurementSource_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Instrumentation.CalibrationStatus (
    CalibrationStatusId    INT IDENTITY(1,1) NOT NULL,
    Code                   NVARCHAR(50) NOT NULL,
    Name                   NVARCHAR(150) NOT NULL,
    Description             NVARCHAR(500) NULL,
    DisplayOrder            INT NOT NULL
        CONSTRAINT DF_Instrumentation_CalibrationStatus_DisplayOrder DEFAULT 0,
    IsActive               BIT NOT NULL
        CONSTRAINT DF_Instrumentation_CalibrationStatus_IsActive DEFAULT 1,
    CreatedAtUtc            DATETIME2(7) NOT NULL
        CONSTRAINT DF_Instrumentation_CalibrationStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc           DATETIME2(7) NOT NULL
        CONSTRAINT DF_Instrumentation_CalibrationStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion              ROWVERSION NOT NULL,

CONSTRAINT PK_Instrumentation_CalibrationStatus
    PRIMARY KEY (CalibrationStatusId),
CONSTRAINT UQ_Instrumentation_CalibrationStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Instrumentation.HistorianRetentionClass (
    HistorianRetentionClassId INT IDENTITY(1,1) NOT NULL,
    Code                       NVARCHAR(50) NOT NULL,
    Name                       NVARCHAR(150) NOT NULL,
    Description                 NVARCHAR(500) NULL,
    DisplayOrder                INT NOT NULL
        CONSTRAINT DF_Instrumentation_HistorianRetentionClass_DisplayOrder DEFAULT 0,
    IsActive                   BIT NOT NULL
        CONSTRAINT DF_Instrumentation_HistorianRetentionClass_IsActive DEFAULT 1,
    CreatedAtUtc                DATETIME2(7) NOT NULL
        CONSTRAINT DF_Instrumentation_HistorianRetentionClass_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc               DATETIME2(7) NOT NULL
        CONSTRAINT DF_Instrumentation_HistorianRetentionClass_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion                  ROWVERSION NOT NULL,

CONSTRAINT PK_Instrumentation_HistorianRetentionClass
    PRIMARY KEY (HistorianRetentionClassId),
CONSTRAINT UQ_Instrumentation_HistorianRetentionClass_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Instrumentation.ThresholdType (
    ThresholdTypeId          INT IDENTITY(1,1) NOT NULL,
    Code                     NVARCHAR(50) NOT NULL,
    Name                     NVARCHAR(150) NOT NULL,
    Description               NVARCHAR(500) NULL,
    DisplayOrder              INT NOT NULL
        CONSTRAINT DF_Instrumentation_ThresholdType_DisplayOrder DEFAULT 0,
    IsActive                 BIT NOT NULL
        CONSTRAINT DF_Instrumentation_ThresholdType_IsActive DEFAULT 1,
    CreatedAtUtc              DATETIME2(7) NOT NULL
        CONSTRAINT DF_Instrumentation_ThresholdType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc             DATETIME2(7) NOT NULL
        CONSTRAINT DF_Instrumentation_ThresholdType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion                ROWVERSION NOT NULL,

CONSTRAINT PK_Instrumentation_ThresholdType
    PRIMARY KEY (ThresholdTypeId),
CONSTRAINT UQ_Instrumentation_ThresholdType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Instrumentation.BackfillJobStatus (
    BackfillJobStatusId      INT IDENTITY(1,1) NOT NULL,
    Code                     NVARCHAR(50) NOT NULL,
    Name                     NVARCHAR(150) NOT NULL,

```

```

Description    NVARCHAR(500) NULL,
DisplayOrder   INT NOT NULL
    CONSTRAINT DF_Instrumentation_BackfillJobStatus_DisplayOrder DEFAULT 0,
IsActive       BIT NOT NULL
    CONSTRAINT DF_Instrumentation_BackfillJobStatus_IsActive DEFAULT 1,
CreatedAtUtc   DATETIME2(7) NOT NULL
    CONSTRAINT DF_Instrumentation_BackfillJobStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc  DATETIME2(7) NOT NULL
    CONSTRAINT DF_Instrumentation_BackfillJobStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion     ROWVERSION NOT NULL,

CONSTRAINT PK_Instrumentation_BackfillJobStatus
    PRIMARY KEY (BackfillJobStatusId),
CONSTRAINT UQ_Instrumentation_BackfillJobStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Instrumentation.LineageRole (
    LineageRoleId INT IDENTITY(1,1) NOT NULL,
    Code          NVARCHAR(50) NOT NULL,
    Name          NVARCHAR(150) NOT NULL,
    Description    NVARCHAR(500) NULL,
    DisplayOrder  INT NOT NULL
    CONSTRAINT DF_Instrumentation_LineageRole_DisplayOrder DEFAULT 0,
    IsActive      BIT NOT NULL
    CONSTRAINT DF_Instrumentation_LineageRole_IsActive DEFAULT 1,
    CreatedAtUtc  DATETIME2(7) NOT NULL
    CONSTRAINT DF_Instrumentation_LineageRole_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Instrumentation_LineageRole_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion    ROWVERSION NOT NULL,

CONSTRAINT PK_Instrumentation_LineageRole
    PRIMARY KEY (LineageRoleId),
CONSTRAINT UQ_Instrumentation_LineageRole_Code
    UNIQUE (Code)
);
GO

```

C.5.4.3 Acquisition nodes, connections and raw points

```

CREATE TABLE Instrumentation.DataAcquisitionNode (
    DataAcquisitionNodeId INT IDENTITY(1,1) NOT NULL,
    UnitId                INT NOT NULL,
    ChannelStatusId       INT NOT NULL,
    Code                  NVARCHAR(80) NOT NULL,
    Name                  NVARCHAR(250) NOT NULL,
    HostName              NVARCHAR(250) NULL,
    NetworkZone           NVARCHAR(100) NULL,
    Description            NVARCHAR(1000) NULL,
    CreatedAtUtc           DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_DataAcquisitionNode_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy             NVARCHAR(100) NOT NULL,
    ModifiedAtUtc          DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_DataAcquisitionNode_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
    ModifiedBy            NVARCHAR(100) NOT NULL,
    IsDeleted              BIT NOT NULL CONSTRAINT DF_Instrumentation_DataAcquisitionNode_IsDeleted DEFAULT 0,
    RowVersion             ROWVERSION NOT NULL,

CONSTRAINT PK_Instrumentation_DataAcquisitionNode PRIMARY KEY (DataAcquisitionNodeId),
CONSTRAINT UQ_Instrumentation_DataAcquisitionNode_Unit_Code UNIQUE (UnitId, Code),
CONSTRAINT FK_Instrumentation_DataAcquisitionNode_Unit
    FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
CONSTRAINT FK_Instrumentation_DataAcquisitionNode_ChannelStatus
    FOREIGN KEY (ChannelStatusId) REFERENCES Instrumentation.ChannelStatus(ChannelStatusId)
);
GO

CREATE TABLE Instrumentation.AcquisitionConnection (
    AcquisitionConnectionId INT IDENTITY(1,1) NOT NULL,
    DataAcquisitionNodeId  INT NOT NULL,
    ChannelStatusId        INT NOT NULL,
    Code                   NVARCHAR(80) NOT NULL,
    Protocol               NVARCHAR(80) NOT NULL,
    Endpoint               NVARCHAR(500) NULL,
    PollIntervalMs         INT NULL,
    IsReadOnly              BIT NOT NULL CONSTRAINT DF_Instrumentation_AcquisitionConnection_IsReadOnly DEFAULT 1,
    CreatedAtUtc            DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_AcquisitionConnection_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy              NVARCHAR(100) NOT NULL,
    ModifiedAtUtc           DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_AcquisitionConnection_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
    ModifiedBy             NVARCHAR(100) NOT NULL,
    IsDeleted               BIT NOT NULL CONSTRAINT DF_Instrumentation_AcquisitionConnection_IsDeleted DEFAULT 0,
    RowVersion              ROWVERSION NOT NULL,

CONSTRAINT PK_Instrumentation_AcquisitionConnection PRIMARY KEY (AcquisitionConnectionId),

```

```

CONSTRAINT UQ_Instrumentation_AcquisitionConnection_Node_Code UNIQUE (DataAcquisitionNodeId, Code),
CONSTRAINT FK_Instrumentation_AcquisitionConnection_DataAcquisitionNode
    FOREIGN KEY (DataAcquisitionNodeId) REFERENCES Instrumentation.DataAcquisitionNode(DataAcquisitionNodeId),
CONSTRAINT FK_Instrumentation_AcquisitionConnection_ChannelStatus
    FOREIGN KEY (ChannelStatusId) REFERENCES Instrumentation.ChannelStatus(ChannelStatusId),
CONSTRAINT CK_Instrumentation_AcquisitionConnection_PollInterval CHECK (PollIntervalMs IS NULL OR PollIntervalMs > 0)
);
GO

CREATE TABLE Instrumentation.AcquisitionPoint (
    AcquisitionPointId INT IDENTITY(1,1) NOT NULL,
    AcquisitionConnectionId INT NOT NULL,
    Code NVARCHAR(100) NOT NULL,
    RawAddress NVARCHAR(300) NOT NULL,
    RawDataType NVARCHAR(80) NULL,
    ScaleFactor DECIMAL(18,8) NULL,
    OffsetValue DECIMAL(18,8) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_AcquisitionPoint_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_AcquisitionPoint_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Instrumentation_AcquisitionPoint PRIMARY KEY (AcquisitionPointId),
    CONSTRAINT UQ_Instrumentation_AcquisitionPoint_Connection_Code UNIQUE (AcquisitionConnectionId, Code),
    CONSTRAINT FK_Instrumentation_AcquisitionPoint_AcquisitionConnection
        FOREIGN KEY (AcquisitionConnectionId) REFERENCES Instrumentation.AcquisitionConnection(AcquisitionConnectionId)
);
GO

```

C.5.4.4 Instruments, sensors and channels

```

CREATE TABLE Instrumentation.Instrument (
    InstrumentId INT IDENTITY(1,1) NOT NULL,
    UnitId INT NOT NULL,
    EquipmentId INT NULL,
    EquipmentLocationId INT NULL,
    SensorStatusId INT NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(250) NOT NULL,
    Manufacturer NVARCHAR(200) NULL,
    ModelNumber NVARCHAR(120) NULL,
    SerialNumber NVARCHAR(120) NULL,
    InstalledOn DATE NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_Instrument_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_Instrument_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Instrumentation_Instrument_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Instrumentation_Instrument PRIMARY KEY (InstrumentId),
    CONSTRAINT UQ_Instrumentation_Instrument_Unit_Code UNIQUE (UnitId, Code),
    CONSTRAINT FK_Instrumentation_Instrument_Unit
        FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_Instrumentation_Instrument_Equipment
        FOREIGN KEY (EquipmentId) REFERENCES ReactorFleet.Equipment(EquipmentId),
    CONSTRAINT FK_Instrumentation_Instrument_EquipmentLocation
        FOREIGN KEY (EquipmentLocationId) REFERENCES ReactorFleet.EquipmentLocation(EquipmentLocationId),
    CONSTRAINT FK_Instrumentation_Instrument_SensorStatus
        FOREIGN KEY (SensorStatusId) REFERENCES Instrumentation.SensorStatus(SensorStatusId)
);
GO

CREATE TABLE Instrumentation.Sensor (
    SensorId INT IDENTITY(1,1) NOT NULL,
    InstrumentId INT NOT NULL,
    SensorTypeId INT NOT NULL,
    SensorStatusId INT NOT NULL,
    EngineeringUnitId INT NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(250) NOT NULL,
    RangeMin DECIMAL(18,6) NULL,
    RangeMax DECIMAL(18,6) NULL,
    AccuracyPercent DECIMAL(9,6) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_Sensor_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_Sensor_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Instrumentation_Sensor_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Instrumentation_Sensor PRIMARY KEY (SensorId),
    CONSTRAINT UQ_Instrumentation_Sensor_Instrument_Code UNIQUE (InstrumentId, Code),
    CONSTRAINT FK_Instrumentation_Sensor_Instrument

```

```

        FOREIGN KEY (InstrumentId) REFERENCES Instrumentation.Instrument(InstrumentId),
    CONSTRAINT FK_Instrumentation_Sensor_SensorType
        FOREIGN KEY (SensorTypeId) REFERENCES Instrumentation.SensorType(SensorTypeId),
    CONSTRAINT FK_Instrumentation_Sensor_SensorStatus
        FOREIGN KEY (SensorStatusId) REFERENCES Instrumentation.SensorStatus(SensorStatusId),
    CONSTRAINT FK_Instrumentation_Sensor_EngineeringUnit
        FOREIGN KEY (EngineeringUnitId) REFERENCES CorePlatform.EngineeringUnit(EngineeringUnitId),
    CONSTRAINT CK_Instrumentation_Sensor_Range CHECK (RangeMin IS NULL OR RangeMax IS NULL OR RangeMax > RangeMin)
);
GO

CREATE TABLE Instrumentation.SensorChannel (
    SensorChannelId INT IDENTITY(1,1) NOT NULL,
    SensorId INT NOT NULL,
    ChannelStatusId INT NOT NULL,
    ChannelCode NVARCHAR(80) NOT NULL,
    ChannelNumber INT NULL,
    IsPrimary BIT NOT NULL CONSTRAINT DF_Instrumentation_SensorChannel_IsPrimary DEFAULT 0,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_SensorChannel_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_SensorChannel_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Instrumentation_SensorChannel PRIMARY KEY (SensorChannelId),
    CONSTRAINT UQ_Instrumentation_SensorChannel_Sensor_Code UNIQUE (SensorId, ChannelCode),
    CONSTRAINT FK_Instrumentation_SensorChannel_Sensor
        FOREIGN KEY (SensorId) REFERENCES Instrumentation.Sensor(SensorId),
    CONSTRAINT FK_Instrumentation_SensorChannel_ChannelStatus
        FOREIGN KEY (ChannelStatusId) REFERENCES Instrumentation.ChannelStatus(ChannelStatusId)
);
GO

```

C.5.4.5 Signal registry, aliases, groups, mapping and lineage

```

CREATE TABLE Instrumentation.Signal (
    SignalId INT IDENTITY(1,1) NOT NULL,
    UnitId INT NOT NULL,
    EquipmentId INT NULL,
    PlantSystemId INT NULL,
    SensorChannelId INT NULL,
    SignalTypeId INT NOT NULL,
    SignalCategoryId INT NOT NULL,
    SignalRoleId INT NOT NULL,
    EngineeringUnitId INT NOT NULL,
    SamplingModeId INT NOT NULL,
    HistorianRetentionClassId INT NOT NULL,
    Tag NVARCHAR(80) NOT NULL,
    Name NVARCHAR(250) NOT NULL,
    Description NVARCHAR(1000) NULL,
    ScanRateHz DECIMAL(12,6) NULL,
    PrecisionDigits INT NULL,
    NormalMin DECIMAL(18,6) NULL,
    NormalMax DECIMAL(18,6) NULL,
    IsSafetyRelated BIT NOT NULL CONSTRAINT DF_Instrumentation_Signal_IsSafetyRelated DEFAULT 0,
    IsHistorized BIT NOT NULL CONSTRAINT DF_Instrumentation_Signal_IsHistorized DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_Signal_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_Signal_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Instrumentation_Signal_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Instrumentation_Signal PRIMARY KEY (SignalId),
    CONSTRAINT UQ_Instrumentation_Signal_Tag UNIQUE (Tag),
    CONSTRAINT FK_Instrumentation_Signal_Unit
        FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_Instrumentation_Signal_Equipment
        FOREIGN KEY (EquipmentId) REFERENCES ReactorFleet.Equipment(EquipmentId),
    CONSTRAINT FK_Instrumentation_Signal_PlantSystem
        FOREIGN KEY (PlantSystemId) REFERENCES ReactorFleet.PlantSystem(PlantSystemId),
    CONSTRAINT FK_Instrumentation_Signal_SensorChannel
        FOREIGN KEY (SensorChannelId) REFERENCES Instrumentation.SensorChannel(SensorChannelId),
    CONSTRAINT FK_Instrumentation_Signal_SignalType
        FOREIGN KEY (SignalTypeId) REFERENCES Instrumentation.SignalType(SignalTypeId),
    CONSTRAINT FK_Instrumentation_Signal_SignalCategory
        FOREIGN KEY (SignalCategoryId) REFERENCES Instrumentation.SignalCategory(SignalCategoryId),
    CONSTRAINT FK_Instrumentation_Signal_SignalRole
        FOREIGN KEY (SignalRoleId) REFERENCES Instrumentation.SignalRole(SignalRoleId),
    CONSTRAINT FK_Instrumentation_Signal_EngineeringUnit
        FOREIGN KEY (EngineeringUnitId) REFERENCES CorePlatform.EngineeringUnit(EngineeringUnitId),
    CONSTRAINT FK_Instrumentation_Signal_SamplingMode
        FOREIGN KEY (SamplingModeId) REFERENCES Instrumentation.SamplingMode(SamplingModeId),
    CONSTRAINT FK_Instrumentation_Signal_HistorianRetentionClass
        FOREIGN KEY (HistorianRetentionClassId) REFERENCES Instrumentation.HistorianRetentionClass(HistorianRetentionClassId),
    CONSTRAINT CK_Instrumentation_Signal_NormalRange CHECK (NormalMin IS NULL OR NormalMax IS NULL OR NormalMax > NormalMin),
    CONSTRAINT CK_Instrumentation_Signal_ScanRate CHECK (ScanRateHz IS NULL OR ScanRateHz > 0)
);

```



```

);
GO

CREATE TABLE Instrumentation.SignalAlias (
    SignalAliasId INT IDENTITY(1,1) NOT NULL,
    SignalId INT NOT NULL,
    AliasType NVARCHAR(50) NOT NULL,
    AliasValue NVARCHAR(150) NOT NULL,
    IsPreferred BIT NOT NULL CONSTRAINT DF_Instrumentation_SignalAlias_IsPreferred DEFAULT 0,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_SignalAlias_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Instrumentation_SignalAlias PRIMARY KEY (SignalAliasId),
    CONSTRAINT UQ_Instrumentation_SignalAlias_Type_Value UNIQUE (AliasType, AliasValue),
    CONSTRAINT FK_Instrumentation_SignalAlias_Signal
        FOREIGN KEY (SignalId) REFERENCES Instrumentation.Signal(SignalId)
);
GO

CREATE TABLE Instrumentation.SignalGroup (
    SignalGroupId INT IDENTITY(1,1) NOT NULL,
    UnitId INT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(250) NOT NULL,
    Description NVARCHAR(1000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_SignalGroup_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_SignalGroup_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Instrumentation_SignalGroup PRIMARY KEY (SignalGroupId),
    CONSTRAINT UQ_Instrumentation_SignalGroup_Code UNIQUE (Code),
    CONSTRAINT FK_Instrumentation_SignalGroup_Unit
        FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId)
);
GO

CREATE TABLE Instrumentation.SignalGroupMember (
    SignalGroupMemberId INT IDENTITY(1,1) NOT NULL,
    SignalGroupId INT NOT NULL,
    SignalId INT NOT NULL,
    DisplayOrder INT NOT NULL CONSTRAINT DF_Instrumentation_SignalGroupMember_DisplayOrder DEFAULT 0,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_SignalGroupMember_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Instrumentation_SignalGroupMember PRIMARY KEY (SignalGroupMemberId),
    CONSTRAINT UQ_Instrumentation_SignalGroupMember_Group_Signal UNIQUE (SignalGroupId, SignalId),
    CONSTRAINT FK_Instrumentation_SignalGroupMember_SignalGroup
        FOREIGN KEY (SignalGroupId) REFERENCES Instrumentation.SignalGroup(SignalGroupId),
    CONSTRAINT FK_Instrumentation_SignalGroupMember_Signal
        FOREIGN KEY (SignalId) REFERENCES Instrumentation.Signal(SignalId)
);
GO

CREATE TABLE Instrumentation.SignalMapping (
    SignalMappingId INT IDENTITY(1,1) NOT NULL,
    SignalId INT NOT NULL,
    AcquisitionPointId INT NOT NULL,
    EffectiveFromUtc DATETIME2(7) NOT NULL,
    EffectiveToUtc DATETIME2(7) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_SignalMapping_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Instrumentation_SignalMapping PRIMARY KEY (SignalMappingId),
    CONSTRAINT UQ_Instrumentation_SignalMapping_Signal_Point_From UNIQUE (SignalId, AcquisitionPointId, EffectiveFromUtc),
    CONSTRAINT FK_Instrumentation_SignalMapping_Signal
        FOREIGN KEY (SignalId) REFERENCES Instrumentation.Signal(SignalId),
    CONSTRAINT FK_Instrumentation_SignalMapping_AcquisitionPoint
        FOREIGN KEY (AcquisitionPointId) REFERENCES Instrumentation.AcquisitionPoint(AcquisitionPointId),
    CONSTRAINT CK_Instrumentation_SignalMapping_Effective CHECK (EffectiveToUtc IS NULL OR EffectiveToUtc > EffectiveFromUtc)
);
GO

CREATE TABLE Instrumentation.SignalDependency (
    SignalDependencyId BIGINT IDENTITY(1,1) NOT NULL,
    SourceSignalId INT NOT NULL,
    TargetSignalId INT NOT NULL,
    DependencyCode NVARCHAR(80) NOT NULL,
    InfluenceWeight DECIMAL(9,6) NOT NULL CONSTRAINT DF_Instrumentation_SignalDependency_InfluenceWeight DEFAULT 1,
    NominalLagSeconds DECIMAL(12,3) NULL,
    Description NVARCHAR(1000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_SignalDependency_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Instrumentation_SignalDependency PRIMARY KEY (SignalDependencyId),
    CONSTRAINT UQ_Instrumentation_SignalDependency_Source_Target_Code UNIQUE (SourceSignalId, TargetSignalId, DependencyCode),
    CONSTRAINT FK_Instrumentation_SignalDependency_SourceSignal

```



```

        FOREIGN KEY (SourceSignalId) REFERENCES Instrumentation.Signal(SignalId),
        CONSTRAINT FK_Instrumentation_SignalDependency_TargetSignal
        FOREIGN KEY (TargetSignalId) REFERENCES Instrumentation.Signal(SignalId),
        CONSTRAINT CK_Instrumentation_SignalDependency_NoSelf CHECK (SourceSignalId <> TargetSignalId)
    );
GO

CREATE TABLE Instrumentation.SignalLineage (
    SignalLineageId INT IDENTITY(1,1) NOT NULL,
    DerivedSignalId INT NOT NULL,
    SourceSignalId INT NOT NULL,
    LineageRoleId INT NOT NULL,
    FormulaFragment NVARCHAR(1000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_SignalLineage_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Instrumentation_SignalLineage PRIMARY KEY (SignalLineageId),
    CONSTRAINT UQ_Instrumentation_SignalLineage_Derived_Source_Role UNIQUE (DerivedSignalId, SourceSignalId, LineageRoleId),
    CONSTRAINT FK_Instrumentation_SignalLineage_DerivedSignal
        FOREIGN KEY (DerivedSignalId) REFERENCES Instrumentation.Signal(SignalId),
    CONSTRAINT FK_Instrumentation_SignalLineage_SourceSignal
        FOREIGN KEY (SourceSignalId) REFERENCES Instrumentation.Signal(SignalId),
    CONSTRAINT FK_Instrumentation_SignalLineage_LineageRole
        FOREIGN KEY (LineageRoleId) REFERENCES Instrumentation.LineageRole(LineageRoleId),
    CONSTRAINT CK_Instrumentation_SignalLineage_NoSelf CHECK (DerivedSignalId <> SourceSignalId)
);
GO

```

C.5.4.6 Limits, deadbands and calibration

```

CREATE TABLE Instrumentation.SignalLimit (
    SignalLimitId INT IDENTITY(1,1) NOT NULL,
    SignalId INT NOT NULL,
    ThresholdTypeId INT NOT NULL,
    LimitCode NVARCHAR(80) NOT NULL,
    LimitValue DECIMAL(18,6) NOT NULL,
    SeverityCode NVARCHAR(40) NULL,
    IsActive BIT NOT NULL CONSTRAINT DF_Instrumentation_SignalLimit_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_SignalLimit_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_SignalLimit_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Instrumentation_SignalLimit PRIMARY KEY (SignalLimitId),
    CONSTRAINT UQ_Instrumentation_SignalLimit_Signal_Code UNIQUE (SignalId, LimitCode),
    CONSTRAINT FK_Instrumentation_SignalLimit_Signal
        FOREIGN KEY (SignalId) REFERENCES Instrumentation.Signal(SignalId),
    CONSTRAINT FK_Instrumentation_SignalLimit_ThresholdType
        FOREIGN KEY (ThresholdTypeId) REFERENCES Instrumentation.ThresholdType(ThresholdTypeId)
);
GO

CREATE TABLE Instrumentation.SignalDeadband (
    SignalDeadbandId INT IDENTITY(1,1) NOT NULL,
    SignalId INT NOT NULL,
    DeadbandValue DECIMAL(18,6) NOT NULL,
    PercentOfRange DECIMAL(9,6) NULL,
    Purpose NVARCHAR(100) NOT NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_SignalDeadband_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_SignalDeadband_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Instrumentation_SignalDeadband PRIMARY KEY (SignalDeadbandId),
    CONSTRAINT UQ_Instrumentation_SignalDeadband_Signal_Purpose UNIQUE (SignalId, Purpose),
    CONSTRAINT FK_Instrumentation_SignalDeadband_Signal
        FOREIGN KEY (SignalId) REFERENCES Instrumentation.Signal(SignalId),
    CONSTRAINT CK_Instrumentation_SignalDeadband_Positive CHECK (DeadbandValue >= 0)
);
GO

CREATE TABLE Instrumentation.CalibrationPlan (
    CalibrationPlanId INT IDENTITY(1,1) NOT NULL,
    SignalId INT NOT NULL,
    SensorId INT NULL,
    CalibrationStatusId INT NOT NULL,
    IntervalDays INT NOT NULL,
    NextDueDate DATE NULL,
    ProcedureReference NVARCHAR(300) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_CalibrationPlan_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_CalibrationPlan_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
);
GO

```

```

    ModifiedBy          NVARCHAR(100) NOT NULL,
    IsDeleted           BIT NOT NULL CONSTRAINT DF_Instrumentation_CalibrationPlan_IsDeleted DEFAULT 0,
    RowVersion          ROWVERSION NOT NULL,

    CONSTRAINT PK_Instrumentation_CalibrationPlan PRIMARY KEY (CalibrationPlanId),
    CONSTRAINT UQ_Instrumentation_CalibrationPlan_Signal UNIQUE (SignalId),
    CONSTRAINT FK_Instrumentation_CalibrationPlan_Signal
        FOREIGN KEY (SignalId) REFERENCES Instrumentation.Signal(SignalId),
    CONSTRAINT FK_Instrumentation_CalibrationPlan_Sensor
        FOREIGN KEY (SensorId) REFERENCES Instrumentation.Sensor(SensorId),
    CONSTRAINT FK_Instrumentation_CalibrationPlan_CalibrationStatus
        FOREIGN KEY (CalibrationStatusId) REFERENCES Instrumentation.CalibrationStatus(CalibrationStatusId),
    CONSTRAINT CK_Instrumentation_CalibrationPlan_Interval CHECK (IntervalDays > 0)
);
GO

CREATE TABLE Instrumentation.CalibrationRecord (
    CalibrationRecordId INT IDENTITY(1,1) NOT NULL,
    CalibrationPlanId    INT NOT NULL,
    PerformedByUserId    INT NOT NULL,
    PerformedAtUtc       DATETIME2(7) NOT NULL,
    AsFoundValue         DECIMAL(18,6) NULL,
    AsLeftValue          DECIMAL(18,6) NULL,
    Passed               BIT NOT NULL,
    Notes                NVARCHAR(1000) NULL,
    CreatedAtUtc         DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_CalibrationRecord_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy            NVARCHAR(100) NOT NULL,
    RowVersion           ROWVERSION NOT NULL,

    CONSTRAINT PK_Instrumentation_CalibrationRecord PRIMARY KEY (CalibrationRecordId),
    CONSTRAINT FK_Instrumentation_CalibrationRecord_CalibrationPlan
        FOREIGN KEY (CalibrationPlanId) REFERENCES Instrumentation.CalibrationPlan(CalibrationPlanId),
    CONSTRAINT FK_Instrumentation_CalibrationRecord_PerformedByUser
        FOREIGN KEY (PerformedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId)
);
GO

```

C.5.4.7 Historian measurements, aggregates, quality and backfill

```

CREATE TABLE Instrumentation.Measurement (
    MeasurementId        BIGINT IDENTITY(1,1) NOT NULL,
    SignalId              INT NOT NULL,
    SignalQualityId       INT NOT NULL,
    MeasurementSourceId   INT NOT NULL,
    TimestampUtc          DATETIME2(7) NOT NULL,
    NumericValue          FLOAT NULL,
    TextValue             NVARCHAR(400) NULL,
    IsEstimated           BIT NOT NULL CONSTRAINT DF_Instrumentation_Measurement_IsEstimated DEFAULT 0,
    InsertedAtUtc         DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_Measurement_InsertedAtUtc DEFAULT
SYSUTCDATETIME(),

    CONSTRAINT PK_Instrumentation_Measurement PRIMARY KEY NONCLUSTERED (MeasurementId),
    CONSTRAINT FK_Instrumentation_Measurement_Signal
        FOREIGN KEY (SignalId) REFERENCES Instrumentation.Signal(SignalId),
    CONSTRAINT FK_Instrumentation_Measurement_SignalQuality
        FOREIGN KEY (SignalQualityId) REFERENCES Instrumentation.SignalQuality(SignalQualityId),
    CONSTRAINT FK_Instrumentation_Measurement_MeasurementSource
        FOREIGN KEY (MeasurementSourceId) REFERENCES Instrumentation.MeasurementSource(MeasurementSourceId),
    CONSTRAINT CK_Instrumentation_Measurement_OneValue CHECK (NumericValue IS NOT NULL OR TextValue IS NOT NULL)
);
GO

CREATE TABLE Instrumentation.MeasurementAggregate (
    MeasurementAggregateId BIGINT IDENTITY(1,1) NOT NULL,
    SignalId                INT NOT NULL,
    SignalQualityId          INT NOT NULL,
    WindowStartUtc          DATETIME2(7) NOT NULL,
    WindowSeconds            INT NOT NULL,
    SampleCount              INT NOT NULL,
    MinValue                 FLOAT NULL,
    MaxValue                 FLOAT NULL,
    AvgValue                 FLOAT NULL,
    LastValue                FLOAT NULL,
    InsertedAtUtc           DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_MeasurementAggregate_InsertedAtUtc DEFAULT
SYSUTCDATETIME(),

    CONSTRAINT PK_Instrumentation_MeasurementAggregate PRIMARY KEY (MeasurementAggregateId),
    CONSTRAINT UQ_Instrumentation_MeasurementAggregate_Signal_Window UNIQUE (SignalId, WindowStartUtc, WindowSeconds),
    CONSTRAINT FK_Instrumentation_MeasurementAggregate_Signal
        FOREIGN KEY (SignalId) REFERENCES Instrumentation.Signal(SignalId),
    CONSTRAINT FK_Instrumentation_MeasurementAggregate_SignalQuality
        FOREIGN KEY (SignalQualityId) REFERENCES Instrumentation.SignalQuality(SignalQualityId),
    CONSTRAINT CK_Instrumentation_MeasurementAggregate_Window CHECK (WindowSeconds > 0 AND SampleCount >= 0)
);
GO

CREATE TABLE Instrumentation.MeasurementAnnotation (
    MeasurementAnnotationId BIGINT IDENTITY(1,1) NOT NULL,

```

```

MeasurementId          BIGINT NOT NULL,
CreatedByUserId         INT NULL,
AnnotationCode          NVARCHAR(80) NOT NULL,
AnnotationText          NVARCHAR(1000) NOT NULL,
CreatedAtUtc            DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_MeasurementAnnotation_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),

CONSTRAINT PK_Instrumentation_MeasurementAnnotation PRIMARY KEY (MeasurementAnnotationId),
CONSTRAINT FK_Instrumentation_MeasurementAnnotation_Measurement
    FOREIGN KEY (MeasurementId) REFERENCES Instrumentation.Measurement(MeasurementId),
CONSTRAINT FK_Instrumentation_MeasurementAnnotation_CreatedByUser
    FOREIGN KEY (CreatedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE Instrumentation.SignalQualityEvent (
    SignalQualityEventId BIGINT IDENTITY(1,1) NOT NULL,
    SignalId              INT NOT NULL,
    SignalQualityId        INT NOT NULL,
    StartedAtUtc           DATETIME2(7) NOT NULL,
    EndedAtUtc             DATETIME2(7) NULL,
    ReasonCode             NVARCHAR(80) NULL,
    Notes                  NVARCHAR(1000) NULL,
    CreatedAtUtc           DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_SignalQualityEvent_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy              NVARCHAR(100) NOT NULL,

CONSTRAINT PK_Instrumentation_SignalQualityEvent PRIMARY KEY (SignalQualityEventId),
CONSTRAINT FK_Instrumentation_SignalQualityEvent_Signal
    FOREIGN KEY (SignalId) REFERENCES Instrumentation.Signal(SignalId),
CONSTRAINT FK_Instrumentation_SignalQualityEvent_SignalQuality
    FOREIGN KEY (SignalQualityId) REFERENCES Instrumentation.SignalQuality(SignalQualityId),
CONSTRAINT CK_Instrumentation_SignalQualityEvent_Time CHECK (EndedAtUtc IS NULL OR EndedAtUtc > StartedAtUtc)
);
GO

CREATE TABLE Instrumentation.DataGap (
    DataGapId             BIGINT IDENTITY(1,1) NOT NULL,
    SignalId              INT NOT NULL,
    GapStartUtc           DATETIME2(7) NOT NULL,
    GapEndUtc             DATETIME2(7) NOT NULL,
    DetectedAtUtc         DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_DataGap_DetectedAtUtc DEFAULT SYSUTCDATETIME(),
    ResolvedAtUtc         DATETIME2(7) NULL,
    Notes                  NVARCHAR(1000) NULL,

CONSTRAINT PK_Instrumentation_DataGap PRIMARY KEY (DataGapId),
CONSTRAINT FK_Instrumentation_DataGap_Signal
    FOREIGN KEY (SignalId) REFERENCES Instrumentation.Signal(SignalId),
CONSTRAINT CK_Instrumentation_DataGap_Time CHECK (GapEndUtc > GapStartUtc)
);
GO

CREATE TABLE Instrumentation.HistorianRetentionPolicy (
    HistorianRetentionPolicyId INT IDENTITY(1,1) NOT NULL,
    HistorianRetentionClassId  INT NOT NULL,
    RawRetentionDays           INT NOT NULL,
    AggregateRetentionDays     INT NOT NULL,
    AggregationWindowSeconds   INT NULL,
    IsActive                   BIT NOT NULL CONSTRAINT DF_Instrumentation_HistorianRetentionPolicy_IsActive DEFAULT 1,
    CreatedAtUtc               DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_HistorianRetentionPolicy_CreatedAtUtc
DEFAULT SYSUTCDATETIME(),
    CreatedBy                  NVARCHAR(100) NOT NULL,
    ModifiedAtUtc              DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_HistorianRetentionPolicy_ModifiedAtUtc
DEFAULT SYSUTCDATETIME(),
    ModifiedBy                  NVARCHAR(100) NOT NULL,
    RowVersion                  ROWVERSION NOT NULL,

CONSTRAINT PK_Instrumentation_HistorianRetentionPolicy PRIMARY KEY (HistorianRetentionPolicyId),
CONSTRAINT UQ_Instrumentation_HistorianRetentionPolicy_Class UNIQUE (HistorianRetentionClassId),
CONSTRAINT FK_Instrumentation_HistorianRetentionPolicy_HistorianRetentionClass
    FOREIGN KEY (HistorianRetentionClassId) REFERENCES Instrumentation.HistorianRetentionClass(HistorianRetentionClassId),
CONSTRAINT CK_Instrumentation_HistorianRetentionPolicy_Days CHECK (RawRetentionDays >= 0 AND AggregateRetentionDays >= 0)
);
GO

CREATE TABLE Instrumentation.HistorianImportBatch (
    HistorianImportBatchId BIGINT IDENTITY(1,1) NOT NULL,
    MeasurementSourceId    INT NOT NULL,
    CreatedByUserId         INT NULL,
    SourceName              NVARCHAR(250) NOT NULL,
    StartedAtUtc            DATETIME2(7) NOT NULL,
    CompletedAtUtc          DATETIME2(7) NULL,
    RowCount                BIGINT NOT NULL CONSTRAINT DF_Instrumentation_HistorianImportBatch_RowCount DEFAULT 0,
    Notes                   NVARCHAR(1000) NULL,

CONSTRAINT PK_Instrumentation_HistorianImportBatch PRIMARY KEY (HistorianImportBatchId),
CONSTRAINT FK_Instrumentation_HistorianImportBatch_MeasurementSource
    FOREIGN KEY (MeasurementSourceId) REFERENCES Instrumentation.MeasurementSource(MeasurementSourceId),
CONSTRAINT FK_Instrumentation_HistorianImportBatch_CreatedByUser
    FOREIGN KEY (CreatedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
CONSTRAINT CK_Instrumentation_HistorianImportBatch_Time CHECK (CompletedAtUtc IS NULL OR CompletedAtUtc >= StartedAtUtc)
);
GO

CREATE TABLE Instrumentation.HistorianImportBatchItem (

```

```

HistorianImportBatchItemId BIGINT IDENTITY(1,1) NOT NULL,
HistorianImportBatchId    BIGINT NOT NULL,
SignalId                  INT NOT NULL,
FirstTimestampUtc         DATETIME2(7) NULL,
LastTimestampUtc         DATETIME2(7) NULL,
RowCount                  BIGINT NOT NULL CONSTRAINT DF_Instrumentation_HistorianImportBatchItem_RowCount DEFAULT 0,
RejectedCount             BIGINT NOT NULL CONSTRAINT DF_Instrumentation_HistorianImportBatchItem_RejectedCount DEFAULT 0,

CONSTRAINT PK_Instrumentation_HistorianImportBatchItem PRIMARY KEY (HistorianImportBatchItemId),
CONSTRAINT FK_Instrumentation_HistorianImportBatchItem_HistorianImportBatch
    FOREIGN KEY (HistorianImportBatchId) REFERENCES Instrumentation.HistorianImportBatch(HistorianImportBatchId),
CONSTRAINT FK_Instrumentation_HistorianImportBatchItem_Signal
    FOREIGN KEY (SignalId) REFERENCES Instrumentation.Signal(SignalId)
);
GO

CREATE TABLE Instrumentation.HistorianBackfillJob (
    HistorianBackfillJobId BIGINT IDENTITY(1,1) NOT NULL,
    SignalId               INT NULL,
    BackfillJobStatusId    INT NOT NULL,
    RequestedByUserId      INT NULL,
    RequestedAtUtc         DATETIME2(7) NOT NULL CONSTRAINT DF_Instrumentation_HistorianBackfillJob_RequestedAtUtc DEFAULT
SYSUTCDATETIME(),
    WindowStartUtc        DATETIME2(7) NOT NULL,
    WindowEndUtc          DATETIME2(7) NOT NULL,
    StartedAtUtc           DATETIME2(7) NULL,
    CompletedAtUtc         DATETIME2(7) NULL,
    Notes                  NVARCHAR(1000) NULL,

CONSTRAINT PK_Instrumentation_HistorianBackfillJob PRIMARY KEY (HistorianBackfillJobId),
CONSTRAINT FK_Instrumentation_HistorianBackfillJob_Signal
    FOREIGN KEY (SignalId) REFERENCES Instrumentation.Signal(SignalId),
CONSTRAINT FK_Instrumentation_HistorianBackfillJob_BackfillJobStatus
    FOREIGN KEY (BackfillJobStatusId) REFERENCES Instrumentation.BackfillJobStatus(BackfillJobStatusId),
CONSTRAINT FK_Instrumentation_HistorianBackfillJob_RequestedByUser
    FOREIGN KEY (RequestedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
CONSTRAINT CK_Instrumentation_HistorianBackfillJob_Window CHECK (WindowEndUtc > WindowStartUtc)
);
GO

```

C.5.5 Indexes and seed examples

Signal lookup is tag-first, while historian reads are signal-and-time-first. The hot table therefore uses a descending timestamp index per signal and can optionally receive a columnstore index when volume justifies it.

```

CREATE INDEX IX_Instrumentation_DataAcquisitionNode_UnitId ON Instrumentation.DataAcquisitionNode(UnitId);
CREATE INDEX IX_Instrumentation_Instrument_UnitId ON Instrumentation.Instrument(UnitId);
CREATE INDEX IX_Instrumentation_Instrument_EquipmentId ON Instrumentation.Instrument(EquipmentId);
CREATE INDEX IX_Instrumentation_Sensor_InstrumentId ON Instrumentation.Sensor(InstrumentId);
CREATE INDEX IX_Instrumentation_SensorChannel_SensorId ON Instrumentation.SensorChannel(SensorId);
CREATE INDEX IX_Instrumentation_Signal_UnitId ON Instrumentation.Signal(UnitId);
CREATE INDEX IX_Instrumentation_Signal_EquipmentId ON Instrumentation.Signal(EquipmentId) WHERE EquipmentId IS NOT NULL;
CREATE INDEX IX_Instrumentation_Signal_PlantSystemId ON Instrumentation.Signal(PlantSystemId) WHERE PlantSystemId IS NOT NULL;
CREATE INDEX IX_Instrumentation_Signal_Category_Role ON Instrumentation.Signal(SignalCategoryId, SignalRoleId);
CREATE INDEX IX_Instrumentation_SignalGroupMember_SignalId ON Instrumentation.SignalGroupMember(SignalId);
CREATE INDEX IX_Instrumentation_SignalMapping_AcquisitionPointId ON Instrumentation.SignalMapping(AcquisitionPointId);
CREATE INDEX IX_Instrumentation_SignalDependency_TargetSignalId ON Instrumentation.SignalDependency(TargetSignalId);
CREATE INDEX IX_Instrumentation_SignalLineage_SourceSignalId ON Instrumentation.SignalLineage(SourceSignalId);
CREATE INDEX IX_Instrumentation_SignalLimit_SignalId ON Instrumentation.SignalLimit(SignalId);
CREATE INDEX IX_Instrumentation_CalibrationRecord_Plan_PerformedAt ON Instrumentation.CalibrationRecord(CalibrationPlanId,
PerformedAtUtc DESC);

-- Hot historian access path: one signal over time.
CREATE INDEX IX_Instrumentation_Measurement_Signal_Timestamp
ON Instrumentation.Measurement(SignalId, TimestampUtc DESC)
INCLUDE (NumericValue, SignalQualityId, MeasurementSourceId);

-- Optional for large demonstrator builds: keep after initial load validation.
-- CREATE CLUSTERED COLUMNSTORE INDEX CCI_Instrumentation_Measurement
-- ON Instrumentation.Measurement;

CREATE INDEX IX_Instrumentation_MeasurementAggregate_Signal_Window
ON Instrumentation.MeasurementAggregate(SignalId, WindowStartUtc DESC, WindowSeconds);
CREATE INDEX IX_Instrumentation_SignalQualityEvent_Signal_Time
ON Instrumentation.SignalQualityEvent(SignalId, StartedAtUtc DESC, EndedAtUtc);
CREATE INDEX IX_Instrumentation_DataGap_Signal_Time
ON Instrumentation.DataGap(SignalId, GapStartUtc DESC, GapEndUtc);
GO

```

C.5.5.1 Minimal seed examples

```
INSERT INTO Instrumentation.SignalQuality (Code, Name, Description, DisplayOrder)
VALUES
('GOOD','Good','Value passed validation and is usable.',1),
('UNCERTAIN','Uncertain','Value exists but quality is reduced.',2),
('BAD','Bad','Value failed quality validation.',3),
('STALE','Stale','Value has not refreshed within the expected interval.',4),
('SUBSTITUTED','Substituted','Value was replaced by a fallback or estimate.',5),
('SIMULATED','Simulated','Value originates from the demonstrator or simulator.',6);
GO

INSERT INTO Instrumentation.SignalType (Code, Name, DisplayOrder)
VALUES
('MEASURED','Measured',1),
('CALCULATED','Calculated',2),
('COMMAND','Command indication',3),
('STATE','State',4),
('SIMULATED','Simulated',5);
GO

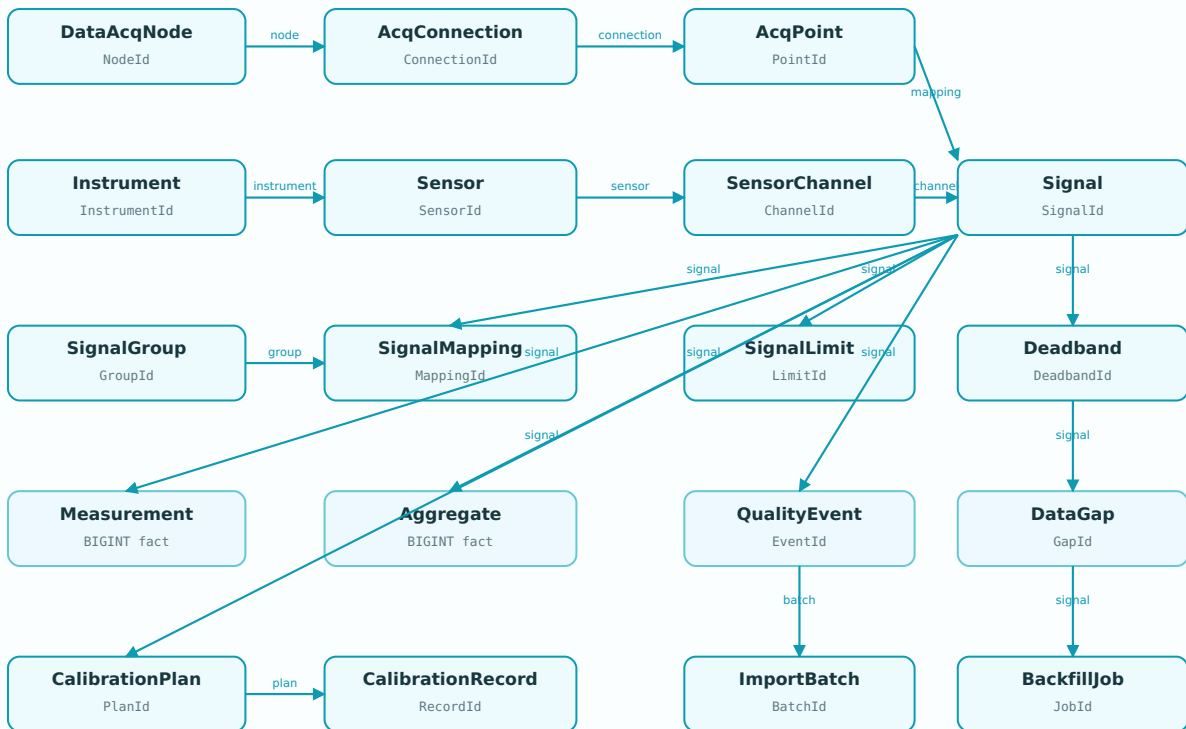
INSERT INTO Instrumentation.SignalCategory (Code, Name, DisplayOrder)
VALUES
('POWER','Power',1),('TEMPERATURE','Temperature',2),('PRESSURE','Pressure',3),
('FLOW','Flow',4),('RADIATION','Radiation',5),('VIBRATION','Vibration',6),('ELECTRICAL','Electrical',7);
GO

INSERT INTO Instrumentation.ThresholdType (Code, Name, DisplayOrder)
VALUES
('LOW_LOW','Low-low',1),('LOW','Low',2),('HIGH','High',3),('HIGH_HIGH','High-high',4),
('RATE','Rate of change',5),('DEVIATION','Deviation',6);
GO
```

C.5.6 ER diagram

The sector has three main flows: acquisition path, physical sensor path, and historian path. They meet at `Instrumentation.Signal`.

Instrumentation sector ER diagram



Lookup tables are omitted for readability. Measurement is the main high-volume fact table.

Figure C.5.1 - Instrumentation sector ER diagram. Lookup tables are omitted from the drawing for readability; they are included in the DDL and FK mapping.

C.5.7 Foreign-key mapping

C.5.7.1 Internal Instrumentation foreign keys

FROM TABLE	FK COLUMN	TO TABLE	PK COLUMN	RELATIONSHIP
DataAcquisitionNode	ChannelStatusId	ChannelStatus	ChannelStatusId	many nodes per channel status
AcquisitionConnection	DataAcquisitionNodeId	DataAcquisitionNode	DataAcquisitionNodeId	many connections per acquisition node
AcquisitionConnection	ChannelStatusId	ChannelStatus	ChannelStatusId	many connections per status
AcquisitionPoint	AcquisitionConnectionId	AcquisitionConnection	AcquisitionConnectionId	many raw points per connection
Instrument	SensorStatusId	SensorStatus	SensorStatusId	many instruments per status
Sensor	InstrumentId	Instrument	InstrumentId	many sensors per instrument
Sensor	SensorTypeId	SensorType	SensorTypeId	many sensors per type
Sensor	SensorStatusId	SensorStatus	SensorStatusId	many sensors per status
SensorChannel	SensorId	Sensor	SensorId	many channels per sensor
SensorChannel	ChannelStatusId	ChannelStatus	ChannelStatusId	many channels per status
Signal	SensorChannelId	SensorChannel	SensorChannelId	many signals per channel, nullable
Signal	SignalTypeId	SignalType	SignalTypeId	many signals per type
Signal	SignalCategoryId	SignalCategory	SignalCategoryId	many signals per category
Signal	SignalRoleId	SignalRole	SignalRoleId	many signals per role
Signal	SamplingModeId	SamplingMode	SamplingModeId	many signals per sampling mode
Signal	HistorianRetentionClassId	HistorianRetentionClasses	HistorianRetentionClassId	many signals per retention class
SignalAlias	SignalId	Signal	SignalId	many aliases per signal
SignalGroupMember	SignalGroupId	SignalGroup	SignalGroupId	many members per group
SignalGroupMember	SignalId	Signal	SignalId	many groups per signal

FROM TABLE	FK COLUMN	TO TABLE	PK COLUMN	RELATIONSHIP
SignalMapping	SignalId	Signal	SignalId	many mappings per signal over time
SignalMapping	AcquisitionPointId	AcquisitionPoint	AcquisitionPointId	many mappings per acquisition point over time
SignalDependency	SourceSignalId	Signal	SignalId	source signal
SignalDependency	TargetSignalId	Signal	SignalId	target signal
SignalLineage	DerivedSignalId	Signal	SignalId	derived signal
SignalLineage	SourceSignalId	Signal	SignalId	source signal
SignalLineage	LineageRoleId	LineageRole	LineageRoleId	role of source signal
SignalLimit	SignalId	Signal	SignalId	many limits per signal
SignalLimit	ThresholdTypeId	ThresholdType	ThresholdTypeId	many limits per threshold type
SignalDeadband	SignalId	Signal	SignalId	many deadband policies per signal
CalibrationPlan	SignalId	Signal	SignalId	one active plan per signal
CalibrationPlan	SensorId	Sensor	SensorId	optional sensor under calibration
CalibrationPlan	CalibrationStatusId	CalibrationStatus	CalibrationStatusId	many plans per calibration status
CalibrationRecord	CalibrationPlanId	CalibrationPlan	CalibrationPlanId	many records per plan
Measurement	SignalId	Signal	SignalId	many measurements per signal
Measurement	SignalQualityId	SignalQuality	SignalQualityId	many measurements per quality
Measurement	MeasurementSourceId	MeasurementSource	MeasurementSourceId	many measurements per source
MeasurementAggregate	SignalId	Signal	SignalId	many aggregate windows per signal
SignalQualityEvent	SignalId	Signal	SignalId	many quality intervals per signal
DataGap	SignalId	Signal	SignalId	many detected gaps per signal
HistorianRetentionPolicy	HistorianRetentionClassId	HistorianRetentionClasses	HistorianRetentionClassId	one policy per class
HistorianImportBatchItem	HistorianImportBatchId	HistorianImportBatch	HistorianImportBatchId	many items per import batch
HistorianImportBatchItem	SignalId	Signal	SignalId	many batch items per signal

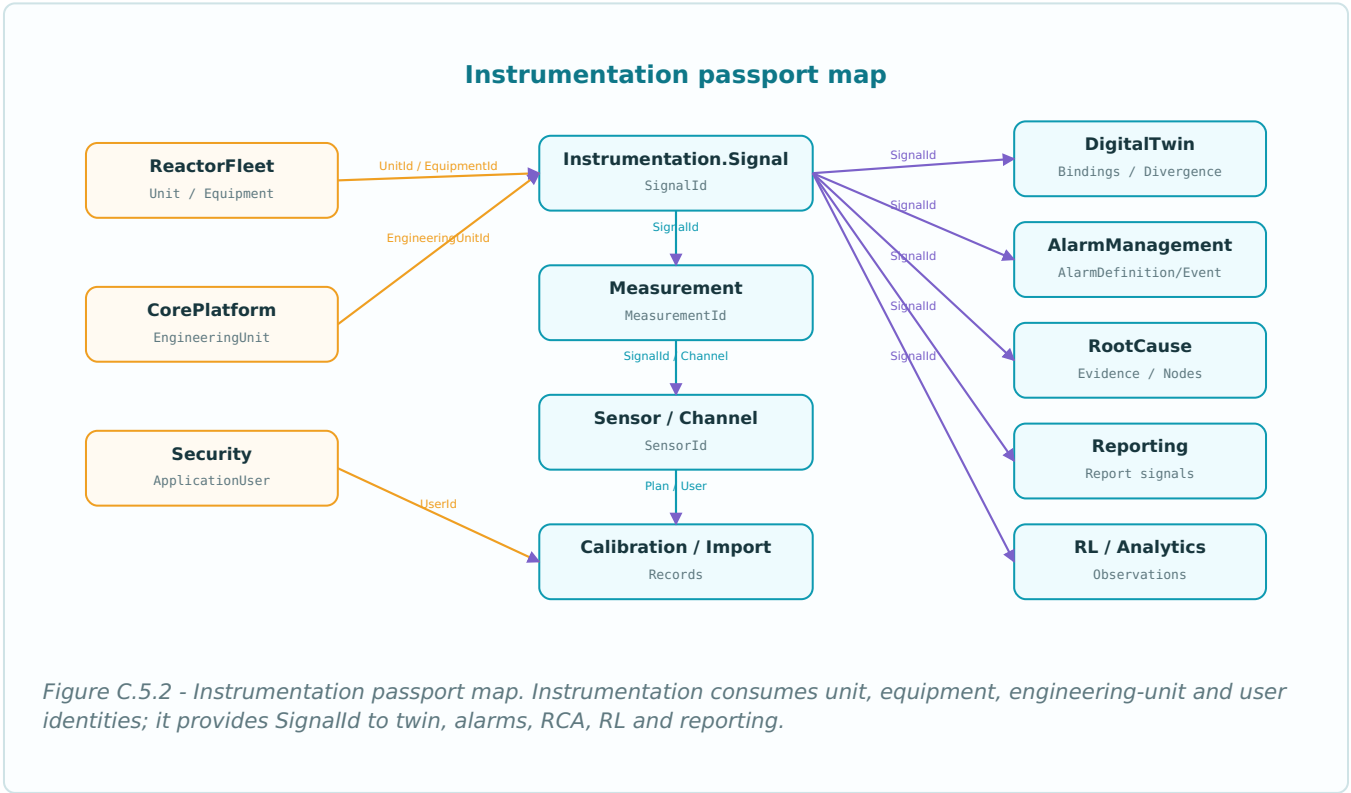
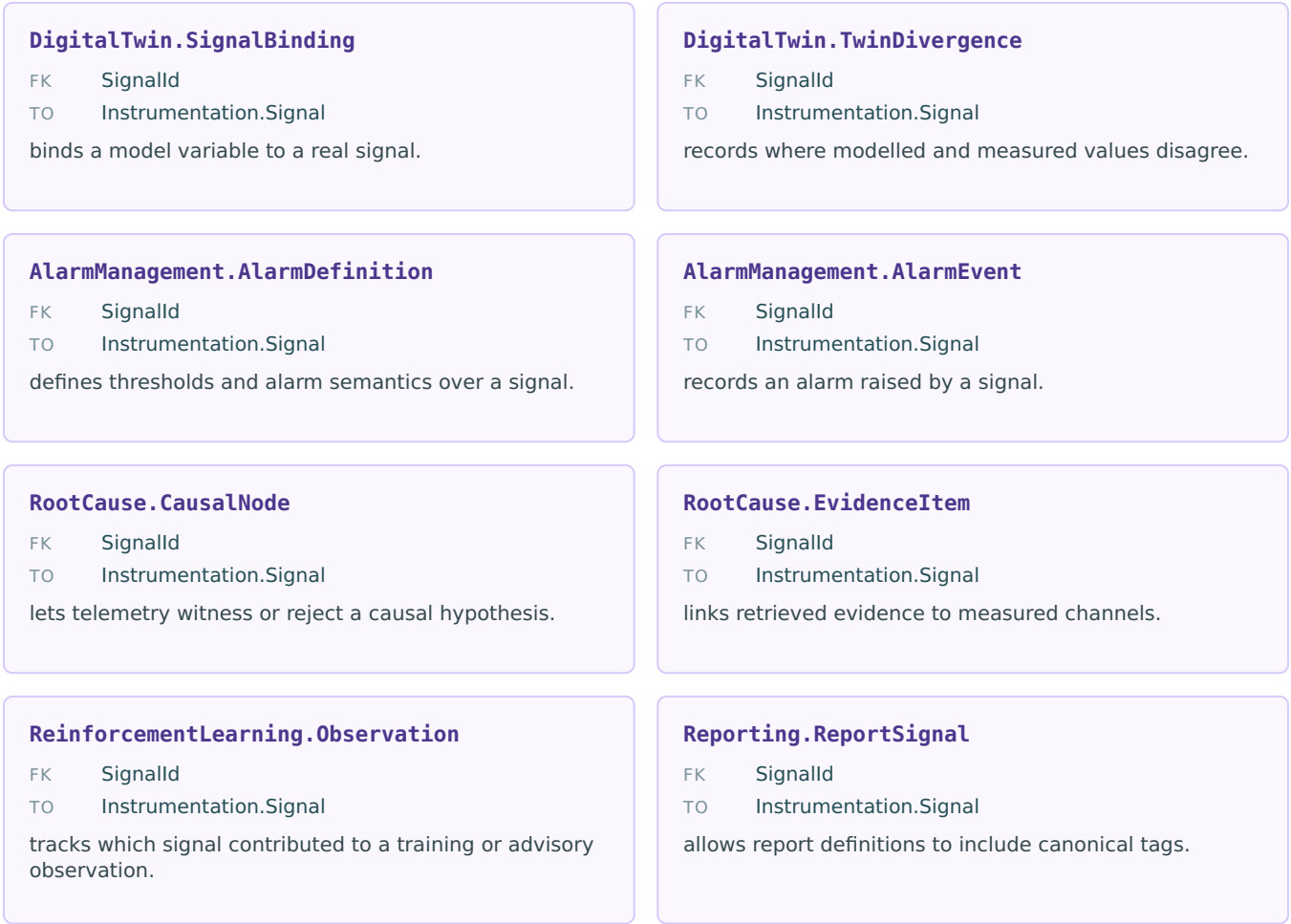
FROM TABLE	FK COLUMN	TO TABLE	PK COLUMN	RELATIONSHIP
HistorianBackfillJob	SignalId	Signal	SignalId	optional signal under backfill
HistorianBackfillJob	BackfillJobStatusId	BackfillJobStatus	BackfillJobStatusId	many jobs per status

C.5.7.2 Passports consumed from previous sectors

FROM TABLE	FK COLUMN	TO TABLE	PK COLUMN	RELATIONSHIP
DataAcquisitionNode	UnitId	ReactorFleet.Unit	UnitId	many acquisition nodes per unit
Instrument	UnitId	ReactorFleet.Unit	UnitId	many instruments per unit
Instrument	EquipmentId	ReactorFleet.Equipment	EquipmentId	optional instrument-to-component binding
Instrument	EquipmentLocationId	ReactorFleet.EquipmentLocation	EquipmentLocationId	optional location binding
Sensor	EngineeringUnitId	CorePlatform.EngineeringUnit	EngineeringUnitId	canonical sensor range unit
Signal	UnitId	ReactorFleet.Unit	UnitId	many signals per unit
Signal	EquipmentId	ReactorFleet.Equipment	EquipmentId	optional signal-to-component binding
Signal	PlantSystemId	ReactorFleet.PlantSystem	PlantSystemId	optional signal-to-system binding
Signal	EngineeringUnitId	CorePlatform.EngineeringUnit	EngineeringUnitId	canonical signal unit
CalibrationRecord	PerformedByUserId	Security.ApplicationUser	ApplicationUserId	optional user who performed calibration
MeasurementAnnotation	CreatedByUserId	Security.ApplicationUser	ApplicationUserId	optional user who annotated value
HistorianImportBatch	CreatedByUserId	Security.ApplicationUser	ApplicationUserId	optional user who started import
HistorianBackfillJob	RequestedByUserId	Security.ApplicationUser	ApplicationUserId	optional user who requested backfill

C.5.7.3 Incoming passports from later sectors

The following relationships are declared in later sectors. They are listed here because `Instrumentation.Signal` is the shared signal identity used by the rest of NEXUS-1.



C.5.8 Useful verification queries

These queries validate that a tag can be traced to its unit, unit of measure, source, quality and acquisition path.

```
-- 1. Find all active historized signals for one unit.
SELECT s.Tag, s.Name, c.Code AS category, u.Symbol AS unit_symbol, r.Code AS retention
FROM Instrumentation.Signal AS s
JOIN Instrumentation.SignalCategory AS c ON c.SignalCategoryId = s.SignalCategoryId
JOIN CorePlatform.EngineeringUnit AS u ON u.EngineeringUnitId = s.EngineeringUnitId
JOIN Instrumentation.HistorianRetentionClass AS r ON r.HistorianRetentionClassId = s.HistorianRetentionClassId
JOIN ReactorFleet.Unit AS ru ON ru.UnitId = s.UnitId
WHERE ru.Code = 'NX1-U1' AND s.IsHistorized = 1 AND s.IsDeleted = 0
ORDER BY s.Tag;

-- 2. Pull the latest ten values for a tag.
DECLARE @tag NVARCHAR(80) = N'NX1-U1.RX.POWER';
SELECT TOP (10) m.TimestampUtc, m.NumericValue, q.Code AS quality, src.Code AS source
FROM Instrumentation.Measurement AS m
JOIN Instrumentation.Signal AS s ON s.SignalId = m.SignalId
JOIN Instrumentation.SignalQuality AS q ON q.SignalQualityId = m.SignalQualityId
JOIN Instrumentation.MeasurementSource AS src ON src.MeasurementSourceId = m.MeasurementSourceId
WHERE s.Tag = @tag
ORDER BY m.TimestampUtc DESC;

-- 3. Find stale or bad signals in the current unit.
SELECT s.Tag, q.Code AS quality, e.StartedAtUtc, e.EndedAtUtc, e.ReasonCode
FROM Instrumentation.SignalQualityEvent AS e
JOIN Instrumentation.Signal AS s ON s.SignalId = e.SignalId
JOIN Instrumentation.SignalQuality AS q ON q.SignalQualityId = e.SignalQualityId
WHERE e.EndedAtUtc IS NULL AND q.Code IN ('BAD', 'STALE', 'UNCERTAIN')
ORDER BY e.StartedAtUtc DESC;

-- 4. Show acquisition path from tag to raw point.
SELECT s.Tag, n.Code AS node_code, c.Protocol, c.Endpoint, p.RawAddress
FROM Instrumentation.Signal AS s
JOIN Instrumentation.SignalMapping AS sm ON sm.SignalId = s.SignalId AND sm.EffectiveToUtc IS NULL
JOIN Instrumentation.AcquisitionPoint AS p ON p.AcquisitionPointId = sm.AcquisitionPointId
JOIN Instrumentation.AcquisitionConnection AS c ON c.AcquisitionConnectionId = p.AcquisitionConnectionId
JOIN Instrumentation.DataAcquisitionNode AS n ON n.DataAcquisitionNodeId = c.DataAcquisitionNodeId
WHERE s.Tag = 'NX1-U1.RX.POWER';
```

C.5.9 Boundary and next sector

Honest boundary. This schema is a demonstrator-grade instrumentation and historian model. It is not a certified I&C database, not a safety-class historian, and not a representation of any real facility. The design deliberately records quality and gaps because a value without quality is not evidence.

The next sector is [DigitalTwin](#). It will bind physical units from [ReactorFleet](#) and measured signals from [Instrumentation](#) into twin models, snapshots and divergence records.

Appendix C.6

DigitalTwin Schema

The DigitalTwin sector binds three things that earlier sectors keep separate: the physical unit, the signals that measure it, and the model that represents it.

The sector depends outward on `ReactorFleet.Unit` for the thing mirrored and on `Instrumentation.Signal` for the measured reality that feeds and judges the model. Nothing points backward from the plant into the twin. The direction of dependency is deliberate: the twin depends on the plant and the instruments, not the reverse.

A digital twin is not only a dashboard. It is a model with a conscience: it stores where its picture of the world agrees with telemetry, and it records where it has drifted.

C.6.1 Sector purpose	C.6.2 Full table list
C.6.3 Lookup tables	C.6.4 SQL DDL
C.6.5 Indexes and seed examples	C.6.6 ER diagram
C.6.7 Foreign-key mapping	C.6.8 Boundary and next sector

C.6.1 Sector purpose

`DigitalTwin` is the keystone sector. It consumes the physical hierarchy from `ReactorFleet`, the signal registry from `Instrumentation`, engineering units from `CorePlatform`, and user identity from `Security`. It provides model, snapshot, divergence and simulation records to the later reasoning and governance sectors.

- `TwinModel` names the physical unit mirrored by the model.
- `SignalBinding` wires a model variable to a real signal.
- `TwinSnapshot` freezes the model state at a timestamp.
- `TwinDivergence` records where modeled value and measured value disagree.
- `SimulationRun` and `WhatIfCase` make the twin usable for training, replay and analysis.

Design choice. The divergence table is not an error log. It is a first-class data object. A twin that hides disagreement becomes unsafe to trust; a twin that records disagreement can be corrected, audited and used honestly.

C.6.2 Full table list

The DigitalTwin sector contains forty-two tables: fifteen typed lookup tables and twenty-seven substantive tables. The model itself is compact, but the surrounding runtime, validation and simulation records make the sector auditable.

LOOKUP

DigitalTwin.TwinModelType

PK TwinModelTypeId
FK None

Classifies a model as physics surrogate, point-kinetics model, thermal-hydraulic approximation, equipment health model, or visualization model.

LOOKUP

DigitalTwin.TwinModelStatus

PK TwinModelStatusId
FK None

Lifecycle of a twin model: draft, validating, active, retired, superseded, failed.

LOOKUP

DigitalTwin.TwinFidelityLevel

PK TwinFidelityLevelId
FK None

Declared fidelity band: illustrative, training, shadow, advisory-ready, validated.

LOOKUP

DigitalTwin.ModelVariableType

PK ModelVariableTypeId
FK None

Classifies model variables: state, input, output, derived, diagnostic, safety-margin.

LOOKUP

DigitalTwin.ModelParameterType

PK ModelParameterTypeId
FK None

Classifies model parameters: constant, calibration, solver, bound, tuning, assumption.

LOOKUP

DigitalTwin.BindingRole

PK BindingRoleId
FK None

Role of a signal binding: input, output witness, calibration witness, divergence witness, fallback.

LOOKUP

DigitalTwin.BindingStatus

PK BindingStatusId
FK None

Binding lifecycle: proposed, active, disabled, stale, rejected.

LOOKUP

DigitalTwin.SnapshotReason

PK SnapshotReasonId
FK None

Why a snapshot was captured: scheduled, alarm, manual, simulation, divergence, training.

LOOKUP

DigitalTwin.DivergenceSeverity

PK DivergenceSeverityId
FK None

Severity of model-vs-reality mismatch: info, warning, critical, invalid.

LOOKUP

DigitalTwin.DivergenceStatus

PK DivergenceStatusId
FK None

Review state of a divergence: open, acknowledged, explained, corrected, waived, closed.

LOOKUP

DigitalTwin.SimulationScenarioType

PK SimulationScenarioTypeId
FK None

Scenario family: what-if, replay, training, validation, regression, RL environment.

LOOKUP

DigitalTwin.SimulationRunStatus

PK SimulationRunStatusId
FK None

Run status: queued, running, completed, failed, cancelled, invalidated.

LOOKUP

DigitalTwin.SynchronizationStatus

PK SynchronizationStatusId
FK None

Synchronization health: locked, drift, diverging, stale, offline.

LOOKUP

DigitalTwin.ValidationStatus

PK ValidationStatusId
FK None

Validation state: unvalidated, testing, passed, failed, expired, waived.

LOOKUP

DigitalTwin.SolverType

PK SolverTypeId
FK None

Numerical solver or update strategy used by a model version.

MODEL

DigitalTwin.TwinModel

PK TwinModelId
FK UnitId, TwinModelTypeId, TwinModelStatusId

Names the physical unit mirrored by the twin and declares its model type, status and fidelity.

MODEL

DigitalTwin.TwinModelVersion

PK TwinModelVersionId
FK TwinModelId, SolverTypeId, ValidationStatusId

Versioned model implementation, solver configuration, code reference, hash and validation state.

MODEL

DigitalTwin.TwinModelComponent

PK TwinModelComponentId
FK TwinModelId, EquipmentId, PlantSystemId

Maps model components to physical equipment and plant systems where that relationship exists.

MODEL**DigitalTwin.TwinVariable**

PK TwinVariableId

FK TwinModelId, EngineeringUnitId

Variable catalogue for model state, inputs, outputs and diagnostics.

MODEL**DigitalTwin.TwinParameter**

PK TwinParameterId

FK TwinModelVersionId, EngineeringUnitId

Versioned constants, bounds, solver parameters and calibration parameters.

BINDING**DigitalTwin.SignalBinding**

PK SignalBindingId

FK TwinModelId, TwinVariableId, SignalId

Wires a model variable to a real instrumentation signal.

BINDING**DigitalTwin.SignalBindingCalibration**

PK SignalBindingCalibrationId

FK SignalBindingId, CalibratedByUserId

Scale, offset, tolerance and calibration provenance for a binding.

RUNTIME**DigitalTwin.TwinRuntimeSession**

PK TwinRuntimeSessionId

FK TwinModelVersionId, StartedByUserId

Runtime instance of a twin model version, including mode, host and time window.

RUNTIME**DigitalTwin.TwinSynchronization**

PK TwinSynchronizationId

FK TwinRuntimeSessionId, SynchronizationStatusId

Records synchronization state between model and telemetry for one runtime session.

RUNTIME**DigitalTwin.TwinStateVector**

PK TwinStateVectorId

FK TwinRuntimeSessionId

Serialized state vector used to restart, replay or inspect a model runtime.

SNAPSHOT**DigitalTwin.TwinSnapshot**

PK TwinSnapshotId

FK TwinRuntimeSessionId, SnapshotReasonId

Captures the model state at one instant.

SNAPSHOT**DigitalTwin.TwinSnapshotValue**

PK TwinSnapshotValueId

FK TwinSnapshotId, TwinVariableId

One model variable value inside a snapshot.

DIVERGENCE**DigitalTwin.TwinDivergence**

PK TwinDivergenceId

FK TwinSnapshotId, SignalId

Where modelled value and measured value disagree. This is the conscience table of the sector.

DIVERGENCE**DigitalTwin.TwinDivergenceReview**

PK TwinDivergenceReviewId
FK TwinDivergenceId , ReviewedByUserId

Human or system review of a divergence: explained, corrected, waived or closed.

SIMULATION**DigitalTwin.SimulationScenario**

PK SimulationScenarioId
FK TwinModelId , CreatedByUserId

Named what-if, replay, validation or training scenario.

SIMULATION**DigitalTwin.SimulationScenarioInput**

PK SimulationScenarioInputId
FK SimulationScenarioId , TwinVariableId

Input variable, initial condition or forcing term for a scenario.

SIMULATION**DigitalTwin.SimulationRun**

PK SimulationRunId
FK SimulationScenarioId , TwinModelVersionId

Concrete execution of a scenario against a model version.

SIMULATION**DigitalTwin.SimulationRunStep**

PK SimulationRunStepId
FK SimulationRunId

Time-step record for replay, validation and training traces.

SIMULATION**DigitalTwin.SimulationRunOutput**

PK SimulationRunOutputId
FK SimulationRunStepId , TwinVariableId

Output variable values emitted by a simulation step.

WHAT-IF**DigitalTwin.WhatIfCase**

PK WhatIfCaseId
FK TwinModelId , RequestedByUserId

A named operator or engineer what-if analysis case.

WHAT-IF**DigitalTwin.WhatIfCaseInput**

PK WhatIfCaseInputId
FK WhatIfCaseId , TwinVariableId

Input override or perturbation supplied to a what-if case.

WHAT-IF**DigitalTwin.WhatIfCaseResult**

PK WhatIfCaseResultId
FK WhatIfCaseId , SimulationRunId

Summarized result and conclusion of a what-if case.

VALIDATION**DigitalTwin.TwinHealthCheck**

PK TwinHealthCheckId
FK TwinRuntimeSessionId ,
SynchronizationStatusId

Periodic operational health check of the running twin.

VALIDATION**DigitalTwin.TwinModelValidation**

PK TwinModelValidationId
FK TwinModelVersionId , ValidationStatusId

Validation campaign for a model version.

VALIDATION**DigitalTwin.TwinValidationMetric**

PK TwinValidationMetricId
FK TwinModelValidationId , TwinVariableId

Metric result such as RMSE, max error, Brier score, coverage or timing error.

GOVERNANCE

DigitalTwin.ModelAssumption

PK ModelAssumptionId
 FK TwinModelVersionId

Versioned assumption, simplification or validity boundary.

GOVERNANCE

DigitalTwin.TwinAnnotation

PK TwinAnnotationId
 FK TwinModelId , CreatedByUserId

General annotation attached to a model, snapshot, divergence or simulation run.

C.6.3 Lookup tables

Lookup values express the lifecycle and epistemic state of the twin: model type, fidelity, binding status, divergence severity, synchronization status and validation status.

LOOKUP TABLE	TYPICAL SEED CODES	MAIN CONSUMER
TwinModelType	POINT_KINETICS , THERMAL_HYDRAULIC , EQUIPMENT_HEALTH , VISUALIZATION	Referenced by TwinModel .
TwinModelStatus	DRAFT , VALIDATING , ACTIVE , RETIRED , SUPERSEDED	Referenced by TwinModel .
TwinFidelityLevel	ILLUSTRATIVE , TRAINING , SHADOW , ADVISORY_READY , VALIDATED	Referenced by TwinModel .
ModelVariableType	STATE , INPUT , OUTPUT , DERIVED , DIAGNOSTIC	Referenced by TwinVariable .
ModelParameterType	CONSTANT , CALIBRATION , SOLVER , BOUND , ASSUMPTION	Referenced by TwinParameter .
BindingRole	INPUT , OUTPUT_WITNESS , CALIBRATION , DIVERGENCE_WITNESS , FALLBACK	Referenced by SignalBinding .
BindingStatus	PROPOSED , ACTIVE , DISABLED , STALE , REJECTED	Referenced by SignalBinding .
SnapshotReason	SCHEDULED , ALARM , MANUAL , SIMULATION , DIVERGENCE , TRAINING	Referenced by TwinSnapshot .
DivergenceSeverity	INFO , WARNING , CRITICAL , INVALID	Referenced by TwinDivergence .
DivergenceStatus	OPEN , ACKNOWLEDGED , EXPLAINED , CORRECTED , WAIVED , CLOSED	Referenced by TwinDivergence .
SimulationScenarioType	WHAT_IF , REPLAY , TRAINING , VALIDATION , REGRESSION , RL_ENVIRONMENT	Referenced by SimulationScenario .
SimulationRunStatus	QUEUED , RUNNING , COMPLETED , FAILED , CANCELLED , INVALIDATED	Referenced by SimulationRun .
SynchronizationStatus	LOCKED , DRIFT , DIVERGING , STALE , OFFLINE	Referenced by synchronization and health-check tables.
ValidationStatus	UNVALIDATED , TESTING , PASSED , FAILED , EXPIRED , WAIVED	Referenced by TwinModelVersion and validations.

LOOKUP TABLE	TYPICAL SEED CODES	MAIN CONSUMER
SolverType	EULER , RK4 , STIFF_SOLVER , DISCRETE_STEP , SURROGATE	Referenced by TwinModelVersion .

C.6.4 SQL DDL

C.6.4.1 Schema

```
IF SCHEMA ID(N'DigitalTwin') IS NULL
EXEC(N'CREATE SCHEMA DigitalTwin');
GO
```

C.6.4.2 DigitalTwin lookup tables

```
CREATE TABLE DigitalTwin.TwinModelType (
    TwinModelTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_DigitalTwin_TwinModelType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_DigitalTwin_TwinModelType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_DigitalTwin_TwinModelType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_DigitalTwin_TwinModelType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_DigitalTwin_TwinModelType
        PRIMARY KEY (TwinModelTypeId),
    CONSTRAINT UQ_DigitalTwin_TwinModelType_Code
        UNIQUE (Code)
);
GO

CREATE TABLE DigitalTwin.TwinModelStatus (
    TwinModelStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_DigitalTwin_TwinModelStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_DigitalTwin_TwinModelStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_DigitalTwin_TwinModelStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_DigitalTwin_TwinModelStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_DigitalTwin_TwinModelStatus
        PRIMARY KEY (TwinModelStatusId),
    CONSTRAINT UQ_DigitalTwin_TwinModelStatus_Code
        UNIQUE (Code)
);
GO

CREATE TABLE DigitalTwin.TwinFidelityLevel (
    TwinFidelityLevelId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_DigitalTwin_TwinFidelityLevel_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_DigitalTwin_TwinFidelityLevel_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_DigitalTwin_TwinFidelityLevel_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_DigitalTwin_TwinFidelityLevel_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_DigitalTwin_TwinFidelityLevel
```

```

        PRIMARY KEY (TwinFidelityLevelId),
        CONSTRAINT UQ_DigitalTwin_TwinFidelityLevel_Code
            UNIQUE (Code)
    );
GO

CREATE TABLE DigitalTwin.ModelVariableType (
    ModelVariableTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_DigitalTwin_ModelVariableType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_DigitalTwin_ModelVariableType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_DigitalTwin_ModelVariableType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_DigitalTwin_ModelVariableType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_DigitalTwin_ModelVariableType
        PRIMARY KEY (ModelVariableTypeId),
    CONSTRAINT UQ_DigitalTwin_ModelVariableType_Code
        UNIQUE (Code)
);
GO

CREATE TABLE DigitalTwin.ModelParameterType (
    ModelParameterTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_DigitalTwin_ModelParameterType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_DigitalTwin_ModelParameterType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_DigitalTwin_ModelParameterType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_DigitalTwin_ModelParameterType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_DigitalTwin_ModelParameterType
        PRIMARY KEY (ModelParameterTypeId),
    CONSTRAINT UQ_DigitalTwin_ModelParameterType_Code
        UNIQUE (Code)
);
GO

CREATE TABLE DigitalTwin.BindingRole (
    BindingRoleId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_DigitalTwin_BindingRole_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_DigitalTwin_BindingRole_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_DigitalTwin_BindingRole_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_DigitalTwin_BindingRole_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_DigitalTwin_BindingRole
        PRIMARY KEY (BindingRoleId),
    CONSTRAINT UQ_DigitalTwin_BindingRole_Code
        UNIQUE (Code)
);
GO

CREATE TABLE DigitalTwin.BindingStatus (
    BindingStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_DigitalTwin_BindingStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_DigitalTwin_BindingStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_DigitalTwin_BindingStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_DigitalTwin_BindingStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_DigitalTwin_BindingStatus
        PRIMARY KEY (BindingStatusId),
    CONSTRAINT UQ_DigitalTwin_BindingStatus_Code
        UNIQUE (Code)
);
GO

```

```

);
GO

CREATE TABLE DigitalTwin.SnapshotReason (
    SnapshotReasonId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_DigitalTwin_SnapshotReason_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_DigitalTwin_SnapshotReason_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_DigitalTwin_SnapshotReason_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_DigitalTwin_SnapshotReason_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_DigitalTwin_SnapshotReason
    PRIMARY KEY (SnapshotReasonId),
    CONSTRAINT UQ_DigitalTwin_SnapshotReason_Code
    UNIQUE (Code)
);
GO

CREATE TABLE DigitalTwin.DivergenceSeverity (
    DivergenceSeverityId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_DigitalTwin_DivergenceSeverity_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_DigitalTwin_DivergenceSeverity_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_DigitalTwin_DivergenceSeverity_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_DigitalTwin_DivergenceSeverity_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_DigitalTwin_DivergenceSeverity
    PRIMARY KEY (DivergenceSeverityId),
    CONSTRAINT UQ_DigitalTwin_DivergenceSeverity_Code
    UNIQUE (Code)
);
GO

CREATE TABLE DigitalTwin.DivergenceStatus (
    DivergenceStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_DigitalTwin_DivergenceStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_DigitalTwin_DivergenceStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_DigitalTwin_DivergenceStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_DigitalTwin_DivergenceStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_DigitalTwin_DivergenceStatus
    PRIMARY KEY (DivergenceStatusId),
    CONSTRAINT UQ_DigitalTwin_DivergenceStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE DigitalTwin.SimulationScenarioType (
    SimulationScenarioTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_DigitalTwin_SimulationScenarioType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_DigitalTwin_SimulationScenarioType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_DigitalTwin_SimulationScenarioType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_DigitalTwin_SimulationScenarioType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_DigitalTwin_SimulationScenarioType
    PRIMARY KEY (SimulationScenarioTypeId),
    CONSTRAINT UQ_DigitalTwin_SimulationScenarioType_Code
    UNIQUE (Code)
);
GO

```

```

CREATE TABLE DigitalTwin.SimulationRunStatus (
    SimulationRunStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_DigitalTwin_SimulationRunStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_DigitalTwin_SimulationRunStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_DigitalTwin_SimulationRunStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_DigitalTwin_SimulationRunStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_DigitalTwin_SimulationRunStatus
    PRIMARY KEY (SimulationRunStatusId),
    CONSTRAINT UQ_DigitalTwin_SimulationRunStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE DigitalTwin.SynchronizationStatus (
    SynchronizationStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_DigitalTwin_SynchronizationStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_DigitalTwin_SynchronizationStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_DigitalTwin_SynchronizationStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_DigitalTwin_SynchronizationStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_DigitalTwin_SynchronizationStatus
    PRIMARY KEY (SynchronizationStatusId),
    CONSTRAINT UQ_DigitalTwin_SynchronizationStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE DigitalTwin.ValidationStatus (
    ValidationStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_DigitalTwin_ValidationStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_DigitalTwin_ValidationStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_DigitalTwin_ValidationStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_DigitalTwin_ValidationStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_DigitalTwin_ValidationStatus
    PRIMARY KEY (ValidationStatusId),
    CONSTRAINT UQ_DigitalTwin_ValidationStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE DigitalTwin.SolverType (
    SolverTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_DigitalTwin_SolverType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_DigitalTwin_SolverType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_DigitalTwin_SolverType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_DigitalTwin_SolverType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_DigitalTwin_SolverType
    PRIMARY KEY (SolverTypeId),
    CONSTRAINT UQ_DigitalTwin_SolverType_Code
    UNIQUE (Code)
);
GO

```

C.6.4.3 Model catalogue, versions, components, variables and parameters

```
CREATE TABLE DigitalTwin.TwinModel (
  TwinModelId INT IDENTITY(1,1) NOT NULL,
  UnitId INT NOT NULL,
  TwinModelTypeId INT NOT NULL,
  TwinModelStatusId INT NOT NULL,
  TwinFidelityLevelId INT NOT NULL,
  Code NVARCHAR(80) NOT NULL,
  Name NVARCHAR(250) NOT NULL,
  Description NVARCHAR(1000) NULL,
  ModelOwner NVARCHAR(200) NULL,
  IsAuthoritative BIT NOT NULL CONSTRAINT DF_DigitalTwin_TwinModel_IsAuthoritative DEFAULT 0,
  CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_TwinModel_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  CreatedBy NVARCHAR(100) NOT NULL,
  ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_TwinModel_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedBy NVARCHAR(100) NOT NULL,
  IsDeleted BIT NOT NULL CONSTRAINT DF_DigitalTwin_TwinModel_IsDeleted DEFAULT 0,
  RowVersion ROWVERSION NOT NULL,

  CONSTRAINT PK_DigitalTwin_TwinModel PRIMARY KEY (TwinModelId),
  CONSTRAINT UQ_DigitalTwin_TwinModel_Unit_Code UNIQUE (UnitId, Code),
  CONSTRAINT FK_DigitalTwin_TwinModel_Unit
    FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
  CONSTRAINT FK_DigitalTwin_TwinModel_TwinModelType
    FOREIGN KEY (TwinModelTypeId) REFERENCES DigitalTwin.TwinModelType(TwinModelTypeId),
  CONSTRAINT FK_DigitalTwin_TwinModel_TwinModelStatus
    FOREIGN KEY (TwinModelStatusId) REFERENCES DigitalTwin.TwinModelStatus(TwinModelStatusId),
  CONSTRAINT FK_DigitalTwin_TwinModel_TwinFidelityLevel
    FOREIGN KEY (TwinFidelityLevelId) REFERENCES DigitalTwin.TwinFidelityLevel(TwinFidelityLevelId)
);
GO

CREATE TABLE DigitalTwin.TwinModelVersion (
  TwinModelVersionId INT IDENTITY(1,1) NOT NULL,
  TwinModelId INT NOT NULL,
  SolverTypeId INT NOT NULL,
  ValidationStatusId INT NOT NULL,
  VersionCode NVARCHAR(50) NOT NULL,
  ModelVersion NVARCHAR(80) NOT NULL,
  SourceReference NVARCHAR(500) NULL,
  ModelHash VARBINARY(32) NULL,
  ConfigurationJson NVARCHAR(MAX) NULL,
  ReleasedAtUtc DATETIME2(7) NULL,
  ApprovedByUserId INT NULL,
  CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_TwinModelVersion_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  CreatedBy NVARCHAR(100) NOT NULL,
  ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_TwinModelVersion_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
  ModifiedBy NVARCHAR(100) NOT NULL,
  IsDeleted BIT NOT NULL CONSTRAINT DF_DigitalTwin_TwinModelVersion_IsDeleted DEFAULT 0,
  RowVersion ROWVERSION NOT NULL,

  CONSTRAINT PK_DigitalTwin_TwinModelVersion PRIMARY KEY (TwinModelVersionId),
  CONSTRAINT UQ_DigitalTwin_TwinModelVersion_Model_Version UNIQUE (TwinModelId, VersionCode),
  CONSTRAINT FK_DigitalTwin_TwinModelVersion_TwinModel
    FOREIGN KEY (TwinModelId) REFERENCES DigitalTwin.TwinModel(TwinModelId),
  CONSTRAINT FK_DigitalTwin_TwinModelVersion_SolverType
    FOREIGN KEY (SolverTypeId) REFERENCES DigitalTwin.SolverType(SolverTypeId),
  CONSTRAINT FK_DigitalTwin_TwinModelVersion_ValidationStatus
    FOREIGN KEY (ValidationStatusId) REFERENCES DigitalTwin.ValidationStatus(ValidationStatusId),
  CONSTRAINT FK_DigitalTwin_TwinModelVersion_ApprovedByUser
    FOREIGN KEY (ApprovedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE DigitalTwin.TwinModelComponent (
  TwinModelComponentId INT IDENTITY(1,1) NOT NULL,
  TwinModelId INT NOT NULL,
  EquipmentId INT NULL,
  PlantSystemId INT NULL,
  Code NVARCHAR(80) NOT NULL,
  Name NVARCHAR(250) NOT NULL,
  Description NVARCHAR(1000) NULL,
  CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_TwinModelComponent_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
  CreatedBy NVARCHAR(100) NOT NULL,
  ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_TwinModelComponent_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
  ModifiedBy NVARCHAR(100) NOT NULL,
  IsDeleted BIT NOT NULL CONSTRAINT DF_DigitalTwin_TwinModelComponent_IsDeleted DEFAULT 0,
  RowVersion ROWVERSION NOT NULL,

  CONSTRAINT PK_DigitalTwin_TwinModelComponent PRIMARY KEY (TwinModelComponentId),
  CONSTRAINT UQ_DigitalTwin_TwinModelComponent_Model_Code UNIQUE (TwinModelId, Code),
  CONSTRAINT FK_DigitalTwin_TwinModelComponent_TwinModel
    FOREIGN KEY (TwinModelId) REFERENCES DigitalTwin.TwinModel(TwinModelId),
  CONSTRAINT FK_DigitalTwin_TwinModelComponent_Equipment
    FOREIGN KEY (EquipmentId) REFERENCES ReactorFleet.Equipment(EquipmentId),
  CONSTRAINT FK_DigitalTwin_TwinModelComponent_PlantSystem
    FOREIGN KEY (PlantSystemId) REFERENCES ReactorFleet.PlantSystem(PlantSystemId),
  CONSTRAINT CK_DigitalTwin_TwinModelComponent_PhysicalAnchor
```

```

        CHECK (EquipmentId IS NOT NULL OR PlantSystemId IS NOT NULL)
    );
GO

CREATE TABLE DigitalTwin.TwinVariable (
    TwinVariableId INT IDENTITY(1,1) NOT NULL,
    TwinModelId INT NOT NULL,
    ModelVariableTypeId INT NOT NULL,
    EngineeringUnitId INT NULL,
    Code NVARCHAR(120) NOT NULL,
    Name NVARCHAR(250) NOT NULL,
    Description NVARCHAR(1000) NULL,
    IsStateVariable BIT NOT NULL CONSTRAINT DF_DigitalTwin_TwinVariable_IsStateVariable DEFAULT 0,
    IsRequired BIT NOT NULL CONSTRAINT DF_DigitalTwin_TwinVariable_IsRequired DEFAULT 0,
    LowerBound FLOAT NULL,
    UpperBound FLOAT NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_TwinVariable_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_TwinVariable_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_DigitalTwin_TwinVariable_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_DigitalTwin_TwinVariable PRIMARY KEY (TwinVariableId),
    CONSTRAINT UQ_DigitalTwin_TwinVariable_ModelCode UNIQUE (TwinModelId, Code),
    CONSTRAINT FK_DigitalTwin_TwinVariable_TwinModel
        FOREIGN KEY (TwinModelId) REFERENCES DigitalTwin.TwinModel(TwinModelId),
    CONSTRAINT FK_DigitalTwin_TwinVariable_ModelVariableType
        FOREIGN KEY (ModelVariableTypeId) REFERENCES DigitalTwin.ModelVariableType(ModelVariableTypeId),
    CONSTRAINT FK_DigitalTwin_TwinVariable_EngineeringUnit
        FOREIGN KEY (EngineeringUnitId) REFERENCES CorePlatform.EngineeringUnit(EngineeringUnitId),
    CONSTRAINT CK_DigitalTwin_TwinVariable_Bounds
        CHECK (LowerBound IS NULL OR UpperBound IS NULL OR LowerBound <= UpperBound)
);
GO

CREATE TABLE DigitalTwin.TwinParameter (
    TwinParameterId INT IDENTITY(1,1) NOT NULL,
    TwinModelVersionId INT NOT NULL,
    ModelParameterTypeId INT NOT NULL,
    EngineeringUnitId INT NULL,
    Code NVARCHAR(120) NOT NULL,
    Name NVARCHAR(250) NOT NULL,
    NumericValue FLOAT NULL,
    TextValue NVARCHAR(1000) NULL,
    JsonValue NVARCHAR(MAX) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_TwinParameter_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_TwinParameter_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_DigitalTwin_TwinParameter PRIMARY KEY (TwinParameterId),
    CONSTRAINT UQ_DigitalTwin_TwinParameter_VersionCode UNIQUE (TwinModelVersionId, Code),
    CONSTRAINT FK_DigitalTwin_TwinParameter_TwinModelVersion
        FOREIGN KEY (TwinModelVersionId) REFERENCES DigitalTwin.TwinModelVersion(TwinModelVersionId),
    CONSTRAINT FK_DigitalTwin_TwinParameter_ModelParameterType
        FOREIGN KEY (ModelParameterTypeId) REFERENCES DigitalTwin.ModelParameterType(ModelParameterTypeId),
    CONSTRAINT FK_DigitalTwin_TwinParameter_EngineeringUnit
        FOREIGN KEY (EngineeringUnitId) REFERENCES CorePlatform.EngineeringUnit(EngineeringUnitId),
    CONSTRAINT CK_DigitalTwin_TwinParameter_OneValue
        CHECK (NumericValue IS NOT NULL OR TextValue IS NOT NULL OR JsonValue IS NOT NULL)
);
GO

```

C.6.4.4 Signal bindings and runtime state

```

CREATE TABLE DigitalTwin.SignalBinding (
    SignalBindingId INT IDENTITY(1,1) NOT NULL,
    TwinModelId INT NOT NULL,
    TwinVariableId INT NOT NULL,
    SignalId INT NOT NULL,
    BindingRoleId INT NOT NULL,
    BindingStatusId INT NOT NULL,
    ModelVariable NVARCHAR(160) NOT NULL,
    ScaleFactor DECIMAL(18,8) NULL,
    OffsetValue DECIMAL(18,8) NULL,
    ToleranceAbs FLOAT NULL,
    TolerancePercent FLOAT NULL,
    EffectiveFromUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_SignalBinding_EffectiveFromUtc DEFAULT SYSUTCDATETIME(),
    EffectiveToUtc DATETIME2(7) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_SignalBinding_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_SignalBinding_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_DigitalTwin_SignalBinding_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_DigitalTwin_SignalBinding PRIMARY KEY (SignalBindingId),

```



```

CONSTRAINT UQ_DigitalTwin_SignalBinding_Model_Variable_Signal UNIQUE (TwinModelId, TwinVariableId, SignalId),
CONSTRAINT FK_DigitalTwin_SignalBinding_TwinModel
    FOREIGN KEY (TwinModelId) REFERENCES DigitalTwin.TwinModel(TwinModelId),
CONSTRAINT FK_DigitalTwin_SignalBinding_TwinVariable
    FOREIGN KEY (TwinVariableId) REFERENCES DigitalTwin.TwinVariable(TwinVariableId),
CONSTRAINT FK_DigitalTwin_SignalBinding_Signal
    FOREIGN KEY (SignalId) REFERENCES Instrumentation.Signal(SignalId),
CONSTRAINT FK_DigitalTwin_SignalBinding_BindingRole
    FOREIGN KEY (BindingRoleId) REFERENCES DigitalTwin.BindingRole(BindingRoleId),
CONSTRAINT FK_DigitalTwin_SignalBinding_BindingStatus
    FOREIGN KEY (BindingStatusId) REFERENCES DigitalTwin.BindingStatus(BindingStatusId),
CONSTRAINT CK_DigitalTwin_SignalBinding_EffectiveRange
    CHECK (EffectiveToUtc IS NULL OR EffectiveToUtc > EffectiveFromUtc)
);
GO

CREATE TABLE DigitalTwin.SignalBindingCalibration (
    SignalBindingCalibrationId INT IDENTITY(1,1) NOT NULL,
    SignalBindingId INT NOT NULL,
    CalibratedByUserId INT NULL,
    CalibratedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_SignalBindingCalibration_CalibratedAtUtc
    DEFAULT SYSUTCDATETIME(),
    ScaleFactor DECIMAL(18,8) NULL,
    OffsetValue DECIMAL(18,8) NULL,
    ToleranceAbs FLOAT NULL,
    TolerancePercent FLOAT NULL,
    Comment NVARCHAR(1000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_SignalBindingCalibration_CreatedAtUtc DEFAULT
    SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_DigitalTwin_SignalBindingCalibration PRIMARY KEY (SignalBindingCalibrationId),
    CONSTRAINT FK_DigitalTwin_SignalBindingCalibration_SignalBinding
        FOREIGN KEY (SignalBindingId) REFERENCES DigitalTwin.SignalBinding(SignalBindingId),
    CONSTRAINT FK_DigitalTwin_SignalBindingCalibration_CalibratedByUser
        FOREIGN KEY (CalibratedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE DigitalTwin.TwinRuntimeSession (
    TwinRuntimeSessionId BIGINT IDENTITY(1,1) NOT NULL,
    TwinModelVersionId INT NOT NULL,
    StartedByUserId INT NULL,
    SessionCode NVARCHAR(100) NOT NULL,
    RuntimeMode NVARCHAR(50) NOT NULL,
    HostName NVARCHAR(250) NULL,
    StartedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_TwinRuntimeSession_StartedAtUtc DEFAULT
    SYSUTCDATETIME(),
    EndedAtUtc DATETIME2(7) NULL,
    IsReadOnly BIT NOT NULL CONSTRAINT DF_DigitalTwin_TwinRuntimeSession_IsReadOnly DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_TwinRuntimeSession_CreatedAtUtc DEFAULT
    SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_DigitalTwin_TwinRuntimeSession PRIMARY KEY (TwinRuntimeSessionId),
    CONSTRAINT UQ_DigitalTwin_TwinRuntimeSession_Code UNIQUE (SessionCode),
    CONSTRAINT FK_DigitalTwin_TwinRuntimeSession_TwinModelVersion
        FOREIGN KEY (TwinModelVersionId) REFERENCES DigitalTwin.TwinModelVersion(TwinModelVersionId),
    CONSTRAINT FK_DigitalTwin_TwinRuntimeSession_StartedByUser
        FOREIGN KEY (StartedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_DigitalTwin_TwinRuntimeSession_TimeRange
        CHECK (EndedAtUtc IS NULL OR EndedAtUtc > StartedAtUtc)
);
GO

CREATE TABLE DigitalTwin.TwinSynchronization (
    TwinSynchronizationId BIGINT IDENTITY(1,1) NOT NULL,
    TwinRuntimeSessionId BIGINT NOT NULL,
    SynchronizationStatusId INT NOT NULL,
    CheckedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_TwinSynchronization_CheckedAtUtc DEFAULT
    SYSUTCDATETIME(),
    SyncLagMs INT NULL,
    MaxDivergencePercent DECIMAL(10,4) NULL,
    BoundSignalCount INT NOT NULL CONSTRAINT DF_DigitalTwin_TwinSynchronization_BoundSignalCount DEFAULT 0,
    DivergingSignalCount INT NOT NULL CONSTRAINT DF_DigitalTwin_TwinSynchronization_DivergingSignalCount DEFAULT 0,
    Notes NVARCHAR(1000) NULL,

    CONSTRAINT PK_DigitalTwin_TwinSynchronization PRIMARY KEY (TwinSynchronizationId),
    CONSTRAINT FK_DigitalTwin_TwinSynchronization_TwinRuntimeSession
        FOREIGN KEY (TwinRuntimeSessionId) REFERENCES DigitalTwin.TwinRuntimeSession(TwinRuntimeSessionId),
    CONSTRAINT FK_DigitalTwin_TwinSynchronization_SynchronizationStatus
        FOREIGN KEY (SynchronizationStatusId) REFERENCES DigitalTwin.SynchronizationStatus(SynchronizationStatusId),
    CONSTRAINT CK_DigitalTwin_TwinSynchronization_Counts CHECK (BoundSignalCount >= 0 AND DivergingSignalCount >= 0)
);
GO

CREATE TABLE DigitalTwin.TwinStateVector (
    TwinStateVectorId BIGINT IDENTITY(1,1) NOT NULL,
    TwinRuntimeSessionId BIGINT NOT NULL,
    CapturedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_TwinStateVector_CapturedAtUtc DEFAULT
    SYSUTCDATETIME(),
    StateJson NVARCHAR(MAX) NOT NULL,
    StateHash VARBINARY(32) NULL,

```



```

CONSTRAINT PK_DigitalTwin_TwinStateVector PRIMARY KEY (TwinStateVectorId),
CONSTRAINT FK_DigitalTwin_TwinStateVector_TwinRuntimeSession
    FOREIGN KEY (TwinRuntimeSessionId) REFERENCES DigitalTwin.TwinRuntimeSession(TwinRuntimeSessionId)
);
GO

```

C.6.4.5 Snapshots and divergence records

```

CREATE TABLE DigitalTwin.TwinSnapshot (
    TwinSnapshotId BIGINT IDENTITY(1,1) NOT NULL,
    TwinRuntimeSessionId BIGINT NOT NULL,
    SnapshotReasonId INT NOT NULL,
    SnapshotAtUtc DATETIME2(7) NOT NULL,
    TimeStepIndex BIGINT NULL,
    StateVectorHash VARBINARY(32) NULL,
    SummaryJson NVARCHAR(MAX) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_TwinSnapshot_CreatedAtUtc DEFAULT SYSUTCDATETIME(),

    CONSTRAINT PK_DigitalTwin_TwinSnapshot PRIMARY KEY (TwinSnapshotId),
    CONSTRAINT FK_DigitalTwin_TwinSnapshot_TwinRuntimeSession
        FOREIGN KEY (TwinRuntimeSessionId) REFERENCES DigitalTwin.TwinRuntimeSession(TwinRuntimeSessionId),
    CONSTRAINT FK_DigitalTwin_TwinSnapshot_SnapshotReason
        FOREIGN KEY (SnapshotReasonId) REFERENCES DigitalTwin.SnapshotReason(SnapshotReasonId)
);
GO

CREATE TABLE DigitalTwin.TwinSnapshotValue (
    TwinSnapshotValueId BIGINT IDENTITY(1,1) NOT NULL,
    TwinSnapshotId BIGINT NOT NULL,
    TwinVariableId INT NOT NULL,
    NumericValue FLOAT NULL,
    TextValue NVARCHAR(1000) NULL,
    JsonValue NVARCHAR(MAX) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_TwinSnapshotValue_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),

    CONSTRAINT PK_DigitalTwin_TwinSnapshotValue PRIMARY KEY (TwinSnapshotValueId),
    CONSTRAINT UQ_DigitalTwin_TwinSnapshotValue_Snapshot_Variable UNIQUE (TwinSnapshotId, TwinVariableId),
    CONSTRAINT FK_DigitalTwin_TwinSnapshotValue_TwinSnapshot
        FOREIGN KEY (TwinSnapshotId) REFERENCES DigitalTwin.TwinSnapshot(TwinSnapshotId),
    CONSTRAINT FK_DigitalTwin_TwinSnapshotValue_TwinVariable
        FOREIGN KEY (TwinVariableId) REFERENCES DigitalTwin.TwinVariable(TwinVariableId),
    CONSTRAINT CK_DigitalTwin_TwinSnapshotValue_OneValue
        CHECK (NumericValue IS NOT NULL OR TextValue IS NOT NULL OR JsonValue IS NOT NULL)
);
GO

CREATE TABLE DigitalTwin.TwinDivergence (
    TwinDivergenceId BIGINT IDENTITY(1,1) NOT NULL,
    TwinSnapshotId BIGINT NOT NULL,
    SignalId INT NOT NULL,
    TwinVariableId INT NULL,
    DivergenceSeverityId INT NOT NULL,
    DivergenceStatusId INT NOT NULL,
    DetectedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_TwinDivergence_DetectedAtUtc DEFAULT
SYSUTCDATETIME(),
    ModeledValue FLOAT NOT NULL,
    MeasuredValue FLOAT NOT NULL,
    DeltaValue FLOAT NOT NULL,
    DeltaPercent DECIMAL(10,4) NULL,
    ThresholdAbs FLOAT NULL,
    ThresholdPercent DECIMAL(10,4) NULL,
    Explanation NVARCHAR(1000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_TwinDivergence_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_DigitalTwin_TwinDivergence PRIMARY KEY (TwinDivergenceId),
    CONSTRAINT FK_DigitalTwin_TwinDivergence_TwinSnapshot
        FOREIGN KEY (TwinSnapshotId) REFERENCES DigitalTwin.TwinSnapshot(TwinSnapshotId),
    CONSTRAINT FK_DigitalTwin_TwinDivergence_Signal
        FOREIGN KEY (SignalId) REFERENCES Instrumentation.Signal(SignalId),
    CONSTRAINT FK_DigitalTwin_TwinDivergence_TwinVariable
        FOREIGN KEY (TwinVariableId) REFERENCES DigitalTwin.TwinVariable(TwinVariableId),
    CONSTRAINT FK_DigitalTwin_TwinDivergence_DivergenceSeverity
        FOREIGN KEY (DivergenceSeverityId) REFERENCES DigitalTwin.DivergenceSeverity(DivergenceSeverityId),
    CONSTRAINT FK_DigitalTwin_TwinDivergence_DivergenceStatus
        FOREIGN KEY (DivergenceStatusId) REFERENCES DigitalTwin.DivergenceStatus(DivergenceStatusId)
);
GO

CREATE TABLE DigitalTwin.TwinDivergenceReview (
    TwinDivergenceReviewId BIGINT IDENTITY(1,1) NOT NULL,
    TwinDivergenceId BIGINT NOT NULL,
    DivergenceStatusId INT NOT NULL,
    ReviewedByUserId INT NULL,
    ReviewedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_TwinDivergenceReview_ReviewedAtUtc DEFAULT
SYSUTCDATETIME(),
    ReviewNote NVARCHAR(2000) NULL,

```

```

    CorrectiveAction      NVARCHAR(1000) NULL,
    CreatedAtUtc          DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_TwinDivergenceReview_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),

    CONSTRAINT PK_DigitalTwin_TwinDivergenceReview PRIMARY KEY (TwinDivergenceReviewId),
    CONSTRAINT FK_DigitalTwin_TwinDivergenceReview_TwinDivergence
        FOREIGN KEY (TwinDivergenceId) REFERENCES DigitalTwin.TwinDivergence(TwinDivergenceId),
    CONSTRAINT FK_DigitalTwin_TwinDivergenceReview_DivergenceStatus
        FOREIGN KEY (DivergenceStatusId) REFERENCES DigitalTwin.DivergenceStatus(DivergenceStatusId),
    CONSTRAINT FK_DigitalTwin_TwinDivergenceReview_ReviewedByUser
        FOREIGN KEY (ReviewedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId)
);
GO

```

C.6.4.6 Simulation scenarios, runs and what-if cases

```

CREATE TABLE DigitalTwin.SimulationScenario (
    SimulationScenarioId INT IDENTITY(1,1) NOT NULL,
    TwinModelId          INT NOT NULL,
    SimulationScenarioTypeId INT NOT NULL,
    CreatedByUserId      INT NULL,
    Code                 NVARCHAR(100) NOT NULL,
    Name                 NVARCHAR(250) NOT NULL,
    Description           NVARCHAR(2000) NULL,
    DefaultHorizonSeconds INT NULL,
    DefaultStepMs         INT NULL,
    CreatedAtUtc          DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_SimulationScenario_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy            NVARCHAR(100) NOT NULL,
    ModifiedAtUtc        DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_SimulationScenario_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
    ModifiedBy           NVARCHAR(100) NOT NULL,
    IsDeleted            BIT NOT NULL CONSTRAINT DF_DigitalTwin_SimulationScenario_IsDeleted DEFAULT 0,
    RowVersion           ROWVERSION NOT NULL,

    CONSTRAINT PK_DigitalTwin_SimulationScenario PRIMARY KEY (SimulationScenarioId),
    CONSTRAINT UQ_DigitalTwin_SimulationScenario_Model_Code UNIQUE (TwinModelId, Code),
    CONSTRAINT FK_DigitalTwin_SimulationScenario_TwinModel
        FOREIGN KEY (TwinModelId) REFERENCES DigitalTwin.TwinModel(TwinModelId),
    CONSTRAINT FK_DigitalTwin_SimulationScenario_SimulationScenarioType
        FOREIGN KEY (SimulationScenarioTypeId) REFERENCES DigitalTwin.SimulationScenarioType(SimulationScenarioTypeId),
    CONSTRAINT FK_DigitalTwin_SimulationScenario_CreatedByUser
        FOREIGN KEY (CreatedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_DigitalTwin_SimulationScenario_Defaults CHECK ((DefaultHorizonSeconds IS NULL OR DefaultHorizonSeconds > 0)
AND (DefaultStepMs IS NULL OR DefaultStepMs > 0))
);
GO

CREATE TABLE DigitalTwin.SimulationScenarioInput (
    SimulationScenarioInputId INT IDENTITY(1,1) NOT NULL,
    SimulationScenarioId      INT NOT NULL,
    TwinVariableId            INT NOT NULL,
    NumericValue              FLOAT NULL,
    TextValue                 NVARCHAR(1000) NULL,
    JsonValue                 NVARCHAR(MAX) NULL,

    CONSTRAINT PK_DigitalTwin_SimulationScenarioInput PRIMARY KEY (SimulationScenarioInputId),
    CONSTRAINT UQ_DigitalTwin_SimulationScenarioInput_Scenario_Variable UNIQUE (SimulationScenarioId, TwinVariableId),
    CONSTRAINT FK_DigitalTwin_SimulationScenarioInput_SimulationScenario
        FOREIGN KEY (SimulationScenarioId) REFERENCES DigitalTwin.SimulationScenario(SimulationScenarioId),
    CONSTRAINT FK_DigitalTwin_SimulationScenarioInput_TwinVariable
        FOREIGN KEY (TwinVariableId) REFERENCES DigitalTwin.TwinVariable(TwinVariableId),
    CONSTRAINT CK_DigitalTwin_SimulationScenarioInput_OneValue
        CHECK (NumericValue IS NOT NULL OR TextValue IS NOT NULL OR JsonValue IS NOT NULL)
);
GO

CREATE TABLE DigitalTwin.SimulationRun (
    SimulationRunId          BIGINT IDENTITY(1,1) NOT NULL,
    SimulationScenarioId     INT NOT NULL,
    TwinModelVersionId       INT NOT NULL,
    SimulationRunStatusId    INT NOT NULL,
    RequestedByUserId        INT NULL,
    RunCode                  NVARCHAR(120) NOT NULL,
    StartedAtUtc             DATETIME2(7) NULL,
    CompletedAtUtc           DATETIME2(7) NULL,
    HorizonSeconds           INT NOT NULL,
    StepMs                   INT NOT NULL,
    ResultSummaryJson        NVARCHAR(MAX) NULL,
    CreatedAtUtc             DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_SimulationRun_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy                NVARCHAR(100) NOT NULL,
    RowVersion               ROWVERSION NOT NULL,

    CONSTRAINT PK_DigitalTwin_SimulationRun PRIMARY KEY (SimulationRunId),
    CONSTRAINT UQ_DigitalTwin_SimulationRun_RunCode UNIQUE (RunCode),
    CONSTRAINT FK_DigitalTwin_SimulationRun_SimulationScenario
        FOREIGN KEY (SimulationScenarioId) REFERENCES DigitalTwin.SimulationScenario(SimulationScenarioId),
    CONSTRAINT FK_DigitalTwin_SimulationRun_TwinModelVersion
        FOREIGN KEY (TwinModelVersionId) REFERENCES DigitalTwin.TwinModelVersion(TwinModelVersionId),

```

```

CONSTRAINT FK_DigitalTwin_SimulationRun_SimulationRunStatus
    FOREIGN KEY (SimulationRunStatusId) REFERENCES DigitalTwin.SimulationRunStatus(SimulationRunStatusId),
CONSTRAINT FK_DigitalTwin_SimulationRun_RequestedByUser
    FOREIGN KEY (RequestedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
CONSTRAINT CK_DigitalTwin_SimulationRun_PositiveStep CHECK (HorizonSeconds > 0 AND StepMs > 0),
CONSTRAINT CK_DigitalTwin_SimulationRun_TimeRange CHECK (CompletedAtUtc IS NULL OR StartedAtUtc IS NULL OR CompletedAtUtc
=> StartedAtUtc)
);
GO

CREATE TABLE DigitalTwin.SimulationRunStep (
    SimulationRunStepId BIGINT IDENTITY(1,1) NOT NULL,
    SimulationRunId      BIGINT NOT NULL,
    StepIndex            BIGINT NOT NULL,
    SimTimeSeconds       DECIMAL(18,6) NOT NULL,
    WallClockAtUtc       DATETIME2(7) NULL,
    StepStateJson        NVARCHAR(MAX) NULL,

    CONSTRAINT PK_DigitalTwin_SimulationRunStep PRIMARY KEY (SimulationRunStepId),
    CONSTRAINT UQ_DigitalTwin_SimulationRunStep_Run_Step UNIQUE (SimulationRunId, StepIndex),
    CONSTRAINT FK_DigitalTwin_SimulationRunStep_SimulationRun
        FOREIGN KEY (SimulationRunId) REFERENCES DigitalTwin.SimulationRun(SimulationRunId),
    CONSTRAINT CK_DigitalTwin_SimulationRunStep_Step CHECK (StepIndex >= 0 AND SimTimeSeconds >= 0)
);
GO

CREATE TABLE DigitalTwin.SimulationRunOutput (
    SimulationRunOutputId BIGINT IDENTITY(1,1) NOT NULL,
    SimulationRunStepId   BIGINT NOT NULL,
    TwinVariableId        INT NOT NULL,
    NumericValue          FLOAT NULL,
    TextValue             NVARCHAR(1000) NULL,
    JsonValue             NVARCHAR(MAX) NULL,

    CONSTRAINT PK_DigitalTwin_SimulationRunOutput PRIMARY KEY (SimulationRunOutputId),
    CONSTRAINT UQ_DigitalTwin_SimulationRunOutput_Step_Variable UNIQUE (SimulationRunStepId, TwinVariableId),
    CONSTRAINT FK_DigitalTwin_SimulationRunOutput_SimulationRunStep
        FOREIGN KEY (SimulationRunStepId) REFERENCES DigitalTwin.SimulationRunStep(SimulationRunStepId),
    CONSTRAINT FK_DigitalTwin_SimulationRunOutput_TwinVariable
        FOREIGN KEY (TwinVariableId) REFERENCES DigitalTwin.TwinVariable(TwinVariableId),
    CONSTRAINT CK_DigitalTwin_SimulationRunOutput_OneValue
        CHECK (NumericValue IS NOT NULL OR TextValue IS NOT NULL OR JsonValue IS NOT NULL)
);
GO

CREATE TABLE DigitalTwin.WhatIfCase (
    WhatIfCaseId          INT IDENTITY(1,1) NOT NULL,
    TwinModelId           INT NOT NULL,
    RequestedByUserId     INT NULL,
    Code                  NVARCHAR(100) NOT NULL,
    Name                  NVARCHAR(250) NOT NULL,
    Question              NVARCHAR(2000) NOT NULL,
    CreatedAtUtc          DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_WhatIfCase_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy             NVARCHAR(100) NOT NULL,
    ModifiedAtUtc         DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_WhatIfCase_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy            NVARCHAR(100) NOT NULL,
    IsDeleted             BIT NOT NULL CONSTRAINT DF_DigitalTwin_WhatIfCase_IsDeleted DEFAULT 0,
    RowVersion            ROWVERSION NOT NULL,

    CONSTRAINT PK_DigitalTwin_WhatIfCase PRIMARY KEY (WhatIfCaseId),
    CONSTRAINT UQ_DigitalTwin_WhatIfCase_Model_Code UNIQUE (TwinModelId, Code),
    CONSTRAINT FK_DigitalTwin_WhatIfCase_TwinModel
        FOREIGN KEY (TwinModelId) REFERENCES DigitalTwin.TwinModel(TwinModelId),
    CONSTRAINT FK_DigitalTwin_WhatIfCase_RequestedByUser
        FOREIGN KEY (RequestedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE DigitalTwin.WhatIfCaseInput (
    WhatIfCaseInputId    INT IDENTITY(1,1) NOT NULL,
    WhatIfCaseId         INT NOT NULL,
    TwinVariableId       INT NOT NULL,
    NumericValue         FLOAT NULL,
    TextValue            NVARCHAR(1000) NULL,
    JsonValue            NVARCHAR(MAX) NULL,

    CONSTRAINT PK_DigitalTwin_WhatIfCaseInput PRIMARY KEY (WhatIfCaseInputId),
    CONSTRAINT UQ_DigitalTwin_WhatIfCaseInput_Case_Variable UNIQUE (WhatIfCaseId, TwinVariableId),
    CONSTRAINT FK_DigitalTwin_WhatIfCaseInput_WhatIfCase
        FOREIGN KEY (WhatIfCaseId) REFERENCES DigitalTwin.WhatIfCase(WhatIfCaseId),
    CONSTRAINT FK_DigitalTwin_WhatIfCaseInput_TwinVariable
        FOREIGN KEY (TwinVariableId) REFERENCES DigitalTwin.TwinVariable(TwinVariableId),
    CONSTRAINT CK_DigitalTwin_WhatIfCaseInput_OneValue
        CHECK (NumericValue IS NOT NULL OR TextValue IS NOT NULL OR JsonValue IS NOT NULL)
);
GO

CREATE TABLE DigitalTwin.WhatIfCaseResult (
    WhatIfCaseResultId   INT IDENTITY(1,1) NOT NULL,
    WhatIfCaseId         INT NOT NULL,
    SimulationRunId       BIGINT NULL,
    ResultText           NVARCHAR(2000) NOT NULL,
    ResultJson           NVARCHAR(MAX) NULL,
    CreatedAtUtc         DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_WhatIfCaseResult_CreatedAtUtc DEFAULT SYSUTCDATETIME(),

```

```

CONSTRAINT PK_DigitalTwin_WhatIfCaseResult PRIMARY KEY (WhatIfCaseResultId),
CONSTRAINT FK_DigitalTwin_WhatIfCaseResult_WhatIfCase
    FOREIGN KEY (WhatIfCaseId) REFERENCES DigitalTwin.WhatIfCase(WhatIfCaseId),
CONSTRAINT FK_DigitalTwin_WhatIfCaseResult_SimulationRun
    FOREIGN KEY (SimulationRunId) REFERENCES DigitalTwin.SimulationRun(SimulationRunId)
);
GO

```

C.6.4.7 Health checks, validation, assumptions and annotations

```

CREATE TABLE DigitalTwin.TwinHealthCheck (
    TwinHealthCheckId BIGINT IDENTITY(1,1) NOT NULL,
    TwinRuntimeSessionId BIGINT NOT NULL,
    SynchronizationStatusId INT NOT NULL,
    CheckedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_TwinHealthCheck_CheckedAtUtc DEFAULT
SYSUTCDATETIME(),
    HealthScore DECIMAL(6,3) NULL,
    SyncLagMs INT NULL,
    DivergenceOpenCount INT NOT NULL CONSTRAINT DF_DigitalTwin_TwinHealthCheck_DivergenceOpenCount DEFAULT 0,
    Message NVARCHAR(1000) NULL,

    CONSTRAINT PK_DigitalTwin_TwinHealthCheck PRIMARY KEY (TwinHealthCheckId),
    CONSTRAINT FK_DigitalTwin_TwinHealthCheck_TwinRuntimeSession
        FOREIGN KEY (TwinRuntimeSessionId) REFERENCES DigitalTwin.TwinRuntimeSession(TwinRuntimeSessionId),
    CONSTRAINT FK_DigitalTwin_TwinHealthCheck_SynchronizationStatus
        FOREIGN KEY (SynchronizationStatusId) REFERENCES DigitalTwin.SynchronizationStatus(SynchronizationStatusId),
    CONSTRAINT CK_DigitalTwin_TwinHealthCheck_HealthScore CHECK (HealthScore IS NULL OR (HealthScore >= 0 AND HealthScore <=
100))
);
GO

CREATE TABLE DigitalTwin.TwinModelValidation (
    TwinModelValidationId INT IDENTITY(1,1) NOT NULL,
    TwinModelVersionId INT NOT NULL,
    ValidationStatusId INT NOT NULL,
    PerformedByUserId INT NULL,
    ValidationCode NVARCHAR(100) NOT NULL,
    StartedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_TwinModelValidation_StartedAtUtc DEFAULT
SYSUTCDATETIME(),
    CompletedAtUtc DATETIME2(7) NULL,
    Summary NVARCHAR(2000) NULL,
    EvidenceUri NVARCHAR(500) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_TwinModelValidation_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_DigitalTwin_TwinModelValidation PRIMARY KEY (TwinModelValidationId),
    CONSTRAINT UQ_DigitalTwin_TwinModelValidation_Version_Code UNIQUE (TwinModelVersionId, ValidationCode),
    CONSTRAINT FK_DigitalTwin_TwinModelValidation_TwinModelVersion
        FOREIGN KEY (TwinModelVersionId) REFERENCES DigitalTwin.TwinModelVersion(TwinModelVersionId),
    CONSTRAINT FK_DigitalTwin_TwinModelValidation_ValidationStatus
        FOREIGN KEY (ValidationStatusId) REFERENCES DigitalTwin.ValidationStatus(ValidationStatusId),
    CONSTRAINT FK_DigitalTwin_TwinModelValidation_PerformedByUser
        FOREIGN KEY (PerformedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_DigitalTwin_TwinModelValidation_TimeRange CHECK (CompletedAtUtc IS NULL OR CompletedAtUtc >= StartedAtUtc)
);
GO

CREATE TABLE DigitalTwin.TwinValidationMetric (
    TwinValidationMetricId INT IDENTITY(1,1) NOT NULL,
    TwinModelValidationId INT NOT NULL,
    TwinVariableId INT NULL,
    MetricCode NVARCHAR(100) NOT NULL,
    MetricName NVARCHAR(250) NOT NULL,
    MetricValue FLOAT NOT NULL,
    ThresholdValue FLOAT NULL,
    Passed BIT NULL,

    CONSTRAINT PK_DigitalTwin_TwinValidationMetric PRIMARY KEY (TwinValidationMetricId),
    CONSTRAINT UQ_DigitalTwin_TwinValidationMetric_Validation_Code_Variable UNIQUE (TwinModelValidationId, MetricCode,
TwinVariableId),
    CONSTRAINT FK_DigitalTwin_TwinValidationMetric_TwinModelValidation
        FOREIGN KEY (TwinModelValidationId) REFERENCES DigitalTwin.TwinModelValidation(TwinModelValidationId),
    CONSTRAINT FK_DigitalTwin_TwinValidationMetric_TwinVariable
        FOREIGN KEY (TwinVariableId) REFERENCES DigitalTwin.TwinVariable(TwinVariableId)
);
GO

CREATE TABLE DigitalTwin.ModelAssumption (
    ModelAssumptionId INT IDENTITY(1,1) NOT NULL,
    TwinModelVersionId INT NOT NULL,
    Code NVARCHAR(100) NOT NULL,
    Statement NVARCHAR(2000) NOT NULL,
    Consequence NVARCHAR(2000) NULL,
    ValidFromUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_ModelAssumption_ValidFromUtc DEFAULT SYSUTCDATETIME(),
    ValidToUtc DATETIME2(7) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_ModelAssumption_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,

```

```

        ModifiedAtUtc      DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_ModelAssumption_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
        ModifiedBy        NVARCHAR(100) NOT NULL,
        RowVersion         ROWVERSION NOT NULL,

        CONSTRAINT PK_DigitalTwin_ModelAssumption PRIMARY KEY (ModelAssumptionId),
        CONSTRAINT UQ_DigitalTwin_ModelAssumption_Version_Code UNIQUE (TwinModelVersionId, Code),
        CONSTRAINT FK_DigitalTwin_ModelAssumption_TwinModelVersion
            FOREIGN KEY (TwinModelVersionId) REFERENCES DigitalTwin.TwinModelVersion(TwinModelVersionId),
        CONSTRAINT CK_DigitalTwin_ModelAssumption_ValidRange CHECK (ValidToUtc IS NULL OR ValidToUtc > ValidFromUtc)
    );
GO

CREATE TABLE DigitalTwin.TwinAnnotation (
    TwinAnnotationId      BIGINT IDENTITY(1,1) NOT NULL,
    TwinModelId           INT NOT NULL,
    TwinSnapshotId        BIGINT NULL,
    TwinDivergenceId      BIGINT NULL,
    SimulationRunId        BIGINT NULL,
    CreatedByUserId        INT NULL,
    AnnotationType         NVARCHAR(80) NOT NULL,
    Body                  NVARCHAR(2000) NOT NULL,
    CreatedAtUtc           DATETIME2(7) NOT NULL CONSTRAINT DF_DigitalTwin_TwinAnnotation_CreatedAtUtc DEFAULT SYSUTCDATETIME(),

    CONSTRAINT PK_DigitalTwin_TwinAnnotation PRIMARY KEY (TwinAnnotationId),
    CONSTRAINT FK_DigitalTwin_TwinAnnotation_TwinModel
        FOREIGN KEY (TwinModelId) REFERENCES DigitalTwin.TwinModel(TwinModelId),
    CONSTRAINT FK_DigitalTwin_TwinAnnotation_TwinSnapshot
        FOREIGN KEY (TwinSnapshotId) REFERENCES DigitalTwin.TwinSnapshot(TwinSnapshotId),
    CONSTRAINT FK_DigitalTwin_TwinAnnotation_TwinDivergence
        FOREIGN KEY (TwinDivergenceId) REFERENCES DigitalTwin.TwinDivergence(TwinDivergenceId),
    CONSTRAINT FK_DigitalTwin_TwinAnnotation_SimulationRun
        FOREIGN KEY (SimulationRunId) REFERENCES DigitalTwin.SimulationRun(SimulationRunId),
    CONSTRAINT FK_DigitalTwin_TwinAnnotation_CreatedByUser
        FOREIGN KEY (CreatedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_DigitalTwin_TwinAnnotation_Target
        CHECK (TwinSnapshotId IS NOT NULL OR TwinDivergenceId IS NOT NULL OR SimulationRunId IS NOT NULL)
    );
GO

```

C.6.5 Indexes and seed examples

The most important access paths are: twin by unit, binding by signal, snapshot by runtime and time, divergence by signal and time, and simulation run by scenario and status.

```

CREATE INDEX IX_DigitalTwin_TwinModel_UnitId
ON DigitalTwin.TwinModel(UnitId);

CREATE INDEX IX_DigitalTwin_TwinModelVersion_TwinModelId
ON DigitalTwin.TwinModelVersion(TwinModelId);

CREATE INDEX IX_DigitalTwin_TwinVariable_TwinModelId
ON DigitalTwin.TwinVariable(TwinModelId);

CREATE INDEX IX_DigitalTwin_SignalBinding_SignalId
ON DigitalTwin.SignalBinding(SignalId);

CREATE INDEX IX_DigitalTwin_SignalBinding_TwinModelId
ON DigitalTwin.SignalBinding(TwinModelId);

CREATE INDEX IX_DigitalTwin_TwinRuntimeSession_TwinModelVersionId_StartedAtUtc
ON DigitalTwin.TwinRuntimeSession(TwinModelVersionId, StartedAtUtc DESC);

CREATE INDEX IX_DigitalTwin_TwinSnapshot_Session_Time
ON DigitalTwin.TwinSnapshot(TwinRuntimeSessionId, SnapshotAtUtc DESC);

CREATE INDEX IX_DigitalTwin_TwinSnapshotValue_Snapshot_Variable
ON DigitalTwin.TwinSnapshotValue(TwinSnapshotId, TwinVariableId);

CREATE INDEX IX_DigitalTwin_TwinDivergence_Signal_DetectedAtUtc
ON DigitalTwin.TwinDivergence(SignalId, DetectedAtUtc DESC);

CREATE INDEX IX_DigitalTwin_TwinDivergence_Status_Severity
ON DigitalTwin.TwinDivergence(DivergenceStatusId, DivergenceSeverityId, DetectedAtUtc DESC);

CREATE INDEX IX_DigitalTwin_SimulationRun_Scenario_Status
ON DigitalTwin.SimulationRun(SimulationScenarioId, SimulationRunStatusId, CreatedAtUtc DESC);

CREATE INDEX IX_DigitalTwin_SimulationRunOutput_Step_Variable
ON DigitalTwin.SimulationRunOutput(SimulationRunStepId, TwinVariableId);
GO

```

C.6.5.1 Minimal seed examples

```

INSERT INTO DigitalTwin.TwinModelType (Code, Name) VALUES
('POINT_KINETICS', 'Point kinetics model'),
('THERMAL_HYDRAULIC', 'Thermal-hydraulic approximation'),
('EQUIPMENT_HEALTH', 'Equipment health model'),
('VISUALIZATION', 'Visualization model');

INSERT INTO DigitalTwin.TwinModelStatus (Code, Name) VALUES
('DRAFT', 'Draft'), ('VALIDATING', 'Validating'), ('ACTIVE', 'Active'),
('RETIRED', 'Retired'), ('SUPERSEDED', 'Superseded');

INSERT INTO DigitalTwin.TwinFidelityLevel (Code, Name) VALUES
('ILLUSTRATIVE', 'Illustrative'), ('TRAINING', 'Training'),
('SHADOW', 'Shadow'), ('ADVISORY_READY', 'Advisory ready'), ('VALIDATED', 'Validated');

INSERT INTO DigitalTwin.ModelVariableType (Code, Name) VALUES
('STATE', 'State'), ('INPUT', 'Input'), ('OUTPUT', 'Output'),
('DERIVED', 'Derived'), ('DIAGNOSTIC', 'Diagnostic');

INSERT INTO DigitalTwin.BindingRole (Code, Name) VALUES
('INPUT', 'Input'), ('OUTPUT_WITNESS', 'Output witness'),
('CALIBRATION', 'Calibration witness'), ('DIVERGENCE_WITNESS', 'Divergence witness');

INSERT INTO DigitalTwin.DivergenceSeverity (Code, Name) VALUES
('INFO', 'Information'), ('WARNING', 'Warning'), ('CRITICAL', 'Critical'), ('INVALID', 'Invalid');

INSERT INTO DigitalTwin.DivergenceStatus (Code, Name) VALUES
('OPEN', 'Open'), ('ACKNOWLEDGED', 'Acknowledged'), ('EXPLAINED', 'Explained'),
('CORRECTED', 'Corrected'), ('WAIVED', 'Waived'), ('CLOSED', 'Closed');
GO

```

C.6.6 ER diagram

The central line is **TwinModel** to **TwinVariable** to **SignalBinding**. Runtime sessions produce snapshots; snapshots produce values; values can be compared to signals and recorded as divergence.

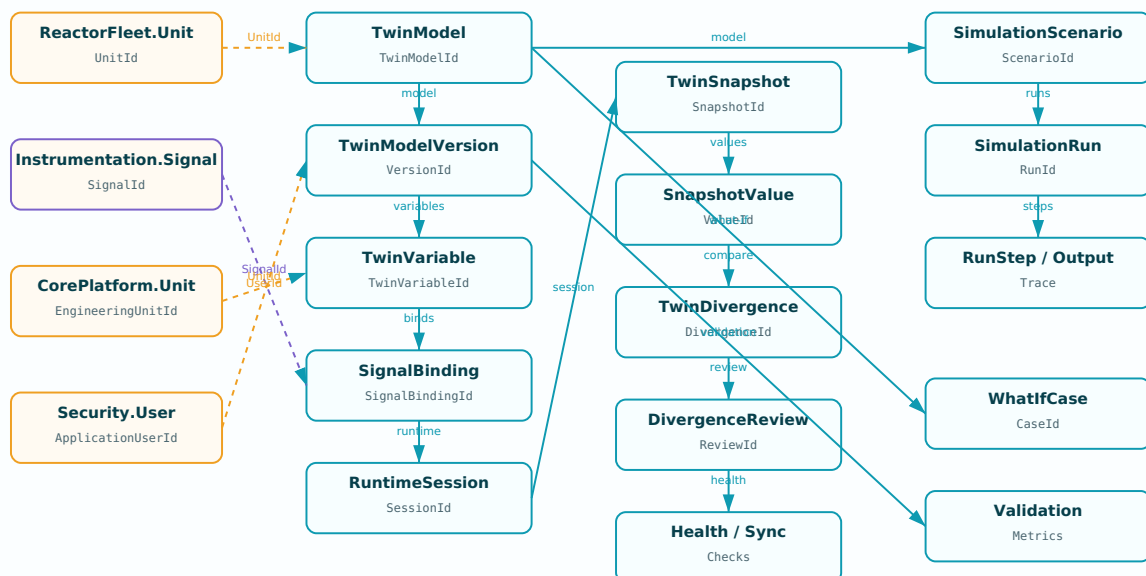


Figure C.6.1 - DigitalTwin sector ER diagram. Lookup tables are omitted from the drawing for readability; they are included in the DDL and FK mapping.

C.6.7 Foreign-key mapping

C.6.7.1 Internal DigitalTwin foreign keys

FROM TABLE	FK COLUMN	TO TABLE	PK COLUMN	RELATIONSHIP
TwinModelVersion	TwinModelId	TwinModel	TwinModelId	Many versions per model.
TwinModelComponent	TwinModelId	TwinModel	TwinModelId	Many model components per model.
TwinVariable	TwinModelId	TwinModel	TwinModelId	Many variables per model.
TwinParameter	TwinModelVersionId	TwinModelVersion	TwinModelVersionId	Parameters are versioned.
SignalBinding	TwinModelId	TwinModel	TwinModelId	Bindings belong to the model.
SignalBinding	TwinVariableId	TwinVariable	TwinVariableId	Binding maps a model variable.
SignalBindingCalibration	SignalBindingId	SignalBinding	SignalBindingId	Calibration belongs to the binding.
TwinRuntimeSession	TwinModelVersionId	TwinModelVersion	TwinModelVersionId	A runtime executes one version.
TwinSynchronization	TwinRuntimeSessionId	TwinRuntimeSession	TwinRuntimeSessionId	Synchronization samples per runtime.
TwinStateVector	TwinRuntimeSessionId	TwinRuntimeSession	TwinRuntimeSessionId	State-vector checkpoints per runtime.
TwinSnapshot	TwinRuntimeSessionId	TwinRuntimeSession	TwinRuntimeSessionId	Many snapshots per runtime.
TwinSnapshotValue	TwinSnapshotId	TwinSnapshot	TwinSnapshotId	Many values per snapshot.
TwinSnapshotValue	TwinVariableId	TwinVariable	TwinVariableId	One value per variable in a snapshot.
TwinDivergence	TwinSnapshotId	TwinSnapshot	TwinSnapshotId	Divergence is anchored to the snapshot where it was seen.
TwinDivergenceReview	TwinDivergenceId	TwinDivergence	TwinDivergenceId	Reviews belong to a divergence.
SimulationScenarioInput	SimulationScenarioId	SimulationScenario	SimulationScenarioId	Inputs belong to a scenario.
SimulationRun	SimulationScenarioId	SimulationScenario	SimulationScenarioId	Runs execute a scenario.

FROM TABLE	FK COLUMN	TO TABLE	PK COLUMN	RELATIONSHIP
SimulationRunStep	SimulationRunId	SimulationRun	SimulationRunId	Steps belong to a run.
SimulationRunOutput	SimulationRunStepId	SimulationRunStep	SimulationRunStepId	Outputs belong to a run step.
WhatIfCaseInput	WhatIfCaseId	WhatIfCase	WhatIfCaseId	Inputs belong to the case.
WhatIfCaseResult	WhatIfCaseId	WhatIfCase	WhatIfCaseId	Results belong to the case.
TwinModelValidation	TwinModelVersionId	TwinModelVersion	TwinModelVersionId	Validation campaigns are version-specific.
TwinValidationMetric	TwinModelValidationId	TwinModelValidation	TwinModelValidationId	Metrics belong to validation campaign.
ModelAssumption	TwinModelVersionId	TwinModelVersion	TwinModelVersionId	Assumptions are versioned.

C.6.7.2 Passports consumed from previous sectors

FROM TABLE	FK COLUMN	TO TABLE	PK COLUMN	RELATIONSHIP
TwinModel	UnitId	ReactorFleet.Unit	UnitId	The twin names the physical unit it mirrors.
TwinModelComponent	EquipmentId	ReactorFleet.Equipment	EquipmentId	A model component may map to a physical equipment item.
TwinModelComponent	PlantSystemId	ReactorFleet.PlantSystem	PlantSystemId	A model component may map to a plant system.
TwinVariable	EngineeringUnitId	CorePlatform.EngineeringUnit	EngineeringUnitId	Variables use the platform unit catalogue.
TwinParameter	EngineeringUnitId	CorePlatform.EngineeringUnit	EngineeringUnitId	Parameters also use the platform unit catalogue.
SignalBinding	SignalId	Instrumentation.Signal	SignalId	The binding wires model variable to a real signal.
TwinDivergence	SignalId	Instrumentation.Signal	SignalId	Divergence references the measured signal that disagreed.
TwinModelVersion	ApprovedByUserId	Security.ApplicationUser	ApplicationUserId	Optional approval identity.
SignalBindingCalibration	CalibratedByUserId	Security.ApplicationUser	ApplicationUserId	Optional calibration identity.
TwinRuntimeSession	StartedByUserId	Security.ApplicationUser	ApplicationUserId	Optional runtime starter identity.
	ReviewedByUserId		ApplicationUserId	

FROM TABLE	FK COLUMN	TO TABLE	PK COLUMN	RELATIONSHIP
TwinDivergenceReview		Security.ApplicationUser		Optional review identity.
SimulationScenario	CreatedByUserId	Security.ApplicationUser	ApplicationUserId	Optional scenario author identity.
SimulationRun	RequestedByUserId	Security.ApplicationUser	ApplicationUserId	Optional run requester identity.
WhatIfCase	RequestedByUserId	Security.ApplicationUser	ApplicationUserId	Optional what-if requester identity.

C.6.7.3 Incoming passports from later sectors

The following relationships are declared in later sectors. They are listed here because the DigitalTwin sector becomes evidence for alarms, root-cause reasoning, reporting and compliance.

AlarmManagement.AlarmEvent

FK TwinDivergenceId
 TO DigitalTwin.TwinDivergence
 A later alarm can be raised by, or linked to, a divergence.

RootCause.EvidenceItem

FK TwinDivergenceId
 TO DigitalTwin.TwinDivergence
 RCA can use divergence as evidence.

ReinforcementLearning.TrainingRun

FK TwinModelVersionId
 TO DigitalTwin.TwinModelVersion
 RL training runs can declare which twin version served as environment.

Reporting.ReportSnapshot

FK TwinSnapshotId
 TO DigitalTwin.TwinSnapshot
 Reports can freeze a twin snapshot as evidence.

Compliance.Evidence

FK TwinModelValidationId
 TO DigitalTwin.TwinModelValidation
 Validation campaigns can become compliance evidence.

DigitalTwin passports - every arrow leaves the twin

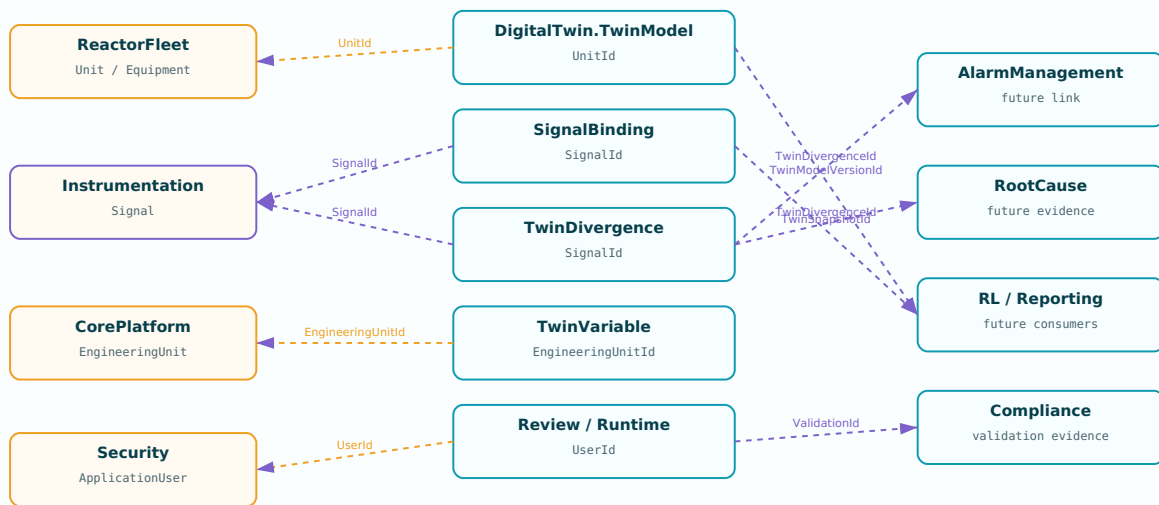


Figure C.6.2 - DigitalTwin passport map. The twin consumes unit, equipment, signal, engineering-unit and user identities; later sectors consume snapshots, divergences, model versions and validation records.

C.6.8 Useful verification queries

These queries validate the core promise of the sector: one physical unit, one model, real signals bound to variables, and visible divergences when reality and model disagree.

```
-- 1. Show every active twin and the unit it mirrors.
SELECT u.Code AS UnitCode,
       tm.Code AS TwinCode,
       tmt.Code AS ModelType,
       tms.Code AS Status,
       tfl.Code AS Fidelity
FROM DigitalTwin.TwinModel AS tm
JOIN ReactorFleet.Unit AS u ON u.UnitId = tm.UnitId
JOIN DigitalTwin.TwinModelType AS tmt ON tmt.TwinModelTypeId = tm.TwinModelTypeId
JOIN DigitalTwin.TwinModelStatus AS tms ON tms.TwinModelStatusId = tm.TwinModelStatusId
JOIN DigitalTwin.TwinFidelityLevel AS tfl ON tfl.TwinFidelityLevelId = tm.TwinFidelityLevelId
WHERE tm.IsDeleted = 0;

-- 2. Trace one model variable to the real signal that feeds it.
SELECT tm.Code AS TwinCode,
       tv.Code AS ModelVariable,
       s.Tag AS SignalTag,
       br.Code AS BindingRole,
       bs.Code AS BindingStatus
FROM DigitalTwin.SignalBinding AS sb
JOIN DigitalTwin.TwinModel AS tm ON tm.TwinModelId = sb.TwinModelId
JOIN DigitalTwin.TwinVariable AS tv ON tv.TwinVariableId = sb.TwinVariableId
JOIN Instrumentation.Signal AS s ON s.SignalId = sb.SignalId
JOIN DigitalTwin.BindingRole AS br ON br.BindingRoleId = sb.BindingRoleId
JOIN DigitalTwin.BindingStatus AS bs ON bs.BindingStatusId = sb.BindingStatusId
WHERE tm.Code = 'NX1-U1-CORE-TWIN';

-- 3. List open divergences with the measured signal and model value.
SELECT td.DetectedAtUtc,
       s.Tag,
       tv.Code AS ModelVariable,
       td.ModeledValue,
       td.MeasuredValue,
       td.DeltaValue,
       ds.Code AS Severity,
       dst.Code AS Status
FROM DigitalTwin.TwinDivergence AS td
JOIN Instrumentation.Signal AS s ON s.SignalId = td.SignalId
LEFT JOIN DigitalTwin.TwinVariable AS tv ON tv.TwinVariableId = td.TwinVariableId
JOIN DigitalTwin.DivergenceSeverity AS ds ON ds.DivergenceSeverityId = td.DivergenceSeverityId
```

```
JOIN DigitalTwin.DivergenceStatus AS dst ON dst.DivergenceStatusId = td.DivergenceStatusId
WHERE dst.Code IN ('OPEN', 'ACKNOWLEDGED')
ORDER BY td.DetectedAtUtc DESC;
```

C.6.9 Boundary and next sector

Honest boundary. This schema describes a NEXUS-1 demonstrator-grade digital twin. It is not a licensed safety-class twin, not a plant control system, and not connected to any real facility. The schema is intentionally read-heavy and audit-heavy: it records models, bindings, snapshots, simulations and divergences, but it does not grant control authority.

The next sector is `AlarmManagement`. It will consume `ReactorFleet.Unit`, `Instrumentation.Signal`, and eventually `DigitalTwin.TwinDivergence` so that model-vs-reality drift can become an explicit alarm source rather than a hidden discrepancy.

Appendix C.7

AlarmManagement Schema

The AlarmManagement sector owns the alarm flood. It records which alarm definitions exist, how they are evaluated, which events fired, who acknowledged them, and which events belonged to the same flood window.

This sector does not decide the root cause. That belongs to `RootCause`. AlarmManagement preserves the raw facts and the operator-facing order: alarm definitions, alarm states, alarm events, flood membership and naive rankings. The later root-cause sector can read these facts, but it must not rewrite them.

Alarm management answers: what lit up, when, for whom, and how did the operator handle it. Root-cause analysis answers: which of those lights were echoes and which node explains the cascade.

C.7.1

Sector purpose

C.7.2

Full table list

C.7.3

Lookup tables

C.7.4

SQL DDL

C.7.5

Indexes and seed examples

C.7.6

ER diagram

C.7.7

Foreign-key mapping

C.7.8

Boundary and next sector

C.7.1 Sector purpose

`AlarmManagement` sits directly above `Instrumentation` and `ReactorFleet`. A definition usually watches a signal, but it is always scoped to a unit. The event keeps the same `UnitId` and `SignalId` so the alarm, historian, digital twin and root-cause engine speak about the same object without translation.

- `AlarmDefinition` is the governed alarmable condition.
- `AlarmCondition`, `AlarmLimit` and `AlarmDeadband` describe how the alarm evaluates.
- `AlarmEvent` is the append-only fact that the alarm fired or returned.
- `AlarmAcknowledgement`, `AlarmComment` and `AlarmOperatorAction` preserve operator response.

- `AlarmFlood` and `AlarmFloodMember` preserve the flood as the operator saw it.
- `AlarmFloodRanking` may store count, priority or arrival rankings, but not the causal verdict.

Design choice. Priority and severity are stored because operators need them. They are not treated as cause. A high-priority downstream alarm remains a downstream alarm. The schema preserves that distinction by keeping alarm flood ranking separate from the later root-cause hypothesis tables.

C.7.2 Full table list

The AlarmManagement sector contains forty-five tables: eighteen typed lookup tables and twenty-seven substantive tables. The split is deliberate. The definitions are governed and mutable; the runtime events are append-only facts; the flood tables capture a window of operator load.

LOOKUP

AlarmManagement.AlarmSeverity

PK `AlarmSeverityId`
FK None

Severity of the alarm condition: information, warning, high, critical, trip.

LOOKUP

AlarmManagement.AlarmPriority

PK `AlarmPriorityId`
FK None

Operator handling priority after rationalization. Priority is not causality.

LOOKUP

AlarmManagement.AlarmState

PK `AlarmStateId`
FK None

Runtime state: active, acknowledged, returned, cleared, suppressed, shelved.

LOOKUP

AlarmManagement.AlarmCategory

PK `AlarmCategoryId`
FK None

Process, safety, security, equipment, instrumentation, digital-twin or advisory category.

LOOKUP

AlarmManagement.AlarmClass

PK `AlarmClassId`
FK None

Annunciation class used for display, filtering and operator workflows.

LOOKUP

AlarmManagement.AlarmConditionType

PK `AlarmConditionTypeId`
FK None

High limit, low limit, rate-of-change, deviation, quality, state-change or composite rule.

LOOKUP

AlarmManagement.AlarmDirection

PK AlarmDirectionId

FK None

Rising, falling, either, state-entered or state-exited.

LOOKUP

AlarmManagement.AlarmLifecycleStatus

PK AlarmLifecycleStatusId

FK None

Draft, rationalized, approved, active, retired or superseded.

LOOKUP

AlarmManagement.AlarmSourceType

PK AlarmSourceTypeId

FK None

Signal, equipment, system, digital twin, manual entry, import or external service source.

LOOKUP

AlarmManagement.AlarmFloodStatus

PK AlarmFloodStatusId

FK None

Detected, open, analyzed, handed-off, closed or archived flood.

LOOKUP

AlarmManagement.AlarmSuppressionReason

PK AlarmSuppressionReasonId

FK None

Reasons for designed suppression: maintenance, startup mode, known nuisance, test, invalid source.

LOOKUP

AlarmManagement.AlarmShelvingReason

PK AlarmShelvingReasonId

FK None

Operator shelving reason with expiry and accountability.

LOOKUP

AlarmManagement.AlarmAckMethod

PK AlarmAckMethodId

FK None

How an alarm was acknowledged: console, mobile, API, batch, system, drill.

LOOKUP

AlarmManagement.NotificationChannel

PK NotificationChannelId

FK None

Console, email, SMS, Teams, API, historian or pager channel.

LOOKUP

AlarmManagement.EscalationStatus

PK EscalationStatusId

FK None

Pending, sent, accepted, failed, cancelled or timed-out escalation.

LOOKUP

AlarmManagement.RationalizationStatus

PK RationalizationStatusId

FK None

Review state of a rationalized alarm: proposed, approved, rejected, expired.

LOOKUP**AlarmManagement.OperatorActionType**

PK OperatorActionTypeId

FK None

Operator response type: observe, diagnose, adjust, isolate, confirm, escalate.

LOOKUP**AlarmManagement.KpiMetricType**

PK KpiMetricTypeId

FK None

Alarm-rate, standing alarm count, flood count, acknowledgement latency and nuisance metrics.

DEFINITION**AlarmManagement.AlarmDefinition**

PK AlarmDefinitionId

FK UnitId, SignalId, EquipmentId

Authoritative definition of one alarmable condition, including source, classification and lifecycle.

DEFINITION**AlarmManagement.AlarmCondition**

PK AlarmConditionId

FK AlarmDefinitionId, AlarmConditionTypeId

One evaluable condition under a definition: high limit, low limit, deviation, quality or composite condition.

DEFINITION**AlarmManagement.AlarmLimit**

PK AlarmLimitId

FK AlarmConditionId, EngineeringUnitId

Numeric limit, limit band and engineering unit used by a condition.

DEFINITION**AlarmManagement.AlarmDeadband**

PK AlarmDeadbandId

FK AlarmConditionId, EngineeringUnitId

Return-to-normal hysteresis, delay and debounce configuration.

DEFINITION**AlarmManagement.AlarmRoutingRule**

PK AlarmRoutingRuleId

FK AlarmDefinitionId, AlarmPriorityId

Routing rule deciding who is notified for a class, priority, unit or time window.

DEFINITION**AlarmManagement.AlarmRoutingRecipient**

PK AlarmRoutingRecipientId

FK AlarmRoutingRuleId, ApplicationUserId

Users, roles or channels that receive notification from a routing rule.

DEFINITION**AlarmManagement.AnnunciatorPanel**

PK AnnunciatorPanelId

FK UnitId

Logical alarm panel displayed by the console.

DEFINITION**AlarmManagement.AnnunciatorTile**

PK AnnunciatorTileId

FK AnnunciatorPanelId, AlarmDefinitionId

Position of an alarm definition on a panel or screen.

RUNTIME**AlarmManagement.AlarmEvent**

PK AlarmEventId

FK AlarmDefinitionId, UnitId, SignalId

Append-only fact that an alarm condition fired, returned or was ignored.

RUNTIME**AlarmManagement.AlarmEventStateHistory**

PK AlarmEventStateHistoryId

FK AlarmEventId, AlarmStateId

State-transition history for an event: active, acknowledged, returned, cleared.

RUNTIME**AlarmManagement.AlarmAcknowledgement**

PK AlarmAcknowledgementId
FK AlarmEventId , AcknowledgedByUserId

Who acknowledged the event, when, how and with which note.

RUNTIME**AlarmManagement.AlarmComment**

PK AlarmCommentId
FK AlarmEventId , CreatedByUserId

Operator or engineer comment attached to an alarm event.

RUNTIME**AlarmManagement.AlarmOperatorAction**

PK AlarmOperatorActionId
FK AlarmEventId , OperatorActionTypeId

Action taken or recommended in response to an alarm.

RUNTIME**AlarmManagement.AlarmEventEvidence**

PK AlarmEventEvidenceId
FK AlarmEventId , MeasurementId

Evidence snapshot tying an event to the signal measurements that supported it.

RUNTIME**AlarmManagement.AlarmNotification**

PK AlarmNotificationId
FK AlarmEventId , NotificationChannelId

Notification attempt generated by a routing or escalation rule.

RUNTIME**AlarmManagement.AlarmEscalation**

PK AlarmEscalationId
FK AlarmEventId , EscalationStatusId

Escalation instance for unacknowledged or critical alarms.

RUNTIME**AlarmManagement.AlarmEscalationAction**

PK AlarmEscalationActionId
FK AlarmEscalationId

Individual action or delivery attempt inside an escalation.

CONTROL**AlarmManagement.AlarmSuppression**

PK AlarmSuppressionId
FK AlarmDefinitionId , SuppressedByUserId

Planned or system-approved suppression with reason and time boundary.

CONTROL**AlarmManagement.AlarmShelving**

PK AlarmShelvingId
FK AlarmDefinitionId , ShelvedByUserId

Temporary operator shelving with expiry and accountability.

CONTROL**AlarmManagement.AlarmStandingOrder**

PK AlarmStandingOrderId
FK AlarmDefinitionId

Approved handling instruction presented when the alarm fires.

FLOOD AlarmManagement.AlarmFlood PK AlarmFloodId FK UnitId , AlarmFloodStatusId Detected alarm flood window for a unit.	FLOOD AlarmManagement.AlarmFloodMember PK AlarmFloodMemberId FK AlarmFloodId , AlarmEventId One alarm event inside a flood, with order, strand and display grouping.
FLOOD AlarmManagement.AlarmFloodSummary PK AlarmFloodSummaryId FK AlarmFloodId Aggregate counts, rates and severity mix for the flood.	FLOOD AlarmManagement.AlarmFloodRanking PK AlarmFloodRankingId FK AlarmFloodId , AlarmDefinitionId Naive ranking by count or priority, explicitly kept separate from root-cause ranking.
GOVERNANCE AlarmManagement.AlarmRationalization PK AlarmRationalizationId FK AlarmDefinitionId , RationalizationStatusId Rationalization record defining purpose, consequence, priority and operator response.	GOVERNANCE AlarmManagement.AlarmRationalizationReview PK AlarmRationalizationReviewId FK AlarmRationalizationId , ReviewedByUserId Review and approval trail for a rationalized alarm.
GOVERNANCE AlarmManagement.AlarmKpiSnapshot PK AlarmKpiSnapshotId FK UnitId , KpiMetricTypeId Periodic KPI row for alarm management performance.	

C.7.3 Lookup tables

Lookup tables make alarm behaviour explicit and queryable. No alarm severity, state, acknowledgement method or suppression reason is stored as a free-text string.

LOOKUP TABLE	TYPICAL SEED CODES	MAIN CONSUMER
AlarmSeverity	INFO , WARN , HIGH , CRITICAL , TRIP	Referenced by AlarmDefinition and AlarmEvent .
AlarmPriority	P1 , P2 , P3 , P4 , ADVISORY	Referenced by routing, rationalization and events.
AlarmState	ACTIVE , ACKNOWLEDGED , RETURNED , CLEARED , SHELVED , SUPPRESSED	Referenced by event state history.
AlarmCategory		Classifies alarm definitions.

LOOKUP TABLE	TYPICAL SEED CODES	MAIN CONSUMER
	PROCESS , SAFETY , EQUIPMENT , INSTRUMENTATION , DIGITAL_TWIN	
AlarmClass	ANNUNCIATOR , EVENT_ONLY , DIAGNOSTIC , ADVISORY	Display and workflow class.
AlarmConditionType	HIGH_LIMIT , LOW_LIMIT , RATE_OF_CHANGE , DEVIATION , QUALITY , COMPOSITE	Defines how AlarmCondition is evaluated.
AlarmDirection	RISING , FALLING , EITHER , STATE_ENTERED , STATE_EXITED	Direction of threshold crossing.
AlarmLifecycleStatus	DRAFT , RATIONALIZED , APPROVED , ACTIVE , RETIRED	Lifecycle of AlarmDefinition .
AlarmSourceType	SIGNAL , EQUIPMENT , SYSTEM , DIGITAL_TWIN , MANUAL , EXTERNAL	Source of an alarm definition or event.
AlarmFloodStatus	DETECTED , OPEN , ANALYZED , HANDED_OFF , CLOSED , ARCHIVED	Lifecycle of AlarmFlood .
AlarmSuppressionReason	MAINTENANCE , STARTUP , TEST , KNOWN_NUISANCE , INVALID_SOURCE	Reason for designed suppression.
AlarmShelvingReason	NUISANCE , DUPLICATE , UNDER_INVESTIGATION , WORK_IN_PROGRESS	Reason for temporary shelving.
AlarmAckMethod	CONSOLE , MOBILE , API , BATCH , SYSTEM	How acknowledgement was made.
NotificationChannel	CONSOLE , EMAIL , SMS , TEAMS , PAGER , API	Delivery channel for notifications.
EscalationStatus	PENDING , SENT , ACCEPTED , FAILED , CANCELLED , TIMED_OUT	Escalation state.
RationalizationStatus	PROPOSED , IN_REVIEW , APPROVED , REJECTED , EXPIRED	Alarm rationalization review state.
OperatorActionType	OBSERVE , DIAGNOSE , ADJUST , ISOLATE , CONFIRM , ESCALATE	Operator response type.
KpiMetricType	ALARM_RATE , FLOOD_COUNT , STANDING_COUNT , ACK_LATENCY , NUISANCE_RATE	Alarm-management KPI metric.

C.7.4 SQL DDL

C.7.4.1 Schema

```
IF SCHEMA_ID(N'AlarmManagement') IS NULL
EXEC(N'CREATE SCHEMA AlarmManagement');
GO
```

C.7.4.2 AlarmManagement lookup tables

```
CREATE TABLE AlarmManagement.AlarmSeverity (
  AlarmSeverityId INT IDENTITY(1,1) NOT NULL,
  Code NVARCHAR(50) NOT NULL,
  Name NVARCHAR(150) NOT NULL,
  Description NVARCHAR(500) NULL,
  DisplayOrder INT NOT NULL
  CONSTRAINT DF_AlarmManagement_AlarmSeverity_DisplayOrder DEFAULT 0,
  IsActive BIT NOT NULL
  CONSTRAINT DF_AlarmManagement_AlarmSeverity_IsActive DEFAULT 1,
  CreatedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_AlarmManagement_AlarmSeverity_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_AlarmManagement_AlarmSeverity_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  RowVersion ROWVERSION NOT NULL,

  CONSTRAINT PK_AlarmManagement_AlarmSeverity
    PRIMARY KEY (AlarmSeverityId),
  CONSTRAINT UQ_AlarmManagement_AlarmSeverity_Code
    UNIQUE (Code)
);
GO

CREATE TABLE AlarmManagement.AlarmPriority (
  AlarmPriorityId INT IDENTITY(1,1) NOT NULL,
  Code NVARCHAR(50) NOT NULL,
  Name NVARCHAR(150) NOT NULL,
  Description NVARCHAR(500) NULL,
  DisplayOrder INT NOT NULL
  CONSTRAINT DF_AlarmManagement_AlarmPriority_DisplayOrder DEFAULT 0,
  IsActive BIT NOT NULL
  CONSTRAINT DF_AlarmManagement_AlarmPriority_IsActive DEFAULT 1,
  CreatedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_AlarmManagement_AlarmPriority_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_AlarmManagement_AlarmPriority_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  RowVersion ROWVERSION NOT NULL,

  CONSTRAINT PK_AlarmManagement_AlarmPriority
    PRIMARY KEY (AlarmPriorityId),
  CONSTRAINT UQ_AlarmManagement_AlarmPriority_Code
    UNIQUE (Code)
);
GO

CREATE TABLE AlarmManagement.AlarmState (
  AlarmStateId INT IDENTITY(1,1) NOT NULL,
  Code NVARCHAR(50) NOT NULL,
  Name NVARCHAR(150) NOT NULL,
  Description NVARCHAR(500) NULL,
  DisplayOrder INT NOT NULL
  CONSTRAINT DF_AlarmManagement_AlarmState_DisplayOrder DEFAULT 0,
  IsActive BIT NOT NULL
  CONSTRAINT DF_AlarmManagement_AlarmState_IsActive DEFAULT 1,
  CreatedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_AlarmManagement_AlarmState_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_AlarmManagement_AlarmState_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  RowVersion ROWVERSION NOT NULL,

  CONSTRAINT PK_AlarmManagement_AlarmState
    PRIMARY KEY (AlarmStateId),
  CONSTRAINT UQ_AlarmManagement_AlarmState_Code
    UNIQUE (Code)
);
GO

CREATE TABLE AlarmManagement.AlarmCategory (
  AlarmCategoryId INT IDENTITY(1,1) NOT NULL,
  Code NVARCHAR(50) NOT NULL,
  Name NVARCHAR(150) NOT NULL,
  Description NVARCHAR(500) NULL,
  DisplayOrder INT NOT NULL
  CONSTRAINT DF_AlarmManagement_AlarmCategory_DisplayOrder DEFAULT 0,
  IsActive BIT NOT NULL
  CONSTRAINT DF_AlarmManagement_AlarmCategory_IsActive DEFAULT 1,
  CreatedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_AlarmManagement_AlarmCategory_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_AlarmManagement_AlarmCategory_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  RowVersion ROWVERSION NOT NULL,

  CONSTRAINT PK_AlarmManagement_AlarmCategory
    PRIMARY KEY (AlarmCategoryId),
  CONSTRAINT UQ_AlarmManagement_AlarmCategory_Code
    UNIQUE (Code)
);
GO

CREATE TABLE AlarmManagement.AlarmClass (
  AlarmClassId INT IDENTITY(1,1) NOT NULL,
  Code NVARCHAR(50) NOT NULL,
  Name NVARCHAR(150) NOT NULL,
```

```

Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_AlarmManagement_AlarmClass_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_AlarmManagement_AlarmClass_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_AlarmManagement_AlarmClass_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_AlarmManagement_AlarmClass_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_AlarmManagement_AlarmClass
PRIMARY KEY (AlarmClassId),
CONSTRAINT UQ_AlarmManagement_AlarmClass_Code
UNIQUE (Code)
);
GO

CREATE TABLE AlarmManagement.AlarmConditionType (
AlarmConditionTypeId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_AlarmManagement_AlarmConditionType_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_AlarmManagement_AlarmConditionType_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_AlarmManagement_AlarmConditionType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_AlarmManagement_AlarmConditionType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_AlarmManagement_AlarmConditionType
PRIMARY KEY (AlarmConditionTypeId),
CONSTRAINT UQ_AlarmManagement_AlarmConditionType_Code
UNIQUE (Code)
);
GO

CREATE TABLE AlarmManagement.AlarmDirection (
AlarmDirectionId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_AlarmManagement_AlarmDirection_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_AlarmManagement_AlarmDirection_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_AlarmManagement_AlarmDirection_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_AlarmManagement_AlarmDirection_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_AlarmManagement_AlarmDirection
PRIMARY KEY (AlarmDirectionId),
CONSTRAINT UQ_AlarmManagement_AlarmDirection_Code
UNIQUE (Code)
);
GO

CREATE TABLE AlarmManagement.AlarmLifecycleStatus (
AlarmLifecycleStatusId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_AlarmManagement_AlarmLifecycleStatus_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_AlarmManagement_AlarmLifecycleStatus_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_AlarmManagement_AlarmLifecycleStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_AlarmManagement_AlarmLifecycleStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_AlarmManagement_AlarmLifecycleStatus
PRIMARY KEY (AlarmLifecycleStatusId),
CONSTRAINT UQ_AlarmManagement_AlarmLifecycleStatus_Code
UNIQUE (Code)
);
GO

CREATE TABLE AlarmManagement.AlarmSourceType (
AlarmSourceTypeId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_AlarmManagement_AlarmSourceType_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_AlarmManagement_AlarmSourceType_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_AlarmManagement_AlarmSourceType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_AlarmManagement_AlarmSourceType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),

```

```

RowVersion    ROWVERSION NOT NULL,

CONSTRAINT PK_AlarmManagement_AlarmSourceType
    PRIMARY KEY (AlarmSourceTypeId),
CONSTRAINT UQ_AlarmManagement_AlarmSourceType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE AlarmManagement.AlarmFloodStatus (
    AlarmFloodStatusId    INT IDENTITY(1,1) NOT NULL,
    Code                  NVARCHAR(50) NOT NULL,
    Name                  NVARCHAR(150) NOT NULL,
    Description            NVARCHAR(500) NULL,
    DisplayOrder          INT NOT NULL
    CONSTRAINT DF_AlarmManagement_AlarmFloodStatus_DisplayOrder DEFAULT 0,
    IsActive              BIT NOT NULL
    CONSTRAINT DF_AlarmManagement_AlarmFloodStatus_IsActive DEFAULT 1,
    CreatedAtUtc           DATETIME2(7) NOT NULL
    CONSTRAINT DF_AlarmManagement_AlarmFloodStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc          DATETIME2(7) NOT NULL
    CONSTRAINT DF_AlarmManagement_AlarmFloodStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion             ROWVERSION NOT NULL,

    CONSTRAINT PK_AlarmManagement_AlarmFloodStatus
        PRIMARY KEY (AlarmFloodStatusId),
    CONSTRAINT UQ_AlarmManagement_AlarmFloodStatus_Code
        UNIQUE (Code)
);
GO

CREATE TABLE AlarmManagement.AlarmSuppressionReason (
    AlarmSuppressionReasonId    INT IDENTITY(1,1) NOT NULL,
    Code                        NVARCHAR(50) NOT NULL,
    Name                        NVARCHAR(150) NOT NULL,
    Description                  NVARCHAR(500) NULL,
    DisplayOrder                INT NOT NULL
    CONSTRAINT DF_AlarmManagement_AlarmSuppressionReason_DisplayOrder DEFAULT 0,
    IsActive                    BIT NOT NULL
    CONSTRAINT DF_AlarmManagement_AlarmSuppressionReason_IsActive DEFAULT 1,
    CreatedAtUtc                 DATETIME2(7) NOT NULL
    CONSTRAINT DF_AlarmManagement_AlarmSuppressionReason_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc                DATETIME2(7) NOT NULL
    CONSTRAINT DF_AlarmManagement_AlarmSuppressionReason_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion                   ROWVERSION NOT NULL,

    CONSTRAINT PK_AlarmManagement_AlarmSuppressionReason
        PRIMARY KEY (AlarmSuppressionReasonId),
    CONSTRAINT UQ_AlarmManagement_AlarmSuppressionReason_Code
        UNIQUE (Code)
);
GO

CREATE TABLE AlarmManagement.AlarmShelvingReason (
    AlarmShelvingReasonId    INT IDENTITY(1,1) NOT NULL,
    Code                     NVARCHAR(50) NOT NULL,
    Name                     NVARCHAR(150) NOT NULL,
    Description               NVARCHAR(500) NULL,
    DisplayOrder             INT NOT NULL
    CONSTRAINT DF_AlarmManagement_AlarmShelvingReason_DisplayOrder DEFAULT 0,
    IsActive                  BIT NOT NULL
    CONSTRAINT DF_AlarmManagement_AlarmShelvingReason_IsActive DEFAULT 1,
    CreatedAtUtc              DATETIME2(7) NOT NULL
    CONSTRAINT DF_AlarmManagement_AlarmShelvingReason_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc             DATETIME2(7) NOT NULL
    CONSTRAINT DF_AlarmManagement_AlarmShelvingReason_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion                ROWVERSION NOT NULL,

    CONSTRAINT PK_AlarmManagement_AlarmShelvingReason
        PRIMARY KEY (AlarmShelvingReasonId),
    CONSTRAINT UQ_AlarmManagement_AlarmShelvingReason_Code
        UNIQUE (Code)
);
GO

CREATE TABLE AlarmManagement.AlarmAckMethod (
    AlarmAckMethodId    INT IDENTITY(1,1) NOT NULL,
    Code                NVARCHAR(50) NOT NULL,
    Name                NVARCHAR(150) NOT NULL,
    Description          NVARCHAR(500) NULL,
    DisplayOrder        INT NOT NULL
    CONSTRAINT DF_AlarmManagement_AlarmAckMethod_DisplayOrder DEFAULT 0,
    IsActive            BIT NOT NULL
    CONSTRAINT DF_AlarmManagement_AlarmAckMethod_IsActive DEFAULT 1,
    CreatedAtUtc        DATETIME2(7) NOT NULL
    CONSTRAINT DF_AlarmManagement_AlarmAckMethod_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc       DATETIME2(7) NOT NULL
    CONSTRAINT DF_AlarmManagement_AlarmAckMethod_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion          ROWVERSION NOT NULL,

    CONSTRAINT PK_AlarmManagement_AlarmAckMethod
        PRIMARY KEY (AlarmAckMethodId),
    CONSTRAINT UQ_AlarmManagement_AlarmAckMethod_Code
        UNIQUE (Code)
);
GO

```

```

CREATE TABLE AlarmManagement.NotificationChannel (
    NotificationChannelId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_AlarmManagement_NotificationChannel_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_AlarmManagement_NotificationChannel_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_AlarmManagement_NotificationChannel_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_AlarmManagement_NotificationChannel_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_AlarmManagement_NotificationChannel
    PRIMARY KEY (NotificationChannelId),
    CONSTRAINT UQ_AlarmManagement_NotificationChannel_Code
    UNIQUE (Code)
);
GO

CREATE TABLE AlarmManagement.EscalationStatus (
    EscalationStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_AlarmManagement_EscalationStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_AlarmManagement_EscalationStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_AlarmManagement_EscalationStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_AlarmManagement_EscalationStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_AlarmManagement_EscalationStatus
    PRIMARY KEY (EscalationStatusId),
    CONSTRAINT UQ_AlarmManagement_EscalationStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE AlarmManagement.RationalizationStatus (
    RationalizationStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_AlarmManagement_RationalizationStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_AlarmManagement_RationalizationStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_AlarmManagement_RationalizationStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_AlarmManagement_RationalizationStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_AlarmManagement_RationalizationStatus
    PRIMARY KEY (RationalizationStatusId),
    CONSTRAINT UQ_AlarmManagement_RationalizationStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE AlarmManagement.OperatorActionType (
    OperatorActionTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_AlarmManagement_OperatorActionType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_AlarmManagement_OperatorActionType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_AlarmManagement_OperatorActionType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_AlarmManagement_OperatorActionType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_AlarmManagement_OperatorActionType
    PRIMARY KEY (OperatorActionTypeId),
    CONSTRAINT UQ_AlarmManagement_OperatorActionType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE AlarmManagement.KpiMetricType (
    KpiMetricTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_AlarmManagement_KpiMetricType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL

```

```

        CONSTRAINT DF_AlarmManagement_KpiMetricType_IsActive DEFAULT 1,
        CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_AlarmManagement_KpiMetricType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_AlarmManagement_KpiMetricType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_AlarmManagement_KpiMetricType
    PRIMARY KEY (KpiMetricTypeId),
    CONSTRAINT UQ_AlarmManagement_KpiMetricType_Code
    UNIQUE (Code)
);
GO

```

C.7.4.3 Alarm definitions and annunciator layout

```

CREATE TABLE AlarmManagement.AlarmDefinition (
    AlarmDefinitionId INT IDENTITY(1,1) NOT NULL,
    UnitId INT NOT NULL,
    SignalId INT NULL,
    EquipmentId INT NULL,
    PlantSystemId INT NULL,
    AlarmSeverityId INT NOT NULL,
    AlarmPriorityId INT NOT NULL,
    AlarmCategoryId INT NOT NULL,
    AlarmClassId INT NOT NULL,
    AlarmLifecycleStatusId INT NOT NULL,
    AlarmSourceTypeId INT NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(250) NOT NULL,
    Description NVARCHAR(1000) NULL,
    OperatorMessage NVARCHAR(1000) NULL,
    SafetySignificant BIT NOT NULL CONSTRAINT DF_AlarmManagement_AlarmDefinition_SafetySignificant DEFAULT 0,
    RequiresAcknowledgement BIT NOT NULL CONSTRAINT DF_AlarmManagement_AlarmDefinition_RequiresAcknowledgement DEFAULT 1,
    IsLatched BIT NOT NULL CONSTRAINT DF_AlarmManagement_AlarmDefinition_IsLatched DEFAULT 0,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmDefinition_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmDefinition_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_AlarmManagement_AlarmDefinition_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_AlarmManagement_AlarmDefinition PRIMARY KEY (AlarmDefinitionId),
    CONSTRAINT UQ_AlarmManagement_AlarmDefinition_Unit_Code UNIQUE (UnitId, Code),
    CONSTRAINT FK_AlarmManagement_AlarmDefinition_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_AlarmManagement_AlarmDefinition_Signal FOREIGN KEY (SignalId) REFERENCES Instrumentation.Signal(SignalId),
    CONSTRAINT FK_AlarmManagement_AlarmDefinition_Equipment FOREIGN KEY (EquipmentId) REFERENCES
ReactorFleet.Equipment(EquipmentId),
    CONSTRAINT FK_AlarmManagement_AlarmDefinition_PlantSystem FOREIGN KEY (PlantSystemId) REFERENCES
ReactorFleet.PlantSystem(PlantSystemId),
    CONSTRAINT FK_AlarmManagement_AlarmDefinition_AlarmSeverity FOREIGN KEY (AlarmSeverityId) REFERENCES
AlarmManagement.AlarmSeverity(AlarmSeverityId),
    CONSTRAINT FK_AlarmManagement_AlarmDefinition_AlarmPriority FOREIGN KEY (AlarmPriorityId) REFERENCES
AlarmManagement.AlarmPriority(AlarmPriorityId),
    CONSTRAINT FK_AlarmManagement_AlarmDefinition_AlarmCategory FOREIGN KEY (AlarmCategoryId) REFERENCES
AlarmManagement.AlarmCategory(AlarmCategoryId),
    CONSTRAINT FK_AlarmManagement_AlarmDefinition_AlarmClass FOREIGN KEY (AlarmClassId) REFERENCES
AlarmManagement.AlarmClass(AlarmClassId),
    CONSTRAINT FK_AlarmManagement_AlarmDefinition_AlarmLifecycleStatus FOREIGN KEY (AlarmLifecycleStatusId) REFERENCES
AlarmManagement.AlarmLifecycleStatus(AlarmLifecycleStatusId),
    CONSTRAINT FK_AlarmManagement_AlarmDefinition_AlarmSourceType FOREIGN KEY (AlarmSourceTypeId) REFERENCES
AlarmManagement.AlarmSourceType(AlarmSourceTypeId)
);
GO

CREATE TABLE AlarmManagement.AlarmCondition (
    AlarmConditionId INT IDENTITY(1,1) NOT NULL,
    AlarmDefinitionId INT NOT NULL,
    AlarmConditionTypeId INT NOT NULL,
    AlarmDirectionId INT NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    ExpressionText NVARCHAR(1000) NULL,
    EvaluationOrder INT NOT NULL CONSTRAINT DF_AlarmManagement_AlarmCondition_EvaluationOrder DEFAULT 0,
    IsPrimaryCondition BIT NOT NULL CONSTRAINT DF_AlarmManagement_AlarmCondition_IsPrimaryCondition DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmCondition_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmCondition_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_AlarmManagement_AlarmCondition_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_AlarmManagement_AlarmCondition PRIMARY KEY (AlarmConditionId),
    CONSTRAINT FK_AlarmManagement_AlarmCondition_AlarmDefinition FOREIGN KEY (AlarmDefinitionId) REFERENCES
AlarmManagement.AlarmDefinition(AlarmDefinitionId),
    CONSTRAINT FK_AlarmManagement_AlarmCondition_AlarmConditionType FOREIGN KEY (AlarmConditionTypeId) REFERENCES
AlarmManagement.AlarmConditionType(AlarmConditionTypeId),
    CONSTRAINT FK_AlarmManagement_AlarmCondition_AlarmDirection FOREIGN KEY (AlarmDirectionId) REFERENCES
AlarmManagement.AlarmDirection(AlarmDirectionId)
);
GO

CREATE TABLE AlarmManagement.AlarmLimit (
    AlarmLimitId INT IDENTITY(1,1) NOT NULL,

```



```

AlarmConditionId INT NOT NULL,
EngineeringUnitId INT NULL,
LimitValue DECIMAL(18,6) NOT NULL,
LimitLowValue DECIMAL(18,6) NULL,
LimitHighValue DECIMAL(18,6) NULL,
EffectiveFromUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmLimit_EffectiveFromUtc DEFAULT SYSUTCDATETIME(),
EffectiveToUtc DATETIME2(7) NULL,
CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmLimit_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
CreatedBy NVARCHAR(100) NOT NULL,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_AlarmManagement_AlarmLimit PRIMARY KEY (AlarmLimitId),
CONSTRAINT FK_AlarmManagement_AlarmLimit_AlarmCondition FOREIGN KEY (AlarmConditionId) REFERENCES
AlarmManagement.AlarmCondition(AlarmConditionId),
CONSTRAINT FK_AlarmManagement_AlarmLimit_EngineeringUnit FOREIGN KEY (EngineeringUnitId) REFERENCES
CorePlatform.EngineeringUnit(EngineeringUnitId)
);
GO

CREATE TABLE AlarmManagement.AlarmDeadband (
AlarmDeadbandId INT IDENTITY(1,1) NOT NULL,
AlarmConditionId INT NOT NULL,
EngineeringUnitId INT NULL,
DeadbandValue DECIMAL(18,6) NULL,
ReturnDelaySeconds INT NOT NULL CONSTRAINT DF_AlarmManagement_AlarmDeadband_ReturnDelaySeconds DEFAULT 0,
OnDelaySeconds INT NOT NULL CONSTRAINT DF_AlarmManagement_AlarmDeadband_OnDelaySeconds DEFAULT 0,
OffDelaySeconds INT NOT NULL CONSTRAINT DF_AlarmManagement_AlarmDeadband_OffDelaySeconds DEFAULT 0,
CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmDeadband_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
CreatedBy NVARCHAR(100) NOT NULL,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_AlarmManagement_AlarmDeadband PRIMARY KEY (AlarmDeadbandId),
CONSTRAINT FK_AlarmManagement_AlarmDeadband_AlarmCondition FOREIGN KEY (AlarmConditionId) REFERENCES
AlarmManagement.AlarmCondition(AlarmConditionId),
CONSTRAINT FK_AlarmManagement_AlarmDeadband_EngineeringUnit FOREIGN KEY (EngineeringUnitId) REFERENCES
CorePlatform.EngineeringUnit(EngineeringUnitId)
);
GO

CREATE TABLE AlarmManagement.AlarmRoutingRule (
AlarmRoutingRuleId INT IDENTITY(1,1) NOT NULL,
AlarmDefinitionId INT NULL,
UnitId INT NULL,
AlarmPriorityId INT NULL,
Name NVARCHAR(150) NOT NULL,
AppliesWhenJson NVARCHAR(MAX) NULL,
IsActive BIT NOT NULL CONSTRAINT DF_AlarmManagement_AlarmRoutingRule_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmRoutingRule_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
CreatedBy NVARCHAR(100) NOT NULL,
ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmRoutingRule_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedBy NVARCHAR(100) NOT NULL,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_AlarmManagement_AlarmRoutingRule PRIMARY KEY (AlarmRoutingRuleId),
CONSTRAINT FK_AlarmManagement_AlarmRoutingRule_AlarmDefinition FOREIGN KEY (AlarmDefinitionId) REFERENCES
AlarmManagement.AlarmDefinition(AlarmDefinitionId),
CONSTRAINT FK_AlarmManagement_AlarmRoutingRule_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
CONSTRAINT FK_AlarmManagement_AlarmRoutingRule_AlarmPriority FOREIGN KEY (AlarmPriorityId) REFERENCES
AlarmManagement.AlarmPriority(AlarmPriorityId)
);
GO

CREATE TABLE AlarmManagement.AlarmRoutingRecipient (
AlarmRoutingRecipientId INT IDENTITY(1,1) NOT NULL,
AlarmRoutingRuleId INT NOT NULL,
ApplicationUserId INT NULL,
ApplicationRoleId INT NULL,
NotificationChannelId INT NOT NULL,
ExternalAddress NVARCHAR(320) NULL,
DisplayOrder INT NOT NULL CONSTRAINT DF_AlarmManagement_AlarmRoutingRecipient_DisplayOrder DEFAULT 0,
CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmRoutingRecipient_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
CreatedBy NVARCHAR(100) NOT NULL,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_AlarmManagement_AlarmRoutingRecipient PRIMARY KEY (AlarmRoutingRecipientId),
CONSTRAINT FK_AlarmManagement_AlarmRoutingRecipient_AlarmRoutingRule FOREIGN KEY (AlarmRoutingRuleId) REFERENCES
AlarmManagement.AlarmRoutingRule(AlarmRoutingRuleId),
CONSTRAINT FK_AlarmManagement_AlarmRoutingRecipient_ApplicationUser FOREIGN KEY (ApplicationUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId),
CONSTRAINT FK_AlarmManagement_AlarmRoutingRecipient_ApplicationRole FOREIGN KEY (ApplicationRoleId) REFERENCES
Security.ApplicationRole(ApplicationRoleId),
CONSTRAINT FK_AlarmManagement_AlarmRoutingRecipient_NotificationChannel FOREIGN KEY (NotificationChannelId) REFERENCES
AlarmManagement.NotificationChannel(NotificationChannelId)
);
GO

CREATE TABLE AlarmManagement.AnnunciatorPanel (
AnnunciatorPanelId INT IDENTITY(1,1) NOT NULL,
UnitId INT NOT NULL,
Code NVARCHAR(80) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AnnunciatorPanel_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
CreatedBy NVARCHAR(100) NOT NULL,
ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AnnunciatorPanel_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedBy NVARCHAR(100) NOT NULL,
IsDeleted BIT NOT NULL CONSTRAINT DF_AlarmManagement_AnnunciatorPanel_IsDeleted DEFAULT 0,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_AlarmManagement_AnnunciatorPanel PRIMARY KEY (AnnunciatorPanelId),

```



```

CONSTRAINT UQ_AlarmManagement_AnnunciatorPanel_Unit_Code UNIQUE (UnitId, Code),
CONSTRAINT FK_AlarmManagement_AnnunciatorPanel_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId)
);
GO

CREATE TABLE AlarmManagement.AnnunciatorTile (
    AnnunciatorTileId INT IDENTITY(1,1) NOT NULL,
    AnnunciatorPanelId INT NOT NULL,
    AlarmDefinitionId INT NOT NULL,
    RowNumber INT NOT NULL,
    ColumnNumber INT NOT NULL,
    Label NVARCHAR(150) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AnnunciatorTile_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_AlarmManagement_AnnunciatorTile PRIMARY KEY (AnnunciatorTileId),
    CONSTRAINT UQ_AlarmManagement_AnnunciatorTile_Panel_Position UNIQUE (AnnunciatorPanelId, RowNumber, ColumnNumber),
    CONSTRAINT FK_AlarmManagement_AnnunciatorTile_AnnunciatorPanel FOREIGN KEY (AnnunciatorPanelId) REFERENCES
AlarmManagement.AnnunciatorPanel(AnnunciatorPanelId),
    CONSTRAINT FK_AlarmManagement_AnnunciatorTile_AlarmDefinition FOREIGN KEY (AlarmDefinitionId) REFERENCES
AlarmManagement.AlarmDefinition(AlarmDefinitionId)
);
GO

```

C.7.4.4 Runtime alarm events and operator response

```

CREATE TABLE AlarmManagement.AlarmEvent (
    AlarmEventId BIGINT IDENTITY(1,1) NOT NULL,
    AlarmDefinitionId INT NOT NULL,
    UnitId INT NOT NULL,
    SignalId INT NULL,
    EquipmentId INT NULL,
    AlarmSeverityId INT NOT NULL,
    AlarmPriorityId INT NOT NULL,
    AlarmSourceTypeId INT NOT NULL,
    RaisedAtUtc DATETIME2(7) NOT NULL,
    ReturnedAtUtc DATETIME2(7) NULL,
    ClearedAtUtc DATETIME2(7) NULL,
    SourceValue DECIMAL(18,6) NULL,
    ThresholdValue DECIMAL(18,6) NULL,
    Message NVARCHAR(1000) NOT NULL,
    TagSnapshot NVARCHAR(80) NULL,
    IsDrill BIT NOT NULL CONSTRAINT DF_AlarmManagement_AlarmEvent_IsDrill DEFAULT 0,
    CorrelationKey NVARCHAR(120) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmEvent_CreatedAtUtc DEFAULT SYSUTCDATETIME(),

    CONSTRAINT PK_AlarmManagement_AlarmEvent PRIMARY KEY (AlarmEventId),
    CONSTRAINT FK_AlarmManagement_AlarmEvent_AlarmDefinition FOREIGN KEY (AlarmDefinitionId) REFERENCES
AlarmManagement.AlarmDefinition(AlarmDefinitionId),
    CONSTRAINT FK_AlarmManagement_AlarmEvent_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_AlarmManagement_AlarmEvent_Signal FOREIGN KEY (SignalId) REFERENCES Instrumentation.Signal(SignalId),
    CONSTRAINT FK_AlarmManagement_AlarmEvent_Equipment FOREIGN KEY (EquipmentId) REFERENCES ReactorFleet.Equipment(EquipmentId),
    CONSTRAINT FK_AlarmManagement_AlarmEvent_AlarmSeverity FOREIGN KEY (AlarmSeverityId) REFERENCES
AlarmManagement.AlarmSeverity(AlarmSeverityId),
    CONSTRAINT FK_AlarmManagement_AlarmEvent_AlarmPriority FOREIGN KEY (AlarmPriorityId) REFERENCES
AlarmManagement.AlarmPriority(AlarmPriorityId),
    CONSTRAINT FK_AlarmManagement_AlarmEvent_AlarmSourceType FOREIGN KEY (AlarmSourceTypeId) REFERENCES
AlarmManagement.AlarmSourceType(AlarmSourceTypeId)
);
GO

CREATE TABLE AlarmManagement.AlarmEventStateHistory (
    AlarmEventStateHistoryId BIGINT IDENTITY(1,1) NOT NULL,
    AlarmEventId BIGINT NOT NULL,
    AlarmStateId INT NOT NULL,
    ChangedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmEventStateHistory_ChangedAtUtc DEFAULT
SYSUTCDATETIME(),
    ChangedByUserId INT NULL,
    Reason NVARCHAR(500) NULL,

    CONSTRAINT PK_AlarmManagement_AlarmEventStateHistory PRIMARY KEY (AlarmEventStateHistoryId),
    CONSTRAINT FK_AlarmManagement_AlarmEventStateHistory_AlarmEvent FOREIGN KEY (AlarmEventId) REFERENCES
AlarmManagement.AlarmEvent(AlarmEventId),
    CONSTRAINT FK_AlarmManagement_AlarmEventStateHistory_AlarmState FOREIGN KEY (AlarmStateId) REFERENCES
AlarmManagement.AlarmState(AlarmStateId),
    CONSTRAINT FK_AlarmManagement_AlarmEventStateHistory_ChangedByUser FOREIGN KEY (ChangedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE AlarmManagement.AlarmAcknowledgement (
    AlarmAcknowledgementId BIGINT IDENTITY(1,1) NOT NULL,
    AlarmEventId BIGINT NOT NULL,
    AlarmAckMethodId INT NOT NULL,
    AcknowledgedByUserId INT NOT NULL,
    AcknowledgedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmAcknowledgement_AcknowledgedAtUtc DEFAULT
SYSUTCDATETIME(),
    AcknowledgementNote NVARCHAR(1000) NULL,

    CONSTRAINT PK_AlarmManagement_AlarmAcknowledgement PRIMARY KEY (AlarmAcknowledgementId),
    CONSTRAINT FK_AlarmManagement_AlarmAcknowledgement_AlarmEvent FOREIGN KEY (AlarmEventId) REFERENCES
AlarmManagement.AlarmEvent(AlarmEventId),
    CONSTRAINT FK_AlarmManagement_AlarmAcknowledgement_AlarmAckMethod FOREIGN KEY (AlarmAckMethodId) REFERENCES
AlarmManagement.AlarmAckMethod(AlarmAckMethodId),

```

```

        CONSTRAINT FK_AlarmManagement_AlarmAcknowledgement_AcknowledgedByUser FOREIGN KEY (AcknowledgedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
    );
GO

CREATE TABLE AlarmManagement.AlarmComment (
    AlarmCommentId BIGINT IDENTITY(1,1) NOT NULL,
    AlarmEventId BIGINT NOT NULL,
    CreatedByUserId INT NOT NULL,
    CommentText NVARCHAR(2000) NOT NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmComment_CreatedAtUtc DEFAULT SYSUTCDATETIME(),

    CONSTRAINT PK_AlarmManagement_AlarmComment PRIMARY KEY (AlarmCommentId),
    CONSTRAINT FK_AlarmManagement_AlarmComment_AlarmEvent FOREIGN KEY (AlarmEventId) REFERENCES
AlarmManagement.AlarmEvent(AlarmEventId),
    CONSTRAINT FK_AlarmManagement_AlarmComment_CreatedByUser FOREIGN KEY (CreatedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE AlarmManagement.AlarmOperatorAction (
    AlarmOperatorActionId BIGINT IDENTITY(1,1) NOT NULL,
    AlarmEventId BIGINT NOT NULL,
    OperatorActionTypeId INT NOT NULL,
    PerformedByUserId INT NOT NULL,
    ActionAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmOperatorAction_ActionAtUtc DEFAULT
SYSUTCDATETIME(),
    ActionText NVARCHAR(1000) NOT NULL,
    IsRecommended BIT NOT NULL CONSTRAINT DF_AlarmManagement_AlarmOperatorAction_IsRecommended DEFAULT 0,

    CONSTRAINT PK_AlarmManagement_AlarmOperatorAction PRIMARY KEY (AlarmOperatorActionId),
    CONSTRAINT FK_AlarmManagement_AlarmOperatorAction_AlarmEvent FOREIGN KEY (AlarmEventId) REFERENCES
AlarmManagement.AlarmEvent(AlarmEventId),
    CONSTRAINT FK_AlarmManagement_AlarmOperatorAction_OperatorActionType FOREIGN KEY (OperatorActionTypeId) REFERENCES
AlarmManagement.OperatorActionType(OperatorActionTypeId),
    CONSTRAINT FK_AlarmManagement_AlarmOperatorAction_PerformedByUser FOREIGN KEY (PerformedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE AlarmManagement.AlarmEventEvidence (
    AlarmEventEvidenceId BIGINT IDENTITY(1,1) NOT NULL,
    AlarmEventId BIGINT NOT NULL,
    MeasurementId BIGINT NOT NULL,
    SignalId INT NOT NULL,
    EvidenceAtUtc DATETIME2(7) NOT NULL,
    EvidenceValue DECIMAL(18,6) NULL,
    EvidenceQuality NVARCHAR(50) NULL,
    EvidenceNote NVARCHAR(500) NULL,

    CONSTRAINT PK_AlarmManagement_AlarmEventEvidence PRIMARY KEY (AlarmEventEvidenceId),
    CONSTRAINT FK_AlarmManagement_AlarmEventEvidence_AlarmEvent FOREIGN KEY (AlarmEventId) REFERENCES
AlarmManagement.AlarmEvent(AlarmEventId),
    CONSTRAINT FK_AlarmManagement_AlarmEventEvidence_Measurement FOREIGN KEY (MeasurementId) REFERENCES
Instrumentation.Measurement(MeasurementId),
    CONSTRAINT FK_AlarmManagement_AlarmEventEvidence_Signal FOREIGN KEY (SignalId) REFERENCES Instrumentation.Signal(SignalId)
);
GO

CREATE TABLE AlarmManagement.AlarmNotification (
    AlarmNotificationId BIGINT IDENTITY(1,1) NOT NULL,
    AlarmEventId BIGINT NOT NULL,
    NotificationChannelId INT NOT NULL,
    RecipientUserId INT NOT NULL,
    RecipientAddress NVARCHAR(320) NULL,
    SentAtUtc DATETIME2(7) NULL,
    DeliveryStatus NVARCHAR(60) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmNotification_DeliveryStatus DEFAULT N'PENDING',
    ErrorText NVARCHAR(1000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmNotification_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),

    CONSTRAINT PK_AlarmManagement_AlarmNotification PRIMARY KEY (AlarmNotificationId),
    CONSTRAINT FK_AlarmManagement_AlarmNotification_AlarmEvent FOREIGN KEY (AlarmEventId) REFERENCES
AlarmManagement.AlarmEvent(AlarmEventId),
    CONSTRAINT FK_AlarmManagement_AlarmNotification_NotificationChannel FOREIGN KEY (NotificationChannelId) REFERENCES
AlarmManagement.NotificationChannel(NotificationChannelId),
    CONSTRAINT FK_AlarmManagement_AlarmNotification_RecipientUser FOREIGN KEY (RecipientUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE AlarmManagement.AlarmEscalation (
    AlarmEscalationId BIGINT IDENTITY(1,1) NOT NULL,
    AlarmEventId BIGINT NOT NULL,
    EscalationStatusId INT NOT NULL,
    EscalationLevel INT NOT NULL CONSTRAINT DF_AlarmManagement_AlarmEscalation_EscalationLevel DEFAULT 1,
    StartedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmEscalation_StartedAtUtc DEFAULT SYSUTCDATETIME(),
    CompletedAtUtc DATETIME2(7) NULL,
    Summary NVARCHAR(1000) NULL,

    CONSTRAINT PK_AlarmManagement_AlarmEscalation PRIMARY KEY (AlarmEscalationId),
    CONSTRAINT FK_AlarmManagement_AlarmEscalation_AlarmEvent FOREIGN KEY (AlarmEventId) REFERENCES
AlarmManagement.AlarmEvent(AlarmEventId),
    CONSTRAINT FK_AlarmManagement_AlarmEscalation_EscalationStatus FOREIGN KEY (EscalationStatusId) REFERENCES
AlarmManagement.EscalationStatus(EscalationStatusId)
);
GO

CREATE TABLE AlarmManagement.AlarmEscalationAction (
    AlarmEscalationActionId BIGINT IDENTITY(1,1) NOT NULL,
    AlarmEscalationId BIGINT NOT NULL,

```

```

NotificationChannelId INT NULL,
TargetUserId INT NULL,
ActionAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmEscalationAction_ActionAtUtc DEFAULT
SYSUTCDATETIME(),
ActionResult NVARCHAR(100) NOT NULL,
ActionNote NVARCHAR(1000) NULL,

CONSTRAINT PK_AlarmManagement_AlarmEscalationAction PRIMARY KEY (AlarmEscalationActionId),
CONSTRAINT FK_AlarmManagement_AlarmEscalationAction_AlarmEscalation FOREIGN KEY (AlarmEscalationId) REFERENCES
AlarmManagement.AlarmEscalation(AlarmEscalationId),
CONSTRAINT FK_AlarmManagement_AlarmEscalationAction_NotificationChannel FOREIGN KEY (NotificationChannelId) REFERENCES
AlarmManagement.NotificationChannel(NotificationChannelId),
CONSTRAINT FK_AlarmManagement_AlarmEscalationAction_TargetUser FOREIGN KEY (TargetUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

```

C.7.4.5 Suppression, shelving, flood windows and rationalization

```

CREATE TABLE AlarmManagement.AlarmSuppression (
    AlarmSuppressionId INT IDENTITY(1,1) NOT NULL,
    AlarmDefinitionId INT NOT NULL,
    AlarmSuppressionReasonId INT NOT NULL,
    SuppressedByUserId INT NOT NULL,
    StartsAtUtc DATETIME2(7) NOT NULL,
    EndsAtUtc DATETIME2(7) NULL,
    ReasonText NVARCHAR(1000) NULL,
    IsActive BIT NOT NULL CONSTRAINT DF_AlarmManagement_AlarmSuppression_IsActive DEFAULT 1,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_AlarmManagement_AlarmSuppression PRIMARY KEY (AlarmSuppressionId),
    CONSTRAINT FK_AlarmManagement_AlarmSuppression_AlarmDefinition FOREIGN KEY (AlarmDefinitionId) REFERENCES
AlarmManagement.AlarmDefinition(AlarmDefinitionId),
    CONSTRAINT FK_AlarmManagement_AlarmSuppression_AlarmSuppressionReason FOREIGN KEY (AlarmSuppressionReasonId) REFERENCES
AlarmManagement.AlarmSuppressionReason(AlarmSuppressionReasonId),
    CONSTRAINT FK_AlarmManagement_AlarmSuppression_SuppressedByUser FOREIGN KEY (SuppressedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE AlarmManagement.AlarmShelving (
    AlarmShelvingId INT IDENTITY(1,1) NOT NULL,
    AlarmDefinitionId INT NOT NULL,
    AlarmShelvingReasonId INT NOT NULL,
    ShelvedByUserId INT NOT NULL,
    ShelvedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmShelving_ShelvedAtUtc DEFAULT SYSUTCDATETIME(),
    ExpiresAtUtc DATETIME2(7) NOT NULL,
    UnshelvedAtUtc DATETIME2(7) NULL,
    ReasonText NVARCHAR(1000) NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_AlarmManagement_AlarmShelving PRIMARY KEY (AlarmShelvingId),
    CONSTRAINT FK_AlarmManagement_AlarmShelving_AlarmDefinition FOREIGN KEY (AlarmDefinitionId) REFERENCES
AlarmManagement.AlarmDefinition(AlarmDefinitionId),
    CONSTRAINT FK_AlarmManagement_AlarmShelving_AlarmShelvingReason FOREIGN KEY (AlarmShelvingReasonId) REFERENCES
AlarmManagement.AlarmShelvingReason(AlarmShelvingReasonId),
    CONSTRAINT FK_AlarmManagement_AlarmShelving_ShelvedByUser FOREIGN KEY (ShelvedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE AlarmManagement.AlarmStandingOrder (
    AlarmStandingOrderId INT IDENTITY(1,1) NOT NULL,
    AlarmDefinitionId INT NOT NULL,
    InstructionText NVARCHAR(2000) NOT NULL,
    EffectiveFromUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmStandingOrder_EffectiveFromUtc DEFAULT
SYSUTCDATETIME(),
    EffectiveToUtc DATETIME2(7) NULL,
    ApprovedByUserId INT NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmStandingOrder_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_AlarmManagement_AlarmStandingOrder PRIMARY KEY (AlarmStandingOrderId),
    CONSTRAINT FK_AlarmManagement_AlarmStandingOrder_AlarmDefinition FOREIGN KEY (AlarmDefinitionId) REFERENCES
AlarmManagement.AlarmDefinition(AlarmDefinitionId),
    CONSTRAINT FK_AlarmManagement_AlarmStandingOrder_ApprovedByUser FOREIGN KEY (ApprovedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE AlarmManagement.AlarmFlood (
    AlarmFloodId BIGINT IDENTITY(1,1) NOT NULL,
    UnitId INT NOT NULL,
    AlarmFloodStatusId INT NOT NULL,
    StartedAtUtc DATETIME2(7) NOT NULL,
    EndedAtUtc DATETIME2(7) NULL,
    DetectionRule NVARCHAR(200) NOT NULL,
    AlarmCount INT NOT NULL CONSTRAINT DF_AlarmManagement_AlarmFlood_AlarmCount DEFAULT 0,
    PeakRatePerMinute DECIMAL(10,2) NULL,
    Summary NVARCHAR(1000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmFlood_CreatedAtUtc DEFAULT SYSUTCDATETIME(),

    CONSTRAINT PK_AlarmManagement_AlarmFlood PRIMARY KEY (AlarmFloodId),

```

```

        CONSTRAINT FK_AlarmManagement_AlarmFlood_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
        CONSTRAINT FK_AlarmManagement_AlarmFlood_AlarmFloodStatus FOREIGN KEY (AlarmFloodStatusId) REFERENCES
AlarmManagement.AlarmFloodStatus(AlarmFloodStatusId)
);
GO

CREATE TABLE AlarmManagement.AlarmFloodMember (
    AlarmFloodMemberId BIGINT IDENTITY(1,1) NOT NULL,
    AlarmFloodId BIGINT NOT NULL,
    AlarmEventId BIGINT NOT NULL,
    ArrivalOrder INT NOT NULL,
    GroupLabel NVARCHAR(150) NULL,
    StrandLabel NVARCHAR(150) NULL,
    IsFirstInFlood BIT NOT NULL CONSTRAINT DF_AlarmManagement_AlarmFloodMember_IsFirstInFlood DEFAULT 0,

    CONSTRAINT PK_AlarmManagement_AlarmFloodMember PRIMARY KEY (AlarmFloodMemberId),
    CONSTRAINT UQ_AlarmManagement_AlarmFloodMember_Flood_Event UNIQUE (AlarmFloodId, AlarmEventId),
    CONSTRAINT FK_AlarmManagement_AlarmFloodMember_AlarmFlood FOREIGN KEY (AlarmFloodId) REFERENCES
AlarmManagement.AlarmFlood(AlarmFloodId),
    CONSTRAINT FK_AlarmManagement_AlarmFloodMember_AlarmEvent FOREIGN KEY (AlarmEventId) REFERENCES
AlarmManagement.AlarmEvent(AlarmEventId)
);
GO

CREATE TABLE AlarmManagement.AlarmFloodSummary (
    AlarmFloodSummaryId BIGINT IDENTITY(1,1) NOT NULL,
    AlarmFloodId BIGINT NOT NULL,
    TotalAlarmCount INT NOT NULL,
    CriticalCount INT NOT NULL CONSTRAINT DF_AlarmManagement_AlarmFloodSummary_CriticalCount DEFAULT 0,
    WarningCount INT NOT NULL CONSTRAINT DF_AlarmManagement_AlarmFloodSummary_WarningCount DEFAULT 0,
    UniqueSignalCount INT NOT NULL CONSTRAINT DF_AlarmManagement_AlarmFloodSummary_UniqueSignalCount DEFAULT 0,
    UniqueEquipmentCount INT NOT NULL CONSTRAINT DF_AlarmManagement_AlarmFloodSummary_UniqueEquipmentCount DEFAULT 0,
    DurationSeconds INT NULL,
    CalculatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmFloodSummary_CalculatedAtUtc DEFAULT
SYSUTCDATETIME(),

    CONSTRAINT PK_AlarmManagement_AlarmFloodSummary PRIMARY KEY (AlarmFloodSummaryId),
    CONSTRAINT FK_AlarmManagement_AlarmFloodSummary_AlarmFlood FOREIGN KEY (AlarmFloodId) REFERENCES
AlarmManagement.AlarmFlood(AlarmFloodId)
);
GO

CREATE TABLE AlarmManagement.AlarmFloodRanking (
    AlarmFloodRankingId BIGINT IDENTITY(1,1) NOT NULL,
    AlarmFloodId BIGINT NOT NULL,
    AlarmDefinitionId INT NOT NULL,
    RankingMethod NVARCHAR(50) NOT NULL,
    RankNumber INT NOT NULL,
    AlarmCount INT NOT NULL CONSTRAINT DF_AlarmManagement_AlarmFloodRanking_AlarmCount DEFAULT 0,
    Score DECIMAL(18,6) NULL,
    Note NVARCHAR(500) NULL,

    CONSTRAINT PK_AlarmManagement_AlarmFloodRanking PRIMARY KEY (AlarmFloodRankingId),
    CONSTRAINT FK_AlarmManagement_AlarmFloodRanking_AlarmFlood FOREIGN KEY (AlarmFloodId) REFERENCES
AlarmManagement.AlarmFlood(AlarmFloodId),
    CONSTRAINT FK_AlarmManagement_AlarmFloodRanking_AlarmDefinition FOREIGN KEY (AlarmDefinitionId) REFERENCES
AlarmManagement.AlarmDefinition(AlarmDefinitionId),
    CONSTRAINT CK_AlarmManagement_AlarmFloodRanking_RankingMethod CHECK (RankingMethod IN (N'COUNT', N'PRIORITY', N'ARRIVAL',
N'RCA_IMPORTED'))
);
GO

CREATE TABLE AlarmManagement.AlarmRationalization (
    AlarmRationalizationId INT IDENTITY(1,1) NOT NULL,
    AlarmDefinitionId INT NOT NULL,
    RationalizationStatusId INT NOT NULL,
    ConsequenceText NVARCHAR(1000) NULL,
    OperatorResponseText NVARCHAR(2000) NULL,
    DesignBasisReference NVARCHAR(500) NULL,
    ProposedPriorityId INT NULL,
    ProposedSeverityId INT NULL,
    ProposedByUserId INT NULL,
    ProposedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmRationalization_ProposedAtUtc DEFAULT
SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_AlarmManagement_AlarmRationalization PRIMARY KEY (AlarmRationalizationId),
    CONSTRAINT FK_AlarmManagement_AlarmRationalization_AlarmDefinition FOREIGN KEY (AlarmDefinitionId) REFERENCES
AlarmManagement.AlarmDefinition(AlarmDefinitionId),
    CONSTRAINT FK_AlarmManagement_AlarmRationalization_RationalizationStatus FOREIGN KEY (RationalizationStatusId) REFERENCES
AlarmManagement.RationalizationStatus(RationalizationStatusId),
    CONSTRAINT FK_AlarmManagement_AlarmRationalization_ProposedPriority FOREIGN KEY (ProposedPriorityId) REFERENCES
AlarmManagement.AlarmPriority(AlarmPriorityId),
    CONSTRAINT FK_AlarmManagement_AlarmRationalization_ProposedSeverity FOREIGN KEY (ProposedSeverityId) REFERENCES
AlarmManagement.AlarmSeverity(AlarmSeverityId),
    CONSTRAINT FK_AlarmManagement_AlarmRationalization_ProposedByUser FOREIGN KEY (ProposedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE AlarmManagement.AlarmRationalizationReview (
    AlarmRationalizationReviewId INT IDENTITY(1,1) NOT NULL,
    AlarmRationalizationId INT NOT NULL,
    ReviewedByUserId INT NOT NULL,
    ReviewedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmRationalizationReview_ReviewedAtUtc
DEFAULT SYSUTCDATETIME(),
    Decision NVARCHAR(40) NOT NULL,
    ReviewNote NVARCHAR(1000) NULL,

    CONSTRAINT PK_AlarmManagement_AlarmRationalizationReview PRIMARY KEY (AlarmRationalizationReviewId),
    CONSTRAINT FK_AlarmManagement_AlarmRationalizationReview_AlarmRationalization FOREIGN KEY (AlarmRationalizationId) REFERENCES

```

```

AlarmManagement.AlarmRationalization(AlarmRationalizationId),
    CONSTRAINT FK_AlarmManagement_AlarmRationalizationReview_ReviewedByUser FOREIGN KEY (ReviewedById) REFERENCES
Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_AlarmManagement_AlarmRationalizationReview_Decision CHECK (Decision IN (N'APPROVE', N'REJECT', N'REVISE',
N'EXPIRE'))
);
GO

CREATE TABLE AlarmManagement.AlarmKpiSnapshot (
    AlarmKpiSnapshotId BIGINT IDENTITY(1,1) NOT NULL,
    UnitId INT NULL,
    KpiMetricTypeId INT NOT NULL,
    PeriodStartUtc DATETIME2(7) NOT NULL,
    PeriodEndUtc DATETIME2(7) NOT NULL,
    MetricValue DECIMAL(18,6) NOT NULL,
    CalculatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AlarmManagement_AlarmKpiSnapshot_CalculatedAtUtc DEFAULT
SYSUTCDATETIME(),

    CONSTRAINT PK_AlarmManagement_AlarmKpiSnapshot PRIMARY KEY (AlarmKpiSnapshotId),
    CONSTRAINT FK_AlarmManagement_AlarmKpiSnapshot_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_AlarmManagement_AlarmKpiSnapshot_KpiMetricType FOREIGN KEY (KpiMetricTypeId) REFERENCES
AlarmManagement.KpiMetricType(KpiMetricTypeId)
);
GO

```

C.7.5 Indexes and seed examples

The most important access paths are unit-plus-time, definition-plus-time, signal-plus-time and flood membership order. Those are the paths used by the alarm list, the flood view, the RCA handoff and reporting.

```

CREATE INDEX IX_AlarmManagement_AlarmDefinition_UnitId
ON AlarmManagement.AlarmDefinition(UnitId);

CREATE INDEX IX_AlarmManagement_AlarmDefinition_SignalId
ON AlarmManagement.AlarmDefinition(SignalId)
WHERE SignalId IS NOT NULL;

CREATE INDEX IX_AlarmManagement_AlarmDefinition_EquipmentId
ON AlarmManagement.AlarmDefinition(EquipmentId)
WHERE EquipmentId IS NOT NULL;

CREATE INDEX IX_AlarmManagement_AlarmCondition_AlarmDefinitionId
ON AlarmManagement.AlarmCondition(AlarmDefinitionId);

CREATE INDEX IX_AlarmManagement_AlarmEvent_Unit_RaisedAtUtc
ON AlarmManagement.AlarmEvent(UnitId, RaisedAtUtc DESC)
INCLUDE (AlarmDefinitionId, AlarmSeverityId, AlarmPriorityId, SignalId, EquipmentId);

CREATE INDEX IX_AlarmManagement_AlarmEvent_Definition_RaisedAtUtc
ON AlarmManagement.AlarmEvent(AlarmDefinitionId, RaisedAtUtc DESC);

CREATE INDEX IX_AlarmManagement_AlarmEvent_Signal_RaisedAtUtc
ON AlarmManagement.AlarmEvent(SignalId, RaisedAtUtc DESC)
WHERE SignalId IS NOT NULL;

CREATE INDEX IX_AlarmManagement_AlarmEvent_CorrelationKey
ON AlarmManagement.AlarmEvent(CorrelationKey)
WHERE CorrelationKey IS NOT NULL;

CREATE INDEX IX_AlarmManagement_AlarmEventStateHistory_Event Time
ON AlarmManagement.AlarmEventStateHistory(AlarmEventId, ChangedAtUtc DESC);

CREATE INDEX IX_AlarmManagement_AlarmFlood_Unit_StartedAtUtc
ON AlarmManagement.AlarmFlood(UnitId, StartedAtUtc DESC);

CREATE INDEX IX_AlarmManagement_AlarmFloodMember_Flood_Order
ON AlarmManagement.AlarmFloodMember(AlarmFloodId, ArrivalOrder);

CREATE INDEX IX_AlarmManagement_AlarmAcknowledgement_Event
ON AlarmManagement.AlarmAcknowledgement(AlarmEventId);

CREATE INDEX IX_AlarmManagement_AlarmNotification_Event_Channel
ON AlarmManagement.AlarmNotification(AlarmEventId, NotificationChannelId);

CREATE INDEX IX_AlarmManagement_AlarmKpiSnapshot_Period
ON AlarmManagement.AlarmKpiSnapshot(KpiMetricTypeId, PeriodStartUtc, PeriodEndUtc);
GO

```

C.7.5.1 Seed examples

```

INSERT INTO AlarmManagement.AlarmSeverity (Code, Name, Description, DisplayOrder)
VALUES
(N'INFO', N'Information', N'Operator information only.', 10),

```

```

(N'WARN', N'Warning', N'Requires operator awareness.', 20),
(N'HIGH', N'High', N'Requires prompt operator response.', 30),
(N'CRITICAL', N'Critical', N'Immediate operator attention.', 40),
(N'TRIP', N'Trip', N'Protective trip or trip-related alarm.', 50);

INSERT INTO AlarmManagement.AlarmState (Code, Name, DisplayOrder)
VALUES
(N'ACTIVE', N'Active', 10),
(N'ACKNOWLEDGED', N'Acknowledged', 20),
(N'RETURNED', N'Returned to normal', 30),
(N'CLEARED', N'Cleared', 40),
(N'SHELVED', N'Shelved', 50),
(N'SUPPRESSED', N'Suppressed', 60);

INSERT INTO AlarmManagement.AlarmFloodStatus (Code, Name, DisplayOrder)
VALUES
(N'DETECTED', N'Detected', 10),
(N'OPEN', N'Open', 20),
(N'ANALYZED', N'Analyzed', 30),
(N'HANDED OFF', N'Handed off to RCA', 40),
(N'CLOSED', N'Closed', 50),
(N'ARCHIVED', N'Archived', 60);
GO

```

C.7.5.2 Verification queries

```

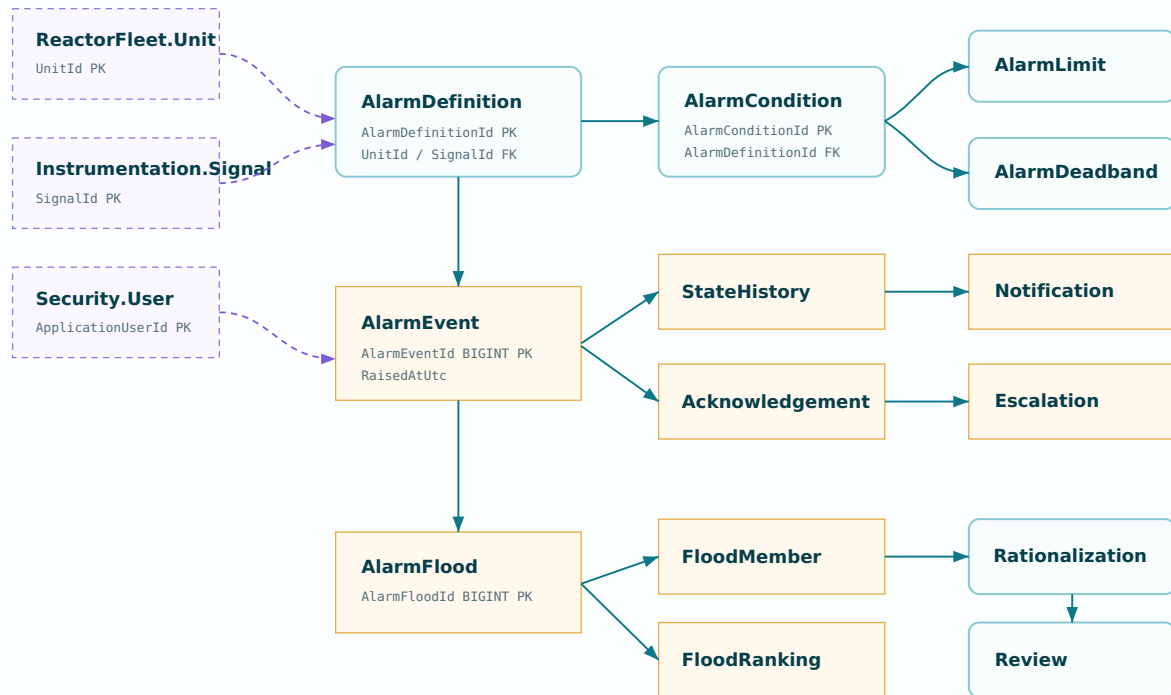
-- Latest alarms for one unit.
SELECT TOP (50)
    u.Code AS UnitCode,
    ae.RaisedAtUtc,
    ad.Code AS AlarmCode,
    sev.Code AS Severity,
    pri.Code AS Priority,
    ae.Message
FROM AlarmManagement.AlarmEvent AS ae
JOIN AlarmManagement.AlarmDefinition AS ad ON ad.AlarmDefinitionId = ae.AlarmDefinitionId
JOIN ReactorFleet.Unit AS u ON u.UnitId = ae.UnitId
JOIN AlarmManagement.AlarmSeverity AS sev ON sev.AlarmSeverityId = ae.AlarmSeverityId
JOIN AlarmManagement.AlarmPriority AS pri ON pri.AlarmPriorityId = ae.AlarmPriorityId
WHERE u.Code = 'NX1-U1'
ORDER BY ae.RaisedAtUtc DESC;

-- A flood as the operator saw it: raw order, not causal order.
SELECT
    fm.ArrivalOrder,
    ae.RaisedAtUtc,
    ad.Code AS AlarmCode,
    ae.Message,
    fm.GroupLabel
FROM AlarmManagement.AlarmFloodMember AS fm
JOIN AlarmManagement.AlarmEvent AS ae ON ae.AlarmEventId = fm.AlarmEventId
JOIN AlarmManagement.AlarmDefinition AS ad ON ad.AlarmDefinitionId = ae.AlarmDefinitionId
WHERE fm.AlarmFloodId = @AlarmFloodId
ORDER BY fm.ArrivalOrder;

-- Naive count ranking, deliberately separate from RootCause ranking.
SELECT
    fr.RankNumber,
    fr.RankingMethod,
    ad.Code,
    fr.AlarmCount,
    fr.Score,
    fr.Note
FROM AlarmManagement.AlarmFloodRanking AS fr
JOIN AlarmManagement.AlarmDefinition AS ad ON ad.AlarmDefinitionId = fr.AlarmDefinitionId
WHERE fr.AlarmFloodId = @AlarmFloodId
ORDER BY fr.RankNumber;

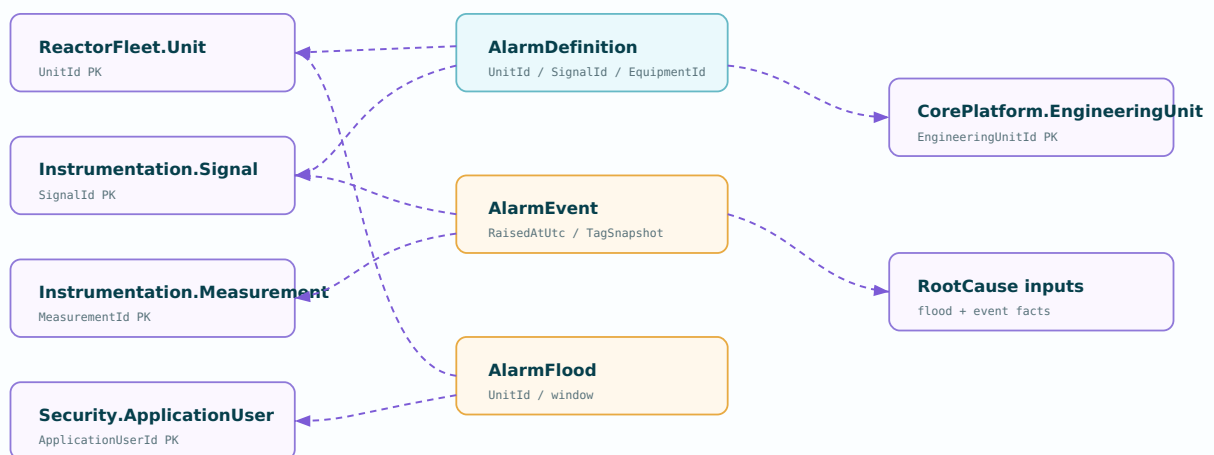
```

C.7.6 ER diagram



Solid edges stay inside AlarmManagement. Dashed violet edges are cross-schema passports to ReactorFleet, Instrumentation and Security.

Figure C.7.1 - AlarmManagement sector ER diagram. Definitions lead to runtime events; runtime events can be acknowledged, notified, escalated and grouped into alarm floods.



Dependencies point from AlarmManagement to the authoritative owner of the referenced row. RootCause later reads AlarmFlood and AlarmEvent; it does not own them.

Figure C.7.2 - AlarmManagement passport map. Cross-schema dependencies point to the owning sector. RootCause later reads these rows but does not own the alarm event facts.

C.7.7 Foreign-key mapping

The table below lists the most important passports. Internal lookup-table foreign keys are present in the DDL but are not repeated here line by line.

FROM TABLE	FK COLUMN	TO TABLE	PK COLUMN	RELATIONSHIP
AlarmDefinition	UnitId	ReactorFleet.Unit	UnitId	Alarm definition belongs to one physical unit.
AlarmDefinition	SignalId	Instrumentation.Signal	SignalId	Signal-driven alarm watches one instrumentation signal.
AlarmDefinition	EquipmentId	ReactorFleet.Equipment	EquipmentId	Equipment alarm is anchored to a physical asset.
AlarmDefinition	PlantSystemId	ReactorFleet.PlantSystem	PlantSystemId	System-level alarm is anchored to a plant system.
AlarmLimit	EngineeringUnitId	CorePlatform.EngineeringUnit	EngineeringUnitId	Thresholds use platform engineering units.
AlarmEvent	UnitId	ReactorFleet.Unit	UnitId	Runtime event belongs to the same unit as later RCA and reporting.
AlarmEvent	SignalId	Instrumentation.Signal	SignalId	Event keeps the signal identity that the historian and twin use.
AlarmEventEvidence	MeasurementId	Instrumentation.Measurement	MeasurementId	Evidence can point to the exact measurement row that supported the event.
AlarmAcknowledgement	AcknowledgedByUserId	Security.ApplicationUser	ApplicationUserId	Human acknowledgement is accountable.
AlarmFlood	UnitId	ReactorFleet.Unit	UnitId	Flood windows are unit-scoped.
AlarmFloodMember	AlarmEventId	AlarmManagement.AlarmEvent	AlarmEventId	A flood is an ordered set of alarm events.
AlarmRationalization	AlarmDefinitionId	AlarmManagement.AlarmDefinition	AlarmDefinitionId	Rationalization governs the definition, not each event.

AlarmDefinition

FK UnitId
TO ReactorFleet.Unit
Defines which unit can raise the alarm.

AlarmDefinition

FK SignalId
TO Instrumentation.Signal
Binds the definition to the same tag the historian records.

AlarmEvent

FK `UnitId`, `SignalId`
TO `ReactorFleet.Unit`, `Instrumentation.Signal`
Makes alarm events joinable to the physical plant and historian.

AlarmEventEvidence

FK `MeasurementId`
TO `Instrumentation.Measurement`
Captures the witness readings used when the event was raised.

AlarmAcknowledgement

FK `AcknowledgedByUserId`
TO `Security.ApplicationUser`
Records accountability for acknowledgement.

AlarmRationalizationReview

FK `ReviewedByUserId`
TO `Security.ApplicationUser`
Records review and approval accountability.

AlarmLimit

FK `EngineeringUnitId`
TO `CorePlatform.EngineeringUnit`
Keeps thresholds unit-safe and queryable.

C.7.8 Boundary and next sector

Honest boundary. This schema manages alarms; it does not claim that the most urgent, first, loudest or most repeated alarm is the origin. The flood is recorded faithfully because the root-cause engine needs honest input. The causal explanation belongs to `RootCause`, where alarm events are joined with topology, telemetry and evidence.

The next schema is `EventManager`. It will store incidents, near misses, operational events, event timelines and handoffs. Alarm events can trigger or support incidents, but an incident is not just an alarm; it is an operational record with ownership, investigation and closure.

Appendix C.8

EventManager Schema

The EventManagement sector is the bridge between raw operational facts and accountable follow-up. AlarmManagement records what lit up. EventManagement records what the organization decided to treat as an event, incident, near miss or drill record.

An operational event can be created manually, imported from a drill, raised from a digital-twin divergence, or opened from an alarm flood. It keeps links to the alarm events and flood window, but it does not copy them. It adds ownership, timeline, participants, impacts, notes, attachments, incident actions and review outcomes.

EventManager answers: what happened, who owns it, what evidence is attached, what actions follow, and when the record is closed.

C.8.1
Sector purpose

C.8.2
Full table list

C.8.3
Lookup tables

C.8.4
SQL DDL

C.8.5
Indexes and seed examples

C.8.6
ER diagram

C.8.7
Foreign-key mapping

C.8.8
Boundary and next sector

C.8.1 Sector purpose

EventManager depends on ReactorFleet, Organization, Security, Instrumentation and AlarmManagement. Its most important row is OperationalEvent: the one place where an operational occurrence receives a title, owner, severity, status and lifecycle.

- OperationalEvent is the base record for incidents, near misses, transients and drills.
- EventAlarmLink and EventFloodLink connect the record back to raw alarm facts.
- EventTimelineEntry gives the event a replayable chronology.
- EventParticipant, EventNote and EventAttachment preserve ownership and evidence.
- NearMiss and Incident specialize the base event without duplicating the base identity.
- IncidentAction and IncidentReview close the loop from event to follow-up.

Design choice. Incidents and near misses are not separate universes. Both are specialized records anchored to one `OperationalEvent`. This lets reporting ask for all events, while specialist screens can filter to incidents or near misses only.

C.8.2 Full table list

The EventManagement sector contains forty-two tables: eighteen typed lookup tables and twenty-four substantive tables. The split keeps classifications stable, while event facts remain accountable and queryable.

LOOKUP

EventManagement.EventType

PK EventTypeId
FK None

Classifies operational events: incident, near miss, transient, drill, maintenance handoff, security or digital-twin divergence.

LOOKUP

EventManagement.EventStatus

PK EventStatusId
FK None

Lifecycle of an operational event: new, triaged, investigating, actioned, closed, archived.

LOOKUP

EventManagement.EventSeverity

PK EventSeverityId
FK None

Severity of the operational record, separate from alarm priority and root-cause confidence.

LOOKUP

EventManagement.EventSourceType

PK EventSourceTypeId
FK None

How the event was created: alarm flood, manual report, drill, import, digital twin, maintenance or security.

LOOKUP

EventManagement.EventRelationshipType

PK EventRelationshipTypeId
FK None

Relationship between events: duplicate, parent, child, related, follow-up, supersedes.

LOOKUP

EventManagement.EventTimelineEntryType

PK EventTimelineEntryTypeId
FK None

Classifies timeline entries: detected, alarmed, acknowledged, action, decision, handoff, closure.

LOOKUP

EventManagement.EventParticipantRole

PK EventParticipantRoleId
FK None

Role of a person or user in an event: owner, reporter, investigator, reviewer, observer.

LOOKUP

EventManagement.EventAttachmentType

PK EventAttachmentTypeId
FK None

Classifies attachments: log export, screenshot, procedure, photo, report, historian extract.

LOOKUP

EventManagement.EventNoteType

PK EventNoteTypeId
FK None

Classifies notes: operator note, engineer note, investigation note, closure note, system note.

LOOKUP

EventManagement.NearMissType

PK NearMissTypeId
FK None

Classifies a near miss by the barrier or condition that prevented escalation.

LOOKUP

EventManagement.BarrierType

PK BarrierTypeId
FK None

Physical, procedural, human, software, interlock, alarm, training or administrative barrier.

LOOKUP

EventManagement.IncidentType

PK IncidentTypeId
FK None

Classifies incidents: operational, equipment, safety, radiation, security, cyber, environmental, training.

LOOKUP

EventManagement.IncidentStatus

PK IncidentStatusId
FK None

Incident lifecycle: opened, investigating, awaiting action, review, closed, reopened.

LOOKUP

EventManagement.IncidentClassification

PK IncidentClassificationId
FK None

Regulatory, internal, training, outage, reliability, quality or reporting classification.

LOOKUP

EventManagement.IncidentActionType

PK IncidentActionTypeId
FK None

Corrective, preventive, compensatory, investigation, training, inspection or reporting action.

LOOKUP

EventManagement.IncidentActionStatus

PK IncidentActionStatusId
FK None

Action state: proposed, assigned, in progress, completed, verified, cancelled, overdue.

LOOKUP

EventManagement.IncidentReviewOutcome

PK IncidentReviewOutcomeId
FK None

Review outcome: accepted, returned, escalated, reopened, closed with actions.

LOOKUP

EventManagement.EventImpactType

PK EventImpactTypeId
FK None

Classifies impact: safety, production, equipment, dose, compliance, availability, training.

CORE**EventManagement.OperationalEvent**

PK OperationalEventId

FK UnitId, EventTypeId, EventStatusId

The central event record: a structured operational occurrence with ownership and lifecycle.

CORE**EventManagement.EventStatusHistory**

PK EventStatusHistoryId

FK OperationalEventId, EventStatusId

Append-only status transition history for the event.

CORE**EventManagement.EventRelationship**

PK EventRelationshipId

FK SourceOperationalEventId,
TargetOperationalEventId

Directed relationship between events.

CORE**EventManagement.EventTag**

PK EventTagId

FK None

Controlled tag catalogue for searching and reporting.

CORE**EventManagement.EventTagAssignment**

PK EventTagAssignmentId

FK OperationalEventId, EventTagId

Many-to-many assignment of controlled tags to events.

LINKS**EventManagement.EventAlarmLink**

PK EventAlarmLinkId

FK OperationalEventId, AlarmEventId

Connects an event to one or more alarm events that triggered or supported it.

LINKS**EventManagement.EventFloodLink**

PK EventFloodLinkId

FK OperationalEventId, AlarmFloodId

Connects an event to an alarm flood window.

LINKS**EventManagement.EventSignalLink**

PK EventSignalLinkId

FK OperationalEventId, SignalId

Signals relevant to the event, including witnesses and context signals.

LINKS**EventManagement.EventEquipmentLink**

PK EventEquipmentLinkId

FK OperationalEventId, EquipmentId

Equipment involved, suspected, affected, or explicitly ruled out at event level.

TIMELINE**EventManagement.EventTimelineEntry**

PK EventTimelineEntryId

FK OperationalEventId,
EventTimelineEntryTypeId

Chronological narrative of what happened, who acted, and what changed.

TIMELINE**EventManagement.EventParticipant**

PK EventParticipantId
FK OperationalEventId , PersonId , ApplicationUserId

People or users involved in the event.

TIMELINE**EventManagement.EventNote**

PK EventNoteId
FK OperationalEventId , EventNoteTypeId

Notes attached to the event lifecycle.

TIMELINE**EventManagement.EventAttachment**

PK EventAttachmentId
FK OperationalEventId , EventAttachmentTypeId

Metadata for stored attachments; binary content is outside the normalized schema.

CONTEXT**EventManagement.EventImpactAssessment**

PK EventImpactAssessmentId
FK OperationalEventId ,
EventImpactTypeId

Structured impact row: safety, production, dose, equipment, compliance or availability.

CONTEXT**EventManagement.EventUnitStateSnapshot**

PK EventUnitStateSnapshotId
FK OperationalEventId , UnitId

Event-time unit context: power, mode, trip status and grid state.

NEAR MISS**EventManagement.NearMiss**

PK NearMissId
FK OperationalEventId , NearMissTypeId

Specialized near-miss record tied to an operational event.

NEAR MISS**EventManagement.NearMissBarrier**

PK NearMissBarrierId
FK NearMissId , BarrierTypeId

Barrier that prevented escalation, and whether it succeeded, weakened or failed.

INCIDENT**EventManagement.Incident**

PK IncidentId
FK OperationalEventId , IncidentTypeId ,
IncidentStatusId

Specialized incident record with investigation and closure fields.

INCIDENT**EventManagement.IncidentClassificationAssignment**

PK IncidentClassificationAssignmentId
FK IncidentId , IncidentClassificationId

Many-to-many classification of an incident.

INCIDENT**EventManagement.IncidentAction**

PK IncidentActionId
FK IncidentId , IncidentActionTypeId ,
IncidentActionStatusId

Corrective or preventive action arising from an incident.

INCIDENT EventManagement.IncidentActionOwner PK IncidentActionOwnerId FK IncidentActionId, PersonId, ApplicationUserId Ownership assignment for one incident action.	INCIDENT EventManagement.IncidentReview PK IncidentReviewId FK IncidentId, IncidentReviewOutcomeId Formal review record for an incident.
INCIDENT EventManagement.IncidentReviewFinding PK IncidentReviewFindingId FK IncidentReviewId Finding or observation from a formal review.	

C.8.3 Lookup tables

Lookup tables keep lifecycle, severity, source, participant role, incident status and action status consistent across the platform. They also make reporting and EF Core navigation explicit.

LOOKUP TABLE	TYPICAL SEED CODES	MAIN CONSUMER
EventType	INCIDENT, NEAR_MISS, TRANSIENT, DRILL, DIVERGENCE	Referenced by OperationalEvent.
EventStatus	NEW, TRIAGED, INVESTIGATING, ACTIONED, CLOSED, ARCHIVED	Referenced by OperationalEvent and status history.
EventSeverity	INFO, MINOR, MODERATE, MAJOR, CRITICAL	Referenced by OperationalEvent.
EventSourceType	ALARM_FLOOD, MANUAL, DIGITAL_TWIN, DRILL, IMPORT, MAINTENANCE	Describes how an event was created.
EventRelationshipType	PARENT, CHILD, RELATED, DUPLICATE, FOLLOW_UP	Referenced by EventRelationship.
EventTimelineEntryType	DETECTED, ALARMED, ACKNOWLEDGED, ACTION, DECISION, CLOSURE	Classifies timeline entries.
EventParticipantRole	OWNER, REPORTER, INVESTIGATOR, REVIEWER, OBSERVER	Referenced by participants.
EventAttachmentType	SCREENSHOT, LOG_EXPORT, REPORT, PHOTO, PROCEDURE	Referenced by attachments.
EventNoteType	OPERATOR, ENGINEERING, INVESTIGATION, CLOSURE, SYSTEM	Referenced by event notes.
NearMissType	BARRIER_SUCCESS, RECOVERED_TRANSIENT, PROCEDURAL_CATCH, DETECTED_EARLY	Referenced by NearMiss.
BarrierType	PHYSICAL, PROCEDURAL, SOFTWARE, HUMAN, INTERLOCK, ALARM	Referenced by near-miss barriers.

LOOKUP TABLE	TYPICAL SEED CODES	MAIN CONSUMER
IncidentType	OPERATIONAL , EQUIPMENT , SAFETY , RADIATION , SECURITY , CYBER	Referenced by Incident .
IncidentStatus	OPEN , INVESTIGATING , AWAITING_ACTION , REVIEW , CLOSED , REOPENED	Referenced by Incident .
IncidentClassification	INTERNAL , REGULATORY , TRAINING , QUALITY , RELIABILITY	Assigned to incidents.
IncidentActionType	CORRECTIVE , PREVENTIVE , COMPENSATORY , TRAINING , INSPECTION	Referenced by actions.
IncidentActionStatus	PROPOSED , ASSIGNED , IN_PROGRESS , COMPLETED , VERIFIED , OVERDUE	Action lifecycle.
IncidentReviewOutcome	ACCEPTED , RETURNED , ESCALATED , REOPENED , CLOSED_WITH_ACTIONS	Referenced by reviews.
EventImpactType	SAFETY , PRODUCTION , EQUIPMENT , DOSE , COMPLIANCE , AVAILABILITY	Referenced by impact assessments.

C.8.4 SQL DDL

C.8.4.1 Schema

```
IF SCHEMA_ID(N'EventManagement') IS NULL
EXEC(N'CREATE SCHEMA EventManagement');
GO
```

C.8.4.2 EventManagement lookup tables

```
CREATE TABLE EventManagement.EventType (
    EventTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_EventManagement_EventType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_EventManagement_EventType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EventManagement_EventType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EventManagement_EventType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EventManagement_EventType
    PRIMARY KEY (EventTypeId),
    CONSTRAINT UQ_EventManagement_EventType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE EventManagement.EventStatus (
    EventStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_EventManagement_EventStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_EventManagement_EventStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EventManagement_EventStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EventManagement_EventStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,
```



```

CONSTRAINT PK_EventManagement_EventStatus
PRIMARY KEY (EventStatusId),
CONSTRAINT UQ_EventManagement_EventStatus_Code
UNIQUE (Code)
);
GO

CREATE TABLE EventManagement.EventSeverity (
EventSeverityId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_EventManagement_EventSeverity_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_EventManagement_EventSeverity_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EventManagement_EventSeverity_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EventManagement_EventSeverity_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_EventManagement_EventSeverity
PRIMARY KEY (EventSeverityId),
CONSTRAINT UQ_EventManagement_EventSeverity_Code
UNIQUE (Code)
);
GO

CREATE TABLE EventManagement.EventSourceType (
EventSourceTypeId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_EventManagement_EventSourceType_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_EventManagement_EventSourceType_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EventManagement_EventSourceType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EventManagement_EventSourceType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_EventManagement_EventSourceType
PRIMARY KEY (EventSourceTypeId),
CONSTRAINT UQ_EventManagement_EventSourceType_Code
UNIQUE (Code)
);
GO

CREATE TABLE EventManagement.EventRelationshipType (
EventRelationshipTypeId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_EventManagement_EventRelationshipType_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_EventManagement_EventRelationshipType_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EventManagement_EventRelationshipType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EventManagement_EventRelationshipType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_EventManagement_EventRelationshipType
PRIMARY KEY (EventRelationshipTypeId),
CONSTRAINT UQ_EventManagement_EventRelationshipType_Code
UNIQUE (Code)
);
GO

CREATE TABLE EventManagement.EventTimelineEntryType (
EventTimelineEntryTypeId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_EventManagement_EventTimelineEntryType_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_EventManagement_EventTimelineEntryType_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EventManagement_EventTimelineEntryType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EventManagement_EventTimelineEntryType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_EventManagement_EventTimelineEntryType
PRIMARY KEY (EventTimelineEntryTypeId),
CONSTRAINT UQ_EventManagement_EventTimelineEntryType_Code
UNIQUE (Code)
);
GO

```

```

CREATE TABLE EventManagement.EventParticipantRole (
    EventParticipantRoleId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_EventManagement_EventParticipantRole_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_EventManagement_EventParticipantRole_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EventManagement_EventParticipantRole_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EventManagement_EventParticipantRole_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EventManagement_EventParticipantRole
    PRIMARY KEY (EventParticipantRoleId),
    CONSTRAINT UQ_EventManagement_EventParticipantRole_Code
    UNIQUE (Code)
);
GO

CREATE TABLE EventManagement.EventAttachmentType (
    EventAttachmentTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_EventManagement_EventAttachmentType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_EventManagement_EventAttachmentType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EventManagement_EventAttachmentType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EventManagement_EventAttachmentType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EventManagement_EventAttachmentType
    PRIMARY KEY (EventAttachmentTypeId),
    CONSTRAINT UQ_EventManagement_EventAttachmentType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE EventManagement.EventNoteType (
    EventNoteTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_EventManagement_EventNoteType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_EventManagement_EventNoteType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EventManagement_EventNoteType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EventManagement_EventNoteType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EventManagement_EventNoteType
    PRIMARY KEY (EventNoteTypeId),
    CONSTRAINT UQ_EventManagement_EventNoteType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE EventManagement.NearMissType (
    NearMissTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_EventManagement_NearMissType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_EventManagement_NearMissType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EventManagement_NearMissType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EventManagement_NearMissType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EventManagement_NearMissType
    PRIMARY KEY (NearMissTypeId),
    CONSTRAINT UQ_EventManagement_NearMissType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE EventManagement.BarrierType (
    BarrierTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_EventManagement_BarrierType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_EventManagement_BarrierType_IsActive DEFAULT 1,

```

```

CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EventManagement_BarrierType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EventManagement_BarrierType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_EventManagement_BarrierType
PRIMARY KEY (BarrierTypeId),
CONSTRAINT UQ_EventManagement_BarrierType_Code
UNIQUE (Code)
);
GO

CREATE TABLE EventManagement.IncidentType (
IncidentTypeId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_EventManagement_IncidentType_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_EventManagement_IncidentType_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EventManagement_IncidentType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EventManagement_IncidentType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_EventManagement_IncidentType
PRIMARY KEY (IncidentTypeId),
CONSTRAINT UQ_EventManagement_IncidentType_Code
UNIQUE (Code)
);
GO

CREATE TABLE EventManagement.IncidentStatus (
IncidentStatusId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_EventManagement_IncidentStatus_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_EventManagement_IncidentStatus_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EventManagement_IncidentStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EventManagement_IncidentStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_EventManagement_IncidentStatus
PRIMARY KEY (IncidentStatusId),
CONSTRAINT UQ_EventManagement_IncidentStatus_Code
UNIQUE (Code)
);
GO

CREATE TABLE EventManagement.IncidentClassification (
IncidentClassificationId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_EventManagement_IncidentClassification_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_EventManagement_IncidentClassification_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EventManagement_IncidentClassification_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EventManagement_IncidentClassification_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_EventManagement_IncidentClassification
PRIMARY KEY (IncidentClassificationId),
CONSTRAINT UQ_EventManagement_IncidentClassification_Code
UNIQUE (Code)
);
GO

CREATE TABLE EventManagement.IncidentActionType (
IncidentActionTypeId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_EventManagement_IncidentActionType_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_EventManagement_IncidentActionType_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EventManagement_IncidentActionType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EventManagement_IncidentActionType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_EventManagement_IncidentActionType
PRIMARY KEY (IncidentActionTypeId),
CONSTRAINT UQ_EventManagement_IncidentActionType_Code

```

```

        UNIQUE (Code)
    );
GO

CREATE TABLE EventManagement.IncidentActionStatus (
    IncidentActionStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_EventManagement_IncidentActionStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_EventManagement_IncidentActionStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EventManagement_IncidentActionStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EventManagement_IncidentActionStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EventManagement_IncidentActionStatus
    PRIMARY KEY (IncidentActionStatusId),
    CONSTRAINT UQ_EventManagement_IncidentActionStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE EventManagement.IncidentReviewOutcome (
    IncidentReviewOutcomeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_EventManagement_IncidentReviewOutcome_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_EventManagement_IncidentReviewOutcome_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EventManagement_IncidentReviewOutcome_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EventManagement_IncidentReviewOutcome_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EventManagement_IncidentReviewOutcome
    PRIMARY KEY (IncidentReviewOutcomeId),
    CONSTRAINT UQ_EventManagement_IncidentReviewOutcome_Code
    UNIQUE (Code)
);
GO

CREATE TABLE EventManagement.EventImpactType (
    EventImpactTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_EventManagement_EventImpactType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_EventManagement_EventImpactType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EventManagement_EventImpactType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EventManagement_EventImpactType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EventManagement_EventImpactType
    PRIMARY KEY (EventImpactTypeId),
    CONSTRAINT UQ_EventManagement_EventImpactType_Code
    UNIQUE (Code)
);
GO

```

C.8.4.3 Base event, status history, relationships and tags

```

CREATE TABLE EventManagement.OperationalEvent (
    OperationalEventId BIGINT IDENTITY(1,1) NOT NULL,
    UnitId INT NOT NULL,
    PlantId INT NULL,
    EventTypeId INT NOT NULL,
    EventStatusId INT NOT NULL,
    EventSeverityId INT NOT NULL,
    EventSourceTypeId INT NOT NULL,
    EventCode NVARCHAR(80) NOT NULL,
    Title NVARCHAR(250) NOT NULL,
    Summary NVARCHAR(2000) NULL,
    DetectedAtUtc DATETIME2(7) NOT NULL,
    ReportedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_EventManagement_OperationalEvent_ReportedAtUtc DEFAULT SYSUTCDATETIME(),
    ClosedAtUtc DATETIME2(7) NULL,
    ReportedByUserId INT NULL,
    OwnerUserId INT NULL,
    OwningPersonId INT NULL,
    SourceReference NVARCHAR(200) NULL,
    IsDrill BIT NOT NULL CONSTRAINT DF_EventManagement_OperationalEvent_IsDrill DEFAULT 0,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_EventManagement_OperationalEvent_CreatedAtUtc DEFAULT SYSUTCDATETIME(),

```

```

    CreatedBy          NVARCHAR(100) NOT NULL,
    ModifiedAtUtc      DATETIME2(7) NOT NULL CONSTRAINT DF_EventManagement_OperationalEvent_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy         NVARCHAR(100) NOT NULL,
    IsDeleted          BIT NOT NULL CONSTRAINT DF_EventManagement_OperationalEvent_IsDeleted DEFAULT 0,
    RowVersion         ROWVERSION NOT NULL,

    CONSTRAINT PK_EventManagement_OperationalEvent PRIMARY KEY (OperationalEventId),
    CONSTRAINT UQ_EventManagement_OperationalEvent_EventCode UNIQUE (EventCode),
    CONSTRAINT FK_EventManagement_OperationalEvent_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_EventManagement_OperationalEvent_Plant FOREIGN KEY (PlantId) REFERENCES Organization.Plant(PlantId),
    CONSTRAINT FK_EventManagement_OperationalEvent_EventType FOREIGN KEY (EventTypeId) REFERENCES
EventManagement.EventType(EventTypeId),
    CONSTRAINT FK_EventManagement_OperationalEvent_EventStatus FOREIGN KEY (EventStatusId) REFERENCES
EventManagement.EventStatus(EventStatusId),
    CONSTRAINT FK_EventManagement_OperationalEvent_EventSeverity FOREIGN KEY (EventSeverityId) REFERENCES
EventManagement.EventSeverity(EventSeverityId),
    CONSTRAINT FK_EventManagement_OperationalEvent_EventSourceType FOREIGN KEY (EventSourceTypeId) REFERENCES
EventManagement.EventSourceType(EventSourceTypeId),
    CONSTRAINT FK_EventManagement_OperationalEvent_ReportedByUser FOREIGN KEY (ReportedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT FK_EventManagement_OperationalEvent_OwnerUser FOREIGN KEY (OwnerUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT FK_EventManagement_OperationalEvent_OwningPerson FOREIGN KEY (OwningPersonId) REFERENCES
Organization.Person(PersonId)
);
GO

CREATE TABLE EventManagement.EventStatusHistory (
    EventStatusHistoryId BIGINT IDENTITY(1,1) NOT NULL,
    OperationalEventId   BIGINT NOT NULL,
    EventStatusId        INT NOT NULL,
    ChangedAtUtc         DATETIME2(7) NOT NULL CONSTRAINT DF_EventManagement_EventStatusHistory_ChangedAtUtc DEFAULT
SYSUTCDATETIME(),
    ChangedByUserId      INT NULL,
    Reason              NVARCHAR(500) NULL,

    CONSTRAINT PK_EventManagement_EventStatusHistory PRIMARY KEY (EventStatusHistoryId),
    CONSTRAINT FK_EventManagement_EventStatusHistory_OperationalEvent FOREIGN KEY (OperationalEventId) REFERENCES
EventManagement.OperationalEvent(OperationalEventId),
    CONSTRAINT FK_EventManagement_EventStatusHistory_EventStatus FOREIGN KEY (EventStatusId) REFERENCES
EventManagement.EventStatus(EventStatusId),
    CONSTRAINT FK_EventManagement_EventStatusHistory_ChangedByUser FOREIGN KEY (ChangedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE EventManagement.EventRelationship (
    EventRelationshipId   BIGINT IDENTITY(1,1) NOT NULL,
    SourceOperationalEventId BIGINT NOT NULL,
    TargetOperationalEventId BIGINT NOT NULL,
    EventRelationshipTypeId INT NOT NULL,
    Note                 NVARCHAR(500) NULL,
    CreatedAtUtc         DATETIME2(7) NOT NULL CONSTRAINT DF_EventManagement_EventRelationship_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy            NVARCHAR(100) NOT NULL,

    CONSTRAINT PK_EventManagement_EventRelationship PRIMARY KEY (EventRelationshipId),
    CONSTRAINT UQ_EventManagement_EventRelationship UNIQUE (SourceOperationalEventId, TargetOperationalEventId,
EventRelationshipTypeId),
    CONSTRAINT CK_EventManagement_EventRelationship_NotSelf CHECK (SourceOperationalEventId <> TargetOperationalEventId),
    CONSTRAINT FK_EventManagement_EventRelationship_SourceEvent FOREIGN KEY (SourceOperationalEventId) REFERENCES
EventManagement.OperationalEvent(OperationalEventId),
    CONSTRAINT FK_EventManagement_EventRelationship_TargetEvent FOREIGN KEY (TargetOperationalEventId) REFERENCES
EventManagement.OperationalEvent(OperationalEventId),
    CONSTRAINT FK_EventManagement_EventRelationship_Type FOREIGN KEY (EventRelationshipTypeId) REFERENCES
EventManagement.EventRelationshipType(EventRelationshipTypeId)
);
GO

CREATE TABLE EventManagement.EventTag (
    EventTagId          INT IDENTITY(1,1) NOT NULL,
    Code                NVARCHAR(50) NOT NULL,
    Name                NVARCHAR(150) NOT NULL,
    Description          NVARCHAR(500) NULL,
    IsActive            BIT NOT NULL CONSTRAINT DF_EventManagement_EventTag_IsActive DEFAULT 1,
    CreatedAtUtc        DATETIME2(7) NOT NULL CONSTRAINT DF_EventManagement_EventTag_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc       DATETIME2(7) NOT NULL CONSTRAINT DF_EventManagement_EventTag_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion          ROWVERSION NOT NULL,

    CONSTRAINT PK_EventManagement_EventTag PRIMARY KEY (EventTagId),
    CONSTRAINT UQ_EventManagement_EventTag_Code UNIQUE (Code)
);
GO

CREATE TABLE EventManagement.EventTagAssignment (
    EventTagAssignmentId BIGINT IDENTITY(1,1) NOT NULL,
    OperationalEventId   BIGINT NOT NULL,
    EventTagId           INT NOT NULL,
    AssignedAtUtc        DATETIME2(7) NOT NULL CONSTRAINT DF_EventManagement_EventTagAssignment_AssignedAtUtc DEFAULT
SYSUTCDATETIME(),
    AssignedByUserId     INT NULL,

    CONSTRAINT PK_EventManagement_EventTagAssignment PRIMARY KEY (EventTagAssignmentId),
    CONSTRAINT UQ_EventManagement_EventTagAssignment UNIQUE (OperationalEventId, EventTagId),
    CONSTRAINT FK_EventManagement_EventTagAssignment_OperationalEvent FOREIGN KEY (OperationalEventId) REFERENCES
EventManagement.OperationalEvent(OperationalEventId),
    CONSTRAINT FK_EventManagement_EventTagAssignment_EventTag FOREIGN KEY (EventTagId) REFERENCES
EventManagement.EventTag(EventTagId),
    CONSTRAINT FK_EventManagement_EventTagAssignment_AssignedByUser FOREIGN KEY (AssignedByUserId) REFERENCES

```

```
Security.ApplicationUser(ApplicationUserId)
);
GO
```

C.8.4.4 Alarm, signal, equipment, timeline, participant and evidence links

```
CREATE TABLE EventManagement.EventAlarmLink (
    EventAlarmLinkId BIGINT IDENTITY(1,1) NOT NULL,
    OperationalEventId BIGINT NOT NULL,
    AlarmEventId BIGINT NOT NULL,
    LinkRole NVARCHAR(50) NOT NULL CONSTRAINT DF_EventManagement_EventAlarmLink_LinkRole DEFAULT N'SUPPORTING',
    Note NVARCHAR(500) NULL,

    CONSTRAINT PK_EventManagement_EventAlarmLink PRIMARY KEY (EventAlarmLinkId),
    CONSTRAINT UQ_EventManagement_EventAlarmLink UNIQUE (OperationalEventId, AlarmEventId),
    CONSTRAINT FK_EventManagement_EventAlarmLink_OperationalEvent FOREIGN KEY (OperationalEventId) REFERENCES
EventManagement.OperationalEvent(OperationalEventId),
    CONSTRAINT FK_EventManagement_EventAlarmLink_AlarmEvent FOREIGN KEY (AlarmEventId) REFERENCES
AlarmManagement.AlarmEvent(AlarmEventId)
);
GO

CREATE TABLE EventManagement.EventFloodLink (
    EventFloodLinkId BIGINT IDENTITY(1,1) NOT NULL,
    OperationalEventId BIGINT NOT NULL,
    AlarmFloodId BIGINT NOT NULL,
    LinkRole NVARCHAR(50) NOT NULL CONSTRAINT DF_EventManagement_EventFloodLink_LinkRole DEFAULT N'TRIGGER',
    Note NVARCHAR(500) NULL,

    CONSTRAINT PK_EventManagement_EventFloodLink PRIMARY KEY (EventFloodLinkId),
    CONSTRAINT UQ_EventManagement_EventFloodLink UNIQUE (OperationalEventId, AlarmFloodId),
    CONSTRAINT FK_EventManagement_EventFloodLink_OperationalEvent FOREIGN KEY (OperationalEventId) REFERENCES
EventManagement.OperationalEvent(OperationalEventId),
    CONSTRAINT FK_EventManagement_EventFloodLink_AlarmFlood FOREIGN KEY (AlarmFloodId) REFERENCES
AlarmManagement.AlarmFlood(AlarmFloodId)
);
GO

CREATE TABLE EventManagement.EventSignalLink (
    EventSignalLinkId BIGINT IDENTITY(1,1) NOT NULL,
    OperationalEventId BIGINT NOT NULL,
    SignalId INT NOT NULL,
    LinkRole NVARCHAR(50) NOT NULL,
    Note NVARCHAR(500) NULL,

    CONSTRAINT PK_EventManagement_EventSignalLink PRIMARY KEY (EventSignalLinkId),
    CONSTRAINT UQ_EventManagement_EventSignalLink UNIQUE (OperationalEventId, SignalId, LinkRole),
    CONSTRAINT FK_EventManagement_EventSignalLink_OperationalEvent FOREIGN KEY (OperationalEventId) REFERENCES
EventManagement.OperationalEvent(OperationalEventId),
    CONSTRAINT FK_EventManagement_EventSignalLink_Signal FOREIGN KEY (SignalId) REFERENCES Instrumentation.Signal(SignalId)
);
GO

CREATE TABLE EventManagement.EventEquipmentLink (
    EventEquipmentLinkId BIGINT IDENTITY(1,1) NOT NULL,
    OperationalEventId BIGINT NOT NULL,
    EquipmentId INT NOT NULL,
    LinkRole NVARCHAR(50) NOT NULL,
    Note NVARCHAR(500) NULL,

    CONSTRAINT PK_EventManagement_EventEquipmentLink PRIMARY KEY (EventEquipmentLinkId),
    CONSTRAINT UQ_EventManagement_EventEquipmentLink UNIQUE (OperationalEventId, EquipmentId, LinkRole),
    CONSTRAINT FK_EventManagement_EventEquipmentLink_OperationalEvent FOREIGN KEY (OperationalEventId) REFERENCES
EventManagement.OperationalEvent(OperationalEventId),
    CONSTRAINT FK_EventManagement_EventEquipmentLink_Equipment FOREIGN KEY (EquipmentId) REFERENCES
ReactorFleet.Equipment(EquipmentId)
);
GO

CREATE TABLE EventManagement.EventTimelineEntry (
    EventTimelineEntryId BIGINT IDENTITY(1,1) NOT NULL,
    OperationalEventId BIGINT NOT NULL,
    EventTimelineEntryTypeId INT NOT NULL,
    EntryAtUtc DATETIME2(7) NOT NULL,
    Title NVARCHAR(200) NOT NULL,
    Body NVARCHAR(2000) NULL,
    EnteredByUserId INT NULL,
    SourceReference NVARCHAR(200) NULL,

    CONSTRAINT PK_EventManagement_EventTimelineEntry PRIMARY KEY (EventTimelineEntryId),
    CONSTRAINT FK_EventManagement_EventTimelineEntry_OperationalEvent FOREIGN KEY (OperationalEventId) REFERENCES
EventManagement.OperationalEvent(OperationalEventId),
    CONSTRAINT FK_EventManagement_EventTimelineEntry_Type FOREIGN KEY (EventTimelineEntryTypeId) REFERENCES
EventManagement.EventTimelineEntryType(EventTimelineEntryTypeId),
    CONSTRAINT FK_EventManagement_EventTimelineEntry_EnteredByUser FOREIGN KEY (EnteredByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE EventManagement.EventParticipant (
    EventParticipantId BIGINT IDENTITY(1,1) NOT NULL,
    OperationalEventId BIGINT NOT NULL,
    EventParticipantRoleId INT NOT NULL,
    PersonId INT NULL,
    ApplicationUserId INT NULL,
```

```

        DisplayNameSnapshot NVARCHAR(200) NULL,
        JoinedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_EventManagement_EventParticipant_JoinedAtUtc DEFAULT
SYSUTCDATETIME(),
        LeftAtUtc DATETIME2(7) NULL,

        CONSTRAINT PK_EventManagement_EventParticipant PRIMARY KEY (EventParticipantId),
        CONSTRAINT FK_EventManagement_EventParticipant_OperationalEvent FOREIGN KEY (OperationalEventId) REFERENCES
EventManagement.OperationalEvent(OperationalEventId),
        CONSTRAINT FK_EventManagement_EventParticipant_Role FOREIGN KEY (EventParticipantRoleId) REFERENCES
EventManagement.EventParticipantRole(EventParticipantRoleId),
        CONSTRAINT FK_EventManagement_EventParticipant_Person FOREIGN KEY (PersonId) REFERENCES Organization.Person(PersonId),
        CONSTRAINT FK_EventManagement_EventParticipant_ApplicationUser FOREIGN KEY (ApplicationUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId),
        CONSTRAINT CK_EventManagement_EventParticipant_Identity CHECK (PersonId IS NOT NULL OR ApplicationUserId IS NOT NULL OR
DisplayNameSnapshot IS NOT NULL)
);
GO

CREATE TABLE EventManagement.EventNote (
    EventNoteId BIGINT IDENTITY(1,1) NOT NULL,
    OperationalEventId BIGINT NOT NULL,
    EventNoteTypeId INT NOT NULL,
    Body NVARCHAR(MAX) NOT NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_EventManagement_EventNote_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedByUserId INT NULL,
    IsInternalOnly BIT NOT NULL CONSTRAINT DF_EventManagement_EventNote_IsInternalOnly DEFAULT 1,

    CONSTRAINT PK_EventManagement_EventNote PRIMARY KEY (EventNoteId),
    CONSTRAINT FK_EventManagement_EventNote_OperationalEvent FOREIGN KEY (OperationalEventId) REFERENCES
EventManagement.OperationalEvent(OperationalEventId),
    CONSTRAINT FK_EventManagement_EventNote_Type FOREIGN KEY (EventNoteTypeId) REFERENCES
EventManagement.EventNoteType(EventNoteTypeId),
    CONSTRAINT FK_EventManagement_EventNote_CreatedByUser FOREIGN KEY (CreatedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE EventManagement.EventAttachment (
    EventAttachmentId BIGINT IDENTITY(1,1) NOT NULL,
    OperationalEventId BIGINT NOT NULL,
    EventAttachmentTypeId INT NOT NULL,
    FileName NVARCHAR(260) NOT NULL,
    ContentType NVARCHAR(100) NULL,
    StorageUri NVARCHAR(1000) NOT NULL,
    Sha256Hash CHAR(64) NULL,
    SizeBytes BIGINT NULL,
    UploadedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_EventManagement_EventAttachment_UploadedAtUtc DEFAULT
SYSUTCDATETIME(),
    UploadedByUserId INT NULL,

    CONSTRAINT PK_EventManagement_EventAttachment PRIMARY KEY (EventAttachmentId),
    CONSTRAINT FK_EventManagement_EventAttachment_OperationalEvent FOREIGN KEY (OperationalEventId) REFERENCES
EventManagement.OperationalEvent(OperationalEventId),
    CONSTRAINT FK_EventManagement_EventAttachment_Type FOREIGN KEY (EventAttachmentTypeId) REFERENCES
EventManagement.EventAttachmentType(EventAttachmentTypeId),
    CONSTRAINT FK_EventManagement_EventAttachment_UploadedByUser FOREIGN KEY (UploadedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

```

C.8.4.5 Impact, near miss, incident, actions and review

```

CREATE TABLE EventManagement.EventImpactAssessment (
    EventImpactAssessmentId BIGINT IDENTITY(1,1) NOT NULL,
    OperationalEventId BIGINT NOT NULL,
    EventImpactTypeId INT NOT NULL,
    ImpactLevel NVARCHAR(50) NOT NULL,
    EstimatedValue DECIMAL(18,4) NULL,
    EngineeringUnitId INT NULL,
    Narrative NVARCHAR(1000) NULL,

    CONSTRAINT PK_EventManagement_EventImpactAssessment PRIMARY KEY (EventImpactAssessmentId),
    CONSTRAINT FK_EventManagement_EventImpactAssessment_OperationalEvent FOREIGN KEY (OperationalEventId) REFERENCES
EventManagement.OperationalEvent(OperationalEventId),
    CONSTRAINT FK_EventManagement_EventImpactAssessment_ImpactType FOREIGN KEY (EventImpactTypeId) REFERENCES
EventManagement.EventImpactType(EventImpactTypeId),
    CONSTRAINT FK_EventManagement_EventImpactAssessment_EngineeringUnit FOREIGN KEY (EngineeringUnitId) REFERENCES
CorePlatform.EngineeringUnit(EngineeringUnitId)
);
GO

CREATE TABLE EventManagement.EventUnitStateSnapshot (
    EventUnitStateSnapshotId BIGINT IDENTITY(1,1) NOT NULL,
    OperationalEventId BIGINT NOT NULL,
    UnitId INT NOT NULL,
    SnapshotAtUtc DATETIME2(7) NOT NULL,
    OperationalModeText NVARCHAR(80) NULL,
    PowerMWe DECIMAL(12,3) NULL,
    ThermalPowerMWt DECIMAL(12,3) NULL,
    ReactorTripActive BIT NOT NULL CONSTRAINT DF_EventManagement_EventUnitStateSnapshot_ReactorTripActive DEFAULT 0,
    GridConnected BIT NOT NULL CONSTRAINT DF_EventManagement_EventUnitStateSnapshot_GridConnected DEFAULT 1,
    Notes NVARCHAR(1000) NULL,

    CONSTRAINT PK_EventManagement_EventUnitStateSnapshot PRIMARY KEY (EventUnitStateSnapshotId),
    CONSTRAINT FK_EventManagement_EventUnitStateSnapshot_OperationalEvent FOREIGN KEY (OperationalEventId) REFERENCES

```



```

EventManagement.OperationalEvent(OperationalEventId),
    CONSTRAINT FK_EventManagement_EventUnitStateSnapshot_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId)
);
GO

CREATE TABLE EventManagement.NearMiss (
    NearMissId BIGINT IDENTITY(1,1) NOT NULL,
    OperationalEventId BIGINT NOT NULL,
    NearMissTypeId INT NOT NULL,
    PotentialSeverityId INT NOT NULL,
    WhatPreventedEscalation NVARCHAR(1000) NULL,
    LessonsLearned NVARCHAR(1000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_EventManagement_NearMiss_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_EventManagement_NearMiss_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EventManagement_NearMiss PRIMARY KEY (NearMissId),
    CONSTRAINT UQ_EventManagement_NearMiss_OperationalEvent UNIQUE (OperationalEventId),
    CONSTRAINT FK_EventManagement_NearMiss_OperationalEvent FOREIGN KEY (OperationalEventId) REFERENCES
EventManagement.OperationalEvent(OperationalEventId),
    CONSTRAINT FK_EventManagement_NearMiss_NearMissType FOREIGN KEY (NearMissTypeId) REFERENCES
EventManagement.NearMissType(NearMissTypeId),
    CONSTRAINT FK_EventManagement_NearMiss_PotentialSeverity FOREIGN KEY (PotentialSeverityId) REFERENCES
EventManagement.EventSeverity(EventSeverityId)
);
GO

CREATE TABLE EventManagement.NearMissBarrier (
    NearMissBarrierId BIGINT IDENTITY(1,1) NOT NULL,
    NearMissId BIGINT NOT NULL,
    BarrierTypeId INT NOT NULL,
    Description NVARCHAR(500) NOT NULL,
    BarrierWorked BIT NOT NULL CONSTRAINT DF_EventManagement_NearMissBarrier_BarrierWorked DEFAULT 1,
    WeaknessNoted NVARCHAR(1000) NULL,

    CONSTRAINT PK_EventManagement_NearMissBarrier PRIMARY KEY (NearMissBarrierId),
    CONSTRAINT FK_EventManagement_NearMissBarrier_NearMiss FOREIGN KEY (NearMissId) REFERENCES EventManagement.NearMiss(NearMissId),
    CONSTRAINT FK_EventManagement_NearMissBarrier_BarrierType FOREIGN KEY (BarrierTypeId) REFERENCES
EventManagement.BarrierType(BarrierTypeId)
);
GO

CREATE TABLE EventManagement.Incident (
    IncidentId BIGINT IDENTITY(1,1) NOT NULL,
    OperationalEventId BIGINT NOT NULL,
    IncidentTypeId INT NOT NULL,
    IncidentStatusId INT NOT NULL,
    IncidentNumber NVARCHAR(80) NOT NULL,
    InvestigationSummary NVARCHAR(2000) NULL,
    OpenedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_EventManagement_Incident_OpenedAtUtc DEFAULT SYSUTCDATETIME(),
    ClosedAtUtc DATETIME2(7) NULL,
    LeadInvestigatorUserId INT NOT NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_EventManagement_Incident_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_EventManagement_Incident_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EventManagement_Incident PRIMARY KEY (IncidentId),
    CONSTRAINT UQ_EventManagement_Incident_OperationalEvent UNIQUE (OperationalEventId),
    CONSTRAINT UQ_EventManagement_Incident_IncidentNumber UNIQUE (IncidentNumber),
    CONSTRAINT FK_EventManagement_Incident_OperationalEvent FOREIGN KEY (OperationalEventId) REFERENCES
EventManagement.OperationalEvent(OperationalEventId),
    CONSTRAINT FK_EventManagement_Incident_IncidentType FOREIGN KEY (IncidentTypeId) REFERENCES
EventManagement.IncidentType(IncidentTypeId),
    CONSTRAINT FK_EventManagement_Incident_IncidentStatus FOREIGN KEY (IncidentStatusId) REFERENCES
EventManagement.IncidentStatus(IncidentStatusId),
    CONSTRAINT FK_EventManagement_Incident_LeadInvestigatorUser FOREIGN KEY (LeadInvestigatorUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE EventManagement.IncidentClassificationAssignment (
    IncidentClassificationAssignmentId BIGINT IDENTITY(1,1) NOT NULL,
    IncidentId BIGINT NOT NULL,
    IncidentClassificationId INT NOT NULL,
    AssignedAtUtc DATETIME2(7) NOT NULL CONSTRAINT
DF_EventManagement_IncidentClassificationAssignment_AssignedAtUtc DEFAULT SYSUTCDATETIME(),
    AssignedByUserId INT NOT NULL,

    CONSTRAINT PK_EventManagement_IncidentClassificationAssignment PRIMARY KEY (IncidentClassificationAssignmentId),
    CONSTRAINT UQ_EventManagement_IncidentClassificationAssignment UNIQUE (IncidentId, IncidentClassificationId),
    CONSTRAINT FK_EventManagement_IncidentClassificationAssignment_Incident FOREIGN KEY (IncidentId) REFERENCES
EventManagement.Incident(IncidentId),
    CONSTRAINT FK_EventManagement_IncidentClassificationAssignment_Classification FOREIGN KEY (IncidentClassificationId) REFERENCES
EventManagement.IncidentClassification(IncidentClassificationId),
    CONSTRAINT FK_EventManagement_IncidentClassificationAssignment_AssignedByUser FOREIGN KEY (AssignedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE EventManagement.IncidentAction (
    IncidentActionId BIGINT IDENTITY(1,1) NOT NULL,
    IncidentId BIGINT NOT NULL,
    IncidentActionTypeId INT NOT NULL,
    IncidentActionStatusId INT NOT NULL,
    Title NVARCHAR(250) NOT NULL,
    Description NVARCHAR(2000) NULL,
    DueAtUtc DATETIME2(7) NULL,

```



```

CompletedAtUtc          DATETIME2(7) NULL,
VerifiedAtUtc           DATETIME2(7) NULL,
VerifiedByUserId        INT NULL,
CreatedAtUtc            DATETIME2(7) NOT NULL CONSTRAINT DF_EventManagement_IncidentAction_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
CreatedBy              NVARCHAR(100) NOT NULL,
ModifiedAtUtc           DATETIME2(7) NOT NULL CONSTRAINT DF_EventManagement_IncidentAction_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
ModifiedBy              NVARCHAR(100) NOT NULL,
RowVersion              ROWVERSION NOT NULL,

CONSTRAINT PK_EventManagement_IncidentAction PRIMARY KEY (IncidentActionId),
CONSTRAINT FK_EventManagement_IncidentAction_Incident FOREIGN KEY (IncidentId) REFERENCES EventManagement.Incident(IncidentId),
CONSTRAINT FK_EventManagement_IncidentAction_Type FOREIGN KEY (IncidentActionTypeId) REFERENCES
EventManagement.IncidentActionType(IncidentActionTypeId),
CONSTRAINT FK_EventManagement_IncidentAction_Status FOREIGN KEY (IncidentActionStatusId) REFERENCES
EventManagement.IncidentActionStatus(IncidentActionStatusId),
CONSTRAINT FK_EventManagement_IncidentAction_VerifiedByUser FOREIGN KEY (VerifiedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE EventManagement.IncidentActionOwner (
    IncidentActionOwnerId BIGINT IDENTITY(1,1) NOT NULL,
    IncidentActionId      BIGINT NOT NULL,
    PersonId              INT NULL,
    ApplicationUserId      INT NULL,
    AssignedAtUtc         DATETIME2(7) NOT NULL CONSTRAINT DF_EventManagement_IncidentActionOwner_AssignedAtUtc DEFAULT
SYSUTCDATETIME(),
    AssignedByUserId      INT NULL,

CONSTRAINT PK_EventManagement_IncidentActionOwner PRIMARY KEY (IncidentActionOwnerId),
CONSTRAINT FK_EventManagement_IncidentActionOwner_IncidentAction FOREIGN KEY (IncidentActionId) REFERENCES
EventManagement.IncidentAction(IncidentActionId),
CONSTRAINT FK_EventManagement_IncidentActionOwner_Person FOREIGN KEY (PersonId) REFERENCES Organization.Person(PersonId),
CONSTRAINT FK_EventManagement_IncidentActionOwner_ApplicationUser FOREIGN KEY (ApplicationUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId),
CONSTRAINT FK_EventManagement_IncidentActionOwner_AssignedByUser FOREIGN KEY (AssignedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId),
CONSTRAINT CK_EventManagement_IncidentActionOwner_Identity CHECK (PersonId IS NOT NULL OR ApplicationUserId IS NOT NULL)
);
GO

CREATE TABLE EventManagement.IncidentReview (
    IncidentReviewId      BIGINT IDENTITY(1,1) NOT NULL,
    IncidentId            BIGINT NOT NULL,
    IncidentReviewOutcomeId INT NOT NULL,
    ReviewedAtUtc         DATETIME2(7) NOT NULL CONSTRAINT DF_EventManagement_IncidentReview_ReviewedAtUtc DEFAULT
SYSUTCDATETIME(),
    ReviewedByUserId      INT NULL,
    ReviewSummary         NVARCHAR(2000) NULL,

CONSTRAINT PK_EventManagement_IncidentReview PRIMARY KEY (IncidentReviewId),
CONSTRAINT FK_EventManagement_IncidentReview_Incident FOREIGN KEY (IncidentId) REFERENCES EventManagement.Incident(IncidentId),
CONSTRAINT FK_EventManagement_IncidentReview_Outcome FOREIGN KEY (IncidentReviewOutcomeId) REFERENCES
EventManagement.IncidentReviewOutcome(IncidentReviewOutcomeId),
CONSTRAINT FK_EventManagement_IncidentReview_ReviewedByUser FOREIGN KEY (ReviewedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE EventManagement.IncidentReviewFinding (
    IncidentReviewFindingId BIGINT IDENTITY(1,1) NOT NULL,
    IncidentReviewId        BIGINT NOT NULL,
    FindingText             NVARCHAR(2000) NOT NULL,
    RequiresAction          BIT NOT NULL CONSTRAINT DF_EventManagement_IncidentReviewFinding_RequiresAction DEFAULT 0,
    CreatedAtUtc            DATETIME2(7) NOT NULL CONSTRAINT DF_EventManagement_IncidentReviewFinding_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),

CONSTRAINT PK_EventManagement_IncidentReviewFinding PRIMARY KEY (IncidentReviewFindingId),
CONSTRAINT FK_EventManagement_IncidentReviewFinding_IncidentReview FOREIGN KEY (IncidentReviewId) REFERENCES
EventManagement.IncidentReview(IncidentReviewId)
);
GO

```

C.8.5 Indexes and seed examples

The most important access paths are unit-plus-time, status-plus-severity, alarm/flood backtracking, chronological timeline replay, and incident action due-date tracking.

```

CREATE INDEX IX_EventManagement_OperationalEvent_Unit_DetectedAtUtc
ON EventManagement.OperationalEvent(UnitId, DetectedAtUtc DESC);
GO

CREATE INDEX IX_EventManagement_OperationalEvent_Status_Severity
ON EventManagement.OperationalEvent(EventStatusId, EventSeverityId, DetectedAtUtc DESC);
GO

CREATE INDEX IX_EventManagement_EventAlarmLink_AlarmEventId
ON EventManagement.EventAlarmLink(AlarmEventId);
GO

```

```

CREATE INDEX IX_EventManagement_EventFloodLink_AlarmFloodId
ON EventManagement.EventFloodLink(AlarmFloodId);
GO

CREATE INDEX IX_EventManagement_EventTimelineEntry_Event_Time
ON EventManagement.EventTimelineEntry(OperationalEventId, EntryAtUtc ASC);
GO

CREATE INDEX IX_EventManagement_EventSignalLink_SignalId
ON EventManagement.EventSignalLink(SignalId);
GO

CREATE INDEX IX_EventManagement_EventEquipmentLink_EquipmentId
ON EventManagement.EventEquipmentLink(EquipmentId);
GO

CREATE INDEX IX_EventManagement_Incident_Status_Due
ON EventManagement.Incident(IncidentStatusId, OpenedAtUtc DESC);
GO

CREATE INDEX IX_EventManagement_IncidentAction_Status_Due
ON EventManagement.IncidentAction(IncidentActionStatusId, DueAtUtc);
GO

```

C.8.5.1 Seed examples

```

INSERT INTO EventManagement.EventType (Code, Name, DisplayOrder) VALUES
(N'INCIDENT', N'Incident', 10),
(N'NEAR MISS', N'Near miss', 20),
(N'TRANSIENT', N'Operational transient', 30),
(N'DRILL', N'Training drill event', 40),
(N'DIVERGENCE', N'Digital twin divergence', 50);

INSERT INTO EventManagement.EventStatus (Code, Name, DisplayOrder) VALUES
(N'NEW', N'New', 10),
(N'TRIAGED', N'Triaged', 20),
(N'INVESTIGATING', N'Investigating', 30),
(N'ACTIONED', N'Actioned', 40),
(N'CLOSED', N'Closed', 50),
(N'ARCHIVED', N'Archived', 60);

INSERT INTO EventManagement.EventSeverity (Code, Name, DisplayOrder) VALUES
(N'INFO', N'Information', 10),
(N'MINOR', N'Minor', 20),
(N'MODERATE', N'Moderate', 30),
(N'MAJOR', N'Major', 40),
(N'CRITICAL', N'Critical', 50);

INSERT INTO EventManagement.EventSourceType (Code, Name, DisplayOrder) VALUES
(N'ALARM_FLOOD', N'Alarm flood', 10),
(N'MANUAL', N'Manual report', 20),
(N'DIGITAL_TWIN', N'Digital twin', 30),
(N'DRILL', N'Drill', 40),
(N'IMPORT', N'Import', 50);

```

C.8.5.2 Verification queries

```

-- Event with its linked alarms and flood.
SELECT
    e.EventCode,
    e.Title,
    es.Code AS EventStatus,
    sev.Code AS EventSeverity,
    ae.AlarmEventId,
    af.AlarmFloodId
FROM EventManagement.OperationalEvent AS e
JOIN EventManagement.EventStatus AS es ON es.EventStatusId = e.EventStatusId
JOIN EventManagement.EventSeverity AS sev ON sev.EventSeverityId = e.EventSeverityId
LEFT JOIN EventManagement.EventAlarmLink AS eal ON eal.OperationalEventId = e.OperationalEventId
LEFT JOIN AlarmManagement.AlarmEvent AS ae ON ae.AlarmEventId = eal.AlarmEventId
LEFT JOIN EventManagement.EventFloodLink AS efl ON efl.OperationalEventId = e.OperationalEventId
LEFT JOIN AlarmManagement.AlarmFlood AS af ON af.AlarmFloodId = efl.AlarmFloodId
WHERE e.EventCode = N'EVT-2026-0418';

-- Timeline for one event, replayable in chronological order.
SELECT
    t.EntryAtUtc,
    tet.Code AS EntryType,
    t.Title,
    t.Body
FROM EventManagement.EventTimelineEntry AS t
JOIN EventManagement.EventTimelineEntryType AS tet ON tet.EventTimelineEntryTypeId = t.EventTimelineEntryTypeId
WHERE t.OperationalEventId = @OperationalEventId
ORDER BY t.EntryAtUtc;

-- Open incident actions that are overdue or nearing due date.
SELECT
    i.IncidentNumber,
    a.Title,
    s.Code AS ActionStatus,

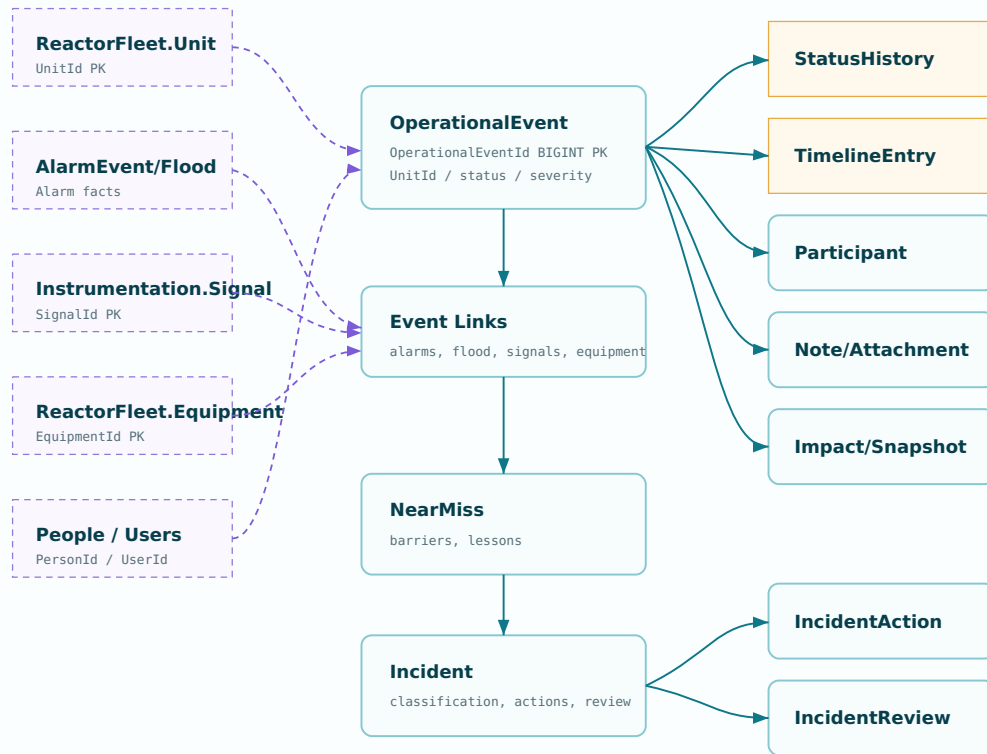
```

```

a.DueAtUtc
FROM EventManagement.IncidentAction AS a
JOIN EventManagement.Incident AS i ON i.IncidentId = a.IncidentId
JOIN EventManagement.IncidentActionStatus AS s ON s.IncidentActionStatusId = a.IncidentActionStatusId
WHERE s.Code NOT IN (N'COMPLETED', N'VERIFIED', N'CANCELLED')
ORDER BY a.DueAtUtc;

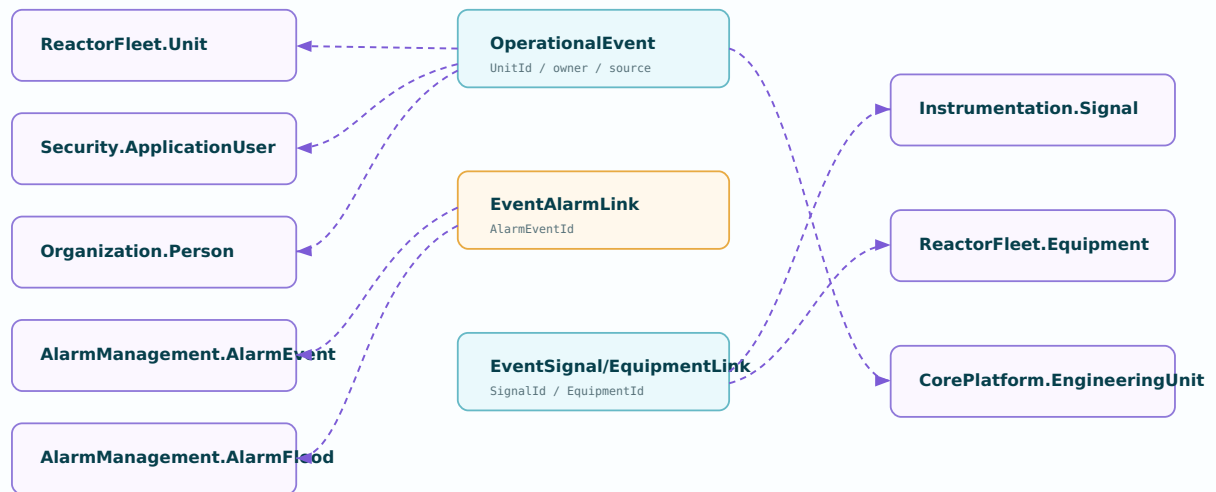
```

C.8.6 ER diagram



Solid edges stay inside EventManagement. Dashed violet edges are cross-schema passports to plant, alarm, signal, equipment and people.

Figure C.8.1 - EventManagement sector ER diagram. OperationalEvent is the hub; alarm links, timeline entries, participants, near misses and incidents hang from it.



EventManagement turns alarm facts into owned operational records. RootCause, Maintenance and Compliance later depend on these event records.

Figure C.8.2 - EventManagement passport map. The sector consumes plant, alarm, signal, equipment and people identities; later RootCause and Maintenance sectors consume the event identity.

C.8.7 Foreign-key mapping

The table below lists the main passports. Lookup-table foreign keys are present in the DDL but are not repeated line by line.

FROM TABLE	FK COLUMN	TO TABLE	PK COLUMN	RELATIONSHIP
OperationalEvent	UnitId	ReactorFleet.Unit	UnitId	Event belongs to the same physical unit used by alarms, twin, RCA and reporting.
OperationalEvent	PlantId	Organization.Plant	PlantId	Optional plant scope for site-level reporting.
OperationalEvent	ReportedByUserId	Security.ApplicationUser	ApplicationUserId	Accountable reporter when the event is user-created.
OperationalEvent	OwnerUserId	Security.ApplicationUser	ApplicationUserId	Accountable event owner.
OperationalEvent	OwningPersonId	Organization.Person	PersonId	Optional person ownership independent of login account.
EventAlarmLink	AlarmEventId	AlarmManagement.AlarmEvent	AlarmEventId	Operational event can be created from or supported by alarm facts.
EventFloodLink	AlarmFloodId	AlarmManagement.AlarmFlood	AlarmFloodId	Event can represent an alarm flood window.
EventSignalLink	SignalId	Instrumentation.Signal	SignalId	

FROM TABLE	FK COLUMN	TO TABLE	PK COLUMN	RELATIONSHIP
				Witness, context or affected signals remain canonical tags.
EventEquipmentLink	EquipmentId	ReactorFleet.Equipment	EquipmentId	Affected or suspected components use the component registry.
EventTimelineEntry	EnteredByUserId	Security.ApplicationUser	ApplicationUserId	Timeline entries can be traced to a user.
EventParticipant	PersonId	Organization.Person	PersonId	Participants can be people in the organization model.
EventImpactAssessment	EngineeringUnitId	CorePlatform.EngineeringUnit	EngineeringUnitId	Quantified impacts use platform unit definitions.
NearMiss	OperationalEventId	EventManager.OperationalEvent	OperationalEventId	Near miss is a specialized view of a base event.
Incident	OperationalEventId	EventManager.OperationalEvent	OperationalEventId	Incident is a specialized view of a base event.
IncidentAction	IncidentId	EventManager.Incident	IncidentId	Corrective/preventive actions belong to one incident.

OperationalEvent

FK UnitId
TO ReactorFleet.Unit

Keeps events unit-scoped and joinable to the physical plant.

OperationalEvent

FK OwnerUserId
TO Security.ApplicationUser

Makes event ownership explicit and auditable.

EventAlarmLink

FK AlarmEventId
TO AlarmManagement.AlarmEvent

Connects structured events to the raw alarm facts that triggered them.

EventFloodLink

FK AlarmFloodId
TO AlarmManagement.AlarmFlood

Lets one incident carry the whole alarm flood window without copying alarms.

EventSignalLink

FK SignalId
TO Instrumentation.Signal

Uses canonical tags for witness and context signals.

EventEquipmentLink

FK EquipmentId
TO ReactorFleet.Equipment

Uses the component registry for affected or suspected equipment.

EventParticipant

FK `PersonId`
TO `Organization.Person`

Connects the event record to the personnel model rather than copying names.

EventImpactAssessment

FK `EngineeringUnitId`
TO `CorePlatform.EngineeringUnit`

Quantified impact remains unit-safe.

C.8.8 Boundary and next sector

Honest boundary. This schema does not infer root cause and does not decide maintenance strategy. It records an accountable operational event and its follow-up. The causal case belongs to `RootCause`; the work order belongs to `Maintenance`; the compliance finding belongs to `Compliance`.

The next schema is `Maintenance`. It will use `ReactorFleet.Equipment` as its physical anchor and may use `EventManagement.IncidentAction` or `OperationalEvent` as the reason a work order, inspection or degradation review was opened.

Appendix C.9

Maintenance Schema

The Maintenance sector turns the physical plant into a maintainable register. ReactorFleet names the equipment; Maintenance tracks asset status, degradation, inspections, work packages, spare parts, schedules and approvals.

This schema is intentionally connected to EventManagement. An incident or operational event may open a work order, but Maintenance does not decide the root cause and does not own the event record. Its job is narrower and more practical: what asset is affected, what condition evidence exists, what work is planned, who owns it, which material is needed, and whether the work has been approved and closed.

Maintenance answers: what needs work, why it needs work, who will do it, what evidence supports it, what parts are required, and when the asset returns to an accountable state.

C.9.1

Sector purpose

C.9.2

Full table list

C.9.3

Lookup tables

C.9.4

SQL DDL

C.9.5

Indexes and seed examples

C.9.6

ER diagram

C.9.7

Foreign-key mapping

C.9.8

Boundary and next sector

C.9.1 Sector purpose

Maintenance depends on ReactorFleet, Organization, Security, Instrumentation, CorePlatform and EventManagement. Its anchor table is Asset. An asset may point to a physical ReactorFleet.Equipment row, but it adds maintenance meaning: status, criticality, spares, condition, degradation, plans and work.

- Asset is the maintainable view of equipment, not a duplicate physical registry.
- AssetCondition and DegradationRecord record health and degradation evidence.
- MaintenancePlan and MaintenanceSchedule define recurring or condition-based work.
- WorkOrder is the executable work package, optionally created from an event or incident action.

- `Inspection` and `InspectionFinding` create structured observations and follow-up actions.
- `SparePart`, `Supplier` and stock tables keep material planning relational instead of free-text.

Design choice. Physical identity stays in `ReactorFleet`. Maintenance adds maintainability, work execution, material planning and condition history. This prevents the same pump or valve from being described twice with competing identities.

C.9.2 Full table list

The Maintenance sector contains forty-six tables: eighteen typed lookup tables and twenty-eight substantive tables. The table count is deliberate: asset health, work execution, scheduling and material control have different lifecycles and should not be collapsed into one overloaded work-order table.

LOOKUP

Maintenance.AssetCategory

PK `AssetCategoryId`
FK None

Groups maintainable items: pump, valve, rod drive, turbine, generator, sensor, building service, robot support.

LOOKUP

Maintenance.AssetStatus

PK `AssetStatusId`
FK None

Runtime maintenance status: in service, degraded, out of service, under maintenance, retired.

LOOKUP

Maintenance.AssetCriticality

PK `AssetCriticalityId`
FK None

Business and engineering criticality used for priority, inspection frequency and reporting.

LOOKUP

Maintenance.ConditionGrade

PK `ConditionGradeId`
FK None

Condition grade from excellent to failed. Used by asset condition and degradation records.

LOOKUP

Maintenance.DegradationMechanism

PK `DegradationMechanismId`
FK None

Corrosion, wear, fatigue, thermal cycling, radiation embrittlement, fouling, vibration or obsolescence.

LOOKUP

Maintenance.InspectionType

PK `InspectionTypeId`
FK None

Visual, vibration, thermography, ultrasonic, radiography, functional, calibration or document inspection.

LOOKUP

Maintenance.FindingSeverity

PK FindingSeverityId

FK None

Severity assigned to inspection findings and active degradation records.

LOOKUP

Maintenance.WorkOrderType

PK WorkOrderTypeId

FK None

Corrective, preventive, predictive, inspection, calibration, modification or emergent work.

LOOKUP

Maintenance.WorkOrderStatus

PK WorkOrderStatusId

FK None

Draft, requested, approved, planned, in progress, blocked, complete, closed or cancelled.

LOOKUP

Maintenance.WorkPriority

PK WorkPriorityId

FK None

Priority used by the planner: routine, low, normal, high, urgent, outage-critical.

LOOKUP

Maintenance.TaskStatus

PK TaskStatusId

FK None

Task-level status independent of the parent work order lifecycle.

LOOKUP

Maintenance.MaintenancePlanType

PK MaintenancePlanTypeId

FK None

Preventive, predictive, condition-based, outage, vendor-recommended or regulatory plan.

LOOKUP

Maintenance.ScheduleBasis

PK ScheduleBasisId

FK None

Calendar interval, running hours, cycles, condition threshold, outage window or manual trigger.

LOOKUP

Maintenance.MaterialUsageType

PK MaterialUsageTypeId

FK None

Planned, reserved, issued, consumed, returned, scrapped or substituted material usage.

LOOKUP

Maintenance.ApprovalStatus

PK ApprovalStatusId

FK None

Approval lifecycle for work, documents and maintenance plans.

LOOKUP

Maintenance.SupplierStatus

PK SupplierStatusId

FK None

Approved, conditional, suspended, retired or evaluation supplier state.

LOOKUP**Maintenance.DocumentType**

PK DocumentTypeId

FK None

Procedure, vendor manual, work package, photo, certificate, calibration sheet or close-out report.

LOOKUP**Maintenance.LabourRole**

PK LabourRoleId

FK None

Planner, technician, engineer, supervisor, radiation protection, QA reviewer or contractor role.

ASSET**Maintenance.Asset**

PK AssetId

FK UnitId , EquipmentId , AssetCategoryId

Maintainable representation of a physical component or equipment item.

ASSET**Maintenance.AssetComponent**

PK AssetComponentId

FK AssetId , ParentAssetComponentId

Subcomponent tree below a maintainable asset: seal, bearing, actuator, motor winding, module.

ASSET**Maintenance.AssetStatusHistory**

PK AssetStatusHistoryId

FK AssetId , AssetStatusId

Append-only state changes for a maintainable asset.

CONDITION**Maintenance.AssetCondition**

PK AssetConditionId

FK AssetId , ConditionGradeId

Point-in-time condition assessment, health score and useful-life estimate.

CONDITION**Maintenance.AssetConditionMeasurement**

PK AssetConditionMeasurementId

FK AssetConditionId , SignalId

Measurement evidence used by a condition assessment.

CONDITION**Maintenance.DegradationRecord**

PK DegradationRecordId

FK AssetId , DegradationMechanismId

Active or historical degradation case for an asset.

CONDITION**Maintenance.DegradationTrendPoint**

PK DegradationTrendPointId

FK DegradationRecordId , EngineeringUnitId

Trend points for degradation rate, thickness loss, vibration growth or other condition metric.

INVENTORY**Maintenance.Supplier**

PK SupplierId

FK CountryId , SupplierStatusId

External supplier or vendor used for spares, services and documents.

INVENTORY**Maintenance.SparePart**

PK SparePartId

FK EngineeringUnitId

Spare part catalogue: part number, unit of issue, manufacturer and description.

INVENTORY**Maintenance.SupplierPart**

PK SupplierPartId

FK SupplierId , SparePartId

Supplier-specific part number, lead time and commercial reference.

INVENTORY**Maintenance.SparePartStock**

PK SparePartStockId
FK SparePartId, PlantId

On-hand, reserved and reorder inventory quantities.

INVENTORY**Maintenance.AssetSparePart**

PK AssetSparePartId
FK AssetId, SparePartId

Mapping of which spare parts are suitable for which assets.

PLAN**Maintenance.MaintenancePlan**

PK MaintenancePlanId
FK AssetId, MaintenancePlanTypeId

Preventive, predictive or condition-based plan for an asset or unit.

PLAN**Maintenance.MaintenancePlanTask**

PK MaintenancePlanTaskId
FK MaintenancePlanId

Reusable tasks that become work-order tasks when the plan is scheduled.

PLAN**Maintenance.MaintenanceSchedule**

PK MaintenanceScheduleId
FK MaintenancePlanId, ScheduleBasisId

Rule and next-due information for a maintenance plan.

PLAN**Maintenance.MaintenanceScheduleOccurrence**

PK MaintenanceScheduleOccurrenceId
FK MaintenanceScheduleId

Planned occurrence generated from a schedule before a work order exists.

PLAN**Maintenance.MaintenanceWindow**

PK MaintenanceWindowId
FK UnitId, CalendarId

Allowed or planned time window for maintenance work.

WORK**Maintenance.WorkOrder**

PK WorkOrderId
FK UnitId, AssetId, WorkOrderStatusId

Executable work package created from a plan, event, incident action or manual request.

WORK**Maintenance.WorkOrderStatusHistory**

PK WorkOrderStatusHistoryId
FK WorkOrderId, WorkOrderStatusId

Append-only lifecycle history for a work order.

WORK**Maintenance.WorkOrderTask**

PK WorkOrderTaskId
FK WorkOrderId, TaskStatusId

Task list inside a work order.

WORK**Maintenance.WorkOrderLabour**

PK WorkOrderLabourId
FK WorkOrderId, LabourRoleId

Estimated or actual labour assignment and hours.

WORK**Maintenance.WorkOrderMaterial**

PK WorkOrderMaterialId
FK WorkOrderId, SparePartId

Planned, reserved, issued or consumed material for a work order.

WORK Maintenance.WorkOrderEventLink PK WorkOrderEventLinkId FK WorkOrderId , OperationalEventId Why a work order was opened, connected to operational events and incident actions.	INSPECTION Maintenance.Inspection PK InspectionId FK AssetId , InspectionTypeId , WorkOrderId An inspection performed or planned for an asset.
INSPECTION Maintenance.InspectionFinding PK InspectionFindingId FK InspectionId , FindingSeverityId Finding, nonconformance or observation produced by an inspection.	EVIDENCE Maintenance.MaintenanceDocument PK MaintenanceDocumentId FK AssetId , WorkOrderId , DocumentTypeId Metadata for procedures, manuals, photos, certificates and work-package files.
EVIDENCE Maintenance.MaintenanceApproval PK MaintenanceApprovalId FK ApprovalStatusId , ApproverUserId Approval record for work orders, plans, documents or inspection close-out.	

C.9.3 Lookup tables

Lookup tables keep status, priority, condition, work type, schedule basis, supplier state and approval state stable. They also make EF Core navigation explicit and keep reporting from depending on text strings.

LOOKUP TABLE	TYPICAL SEED CODES	MAIN CONSUMER
AssetCategory	PUMP , VALVE , ROD_DRIVE , TURBINE , GENERATOR , SENSOR	Referenced by Asset .
AssetStatus	IN_SERVICE , DEGRADED , OUT_OF_SERVICE , MAINTENANCE , RETIRED	Referenced by Asset and status history.
AssetCriticality	LOW , MEDIUM , HIGH , CRITICAL , SAFETY_RELATED	Referenced by Asset .
ConditionGrade	A , B , C , D , E , FAILED	Referenced by condition and degradation.
DegradationMechanism	CORROSION , WEAR , FATIGUE , VIBRATION , THERMAL_CYCLE , OBSOLESCENCE	Referenced by DegradationRecord .
InspectionType	VISUAL , VIBRATION , THERMOGRAPHY , ULTRASONIC , CALIBRATION	Referenced by Inspection .

LOOKUP TABLE	TYPICAL SEED CODES	MAIN CONSUMER
FindingSeverity	INFO, MINOR, MAJOR, CRITICAL	Referenced by findings and degradation.
WorkOrderType	CORRECTIVE, PREVENTIVE, PREDICTIVE, INSPECTION, CALIBRATION	Referenced by WorkOrder.
WorkOrderStatus	DRAFT, REQUESTED, APPROVED, PLANNED, IN_PROGRESS, COMPLETE, CLOSED	Referenced by work orders and history.
WorkPriority	ROUTINE, LOW, NORMAL, HIGH, URGENT, OUTAGE_CRITICAL	Referenced by work orders.
TaskStatus	NOT_STARTED, IN_PROGRESS, BLOCKED, DONE, SKIPPED	Referenced by work order tasks.
MaintenancePlanType	PREVENTIVE, PREDICTIVE, CONDITION_BASED, OUTAGE, REGULATORY	Referenced by plans.
ScheduleBasis	CALENDAR, RUN_HOURS, CYCLES, CONDITION, OUTAGE_WINDOW	Referenced by schedules.
MaterialUsageType	PLANNED, RESERVED, ISSUED, CONSUMED, RETURNED	Referenced by materials.
ApprovalStatus	PENDING, APPROVED, REJECTED, RETURNED, CANCELLED	Referenced by approvals.
SupplierStatus	APPROVED, CONDITIONAL, SUSPENDED, RETIRED	Referenced by suppliers.
DocumentType	PROCEDURE, MANUAL, WORK_PACKAGE, PHOTO, CERTIFICATE, REPORT	Referenced by documents.
LabourRole	TECHNICIAN, ENGINEER, SUPERVISOR, QA, RP, CONTRACTOR	Referenced by labour rows.

C.9.4 SQL DDL

C.9.4.1 Schema

```
IF SCHEMA_ID(N'Maintenance') IS NULL
EXEC(N'CREATE SCHEMA Maintenance');
GO
```

C.9.4.2 Maintenance lookup tables

```
CREATE TABLE Maintenance.AssetCategory (
  AssetCategoryId INT IDENTITY(1,1) NOT NULL,
  Code NVARCHAR(50) NOT NULL,
  Name NVARCHAR(150) NOT NULL,
  Description NVARCHAR(500) NULL,
  DisplayOrder INT NOT NULL
  CONSTRAINT DF_Maintenance_AssetCategory_DisplayOrder DEFAULT 0,
  IsActive BIT NOT NULL
  CONSTRAINT DF_Maintenance_AssetCategory_IsActive DEFAULT 1,
  CreatedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_Maintenance_AssetCategory_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_Maintenance_AssetCategory_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  RowVersion ROWVERSION NOT NULL,
  CONSTRAINT PK_Maintenance_AssetCategory
    PRIMARY KEY (AssetCategoryId),
  CONSTRAINT UQ_Maintenance_AssetCategory_Code
    UNIQUE (Code)
```

```

);
GO

CREATE TABLE Maintenance.AssetStatus (
    AssetStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Maintenance_AssetStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Maintenance_AssetStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Maintenance_AssetStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Maintenance_AssetStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Maintenance_AssetStatus
        PRIMARY KEY (AssetStatusId),
    CONSTRAINT UQ_Maintenance_AssetStatus_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Maintenance.AssetCriticality (
    AssetCriticalityId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Maintenance_AssetCriticality_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Maintenance_AssetCriticality_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Maintenance_AssetCriticality_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Maintenance_AssetCriticality_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Maintenance_AssetCriticality
        PRIMARY KEY (AssetCriticalityId),
    CONSTRAINT UQ_Maintenance_AssetCriticality_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Maintenance.ConditionGrade (
    ConditionGradeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Maintenance_ConditionGrade_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Maintenance_ConditionGrade_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Maintenance_ConditionGrade_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Maintenance_ConditionGrade_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Maintenance_ConditionGrade
        PRIMARY KEY (ConditionGradeId),
    CONSTRAINT UQ_Maintenance_ConditionGrade_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Maintenance.DegradationMechanism (
    DegradationMechanismId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Maintenance_DegradationMechanism_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Maintenance_DegradationMechanism_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Maintenance_DegradationMechanism_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Maintenance_DegradationMechanism_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Maintenance_DegradationMechanism
        PRIMARY KEY (DegradationMechanismId),
    CONSTRAINT UQ_Maintenance_DegradationMechanism_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Maintenance.InspectionType (
    InspectionTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Maintenance_InspectionType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Maintenance_InspectionType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Maintenance_InspectionType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Maintenance_InspectionType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Maintenance_InspectionType
        PRIMARY KEY (InspectionTypeId),
    CONSTRAINT UQ_Maintenance_InspectionType_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Maintenance.FindingSeverity (
    FindingSeverityId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Maintenance_FindingSeverity_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Maintenance_FindingSeverity_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL

```

```

        CONSTRAINT DF_Maintenance_FindingSeverity_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
        ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Maintenance_FindingSeverity_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
        RowVersion ROWVERSION NOT NULL,

        CONSTRAINT PK_Maintenance_FindingSeverity
        PRIMARY KEY (FindingSeverityId),
        CONSTRAINT UQ_Maintenance_FindingSeverity_Code
        UNIQUE (Code)
    );
GO

CREATE TABLE Maintenance.WorkOrderType (
    WorkOrderTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_Maintenance_WorkOrderType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_Maintenance_WorkOrderType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Maintenance_WorkOrderType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Maintenance_WorkOrderType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Maintenance_WorkOrderType
    PRIMARY KEY (WorkOrderTypeId),
    CONSTRAINT UQ_Maintenance_WorkOrderType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Maintenance.WorkOrderStatus (
    WorkOrderStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_Maintenance_WorkOrderStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_Maintenance_WorkOrderStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Maintenance_WorkOrderStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Maintenance_WorkOrderStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Maintenance_WorkOrderStatus
    PRIMARY KEY (WorkOrderStatusId),
    CONSTRAINT UQ_Maintenance_WorkOrderStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Maintenance.WorkPriority (
    WorkPriorityId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_Maintenance_WorkPriority_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_Maintenance_WorkPriority_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Maintenance_WorkPriority_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Maintenance_WorkPriority_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Maintenance_WorkPriority
    PRIMARY KEY (WorkPriorityId),
    CONSTRAINT UQ_Maintenance_WorkPriority_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Maintenance.TaskStatus (
    TaskStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_Maintenance_TaskStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_Maintenance_TaskStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Maintenance_TaskStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Maintenance_TaskStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Maintenance_TaskStatus
    PRIMARY KEY (TaskStatusId),
    CONSTRAINT UQ_Maintenance_TaskStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Maintenance.MaintenancePlanType (
    MaintenancePlanTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_Maintenance_MaintenancePlanType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_Maintenance_MaintenancePlanType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Maintenance_MaintenancePlanType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Maintenance_MaintenancePlanType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Maintenance_MaintenancePlanType
    PRIMARY KEY (MaintenancePlanTypeId),
    CONSTRAINT UQ_Maintenance_MaintenancePlanType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Maintenance.ScheduleBasis (

```

```

ScheduleBasisId      INT IDENTITY(1,1) NOT NULL,
Code                 NVARCHAR(50) NOT NULL,
Name                 NVARCHAR(150) NOT NULL,
Description           NVARCHAR(500) NULL,
DisplayOrder         INT NOT NULL
    CONSTRAINT DF_Maintenance_ScheduleBasis_DisplayOrder DEFAULT 0,
IsActive             BIT NOT NULL
    CONSTRAINT DF_Maintenance_ScheduleBasis_IsActive DEFAULT 1,
CreatedAtUtc          DATETIME2(7) NOT NULL
    CONSTRAINT DF_Maintenance_ScheduleBasis_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc          DATETIME2(7) NOT NULL
    CONSTRAINT DF_Maintenance_ScheduleBasis_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion            ROWVERSION NOT NULL,

CONSTRAINT PK_Maintenance_ScheduleBasis
    PRIMARY KEY (ScheduleBasisId),
CONSTRAINT UQ_Maintenance_ScheduleBasis_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Maintenance.MaterialUsageType (
    MaterialUsageTypeId INT IDENTITY(1,1) NOT NULL,
    Code                 NVARCHAR(50) NOT NULL,
    Name                 NVARCHAR(150) NOT NULL,
    Description           NVARCHAR(500) NULL,
    DisplayOrder         INT NOT NULL
    CONSTRAINT DF_Maintenance_MaterialUsageType_DisplayOrder DEFAULT 0,
    IsActive             BIT NOT NULL
    CONSTRAINT DF_Maintenance_MaterialUsageType_IsActive DEFAULT 1,
    CreatedAtUtc          DATETIME2(7) NOT NULL
    CONSTRAINT DF_Maintenance_MaterialUsageType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc          DATETIME2(7) NOT NULL
    CONSTRAINT DF_Maintenance_MaterialUsageType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion            ROWVERSION NOT NULL,

CONSTRAINT PK_Maintenance_MaterialUsageType
    PRIMARY KEY (MaterialUsageTypeId),
CONSTRAINT UQ_Maintenance_MaterialUsageType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Maintenance.ApprovalStatus (
    ApprovalStatusId     INT IDENTITY(1,1) NOT NULL,
    Code                 NVARCHAR(50) NOT NULL,
    Name                 NVARCHAR(150) NOT NULL,
    Description           NVARCHAR(500) NULL,
    DisplayOrder         INT NOT NULL
    CONSTRAINT DF_Maintenance_ApprovalStatus_DisplayOrder DEFAULT 0,
    IsActive             BIT NOT NULL
    CONSTRAINT DF_Maintenance_ApprovalStatus_IsActive DEFAULT 1,
    CreatedAtUtc          DATETIME2(7) NOT NULL
    CONSTRAINT DF_Maintenance_ApprovalStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc          DATETIME2(7) NOT NULL
    CONSTRAINT DF_Maintenance_ApprovalStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion            ROWVERSION NOT NULL,

CONSTRAINT PK_Maintenance_ApprovalStatus
    PRIMARY KEY (ApprovalStatusId),
CONSTRAINT UQ_Maintenance_ApprovalStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Maintenance.SupplierStatus (
    SupplierStatusId     INT IDENTITY(1,1) NOT NULL,
    Code                 NVARCHAR(50) NOT NULL,
    Name                 NVARCHAR(150) NOT NULL,
    Description           NVARCHAR(500) NULL,
    DisplayOrder         INT NOT NULL
    CONSTRAINT DF_Maintenance_SupplierStatus_DisplayOrder DEFAULT 0,
    IsActive             BIT NOT NULL
    CONSTRAINT DF_Maintenance_SupplierStatus_IsActive DEFAULT 1,
    CreatedAtUtc          DATETIME2(7) NOT NULL
    CONSTRAINT DF_Maintenance_SupplierStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc          DATETIME2(7) NOT NULL
    CONSTRAINT DF_Maintenance_SupplierStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion            ROWVERSION NOT NULL,

CONSTRAINT PK_Maintenance_SupplierStatus
    PRIMARY KEY (SupplierStatusId),
CONSTRAINT UQ_Maintenance_SupplierStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Maintenance.DocumentType (
    DocumentTypeId       INT IDENTITY(1,1) NOT NULL,
    Code                 NVARCHAR(50) NOT NULL,
    Name                 NVARCHAR(150) NOT NULL,
    Description           NVARCHAR(500) NULL,
    DisplayOrder         INT NOT NULL
    CONSTRAINT DF_Maintenance_DocumentType_DisplayOrder DEFAULT 0,
    IsActive             BIT NOT NULL
    CONSTRAINT DF_Maintenance_DocumentType_IsActive DEFAULT 1,
    CreatedAtUtc          DATETIME2(7) NOT NULL
    CONSTRAINT DF_Maintenance_DocumentType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc          DATETIME2(7) NOT NULL
    CONSTRAINT DF_Maintenance_DocumentType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion            ROWVERSION NOT NULL,

CONSTRAINT PK_Maintenance_DocumentType
    PRIMARY KEY (DocumentTypeId),
CONSTRAINT UQ_Maintenance_DocumentType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Maintenance.LabourRole (
    LabourRoleId         INT IDENTITY(1,1) NOT NULL,
    Code                 NVARCHAR(50) NOT NULL,
    Name                 NVARCHAR(150) NOT NULL,
    Description           NVARCHAR(500) NULL,
    DisplayOrder         INT NOT NULL
    CONSTRAINT DF_Maintenance_LabourRole_DisplayOrder DEFAULT 0,
    IsActive             BIT NOT NULL
    CONSTRAINT DF_Maintenance_LabourRole_IsActive DEFAULT 1,
    CreatedAtUtc          DATETIME2(7) NOT NULL
    CONSTRAINT DF_Maintenance_LabourRole_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc          DATETIME2(7) NOT NULL
    CONSTRAINT DF_Maintenance_LabourRole_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion            ROWVERSION NOT NULL,

```



```

CONSTRAINT PK_Maintenance_LabourRole
PRIMARY KEY (LabourRoleId),
CONSTRAINT UQ_Maintenance_LabourRole_Code
UNIQUE (Code)
);
GO

```

C.9.4.3 Assets, condition and degradation

```

CREATE TABLE Maintenance.Asset (
    AssetId INT IDENTITY(1,1) NOT NULL,
    UnitId INT NOT NULL,
    EquipmentId INT NULL,
    SystemId INT NULL,
    AssetCategoryId INT NOT NULL,
    AssetStatusId INT NOT NULL,
    AssetCriticalityId INT NOT NULL,
    AssetCode NVARCHAR(80) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    Description NVARCHAR(1000) NULL,
    LocationText NVARCHAR(250) NULL,
    Manufacturer NVARCHAR(150) NULL,
    ModelNumber NVARCHAR(120) NULL,
    SerialNumber NVARCHAR(120) NULL,
    CommissionedAtUtc DATETIME2(7) NULL,
    RetiredAtUtc DATETIME2(7) NULL,
    IsSafetyRelated BIT NOT NULL CONSTRAINT DF_Maintenance_Asset_IsSafetyRelated DEFAULT 0,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Maintenance_Asset_CreatedAtUtc DEFAULT
    SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Maintenance_Asset_ModifiedAtUtc DEFAULT
    SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Maintenance_Asset_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Maintenance_Asset PRIMARY KEY (AssetId),
    CONSTRAINT UQ_Maintenance_Asset_AssetCode UNIQUE (AssetCode),
    CONSTRAINT FK_Maintenance_Asset_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_Maintenance_Asset_Equipment FOREIGN KEY (EquipmentId) REFERENCES
    ReactorFleet.Equipment(EquipmentId),
    CONSTRAINT FK_Maintenance_Asset_System FOREIGN KEY (SystemId) REFERENCES
    ReactorFleet.System(SystemId),
    CONSTRAINT FK_Maintenance_Asset_AssetCategory FOREIGN KEY (AssetCategoryId) REFERENCES
    Maintenance.AssetCategory(AssetCategoryId),
    CONSTRAINT FK_Maintenance_Asset_AssetStatus FOREIGN KEY (AssetStatusId) REFERENCES
    Maintenance.AssetStatus(AssetStatusId),
    CONSTRAINT FK_Maintenance_Asset_AssetCriticality FOREIGN KEY (AssetCriticalityId) REFERENCES
    Maintenance.AssetCriticality(AssetCriticalityId)
);
GO

CREATE TABLE Maintenance.AssetComponent (
    AssetComponentId INT IDENTITY(1,1) NOT NULL,
    AssetId INT NOT NULL,
    ParentAssetComponentId INT NULL,
    ComponentCode NVARCHAR(80) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    Description NVARCHAR(1000) NULL,
    IsReplaceable BIT NOT NULL CONSTRAINT DF_Maintenance_AssetComponent_IsReplaceable DEFAULT
    1,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Maintenance_AssetComponent_CreatedAtUtc
    DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Maintenance_AssetComponent_ModifiedAtUtc
    DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Maintenance_AssetComponent_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Maintenance_AssetComponent PRIMARY KEY (AssetComponentId),
    CONSTRAINT UQ_Maintenance_AssetComponent_Asset_ComponentCode UNIQUE (AssetId, ComponentCode),
    CONSTRAINT FK_Maintenance_AssetComponent_Asset FOREIGN KEY (AssetId) REFERENCES
    Maintenance.Asset(AssetId),
    CONSTRAINT FK_Maintenance_AssetComponent_Parent FOREIGN KEY (ParentAssetComponentId) REFERENCES
    Maintenance.AssetComponent(AssetComponentId)
);
GO

CREATE TABLE Maintenance.AssetStatusHistory (
    AssetStatusHistoryId BIGINT IDENTITY(1,1) NOT NULL,
    AssetId INT NOT NULL,
    AssetStatusId INT NOT NULL,
    ChangedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Maintenance_AssetStatusHistory_ChangedAtUtc
    DEFAULT SYSUTCDATETIME(),
    ChangedByUserId INT NULL,
    Reason NVARCHAR(500) NULL,

    CONSTRAINT PK_Maintenance_AssetStatusHistory PRIMARY KEY (AssetStatusHistoryId),
    CONSTRAINT FK_Maintenance_AssetStatusHistory_Asset FOREIGN KEY (AssetId) REFERENCES
    Maintenance.Asset(AssetId),
    CONSTRAINT FK_Maintenance_AssetStatusHistory_AssetStatus FOREIGN KEY (AssetStatusId) REFERENCES
    Maintenance.AssetStatus(AssetStatusId),
    CONSTRAINT FK_Maintenance_AssetStatusHistory_ChangedByUser FOREIGN KEY (ChangedByUserId) REFERENCES
    Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE Maintenance.AssetCondition (
    AssetConditionId BIGINT IDENTITY(1,1) NOT NULL,
    AssetId INT NOT NULL,
    ConditionGradeId INT NOT NULL,
    AssessedAtUtc DATETIME2(7) NOT NULL,
    AssessedByUserId INT NULL,
    HealthScorePercent DECIMAL(5,2) NULL,
    RemainingUsefulLifeDays INT NULL,
    Basis NVARCHAR(500) NULL,
    Notes NVARCHAR(2000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT
    DF_Maintenance_AssetCondition_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,

    CONSTRAINT PK_Maintenance_AssetCondition PRIMARY KEY (AssetConditionId),
    CONSTRAINT CK_Maintenance_AssetCondition_HealthScore CHECK (HealthScorePercent IS NULL OR
    (HealthScorePercent >= 0 AND HealthScorePercent <= 100)),
    CONSTRAINT FK_Maintenance_AssetCondition_Asset FOREIGN KEY (AssetId) REFERENCES
    Maintenance.Asset(AssetId),
    CONSTRAINT FK_Maintenance_AssetCondition_ConditionGrade FOREIGN KEY (ConditionGradeId) REFERENCES
    Maintenance.ConditionGrade(ConditionGradeId),
    CONSTRAINT FK_Maintenance_AssetCondition_AssessedByUser FOREIGN KEY (AssessedByUserId) REFERENCES
    Security.ApplicationUser(ApplicationUserId)
);
GO

```

```

);
GO

CREATE TABLE Maintenance.AssetConditionMeasurement (
    AssetConditionMeasurementId BIGINT IDENTITY(1,1) NOT NULL,
    AssetConditionId BIGINT NOT NULL,
    SignalId INT NOT NULL,
    EngineeringUnitId INT NOT NULL,
    MeasuredValue FLOAT NOT NULL,
    MeasuredAtUtc DATETIME2(7) NOT NULL,
    MeasurementNote NVARCHAR(500) NULL,

    CONSTRAINT PK_Maintenance_AssetConditionMeasurement PRIMARY KEY (AssetConditionMeasurementId),
    CONSTRAINT FK_Maintenance_AssetConditionMeasurement_AssetCondition FOREIGN KEY (AssetConditionId)
    REFERENCES Maintenance.AssetCondition(AssetConditionId),
    CONSTRAINT FK_Maintenance_AssetConditionMeasurement_Signal FOREIGN KEY (SignalId) REFERENCES
    Instrumentation.Signal(SignalId),
    CONSTRAINT FK_Maintenance_AssetConditionMeasurement_EngineeringUnit FOREIGN KEY (EngineeringUnitId)
    REFERENCES CorePlatform.EngineeringUnit(EngineeringUnitId)
);
GO

CREATE TABLE Maintenance.DegradationRecord (
    DegradationRecordId BIGINT IDENTITY(1,1) NOT NULL,
    AssetId INT NOT NULL,
    AssetComponentId INT NOT NULL,
    DegradationMechanismId INT NOT NULL,
    FindingSeverityId INT NOT NULL,
    ConditionGradeId INT NOT NULL,
    DetectedAtUtc DATETIME2(7) NOT NULL,
    DetectedByUserId INT NOT NULL,
    Description NVARCHAR(2000) NOT NULL,
    EstimatedRatePerYear DECIMAL(12,6) NULL,
    IsActive BIT NOT NULL CONSTRAINT DF_Maintenance_DegradationRecord_IsActive DEFAULT
    1,
    ClosedAtUtc DATETIME2(7) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT
    DF_Maintenance_DegradationRecord_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT
    DF_Maintenance_DegradationRecord_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Maintenance_DegradationRecord PRIMARY KEY (DegradationRecordId),
    CONSTRAINT FK_Maintenance_DegradationRecord_Asset FOREIGN KEY (AssetId) REFERENCES
    Maintenance.Asset(AssetId),
    CONSTRAINT FK_Maintenance_DegradationRecord_AssetComponent FOREIGN KEY (AssetComponentId) REFERENCES
    Maintenance.AssetComponent(AssetComponentId),
    CONSTRAINT FK_Maintenance_DegradationRecord_DegradationMechanism FOREIGN KEY
    (DegradationMechanismId) REFERENCES Maintenance.DegradationMechanism(DegradationMechanismId),
    CONSTRAINT FK_Maintenance_DegradationRecord_FindingSeverity FOREIGN KEY (FindingSeverityId)
    REFERENCES Maintenance.FindingSeverity(FindingSeverityId),
    CONSTRAINT FK_Maintenance_DegradationRecord_ConditionGrade FOREIGN KEY (ConditionGradeId) REFERENCES
    Maintenance.ConditionGrade(ConditionGradeId),
    CONSTRAINT FK_Maintenance_DegradationRecord_DetectedByUser FOREIGN KEY (DetectedByUserId) REFERENCES
    Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE Maintenance.DegradationTrendPoint (
    DegradationTrendPointId BIGINT IDENTITY(1,1) NOT NULL,
    DegradationRecordId BIGINT NOT NULL,
    EngineeringUnitId INT NOT NULL,
    SourceSignalId INT NOT NULL,
    MeasuredAtUtc DATETIME2(7) NOT NULL,
    Value FLOAT NOT NULL,
    Note NVARCHAR(500) NULL,

    CONSTRAINT PK_Maintenance_DegradationTrendPoint PRIMARY KEY (DegradationTrendPointId),
    CONSTRAINT FK_Maintenance_DegradationTrendPoint_DegradationRecord FOREIGN KEY (DegradationRecordId)
    REFERENCES Maintenance.DegradationRecord(DegradationRecordId),
    CONSTRAINT FK_Maintenance_DegradationTrendPoint_EngineeringUnit FOREIGN KEY (EngineeringUnitId)
    REFERENCES CorePlatform.EngineeringUnit(EngineeringUnitId),
    CONSTRAINT FK_Maintenance_DegradationTrendPoint_SourceSignal FOREIGN KEY (SourceSignalId) REFERENCES
    Instrumentation.Signal(SignalId)
);
GO

```

C.9.4.4 Suppliers, spares, plans and schedules

```

CREATE TABLE Maintenance.Supplier (
    SupplierId INT IDENTITY(1,1) NOT NULL,
    SupplierStatusId INT NOT NULL,
    CountryId INT NOT NULL,
    SupplierCode NVARCHAR(80) NOT NULL,
    Name NVARCHAR(250) NOT NULL,
    ContactEmail NVARCHAR(250) NULL,
    ContactPhone NVARCHAR(80) NULL,
    WebsiteUrl NVARCHAR(500) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Maintenance_Supplier_CreatedAtUtc DEFAULT
    SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Maintenance_Supplier_ModifiedAtUtc DEFAULT
    SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Maintenance_Supplier_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Maintenance_Supplier PRIMARY KEY (SupplierId),
    CONSTRAINT UQ_Maintenance_Supplier_Code UNIQUE (SupplierCode),
    CONSTRAINT FK_Maintenance_Supplier_SupplierStatus FOREIGN KEY (SupplierStatusId) REFERENCES
    Maintenance.SupplierStatus(SupplierStatusId),
    CONSTRAINT FK_Maintenance_Supplier_Country FOREIGN KEY (CountryId) REFERENCES
    CorePlatform.Country(CountryId)
);
GO

CREATE TABLE Maintenance.SparePart (
    SparePartId INT IDENTITY(1,1) NOT NULL,
    EngineeringUnitId INT NOT NULL,
    PartNumber NVARCHAR(120) NOT NULL,
    Name NVARCHAR(250) NOT NULL,
    Description NVARCHAR(1000) NULL,
    Manufacturer NVARCHAR(150) NULL,
    IsCriticalSpare BIT NOT NULL CONSTRAINT DF_Maintenance_SparePart_IsCriticalSpare DEFAULT 0,
    IsActive BIT NOT NULL CONSTRAINT DF_Maintenance_SparePart_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Maintenance_SparePart_CreatedAtUtc DEFAULT
    SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Maintenance_SparePart_ModifiedAtUtc DEFAULT
    SYSUTCDATETIME()
);
GO

```

```

SYSUTCDATETIME()),
ModifiedBy NVARCHAR(100) NOT NULL,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_Maintenance_SparePart PRIMARY KEY (SparePartId),
CONSTRAINT UQ_Maintenance_SparePart_PartNumber UNIQUE (PartNumber),
CONSTRAINT FK_Maintenance_SparePart_EngineeringUnit FOREIGN KEY (EngineeringUnitId) REFERENCES
CorePlatform.EngineeringUnit(EngineeringUnitId)
);
GO

CREATE TABLE Maintenance.SupplierPart (
    SupplierPartId INT IDENTITY(1,1) NOT NULL,
    SupplierId INT NOT NULL,
    SparePartId INT NOT NULL,
    SupplierPartNumber NVARCHAR(120) NOT NULL,
    LeadTimeDays INT NULL,
    MinimumOrderQuantity DECIMAL(18,4) NULL,
    IsPreferred BIT NOT NULL CONSTRAINT DF_Maintenance_SupplierPart_IsPreferred DEFAULT 0,

    CONSTRAINT PK_Maintenance_SupplierPart PRIMARY KEY (SupplierPartId),
    CONSTRAINT UQ_Maintenance_SupplierPart UNIQUE (SupplierId, SparePartId, SupplierPartNumber),
    CONSTRAINT FK_Maintenance_SupplierPart_Supplier FOREIGN KEY (SupplierId) REFERENCES
Maintenance.Supplier(SupplierId),
    CONSTRAINT FK_Maintenance_SupplierPart_SparePart FOREIGN KEY (SparePartId) REFERENCES
Maintenance.SparePart(SparePartId)
);
GO

CREATE TABLE Maintenance.SparePartStock (
    SparePartStockId INT IDENTITY(1,1) NOT NULL,
    SparePartId INT NOT NULL,
    PlantId INT NOT NULL,
    LocationCode NVARCHAR(80) NOT NULL,
    QuantityOnHand DECIMAL(18,4) NOT NULL CONSTRAINT DF_Maintenance_SparePartStock_QuantityOnHand
DEFAULT 0,
    QuantityReserved DECIMAL(18,4) NOT NULL CONSTRAINT DF_Maintenance_SparePartStock_QuantityReserved
DEFAULT 0,
    ReorderPoint DECIMAL(18,4) NULL,
    LastCountedAtUtc DATETIME2(7) NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Maintenance_SparePartStock PRIMARY KEY (SparePartStockId),
    CONSTRAINT UQ_Maintenance_SparePartStock UNIQUE (SparePartId, PlantId, LocationCode),
    CONSTRAINT CK_Maintenance_SparePartStock_NonNegative CHECK (QuantityOnHand >= 0 AND QuantityReserved
>= 0),
    CONSTRAINT FK_Maintenance_SparePartStock_SparePart FOREIGN KEY (SparePartId) REFERENCES
Maintenance.SparePart(SparePartId),
    CONSTRAINT FK_Maintenance_SparePartStock_Plant FOREIGN KEY (PlantId) REFERENCES
Organization.Plant(PlantId)
);
GO

CREATE TABLE Maintenance.AssetSparePart (
    AssetSparePartId INT IDENTITY(1,1) NOT NULL,
    AssetId INT NOT NULL,
    SparePartId INT NOT NULL,
    QuantityTypical DECIMAL(18,4) NULL,
    Note NVARCHAR(500) NULL,

    CONSTRAINT PK_Maintenance_AssetSparePart PRIMARY KEY (AssetSparePartId),
    CONSTRAINT UQ_Maintenance_AssetSparePart UNIQUE (AssetId, SparePartId),
    CONSTRAINT FK_Maintenance_AssetSparePart_Asset FOREIGN KEY (AssetId) REFERENCES
Maintenance.Asset(AssetId),
    CONSTRAINT FK_Maintenance_AssetSparePart_SparePart FOREIGN KEY (SparePartId) REFERENCES
Maintenance.SparePart(SparePartId)
);
GO

CREATE TABLE Maintenance.MaintenancePlan (
    MaintenancePlanId INT IDENTITY(1,1) NOT NULL,
    UnitId INT NOT NULL,
    AssetId INT NULL,
    MaintenancePlanTypeId INT NOT NULL,
    PlanCode NVARCHAR(80) NOT NULL,
    Title NVARCHAR(250) NOT NULL,
    Description NVARCHAR(2000) NULL,
    EffectiveFromUtc DATETIME2(7) NOT NULL,
    EffectiveToUtc DATETIME2(7) NULL,
    OwnerTeamId INT NULL,
    IsActive BIT NOT NULL CONSTRAINT DF_Maintenance_MaintenancePlan_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Maintenance_MaintenancePlan_CreatedAtUtc
DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Maintenance_MaintenancePlan_ModifiedAtUtc
DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Maintenance_MaintenancePlan PRIMARY KEY (MaintenancePlanId),
    CONSTRAINT UQ_Maintenance_MaintenancePlan_PlanCode UNIQUE (PlanCode),
    CONSTRAINT FK_Maintenance_MaintenancePlan_Unit FOREIGN KEY (UnitId) REFERENCES
ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_Maintenance_MaintenancePlan_Asset FOREIGN KEY (AssetId) REFERENCES
Maintenance.Asset(AssetId),
    CONSTRAINT FK_Maintenance_MaintenancePlan_Type FOREIGN KEY (MaintenancePlanTypeId) REFERENCES
Maintenance.MaintenancePlanType(MaintenancePlanTypeId),
    CONSTRAINT FK_Maintenance_MaintenancePlan_OwnerTeam FOREIGN KEY (OwnerTeamId) REFERENCES
Organization.Team(TeamId)
);
GO

CREATE TABLE Maintenance.MaintenancePlanTask (
    MaintenancePlanTaskId INT IDENTITY(1,1) NOT NULL,
    MaintenancePlanId INT NOT NULL,
    SequenceNo INT NOT NULL,
    Title NVARCHAR(250) NOT NULL,
    InstructionText NVARCHAR(4000) NULL,
    EstimatedHours DECIMAL(10,2) NULL,
    RequiresApproval BIT NOT NULL CONSTRAINT DF_Maintenance_MaintenancePlanTask_RequiresApproval
DEFAULT 0,

    CONSTRAINT PK_Maintenance_MaintenancePlanTask PRIMARY KEY (MaintenancePlanTaskId),
    CONSTRAINT UQ_Maintenance_MaintenancePlanTask_Sequence UNIQUE (MaintenancePlanId, SequenceNo),
    CONSTRAINT FK_Maintenance_MaintenancePlanTask_Plan FOREIGN KEY (MaintenancePlanId) REFERENCES
Maintenance.MaintenancePlan(MaintenancePlanId)
);
GO

CREATE TABLE Maintenance.MaintenanceSchedule (
    MaintenanceScheduleId INT IDENTITY(1,1) NOT NULL,
    MaintenancePlanId INT NOT NULL,
    ScheduleBasisId INT NOT NULL,
    IntervalValue INT NULL,
    IntervalUnit NVARCHAR(30) NULL,
    NextDueAtUtc DATETIME2(7) NULL,
    NextDueCounterValue DECIMAL(18,4) NULL,
    IsActive BIT NOT NULL CONSTRAINT DF_Maintenance_MaintenanceSchedule_IsActive DEFAULT 1,
    RowVersion ROWVERSION NOT NULL,

```

```

CONSTRAINT PK_Maintenance_MaintenanceSchedule PRIMARY KEY (MaintenanceScheduleId),
CONSTRAINT FK_Maintenance_MaintenanceSchedule_Plan FOREIGN KEY (MaintenancePlanId) REFERENCES
Maintenance.MaintenancePlan(MaintenancePlanId),
CONSTRAINT FK_Maintenance_MaintenanceSchedule_Basis FOREIGN KEY (ScheduleBasisId) REFERENCES
Maintenance.ScheduleBasis(ScheduleBasisId)
);
GO

CREATE TABLE Maintenance.MaintenanceScheduleOccurrence (
MaintenanceScheduleOccurrenceId BIGINT IDENTITY(1,1) NOT NULL,
MaintenanceScheduleId INT NOT NULL,
PlannedStartUtc DATETIME2(7) NOT NULL,
PlannedEndUtc DATETIME2(7) NULL,
GeneratedWorkOrderId BIGINT NULL,
IsSkipped BIT NOT NULL CONSTRAINT
DF_Maintenance_MaintenanceScheduleOccurrence_IsSkipped DEFAULT 0,
SkipReason NVARCHAR(500) NULL,

CONSTRAINT PK_Maintenance_MaintenanceScheduleOccurrence PRIMARY KEY
(MaintenanceScheduleOccurrenceId),
CONSTRAINT FK_Maintenance_MaintenanceScheduleOccurrence_Schedule FOREIGN KEY (MaintenanceScheduleId)
REFERENCES Maintenance.MaintenanceSchedule(MaintenanceScheduleId)
);
GO

CREATE TABLE Maintenance.MaintenanceWindow (
MaintenanceWindowId INT IDENTITY(1,1) NOT NULL,
UnitId INT NOT NULL,
CalendarId INT NULL,
WindowCode NVARCHAR(80) NOT NULL,
Name NVARCHAR(200) NOT NULL,
StartsAtUtc DATETIME2(7) NOT NULL,
EndsAtUtc DATETIME2(7) NOT NULL,
IsOutageWindow BIT NOT NULL CONSTRAINT DF_Maintenance_MaintenanceWindow_IsOutageWindow DEFAULT
0,
Notes NVARCHAR(1000) NULL,

CONSTRAINT PK_Maintenance_MaintenanceWindow PRIMARY KEY (MaintenanceWindowId),
CONSTRAINT UQ_Maintenance_MaintenanceWindow_WindowCode UNIQUE (WindowCode),
CONSTRAINT CK_Maintenance_MaintenanceWindow_Time CHECK (EndsAtUtc > StartsAtUtc),
CONSTRAINT FK_Maintenance_MaintenanceWindow_Unit FOREIGN KEY (UnitId) REFERENCES
ReactorFleet.Unit(UnitId),
CONSTRAINT FK_Maintenance_MaintenanceWindow_Calendar FOREIGN KEY (CalendarId) REFERENCES
CorePlatform.Calendar(CalendarId)
);
GO

```

C.9.4.5 Work orders, inspections, evidence and approvals

```

CREATE TABLE Maintenance.WorkOrder (
WorkOrderId BIGINT IDENTITY(1,1) NOT NULL,
UnitId INT NOT NULL,
AssetId INT NULL,
MaintenancePlanId INT NULL,
MaintenanceWindowId INT NULL,
WorkOrderTypeId INT NOT NULL,
WorkOrderStatusId INT NOT NULL,
WorkPriorityId INT NOT NULL,
WorkOrderCode NVARCHAR(80) NOT NULL,
Title NVARCHAR(250) NOT NULL,
Description NVARCHAR(2000) NULL,
RequestedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Maintenance_WorkOrder_RequestedAtUtc
DEFAULT SYSUTCDATETIME(),
DueAtUtc DATETIME2(7) NULL,
StartedAtUtc DATETIME2(7) NULL,
CompletedAtUtc DATETIME2(7) NULL,
ClosedAtUtc DATETIME2(7) NULL,
RequestedByUserId INT NULL,
AssignedTeamId INT NULL,
AssignedPersonId INT NULL,
OriginOperationalEventId BIGINT NULL,
OriginIncidentActionId BIGINT NULL,
CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Maintenance_WorkOrder_CreatedAtUtc
DEFAULT SYSUTCDATETIME(),
CreatedBy NVARCHAR(100) NOT NULL,
ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Maintenance_WorkOrder_ModifiedAtUtc
DEFAULT SYSUTCDATETIME(),
ModifiedBy NVARCHAR(100) NOT NULL,
IsDeleted BIT NOT NULL CONSTRAINT DF_Maintenance_WorkOrder_IsDeleted DEFAULT 0,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_Maintenance_WorkOrder PRIMARY KEY (WorkOrderId),
CONSTRAINT UQ_Maintenance_WorkOrder_Code UNIQUE (WorkOrderCode),
CONSTRAINT FK_Maintenance_WorkOrder_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
CONSTRAINT FK_Maintenance_WorkOrder_Asset FOREIGN KEY (AssetId) REFERENCES
Maintenance.Asset(AssetId),
CONSTRAINT FK_Maintenance_WorkOrder_MaintenancePlan FOREIGN KEY (MaintenancePlanId) REFERENCES
Maintenance.MaintenancePlan(MaintenancePlanId),
CONSTRAINT FK_Maintenance_WorkOrder_MaintenanceWindow FOREIGN KEY (MaintenanceWindowId) REFERENCES
Maintenance.MaintenanceWindow(MaintenanceWindowId),
CONSTRAINT FK_Maintenance_WorkOrder_Type FOREIGN KEY (WorkOrderTypeId) REFERENCES
Maintenance.WorkOrderType(WorkOrderTypeId),
CONSTRAINT FK_Maintenance_WorkOrder_Status FOREIGN KEY (WorkOrderStatusId) REFERENCES
Maintenance.WorkOrderStatus(WorkOrderStatusId),
CONSTRAINT FK_Maintenance_WorkOrder_Priority FOREIGN KEY (WorkPriorityId) REFERENCES
Maintenance.WorkPriority(WorkPriorityId),
CONSTRAINT FK_Maintenance_WorkOrder_RequestedByUser FOREIGN KEY (RequestedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId),
CONSTRAINT FK_Maintenance_WorkOrder_AssignedTeam FOREIGN KEY (AssignedTeamId) REFERENCES
Organization.Team(TeamId),
CONSTRAINT FK_Maintenance_WorkOrder_AssignedPerson FOREIGN KEY (AssignedPersonId) REFERENCES
Organization.Person(PersonId),
CONSTRAINT FK_Maintenance_WorkOrder-OriginOperationalEvent FOREIGN KEY (OriginOperationalEventId)
REFERENCES EventManagement.OperationalEvent(OperationalEventId),
CONSTRAINT FK_Maintenance_WorkOrder-OriginIncidentAction FOREIGN KEY (OriginIncidentActionId)
REFERENCES EventManagement.IncidentAction(IncidentActionId)
);
GO

CREATE TABLE Maintenance.WorkOrderStatusHistory (
WorkOrderStatusHistoryId BIGINT IDENTITY(1,1) NOT NULL,
WorkOrderId BIGINT NOT NULL,
WorkOrderStatusId INT NOT NULL,
ChangedAtUtc DATETIME2(7) NOT NULL CONSTRAINT
DF_Maintenance_WorkOrderStatusHistory_ChangedAtUtc DEFAULT SYSUTCDATETIME(),
ChangedByUserId INT NULL,
Reason NVARCHAR(500) NULL,

CONSTRAINT PK_Maintenance_WorkOrderStatusHistory PRIMARY KEY (WorkOrderStatusHistoryId),
CONSTRAINT FK_Maintenance_WorkOrderStatusHistory_WorkOrder FOREIGN KEY (WorkOrderId) REFERENCES

```

```

Maintenance.WorkOrder(WorkOrderId),
CONSTRAINT FK_Maintenance_WorkOrderStatusHistory_Status FOREIGN KEY (WorkOrderStatusId) REFERENCES
Maintenance.WorkOrderStatus(WorkOrderStatusId),
CONSTRAINT FK_Maintenance_WorkOrderStatusHistory_ChangedByUser FOREIGN KEY (ChangedByUserId)
REFERENCES Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE Maintenance.WorkOrderTask (
    WorkOrderTaskId BIGINT IDENTITY(1,1) NOT NULL,
    WorkOrderId BIGINT NOT NULL,
    TaskStatusId INT NOT NULL,
    SequenceNo INT NOT NULL,
    Title NVARCHAR(250) NOT NULL,
    InstructionText NVARCHAR(4000) NULL,
    CompletedAtUtc DATETIME2(7) NULL,
    CompletedByUserId INT NULL,

    CONSTRAINT PK_Maintenance_WorkOrderTask PRIMARY KEY (WorkOrderTaskId),
    CONSTRAINT UQ_Maintenance_WorkOrderTask_Sequence UNIQUE (WorkOrderId, SequenceNo),
    CONSTRAINT FK_Maintenance_WorkOrderTask_WorkOrder FOREIGN KEY (WorkOrderId) REFERENCES
Maintenance.WorkOrder(WorkOrderId),
    CONSTRAINT FK_Maintenance_WorkOrderTask_TaskStatus FOREIGN KEY (TaskStatusId) REFERENCES
Maintenance.TaskStatus(TaskStatusId),
    CONSTRAINT FK_Maintenance_WorkOrderTask_CompletedByUser FOREIGN KEY (CompletedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE Maintenance.WorkOrderLabour (
    WorkOrderLabourId BIGINT IDENTITY(1,1) NOT NULL,
    WorkOrderId BIGINT NOT NULL,
    LabourRoleId INT NOT NULL,
    PersonId INT NULL,
    TeamId INT NULL,
    PlannedHours DECIMAL(10,2) NULL,
    ActualHours DECIMAL(10,2) NULL,
    Notes NVARCHAR(500) NULL,

    CONSTRAINT PK_Maintenance_WorkOrderLabour PRIMARY KEY (WorkOrderLabourId),
    CONSTRAINT FK_Maintenance_WorkOrderLabour_WorkOrder FOREIGN KEY (WorkOrderId) REFERENCES
Maintenance.WorkOrder(WorkOrderId),
    CONSTRAINT FK_Maintenance_WorkOrderLabour_LabourRole FOREIGN KEY (LabourRoleId) REFERENCES
Maintenance.LabourRole(LabourRoleId),
    CONSTRAINT FK_Maintenance_WorkOrderLabour_Person FOREIGN KEY (PersonId) REFERENCES
Organization.Person(PersonId),
    CONSTRAINT FK_Maintenance_WorkOrderLabour_Team FOREIGN KEY (TeamId) REFERENCES
Organization.Team(TeamId)
);
GO

CREATE TABLE Maintenance.WorkOrderMaterial (
    WorkOrderMaterialId BIGINT IDENTITY(1,1) NOT NULL,
    WorkOrderId BIGINT NOT NULL,
    SparePartId INT NOT NULL,
    MaterialUsageTypeId INT NOT NULL,
    Quantity DECIMAL(18,4) NOT NULL,
    IssuedAtUtc DATETIME2(7) NULL,
    Notes NVARCHAR(500) NULL,

    CONSTRAINT PK_Maintenance_WorkOrderMaterial PRIMARY KEY (WorkOrderMaterialId),
    CONSTRAINT CK_Maintenance_WorkOrderMaterial_Quantity CHECK (Quantity > 0),
    CONSTRAINT FK_Maintenance_WorkOrderMaterial_WorkOrder FOREIGN KEY (WorkOrderId) REFERENCES
Maintenance.WorkOrder(WorkOrderId),
    CONSTRAINT FK_Maintenance_WorkOrderMaterial_SparePart FOREIGN KEY (SparePartId) REFERENCES
Maintenance.SparePart(SparePartId),
    CONSTRAINT FK_Maintenance_WorkOrderMaterial_UsageType FOREIGN KEY (MaterialUsageTypeId) REFERENCES
Maintenance.MaterialUsageType(MaterialUsageTypeId)
);
GO

CREATE TABLE Maintenance.WorkOrderEventLink (
    WorkOrderEventLinkId BIGINT IDENTITY(1,1) NOT NULL,
    WorkOrderId BIGINT NOT NULL,
    OperationalEventId BIGINT NOT NULL,
    IncidentActionId BIGINT NULL,
    LinkReason NVARCHAR(500) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT
DF_Maintenance_WorkOrderEventLink_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,

    CONSTRAINT PK_Maintenance_WorkOrderEventLink PRIMARY KEY (WorkOrderEventLinkId),
    CONSTRAINT UQ_Maintenance_WorkOrderEventLink UNIQUE (WorkOrderId, OperationalEventId,
    IncidentActionId),
    CONSTRAINT FK_Maintenance_WorkOrderEventLink_WorkOrder FOREIGN KEY (WorkOrderId) REFERENCES
Maintenance.WorkOrder(WorkOrderId),
    CONSTRAINT FK_Maintenance_WorkOrderEventLink_OperationalEvent FOREIGN KEY (OperationalEventId)
REFERENCES EventManagement.OperationalEvent(OperationalEventId),
    CONSTRAINT FK_Maintenance_WorkOrderEventLink_IncidentAction FOREIGN KEY (IncidentActionId)
REFERENCES EventManagement.IncidentAction(IncidentActionId)
);
GO

CREATE TABLE Maintenance.Inspection (
    InspectionId BIGINT IDENTITY(1,1) NOT NULL,
    AssetId INT NOT NULL,
    WorkOrderId BIGINT NULL,
    InspectionTypeId INT NOT NULL,
    InspectionCode NVARCHAR(80) NOT NULL,
    PlannedAtUtc DATETIME2(7) NULL,
    StartedAtUtc DATETIME2(7) NULL,
    CompletedAtUtc DATETIME2(7) NULL,
    PerformedByUserId INT NULL,
    Summary NVARCHAR(2000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Maintenance_Inspection_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Maintenance_Inspection_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Maintenance_Inspection PRIMARY KEY (InspectionId),
    CONSTRAINT UQ_Maintenance_Inspection_Code UNIQUE (InspectionCode),
    CONSTRAINT FK_Maintenance_Inspection_Asset FOREIGN KEY (AssetId) REFERENCES
Maintenance.Asset(AssetId),
    CONSTRAINT FK_Maintenance_Inspection_WorkOrder FOREIGN KEY (WorkOrderId) REFERENCES
Maintenance.WorkOrder(WorkOrderId),
    CONSTRAINT FK_Maintenance_Inspection_InspectionType FOREIGN KEY (InspectionTypeId) REFERENCES
Maintenance.InspectionType(InspectionTypeId),
    CONSTRAINT FK_Maintenance_Inspection_PerformedByUser FOREIGN KEY (PerformedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE Maintenance.InspectionFinding (
    InspectionFindingId BIGINT IDENTITY(1,1) NOT NULL,
    InspectionId BIGINT NOT NULL,
    AssetComponentId INT NULL,

```

```

FindingSeverityId INT NOT NULL,
CreatedWorkOrderId BIGINT NULL,
FindingCode NVARCHAR(80) NOT NULL,
Description NVARCHAR(2000) NOT NULL,
RecommendedAction NVARCHAR(2000) NULL,
IsClosed BIT NOT NULL CONSTRAINT DF_Maintenance_InspectionFinding_IsClosed DEFAULT 0,
ClosedAtUtc DATETIME2(7) NULL,

CONSTRAINT PK_Maintenance_InspectionFinding PRIMARY KEY (InspectionFindingId),
CONSTRAINT UQ_Maintenance_InspectionFinding_Code UNIQUE (FindingCode),
CONSTRAINT FK_Maintenance_InspectionFinding_Inspection FOREIGN KEY (InspectionId) REFERENCES
Maintenance.Inspection(InspectionId),
CONSTRAINT FK_Maintenance_InspectionFinding_AssetComponent FOREIGN KEY (AssetComponentId) REFERENCES
Maintenance.AssetComponent(AssetComponentId),
CONSTRAINT FK_Maintenance_InspectionFinding_FindingSeverity FOREIGN KEY (FindingSeverityId)
REFERENCES Maintenance.FindingSeverity(FindingSeverityId),
CONSTRAINT FK_Maintenance_InspectionFinding_CreatedWorkOrder FOREIGN KEY (CreatedWorkOrderId)
REFERENCES Maintenance.WorkOrder(WorkOrderId)
);
GO

CREATE TABLE Maintenance.MaintenanceDocument (
MaintenanceDocumentId BIGINT IDENTITY(1,1) NOT NULL,
DocumentTypeId INT NOT NULL,
AssetId INT NULL,
WorkOrderId BIGINT NULL,
InspectionId BIGINT NULL,
SupplierId INT NULL,
DocumentCode NVARCHAR(80) NOT NULL,
Title NVARCHAR(250) NOT NULL,
StorageUri NVARCHAR(1000) NOT NULL,
VersionLabel NVARCHAR(50) NULL,
UploadedAtUtc DATETIME2(7) NOT NULL CONSTRAINT
DF_Maintenance_MaintenanceDocument_UploadedAtUtc DEFAULT SYSUTCDATETIME(),
UploadedByUserId INT NULL,

CONSTRAINT PK_Maintenance_MaintenanceDocument PRIMARY KEY (MaintenanceDocumentId),
CONSTRAINT UQ_Maintenance_MaintenanceDocument_Code UNIQUE (DocumentCode),
CONSTRAINT FK_Maintenance_MaintenanceDocument_DocumentType FOREIGN KEY (DocumentTypeId) REFERENCES
Maintenance.DocumentType(DocumentTypeId),
CONSTRAINT FK_Maintenance_MaintenanceDocument_Asset FOREIGN KEY (AssetId) REFERENCES
Maintenance.Asset(AssetId),
CONSTRAINT FK_Maintenance_MaintenanceDocument_WorkOrder FOREIGN KEY (WorkOrderId) REFERENCES
Maintenance.WorkOrder(WorkOrderId),
CONSTRAINT FK_Maintenance_MaintenanceDocument_Inspection FOREIGN KEY (InspectionId) REFERENCES
Maintenance.Inspection(InspectionId),
CONSTRAINT FK_Maintenance_MaintenanceDocument_Supplier FOREIGN KEY (SupplierId) REFERENCES
Maintenance.Supplier(SupplierId),
CONSTRAINT FK_Maintenance_MaintenanceDocument_UploadedByUser FOREIGN KEY (UploadedByUserId)
REFERENCES Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE Maintenance.MaintenanceApproval (
MaintenanceApprovalId BIGINT IDENTITY(1,1) NOT NULL,
ApprovalStatusId INT NOT NULL,
WorkOrderId BIGINT NULL,
MaintenancePlanId INT NULL,
MaintenanceDocumentId BIGINT NULL,
InspectionId BIGINT NULL,
ApproverUserId INT NOT NULL,
RequestedAtUtc DATETIME2(7) NOT NULL CONSTRAINT
DF_Maintenance_MaintenanceApproval_RequestedAtUtc DEFAULT SYSUTCDATETIME(),
DecidedAtUtc DATETIME2(7) NULL,
DecisionNote NVARCHAR(1000) NULL,

CONSTRAINT PK_Maintenance_MaintenanceApproval PRIMARY KEY (MaintenanceApprovalId),
CONSTRAINT FK_Maintenance_MaintenanceApproval_ApprovalStatus FOREIGN KEY (ApprovalStatusId)
REFERENCES Maintenance.ApprovalStatus(ApprovalStatusId),
CONSTRAINT FK_Maintenance_MaintenanceApproval_WorkOrder FOREIGN KEY (WorkOrderId) REFERENCES
Maintenance.WorkOrder(WorkOrderId),
CONSTRAINT FK_Maintenance_MaintenanceApproval_MaintenancePlan FOREIGN KEY (MaintenancePlanId)
REFERENCES Maintenance.MaintenancePlan(MaintenancePlanId),
CONSTRAINT FK_Maintenance_MaintenanceApproval_MaintenanceDocument FOREIGN KEY
(MaintenanceDocumentId) REFERENCES Maintenance.MaintenanceDocument(MaintenanceDocumentId),
CONSTRAINT FK_Maintenance_MaintenanceApproval_Inspection FOREIGN KEY (InspectionId) REFERENCES
Maintenance.Inspection(InspectionId),
CONSTRAINT FK_Maintenance_MaintenanceApproval_ApproverUser FOREIGN KEY (ApproverUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId),
CONSTRAINT CK_Maintenance_MaintenanceApproval_Target CHECK (
WorkOrderId IS NOT NULL OR MaintenancePlanId IS NOT NULL OR MaintenanceDocumentId IS NOT NULL OR
InspectionId IS NOT NULL
)
);
GO

```

C.9.4.6 Deferred relationship

```

ALTER TABLE Maintenance.MaintenanceScheduleOccurrence
ADD CONSTRAINT FK_Maintenance_MaintenanceScheduleOccurrence_GeneratedWorkOrder
FOREIGN KEY (GeneratedWorkOrderId) REFERENCES Maintenance.WorkOrder(WorkOrderId);
GO

```

C.9.5 Indexes and seed examples

The important access paths are asset by unit, work order by status and due date, condition history by asset, active degradation cases, material availability and event-originated work.

```

CREATE INDEX IX_Maintenance_Asset_UnitId ON Maintenance.Asset(UnitId);
CREATE INDEX IX_Maintenance_Asset_EquipmentId ON Maintenance.Asset(EquipmentId) WHERE EquipmentId IS NOT
NULL;
CREATE INDEX IX_Maintenance_Asset_Status_Criticality ON Maintenance.Asset(AssetStatusId,
AssetCriticalityId);

CREATE INDEX IX_Maintenance_AssetCondition_Asset_AssessedAtUtc
ON Maintenance.AssetCondition(AssetId, AssessedAtUtc DESC);

```



```

CREATE INDEX IX_Maintenance_DegradationRecord_Asset_IsActive
ON Maintenance.DegradationRecord(AssetId, IsActive, DetectedAtUtc DESC);

CREATE INDEX IX_Maintenance_WorkOrder_Unit_Status_Due
ON Maintenance.WorkOrder(UnitId, WorkOrderStatusId, DueAtUtc);

CREATE INDEX IX_Maintenance_WorkOrder_Asset_Status
ON Maintenance.WorkOrder(AssetId, WorkOrderStatusId) WHERE AssetId IS NOT NULL;

CREATE INDEX IX_Maintenance_WorkOrder_OriginOperationalEvent
ON Maintenance.WorkOrder(OriginOperationalEventId) WHERE OriginOperationalEventId IS NOT NULL;

CREATE INDEX IX_Maintenance_WorkOrderTask_WorkOrder_Status
ON Maintenance.WorkOrderTask(WorkOrderId, TaskStatusId, SequenceNo);

CREATE INDEX IX_Maintenance_WorkOrderMaterial_WorkOrder
ON Maintenance.WorkOrderMaterial(WorkOrderId, SparePartId);

CREATE INDEX IX_Maintenance_Inspection_Asset_CompletedAtUtc
ON Maintenance.Inspection(AssetId, CompletedAtUtc DESC);

CREATE INDEX IX_Maintenance_InspectionFinding_Inspection_Severity
ON Maintenance.InspectionFinding(InspectionId, FindingSeverityId);

CREATE INDEX IX_Maintenance_MaintenanceSchedule_NextDue
ON Maintenance.MaintenanceSchedule(NextDueAtUtc) WHERE IsActive = 1;

CREATE INDEX IX_Maintenance_SparePartStock_Plant_Reorder
ON Maintenance.SparePartStock(PlantId, SparePartId, QuantityOnHand, ReorderPoint);

CREATE INDEX IX_Maintenance_MaintenanceDocument_WorkOrder
ON Maintenance.MaintenanceDocument(WorkOrderId) WHERE WorkOrderId IS NOT NULL;
GO

```

C.9.5.1 Seed examples

```

INSERT INTO Maintenance.AssetStatus (Code, Name, DisplayOrder) VALUES
(N'SERVICE', N'In service', 10),
(N'DEGRADED', N'Degraded', 20),
(N'OUT_OF_SERVICE', N'Out of service', 30),
(N'MAINTENANCE', N'Under maintenance', 40),
(N'RETIRED', N'Retired', 90);

INSERT INTO Maintenance.WorkOrderStatus (Code, Name, DisplayOrder) VALUES
(N'DRAFT', N'Draft', 10),
(N'REQUESTED', N'Requested', 20),
(N'APPROVED', N'Approved', 30),
(N'PLANNED', N'Planned', 40),
(N'IN_PROGRESS', N'In progress', 50),
(N'COMPLETE', N'Complete', 80),
(N'CLOSED', N'Closed', 90),
(N'CANCELLED', N'Cancelled', 99);

INSERT INTO Maintenance.WorkOrderType (Code, Name, DisplayOrder) VALUES
(N'CORRECTIVE', N'Corrective maintenance', 10),
(N'PREVENTIVE', N'Preventive maintenance', 20),
(N'PREDICTIVE', N'Predictive maintenance', 30),
(N'INSPECTION', N'Inspection', 40),
(N'CALIBRATION', N'Calibration', 50);

INSERT INTO Maintenance.DegradationMechanism (Code, Name, DisplayOrder) VALUES
(N'WEAR', N'Wear', 10),
(N'FATIGUE', N'Fatigue', 20),
(N'CORROSION', N'Corrosion', 30),
(N'VIBRATION', N'Vibration', 40),
(N'THERMAL_CYCLE', N'Thermal cycling', 50),
(N'OBSOLESCENCE', N'Obsolescence', 90);
GO

```

C.9.5.2 Verification queries

```

-- 1. Assets by unit, with their physical equipment passport.
SELECT u.Code AS UnitCode,
       a.AssetCode,
       a.Name,
       ac.Code AS Category,
       ast.Code AS Status,
       e.EquipmentCode
FROM Maintenance.Asset AS a
JOIN ReactorFleet.Unit AS u ON u.UnitId = a.UnitId
LEFT JOIN ReactorFleet.Equipment AS e ON e.EquipmentId = a.EquipmentId
JOIN Maintenance.AssetCategory AS ac ON ac.AssetCategoryId = a.AssetCategoryId
JOIN Maintenance.AssetStatus AS ast ON ast.AssetStatusId = a.AssetStatusId
WHERE a.IsDeleted = 0
ORDER BY u.Code, a.AssetCode;

-- 2. Open work orders by unit and due date.
SELECT u.Code AS UnitCode,
       wo.WorkOrderCode,
       wo.Title,
       s.Code AS Status,
       p.Code AS Priority,
       wo.DueAtUtc
FROM Maintenance.WorkOrder AS wo
JOIN ReactorFleet.Unit AS u ON u.UnitId = wo.UnitId
JOIN Maintenance.WorkOrderStatus AS s ON s.WorkOrderStatusId = wo.WorkOrderStatusId
JOIN Maintenance.WorkPriority AS p ON p.WorkPriorityId = wo.WorkPriorityId
WHERE wo.IsDeleted = 0
      AND s.Code NOT IN (N'CLOSED', N'CANCELLED')
ORDER BY wo.DueAtUtc;

-- 3. Work orders opened because of operational events or incident actions.
SELECT wo.WorkOrderCode,
       ev.EventCode,
       ia.ActionCode,
       wo.Title
FROM Maintenance.WorkOrder AS wo
LEFT JOIN EventManagement.OperationalEvent AS ev ON ev.OperationalEventId = wo.OriginOperationalEventId
LEFT JOIN EventManagement.IncidentAction AS ia ON ia.IncidentActionId = wo.OriginIncidentActionId
WHERE wo.OriginOperationalEventId IS NOT NULL
      OR wo.OriginIncidentActionId IS NOT NULL;

```

```
-- 4. Latest condition assessment per asset.
SELECT a.AssetCode,
       a.Name,
       c.AssessedAtUtc,
       g.Code AS ConditionGrade,
       c.HealthScorePercent,
       c.RemainingUsefulLifeDays
FROM Maintenance.Asset AS a
OUTER APPLY (
    SELECT TOP (1) *
    FROM Maintenance.AssetCondition AS c
    WHERE c.AssetId = a.AssetId
    ORDER BY c.AssessedAtUtc DESC
) AS c
LEFT JOIN Maintenance.ConditionGrade AS g ON g.ConditionGradeId = c.ConditionGradeId
ORDER BY a.AssetCode;

-- 5. Active degradation cases with trend evidence.
SELECT a.AssetCode,
       m.Code AS Mechanism,
       sev.Code AS Severity,
       d.DetectedAtUtc,
       COUNT(tp.DegradationTrendPointId) AS TrendPoints
FROM Maintenance.DegradationRecord AS d
JOIN Maintenance.Asset AS a ON a.AssetId = d.AssetId
JOIN Maintenance.DegradationMechanism AS m ON m.DegradationMechanismId = d.DegradationMechanismId
JOIN Maintenance.FindingSeverity AS sev ON sev.FindingSeverityId = d.FindingSeverityId
LEFT JOIN Maintenance.DegradationTrendPoint AS tp ON tp.DegradationRecordId = d.DegradationRecordId
WHERE d.IsActive = 1
GROUP BY a.AssetCode, m.Code, sev.Code, d.DetectedAtUtc
ORDER BY d.DetectedAtUtc DESC;
```

C.9.6 ER diagram

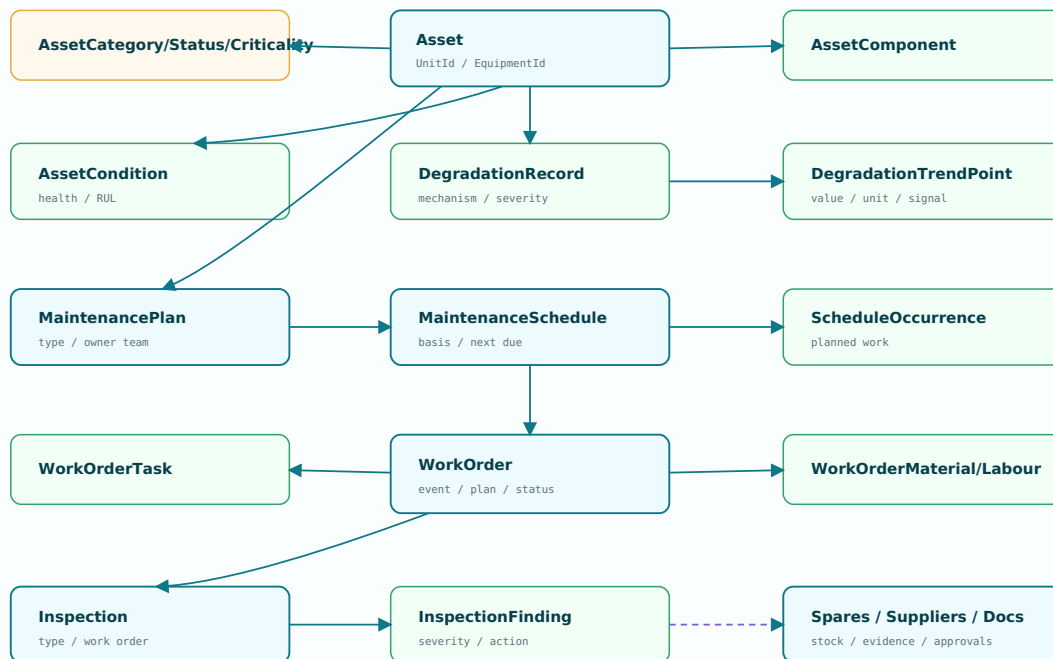
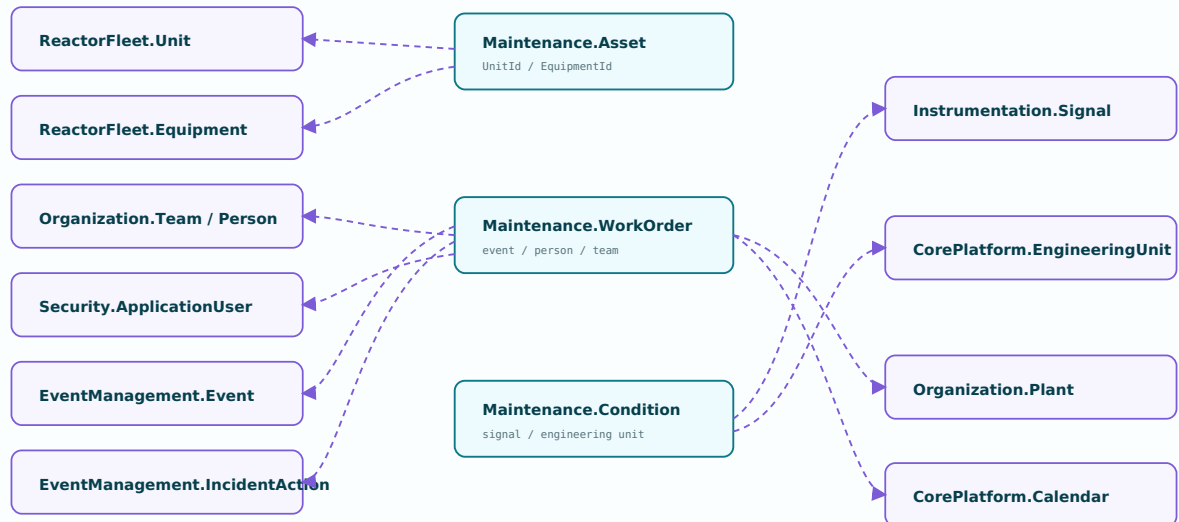


Figure C.9.1 - Maintenance sector ER diagram. Asset is the hub; condition, degradation, plans, work orders, inspections, spares, documents and approvals hang from it.



Maintenance is deliberately referential: it does not invent a plant, person, signal, unit of measure or event identity. It consumes their passports.

Figure C.9.2 - Maintenance passport map. The sector consumes plant, equipment, signal, unit-of-measure, people and event identities instead of duplicating them.

C.9.7 Foreign-key mapping

The table below lists the main passports. Lookup-table foreign keys are present in the DDL but are not repeated line by line.

Maintenance.Asset

FK UnitId
TO ReactorFleet.Unit(UnitId)
Every maintainable asset belongs to a physical unit.

Maintenance.Asset

FK EquipmentId
TO ReactorFleet.Equipment(EquipmentId)
Optional link to the physical equipment registry.

Maintenance.Asset

FK SystemId
TO ReactorFleet.System(SystemId)
Optional plant system that owns or contains the asset.

Maintenance.AssetStatusHistory

FK ChangedByUserId
TO Security.ApplicationUser(ApplicationUserId)
User who changed the asset status.

Maintenance.AssetConditionMeasurement

FK SignalId
TO Instrumentation.Signal(SignalId)
Signal used as measurement evidence.

Maintenance.AssetConditionMeasurement

FK EngineeringUnitId
TO CorePlatform.EngineeringUnit(EngineeringUnitId)
Unit of measurement.

Maintenance.DegradationTrendPoint

FK SourceSignalId
TO Instrumentation.Signal(SignalId)
Optional signal behind the trend point.

Maintenance.Supplier

FK CountryId
TO CorePlatform.Country(CountryId)
Supplier geography.

Maintenance.SparePartStock

FK PlantId
TO Organization.Plant(PlantId)
Inventory location at plant level.

Maintenance.MaintenancePlan

FK OwnerTeamId
TO Organization.Team(TeamId)
Team that owns the plan.

Maintenance.MaintenanceWindow

FK CalendarId
TO CorePlatform.Calendar(CalendarId)
Calendar used by the window.

Maintenance.WorkOrder

FK AssignedTeamId
TO Organization.Team(TeamId)
Team assigned to execute the work.

Maintenance.WorkOrder

FK AssignedPersonId
TO Organization.Person(PersonId)
Person assigned to execute or lead the work.

Maintenance.WorkOrder

FK OriginOperationalEventId
TO EventManagement.OperationalEvent(OperationalEventId)
Operational event that triggered the work order.

Maintenance.WorkOrder

FK OriginIncidentActionId
TO EventManagement.IncidentAction(IncidentActionId)
Incident action that requires maintenance work.

Maintenance.WorkOrderLabour

FK PersonId
TO Organization.Person(PersonId)
Person assigned to a labour row.

Maintenance.Inspection

FK PerformedByUserId
TO Security.ApplicationUser(ApplicationUserId)
User who performed or recorded the inspection.

Maintenance.MaintenanceApproval

FK ApproverUserId
TO Security.ApplicationUser(ApplicationUserId)
Approver for a work order, document, plan or inspection.

C.9.8 Boundary and next sector

Honest boundary. This schema records maintenance reality: condition, degradation, inspections, work orders, material and approvals. It does not infer causality, it does not generate regulatory findings, and it does not claim that a work order proves the cause of an event. Root-cause reasoning belongs to [RootCause](#); audit and regulatory accountability belong later to [Compliance](#) and [Audit](#).

The next schema is `RootCause`. It will use `EventManager.OperationalEvent`, `AlarmManagement.AlarmEvent`, `Instrumentation.Signal`, `ReactorFleet.Equipment` and the maintenance history above as grounded evidence for hypotheses, causal nodes, causal edges and rejected candidates.

Appendix C.10

RootCause Schema

The RootCause sector is the database home of the causal graph and the grounded analysis run. It receives alarm floods, operational events, incidents, telemetry windows, registry facts, maintenance history and documents, then records how a candidate cause was tested, supported, rejected or selected.

The design deliberately separates the engineered graph from the analysis result. The graph is versioned and maintained as an engineering artefact. An analysis pins one graph version, builds one sealed grounding context, tests candidate nodes, records evidence and produces either a conclusion or an explicit abstention.

The graph decides what can cause what. The grounding context decides what actually moved. The conclusion records what survived both - or why the engine refused to assert a cause.

C.10.1

Sector purpose

C.10.2

Full table list

C.10.3

Lookup tables

C.10.4

SQL DDL

C.10.5

Indexes and seed examples

C.10.6

ER diagram

C.10.7

Foreign-key mapping

C.10.8

Boundary and next sector

C.10.1 Sector purpose

RootCause sits downstream of ReactorFleet, Instrumentation, AlarmManagement, EventManagement and Maintenance. It does not own the plant, the measurements, the alarms or the work orders. It owns the causal reasoning artefacts that bind those facts into an auditable explanation.

- CausalGraph, CausalGraphVersion, CausalNode and CausalEdge define the engineered causal backbone.
- CausalEdgeDelayWindow records timing assumptions used by the node test.
- RootCauseAnalysis pins a graph version and defines the alarm/event window being analysed.

- `GroundingContext` seals the registry slice, telemetry window and retrieved documents used by every method.
- `AnalysisCandidate` and `AnalysisHypothesis` record what was tested, not just what won.
- `RejectedCandidate` preserves ruled-out candidates as first-class audit records.
- `TelemetryWitness`, `RegistryWitness` and `DocumentWitness` implement the grounding triangle.
- `AnalysisConclusion` records the selected cause or a structured abstention.

Design choice. The language-model output is not the authority. It is stored in `AnalysisModelRun` and `AnalysisModelOutput` as downstream explanation artefact. Candidate ranking, evidence, validation checks and conclusion are structured tables.

C.10.2 Full table list

The RootCause sector contains forty-four tables: fourteen typed lookup tables and thirty substantive tables. The table count is justified by auditability: a rejected candidate, a delay-window failure and a document witness are different facts with different lifecycles.

LOOKUP

RootCause.CausalNodeType

PK `CausalNodeTypeId`

FK None

Kinds of node in the causal graph: component, condition, signal, protection action, external stressor or artefact.

LOOKUP

RootCause.CausalEdgeType

PK `CausalEdgeTypeId`

FK None

Directed causal relation: physical propagation, control influence, common cause, artefact, rejected candidate.

LOOKUP

RootCause.EdgeProvenanceType

PK `EdgeProvenanceTypeId`

FK None

Why an edge exists: fault-tree backbone, topology, engineering review, learned weight, proposed, rejected.

LOOKUP

RootCause.EdgeStatus

PK `EdgeStatusId`

FK None

Lifecycle state of a causal edge: draft, active, superseded, rejected, retired.

LOOKUP

RootCause.AnalysisStatus

PK `AnalysisStatusId`

FK None

Lifecycle of a root-cause analysis: opened, context built, running, concluded, abstained, closed.

LOOKUP

RootCause.CandidateRole

PK `CandidateRoleId`

FK None

Candidate classification: origin, proximate, parallel, contributing, artefact, ruled out.

LOOKUP

RootCause.CandidateStatus

PK CandidateStatusId
FK None

Candidate state: proposed, tested, supported, weakened, rejected, selected.

LOOKUP

RootCause.HypothesisStatus

PK HypothesisStatusId
FK None

Hypothesis lifecycle: generated, grounded, challenged, accepted, rejected, abstained.

LOOKUP

RootCause.EvidenceType

PK EvidenceTypeId
FK None

Evidence kind: alarm, telemetry, registry, document, maintenance, event, model output, operator note.

LOOKUP

RootCause.WitnessType

PK WitnessTypeId
FK None

Witness class for the grounding triangle: telemetry, registry, documentation, maintenance or event.

LOOKUP

RootCause.ConfidenceBand

PK ConfidenceBandId
FK None

Human-readable confidence band: low, medium, high, grounded, insufficient evidence.

LOOKUP

RootCause.GroundingSourceType

PK GroundingSourceTypeId
FK None

Source type included in the unified context: alarm flood, telemetry window, registry slice, corpus document.

LOOKUP

RootCause.ValidationResult

PK ValidationResultId
FK None

Validation outcome: passed, failed, warning, not applicable, not enough evidence.

LOOKUP

RootCause.AbstentionReason

PK AbstentionReasonId
FK None

Reason the engine refuses to assert a cause: missing graph, weak grounding, stale context, conflicting witnesses.

GRAPH

RootCause.CausalGraph

PK CausalGraphId
FK UnitId

Named causal graph for a unit, system or reusable graph family.

GRAPH

RootCause.CausalGraphVersion

PK CausalGraphVersionId
FK CausalGraphId

Versioned immutable graph release used by analyses.

GRAPH

RootCause.CausalNode

PK CausalNodeId
FK CausalGraphVersionId, CausalNodeTypeId

A component, condition, signal or artefact node in a versioned causal graph.

GRAPH

RootCause.CausalNodeEquipmentLink

PK CausalNodeEquipmentLinkId
FK CausalNodeId, EquipmentId

Binds a causal node to physical equipment in ReactorFleet.

GRAPH

RootCause.CausalNodeSignalLink

PK CausalNodeSignalLinkId
FK CausalNodeId, SignalId

Binds a causal node to one or more instrumentation signals.

GRAPH

RootCause.CausalEdge

PK CausalEdgeId
FK FromCausalNodeId, ToCausalNodeId

Directed causal influence carrying type, provenance, status, weight and delay.

GRAPH

RootCause.CausalEdgeDelayWindow

PK CausalEdgeDelayWindowId
FK CausalEdgeId

Expected propagation window used by the node test and path fit.

GRAPH

RootCause.CausalEdgeEvidence

PK CausalEdgeEvidenceId
FK CausalEdgeId, EvidenceTypeId

Evidence or document reference that justifies a graph edge.

GRAPH

RootCause.CausalGraphChangeRequest

PK CausalGraphChangeRequestId
FK CausalGraphId, RequestedByUserId

Versioned workflow for proposing graph changes.

GRAPH

RootCause.CausalGraphChangeReview

PK CausalGraphChangeReviewId
FK CausalGraphChangeRequestId

Engineering review and approval/rejection of a graph change request.

ANALYSIS

RootCause.RootCauseAnalysis

PK RootCauseAnalysisId
FK UnitId, CausalGraphVersionId

One analysis run against an alarm flood, event, incident or manual request.

ANALYSIS

RootCause.AnalysisEventLink

PK AnalysisEventLinkId
FK RootCauseAnalysisId, OperationalEventId

Connects an analysis to operational events and incidents.

ANALYSIS

RootCause.AnalysisAlarm

PK AnalysisAlarmId
FK RootCauseAnalysisId , AlarmEventId

Alarm events included in the flood being analysed.

ANALYSIS

RootCause.GroundingContext

PK GroundingContextId
FK RootCauseAnalysisId

Sealed unified context: registry slice, telemetry window, alarm flood and retrieved documents.

ANALYSIS

RootCause.GroundingContextSignalWindow

PK GroundingContextSignalWindowId
FK GroundingContextId , SignalId

Telemetry window boundaries for each signal included in the context.

ANALYSIS

RootCause.GroundingContextDocument

PK GroundingContextDocumentId
FK GroundingContextId

Retrieved document or corpus fragment included in the context.

ANALYSIS

RootCause.AnalysisCandidate

PK AnalysisCandidateId
FK RootCauseAnalysisId , CausalNodeId

Node tested as a possible root, proximate cause, artefact or rejected candidate.

ANALYSIS

RootCause.AnalysisHypothesis

PK AnalysisHypothesisId
FK AnalysisCandidateId

Structured hypothesis derived from a candidate after graph and grounding checks.

ANALYSIS

RootCause.HypothesisEvidence

PK HypothesisEvidenceId
FK AnalysisHypothesisId , EvidenceTypeId

Typed evidence item attached to a hypothesis.

ANALYSIS

RootCause.RejectedCandidate

PK RejectedCandidateId
FK AnalysisCandidateId

Explicit rejection record so ruled-out candidates remain auditable.

ANALYSIS

RootCause.CausalPath

PK CausalPathId
FK AnalysisCandidateId

Forward path explained by a candidate.

ANALYSIS

RootCause.CausalPathStep

PK CausalPathStepId
FK CausalPathId , CausalEdgeId

One edge traversal in an explained path, including delay-fit result.

WITNESS

RootCause.TelemetryWitness

PK TelemetryWitnessId
FK AnalysisHypothesisId , SignalId

Telemetry corroboration or contradiction for a hypothesis.

WITNESS

RootCause.RegistryWitness

PK RegistryWitnessId
FK AnalysisHypothesisId , EquipmentId

Registry and maintenance facts that support or weaken a hypothesis.

WITNESS

RootCause.DocumentWitness

PK DocumentWitnessId
FK AnalysisHypothesisId

Document/corpus evidence retrieved for the explanation.

OUTPUT

RootCause.AnalysisConclusion

PK AnalysisConclusionId
FK RootCauseAnalysisId , AnalysisHypothesisId

Final conclusion, selected hypothesis or explicit abstention.

OUTPUT

RootCause.AnalysisModelRun

PK AnalysisModelRunId
FK RootCauseAnalysisId

Deterministic model or downstream LLM explanation run metadata.

OUTPUT

RootCause.AnalysisModelOutput

PK AnalysisModelOutputId
FK AnalysisModelRunId

Structured machine-readable output, prompt hash, response hash and citations.

OUTPUT

RootCause.AnalysisValidationCheck

PK AnalysisValidationCheckId
FK RootCauseAnalysisId

H1-H10 style validation checks applied to grounding, schema and evidence.

OUTPUT

RootCause.AnalysisAuditTrail

PK AnalysisAuditTrailId
FK RootCauseAnalysisId

Append-only lifecycle trace for analysis events.

OUTPUT

RootCause.AnalysisComment

PK AnalysisCommentId
FK RootCauseAnalysisId

Human review comment attached to an analysis.

OUTPUT

RootCause.AnalysisTag

PK AnalysisTagId
FK None

Reusable tag for grouping analyses: feedwater, EMI, training, regression, validation.

OUTPUT

RootCause.AnalysisTagAssignment

PK AnalysisTagAssignmentId
FK RootCauseAnalysisId , AnalysisTagId

Many-to-many assignment of tags to analyses.

C.10.3 Lookup tables

Lookup tables keep graph types, candidate states, evidence classes, witness classes, validation outcomes and abstention reasons explicit. This is especially important because root-cause analysis must be reproducible and explainable after the fact.

LOOKUP TABLE	TYPICAL SEED CODES	MAIN CONSUMER
CausalNodeType	COMPONENT , CONDITION , SIGNAL , EVENT , ARTEFACT , PROTECTION	Referenced by CausalNode .
CausalEdgeType	PHYSICAL , CONTROL , COMMON_CAUSE , ARTEFACT , REJECTED	Referenced by CausalEdge .
EdgeProvenanceType	FAULT_TREE , TOPOLOGY , ENGINEERED , LEARNED_WEIGHT , PROPOSED	Referenced by CausalEdge .
EdgeStatus	DRAFT , ACTIVE , SUPERSEDED , REJECTED , RETIRED	Referenced by CausalEdge .
AnalysisStatus	OPENED , CONTEXT_BUILT , RUNNING , CONCLUDED , ABSTAINED , CLOSED	Referenced by RootCauseAnalysis .
CandidateRole	ORIGIN , PROXIMATE , PARALLEL , CONTRIBUTING , ARTEFACT , RULED_OUT	Referenced by AnalysisCandidate .
CandidateStatus	PROPOSED , TESTED , SUPPORTED , WEAKENED , REJECTED , SELECTED	Referenced by AnalysisCandidate .
HypothesisStatus	GENERATED , GROUNDED , CHALLENGED , ACCEPTED , REJECTED , ABSTAINED	Referenced by AnalysisHypothesis .
EvidenceType	ALARM , TELEMETRY , REGISTRY , DOCUMENT , MAINTENANCE , EVENT , MODEL	Referenced by evidence tables.
WitnessType	TELEMETRY , REGISTRY , DOCUMENT , MAINTENANCE , EVENT	Referenced by witness rows.
ConfidenceBand	LOW , MEDIUM , HIGH , GROUNDED , INSUFFICIENT	Referenced by hypotheses and conclusions.
GroundingSourceType	ALARM_FLOOD , TELEMETRY_WINDOW , REGISTRY_SLICE , CORPUS_DOCUMENT	Referenced by context sources.
ValidationResult	PASS , FAIL , WARNING , NA , INSUFFICIENT	Referenced by validation checks and path steps.
AbstentionReason	MISSING_GRAPH , WEAK_GROUNDING , CONFLICTING_WITNESSES , STALE_CONTEXT	Referenced by conclusions.

C.10.4 SQL DDL

C.10.4.1 Schema

```
IF SCHEMA_ID(N'RootCause') IS NULL  
EXEC(N'CREATE SCHEMA RootCause');  
GO
```

C.10.4.2 RootCause lookup tables

```
CREATE TABLE RootCause.CausalNodeType (
  CausalNodeId INT IDENTITY(1,1) NOT NULL,
  Code NVARCHAR(50) NOT NULL,
  Name NVARCHAR(150) NOT NULL,
  Description NVARCHAR(500) NULL,
  DisplayOrder INT NOT NULL
  CONSTRAINT DF_RootCause_CausalNodeType_DisplayOrder DEFAULT 0,
  IsActive BIT NOT NULL
  CONSTRAINT DF_RootCause_CausalNodeType_IsActive DEFAULT 1,
  CreatedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_RootCause_CausalNodeType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_RootCause_CausalNodeType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  RowVersion ROWVERSION NOT NULL,

  CONSTRAINT PK_RootCause_CausalNodeType
    PRIMARY KEY (CausalNodeId),
  CONSTRAINT UQ_RootCause_CausalNodeType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE RootCause.CausalEdgeType (
  CausalEdgeTypeId INT IDENTITY(1,1) NOT NULL,
  Code NVARCHAR(50) NOT NULL,
  Name NVARCHAR(150) NOT NULL,
  Description NVARCHAR(500) NULL,
  DisplayOrder INT NOT NULL
  CONSTRAINT DF_RootCause_CausalEdgeType_DisplayOrder DEFAULT 0,
  IsActive BIT NOT NULL
  CONSTRAINT DF_RootCause_CausalEdgeType_IsActive DEFAULT 1,
  CreatedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_RootCause_CausalEdgeType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_RootCause_CausalEdgeType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  RowVersion ROWVERSION NOT NULL,

  CONSTRAINT PK_RootCause_CausalEdgeType
    PRIMARY KEY (CausalEdgeTypeId),
  CONSTRAINT UQ_RootCause_CausalEdgeType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE RootCause.EdgeProvenanceType (
  EdgeProvenanceTypeId INT IDENTITY(1,1) NOT NULL,
  Code NVARCHAR(50) NOT NULL,
  Name NVARCHAR(150) NOT NULL,
  Description NVARCHAR(500) NULL,
  DisplayOrder INT NOT NULL
  CONSTRAINT DF_RootCause_EdgeProvenanceType_DisplayOrder DEFAULT 0,
  IsActive BIT NOT NULL
  CONSTRAINT DF_RootCause_EdgeProvenanceType_IsActive DEFAULT 1,
  CreatedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_RootCause_EdgeProvenanceType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_RootCause_EdgeProvenanceType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  RowVersion ROWVERSION NOT NULL,

  CONSTRAINT PK_RootCause_EdgeProvenanceType
    PRIMARY KEY (EdgeProvenanceTypeId),
  CONSTRAINT UQ_RootCause_EdgeProvenanceType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE RootCause.EdgeStatus (
  EdgeStatusId INT IDENTITY(1,1) NOT NULL,
  Code NVARCHAR(50) NOT NULL,
  Name NVARCHAR(150) NOT NULL,
  Description NVARCHAR(500) NULL,
  DisplayOrder INT NOT NULL
  CONSTRAINT DF_RootCause_EdgeStatus_DisplayOrder DEFAULT 0,
  IsActive BIT NOT NULL
  CONSTRAINT DF_RootCause_EdgeStatus_IsActive DEFAULT 1,
  CreatedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_RootCause_EdgeStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_RootCause_EdgeStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  RowVersion ROWVERSION NOT NULL,

  CONSTRAINT PK_RootCause_EdgeStatus
    PRIMARY KEY (EdgeStatusId),
  CONSTRAINT UQ_RootCause_EdgeStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE RootCause.AnalysisStatus (
  AnalysisStatusId INT IDENTITY(1,1) NOT NULL,
  Code NVARCHAR(50) NOT NULL,
  Name NVARCHAR(150) NOT NULL,
  Description NVARCHAR(500) NULL,
  DisplayOrder INT NOT NULL
  CONSTRAINT DF_RootCause_AnalysisStatus_DisplayOrder DEFAULT 0,
  IsActive BIT NOT NULL
  CONSTRAINT DF_RootCause_AnalysisStatus_IsActive DEFAULT 1,
  CreatedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_RootCause_AnalysisStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_RootCause_AnalysisStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  RowVersion ROWVERSION NOT NULL,

  CONSTRAINT PK_RootCause_AnalysisStatus
    PRIMARY KEY (AnalysisStatusId),
  CONSTRAINT UQ_RootCause_AnalysisStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE RootCause.CandidateRole (
  CandidateRoleId INT IDENTITY(1,1) NOT NULL,
  Code NVARCHAR(50) NOT NULL,
  Name NVARCHAR(150) NOT NULL,
  Description NVARCHAR(500) NULL,
  DisplayOrder INT NOT NULL
  CONSTRAINT DF_RootCause_CandidateRole_DisplayOrder DEFAULT 0,
  IsActive BIT NOT NULL
  CONSTRAINT DF_RootCause_CandidateRole_IsActive DEFAULT 1,
  CreatedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_RootCause_CandidateRole_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_RootCause_CandidateRole_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  RowVersion ROWVERSION NOT NULL,
```

```

RowVersion    ROWVERSION NOT NULL,

CONSTRAINT PK_RootCause_CandidateRole
PRIMARY KEY (CandidateRoleId),
CONSTRAINT UQ_RootCause_CandidateRole_Code
UNIQUE (Code)
);
GO

CREATE TABLE RootCause.CandidateStatus (
CandidateStatusId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_RootCause_CandidateStatus_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_RootCause_CandidateStatus_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RootCause_CandidateStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RootCause_CandidateStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_RootCause_CandidateStatus
PRIMARY KEY (CandidateStatusId),
CONSTRAINT UQ_RootCause_CandidateStatus_Code
UNIQUE (Code)
);
GO

CREATE TABLE RootCause.HypothesisStatus (
HypothesisStatusId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_RootCause_HypothesisStatus_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_RootCause_HypothesisStatus_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RootCause_HypothesisStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RootCause_HypothesisStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_RootCause_HypothesisStatus
PRIMARY KEY (HypothesisStatusId),
CONSTRAINT UQ_RootCause_HypothesisStatus_Code
UNIQUE (Code)
);
GO

CREATE TABLE RootCause.EvidenceType (
EvidenceTypeId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_RootCause_EvidenceType_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_RootCause_EvidenceType_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RootCause_EvidenceType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RootCause_EvidenceType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_RootCause_EvidenceType
PRIMARY KEY (EvidenceTypeId),
CONSTRAINT UQ_RootCause_EvidenceType_Code
UNIQUE (Code)
);
GO

CREATE TABLE RootCause.WitnessType (
WitnessTypeId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_RootCause_WitnessType_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_RootCause_WitnessType_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RootCause_WitnessType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RootCause_WitnessType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_RootCause_WitnessType
PRIMARY KEY (WitnessTypeId),
CONSTRAINT UQ_RootCause_WitnessType_Code
UNIQUE (Code)
);
GO

CREATE TABLE RootCause.ConfidenceBand (
ConfidenceBandId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_RootCause_ConfidenceBand_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_RootCause_ConfidenceBand_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RootCause_ConfidenceBand_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RootCause_ConfidenceBand_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_RootCause_ConfidenceBand
PRIMARY KEY (ConfidenceBandId),
CONSTRAINT UQ_RootCause_ConfidenceBand_Code
UNIQUE (Code)
);
GO

CREATE TABLE RootCause.GroundingSourceType (
GroundingSourceTypeId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_RootCause_GroundingSourceType_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_RootCause_GroundingSourceType_IsActive DEFAULT 1,

```

```

    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_RootCause_GroundingSourceType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_RootCause_GroundingSourceType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_RootCause_GroundingSourceType
    PRIMARY KEY (GroundingSourceTypeId),
    CONSTRAINT UQ_RootCause_GroundingSourceType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE RootCause.ValidationResult (
    ValidationResultId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_RootCause_ValidationResult_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_RootCause_ValidationResult_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_RootCause_ValidationResult_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_RootCause_ValidationResult_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_RootCause_ValidationResult
    PRIMARY KEY (ValidationResultId),
    CONSTRAINT UQ_RootCause_ValidationResult_Code
    UNIQUE (Code)
);
GO

CREATE TABLE RootCause.AbstentionReason (
    AbstentionReasonId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_RootCause_AbstentionReason_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_RootCause_AbstentionReason_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_RootCause_AbstentionReason_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_RootCause_AbstentionReason_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_RootCause_AbstentionReason
    PRIMARY KEY (AbstentionReasonId),
    CONSTRAINT UQ_RootCause_AbstentionReason_Code
    UNIQUE (Code)
);
GO

```

C.10.4.3 Causal graph, nodes, edges and graph governance

```

CREATE TABLE RootCause.CausalGraph (
    CausalGraphId INT IDENTITY(1,1) NOT NULL,
    UnitId INT NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    Description NVARCHAR(1000) NULL,
    IsReusableTemplate BIT NOT NULL CONSTRAINT DF_RootCause_CausalGraph_IsReusableTemplate DEFAULT 0,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RootCause_CausalGraph_CreatedAtUtc DEFAULT
    SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RootCause_CausalGraph_ModifiedAtUtc DEFAULT
    SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_RootCause_CausalGraph_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_RootCause_CausalGraph PRIMARY KEY (CausalGraphId),
    CONSTRAINT UQ_RootCause_CausalGraph_Code UNIQUE (Code),
    CONSTRAINT FK_RootCause_CausalGraph_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId)
);
GO

CREATE TABLE RootCause.CausalGraphVersion (
    CausalGraphVersionId INT IDENTITY(1,1) NOT NULL,
    CausalGraphId INT NOT NULL,
    VersionNumber INT NOT NULL,
    VersionLabel NVARCHAR(50) NOT NULL,
    EffectiveFromUtc DATETIME2(7) NOT NULL,
    EffectiveToUtc DATETIME2(7) NULL,
    IsCurrent BIT NOT NULL CONSTRAINT DF_RootCause_CausalGraphVersion_IsCurrent DEFAULT 0,
    ApprovedByUserId INT NOT NULL,
    ApprovedAtUtc DATETIME2(7) NULL,
    ChangeSummary NVARCHAR(1000) NULL,
    GraphHash VARBINARY(32) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RootCause_CausalGraphVersion_CreatedAtUtc
    DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_RootCause_CausalGraphVersion PRIMARY KEY (CausalGraphVersionId),
    CONSTRAINT UQ_RootCause_CausalGraphVersion_Number UNIQUE (CausalGraphId, VersionNumber),
    CONSTRAINT FK_RootCause_CausalGraphVersion_CausalGraph FOREIGN KEY (CausalGraphId) REFERENCES
    RootCause.CausalGraph(CausalGraphId),
    CONSTRAINT FK_RootCause_CausalGraphVersion_ApprovedBy FOREIGN KEY (ApprovedByUserId) REFERENCES
    Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE RootCause.CausalNode (
    CausalNodeId INT IDENTITY(1,1) NOT NULL,
    CausalGraphVersionId INT NOT NULL,
    CausalNodeTypeId INT NOT NULL,
    NodeCode NVARCHAR(100) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    Statement NVARCHAR(500) NOT NULL,
    Description NVARCHAR(1500) NULL,
    IsAlarmable BIT NOT NULL CONSTRAINT DF_RootCause_CausalNode_IsAlarmable DEFAULT 0,
    IsCandidateEnabled BIT NOT NULL CONSTRAINT DF_RootCause_CausalNode_IsCandidateEnabled DEFAULT 1,
    DisplayOrder INT NOT NULL CONSTRAINT DF_RootCause_CausalNode_DisplayOrder DEFAULT 0,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RootCause_CausalNode_CreatedAtUtc DEFAULT
    SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RootCause_CausalNode_ModifiedAtUtc DEFAULT
    SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,

```

```

IsDeleted          BIT NOT NULL CONSTRAINT DF_RootCause_CausalNode_IsDeleted DEFAULT 0,
RowVersion         ROWVERSION NOT NULL,

CONSTRAINT PK_RootCause_CausalNode PRIMARY KEY (CausalNodeId),
CONSTRAINT UQ_RootCause_CausalNode_Code UNIQUE (CausalGraphVersionId, NodeCode),
CONSTRAINT FK_RootCause_CausalNode_GraphVersion FOREIGN KEY (CausalGraphVersionId) REFERENCES
RootCause.CausalGraphVersion(CausalGraphVersionId),
CONSTRAINT FK_RootCause_CausalNode_NodeType FOREIGN KEY (CausalNodeTypeId) REFERENCES
RootCause.CausalNodeType(CausalNodeTypeId)
);
GO

CREATE TABLE RootCause.CausalNodeEquipmentLink (
    CausalNodeEquipmentLinkId INT IDENTITY(1,1) NOT NULL,
    CausalNodeId              INT NOT NULL,
    EquipmentId               INT NOT NULL,
    LinkNote                  NVARCHAR(500) NULL,
    IsPrimary                 BIT NOT NULL CONSTRAINT DF_RootCause_CausalNodeEquipmentLink_IsPrimary
    DEFAULT 0,

    CONSTRAINT PK_RootCause_CausalNodeEquipmentLink PRIMARY KEY (CausalNodeEquipmentLinkId),
    CONSTRAINT UQ_RootCause_CausalNodeEquipmentLink_NodeEquipment UNIQUE (CausalNodeId, EquipmentId),
    CONSTRAINT FK_RootCause_CausalNodeEquipmentLink_CausalNode FOREIGN KEY (CausalNodeId) REFERENCES
    RootCause.CausalNode(CausalNodeId),
    CONSTRAINT FK_RootCause_CausalNodeEquipmentLink_Equipment FOREIGN KEY (EquipmentId) REFERENCES
    ReactorFleet.Equipment(EquipmentId)
);
GO

CREATE TABLE RootCause.CausalNodeSignalLink (
    CausalNodeSignalLinkId INT IDENTITY(1,1) NOT NULL,
    CausalNodeId            INT NOT NULL,
    SignalId                INT NOT NULL,
    LinkNote                NVARCHAR(500) NULL,
    IsPrimary               BIT NOT NULL CONSTRAINT DF_RootCause_CausalNodeSignalLink_IsPrimary DEFAULT 0,

    CONSTRAINT PK_RootCause_CausalNodeSignalLink PRIMARY KEY (CausalNodeSignalLinkId),
    CONSTRAINT UQ_RootCause_CausalNodeSignalLink_NodeSignal UNIQUE (CausalNodeId, SignalId),
    CONSTRAINT FK_RootCause_CausalNodeSignalLink_CausalNode FOREIGN KEY (CausalNodeId) REFERENCES
    RootCause.CausalNode(CausalNodeId),
    CONSTRAINT FK_RootCause_CausalNodeSignalLink_Signal FOREIGN KEY (SignalId) REFERENCES
    Instrumentation.Signal(SignalId)
);
GO

CREATE TABLE RootCause.CausalEdge (
    CausalEdgeId            INT IDENTITY(1,1) NOT NULL,
    CausalGraphVersionId    INT NOT NULL,
    FromCausalNodeId        INT NOT NULL,
    ToCausalNodeId          INT NOT NULL,
    CausalEdgeTypeId        INT NOT NULL,
    EdgeProvenanceTypeId    INT NOT NULL,
    EdgeStatusId            INT NOT NULL,
    EdgeWeight              DECIMAL(9,6) NOT NULL CONSTRAINT DF_RootCause_CausalEdge_EdgeWeight DEFAULT 1.0,
    Polarity                SMALLINT NOT NULL CONSTRAINT DF_RootCause_CausalEdge_Polarity DEFAULT 1,
    MinimumDelaySeconds     INT NULL,
    ExpectedDelaySeconds    INT NULL,
    MaximumDelaySeconds     INT NULL,
    Rationale               NVARCHAR(1500) NULL,
    CreatedAtUtc            DATETIME2(7) NOT NULL CONSTRAINT DF_RootCause_CausalEdge_CreatedAtUtc DEFAULT
    SYSUTCDATETIME(),
    CreatedBy               NVARCHAR(100) NOT NULL,
    ModifiedAtUtc           DATETIME2(7) NOT NULL CONSTRAINT DF_RootCause_CausalEdge_ModifiedAtUtc DEFAULT
    SYSUTCDATETIME(),
    ModifiedBy              NVARCHAR(100) NOT NULL,
    IsDeleted               BIT NOT NULL CONSTRAINT DF_RootCause_CausalEdge_IsDeleted DEFAULT 0,
    RowVersion              ROWVERSION NOT NULL,

    CONSTRAINT PK_RootCause_CausalEdge PRIMARY KEY (CausalEdgeId),
    CONSTRAINT UQ_RootCause_CausalEdge_FromTo UNIQUE (CausalGraphVersionId, FromCausalNodeId,
    ToCausalNodeId, CausalEdgeTypeId),
    CONSTRAINT CK_RootCause_CausalEdge_NoSelfLoop CHECK (FromCausalNodeId <> ToCausalNodeId),
    CONSTRAINT CK_RootCause_CausalEdge_Polarity CHECK (Polarity IN (-1, 1)),
    CONSTRAINT CK_RootCause_CausalEdge_DelayOrder CHECK (MinimumDelaySeconds IS NULL OR
    MaximumDelaySeconds IS NULL OR MinimumDelaySeconds <= MaximumDelaySeconds),
    CONSTRAINT FK_RootCause_CausalEdge_GraphVersion FOREIGN KEY (CausalGraphVersionId) REFERENCES
    RootCause.CausalGraphVersion(CausalGraphVersionId),
    CONSTRAINT FK_RootCause_CausalEdge_FromNode FOREIGN KEY (FromCausalNodeId) REFERENCES
    RootCause.CausalNode(CausalNodeId),
    CONSTRAINT FK_RootCause_CausalEdge_ToNode FOREIGN KEY (ToCausalNodeId) REFERENCES
    RootCause.CausalNode(CausalNodeId),
    CONSTRAINT FK_RootCause_CausalEdge_EdgeType FOREIGN KEY (CausalEdgeTypeId) REFERENCES
    RootCause.CausalEdgeType(CausalEdgeTypeId),
    CONSTRAINT FK_RootCause_CausalEdge_Provenance FOREIGN KEY (EdgeProvenanceTypeId) REFERENCES
    RootCause.EdgeProvenanceType(EdgeProvenanceTypeId),
    CONSTRAINT FK_RootCause_CausalEdge_Status FOREIGN KEY (EdgeStatusId) REFERENCES
    RootCause.EdgeStatus(EdgeStatusId)
);
GO

CREATE TABLE RootCause.CausalEdgeDelayWindow (
    CausalEdgeDelayWindowId INT IDENTITY(1,1) NOT NULL,
    CausalEdgeId            INT NOT NULL,
    WindowName              NVARCHAR(100) NOT NULL,
    MinimumDelaySeconds     INT NOT NULL,
    ExpectedDelaySeconds    INT NOT NULL,
    MaximumDelaySeconds     INT NOT NULL,
    AppliesWhen              NVARCHAR(500) NULL,
    IsDefault                BIT NOT NULL CONSTRAINT DF_RootCause_CausalEdgeDelayWindow_IsDefault DEFAULT
    0,

    CONSTRAINT PK_RootCause_CausalEdgeDelayWindow PRIMARY KEY (CausalEdgeDelayWindowId),
    CONSTRAINT CK_RootCause_CausalEdgeDelayWindow_Order CHECK (MinimumDelaySeconds <= ExpectedDelaySeconds
    AND ExpectedDelaySeconds <= MaximumDelaySeconds),
    CONSTRAINT FK_RootCause_CausalEdgeDelayWindow_CausalEdge FOREIGN KEY (CausalEdgeId) REFERENCES
    RootCause.CausalEdge(CausalEdgeId)
);
GO

CREATE TABLE RootCause.CausalEdgeEvidence (
    CausalEdgeEvidenceId INT IDENTITY(1,1) NOT NULL,
    CausalEdgeId          INT NOT NULL,
    EvidenceTypeId         INT NOT NULL,
    EvidenceReference       NVARCHAR(300) NOT NULL,
    EvidenceSummary         NVARCHAR(1200) NOT NULL,
    SourceUrl              NVARCHAR(1000) NULL,
    SourceHash              VARBINARY(32) NULL,
    AddedByUserId           INT NULL,
    AddedAtUtc             DATETIME2(7) NOT NULL CONSTRAINT DF_RootCause_CausalEdgeEvidence_AddedAtUtc
    DEFAULT SYSUTCDATETIME(),

    CONSTRAINT PK_RootCause_CausalEdgeEvidence PRIMARY KEY (CausalEdgeEvidenceId),
    CONSTRAINT FK_RootCause_CausalEdgeEvidence_CausalEdge FOREIGN KEY (CausalEdgeId) REFERENCES
    RootCause.CausalEdge(CausalEdgeId),
    CONSTRAINT FK_RootCause_CausalEdgeEvidence_EvidenceType FOREIGN KEY (EvidenceTypeId) REFERENCES
    RootCause.EvidenceType(EvidenceTypeId),
    CONSTRAINT FK_RootCause_CausalEdgeEvidence_AddedBy FOREIGN KEY (AddedByUserId) REFERENCES
    Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE RootCause.CausalGraphChangeRequest (
    CausalGraphChangeRequestId INT IDENTITY(1,1) NOT NULL,
    CausalGraphId              INT NOT NULL,

```

```

RequestedByUserId INT NOT NULL,
RequestedAtUtc DATETIME2(7) NOT NULL CONSTRAINT
DF_RootCause_CausalGraphChangeRequest_RequestedAtUtc DEFAULT SYSUTCDATETIME(),
ChangeTitle NVARCHAR(200) NOT NULL,
ChangeDescription NVARCHAR(1500) NOT NULL,
ProposedPatchJson NVARCHAR(MAX) NOT NULL,
StatusCode NVARCHAR(40) NOT NULL,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_RootCause_CausalGraphChangeRequest PRIMARY KEY (CausalGraphChangeRequestId),
CONSTRAINT FK_RootCause_CausalGraphChangeRequest_Graph FOREIGN KEY (CausalGraphId) REFERENCES
RootCause.CausalGraph(CausalGraphId),
CONSTRAINT FK_RootCause_CausalGraphChangeRequest_RequestedBy FOREIGN KEY (RequestedByUserId)
REFERENCES Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE RootCause.CausalGraphChangeReview (
CausalGraphChangeReviewId INT IDENTITY(1,1) NOT NULL,
CausalGraphChangeRequestId INT NOT NULL,
ReviewedByUserId INT NOT NULL,
ReviewedAtUtc DATETIME2(7) NOT NULL CONSTRAINT
DF_RootCause_CausalGraphChangeReview_ReviewedAtUtc DEFAULT SYSUTCDATETIME(),
IsApproved BIT NOT NULL,
ReviewNote NVARCHAR(1500) NULL,

CONSTRAINT PK_RootCause_CausalGraphChangeReview PRIMARY KEY (CausalGraphChangeReviewId),
CONSTRAINT FK_RootCause_CausalGraphChangeReview_Request FOREIGN KEY (CausalGraphChangeRequestId)
REFERENCES RootCause.CausalGraphChangeRequest(CausalGraphChangeRequestId),
CONSTRAINT FK_RootCause_CausalGraphChangeReview_ReviewedBy FOREIGN KEY (ReviewedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

```

C.10.4.4 Analysis, alarm/event links and sealed grounding context

```

CREATE TABLE RootCause.RootCauseAnalysis (
RootCauseAnalysisId BIGINT IDENTITY(1,1) NOT NULL,
UnitId INT NOT NULL,
CausalGraphVersionId INT NOT NULL,
AnalysisStatusId INT NOT NULL,
AlarmFloodId BIGINT NULL,
OperationalEventId BIGINT NULL,
IncidentId BIGINT NULL,
RequestedByUserId INT NULL,
Title NVARCHAR(250) NOT NULL,
AnalysisWindowStartUtc DATETIME2(7) NOT NULL,
AnalysisWindowEndUtc DATETIME2(7) NOT NULL,
StartedAtUtc DATETIME2(7) NULL,
CompletedAtUtc DATETIME2(7) NULL,
DeterministicSeed INT NULL,
InputHash VARBINARY(32) NULL,
CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RootCause_RootCauseAnalysis_CreatedAtUtc
DEFAULT SYSUTCDATETIME(),
CreatedBy NVARCHAR(100) NOT NULL,
ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RootCause_RootCauseAnalysis_ModifiedAtUtc
DEFAULT SYSUTCDATETIME(),
ModifiedBy NVARCHAR(100) NOT NULL,
IsDeleted BIT NOT NULL CONSTRAINT DF_RootCause_RootCauseAnalysis_IsDeleted DEFAULT 0,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_RootCause_RootCauseAnalysis PRIMARY KEY (RootCauseAnalysisId),
CONSTRAINT CK_RootCause_RootCauseAnalysis_Window CHECK (AnalysisWindowStartUtc <
AnalysisWindowEndUtc),
CONSTRAINT FK_RootCause_RootCauseAnalysis_Unit FOREIGN KEY (UnitId) REFERENCES
ReactorFleet.Unit(UnitId),
CONSTRAINT FK_RootCause_RootCauseAnalysis_GraphVersion FOREIGN KEY (CausalGraphVersionId) REFERENCES
RootCause.CausalGraphVersion(CausalGraphVersionId),
CONSTRAINT FK_RootCause_RootCauseAnalysis_Status FOREIGN KEY (AnalysisStatusId) REFERENCES
RootCause.AnalysisStatus(AnalysisStatusId),
CONSTRAINT FK_RootCause_RootCauseAnalysis_AlarmFlood FOREIGN KEY (AlarmFloodId) REFERENCES
AlarmManagement.AlarmFlood(AlarmFloodId),
CONSTRAINT FK_RootCause_RootCauseAnalysis_OperationalEvent FOREIGN KEY (OperationalEventId) REFERENCES
EventManagement.OperationalEvent(OperationalEventId),
CONSTRAINT FK_RootCause_RootCauseAnalysis_Incident FOREIGN KEY (IncidentId) REFERENCES
EventManagement.Incident(IncidentId),
CONSTRAINT FK_RootCause_RootCauseAnalysis_RequestedBy FOREIGN KEY (RequestedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE RootCause.AnalysisEventLink (
AnalysisEventLinkId BIGINT IDENTITY(1,1) NOT NULL,
RootCauseAnalysisId BIGINT NOT NULL,
OperationalEventId BIGINT NULL,
IncidentId BIGINT NULL,
LinkRole NVARCHAR(50) NOT NULL,

CONSTRAINT PK_RootCause_AnalysisEventLink PRIMARY KEY (AnalysisEventLinkId),
CONSTRAINT FK_RootCause_AnalysisEventLink_Analysis FOREIGN KEY (RootCauseAnalysisId) REFERENCES
RootCause.RootCauseAnalysis(RootCauseAnalysisId),
CONSTRAINT FK_RootCause_AnalysisEventLink_Event FOREIGN KEY (OperationalEventId) REFERENCES
EventManagement.OperationalEvent(OperationalEventId),
CONSTRAINT FK_RootCause_AnalysisEventLink_Incident FOREIGN KEY (IncidentId) REFERENCES
EventManagement.Incident(IncidentId)
);
GO

CREATE TABLE RootCause.AnalysisAlarm (
AnalysisAlarmId BIGINT IDENTITY(1,1) NOT NULL,
RootCauseAnalysisId BIGINT NOT NULL,
AlarmEventId BIGINT NOT NULL,
ArrivalOrder INT NOT NULL,
IncludedInFlood BIT NOT NULL CONSTRAINT DF_RootCause_AnalysisAlarm_IncludedInFlood DEFAULT 1,
AlarmWeight DECIMAL(9,6) NULL,

CONSTRAINT PK_RootCause_AnalysisAlarm PRIMARY KEY (AnalysisAlarmId),
CONSTRAINT UQ_RootCause_AnalysisAlarm_AnalysisAlarm UNIQUE (RootCauseAnalysisId, AlarmEventId),
CONSTRAINT FK_RootCause_AnalysisAlarm_Analysis FOREIGN KEY (RootCauseAnalysisId) REFERENCES
RootCause.RootCauseAnalysis(RootCauseAnalysisId),
CONSTRAINT FK_RootCause_AnalysisAlarm_AlarmEvent FOREIGN KEY (AlarmEventId) REFERENCES
AlarmManagement.AlarmEvent(AlarmEventId)
);
GO

CREATE TABLE RootCause.GroundingContext (
GroundingContextId BIGINT IDENTITY(1,1) NOT NULL,
RootCauseAnalysisId BIGINT NOT NULL,
ContextBuiltAtUtc DATETIME2(7) NOT NULL CONSTRAINT
DF_RootCause_GroundingContext_ContextBuiltAtUtc DEFAULT SYSUTCDATETIME(),
WindowStartUtc DATETIME2(7) NOT NULL,
WindowEndUtc DATETIME2(7) NOT NULL,
RegistrySliceHash VARBINARY(32) NULL,
TelemetrySliceHash VARBINARY(32) NULL,
DocumentSetHash VARBINARY(32) NULL,
ContextJson NVARCHAR(MAX) NOT NULL,

```



```

IsSealed          BIT NOT NULL CONSTRAINT DF_RootCause_GroundingContext_IsSealed DEFAULT 1,

CONSTRAINT PK_RootCause_GroundingContext PRIMARY KEY (GroundingContextId),
CONSTRAINT UQ_RootCause_GroundingContext_Analysis UNIQUE (RootCauseAnalysisId),
CONSTRAINT CK_RootCause_GroundingContext_Window CHECK (WindowStartUtc < WindowEndUtc),
CONSTRAINT FK_RootCause_GroundingContext_Analysis FOREIGN KEY (RootCauseAnalysisId) REFERENCES
RootCause.RootCauseAnalysis(RootCauseAnalysisId)
);
GO

CREATE TABLE RootCause.GroundingContextSignalWindow (
    GroundingContextSignalWindowId BIGINT IDENTITY(1,1) NOT NULL,
    GroundingContextId             BIGINT NOT NULL,
    SignalId                       INT NOT NULL,
    WindowStartUtc                 DATETIME2(7) NOT NULL,
    WindowEndUtc                   DATETIME2(7) NOT NULL,
    SampleCount                    INT NOT NULL CONSTRAINT
DF_RootCause_GroundingContextSignalWindow_SampleCount DEFAULT 0,
    MinimumValue                   FLOAT NULL,
    MaximumValue                   FLOAT NULL,
    MeanValue                      FLOAT NULL,
    QualitySummary                  NVARCHAR(250) NULL,

    CONSTRAINT PK_RootCause_GroundingContextSignalWindow PRIMARY KEY (GroundingContextSignalWindowId),
    CONSTRAINT FK_RootCause_GroundingContextSignalWindow_Context FOREIGN KEY (GroundingContextId)
REFERENCES RootCause.GroundingContext(GroundingContextId),
    CONSTRAINT FK_RootCause_GroundingContextSignalWindow_Signal FOREIGN KEY (SignalId) REFERENCES
Instrumentation.Signal(SignalId)
);
GO

CREATE TABLE RootCause.GroundingContextDocument (
    GroundingContextDocumentId BIGINT IDENTITY(1,1) NOT NULL,
    GroundingContextId         BIGINT NOT NULL,
    GroundingSourceTypeId      INT NOT NULL,
    DocumentKey                 NVARCHAR(200) NOT NULL,
    Title                       NVARCHAR(300) NOT NULL,
    SourceUri                   NVARCHAR(1000) NULL,
    Snippet                     NVARCHAR(2000) NULL,
    RelevanceScore              DECIMAL(9,6) NULL,
    SourceHash                   VARBINARY(32) NULL,

    CONSTRAINT PK_RootCause_GroundingContextDocument PRIMARY KEY (GroundingContextDocumentId),
    CONSTRAINT FK_RootCause_GroundingContextDocument_Context FOREIGN KEY (GroundingContextId) REFERENCES
RootCause.GroundingContext(GroundingContextId),
    CONSTRAINT FK_RootCause_GroundingContextDocument_SourceType FOREIGN KEY (GroundingSourceTypeId)
REFERENCES RootCause.GroundingSourceType(GroundingSourceTypeId)
);
GO

```

C.10.4.5 Candidates, hypotheses, evidence, rejections and causal paths

```

CREATE TABLE RootCause.AnalysisCandidate (
    AnalysisCandidateId BIGINT IDENTITY(1,1) NOT NULL,
    RootCauseAnalysisId BIGINT NOT NULL,
    CausalNodeId         INT NOT NULL,
    CandidateRoleId      INT NOT NULL,
    CandidateStatusId    INT NOT NULL,
    CoverageCount         INT NOT NULL CONSTRAINT DF_RootCause_AnalysisCandidate_CoverageCount DEFAULT 0,
    TotalAlarmCount       INT NOT NULL CONSTRAINT DF_RootCause_AnalysisCandidate_TotalAlarmCount DEFAULT 0,
    CoverageScore         DECIMAL(9,6) NOT NULL CONSTRAINT DF_RootCause_AnalysisCandidate_CoverageScore
DEFAULT 0,
    TimingFitScore        DECIMAL(9,6) NOT NULL CONSTRAINT DF_RootCause_AnalysisCandidate_TimingFitScore
DEFAULT 0,
    GroundingScore        DECIMAL(9,6) NOT NULL CONSTRAINT DF_RootCause_AnalysisCandidate_GroundingScore
DEFAULT 0,
    CompositeScore        DECIMAL(9,6) NOT NULL CONSTRAINT DF_RootCause_AnalysisCandidate_CompositeScore
DEFAULT 0,
    RankOrder             INT NULL,
    Explanation            NVARCHAR(1500) NULL,
    TestedAtUtc            DATETIME2(7) NULL,

    CONSTRAINT PK_RootCause_AnalysisCandidate PRIMARY KEY (AnalysisCandidateId),
    CONSTRAINT UQ_RootCause_AnalysisCandidate_AnalysisNode UNIQUE (RootCauseAnalysisId, CausalNodeId),
    CONSTRAINT FK_RootCause_AnalysisCandidate_Analysis FOREIGN KEY (RootCauseAnalysisId) REFERENCES
RootCause.RootCauseAnalysis(RootCauseAnalysisId),
    CONSTRAINT FK_RootCause_AnalysisCandidate_CausalNode FOREIGN KEY (CausalNodeId) REFERENCES
RootCause.CausalNode(CausalNodeId),
    CONSTRAINT FK_RootCause_AnalysisCandidate_Role FOREIGN KEY (CandidateRoleId) REFERENCES
RootCause.CandidateRole(CandidateRoleId),
    CONSTRAINT FK_RootCause_AnalysisCandidate_Status FOREIGN KEY (CandidateStatusId) REFERENCES
RootCause.CandidateStatus(CandidateStatusId)
);
GO

CREATE TABLE RootCause.AnalysisHypothesis (
    AnalysisHypothesisId BIGINT IDENTITY(1,1) NOT NULL,
    AnalysisCandidateId  BIGINT NOT NULL,
    HypothesisStatusId   INT NOT NULL,
    ConfidenceBandId     INT NOT NULL,
    HypothesisStatement  NVARCHAR(1000) NOT NULL,
    StructuredJson        NVARCHAR(MAX) NULL,
    ConfidenceScore       DECIMAL(9,6) NULL,
    CreatedAtUtc          DATETIME2(7) NOT NULL CONSTRAINT DF_RootCause_AnalysisHypothesis_CreatedAtUtc
DEFAULT SYSUTCDATETIME(),
    RowVersion            ROWVERSION NOT NULL,

    CONSTRAINT PK_RootCause_AnalysisHypothesis PRIMARY KEY (AnalysisHypothesisId),
    CONSTRAINT FK_RootCause_AnalysisHypothesis_Candidate FOREIGN KEY (AnalysisCandidateId) REFERENCES
RootCause.AnalysisCandidate(AnalysisCandidateId),
    CONSTRAINT FK_RootCause_AnalysisHypothesis_Status FOREIGN KEY (HypothesisStatusId) REFERENCES
RootCause.HypothesisStatus(HypothesisStatusId),
    CONSTRAINT FK_RootCause_AnalysisHypothesis_ConfidenceBand FOREIGN KEY (ConfidenceBandId) REFERENCES
RootCause.ConfidenceBand(ConfidenceBandId)
);
GO

CREATE TABLE RootCause.HypothesisEvidence (
    HypothesisEvidenceId BIGINT IDENTITY(1,1) NOT NULL,
    AnalysisHypothesisId  BIGINT NOT NULL,
    EvidenceTypeId        INT NOT NULL,
    WitnessTypeId         INT NULL,
    AlarmEventId          BIGINT NULL,
    SignalId              INT NULL,
    MeasurementId         BIGINT NULL,
    EquipmentId           INT NULL,
    WorkOrderId           BIGINT NULL,
    InspectionFindingId   BIGINT NULL,
    EvidenceSummary        NVARCHAR(2000) NOT NULL,
    EvidenceJson           NVARCHAR(MAX) NULL,
    SupportsHypothesis     BIT NOT NULL CONSTRAINT DF_RootCause_HypothesisEvidence_SupportsHypothesis
DEFAULT 1,
    Weight                DECIMAL(9,6) NULL,

    CONSTRAINT PK_RootCause_HypothesisEvidence PRIMARY KEY (HypothesisEvidenceId),
    CONSTRAINT FK_RootCause_HypothesisEvidence_Hypothesis FOREIGN KEY (AnalysisHypothesisId) REFERENCES

```



```

RootCause.AnalysisHypothesis(AnalysisHypothesisId),
CONSTRAINT FK_RootCause_HypothesisEvidence_EvidenceType FOREIGN KEY (EvidenceTypeId) REFERENCES
RootCause.EvidenceType(EvidenceTypeId),
CONSTRAINT FK_RootCause_HypothesisEvidence_WitnessType FOREIGN KEY (WitnessTypeId) REFERENCES
RootCause.WitnessType(WitnessTypeId),
CONSTRAINT FK_RootCause_HypothesisEvidence_AlarmEvent FOREIGN KEY (AlarmEventId) REFERENCES
AlarmManagement.AlarmEvent(AlarmEventId),
CONSTRAINT FK_RootCause_HypothesisEvidence_Signal FOREIGN KEY (SignalId) REFERENCES
Instrumentation.Signal(SignalId),
CONSTRAINT FK_RootCause_HypothesisEvidence_Measurement FOREIGN KEY (MeasurementId) REFERENCES
Instrumentation.Measurement(MeasurementId),
CONSTRAINT FK_RootCause_HypothesisEvidence_Equipment FOREIGN KEY (EquipmentId) REFERENCES
ReactorFleet.Equipment(EquipmentId),
CONSTRAINT FK_RootCause_HypothesisEvidence_WorkOrder FOREIGN KEY (WorkOrderId) REFERENCES
Maintenance.WorkOrder(WorkOrderId),
CONSTRAINT FK_RootCause_HypothesisEvidence_InspectionFinding FOREIGN KEY (InspectionFindingId)
REFERENCES Maintenance.InspectionFinding(InspectionFindingId)
);
GO

CREATE TABLE RootCause.RejectedCandidate (
    RejectedCandidateId BIGINT IDENTITY(1,1) NOT NULL,
    AnalysisCandidateId BIGINT NOT NULL,
    RejectionReason NVARCHAR(1000) NOT NULL,
    FailedValidationJson NVARCHAR(MAX) NULL,
    RejectedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RootCause_RejectedCandidate_RejectedAtUtc
    DEFAULT SYSUTCDATETIME(),

    CONSTRAINT PK_RootCause_RejectedCandidate PRIMARY KEY (RejectedCandidateId),
    CONSTRAINT UQ_RootCause_RejectedCandidate_Candidate UNIQUE (AnalysisCandidateId),
    CONSTRAINT FK_RootCause_RejectedCandidate_Candidate FOREIGN KEY (AnalysisCandidateId) REFERENCES
    RootCause.AnalysisCandidate(AnalysisCandidateId)
);
GO

CREATE TABLE RootCause.CausalPath (
    CausalPathId BIGINT IDENTITY(1,1) NOT NULL,
    AnalysisCandidateId BIGINT NOT NULL,
    PathName NVARCHAR(150) NOT NULL,
    ExplainedAlarmCount INT NOT NULL CONSTRAINT DF_RootCause_CausalPath_ExplainedAlarmCount DEFAULT 0,
    PathScore DECIMAL(9,6) NULL,
    IsPrimaryPath BIT NOT NULL CONSTRAINT DF_RootCause_CausalPath_IsPrimaryPath DEFAULT 0,

    CONSTRAINT PK_RootCause_CausalPath PRIMARY KEY (CausalPathId),
    CONSTRAINT FK_RootCause_CausalPath_Candidate FOREIGN KEY (AnalysisCandidateId) REFERENCES
    RootCause.AnalysisCandidate(AnalysisCandidateId)
);
GO

CREATE TABLE RootCause.CausalPathStep (
    CausalPathStepId BIGINT IDENTITY(1,1) NOT NULL,
    CausalPathId BIGINT NOT NULL,
    StepOrder INT NOT NULL,
    CausalEdgeId INT NOT NULL,
    FromObservedAtUtc DATETIME2(7) NULL,
    ToObservedAtUtc DATETIME2(7) NULL,
    ObservedDelaySeconds INT NULL,
    ValidationResultId INT NOT NULL,
    DelayFitNote NVARCHAR(1000) NULL,

    CONSTRAINT PK_RootCause_CausalPathStep PRIMARY KEY (CausalPathStepId),
    CONSTRAINT UQ_RootCause_CausalPathStep_PathOrder UNIQUE (CausalPathId, StepOrder),
    CONSTRAINT FK_RootCause_CausalPathStep_CausalPath FOREIGN KEY (CausalPathId) REFERENCES
    RootCause.CausalPath(CausalPathId),
    CONSTRAINT FK_RootCause_CausalPathStep_CausalEdge FOREIGN KEY (CausalEdgeId) REFERENCES
    RootCause.CausalEdge(CausalEdgeId),
    CONSTRAINT FK_RootCause_CausalPathStep_ValidationResult FOREIGN KEY (ValidationResultId) REFERENCES
    RootCause.ValidationResult(ValidationResultId)
);
GO

```

C.10.4.6 Witnesses, conclusion, model output, validation and audit trail

```

CREATE TABLE RootCause.TelemetryWitness (
    TelemetryWitnessId BIGINT IDENTITY(1,1) NOT NULL,
    AnalysisHypothesisId BIGINT NOT NULL,
    SignalId INT NOT NULL,
    WitnessTypeId INT NOT NULL,
    WindowStartUtc DATETIME2(7) NOT NULL,
    WindowEndUtc DATETIME2(7) NOT NULL,
    ObservationSummary NVARCHAR(1500) NOT NULL,
    Corroborates BIT NOT NULL,
    QualityScore DECIMAL(9,6) NULL,

    CONSTRAINT PK_RootCause_TelemetryWitness PRIMARY KEY (TelemetryWitnessId),
    CONSTRAINT FK_RootCause_TelemetryWitness_Hypothesis FOREIGN KEY (AnalysisHypothesisId) REFERENCES
    RootCause.AnalysisHypothesis(AnalysisHypothesisId),
    CONSTRAINT FK_RootCause_TelemetryWitness_Signal FOREIGN KEY (SignalId) REFERENCES
    Instrumentation.Signal(SignalId),
    CONSTRAINT FK_RootCause_TelemetryWitness_WitnessType FOREIGN KEY (WitnessTypeId) REFERENCES
    RootCause.WitnessType(WitnessTypeId)
);
GO

CREATE TABLE RootCause.RegistryWitness (
    RegistryWitnessId BIGINT IDENTITY(1,1) NOT NULL,
    AnalysisHypothesisId BIGINT NOT NULL,
    EquipmentId INT NULL,
    AssetId INT NULL,
    WitnessTypeId INT NOT NULL,
    RegistrySummary NVARCHAR(1500) NOT NULL,
    Corroborates BIT NOT NULL,

    CONSTRAINT PK_RootCause_RegistryWitness PRIMARY KEY (RegistryWitnessId),
    CONSTRAINT FK_RootCause_RegistryWitness_Hypothesis FOREIGN KEY (AnalysisHypothesisId) REFERENCES
    RootCause.AnalysisHypothesis(AnalysisHypothesisId),
    CONSTRAINT FK_RootCause_RegistryWitness_Equipment FOREIGN KEY (EquipmentId) REFERENCES
    ReactorFleet.Equipment(EquipmentId),
    CONSTRAINT FK_RootCause_RegistryWitness_Asset FOREIGN KEY (AssetId) REFERENCES
    Maintenance.Asset(AssetId),
    CONSTRAINT FK_RootCause_RegistryWitness_WitnessType FOREIGN KEY (WitnessTypeId) REFERENCES
    RootCause.WitnessType(WitnessTypeId)
);
GO

CREATE TABLE RootCause.DocumentWitness (
    DocumentWitnessId BIGINT IDENTITY(1,1) NOT NULL,
    AnalysisHypothesisId BIGINT NOT NULL,
    GroundingContextDocumentId BIGINT NULL,
    WitnessTypeId INT NOT NULL,
    CitationKey NVARCHAR(120) NOT NULL,
    QuotedText NVARCHAR(1500) NULL,
    Summary NVARCHAR(1500) NOT NULL,
    Corroborates BIT NOT NULL,

    CONSTRAINT PK_RootCause_DocumentWitness PRIMARY KEY (DocumentWitnessId),

```

```

CONSTRAINT FK_RootCause_DocumentWitness_Hypothesis FOREIGN KEY (AnalysisHypothesisId) REFERENCES
RootCause.AnalysisHypothesis(AnalysisHypothesisId),
CONSTRAINT FK_RootCause_DocumentWitness_ContextDocument FOREIGN KEY (GroundingContextDocumentId)
REFERENCES RootCause.GroundingContextDocument(GroundingContextDocumentId),
CONSTRAINT FK_RootCause_DocumentWitness_WitnessType FOREIGN KEY (WitnessTypeId) REFERENCES
RootCause.WitnessType(WitnessTypeId)
);
GO

CREATE TABLE RootCause.AnalysisConclusion (
    AnalysisConclusionId BIGINT IDENTITY(1,1) NOT NULL,
    RootCauseAnalysisId BIGINT NOT NULL,
    AnalysisHypothesisId BIGINT NOT NULL,
    ConfidenceBandId INT NOT NULL,
    AbstentionReasonId INT NOT NULL,
    ConclusionStatement NVARCHAR(2000) NOT NULL,
    OperatorSummary NVARCHAR(2000) NULL,
    IsAbstention BIT NOT NULL CONSTRAINT DF_RootCause_AnalysisConclusion_IsAbstention DEFAULT 0,
    ConcludedByUserId INT NOT NULL,
    ConcludedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RootCause_AnalysisConclusion_ConcludedAtUtc
    DEFAULT SYSUTCDATETIME(),
    CONSTRAINT PK_RootCause_AnalysisConclusion PRIMARY KEY (AnalysisConclusionId),
    CONSTRAINT UQ_RootCause_AnalysisConclusion_Analysis UNIQUE (RootCauseAnalysisId),
    CONSTRAINT FK_RootCause_AnalysisConclusion_Analysis FOREIGN KEY (RootCauseAnalysisId) REFERENCES
    RootCause.RootCauseAnalysis(RootCauseAnalysisId),
    CONSTRAINT FK_RootCause_AnalysisConclusion_Hypothesis FOREIGN KEY (AnalysisHypothesisId) REFERENCES
    RootCause.AnalysisHypothesis(AnalysisHypothesisId),
    CONSTRAINT FK_RootCause_AnalysisConclusion_ConfidenceBand FOREIGN KEY (ConfidenceBandId) REFERENCES
    RootCause.ConfidenceBand(ConfidenceBandId),
    CONSTRAINT FK_RootCause_AnalysisConclusion_AbstentionReason FOREIGN KEY (AbstentionReasonId)
    REFERENCES RootCause.AbstentionReason(AbstentionReasonId),
    CONSTRAINT FK_RootCause_AnalysisConclusion_ConcludedBy FOREIGN KEY (ConcludedByUserId) REFERENCES
    Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE RootCause.AnalysisModelRun (
    AnalysisModelRunId BIGINT IDENTITY(1,1) NOT NULL,
    RootCauseAnalysisId BIGINT NOT NULL,
    ModelName NVARCHAR(150) NOT NULL,
    ModelVersion NVARCHAR(80) NOT NULL,
    IsDeterministic BIT NOT NULL CONSTRAINT DF_RootCause_AnalysisModelRun_IsDeterministic DEFAULT 1,
    Temperature DECIMAL(6,4) NULL,
    PromptHash VARBINARY(32) NULL,
    InputHash VARBINARY(32) NULL,
    StartedAtUtc DATETIME2(7) NOT NULL,
    CompletedAtUtc DATETIME2(7) NULL,
    CONSTRAINT PK_RootCause_AnalysisModelRun PRIMARY KEY (AnalysisModelRunId),
    CONSTRAINT FK_RootCause_AnalysisModelRun_Analysis FOREIGN KEY (RootCauseAnalysisId) REFERENCES
    RootCause.RootCauseAnalysis(RootCauseAnalysisId)
);
GO

CREATE TABLE RootCause.AnalysisModelOutput (
    AnalysisModelOutputId BIGINT IDENTITY(1,1) NOT NULL,
    AnalysisModelRunId BIGINT NOT NULL,
    OutputSchemaName NVARCHAR(150) NOT NULL,
    OutputJson NVARCHAR(MAX) NOT NULL,
    OutputHash VARBINARY(32) NULL,
    CitationCount INT NOT NULL CONSTRAINT DF_RootCause_AnalysisModelOutput_CitationCount DEFAULT
    0,
    IsSchemaValid BIT NOT NULL CONSTRAINT DF_RootCause_AnalysisModelOutput_IsSchemaValid DEFAULT
    0,
    CONSTRAINT PK_RootCause_AnalysisModelOutput PRIMARY KEY (AnalysisModelOutputId),
    CONSTRAINT FK_RootCause_AnalysisModelOutput_ModelRun FOREIGN KEY (AnalysisModelRunId) REFERENCES
    RootCause.AnalysisModelRun(AnalysisModelRunId)
);
GO

CREATE TABLE RootCause.AnalysisValidationCheck (
    AnalysisValidationCheckId BIGINT IDENTITY(1,1) NOT NULL,
    RootCauseAnalysisId BIGINT NOT NULL,
    ValidationResultId INT NOT NULL,
    CheckCode NVARCHAR(50) NOT NULL,
    CheckName NVARCHAR(200) NOT NULL,
    CheckDescription NVARCHAR(1000) NULL,
    ResultDetail NVARCHAR(1500) NULL,
    CheckedAtUtc DATETIME2(7) NOT NULL CONSTRAINT
    DF_RootCause_AnalysisValidationCheck_CheckedAtUtc DEFAULT SYSUTCDATETIME(),
    CONSTRAINT PK_RootCause_AnalysisValidationCheck PRIMARY KEY (AnalysisValidationCheckId),
    CONSTRAINT FK_RootCause_AnalysisValidationCheck_Analysis FOREIGN KEY (RootCauseAnalysisId) REFERENCES
    RootCause.RootCauseAnalysis(RootCauseAnalysisId),
    CONSTRAINT FK_RootCause_AnalysisValidationCheck_Result FOREIGN KEY (ValidationResultId) REFERENCES
    RootCause.ValidationResult(ValidationResultId)
);
GO

CREATE TABLE RootCause.AnalysisAuditTrail (
    AnalysisAuditTrailId BIGINT IDENTITY(1,1) NOT NULL,
    RootCauseAnalysisId BIGINT NOT NULL,
    ActorUserId INT NOT NULL,
    EventAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RootCause_AnalysisAuditTrail_EventAtUtc
    DEFAULT SYSUTCDATETIME(),
    EventCode NVARCHAR(80) NOT NULL,
    EventSummary NVARCHAR(1000) NOT NULL,
    PayloadJson NVARCHAR(MAX) NULL,
    CONSTRAINT PK_RootCause_AnalysisAuditTrail PRIMARY KEY (AnalysisAuditTrailId),
    CONSTRAINT FK_RootCause_AnalysisAuditTrail_Analysis FOREIGN KEY (RootCauseAnalysisId) REFERENCES
    RootCause.RootCauseAnalysis(RootCauseAnalysisId),
    CONSTRAINT FK_RootCause_AnalysisAuditTrail_Actor FOREIGN KEY (ActorUserId) REFERENCES
    Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE RootCause.AnalysisComment (
    AnalysisCommentId BIGINT IDENTITY(1,1) NOT NULL,
    RootCauseAnalysisId BIGINT NOT NULL,
    AuthorUserId INT NOT NULL,
    CommentText NVARCHAR(2000) NOT NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RootCause_AnalysisComment_CreatedAtUtc DEFAULT
    SYSUTCDATETIME(),
    CONSTRAINT PK_RootCause_AnalysisComment PRIMARY KEY (AnalysisCommentId),
    CONSTRAINT FK_RootCause_AnalysisComment_Analysis FOREIGN KEY (RootCauseAnalysisId) REFERENCES
    RootCause.RootCauseAnalysis(RootCauseAnalysisId),
    CONSTRAINT FK_RootCause_AnalysisComment_Author FOREIGN KEY (AuthorUserId) REFERENCES
    Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE RootCause.AnalysisTag (
    AnalysisTagId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    IsActive BIT NOT NULL CONSTRAINT DF_RootCause_AnalysisTag_IsActive DEFAULT 1,
    CONSTRAINT PK_RootCause_AnalysisTag PRIMARY KEY (AnalysisTagId),
    CONSTRAINT UQ_RootCause_AnalysisTag_Code UNIQUE (Code)
);
GO

```

```

);
GO

CREATE TABLE RootCause.AnalysisTagAssignment (
    AnalysisTagAssignmentId BIGINT IDENTITY(1,1) NOT NULL,
    RootCauseAnalysisId BIGINT NOT NULL,
    AnalysisTagId INT NOT NULL,

    CONSTRAINT PK_RootCause_AnalysisTagAssignment PRIMARY KEY (AnalysisTagAssignmentId),
    CONSTRAINT UQ_RootCause_AnalysisTagAssignment UNIQUE (RootCauseAnalysisId, AnalysisTagId),
    CONSTRAINT FK_RootCause_AnalysisTagAssignment_Analysis FOREIGN KEY (RootCauseAnalysisId) REFERENCES
        RootCause.RootCauseAnalysis(RootCauseAnalysisId),
    CONSTRAINT FK_RootCause_AnalysisTagAssignment_Tag FOREIGN KEY (AnalysisTagId) REFERENCES
        RootCause.AnalysisTag(AnalysisTagId)
);
GO

```

C.10.5 Indexes and seed examples

The critical access paths are graph traversal, current graph version lookup, unit/time analysis lookup, candidate ranking, evidence by hypothesis and append-only analysis audit retrieval.

```

CREATE INDEX IX_RootCause_CausalGraph_UnitId
ON RootCause.CausalGraph(UnitId);
GO

CREATE INDEX IX_RootCause_CausalGraphVersion_Current
ON RootCause.CausalGraphVersion(CausalGraphId, IsCurrent, EffectiveFromUtc DESC);
GO

CREATE INDEX IX_RootCause_CausalNode_GraphVersion_Type
ON RootCause.CausalNode(CausalGraphVersionId, CausalNodeTypeId, IsDeleted);
GO

CREATE INDEX IX_RootCause_CausalEdge_FromNode
ON RootCause.CausalEdge(FromCausalNodeId, EdgeStatusId);
GO

CREATE INDEX IX_RootCause_CausalEdge_ToNode
ON RootCause.CausalEdge(ToCausalNodeId, EdgeStatusId);
GO

CREATE INDEX IX_RootCause_RootCauseAnalysis_Unit_Window
ON RootCause.RootCauseAnalysis(UnitId, AnalysisWindowStartUtc DESC, AnalysisWindowEndUtc DESC);
GO

CREATE INDEX IX_RootCause_AnalysisAlarm_AlarmEventId
ON RootCause.AnalysisAlarm(AlarmEventId);
GO

CREATE INDEX IX_RootCause_AnalysisCandidate_Analysis_Rank
ON RootCause.AnalysisCandidate(RootCauseAnalysisId, RankOrder, CompositeScore DESC);
GO

CREATE INDEX IX_RootCause_HypothesisEvidence_Hypothesis_Type
ON RootCause.HypothesisEvidence(AnalysisHypothesisId, EvidenceTypeId);
GO

CREATE INDEX IX_RootCause_GroundingContextSignalWindow_Signal_Time
ON RootCause.GroundingContextSignalWindow(SignalId, WindowStartUtc, WindowEndUtc);
GO

CREATE INDEX IX_RootCause_AnalysisAuditTrail_Analysis_Time
ON RootCause.AnalysisAuditTrail(RootCauseAnalysisId, EventAtUtc DESC);
GO

```

C.10.5.1 Seed examples

```

INSERT INTO RootCause.CausalNodeType (Code, Name, Description)
VALUES ('COMPONENT','Component','Physical component or equipment item'),
       ('CONDITION','Condition','State or degradation condition'),
       ('SIGNAL','Signal','Instrumentation signal or derived signal'),
       ('EVENT','Event','Operational event or protection action'),
       ('ARTEFACT','Artefact','Non-physical measurement artefact');

INSERT INTO RootCause.CausalEdgeType (Code, Name, Description)
VALUES ('PHYSICAL','Physical propagation','Physical cause-effect propagation'),
       ('CONTROL','Control influence','Controller or actuation influence'),
       ('COMMON_CAUSE','Common cause','Shared upstream stressor'),
       ('ARTEFACT','Artefact','Signal corruption rather than physical change'),
       ('REJECTED','Rejected','Candidate edge kept for audit but not used as backbone');

INSERT INTO RootCause.EdgeProvenanceType (Code, Name, Description)
VALUES ('FAULT_TREE','Fault-tree backbone','Engineered edge from fault/event tree'),
       ('TOPOLOGY','Topology','Edge from plant topology and system dependency'),
       ('ENGINEERED','Engineering review','Edge approved by engineering review'),
       ('LEARNED_WEIGHT','Learned weight','Data-informed weight, subordinate to engineering'),
       ('PROPOSED','Proposed','Candidate edge awaiting review');

INSERT INTO RootCause.CandidateRole (Code, Name, Description)
VALUES ('ORIGIN','Origin','Candidate root cause'),
       ('PROXIMATE','Proximate','Downstream but near the visible failure'),
       ('PARALLEL','Parallel','Independent overlapping contributor'),
       ('CONTRIBUTING','Contributing','Minor contributor'),
       ('RULED_OUT','Ruled out','Candidate explicitly rejected');

INSERT INTO RootCause.ValidationResult (Code, Name, Description)
VALUES ('PASS','Pass','Validation passed'),
       ('FAIL','Fail','Validation failed'),
       ('WARNING','Warning','Validation produced a warning'),
       ('INSUFFICIENT','Insufficient evidence','Not enough evidence to assert');
GO

```

C.10.5.2 Verification queries

```
-- 1. Show active graph versions for a unit.
SELECT g.Code, gv.VersionLabel, gv.EffectiveFromUtc, gv.IsCurrent
FROM RootCause.CausalGraph g
JOIN RootCause.CausalGraphVersion gv ON gv.CausalGraphId = g.CausalGraphId
WHERE g.UnitId = @UnitId AND gv.IsCurrent = 1;

-- 2. Traverse the outgoing causal edges of one node.
SELECT fn.NodeCode AS from_node, tn.NodeCode AS to_node, et.Code AS edge_type,
       ep.Code AS provenance, e.EdgeWeight, e.ExpectedDelaySeconds
FROM RootCause.CausalEdge e
JOIN RootCause.CausalNode fn ON fn.CausalNodeId = e.FromCausalNodeId
JOIN RootCause.CausalNode tn ON tn.CausalNodeId = e.ToCausalNodeId
JOIN RootCause.CausalEdgeType et ON et.CausalEdgeTypeId = e.CausalEdgeTypeId
JOIN RootCause.EdgeProvenanceType ep ON ep.EdgeProvenanceTypeId = e.EdgeProvenanceTypeId
WHERE e.FromCausalNodeId = @CausalNodeId;

-- 3. Rank candidates for an analysis.
SELECT c.RankOrder, n.NodeCode, n.Statement, cr.Code AS role_code,
       cs.Code AS status_code, c.CoverageScore, c.TimingFitScore,
       c.GroundingScore, c.CompositeScore
FROM RootCause.AnalysisCandidate c
JOIN RootCause.CausalNode n ON n.CausalNodeId = c.CausalNodeId
JOIN RootCause.CandidateRole cr ON cr.CandidateRoleId = c.CandidateRoleId
JOIN RootCause.CandidateStatus cs ON cs.CandidateStatusId = c.CandidateStatusId
WHERE c.RootCauseAnalysisId = @RootCauseAnalysisId
ORDER BY c.RankOrder;

-- 4. Show conclusion and explicit abstention status.
SELECT a.RootCauseAnalysisId, a.Title, cb.Code AS confidence_band,
       ar.Code AS abstention_reason, c.IsAbstention, c.ConclusionStatement
FROM RootCause.AnalysisConclusion c
JOIN RootCause.RootCauseAnalysis a ON a.RootCauseAnalysisId = c.RootCauseAnalysisId
JOIN RootCause.ConfidenceBand cb ON cb.ConfidenceBandId = c.ConfidenceBandId
LEFT JOIN RootCause.AbstentionReason ar ON ar.AbstentionReasonId = c.AbstentionReasonId
WHERE a.RootCauseAnalysisId = @RootCauseAnalysisId;
GO
```

C.10.6 ER diagram

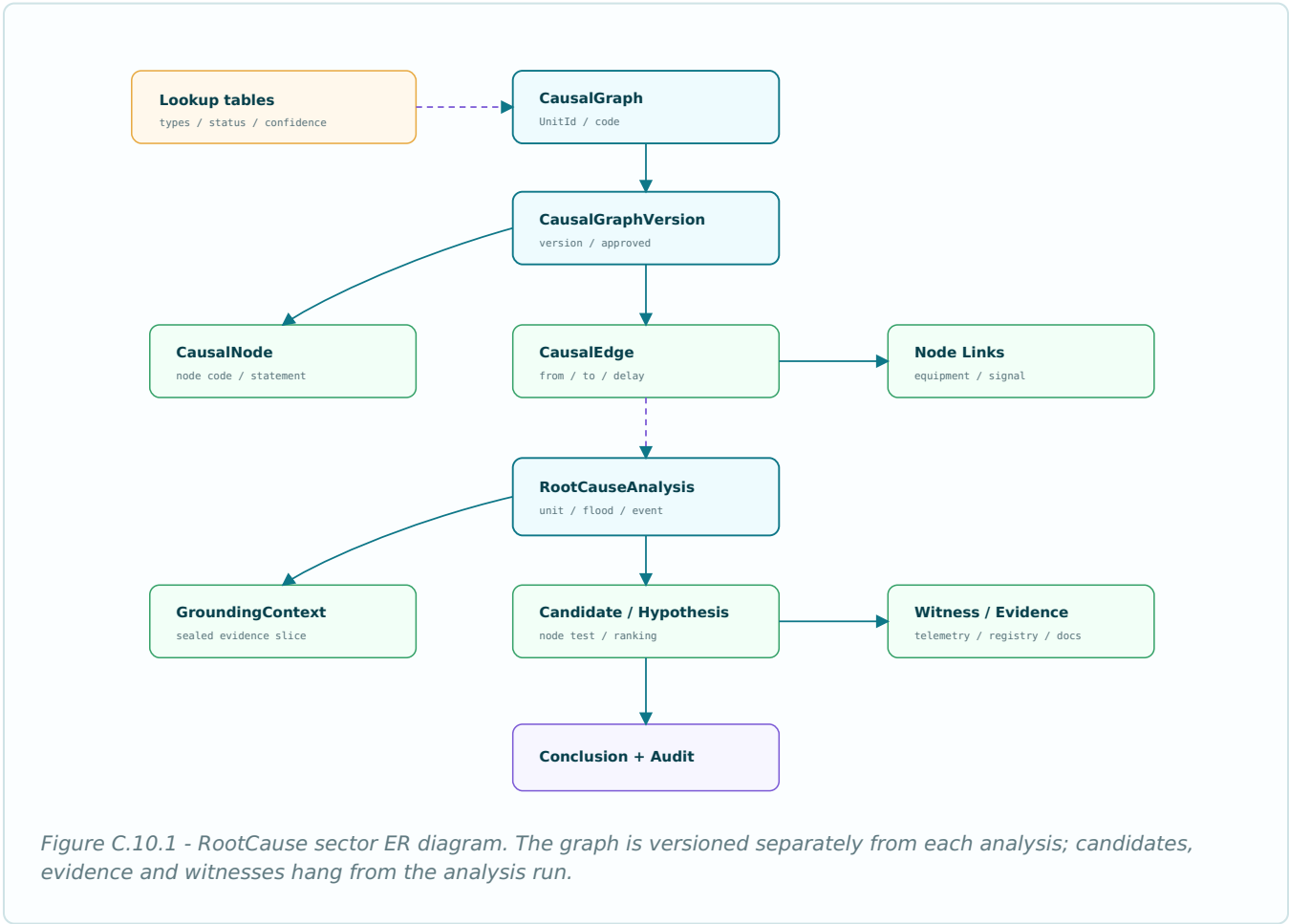
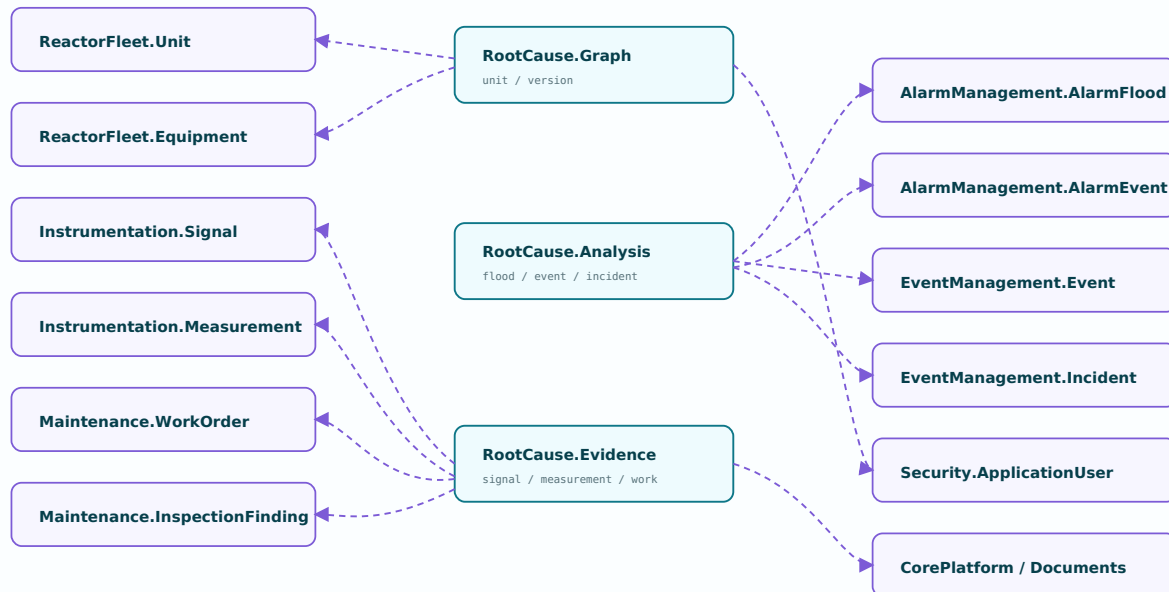


Figure C.10.1 - RootCause sector ER diagram. The graph is versioned separately from each analysis; candidates, evidence and witnesses hang from the analysis run.



RootCause is the integration point for evidence, not a duplicate store. The passports bring in unit, signal, alarm, event, maintenance and user identity.

Figure C.10.2 - RootCause passport map. RootCause consumes unit, equipment, signal, measurement, alarm, event, incident, maintenance and user identities through explicit foreign keys.

C.10.7 Foreign-key mapping

The table below lists the main passports. Lookup-table foreign keys are present in the DDL but are not repeated line by line.

RootCause.CausalGraph

FK UnitId
TO ReactorFleet.Unit
Optional unit ownership for a graph.

RootCause.CausalGraphVersion

FK CausalGraphId
TO RootCause.CausalGraph
Version belongs to a graph.

RootCause.CausalGraphVersion

FK ApprovedByUserId
TO Security.ApplicationUser
Engineering/user approval.

RootCause.CausalNode

FK CausalGraphVersionId
TO RootCause.CausalGraphVersion
Node belongs to a versioned graph.

RootCause.CausalNodeEquipmentLink

FK EquipmentId
TO ReactorFleet.Equipment
Causal node binds to physical equipment.

RootCause.CausalNodeSignalLink

FK SignalId
TO Instrumentation.Signal
Causal node binds to measured signal.

RootCause.CausalEdge

FK FromCausalNodeId
TO RootCause.CausalNode
Directed source of causal influence.

RootCause.CausalEdge

FK ToCausalNodeId
TO RootCause.CausalNode
Directed target of causal influence.

RootCause.CausalEdgeDelayWindow

FK CausalEdgeId
TO RootCause.CausalEdge
Delay windows belong to an edge.

RootCause.CausalEdgeEvidence

FK CausalEdgeId
TO RootCause.CausalEdge
Evidence justifies an edge.

RootCause.RootCauseAnalysis

FK UnitId
TO ReactorFleet.Unit
Analysis is unit-scoped.

RootCause.RootCauseAnalysis

FK CausalGraphVersionId
TO RootCause.CausalGraphVersion
Analysis pins graph version.

RootCause.RootCauseAnalysis

FK AlarmFloodId
TO AlarmManagement.AlarmFlood
Analysis may originate from an alarm flood.

RootCause.RootCauseAnalysis

FK OperationalEventId
TO EventManagement.OperationalEvent
Analysis may originate from an operational event.

RootCause.RootCauseAnalysis

FK IncidentId
TO EventManagement.Incident
Analysis may originate from an incident.

RootCause.AnalysisAlarm

FK AlarmEventId
TO AlarmManagement.AlarmEvent
Alarm included in analysed flood.

RootCause.GroundingContextSignalWindow

FK SignalId
TO Instrumentation.Signal
Telemetry included in unified context.

RootCause.AnalysisCandidate

FK CausalNodeId
TO RootCause.CausalNode
Node tested as candidate.

RootCause.HypothesisEvidence

FK MeasurementId
TO Instrumentation.Measurement
Telemetry sample evidence.

RootCause.HypothesisEvidence

FK WorkOrderId
TO Maintenance.WorkOrder
Maintenance evidence.

RootCause.HypothesisEvidence

FK InspectionFindingId
TO Maintenance.InspectionFinding
Inspection evidence.

RootCause.RegistryWitness

FK AssetId
TO Maintenance.Asset
Maintenance registry witness.

RootCause.AnalysisConclusion

FK AnalysisHypothesisId
TO RootCause.AnalysisHypothesis
Selected or abstained hypothesis.

RootCause.AnalysisAuditTrail

FK ActorUserId
TO Security.ApplicationUser
User action audit.

C.10.8 Boundary and next sector

Honest boundary. This schema records an engineered, grounded and auditable root-cause workflow. It is not a probabilistic claim about a real plant and it is not an autonomous control system. It stores why a cause was selected, why other candidates were rejected, what evidence was used, which graph version was pinned, and whether the engine abstained.

The next schema is **ReinforcementLearning**. It will store the Q-table, state/action catalogue, training runs, episodes, rewards and advisory policy output used by the Optimization screen. It depends on the plant and the digital twin, while RootCause remains focused on explanation after an event.

Appendix C.11

ReinforcementLearning Schema

The ReinforcementLearning sector is the database home of the Optimization screen. It stores the safe training environment, the discrete state and action catalogues, the reward formula, the Q-table, the episodes that filled it, the extracted policy, the evaluation metrics, and the advisory recommendation shown to a human.

The central design choice is legibility. The learned result is not a hidden neural network. It is a table: one value for every state-action pair, and one selected action for every state. The schema therefore preserves both the learning trace and the final policy.

The agent learns only in a safe model. The database records what it tried, what it received as reward, what table it produced, and what advice was finally allowed through the safety clamp.

C.11.1

Sector purpose

C.11.2

Full table list

C.11.3

Lookup tables

C.11.4

SQL DDL

C.11.5

Indexes and seed examples

C.11.6

ER diagram

C.11.7

Foreign-key mapping

C.11.8

Boundary and next sector

C.11.1 Sector purpose

ReinforcementLearning sits downstream of ReactorFleet and DigitalTwin. It does not own the physical unit or the twin model. It owns the learning apparatus: state space, action space, reward function, training run, episode log, Q-table, policy, evaluation and advisory output.

- EnvironmentModel names the safe world where learning is allowed.
- StateSpace and StateDefinition define the discrete situations the agent can recognise.
- ActionSpace and ActionDefinition define the available rod nudges or future advisory actions.

- **RewardFunction** stores the designed scoring rule, including error weight and movement penalty.
- **TrainingRun**, **Episode** and **EpisodeStep** record the experience stream.
- **QTable** and **QTableEntry** persist the learned values.
- **Policy** and **PolicyEntry** extract the readable best-action table from the Q-table.
- **PolicyDeployment**, **SafetyClamp** and **AdvisoryRecommendation** keep advice bounded and accountable.

Design choice. The schema stores learning and advice separately. Training may be experimental. A policy becomes usable only after evaluation, validation, deployment and clamp configuration.

C.11.2 Full table list

The ReinforcementLearning sector contains thirty-seven tables: twelve typed lookup tables and twenty-five substantive tables. It is large enough to make learning reproducible without hiding the result behind a black box.

LOOKUP

ReinforcementLearning.LearningAlgorithm

PK LearningAlgorithmId
FK None

Algorithm catalogue: tabular Q-learning now, future SARSA or actor-critic variants later.

LOOKUP

ReinforcementLearning.EnvironmentModelType

PK EnvironmentModelTypeId
FK None

Kind of environment model used during training: surrogate, point-kinetics twin, replay, or offline dataset.

LOOKUP

ReinforcementLearning.TrainingRunStatus

PK TrainingRunStatusId
FK None

Lifecycle of a training run: queued, running, completed, failed, cancelled, archived.

LOOKUP

ReinforcementLearning.EpisodeStatus

PK EpisodeStatusId
FK None

Lifecycle and outcome of one episode.

LOOKUP

ReinforcementLearning.StateSpaceType

PK StateSpaceTypeId
FK None

State representation: discrete bins, continuous vector, replay-only, hybrid.

LOOKUP

ReinforcementLearning.ActionSpaceType

PK ActionSpaceTypeId
FK None

Action representation: discrete rod nudges, continuous actuator values, advisory labels.

LOOKUP

ReinforcementLearning.RewardFunctionType

PK RewardFunctionTypeId

FK None

Reward family: squared error plus movement cost, custom SQL, external function, frozen formula.

LOOKUP

ReinforcementLearning.PolicyStatus

PK PolicyStatusId

FK None

Policy lifecycle: draft, training, candidate, validated, deployed, retired.

LOOKUP

ReinforcementLearning.AdvisoryMode

PK AdvisoryModeId

FK None

How advice is used: simulation, training, shadow, advisory, disabled.

LOOKUP

ReinforcementLearning.RecommendationStatus

PK RecommendationStatusId

FK None

Recommendation lifecycle: proposed, clamped, shown, accepted, rejected, expired.

LOOKUP

ReinforcementLearning.SafetyClampType

PK SafetyClampTypeId

FK None

Clamp category: action bounds, state coverage, confidence floor, rate limit, protected-mode block.

LOOKUP

ReinforcementLearning.EvaluationMetricType

PK EvaluationMetricTypeId

FK None

Evaluation metric catalogue: average reward, target error, movement count, violation rate, coverage.

MODEL

ReinforcementLearning.EnvironmentModel

PK EnvironmentModelId

FK UnitId , TwinModelId

Named training environment: the safe world in which the agent is allowed to fail.

MODEL

ReinforcementLearning.StateSpace

PK StateSpaceId

FK StateSpaceTypeId

Definition of a state space, such as power deviation x trend bins.

MODEL

ReinforcementLearning.StateDefinition

PK StateDefinitionId

FK StateSpaceId

One discrete state row, with bin boundaries and a stable state index.

MODEL

ReinforcementLearning.ActionSpace

PK ActionSpaceId

FK ActionSpaceTypeId

Definition of available actions for an agent.

MODEL

ReinforcementLearning.ActionDefinition

PK ActionDefinitionId
FK ActionSpaceId

One action, such as insert hard, insert gently, hold, withdraw gently, withdraw hard.

MODEL

ReinforcementLearning.RewardFunction

PK RewardFunctionId
FK RewardFunctionTypeId

Stored reward formula and coefficients.

MODEL

ReinforcementLearning.HyperparameterSet

PK HyperparameterSetId
FK None

Learning rate, discount factor, epsilon schedule and training horizon.

MODEL

ReinforcementLearning.Experiment

PK ExperimentId
FK UnitId

Named experiment grouping runs, policies and evaluation results.

TRAINING

ReinforcementLearning.TrainingRun

PK TrainingRunId
FK ExperimentId , EnvironmentModelId

One deterministic training run with pinned environment, reward, states, actions and hyperparameters.

TRAINING

ReinforcementLearning.Episode

PK EpisodeId
FK TrainingRunId

One rollout against the environment.

TRAINING

ReinforcementLearning.EpisodeStep

PK EpisodeStepId
FK EpisodeId , StateDefinitionId , ActionDefinitionId

One transition tuple: state, action, reward, next state and done flag.

TRAINING

ReinforcementLearning.QTable

PK QTableId
FK TrainingRunId , StateSpaceId , ActionSpaceId

A complete Q-table snapshot produced by a training run.

TRAINING

ReinforcementLearning.QTableEntry

PK QTableEntryId
FK QTableId , StateDefinitionId , ActionDefinitionId

One Q-value for one state-action pair.

TRAINING

ReinforcementLearning.ModelArtifact

PK ModelArtifactId
FK TrainingRunId

Saved artifact hashes and paths for reproducibility.

TRAINING

ReinforcementLearning.TrainingRunAuditTrail

PK TrainingRunAuditTrailId
FK TrainingRunId

Append-only lifecycle and diagnostic trace of training.

POLICY

ReinforcementLearning.Policy

PK PolicyId
FK QTableId , PolicyStatusId

Readable policy extracted from the Q-table: best action per state.

POLICY

ReinforcementLearning.PolicyEntry

PK PolicyEntryId
FK PolicyId, StateDefinitionId,
BestActionDefinitionId

One policy decision per state with Q-value and margin.

POLICY

ReinforcementLearning.PolicyEvaluation

PK PolicyEvaluationId
FK PolicyId, EnvironmentModelId

Evaluation run for a frozen policy.

POLICY

ReinforcementLearning.PolicyEvaluationMetric

PK PolicyEvaluationMetricId
FK PolicyEvaluationId, EvaluationMetricTypeId

One metric result for a policy evaluation.

POLICY

ReinforcementLearning.PolicyDeployment

PK PolicyDeploymentId
FK PolicyId, AdvisoryModeId

Controlled deployment of a policy into a simulation, shadow or advisory context.

ADVISORY

ReinforcementLearning.SafetyClamp

PK SafetyClampId
FK UnitId

Named safety envelope that can block or alter an advisory recommendation.

ADVISORY

ReinforcementLearning.SafetyClampRule

PK SafetyClampRuleId
FK SafetyClampId, SafetyClampTypeId

One rule in a clamp: max nudge, confidence floor, protected mode block, state coverage check.

ADVISORY

ReinforcementLearning.AdvisorySession

PK AdvisorySessionId
FK PolicyDeploymentId, UnitId

A session in which a deployed policy is asked for advisory recommendations.

ADVISORY

ReinforcementLearning.AdvisoryRecommendation

PK AdvisoryRecommendationId
FK AdvisorySessionId, StateDefinitionId

One recommendation returned by a policy, including original and clamped action.

ADVISORY

ReinforcementLearning.AdvisoryRecommendationReason

PK AdvisoryRecommendationReasonId
FK AdvisoryRecommendationId

Readable explanation bars: Q-values and why-this-action reasons.

C.11.3 Lookup tables

Lookup tables make algorithm, model type, run status, policy status, advisory mode, recommendation status and safety-clamp type explicit and enforceable.

LOOKUP TABLE	TYPICAL SEED CODES	MAIN CONSUMER
LearningAlgorithm	Q_LEARNING, SARSA, MONTE_CARLO, MANUAL_POLICY	Referenced by TrainingRun.
EnvironmentModelType	SURROGATE, POINT_KINETICS, REPLAY, OFFLINE_DATASET	Referenced by EnvironmentModel.
TrainingRunStatus	QUEUED, RUNNING, COMPLETED, FAILED, CANCELLED	Referenced by TrainingRun.
EpisodeStatus	OPEN, COMPLETED, TERMINATED, FAILED	Referenced by Episode.
StateSpaceType	DISCRETE_BINS, CONTINUOUS_VECTOR, HYBRID	Referenced by StateSpace.
ActionSpaceType	DISCRETE_PCM, CONTINUOUS, ADVISORY_LABELS	Referenced by ActionSpace.
RewardFunctionType	SQUARED_ERROR_MOVE_COST, CUSTOM_SQL, EXTERNAL	Referenced by RewardFunction.
PolicyStatus	DRAFT, CANDIDATE, VALIDATED, DEPLOYED, RETIRED	Referenced by Policy.
AdvisoryMode	SIMULATION, TRAINING, SHADOW, ADVISORY, DISABLED	Referenced by PolicyDeployment.
RecommendationStatus	PROPOSED, CLAMPED, SHOWN, ACCEPTED, REJECTED, EXPIRED	Referenced by AdvisoryRecommendation.
SafetyClampType	ACTION_BOUND, STATE_COVERAGE, CONFIDENCE_FLOOR, RATE_LIMIT, PROTECTION_BLOCK	Referenced by SafetyClampRule.
EvaluationMetricType	AVERAGE_REWARD, MEAN_ABS_ERROR, MOVEMENT_COUNT, CLAMP_RATE, COVERAGE	Referenced by PolicyEvaluationMetric.

C.11.4 SQL DDL

C.11.4.1 Schema

```
IF SCHEMA_ID(N'ReinforcementLearning') IS NULL
EXEC(N'CREATE SCHEMA ReinforcementLearning');
GO
```

C.11.4.2 ReinforcementLearning lookup tables

```
CREATE TABLE ReinforcementLearning.LearningAlgorithm (
    LearningAlgorithmId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_ReinforcementLearning_LearningAlgorithm_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_ReinforcementLearning_LearningAlgorithm_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReinforcementLearning_LearningAlgorithm_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReinforcementLearning_LearningAlgorithm_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,
    CONSTRAINT PK_ReinforcementLearning_LearningAlgorithm
        PRIMARY KEY (LearningAlgorithmId),
    CONSTRAINT UQ_ReinforcementLearning_LearningAlgorithm_Code
        UNIQUE (Code)
);
```

GO

```
CREATE TABLE ReinforcementLearning.EnvironmentModelType (
    EnvironmentModelTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_ReinforcementLearning_EnvironmentModelType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_ReinforcementLearning_EnvironmentModelType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReinforcementLearning_EnvironmentModelType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReinforcementLearning_EnvironmentModelType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReinforcementLearning_EnvironmentModelType
    PRIMARY KEY (EnvironmentModelTypeId),
    CONSTRAINT UQ_ReinforcementLearning_EnvironmentModelType_Code
    UNIQUE (Code)
);
GO
```

```
CREATE TABLE ReinforcementLearning.TrainingRunStatus (
    TrainingRunStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_ReinforcementLearning_TrainingRunStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_ReinforcementLearning_TrainingRunStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReinforcementLearning_TrainingRunStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReinforcementLearning_TrainingRunStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReinforcementLearning_TrainingRunStatus
    PRIMARY KEY (TrainingRunStatusId),
    CONSTRAINT UQ_ReinforcementLearning_TrainingRunStatus_Code
    UNIQUE (Code)
);
GO
```

```
CREATE TABLE ReinforcementLearning.EpisodeStatus (
    EpisodeStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_ReinforcementLearning_EpisodeStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_ReinforcementLearning_EpisodeStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReinforcementLearning_EpisodeStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReinforcementLearning_EpisodeStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReinforcementLearning_EpisodeStatus
    PRIMARY KEY (EpisodeStatusId),
    CONSTRAINT UQ_ReinforcementLearning_EpisodeStatus_Code
    UNIQUE (Code)
);
GO
```

```
CREATE TABLE ReinforcementLearning.StateSpaceType (
    StateSpaceTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_ReinforcementLearning_StateSpaceType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_ReinforcementLearning_StateSpaceType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReinforcementLearning_StateSpaceType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReinforcementLearning_StateSpaceType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReinforcementLearning_StateSpaceType
    PRIMARY KEY (StateSpaceTypeId),
    CONSTRAINT UQ_ReinforcementLearning_StateSpaceType_Code
    UNIQUE (Code)
);
GO
```

```
CREATE TABLE ReinforcementLearning.ActionSpaceType (
    ActionSpaceTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_ReinforcementLearning_ActionSpaceType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_ReinforcementLearning_ActionSpaceType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReinforcementLearning_ActionSpaceType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReinforcementLearning_ActionSpaceType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReinforcementLearning_ActionSpaceType
    PRIMARY KEY (ActionSpaceTypeId),
    CONSTRAINT UQ_ReinforcementLearning_ActionSpaceType_Code
    UNIQUE (Code)
);
GO
```

```
CREATE TABLE ReinforcementLearning.RewardFunctionType (
    RewardFunctionTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_ReinforcementLearning_RewardFunctionType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_ReinforcementLearning_RewardFunctionType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReinforcementLearning_RewardFunctionType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_ReinforcementLearning_RewardFunctionType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReinforcementLearning_RewardFunctionType

```

```

        PRIMARY KEY (RewardFunctionTypeId),
        CONSTRAINT UQ_ReinforcementLearning_RewardFunctionType_Code
            UNIQUE (Code)
    );
GO

CREATE TABLE ReinforcementLearning.PolicyStatus (
    PolicyStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_ReinforcementLearning_PolicyStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_ReinforcementLearning_PolicyStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_ReinforcementLearning_PolicyStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_ReinforcementLearning_PolicyStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReinforcementLearning_PolicyStatus
        PRIMARY KEY (PolicyStatusId),
    CONSTRAINT UQ_ReinforcementLearning_PolicyStatus_Code
        UNIQUE (Code)
);
GO

CREATE TABLE ReinforcementLearning.AdvisoryMode (
    AdvisoryModeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_ReinforcementLearning_AdvisoryMode_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_ReinforcementLearning_AdvisoryMode_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_ReinforcementLearning_AdvisoryMode_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_ReinforcementLearning_AdvisoryMode_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReinforcementLearning_AdvisoryMode
        PRIMARY KEY (AdvisoryModeId),
    CONSTRAINT UQ_ReinforcementLearning_AdvisoryMode_Code
        UNIQUE (Code)
);
GO

CREATE TABLE ReinforcementLearning.RecommendationStatus (
    RecommendationStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_ReinforcementLearning_RecommendationStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_ReinforcementLearning_RecommendationStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_ReinforcementLearning_RecommendationStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_ReinforcementLearning_RecommendationStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReinforcementLearning_RecommendationStatus
        PRIMARY KEY (RecommendationStatusId),
    CONSTRAINT UQ_ReinforcementLearning_RecommendationStatus_Code
        UNIQUE (Code)
);
GO

CREATE TABLE ReinforcementLearning.SafetyClampType (
    SafetyClampTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_ReinforcementLearning_SafetyClampType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_ReinforcementLearning_SafetyClampType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_ReinforcementLearning_SafetyClampType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_ReinforcementLearning_SafetyClampType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReinforcementLearning_SafetyClampType
        PRIMARY KEY (SafetyClampTypeId),
    CONSTRAINT UQ_ReinforcementLearning_SafetyClampType_Code
        UNIQUE (Code)
);
GO

CREATE TABLE ReinforcementLearning.EvaluationMetricType (
    EvaluationMetricTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_ReinforcementLearning_EvaluationMetricType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_ReinforcementLearning_EvaluationMetricType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_ReinforcementLearning_EvaluationMetricType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_ReinforcementLearning_EvaluationMetricType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReinforcementLearning_EvaluationMetricType
        PRIMARY KEY (EvaluationMetricTypeId),
    CONSTRAINT UQ_ReinforcementLearning_EvaluationMetricType_Code
        UNIQUE (Code)
);
GO

```

C.11.4.3 Environment, state space, action space, reward and experiment

```

CREATE TABLE ReinforcementLearning.EnvironmentModel (
    EnvironmentModelId INT IDENTITY(1,1) NOT NULL,
    EnvironmentModelTypeId INT NOT NULL,

```

```

UnitId INT NOT NULL,
TwinModelId INT NOT NULL,
Code NVARCHAR(80) NOT NULL,
Name NVARCHAR(200) NOT NULL,
Description NVARCHAR(1000) NULL,
VersionLabel NVARCHAR(50) NOT NULL,
TimeStepSeconds DECIMAL(10,4) NOT NULL,
IsDeterministic BIT NOT NULL CONSTRAINT DF_RL_EnvironmentModel_IsDeterministic DEFAULT 1,
RandomSeed INT NOT NULL,
ModelHash VARBINARY(32) NULL,
CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RL_EnvironmentModel_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
CreatedBy NVARCHAR(100) NOT NULL,
ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RL_EnvironmentModel_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedBy NVARCHAR(100) NOT NULL,
IsDeleted BIT NOT NULL CONSTRAINT DF_RL_EnvironmentModel_IsDeleted DEFAULT 0,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_ReinforcementLearning_EnvironmentModel PRIMARY KEY (EnvironmentModelId),
CONSTRAINT UQ_ReinforcementLearning_EnvironmentModel_Code UNIQUE (Code),
CONSTRAINT FK_RL_EnvironmentModel_Type FOREIGN KEY (EnvironmentModelTypeId) REFERENCES
ReinforcementLearning.EnvironmentModelType(EnvironmentModelTypeId),
CONSTRAINT FK_RL_EnvironmentModel_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
CONSTRAINT FK_RL_EnvironmentModel_TwinModel FOREIGN KEY (TwinModelId) REFERENCES
DigitalTwin.TwinModel(TwinModelId),
CONSTRAINT CK_RL_EnvironmentModel_TimeStep CHECK (TimeStepSeconds > 0)
);
GO

CREATE TABLE ReinforcementLearning.StateSpace (
    StateSpaceId INT IDENTITY(1,1) NOT NULL,
    StateSpaceTypeId INT NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    Description NVARCHAR(1000) NULL,
    DimensionCount INT NOT NULL,
    IsDiscrete BIT NOT NULL CONSTRAINT DF_RL_StateSpace_IsDiscrete DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RL_StateSpace_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RL_StateSpace_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_RL_StateSpace_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReinforcementLearning_StateSpace PRIMARY KEY (StateSpaceId),
    CONSTRAINT UQ_ReinforcementLearning_StateSpace_Code UNIQUE (Code),
    CONSTRAINT FK_RL_StateSpace_Type FOREIGN KEY (StateSpaceTypeId) REFERENCES
ReinforcementLearning.StateSpaceType(StateSpaceTypeId),
CONSTRAINT CK_RL_StateSpace_Dimensions CHECK (DimensionCount > 0)
);
GO

CREATE TABLE ReinforcementLearning.StateDefinition (
    StateDefinitionId INT IDENTITY(1,1) NOT NULL,
    StateSpaceId INT NOT NULL,
    StateIndex INT NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    DeviationBin NVARCHAR(50) NULL,
    TrendBin NVARCHAR(50) NULL,
    LowerBoundJson NVARCHAR(1000) NULL,
    UpperBoundJson NVARCHAR(1000) NULL,
    IsTerminal BIT NOT NULL CONSTRAINT DF_RL_StateDefinition_IsTerminal DEFAULT 0,
    DisplayOrder INT NOT NULL CONSTRAINT DF_RL_StateDefinition_DisplayOrder DEFAULT 0,

    CONSTRAINT PK_ReinforcementLearning_StateDefinition PRIMARY KEY (StateDefinitionId),
    CONSTRAINT UQ_RL_StateDefinition_Index UNIQUE (StateSpaceId, StateIndex),
    CONSTRAINT UQ_RL_StateDefinition_Code UNIQUE (StateSpaceId, Code),
    CONSTRAINT FK_RL_StateDefinition_StateSpace FOREIGN KEY (StateSpaceId) REFERENCES
ReinforcementLearning.StateSpace(StateSpaceId)
);
GO

CREATE TABLE ReinforcementLearning.ActionSpace (
    ActionSpaceId INT IDENTITY(1,1) NOT NULL,
    ActionSpaceTypeId INT NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    Description NVARCHAR(1000) NULL,
    EngineeringUnitId INT NOT NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RL_ActionSpace_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RL_ActionSpace_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_RL_ActionSpace_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReinforcementLearning_ActionSpace PRIMARY KEY (ActionSpaceId),
    CONSTRAINT UQ_ReinforcementLearning_ActionSpace_Code UNIQUE (Code),
    CONSTRAINT FK_RL_ActionSpace_Type FOREIGN KEY (ActionSpaceTypeId) REFERENCES
ReinforcementLearning.ActionSpaceType(ActionSpaceTypeId),
CONSTRAINT FK_RL_ActionSpace_EngineeringUnit FOREIGN KEY (EngineeringUnitId) REFERENCES
CorePlatform.EngineeringUnit(EngineeringUnitId)
);
GO

CREATE TABLE ReinforcementLearning.ActionDefinition (
    ActionDefinitionId INT IDENTITY(1,1) NOT NULL,
    ActionSpaceId INT NOT NULL,
    ActionIndex INT NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    ActionValue DECIMAL(18,6) NOT NULL,
    IsNoOp BIT NOT NULL CONSTRAINT DF_RL_ActionDefinition_IsNoOp DEFAULT 0,
    DisplayOrder INT NOT NULL CONSTRAINT DF_RL_ActionDefinition_DisplayOrder DEFAULT 0,

    CONSTRAINT PK_ReinforcementLearning_ActionDefinition PRIMARY KEY (ActionDefinitionId),
    CONSTRAINT UQ_RL_ActionDefinition_Index UNIQUE (ActionSpaceId, ActionIndex),
    CONSTRAINT UQ_RL_ActionDefinition_Code UNIQUE (ActionSpaceId, Code),
    CONSTRAINT FK_RL_ActionDefinition_ActionSpace FOREIGN KEY (ActionSpaceId) REFERENCES
ReinforcementLearning.ActionSpace(ActionSpaceId)
);
GO

CREATE TABLE ReinforcementLearning.RewardFunction (
    RewardFunctionId INT IDENTITY(1,1) NOT NULL,
    RewardFunctionTypeId INT NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    FormulaText NVARCHAR(2000) NOT NULL,
    ErrorWeight DECIMAL(18,6) NOT NULL CONSTRAINT DF_RL_RewardFunction_ErrorWeight DEFAULT 100.0,
    MovePenalty DECIMAL(18,6) NOT NULL CONSTRAINT DF_RL_RewardFunction_MovePenalty DEFAULT 0.3,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RL_RewardFunction_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RL_RewardFunction_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_RL_RewardFunction_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReinforcementLearning_RewardFunction PRIMARY KEY (RewardFunctionId),

```



```

CONSTRAINT UQ_ReinforcementLearning_RewardFunction_Code UNIQUE (Code),
CONSTRAINT FK_RL_RewardFunction_Type FOREIGN KEY (RewardFunctionTypeId) REFERENCES
ReinforcementLearning.RewardFunctionType(RewardFunctionTypeId)
);
GO

CREATE TABLE ReinforcementLearning.HyperparameterSet (
HyperparameterSetId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(80) NOT NULL,
Name NVARCHAR(200) NOT NULL,
LearningRateAlpha DECIMAL(10,6) NOT NULL,
DiscountGamma DECIMAL(10,6) NOT NULL,
EpsilonStart DECIMAL(10,6) NOT NULL,
EpsilonEnd DECIMAL(10,6) NOT NULL,
EpsilonDecay DECIMAL(10,6) NOT NULL,
EpisodeCount INT NOT NULL,
StepsPerEpisode INT NOT NULL,
RandomSeed INT NOT NULL,
CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RL_HyperparameterSet_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
CreatedBy NVARCHAR(100) NOT NULL,
ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RL_HyperparameterSet_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedBy NVARCHAR(100) NOT NULL,
IsDeleted BIT NOT NULL CONSTRAINT DF_RL_HyperparameterSet_IsDeleted DEFAULT 0,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_ReinforcementLearning_HyperparameterSet PRIMARY KEY (HyperparameterSetId),
CONSTRAINT UQ_ReinforcementLearning_HyperparameterSet_Code UNIQUE (Code),
CONSTRAINT CK_RL_HyperparameterSet_Alpha CHECK (LearningRateAlpha > 0 AND LearningRateAlpha <= 1),
CONSTRAINT CK_RL_HyperparameterSet_Gamma CHECK (DiscountGamma >= 0 AND DiscountGamma <= 1),
CONSTRAINT CK_RL_HyperparameterSet_Epsilon CHECK (EpsilonStart >= 0 AND EpsilonStart <= 1 AND EpsilonEnd >=
0 AND EpsilonEnd <= 1),
CONSTRAINT CK_RL_HyperparameterSet_Counts CHECK (EpisodeCount > 0 AND StepsPerEpisode > 0)
);
GO

CREATE TABLE ReinforcementLearning.Experiment (
ExperimentId INT IDENTITY(1,1) NOT NULL,
UnitId INT NOT NULL,
Code NVARCHAR(80) NOT NULL,
Name NVARCHAR(200) NOT NULL,
Objective NVARCHAR(1000) NULL,
OwnerUserId INT NOT NULL,
CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RL_Experiment_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
CreatedBy NVARCHAR(100) NOT NULL,
ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RL_Experiment_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedBy NVARCHAR(100) NOT NULL,
IsDeleted BIT NOT NULL CONSTRAINT DF_RL_Experiment_IsDeleted DEFAULT 0,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_ReinforcementLearning_Experiment PRIMARY KEY (ExperimentId),
CONSTRAINT UQ_ReinforcementLearning_Experiment_Code UNIQUE (Code),
CONSTRAINT FK_RL_Experiment_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
CONSTRAINT FK_RL_Experiment_OwnerUser FOREIGN KEY (OwnerUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

```

C.11.4.4 Training runs, episodes, Q-tables and artifacts

```

CREATE TABLE ReinforcementLearning.TrainingRun (
TrainingRunId INT IDENTITY(1,1) NOT NULL,
ExperimentId INT NOT NULL,
EnvironmentModelId INT NOT NULL,
StateSpaceId INT NOT NULL,
ActionSpaceId INT NOT NULL,
RewardFunctionId INT NOT NULL,
HyperparameterSetId INT NOT NULL,
LearningAlgorithmId INT NOT NULL,
TrainingRunStatusId INT NOT NULL,
Code NVARCHAR(80) NOT NULL,
StartedAtUtc DATETIME2(7) NULL,
CompletedAtUtc DATETIME2(7) NULL,
EpisodeCountCompleted INT NOT NULL CONSTRAINT DF_RL_TrainingRun_EpisodeCountCompleted DEFAULT 0,
TotalReward DECIMAL(20,6) NULL,
AverageReward DECIMAL(20,6) NULL,
RunSeed INT NOT NULL,
TrainingHash VARBINARY(32) NULL,
CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RL_TrainingRun_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
CreatedBy NVARCHAR(100) NOT NULL,
ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RL_TrainingRun_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedBy NVARCHAR(100) NOT NULL,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_ReinforcementLearning_TrainingRun PRIMARY KEY (TrainingRunId),
CONSTRAINT UQ_ReinforcementLearning_TrainingRun_Code UNIQUE (Code),
CONSTRAINT FK_RL_TrainingRun_Experiment FOREIGN KEY (ExperimentId) REFERENCES
ReinforcementLearning.Experiment(ExperimentId),
CONSTRAINT FK_RL_TrainingRun_EnvironmentModel FOREIGN KEY (EnvironmentModelId) REFERENCES
ReinforcementLearning.EnvironmentModel(EnvironmentModelId),
CONSTRAINT FK_RL_TrainingRun_StateSpace FOREIGN KEY (StateSpaceId) REFERENCES
ReinforcementLearning.StateSpace(StateSpaceId),
CONSTRAINT FK_RL_TrainingRun_ActionSpace FOREIGN KEY (ActionSpaceId) REFERENCES
ReinforcementLearning.ActionSpace(ActionSpaceId),
CONSTRAINT FK_RL_TrainingRun_RewardFunction FOREIGN KEY (RewardFunctionId) REFERENCES
ReinforcementLearning.RewardFunction(RewardFunctionId),
CONSTRAINT FK_RL_TrainingRun_HyperparameterSet FOREIGN KEY (HyperparameterSetId) REFERENCES
ReinforcementLearning.HyperparameterSet(HyperparameterSetId),
CONSTRAINT FK_RL_TrainingRun_LearningAlgorithm FOREIGN KEY (LearningAlgorithmId) REFERENCES
ReinforcementLearning.LearningAlgorithm(LearningAlgorithmId),
CONSTRAINT FK_RL_TrainingRun_Status FOREIGN KEY (TrainingRunStatusId) REFERENCES
ReinforcementLearning.TrainingRunStatus(TrainingRunStatusId)
);
GO

CREATE TABLE ReinforcementLearning.Episode (
EpisodeId BIGINT IDENTITY(1,1) NOT NULL,
TrainingRunId INT NOT NULL,
EpisodeStatusId INT NOT NULL,
EpisodeNumber INT NOT NULL,
TargetPowerPercent DECIMAL(9,4) NOT NULL,
InitialPowerPercent DECIMAL(9,4) NULL,
FinalPowerPercent DECIMAL(9,4) NULL,
StepCount INT NOT NULL CONSTRAINT DF_RL_Episode_StepCount DEFAULT 0,
TotalReward DECIMAL(20,6) NULL,
StartedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RL_Episode_StartedAtUtc DEFAULT SYSUTCDATETIME(),
CompletedAtUtc DATETIME2(7) NULL,

CONSTRAINT PK_ReinforcementLearning_Episode PRIMARY KEY (EpisodeId),
CONSTRAINT UQ_RL_Episode_Run_Number UNIQUE (TrainingRunId, EpisodeNumber),
CONSTRAINT FK_RL_Episode_TrainingRun FOREIGN KEY (TrainingRunId) REFERENCES
ReinforcementLearning.TrainingRun(TrainingRunId),
CONSTRAINT FK_RL_Episode_Status FOREIGN KEY (EpisodeStatusId) REFERENCES

```

```

ReinforcementLearning.EpisodeStatus(EpisodeStatusId),
CONSTRAINT CK_RL_Episode_Target CHECK (TargetPowerPercent >= 0 AND TargetPowerPercent <= 120)
);
GO

CREATE TABLE ReinforcementLearning.EpisodeStep (
    EpisodeStepId BIGINT IDENTITY(1,1) NOT NULL,
    EpisodeId BIGINT NOT NULL,
    StepNumber INT NOT NULL,
    StateDefinitionId INT NOT NULL,
    ActionDefinitionId INT NOT NULL,
    NextStateDefinitionId INT NOT NULL,
    PowerPercentBefore DECIMAL(12,6) NULL,
    PowerPercentAfter DECIMAL(12,6) NULL,
    ErrorBefore DECIMAL(12,6) NULL,
    Reward DECIMAL(20,6) NOT NULL,
    IsExploratoryAction BIT NOT NULL CONSTRAINT DF_RL_EpisodeStep_IsExploratoryAction DEFAULT 0,
    IsTerminal BIT NOT NULL CONSTRAINT DF_RL_EpisodeStep_IsTerminal DEFAULT 0,
    OccurredAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RL_EpisodeStep_OccurredAtUtc DEFAULT
SYSUTCDATETIME(),

    CONSTRAINT PK_ReinforcementLearning_EpisodeStep PRIMARY KEY (EpisodeStepId),
    CONSTRAINT UQ_RL_EpisodeStep_Number UNIQUE (EpisodeId, StepNumber),
    CONSTRAINT FK_RL_EpisodeStep_Episode FOREIGN KEY (EpisodeId) REFERENCES
ReinforcementLearning.Episode(EpisodeId),
    CONSTRAINT FK_RL_EpisodeStep_State FOREIGN KEY (StateDefinitionId) REFERENCES
ReinforcementLearning.StateDefinition(StateDefinitionId),
    CONSTRAINT FK_RL_EpisodeStep_Action FOREIGN KEY (ActionDefinitionId) REFERENCES
ReinforcementLearning.ActionDefinition(ActionDefinitionId),
    CONSTRAINT FK_RL_EpisodeStep_NextState FOREIGN KEY (NextStateDefinitionId) REFERENCES
ReinforcementLearning.StateDefinition(StateDefinitionId)
);
GO

CREATE TABLE ReinforcementLearning.QTable (
    QTableId INT IDENTITY(1,1) NOT NULL,
    TrainingRunId INT NOT NULL,
    StateSpaceId INT NOT NULL,
    ActionSpaceId INT NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    SnapshotAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RL_QTable_SnapshotAtUtc DEFAULT SYSUTCDATETIME(),
    EntryCount INT NOT NULL,
    IsFinal BIT NOT NULL CONSTRAINT DF_RL_QTable_IsFinal DEFAULT 0,
    TableHash VARBINARY(32) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RL_QTable_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReinforcementLearning_QTable PRIMARY KEY (QTableId),
    CONSTRAINT UQ_ReinforcementLearning_QTable_Code UNIQUE (Code),
    CONSTRAINT FK_RL_QTable_TrainingRun FOREIGN KEY (TrainingRunId) REFERENCES
ReinforcementLearning.TrainingRun(TrainingRunId),
    CONSTRAINT FK_RL_QTable_StateSpace FOREIGN KEY (StateSpaceId) REFERENCES
ReinforcementLearning.StateSpace(StateSpaceId),
    CONSTRAINT FK_RL_QTable_ActionSpace FOREIGN KEY (ActionSpaceId) REFERENCES
ReinforcementLearning.ActionSpace(ActionSpaceId),
    CONSTRAINT CK_RL_QTable_EntryCount CHECK (EntryCount > 0)
);
GO

CREATE TABLE ReinforcementLearning.QTableEntry (
    QTableEntryId BIGINT IDENTITY(1,1) NOT NULL,
    QTableId INT NOT NULL,
    StateDefinitionId INT NOT NULL,
    ActionDefinitionId INT NOT NULL,
    QValue DECIMAL(20,10) NOT NULL,
    VisitCount INT NOT NULL CONSTRAINT DF_RL_QTableEntry_VisitCount DEFAULT 0,
    LastUpdatedAtUtc DATETIME2(7) NULL,

    CONSTRAINT PK_ReinforcementLearning_QTableEntry PRIMARY KEY (QTableEntryId),
    CONSTRAINT UQ_RL_QTableEntry_StateAction UNIQUE (QTableId, StateDefinitionId, ActionDefinitionId),
    CONSTRAINT FK_RL_QTableEntry_QTable FOREIGN KEY (QTableId) REFERENCES
ReinforcementLearning.QTable(QTableId),
    CONSTRAINT FK_RL_QTableEntry_State FOREIGN KEY (StateDefinitionId) REFERENCES
ReinforcementLearning.StateDefinition(StateDefinitionId),
    CONSTRAINT FK_RL_QTableEntry_Action FOREIGN KEY (ActionDefinitionId) REFERENCES
ReinforcementLearning.ActionDefinition(ActionDefinitionId)
);
GO

CREATE TABLE ReinforcementLearning.ModelArtifact (
    ModelArtifactId INT IDENTITY(1,1) NOT NULL,
    TrainingRunId INT NOT NULL,
    ArtifactType NVARCHAR(50) NOT NULL,
    StorageUri NVARCHAR(1000) NOT NULL,
    Sha256Hash VARBINARY(32) NULL,
    SizeBytes BIGINT NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RL_ModelArtifact_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,

    CONSTRAINT PK_ReinforcementLearning_ModelArtifact PRIMARY KEY (ModelArtifactId),
    CONSTRAINT FK_RL_ModelArtifact_TrainingRun FOREIGN KEY (TrainingRunId) REFERENCES
ReinforcementLearning.TrainingRun(TrainingRunId)
);
GO

CREATE TABLE ReinforcementLearning.TrainingRunAuditTrail (
    TrainingRunAuditTrailId BIGINT IDENTITY(1,1) NOT NULL,
    TrainingRunId INT NOT NULL,
    EventType NVARCHAR(80) NOT NULL,
    Message NVARCHAR(1000) NOT NULL,
    DetailsJson NVARCHAR(MAX) NULL,
    OccurredAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RL_TrainingRunAuditTrail_OccurredAtUtc DEFAULT
SYSUTCDATETIME(),

    CONSTRAINT PK_ReinforcementLearning_TrainingRunAuditTrail PRIMARY KEY (TrainingRunAuditTrailId),
    CONSTRAINT FK_RL_TrainingRunAuditTrail_TrainingRun FOREIGN KEY (TrainingRunId) REFERENCES
ReinforcementLearning.TrainingRun(TrainingRunId)
);
GO

```

C.11.4.5 Policy, evaluation and deployment

```

CREATE TABLE ReinforcementLearning.Policy (
    PolicyId INT IDENTITY(1,1) NOT NULL,
    QTableId INT NOT NULL,
    PolicyStatusId INT NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    ExtractedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RL_Policy_ExtractedAtUtc DEFAULT SYSUTCDATETIME(),
    EntryCount INT NOT NULL,
    PolicyHash VARBINARY(32) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RL_Policy_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RL_Policy_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,

```

```

IsDeleted BIT NOT NULL CONSTRAINT DF_RL_Policy_IsDeleted DEFAULT 0,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_ReinforcementLearning_Policy PRIMARY KEY (PolicyId),
CONSTRAINT UQ_ReinforcementLearning_Policy_Code UNIQUE (Code),
CONSTRAINT FK_RL_Policy_QTable FOREIGN KEY (QTableId) REFERENCES ReinforcementLearning.QTable(QTableId),
CONSTRAINT FK_RL_Policy_Status FOREIGN KEY (PolicyStatusId) REFERENCES
ReinforcementLearning.PolicyStatus(PolicyStatusId)
);
GO

CREATE TABLE ReinforcementLearning.PolicyEntry (
    PolicyEntryId BIGINT IDENTITY(1,1) NOT NULL,
    PolicyId INT NOT NULL,
    StateDefinitionId INT NOT NULL,
    BestActionDefinitionId INT NOT NULL,
    BestQValue DECIMAL(20,10) NOT NULL,
    SecondBestQValue DECIMAL(20,10) NULL,
    ActionMargin DECIMAL(20,10) NULL,
    IsTie BIT NOT NULL CONSTRAINT DF_RL_PolicyEntry_IsTie DEFAULT 0,

    CONSTRAINT PK_ReinforcementLearning_PolicyEntry PRIMARY KEY (PolicyEntryId),
    CONSTRAINT UQ_RL_PolicyEntry_State UNIQUE (PolicyId, StateDefinitionId),
    CONSTRAINT FK_RL_PolicyEntry_Policy FOREIGN KEY (PolicyId) REFERENCES
ReinforcementLearning.Policy(PolicyId),
    CONSTRAINT FK_RL_PolicyEntry_State FOREIGN KEY (StateDefinitionId) REFERENCES
ReinforcementLearning.StateDefinition(StateDefinitionId),
    CONSTRAINT FK_RL_PolicyEntry_BestAction FOREIGN KEY (BestActionDefinitionId) REFERENCES
ReinforcementLearning.ActionDefinition(ActionDefinitionId)
);
GO

CREATE TABLE ReinforcementLearning.PolicyEvaluation (
    PolicyEvaluationId INT IDENTITY(1,1) NOT NULL,
    PolicyId INT NOT NULL,
    EnvironmentModelId INT NOT NULL,
    EvaluationCode NVARCHAR(80) NOT NULL,
    EpisodeCount INT NOT NULL,
    StartedAtUtc DATETIME2(7) NOT NULL,
    CompletedAtUtc DATETIME2(7) NULL,
    Passed BIT NULL,
    Summary NVARCHAR(1000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RL_PolicyEvaluation_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,

    CONSTRAINT PK_ReinforcementLearning_PolicyEvaluation PRIMARY KEY (PolicyEvaluationId),
    CONSTRAINT UQ_ReinforcementLearning_PolicyEvaluation_Code UNIQUE (EvaluationCode),
    CONSTRAINT FK_RL_PolicyEvaluation_Policy FOREIGN KEY (PolicyId) REFERENCES
ReinforcementLearning.Policy(PolicyId),
    CONSTRAINT FK_RL_PolicyEvaluation_EnvironmentModel FOREIGN KEY (EnvironmentModelId) REFERENCES
ReinforcementLearning.EnvironmentModel(EnvironmentModelId),
    CONSTRAINT CK_RL_PolicyEvaluation_EpisodeCount CHECK (EpisodeCount > 0)
);
GO

CREATE TABLE ReinforcementLearning.PolicyEvaluationMetric (
    PolicyEvaluationMetricId INT IDENTITY(1,1) NOT NULL,
    PolicyEvaluationId INT NOT NULL,
    EvaluationMetricTypeId INT NOT NULL,
    MetricValue DECIMAL(20,8) NOT NULL,
    MetricUnit NVARCHAR(40) NULL,
    ThresholdValue DECIMAL(20,8) NULL,
    Passed BIT NULL,

    CONSTRAINT PK_ReinforcementLearning_PolicyEvaluationMetric PRIMARY KEY (PolicyEvaluationMetricId),
    CONSTRAINT UQ_RL_PolicyEvaluationMetric_Type UNIQUE (PolicyEvaluationId, EvaluationMetricTypeId),
    CONSTRAINT FK_RL_PolicyEvaluationMetric_Evaluation FOREIGN KEY (PolicyEvaluationId) REFERENCES
ReinforcementLearning.PolicyEvaluation(PolicyEvaluationId),
    CONSTRAINT FK_RL_PolicyEvaluationMetric_Type FOREIGN KEY (EvaluationMetricTypeId) REFERENCES
ReinforcementLearning.EvaluationMetricType(EvaluationMetricTypeId)
);
GO

CREATE TABLE ReinforcementLearning.PolicyDeployment (
    PolicyDeploymentId INT IDENTITY(1,1) NOT NULL,
    PolicyId INT NOT NULL,
    AdvisoryModeId INT NOT NULL,
    UnitId INT NOT NULL,
    DeployedByUserId INT NOT NULL,
    DeployedAtUtc DATETIME2(7) NOT NULL,
    RetiredAtUtc DATETIME2(7) NULL,
    IsActive BIT NOT NULL CONSTRAINT DF_RL_PolicyDeployment_IsActive DEFAULT 1,
    DeploymentNote NVARCHAR(1000) NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReinforcementLearning_PolicyDeployment PRIMARY KEY (PolicyDeploymentId),
    CONSTRAINT FK_RL_PolicyDeployment_Policy FOREIGN KEY (PolicyId) REFERENCES
ReinforcementLearning.Policy(PolicyId),
    CONSTRAINT FK_RL_PolicyDeployment_AdvisoryMode FOREIGN KEY (AdvisoryModeId) REFERENCES
ReinforcementLearning.AdvisoryMode(AdvisoryModeId),
    CONSTRAINT FK_RL_PolicyDeployment_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_RL_PolicyDeployment_DeployedBy FOREIGN KEY (DeployedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

```

C.11.4.6 Safety clamps and advisory recommendations

```

CREATE TABLE ReinforcementLearning.SafetyClamp (
    SafetyClampId INT IDENTITY(1,1) NOT NULL,
    UnitId INT NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    Description NVARCHAR(1000) NULL,
    IsActive BIT NOT NULL CONSTRAINT DF_RL_SafetyClamp_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RL_SafetyClamp_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RL_SafetyClamp_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_ReinforcementLearning_SafetyClamp PRIMARY KEY (SafetyClampId),
    CONSTRAINT UQ_ReinforcementLearning_SafetyClamp_Code UNIQUE (Code),
    CONSTRAINT FK_RL_SafetyClamp_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId)
);
GO

CREATE TABLE ReinforcementLearning.SafetyClampRule (
    SafetyClampRuleId INT IDENTITY(1,1) NOT NULL,
    SafetyClampId INT NOT NULL,
    SafetyClampTypeId INT NOT NULL,
    RuleCode NVARCHAR(80) NOT NULL,
    RuleExpression NVARCHAR(1000) NOT NULL,
    ClampAction NVARCHAR(100) NOT NULL,
    DisplayOrder INT NOT NULL CONSTRAINT DF_RL_SafetyClampRule_DisplayOrder DEFAULT 0,

```

```

IsActive          BIT NOT NULL CONSTRAINT DF_RL_SafetyClampRule_IsActive DEFAULT 1,

CONSTRAINT PK_ReinforcementLearning_SafetyClampRule PRIMARY KEY (SafetyClampRuleId),
CONSTRAINT UQ_RL_SafetyClampRule_Code UNIQUE (SafetyClampId, RuleCode),
CONSTRAINT FK_RL_SafetyClampRule_Clamp FOREIGN KEY (SafetyClampId) REFERENCES
ReinforcementLearning.SafetyClamp(SafetyClampId),
CONSTRAINT FK_RL_SafetyClampRule_Type FOREIGN KEY (SafetyClampTypeId) REFERENCES
ReinforcementLearning.SafetyClampType(SafetyClampTypeId)
);
GO

CREATE TABLE ReinforcementLearning.AdvisorySession (
  AdvisorySessionId BIGINT IDENTITY(1,1) NOT NULL,
  PolicyDeploymentId INT NOT NULL,
  UnitId             INT NOT NULL,
  StartedByUserId    INT NOT NULL,
  StartedAtUtc       DATETIME2(7) NOT NULL CONSTRAINT DF_RL_AdvisorySession_StartedAtUtc DEFAULT
SYSUTCDATETIME(),
  EndedAtUtc         DATETIME2(7) NULL,
  SessionNote        NVARCHAR(1000) NULL,

  CONSTRAINT PK_ReinforcementLearning_AdvisorySession PRIMARY KEY (AdvisorySessionId),
  CONSTRAINT FK_RL_AdvisorySession_Deployment FOREIGN KEY (PolicyDeploymentId) REFERENCES
ReinforcementLearning.PolicyDeployment(PolicyDeploymentId),
  CONSTRAINT FK_RL_AdvisorySession_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
  CONSTRAINT FK_RL_AdvisorySession_User FOREIGN KEY (StartedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE ReinforcementLearning.AdvisoryRecommendation (
  AdvisoryRecommendationId BIGINT IDENTITY(1,1) NOT NULL,
  AdvisorySessionId        BIGINT NOT NULL,
  RecommendationStatusId    INT NOT NULL,
  StateDefinitionId         INT NOT NULL,
  RecommendedActionDefinitionId INT NOT NULL,
  ClampedActionDefinitionId INT NOT NULL,
  ObservedPowerPercent      DECIMAL(12,6) NULL,
  TargetPowerPercent        DECIMAL(12,6) NULL,
  ConfidenceScore           DECIMAL(10,6) NULL,
  WasClamped                BIT NOT NULL CONSTRAINT DF_RL_AdvisoryRecommendation_WasClamped DEFAULT 0,
  ClampReason               NVARCHAR(500) NULL,
  RequestedAtUtc            DATETIME2(7) NOT NULL CONSTRAINT DF_RL_AdvisoryRecommendation_RequestedAtUtc
DEFAULT SYSUTCDATETIME(),
  ExpiresAtUtc              DATETIME2(7) NULL,

  CONSTRAINT PK_ReinforcementLearning_AdvisoryRecommendation PRIMARY KEY (AdvisoryRecommendationId),
  CONSTRAINT FK_RL_AdvisoryRecommendation_Session FOREIGN KEY (AdvisorySessionId) REFERENCES
ReinforcementLearning.AdvisorySession(AdvisorySessionId),
  CONSTRAINT FK_RL_AdvisoryRecommendation_Status FOREIGN KEY (RecommendationStatusId) REFERENCES
ReinforcementLearning.RecommendationStatus(RecommendationStatusId),
  CONSTRAINT FK_RL_AdvisoryRecommendation_State FOREIGN KEY (StateDefinitionId) REFERENCES
ReinforcementLearning.StateDefinition(StateDefinitionId),
  CONSTRAINT FK_RL_AdvisoryRecommendation_Action FOREIGN KEY (RecommendedActionDefinitionId) REFERENCES
ReinforcementLearning.ActionDefinition(ActionDefinitionId),
  CONSTRAINT FK_RL_AdvisoryRecommendation_ClampedAction FOREIGN KEY (ClampedActionDefinitionId) REFERENCES
ReinforcementLearning.ActionDefinition(ActionDefinitionId),
  CONSTRAINT CK_RL_AdvisoryRecommendation_Confidence CHECK (ConfidenceScore IS NULL OR (ConfidenceScore >= 0
AND ConfidenceScore <= 1))
);
GO

CREATE TABLE ReinforcementLearning.AdvisoryRecommendationReason (
  AdvisoryRecommendationReasonId BIGINT IDENTITY(1,1) NOT NULL,
  AdvisoryRecommendationId        BIGINT NOT NULL,
  ActionDefinitionId              INT NOT NULL,
  QValue                         DECIMAL(20,10) NULL,
  ReasonLabel                    NVARCHAR(200) NOT NULL,
  ReasonText                     NVARCHAR(1000) NULL,
  DisplayOrder                   INT NOT NULL CONSTRAINT DF_RL_AdvisoryRecommendationReason_DisplayOrder
DEFAULT 0,

  CONSTRAINT PK_ReinforcementLearning_AdvisoryRecommendationReason PRIMARY KEY
(AdvisoryRecommendationReasonId),
  CONSTRAINT FK_RL_AdvisoryRecommendationReason_Recommendation FOREIGN KEY (AdvisoryRecommendationId)
REFERENCES ReinforcementLearning.AdvisoryRecommendation(AdvisoryRecommendationId),
  CONSTRAINT FK_RL_AdvisoryRecommendationReason_Action FOREIGN KEY (ActionDefinitionId) REFERENCES
ReinforcementLearning.ActionDefinition(ActionDefinitionId)
);
GO

```

C.11.5 Indexes and seed examples

The critical access paths are run-by-experiment, episode-by-run, step-by-episode, Q-values by table and state, policy entry by state, and recent advisory recommendations by session.

```

CREATE INDEX IX_RL_EnvironmentModel_UnitId ON ReinforcementLearning.EnvironmentModel(UnitId);
CREATE INDEX IX_RL_EnvironmentModel_TwinModelId ON ReinforcementLearning.EnvironmentModel(TwinModelId);
CREATE INDEX IX_RL_StateDefinition_StateSpaceId ON ReinforcementLearning.StateDefinition(StateSpaceId,
StateIndex);
CREATE INDEX IX_RL_ActionDefinition_ActionSpaceId ON ReinforcementLearning.ActionDefinition(ActionSpaceId,
ActionIndex);
CREATE INDEX IX_RL_Experiment_UnitId ON ReinforcementLearning.Experiment(UnitId);
CREATE INDEX IX_RL_TrainingRun_ExperimentId ON ReinforcementLearning.TrainingRun(ExperimentId, CreatedAtUtc
DESC);
CREATE INDEX IX_RL_TrainingRun_Status ON ReinforcementLearning.TrainingRun(TrainingRunStatusId, StartedAtUtc
DESC);
CREATE INDEX IX_RL_Episode_TrainingRunId ON ReinforcementLearning.Episode(TrainingRunId, EpisodeNumber);
CREATE INDEX IX_RL_EpisodeStep_EpisodeId ON ReinforcementLearning.EpisodeStep(EpisodeId, StepNumber);
CREATE INDEX IX_RL_QTable_TrainingRunId ON ReinforcementLearning.QTable(TrainingRunId, IsFinal DESC);
CREATE INDEX IX_RL_QTableEntry_QTable_State ON ReinforcementLearning.QTableEntry(QTableId, StateDefinitionId);
CREATE INDEX IX_RL_Policy_Status ON ReinforcementLearning.Policy(PolicyStatusId, ExtractedAtUtc DESC);
CREATE INDEX IX_RL_PolicyEntry_PolicyId ON ReinforcementLearning.PolicyEntry(PolicyId, StateDefinitionId);
CREATE INDEX IX_RL_PolicyDeployment_Unit_Active ON ReinforcementLearning.PolicyDeployment(UnitId, IsActive)
INCLUDE (PolicyId, AdvisoryModeId);
CREATE INDEX IX_RL_AdvisorySession_Deployment ON ReinforcementLearning.AdvisorySession(PolicyDeploymentId,
StartedAtUtc DESC);
CREATE INDEX IX_RL_AdvisoryRecommendation_Session ON
ReinforcementLearning.AdvisoryRecommendation(AdvisorySessionId, RequestedAtUtc DESC);
CREATE INDEX IX_RL_TrainingRunAuditTrail_Run ON ReinforcementLearning.TrainingRunAuditTrail(TrainingRunId,
OccurredAtUtc DESC);
GO

```

C.11.5.1 Seed examples

```
INSERT INTO ReinforcementLearning.LearningAlgorithm (Code, Name, Description)
VALUES ('Q_LEARNING', 'Tabular Q-learning', 'Readable state-action table updated by the Q-learning rule.');
```

```
INSERT INTO ReinforcementLearning.StateSpaceType (Code, Name)
VALUES ('DISCRETE_BINS', 'Discrete bins');
```

```
INSERT INTO ReinforcementLearning.ActionSpaceType (Code, Name)
VALUES ('DISCRETE_PCM', 'Discrete pcm actions');
```

```
INSERT INTO ReinforcementLearning.RewardFunctionType (Code, Name)
VALUES ('SQUARED_ERROR_MOVE_COST', 'Squared error plus movement cost');
```

```
INSERT INTO ReinforcementLearning.PolicyStatus (Code, Name)
VALUES ('DRAFT', 'Draft'), ('CANDIDATE', 'Candidate'), ('VALIDATED', 'Validated'), ('DEPLOYED', 'Deployed'),
('RETIRED', 'Retired');
```

```
INSERT INTO ReinforcementLearning.ActionDefinition (ActionSpaceId, ActionIndex, Code, Name, ActionValue, IsNoOp,
DisplayOrder)
VALUES
(@RodActionSpaceId, 0, 'INSERT_HARD', 'Insert hard', -8.0, 0, 10),
(@RodActionSpaceId, 1, 'INSERT_GENTLY', 'Insert gently', -2.0, 0, 20),
(@RodActionSpaceId, 2, 'HOLD', 'Hold', 0.0, 1, 30),
(@RodActionSpaceId, 3, 'WITHDRAW_GENTLY', 'Withdraw gently', 2.0, 0, 40),
(@RodActionSpaceId, 4, 'WITHDRAW_HARD', 'Withdraw hard', 8.0, 0, 50);
```

```
INSERT INTO ReinforcementLearning.RewardFunction
(RewardFunctionTypeId, Code, Name, FormulaText, ErrorWeight, MovePenalty, CreatedBy, ModifiedBy)
VALUES
(@SquaredErrorMoveCostTypeId, 'POWER_TRACKING_V1', 'Power tracking reward v1',
'-(power - target)^2 * 100 - 0.3 when action != hold', 100.0, 0.3, 'seed', 'seed');
```

```
GO
```

C.11.5.2 Verification queries

```
-- The 35 x 5 policy should have one entry per state and one selected action per state.
SELECT p.Code, COUNT(*) AS policy_entries
FROM ReinforcementLearning.Policy AS p
JOIN ReinforcementLearning.PolicyEntry AS pe ON pe.PolicyId = p.PolicyId
GROUP BY p.Code;
```

```
-- A final Q-table for the standard NEXUS-1 demo should contain 175 state-action values.
SELECT qt.Code, COUNT(*) AS q_values
FROM ReinforcementLearning.QTable AS qt
JOIN ReinforcementLearning.QTableEntry AS qte ON qte.QTableId = qt.QTableId
WHERE qt.IsFinal = 1
GROUP BY qt.Code;
```

```
-- Read the policy in console form: deviation/trend state to best rod action.
SELECT s.StateIndex, s.Code AS state_code, a.Code AS best_action, pe.BestQValue, pe.ActionMargin
FROM ReinforcementLearning.PolicyEntry AS pe
JOIN ReinforcementLearning.StateDefinition AS s ON s.StateDefinitionId = pe.StateDefinitionId
JOIN ReinforcementLearning.ActionDefinition AS a ON a.ActionDefinitionId = pe.BestActionDefinitionId
WHERE pe.PolicyId = @PolicyId
ORDER BY s.StateIndex;
```

```
-- Review advisory recommendations that were clamped before display.
SELECT r.AdvisoryRecommendationId, r.RequestedAtUtc, s.Code AS state_code,
a.Code AS recommended_action, ca.Code AS clamped_action, r.ClampReason
FROM ReinforcementLearning.AdvisoryRecommendation AS r
JOIN ReinforcementLearning.StateDefinition AS s ON s.StateDefinitionId = r.StateDefinitionId
JOIN ReinforcementLearning.ActionDefinition AS a ON a.ActionDefinitionId = r.RecommendedActionDefinitionId
LEFT JOIN ReinforcementLearning.ActionDefinition AS ca ON ca.ActionDefinitionId = r.ClampedActionDefinitionId
WHERE r.WasClamped = 1
ORDER BY r.RequestedAtUtc DESC;
GO
```

C.11.6 ER diagram

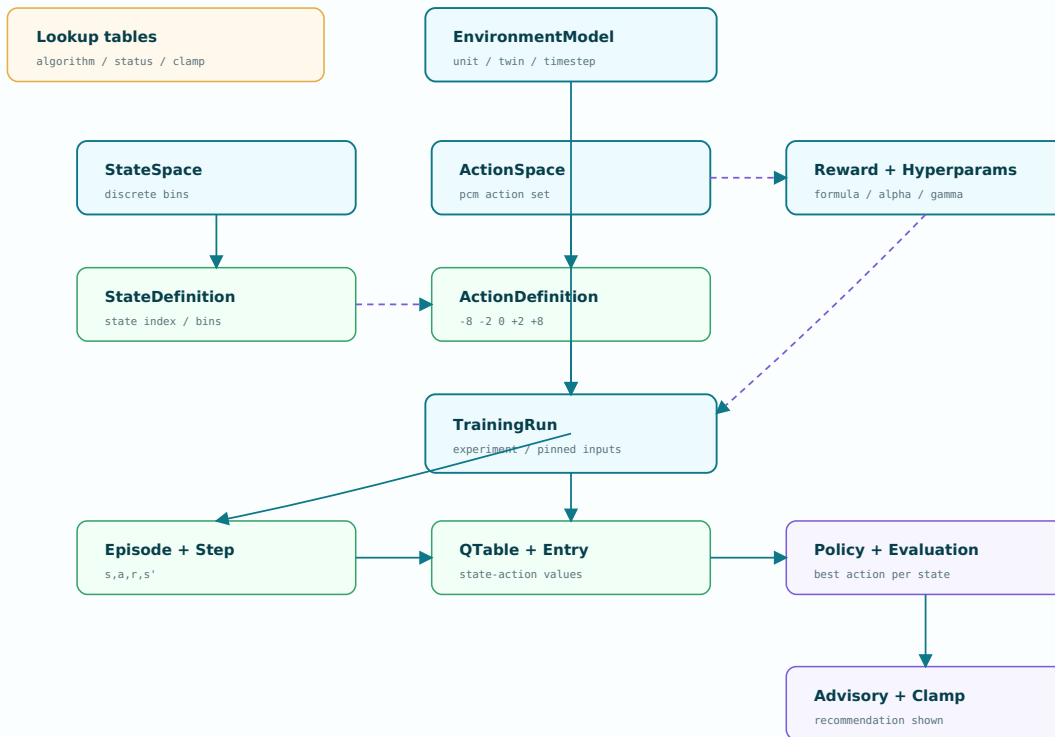
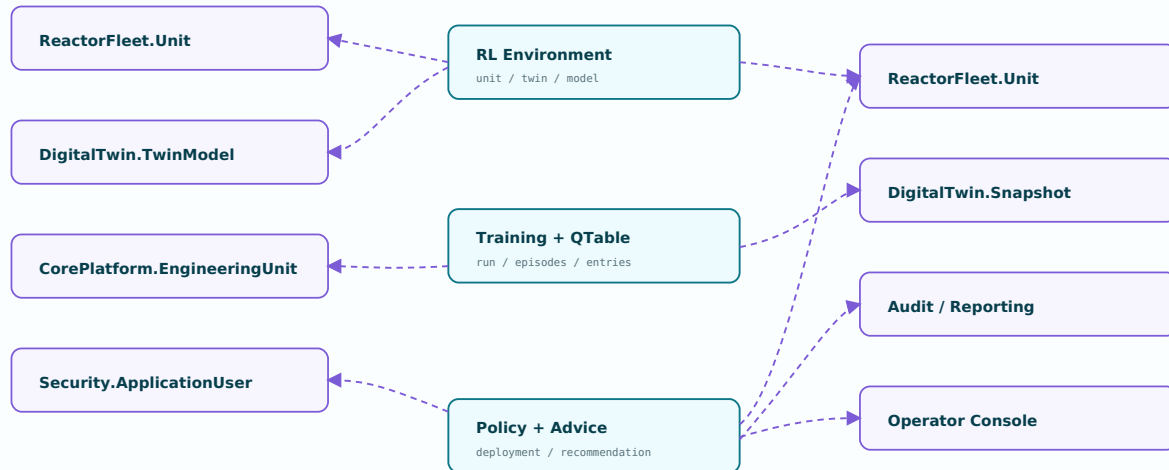


Figure C.11.1 - ReinforcementLearning sector ER diagram. The training apparatus produces a Q-table, the Q-table produces a readable policy, and the deployed policy produces clamped advisory recommendations.



ReinforcementLearning never owns the plant. It points to Unit and TwinModel, learns in a safe environment, then exposes a policy and advisory recommendation under clamps.

Figure C.11.2 - ReinforcementLearning passport map. ReinforcementLearning consumes Unit, TwinModel, EngineeringUnit and user identity through explicit foreign keys.

C.11.7 Foreign-key mapping

The table below lists the main passports. Lookup-table foreign keys are present in the DDL but are not repeated line by line.

EnvironmentModel

FK EnvironmentModelTypeId
TO ReinforcementLearning.EnvironmentModelType
Classifies the safe training world.

EnvironmentModel

FK UnitId
TO ReactorFleet.Unit
Pins the environment to the unit being simulated or mirrored.

EnvironmentModel

FK TwinModelId
TO DigitalTwin.TwinModel
Optionally binds training to a specific digital twin model.

StateSpace

FK StateSpaceTypeId
TO ReinforcementLearning.StateSpaceType
Declares the state representation.

StateDefinition

FK StateSpaceId
TO ReinforcementLearning.StateSpace
Each state belongs to exactly one state space.

ActionSpace

FK ActionSpaceTypeId
TO ReinforcementLearning.ActionSpaceType
Declares discrete pcm actions or future continuous action spaces.

ActionSpace

FK EngineeringUnitId
TO CorePlatform.EngineeringUnit
Uses a real unit such as pcm for rod nudges.

ActionDefinition

FK ActionSpaceId
TO ReinforcementLearning.ActionSpace
Each action belongs to one action space.

RewardFunction

FK RewardFunctionTypeId
TO ReinforcementLearning.RewardFunctionType
Classifies the reward formula.

Experiment

FK UnitId
TO ReactorFleet.Unit
Groups learning work around one physical unit.

Experiment

FK OwnerUserId
TO Security.ApplicationUser
Records accountable owner.

TrainingRun

FK ExperimentId
TO ReinforcementLearning.Experiment
Groups runs inside an experiment.

TrainingRun

FK EnvironmentModelId
TO ReinforcementLearning.EnvironmentModel
Pins the environment used for learning.

TrainingRun

FK StateSpaceId
TO ReinforcementLearning.StateSpace
Pins state encoding.

TrainingRun

FK ActionSpaceId
TO ReinforcementLearning.ActionSpace
Pins action catalogue.

TrainingRun

FK RewardFunctionId
TO ReinforcementLearning.RewardFunction
Pins reward formula.

TrainingRun

FK HyperparameterSetId
TO ReinforcementLearning.HyperparameterSet
Pins alpha, gamma, epsilon and episode counts.

Episode

FK TrainingRunId
TO ReinforcementLearning.TrainingRun
Episode belongs to a run.

EpisodeStep

FK EpisodeId
TO ReinforcementLearning.Episode
Transition belongs to an episode.

EpisodeStep

FK StateDefinitionId
TO ReinforcementLearning.StateDefinition
Transition starts from this state.

EpisodeStep

FK ActionDefinitionId
TO ReinforcementLearning.ActionDefinition
Transition applied this action.

QTable

FK TrainingRunId
TO ReinforcementLearning.TrainingRun
Q-table generated by a run.

QTableEntry

FK QTableId
TO ReinforcementLearning.QTable
One Q-value belongs to one table.

Policy

FK QTableId
TO ReinforcementLearning.QTable
Readable policy extracted from the table.

PolicyEntry

FK PolicyId
TO ReinforcementLearning.Policy
One selected action per state.

PolicyDeployment

FK PolicyId
TO ReinforcementLearning.Policy
Deploys a validated policy.

PolicyDeployment

FK UnitId
TO ReactorFleet.Unit
Policy deployment is unit-scoped.

SafetyClamp

FK UnitId
TO ReactorFleet.Unit
Safety clamp is unit-scoped.

AdvisorySession

FK PolicyDeploymentId
TO ReinforcementLearning.PolicyDeployment
Session asks a deployed policy for advice.

AdvisoryRecommendation

FK StateDefinitionId
TO ReinforcementLearning.StateDefinition
Recommendation was made for this observed state.

AdvisoryRecommendation

FK RecommendedActionDefinitionId
TO ReinforcementLearning.ActionDefinition
Raw policy action.

AdvisoryRecommendation

FK ClampedActionDefinitionId
TO ReinforcementLearning.ActionDefinition
Displayed action after clamp, when changed.

C.11.8 Boundary and next sector

Honest boundary. This schema stores an interpretable learning agent for a demonstration digital twin. It does not give the agent control authority over a plant. Training happens in a safe environment; deployment is advisory; recommendations are clamped; and every row is designed so a human can inspect the table rather than trust a hidden model.

The next schema is **Robotics**. It will store robots, vehicles, capabilities, missions, routes, readiness checks and mission telemetry. Unlike ReinforcementLearning, which learns a table for operator advice, Robotics models physical and remote assets that can be dispatched inside the plant demonstrator.

Appendix C.12

Robotics Schema

The Robotics sector stores the mobile and remote assets that the NEXUS-1 console shows as robots, vehicles and mission-ready equipment. It knows what each robot is, what it can do, where it is allowed to operate, which mission it is assigned to, and whether it is ready to dispatch.

This schema does not own plant topology, sensor identity, human identity, maintenance work orders or operational events. It connects to those sectors by passports: `UnitId`, `EquipmentId`, `EquipmentLocationId`, `SignalId`, `ApplicationUserId`, `WorkOrderId` and `OperationalEventId`.

Robotics answers: which robot can go, what it can carry, where it is allowed to move, what mission it is on, what telemetry it is sending, and what blocked dispatch if it could not go.

C.12.1 Sector purpose	C.12.2 Full table list
C.12.3 Lookup tables	C.12.4 SQL DDL
C.12.5 Indexes and seed examples	C.12.6 ER diagram
C.12.7 Foreign-key mapping	C.12.8 Boundary and next sector

C.12.1 Sector purpose

`Robotics` is operational rather than analytical. It is the place where a crawler, rover, drone or manipulator becomes a row with a status, a battery state, a payload, a work envelope and a mission. The schema is deliberately separate from `Maintenance`; a robot may need a work order, but the work order is still owned by Maintenance.

- `RobotModel` and `Robot` describe the fleet.
- `RobotCapability`, `RobotCapabilityAssignment` and `RobotPayload` describe what each robot can do and carry.
- `RobotSensorBinding` makes robot-mounted sensors normal `Instrumentation.Signal` records.

- `RobotTelemetry`, `RobotLocationHistory` and `RobotHealthSnapshot` store the runtime state.
- `Mission`, `MissionTask`, `MissionRoute` and `MissionRouteWaypoint` store dispatch and route planning.
- `MissionReadinessAssessment` and `MissionReadinessItem` store the dispatch gate.

Design choice. Robot telemetry is not mixed with plant instrumentation history. Robot-mounted sensors are bound to `Instrumentation.Signal`, but robot health metrics remain in `Robotics.RobotTelemetry`. This keeps plant measurements and asset telemetry related but not confused.

C.12.2 Full table list

The Robotics sector contains thirty-eight tables: fifteen typed lookup tables and twenty-three substantive tables. The tables fall into six groups: fleet, telemetry, docking, mission, readiness and integration.

LOOKUP

Robotics.RobotType

PK `RobotTypeId`

FK None

Robot category: crawler, drone, manipulator, rover, inspection arm or support vehicle.

LOOKUP

Robotics.RobotStatus

PK `RobotStatusId`

FK None

Operational state of a robot: available, charging, deployed, faulted, maintenance or retired.

LOOKUP

Robotics.RobotCapabilityType

PK `RobotCapabilityTypeId`

FK None

Capability catalogue: visual inspection, radiation survey, thermal imaging, sample pickup, valve manipulation.

LOOKUP

Robotics.PayloadType

PK `PayloadTypeId`

FK None

Payload category: camera, dosimeter, thermal sensor, manipulator, sample container, LiDAR.

LOOKUP

Robotics.MissionType

PK `MissionTypeId`

FK None

Mission category: inspection, radiation survey, patrol, delivery, recovery, training drill.

LOOKUP

Robotics.MissionStatus

PK `MissionStatusId`

FK None

Mission lifecycle: planned, approved, active, paused, completed, aborted, cancelled.

LOOKUP

Robotics.MissionPriority

PK MissionPriorityId

FK None

Priority tier used by dispatch and readiness views.

LOOKUP

Robotics.MissionTaskStatus

PK MissionTaskStatusId

FK None

Task lifecycle inside a mission: pending, active, completed, skipped, failed.

LOOKUP

Robotics.RouteType

PK RouteTypeId

FK None

Route family: predefined, operator drawn, autonomous, return-to-base, emergency extraction.

LOOKUP

Robotics.WaypointType

PK WaypointTypeId

FK None

Waypoint purpose: start, transit, inspection, sample, hold, charging, exit.

LOOKUP

Robotics.BatteryStatus

PK BatteryStatusId

FK None

Battery status used by fleet cards and readiness gates.

LOOKUP

Robotics.CommunicationStatus

PK CommunicationStatusId

FK None

Radio/network condition: nominal, degraded, lost, local-only, simulated.

LOOKUP

Robotics.ReadinessStatus

PK ReadinessStatusId

FK None

Readiness assessment outcome: ready, conditional, blocked, expired, unknown.

LOOKUP

Robotics.DockingStationStatus

PK DockingStationStatusId

FK None

Docking station state: available, occupied, charging, faulted, isolated.

LOOKUP

Robotics.TelemetryQuality

PK TelemetryQualityId

FK None

Quality of robot telemetry samples: good, estimated, stale, lost, simulated.

FLEET

Robotics.RobotModel

PK RobotModelId

FK RobotTypeId

Manufacturer/model record with physical limits, battery capacity and navigation profile.

FLEET**Robotics.Robot**

PK RobotId
FK RobotModelId , UnitId , RobotStatusId

One physical or simulated robotic asset in the plant demonstrator.

FLEET**Robotics.RobotCapability**

PK RobotCapabilityId
FK RobotCapabilityTypeId , EngineeringUnitId

Governed capability definition, optionally measured in an engineering unit.

FLEET**Robotics.RobotCapabilityAssignment**

PK RobotCapabilityAssignmentId
FK RobotId , RobotCapabilityId

Which robot can perform which capability, with rating and validation date.

FLEET**Robotics.RobotPayload**

PK RobotPayloadId
FK RobotId , PayloadTypeId

Payload mounted on a robot: camera, detector, manipulator or sample tool.

FLEET**Robotics.RobotSensorBinding**

PK RobotSensorBindingId
FK RobotId , SignalId

Binds robot-mounted sensors to the Instrumentation signal catalogue.

FLEET**Robotics.RobotCommunicationLink**

PK RobotCommunicationLinkId
FK RobotId , CommunicationStatusId

Current or configured communication channel for a robot.

FLEET**Robotics.RobotWorkEnvelope**

PK RobotWorkEnvelopeId
FK RobotId , UnitId , EquipmentLocationId

Where the robot is authorised to operate inside a unit or location.

TELEMETRY**Robotics.RobotTelemetry**

PK RobotTelemetryId
FK RobotId , EngineeringUnitId ,
TelemetryQualityId

High-volume telemetry metric: battery %, dose rate, temperature, speed, link quality.

TELEMETRY**Robotics.RobotLocationHistory**

PK RobotLocationHistoryId
FK RobotId , UnitId , EquipmentLocationId

Timestamped location trace for missions and playback.

TELEMETRY**Robotics.RobotHealthSnapshot**

PK RobotHealthSnapshotId
FK RobotId , BatteryStatusId ,
CommunicationStatusId

Periodic health summary used by mission-readiness screens.

DOCKING

Robotics.DockingStation

PK DockingStationId
FK UnitId, EquipmentLocationId,
 DockingStationStatusId

Charging/docking point in the demonstrator plant.

DOCKING

Robotics.ChargingSession

PK ChargingSessionId
FK RobotId, DockingStationId

Charging interval, energy delivered and battery state before/after.

MISSION

Robotics.Mission

PK MissionId
FK UnitId, MissionTypeId, MissionStatusId

Mission header: purpose, scope, priority, requested user, target unit and schedule.

MISSION

Robotics.MissionRobotAssignment

PK MissionRobotAssignmentId
FK MissionId, RobotId

Robot assigned to a mission with role and planned start/end.

MISSION

Robotics.MissionTask

PK MissionTaskId
FK MissionId, MissionTaskStatusId

Task list inside a mission: inspect, measure, capture image, sample, return.

MISSION

Robotics.MissionRoute

PK MissionRouteId
FK MissionId, RouteTypeId

Route plan for a mission.

MISSION

Robotics.MissionRouteWaypoint

PK MissionRouteWaypointId
FK MissionRouteId, WaypointTypeId

One ordered waypoint in a mission route.

MISSION

Robotics.MissionPayloadUsage

PK MissionPayloadUsageId
FK MissionId, RobotPayloadId

Payload used in a mission and the operational purpose for which it was used.

MISSION

Robotics.MissionEvent

PK MissionEventId
FK MissionId, RobotId

Append-only mission timeline: dispatch, waypoint reached, measurement captured, fault, return.

READINESS

Robotics.MissionChecklist

PK MissionChecklistId
FK MissionTypeId

Checklist template for a mission type.

READINESS Robotics.MissionChecklistItem PK MissionChecklistItemId FK MissionChecklistId One checklist rule, such as battery threshold or required capability.	READINESS Robotics.MissionReadinessAssessment PK MissionReadinessAssessmentId FK MissionId , ReadinessStatusId Readiness assessment for a mission before dispatch.
READINESS Robotics.MissionReadinessItem PK MissionReadinessItemId FK MissionReadinessAssessmentId , ReadinessStatusId One pass/fail/conditional readiness check.	INTEGRATION Robotics.RobotMaintenanceLink PK RobotMaintenanceLinkId FK RobotId , WorkOrderId Links robot faults or service requirements to Maintenance work orders.
INTEGRATION Robotics.MissionOperationalEventLink PK MissionOperationalEventLinkId FK MissionId , OperationalEventId Connects a robotic mission to an EventManagement operational event.	

C.12.3 Lookup tables

Lookup tables make robot and mission states explicit. No robot status, mission priority, waypoint type or readiness result is stored as free text.

LOOKUP TABLE	TYPICAL SEED CODES	MAIN CONSUMER
RobotType	CRAWLER , DRONE , ROVER , MANIPULATOR , VEHICLE	Referenced by RobotModel .
RobotStatus	AVAILABLE , CHARGING , DEPLOYED , FAULTED , MAINTENANCE , RETIRED	Referenced by Robot .
RobotCapabilityType	VISUAL , RADIATION_SURVEY , THERMAL , SAMPLE_PICKUP , VALVE_MANIPULATION	Referenced by RobotCapability .
PayloadType	CAMERA , DOSIMETER , THERMAL_SENSOR , MANIPULATOR , SAMPLE_CANISTER	Referenced by RobotPayload .
MissionType	INSPECTION , RADIATION_SURVEY , PATROL , DELIVERY , RECOVERY , DRILL	Referenced by Mission and checklist templates.
MissionStatus	PLANNED , APPROVED , ACTIVE , PAUSED , COMPLETED , ABORTED	Referenced by Mission .
MissionPriority		Referenced by Mission .

LOOKUP TABLE	TYPICAL SEED CODES	MAIN CONSUMER
	LOW , NORMAL , HIGH , URGENT , EMERGENCY	
MissionTaskStatus	PENDING , ACTIVE , COMPLETED , SKIPPED , FAILED	Referenced by MissionTask .
RouteType	PREDEFINED , OPERATOR_DRAWN , AUTONOMOUS , RETURN_TO_BASE	Referenced by MissionRoute .
WaypointType	START , TRANSIT , INSPECT , SAMPLE , HOLD , CHARGE , EXIT	Referenced by MissionRouteWaypoint .
BatteryStatus	NOMINAL , LOW , CRITICAL , CHARGING , UNKNOWN	Referenced by RobotHealthSnapshot .
CommunicationStatus	NOMINAL , DEGRADED , LOST , LOCAL_ONLY , SIMULATED	Referenced by links and health snapshots.
ReadinessStatus	READY , CONDITIONAL , BLOCKED , EXPIRED , UNKNOWN	Referenced by readiness assessments.
DockingStationStatus	AVAILABLE , OCCUPIED , CHARGING , FAULTED , ISOLATED	Referenced by DockingStation .
TelemetryQuality	GOOD , ESTIMATED , STALE , LOST , SIMULATED	Referenced by RobotTelemetry .

C.12.4 SQL DDL

C.12.4.1 Schema

```
IF SCHEMA_ID(N'Robotics') IS NULL
EXEC(N'CREATE SCHEMA Robotics');
GO
```

C.12.4.2 Robotics lookup tables

```
CREATE TABLE Robotics.RobotType (
    RobotTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Robotics_RobotType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Robotics_RobotType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_RobotType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_RobotType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Robotics_RobotType
    PRIMARY KEY (RobotTypeId),
    CONSTRAINT UQ_Robotics_RobotType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Robotics.RobotStatus (
    RobotStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Robotics_RobotStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Robotics_RobotStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_RobotStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_RobotStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Robotics_RobotStatus
    PRIMARY KEY (RobotStatusId),
    CONSTRAINT UQ_Robotics_RobotStatus_Code
    UNIQUE (Code)
);
GO
```



```

CREATE TABLE Robotics.RobotCapabilityType (
    RobotCapabilityTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Robotics_RobotCapabilityType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Robotics_RobotCapabilityType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_RobotCapabilityType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_RobotCapabilityType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Robotics_RobotCapabilityType
    PRIMARY KEY (RobotCapabilityTypeId),
    CONSTRAINT UQ_Robotics_RobotCapabilityType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Robotics.PayloadType (
    PayloadTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Robotics_PayloadType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Robotics_PayloadType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_PayloadType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_PayloadType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Robotics_PayloadType
    PRIMARY KEY (PayloadTypeId),
    CONSTRAINT UQ_Robotics_PayloadType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Robotics.MissionType (
    MissionTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Robotics_MissionType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Robotics_MissionType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_MissionType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_MissionType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Robotics_MissionType
    PRIMARY KEY (MissionTypeId),
    CONSTRAINT UQ_Robotics_MissionType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Robotics.MissionStatus (
    MissionStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Robotics_MissionStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Robotics_MissionStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_MissionStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_MissionStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Robotics_MissionStatus
    PRIMARY KEY (MissionStatusId),
    CONSTRAINT UQ_Robotics_MissionStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Robotics.MissionPriority (
    MissionPriorityId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Robotics_MissionPriority_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Robotics_MissionPriority_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_MissionPriority_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_MissionPriority_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Robotics_MissionPriority
    PRIMARY KEY (MissionPriorityId),
    CONSTRAINT UQ_Robotics_MissionPriority_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Robotics.MissionTaskStatus (
    MissionTaskStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Robotics_MissionTaskStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Robotics_MissionTaskStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_MissionTaskStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_MissionTaskStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Robotics_MissionTaskStatus
    PRIMARY KEY (MissionTaskStatusId),
    CONSTRAINT UQ_Robotics_MissionTaskStatus_Code
    UNIQUE (Code)
);
GO

```

```

CREATE TABLE Robotics.RouteType (
    RouteTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Robotics_RouteType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Robotics_RouteType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_RouteType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_RouteType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Robotics_RouteType
        PRIMARY KEY (RouteTypeId),
    CONSTRAINT UQ_Robotics_RouteType_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Robotics.WaypointType (
    WaypointTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Robotics_WaypointType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Robotics_WaypointType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_WaypointType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_WaypointType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Robotics_WaypointType
        PRIMARY KEY (WaypointTypeId),
    CONSTRAINT UQ_Robotics_WaypointType_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Robotics.BatteryStatus (
    BatteryStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Robotics_BatteryStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Robotics_BatteryStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_BatteryStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_BatteryStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Robotics_BatteryStatus
        PRIMARY KEY (BatteryStatusId),
    CONSTRAINT UQ_Robotics_BatteryStatus_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Robotics.CommunicationStatus (
    CommunicationStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Robotics_CommunicationStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Robotics_CommunicationStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_CommunicationStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_CommunicationStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Robotics_CommunicationStatus
        PRIMARY KEY (CommunicationStatusId),
    CONSTRAINT UQ_Robotics_CommunicationStatus_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Robotics.ReadinessStatus (
    ReadinessStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Robotics_ReadinessStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Robotics_ReadinessStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_ReadinessStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_ReadinessStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Robotics_ReadinessStatus
        PRIMARY KEY (ReadinessStatusId),
    CONSTRAINT UQ_Robotics_ReadinessStatus_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Robotics.DockingStationStatus (
    DockingStationStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Robotics_DockingStationStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Robotics_DockingStationStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_DockingStationStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_DockingStationStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Robotics_DockingStationStatus
        PRIMARY KEY (DockingStationStatusId),
    CONSTRAINT UQ_Robotics_DockingStationStatus_Code
        UNIQUE (Code)
);
GO

```

```

CREATE TABLE Robotics.TelemetryQuality (
    TelemetryQualityId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Robotics_TelemetryQuality_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Robotics_TelemetryQuality_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_TelemetryQuality_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_TelemetryQuality_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Robotics_TelemetryQuality
    PRIMARY KEY (TelemetryQualityId),
    CONSTRAINT UQ_Robotics_TelemetryQuality_Code
    UNIQUE (Code)
);
GO

```

C.12.4.3 Robot fleet, capabilities, payloads and work envelopes

```

CREATE TABLE Robotics.RobotModel (
    RobotModelId INT IDENTITY(1,1) NOT NULL,
    RobotTypeId INT NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Manufacturer NVARCHAR(150) NOT NULL,
    ModelName NVARCHAR(150) NOT NULL,
    Description NVARCHAR(1000) NULL,
    MaxPayloadKg DECIMAL(10,3) NULL,
    MaxSpeedMps DECIMAL(10,3) NULL,
    BatteryCapacityWh DECIMAL(12,2) NULL,
    NominalRuntimeMin INT NOT NULL,
    IsAutonomousCapable BIT NOT NULL
    CONSTRAINT DF_Robotics_RobotModel_IsAutonomousCapable DEFAULT 0,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_RobotModel_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_RobotModel_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL
    CONSTRAINT DF_Robotics_RobotModel_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Robotics_RobotModel PRIMARY KEY (RobotModelId),
    CONSTRAINT UQ_Robotics_RobotModel_Code UNIQUE (Code),
    CONSTRAINT FK_Robotics_RobotModel_RobotType FOREIGN KEY (RobotTypeId) REFERENCES
    Robotics.RobotType(RobotTypeId),
    CONSTRAINT CK_Robotics_RobotModel_MaxPayload CHECK (MaxPayloadKg IS NULL OR MaxPayloadKg >= 0),
    CONSTRAINT CK_Robotics_RobotModel_MaxSpeed CHECK (MaxSpeedMps IS NULL OR MaxSpeedMps >= 0)
);
GO

CREATE TABLE Robotics.Robot (
    RobotId INT IDENTITY(1,1) NOT NULL,
    RobotModelId INT NOT NULL,
    RobotStatusId INT NOT NULL,
    HomeUnitId INT NOT NULL,
    HomeDockingStationId INT NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    SerialNumber NVARCHAR(120) NULL,
    CommissionedAtUtc DATETIME2(7) NULL,
    RetiredAtUtc DATETIME2(7) NULL,
    LastSeenAtUtc DATETIME2(7) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_Robot_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_Robot_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL
    CONSTRAINT DF_Robotics_Robot_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Robotics_Robot PRIMARY KEY (RobotId),
    CONSTRAINT UQ_Robotics_Robot_Code UNIQUE (Code),
    CONSTRAINT FK_Robotics_Robot_RobotModel FOREIGN KEY (RobotModelId) REFERENCES
    Robotics.RobotModel(RobotModelId),
    CONSTRAINT FK_Robotics_Robot_RobotStatus FOREIGN KEY (RobotStatusId) REFERENCES
    Robotics.RobotStatus(RobotStatusId),
    CONSTRAINT FK_Robotics_Robot_HomeUnit FOREIGN KEY (HomeUnitId) REFERENCES ReactorFleet.Unit(UnitId)
);
GO

CREATE TABLE Robotics.RobotCapability (
    RobotCapabilityId INT IDENTITY(1,1) NOT NULL,
    RobotCapabilityTypeId INT NOT NULL,
    EngineeringUnitId INT NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    Description NVARCHAR(1000) NULL,
    NominalValue DECIMAL(18,6) NULL,
    MinimumValue DECIMAL(18,6) NULL,
    MaximumValue DECIMAL(18,6) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_RobotCapability_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Robotics_RobotCapability_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL
    CONSTRAINT DF_Robotics_RobotCapability_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Robotics_RobotCapability PRIMARY KEY (RobotCapabilityId),
    CONSTRAINT UQ_Robotics_RobotCapability_Code UNIQUE (Code),
    CONSTRAINT FK_Robotics_RobotCapability_Type FOREIGN KEY (RobotCapabilityTypeId) REFERENCES
    Robotics.RobotCapabilityType(RobotCapabilityTypeId),
    CONSTRAINT FK_Robotics_RobotCapability_EngineeringUnit FOREIGN KEY (EngineeringUnitId) REFERENCES
    CorePlatform.EngineeringUnit(EngineeringUnitId)
);
GO

CREATE TABLE Robotics.RobotCapabilityAssignment (
    RobotCapabilityAssignmentId INT IDENTITY(1,1) NOT NULL,
    RobotId INT NOT NULL,
    RobotCapabilityId INT NOT NULL,
    RatingValue DECIMAL(18,6) NULL,
    ValidatedAtUtc DATETIME2(7) NULL,
    ValidatedByUserId INT NOT NULL,
    Notes NVARCHAR(1000) NULL,

    CONSTRAINT PK_Robotics_RobotCapabilityAssignment PRIMARY KEY (RobotCapabilityAssignmentId),
    CONSTRAINT UQ_Robotics_RobotCapabilityAssignment UNIQUE (RobotId, RobotCapabilityId),
    CONSTRAINT FK_Robotics_RobotCapabilityAssignment_Robot FOREIGN KEY (RobotId) REFERENCES
    Robotics.Robot(RobotId),
    CONSTRAINT FK_Robotics_RobotCapabilityAssignment_Capability FOREIGN KEY (RobotCapabilityId) REFERENCES
    Robotics.RobotCapability(RobotCapabilityId),
    CONSTRAINT FK_Robotics_RobotCapabilityAssignment_ValidatedBy FOREIGN KEY (ValidatedByUserId) REFERENCES
    Security.ApplicationUser(ApplicationUserId)
);
GO

```

```

);
GO

CREATE TABLE Robotics.RobotPayload (
    RobotPayloadId INT IDENTITY(1,1) NOT NULL,
    RobotId INT NOT NULL,
    PayloadTypeId INT NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    SerialNumber NVARCHAR(120) NULL,
    InstalledAtUtc DATETIME2(7) NULL,
    RemovedAtUtc DATETIME2(7) NULL,
    WeightKg DECIMAL(10,3) NULL,
    IsPrimaryPayload BIT NOT NULL CONSTRAINT DF_Robotics_RobotPayload_IsPrimaryPayload DEFAULT 0,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Robotics_RobotPayload_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Robotics_RobotPayload_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Robotics_RobotPayload_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Robotics_RobotPayload PRIMARY KEY (RobotPayloadId),
    CONSTRAINT UQ_Robotics_RobotPayload Code UNIQUE (Code),
    CONSTRAINT FK_Robotics_RobotPayload_Robot FOREIGN KEY (RobotId) REFERENCES Robotics.Robot(RobotId),
    CONSTRAINT FK_Robotics_RobotPayload_PayloadType FOREIGN KEY (PayloadTypeId) REFERENCES Robotics.PayloadType(PayloadTypeId),
    CONSTRAINT CK_Robotics_RobotPayload_Weight CHECK (WeightKg IS NULL OR WeightKg >= 0)
);
GO

CREATE TABLE Robotics.RobotSensorBinding (
    RobotSensorBindingId INT IDENTITY(1,1) NOT NULL,
    RobotId INT NOT NULL,
    SignalId INT NOT NULL,
    PayloadId INT NOT NULL,
    LocalChannelName NVARCHAR(120) NULL,
    IsPrimary BIT NOT NULL CONSTRAINT DF_Robotics_RobotSensorBinding_IsPrimary DEFAULT 0,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Robotics_RobotSensorBinding_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Robotics_RobotSensorBinding_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Robotics_RobotSensorBinding_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Robotics_RobotSensorBinding PRIMARY KEY (RobotSensorBindingId),
    CONSTRAINT UQ_Robotics_RobotSensorBinding UNIQUE (RobotId, SignalId),
    CONSTRAINT FK_Robotics_RobotSensorBinding_Robot FOREIGN KEY (RobotId) REFERENCES Robotics.Robot(RobotId),
    CONSTRAINT FK_Robotics_RobotSensorBinding_Signal FOREIGN KEY (SignalId) REFERENCES Instrumentation.Signal(SignalId),
    CONSTRAINT FK_Robotics_RobotSensorBinding_Payload FOREIGN KEY (PayloadId) REFERENCES Robotics.RobotPayload(RobotPayloadId)
);
GO

CREATE TABLE Robotics.RobotCommunicationLink (
    RobotCommunicationLinkId INT IDENTITY(1,1) NOT NULL,
    RobotId INT NOT NULL,
    CommunicationStatusId INT NOT NULL,
    LinkName NVARCHAR(120) NOT NULL,
    NetworkAddress NVARCHAR(200) NULL,
    SignalStrengthPercent DECIMAL(5,2) NULL,
    LastHeartbeatAtUtc DATETIME2(7) NULL,
    Notes NVARCHAR(1000) NULL,

    CONSTRAINT PK_Robotics_RobotCommunicationLink PRIMARY KEY (RobotCommunicationLinkId),
    CONSTRAINT FK_Robotics_RobotCommunicationLink_Robot FOREIGN KEY (RobotId) REFERENCES Robotics.Robot(RobotId),
    CONSTRAINT FK_Robotics_RobotCommunicationLink_Status FOREIGN KEY (CommunicationStatusId) REFERENCES Robotics.CommunicationStatus(CommunicationStatusId),
    CONSTRAINT CK_Robotics_RobotCommunicationLink_Signal CHECK (SignalStrengthPercent IS NULL OR SignalStrengthPercent BETWEEN 0 AND 100)
);
GO

CREATE TABLE Robotics.RobotWorkEnvelope (
    RobotWorkEnvelopeId INT IDENTITY(1,1) NOT NULL,
    RobotId INT NOT NULL,
    UnitId INT NOT NULL,
    EquipmentLocationId INT NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    BoundaryJson NVARCHAR(MAX) NULL,
    IsActive BIT NOT NULL CONSTRAINT DF_Robotics_RobotWorkEnvelope_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Robotics_RobotWorkEnvelope_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Robotics_RobotWorkEnvelope_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Robotics_RobotWorkEnvelope PRIMARY KEY (RobotWorkEnvelopeId),
    CONSTRAINT FK_Robotics_RobotWorkEnvelope_Robot FOREIGN KEY (RobotId) REFERENCES Robotics.Robot(RobotId),
    CONSTRAINT FK_Robotics_RobotWorkEnvelope_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_Robotics_RobotWorkEnvelope_EquipmentLocation FOREIGN KEY (EquipmentLocationId) REFERENCES ReactorFleet.EquipmentLocation(EquipmentLocationId)
);
GO

```

C.12.4.4 Robot telemetry, location history and health snapshots

```

CREATE TABLE Robotics.RobotTelemetry (
    RobotTelemetryId BIGINT IDENTITY(1,1) NOT NULL,
    RobotId INT NOT NULL,
    EngineeringUnitId INT NOT NULL,
    TelemetryQualityId INT NOT NULL,
    TimestampUtc DATETIME2(7) NOT NULL,
    MetricCode NVARCHAR(60) NOT NULL,
    NumericValue FLOAT NULL,
    TextValue NVARCHAR(500) NULL,
    SourceSequence BIGINT NULL,
    IsSimulated BIT NOT NULL CONSTRAINT DF_Robotics_RobotTelemetry_IsSimulated DEFAULT 1,
    IngestedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Robotics_RobotTelemetry_IngestedAtUtc DEFAULT SYSUTCDATETIME(),

    CONSTRAINT PK_Robotics_RobotTelemetry PRIMARY KEY (RobotTelemetryId),
    CONSTRAINT FK_Robotics_RobotTelemetry_Robot FOREIGN KEY (RobotId) REFERENCES Robotics.Robot(RobotId),
    CONSTRAINT FK_Robotics_RobotTelemetry_EngineeringUnit FOREIGN KEY (EngineeringUnitId) REFERENCES CorePlatform.EngineeringUnit(EngineeringUnitId),
    CONSTRAINT FK_Robotics_RobotTelemetry_Quality FOREIGN KEY (TelemetryQualityId) REFERENCES Robotics.TelemetryQuality(TelemetryQualityId),
    CONSTRAINT CK_Robotics_RobotTelemetry_Value CHECK (NumericValue IS NOT NULL OR TextValue IS NOT NULL)
);
GO

CREATE TABLE Robotics.RobotLocationHistory (
    RobotLocationHistoryId BIGINT IDENTITY(1,1) NOT NULL,

```

```

RobotId          INT NOT NULL,
UnitId           INT NULL,
EquipmentLocationId INT NULL,
TimestampUtc     DATETIME2(7) NOT NULL,
PositionX        DECIMAL(18,6) NULL,
PositionY        DECIMAL(18,6) NULL,
PositionZ        DECIMAL(18,6) NULL,
HeadingDegrees   DECIMAL(9,4) NULL,
CoordinateFrame  NVARCHAR(50) NOT NULL CONSTRAINT DF_Robotics_RobotLocationHistory_Frame DEFAULT
N'LOCAL',
IsEstimated      BIT NOT NULL CONSTRAINT DF_Robotics_RobotLocationHistory_IsEstimated DEFAULT 0,

CONSTRAINT PK_Robotics_RobotLocationHistory PRIMARY KEY (RobotLocationHistoryId),
CONSTRAINT FK_Robotics_RobotLocationHistory_Robot FOREIGN KEY (RobotId) REFERENCES Robotics.Robot(RobotId),
CONSTRAINT FK_Robotics_RobotLocationHistory_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
CONSTRAINT FK_Robotics_RobotLocationHistory_EquipmentLocation FOREIGN KEY (EquipmentLocationId) REFERENCES
ReactorFleet.EquipmentLocation(EquipmentLocationId),
CONSTRAINT CK_Robotics_RobotLocationHistory_Heading CHECK (HeadingDegrees IS NULL OR HeadingDegrees BETWEEN
0 AND 360)
);
GO

CREATE TABLE Robotics.RobotHealthSnapshot (
RobotHealthSnapshotId BIGINT IDENTITY(1,1) NOT NULL,
RobotId               INT NOT NULL,
BatteryStatusId       INT NOT NULL,
CommunicationStatusId INT NOT NULL,
SnapshotAtUtc         DATETIME2(7) NOT NULL,
BatteryPercent         DECIMAL(5,2) NULL,
EstimatedRuntimeMin   INT NULL,
CpuLoadPercent        DECIMAL(5,2) NULL,
FaultCount            INT NOT NULL CONSTRAINT DF_Robotics_RobotHealthSnapshot_FaultCount DEFAULT 0,
Summary               NVARCHAR(1000) NULL,

CONSTRAINT PK_Robotics_RobotHealthSnapshot PRIMARY KEY (RobotHealthSnapshotId),
CONSTRAINT FK_Robotics_RobotHealthSnapshot_Robot FOREIGN KEY (RobotId) REFERENCES Robotics.Robot(RobotId),
CONSTRAINT FK_Robotics_RobotHealthSnapshot_BatteryStatus FOREIGN KEY (BatteryStatusId) REFERENCES
Robotics.BatteryStatus(BatteryStatusId),
CONSTRAINT FK_Robotics_RobotHealthSnapshot_CommStatus FOREIGN KEY (CommunicationStatusId) REFERENCES
Robotics.CommunicationStatus(CommunicationStatusId),
CONSTRAINT CK_Robotics_RobotHealthSnapshot_Battery CHECK (BatteryPercent IS NULL OR BatteryPercent BETWEEN 0
AND 100),
CONSTRAINT CK_Robotics_RobotHealthSnapshot_Cpu CHECK (CpuLoadPercent IS NULL OR CpuLoadPercent BETWEEN 0 AND
100)
);
GO

```

C.12.4.5 Docking stations and charging sessions

```

CREATE TABLE Robotics.DockingStation (
DockingStationId INT IDENTITY(1,1) NOT NULL,
UnitId           INT NOT NULL,
EquipmentLocationId INT NULL,
DockingStationStatusId INT NOT NULL,
Code             NVARCHAR(80) NOT NULL,
Name            NVARCHAR(200) NOT NULL,
MaxConcurrentRobots INT NOT NULL CONSTRAINT DF_Robotics_DockingStation_MaxConcurrentRobots DEFAULT 1,
MaxChargePowerKw DECIMAL(10,3) NULL,
CreatedAtUtc     DATETIME2(7) NOT NULL CONSTRAINT DF_Robotics_DockingStation_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
CreatedBy        NVARCHAR(100) NOT NULL,
ModifiedAtUtc    DATETIME2(7) NOT NULL CONSTRAINT DF_Robotics_DockingStation_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
ModifiedBy       NVARCHAR(100) NOT NULL,
IsDeleted        BIT NOT NULL CONSTRAINT DF_Robotics_DockingStation_IsDeleted DEFAULT 0,
RowVersion       ROWVERSION NOT NULL,

CONSTRAINT PK_Robotics_DockingStation PRIMARY KEY (DockingStationId),
CONSTRAINT UQ_Robotics_DockingStation_Code UNIQUE (Code),
CONSTRAINT FK_Robotics_DockingStation_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
CONSTRAINT FK_Robotics_DockingStation_EquipmentLocation FOREIGN KEY (EquipmentLocationId) REFERENCES
ReactorFleet.EquipmentLocation(EquipmentLocationId),
CONSTRAINT FK_Robotics_DockingStation_Status FOREIGN KEY (DockingStationStatusId) REFERENCES
Robotics.DockingStationStatus(DockingStationStatusId),
CONSTRAINT CK_Robotics_DockingStation_MaxRobots CHECK (MaxConcurrentRobots > 0)
);
GO

ALTER TABLE Robotics.Robot
ADD CONSTRAINT FK_Robotics_Robot_HomeDockingStation
FOREIGN KEY (HomeDockingStationId) REFERENCES Robotics.DockingStation(DockingStationId);
GO

CREATE TABLE Robotics.ChargingSession (
ChargingSessionId BIGINT IDENTITY(1,1) NOT NULL,
RobotId           INT NOT NULL,
DockingStationId INT NOT NULL,
StartedAtUtc      DATETIME2(7) NOT NULL,
EndedAtUtc        DATETIME2(7) NULL,
StartBatteryPercent DECIMAL(5,2) NULL,
EndBatteryPercent DECIMAL(5,2) NULL,
EnergyDeliveredKwh DECIMAL(12,4) NULL,
StartedByUserId   INT NULL,

CONSTRAINT PK_Robotics_ChargingSession PRIMARY KEY (ChargingSessionId),
CONSTRAINT FK_Robotics_ChargingSession_Robot FOREIGN KEY (RobotId) REFERENCES Robotics.Robot(RobotId),
CONSTRAINT FK_Robotics_ChargingSession_DockingStation FOREIGN KEY (DockingStationId) REFERENCES
Robotics.DockingStation(DockingStationId),
CONSTRAINT FK_Robotics_ChargingSession_StartedBy FOREIGN KEY (StartedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId),
CONSTRAINT CK_Robotics_ChargingSession_BatteryStart CHECK (StartBatteryPercent IS NULL OR
StartBatteryPercent BETWEEN 0 AND 100),
CONSTRAINT CK_Robotics_ChargingSession_BatteryEnd CHECK (EndBatteryPercent IS NULL OR EndBatteryPercent
BETWEEN 0 AND 100),
CONSTRAINT CK_Robotics_ChargingSession_Time CHECK (EndedAtUtc IS NULL OR EndedAtUtc >= StartedAtUtc)
);
GO

```

C.12.4.6 Missions, assignments, tasks, routes, waypoints and events

```

CREATE TABLE Robotics.Mission (
MissionId          BIGINT IDENTITY(1,1) NOT NULL,
UnitId             INT NOT NULL,
MissionTypeId      INT NOT NULL,
MissionStatusId    INT NOT NULL,
MissionPriorityId  INT NOT NULL,
RequestedByUserId  INT NULL,
ApprovedByUserId   INT NULL,
Code              NVARCHAR(80) NOT NULL,
Title              NVARCHAR(250) NOT NULL,
Objective          NVARCHAR(2000) NULL,

```

```

RequestedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Robotics_Mission_RequestedAtUtc DEFAULT
SYSUTCDATETIME(),
PlannedStartUtc DATETIME2(7) NULL,
PlannedEndUtc DATETIME2(7) NULL,
ActualStartUtc DATETIME2(7) NULL,
ActualEndUtc DATETIME2(7) NULL,
CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Robotics_Mission_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
CreatedBy NVARCHAR(100) NOT NULL,
ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Robotics_Mission_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
ModifiedBy NVARCHAR(100) NOT NULL,
IsDeleted BIT NOT NULL CONSTRAINT DF_Robotics_Mission_IsDeleted DEFAULT 0,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_Robotics_Mission PRIMARY KEY (MissionId),
CONSTRAINT UQ_Robotics_Mission_Code UNIQUE (Code),
CONSTRAINT FK_Robotics_Mission_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
CONSTRAINT FK_Robotics_Mission_Type FOREIGN KEY (MissionTypeId) REFERENCES
Robotics.MissionType(MissionTypeId),
CONSTRAINT FK_Robotics_Mission_Status FOREIGN KEY (MissionStatusId) REFERENCES
Robotics.MissionStatus(MissionStatusId),
CONSTRAINT FK_Robotics_Mission_Priority FOREIGN KEY (MissionPriorityId) REFERENCES
Robotics.MissionPriority(MissionPriorityId),
CONSTRAINT FK_Robotics_Mission_RequestedBy FOREIGN KEY (RequestedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId),
CONSTRAINT FK_Robotics_Mission_ApprovedBy FOREIGN KEY (ApprovedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId),
CONSTRAINT CK_Robotics_Mission_PlannedTime CHECK (PlannedEndUtc IS NULL OR PlannedStartUtc IS NULL OR
PlannedEndUtc >= PlannedStartUtc),
CONSTRAINT CK_Robotics_Mission_ActualTime CHECK (ActualEndUtc IS NULL OR ActualStartUtc IS NULL OR
ActualEndUtc >= ActualStartUtc)
);
GO

CREATE TABLE Robotics.MissionRobotAssignment (
MissionRobotAssignmentId BIGINT IDENTITY(1,1) NOT NULL,
MissionId BIGINT NOT NULL,
RobotId INT NOT NULL,
AssignmentRole NVARCHAR(100) NOT NULL,
AssignedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Robotics_MissionRobotAssignment_AssignedAtUtc
DEFAULT SYSUTCDATETIME(),
AssignedByUserId INT NULL,
PlannedStartUtc DATETIME2(7) NULL,
PlannedEndUtc DATETIME2(7) NULL,

CONSTRAINT PK_Robotics_MissionRobotAssignment PRIMARY KEY (MissionRobotAssignmentId),
CONSTRAINT UQ_Robotics_MissionRobotAssignment UNIQUE (MissionId, RobotId),
CONSTRAINT FK_Robotics_MissionRobotAssignment_Mission FOREIGN KEY (MissionId) REFERENCES
Robotics.Mission(MissionId),
CONSTRAINT FK_Robotics_MissionRobotAssignment_Robot FOREIGN KEY (RobotId) REFERENCES
Robotics.Robot(RobotId),
CONSTRAINT FK_Robotics_MissionRobotAssignment_AssignedBy FOREIGN KEY (AssignedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE Robotics.MissionTask (
MissionTaskId BIGINT IDENTITY(1,1) NOT NULL,
MissionId BIGINT NOT NULL,
RobotId INT NULL,
MissionTaskStatusId INT NOT NULL,
TaskOrder INT NOT NULL,
Title NVARCHAR(250) NOT NULL,
Instruction NVARCHAR(2000) NULL,
TargetEquipmentId INT NULL,
StartedAtUtc DATETIME2(7) NULL,
CompletedAtUtc DATETIME2(7) NULL,

CONSTRAINT PK_Robotics_MissionTask PRIMARY KEY (MissionTaskId),
CONSTRAINT UQ_Robotics_MissionTask_Order UNIQUE (MissionId, TaskOrder),
CONSTRAINT FK_Robotics_MissionTask_Mission FOREIGN KEY (MissionId) REFERENCES Robotics.Mission(MissionId),
CONSTRAINT FK_Robotics_MissionTask_Robot FOREIGN KEY (RobotId) REFERENCES Robotics.Robot(RobotId),
CONSTRAINT FK_Robotics_MissionTask_Status FOREIGN KEY (MissionTaskStatusId) REFERENCES
Robotics.MissionTaskStatus(MissionTaskStatusId),
CONSTRAINT FK_Robotics_MissionTask_TargetEquipment FOREIGN KEY (TargetEquipmentId) REFERENCES
ReactorFleet.Equipment(EquipmentId),
CONSTRAINT CK_Robotics_MissionTask_Order CHECK (TaskOrder > 0)
);
GO

CREATE TABLE Robotics.MissionRoute (
MissionRouteId BIGINT IDENTITY(1,1) NOT NULL,
MissionId BIGINT NOT NULL,
RouteTypeId INT NOT NULL,
Name NVARCHAR(200) NOT NULL,
RouteJson NVARCHAR(MAX) NULL,
IsApproved BIT NOT NULL CONSTRAINT DF_Robotics_MissionRoute_IsApproved DEFAULT 0,
ApprovedByUserId INT NULL,
ApprovedAtUtc DATETIME2(7) NULL,

CONSTRAINT PK_Robotics_MissionRoute PRIMARY KEY (MissionRouteId),
CONSTRAINT FK_Robotics_MissionRoute_Mission FOREIGN KEY (MissionId) REFERENCES Robotics.Mission(MissionId),
CONSTRAINT FK_Robotics_MissionRoute_RouteType FOREIGN KEY (RouteTypeId) REFERENCES
Robotics.RouteType(RouteTypeId),
CONSTRAINT FK_Robotics_MissionRoute_ApprovedBy FOREIGN KEY (ApprovedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE Robotics.MissionRouteWaypoint (
MissionRouteWaypointId BIGINT IDENTITY(1,1) NOT NULL,
MissionRouteId BIGINT NOT NULL,
WaypointTypeId INT NOT NULL,
EquipmentLocationId INT NULL,
SequenceNo INT NOT NULL,
Label NVARCHAR(150) NOT NULL,
PositionX DECIMAL(18,6) NULL,
PositionY DECIMAL(18,6) NULL,
PositionZ DECIMAL(18,6) NULL,
HoldSeconds INT NULL,

CONSTRAINT PK_Robotics_MissionRouteWaypoint PRIMARY KEY (MissionRouteWaypointId),
CONSTRAINT UQ_Robotics_MissionRouteWaypoint_Seq UNIQUE (MissionRouteId, SequenceNo),
CONSTRAINT FK_Robotics_MissionRouteWaypoint_Route FOREIGN KEY (MissionRouteId) REFERENCES
Robotics.MissionRoute(MissionRouteId),
CONSTRAINT FK_Robotics_MissionRouteWaypoint_Type FOREIGN KEY (WaypointTypeId) REFERENCES
Robotics.WaypointType(WaypointTypeId),
CONSTRAINT FK_Robotics_MissionRouteWaypoint_Location FOREIGN KEY (EquipmentLocationId) REFERENCES
ReactorFleet.EquipmentLocation(EquipmentLocationId),
CONSTRAINT CK_Robotics_MissionRouteWaypoint_Seq CHECK (SequenceNo > 0),
CONSTRAINT CK_Robotics_MissionRouteWaypoint_Hold CHECK (HoldSeconds IS NULL OR HoldSeconds >= 0)
);
GO

CREATE TABLE Robotics.MissionPayloadUsage (
MissionPayloadUsageId BIGINT IDENTITY(1,1) NOT NULL,
MissionId BIGINT NOT NULL,
RobotPayloadId INT NOT NULL,
Purpose NVARCHAR(250) NOT NULL,
UsedFromUtc DATETIME2(7) NULL,
UsedToUtc DATETIME2(7) NULL,

CONSTRAINT PK_Robotics_MissionPayloadUsage PRIMARY KEY (MissionPayloadUsageId),
CONSTRAINT FK_Robotics_MissionPayloadUsage_Mission FOREIGN KEY (MissionId) REFERENCES
Robotics.Mission(MissionId),
CONSTRAINT FK_Robotics_MissionPayloadUsage_Payload FOREIGN KEY (RobotPayloadId) REFERENCES
Robotics.RobotPayload(RobotPayloadId)
);

```



```
GO

CREATE TABLE Robotics.MissionEvent (
    MissionEventId BIGINT IDENTITY(1,1) NOT NULL,
    MissionId BIGINT NOT NULL,
    RobotId INT NULL,
    MissionTaskId BIGINT NULL,
    OccurredAtUtc DATETIME2(7) NOT NULL,
    EventCode NVARCHAR(80) NOT NULL,
    Title NVARCHAR(250) NOT NULL,
    Detail NVARCHAR(2000) NULL,
    IsFault BIT NOT NULL CONSTRAINT DF_Robotics_MissionEvent_IsFault DEFAULT 0,
    RecordedByUserId INT NULL,

    CONSTRAINT PK_Robotics_MissionEvent PRIMARY KEY (MissionEventId),
    CONSTRAINT FK_Robotics_MissionEvent_Mission FOREIGN KEY (MissionId) REFERENCES Robotics.Mission(MissionId),
    CONSTRAINT FK_Robotics_MissionEvent_Robot FOREIGN KEY (RobotId) REFERENCES Robotics.Robot(RobotId),
    CONSTRAINT FK_Robotics_MissionEvent_Task FOREIGN KEY (MissionTaskId) REFERENCES
        Robotics.MissionTask(MissionTaskId),
    CONSTRAINT FK_Robotics_MissionEvent_RecordedBy FOREIGN KEY (RecordedByUserId) REFERENCES
        Security.ApplicationUser(ApplicationUserId)
);
GO
```

C.12.4.7 Readiness gates and integration links

```
CREATE TABLE Robotics.MissionChecklist (
    MissionChecklistId INT IDENTITY(1,1) NOT NULL,
    MissionTypeId INT NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    Description NVARCHAR(1000) NULL,
    IsActive BIT NOT NULL CONSTRAINT DF_Robotics_MissionChecklist_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Robotics_MissionChecklist_CreatedAtUtc DEFAULT
        SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Robotics_MissionChecklist_ModifiedAtUtc DEFAULT
        SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Robotics_MissionChecklist PRIMARY KEY (MissionChecklistId),
    CONSTRAINT UQ_Robotics_MissionChecklist_Code UNIQUE (Code),
    CONSTRAINT FK_Robotics_MissionChecklist_MissionType FOREIGN KEY (MissionTypeId) REFERENCES
        Robotics.MissionType(MissionTypeId)
);
GO

CREATE TABLE Robotics.MissionChecklistItem (
    MissionChecklistItemId INT IDENTITY(1,1) NOT NULL,
    MissionChecklistId INT NOT NULL,
    SequenceNo INT NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Text NVARCHAR(500) NOT NULL,
    RequiredCapabilityId INT NULL,
    MinimumBatteryPercent DECIMAL(5,2) NULL,
    IsBlocking BIT NOT NULL CONSTRAINT DF_Robotics_MissionChecklistItem_IsBlocking DEFAULT 1,

    CONSTRAINT PK_Robotics_MissionChecklistItem PRIMARY KEY (MissionChecklistItemId),
    CONSTRAINT UQ_Robotics_MissionChecklistItem_Seq UNIQUE (MissionChecklistId, SequenceNo),
    CONSTRAINT FK_Robotics_MissionChecklistItem_Checklist FOREIGN KEY (MissionChecklistId) REFERENCES
        Robotics.MissionChecklist(MissionChecklistId),
    CONSTRAINT FK_Robotics_MissionChecklistItem_Capability FOREIGN KEY (RequiredCapabilityId) REFERENCES
        Robotics.RobotCapability(RobotCapabilityId),
    CONSTRAINT CK_Robotics_MissionChecklistItem_Battery CHECK (MinimumBatteryPercent IS NULL OR
        MinimumBatteryPercent BETWEEN 0 AND 100)
);
GO

CREATE TABLE Robotics.MissionReadinessAssessment (
    MissionReadinessAssessmentId BIGINT IDENTITY(1,1) NOT NULL,
    MissionId BIGINT NOT NULL,
    ReadinessStatusId INT NOT NULL,
    AssessedAtUtc DATETIME2(7) NOT NULL CONSTRAINT
        DF_Robotics_MissionReadinessAssessment_AssessedAtUtc DEFAULT SYSUTCDATETIME(),
    AssessedByUserId INT NULL,
    Summary NVARCHAR(1000) NULL,

    CONSTRAINT PK_Robotics_MissionReadinessAssessment PRIMARY KEY (MissionReadinessAssessmentId),
    CONSTRAINT FK_Robotics_MissionReadinessAssessment_Mission FOREIGN KEY (MissionId) REFERENCES
        Robotics.Mission(MissionId),
    CONSTRAINT FK_Robotics_MissionReadinessAssessment_Status FOREIGN KEY (ReadinessStatusId) REFERENCES
        Robotics.ReadinessStatus(ReadinessStatusId),
    CONSTRAINT FK_Robotics_MissionReadinessAssessment_AssessedBy FOREIGN KEY (AssessedByUserId) REFERENCES
        Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE Robotics.MissionReadinessItem (
    MissionReadinessItemId BIGINT IDENTITY(1,1) NOT NULL,
    MissionReadinessAssessmentId BIGINT NOT NULL,
    MissionChecklistItemId INT NULL,
    ReadinessStatusId INT NOT NULL,
    CheckName NVARCHAR(200) NOT NULL,
    Detail NVARCHAR(1000) NULL,
    IsBlocking BIT NOT NULL CONSTRAINT DF_Robotics_MissionReadinessItem_IsBlocking DEFAULT 1,

    CONSTRAINT PK_Robotics_MissionReadinessItem PRIMARY KEY (MissionReadinessItemId),
    CONSTRAINT FK_Robotics_MissionReadinessItem_Assessment FOREIGN KEY (MissionReadinessAssessmentId) REFERENCES
        Robotics.MissionReadinessAssessment(MissionReadinessAssessmentId),
    CONSTRAINT FK_Robotics_MissionReadinessItem_ChecklistItem FOREIGN KEY (MissionChecklistItemId) REFERENCES
        Robotics.MissionChecklistItem(MissionChecklistItemId),
    CONSTRAINT FK_Robotics_MissionReadinessItem_Status FOREIGN KEY (ReadinessStatusId) REFERENCES
        Robotics.ReadinessStatus(ReadinessStatusId)
);
GO

CREATE TABLE Robotics.RobotMaintenanceLink (
    RobotMaintenanceLinkId BIGINT IDENTITY(1,1) NOT NULL,
    RobotId INT NOT NULL,
    WorkOrderId BIGINT NOT NULL,
    LinkedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Robotics_RobotMaintenanceLink_LinkedAtUtc DEFAULT
        SYSUTCDATETIME(),
    LinkedByUserId INT NULL,
    Reason NVARCHAR(500) NULL,

    CONSTRAINT PK_Robotics_RobotMaintenanceLink PRIMARY KEY (RobotMaintenanceLinkId),
    CONSTRAINT UQ_Robotics_RobotMaintenanceLink UNIQUE (RobotId, WorkOrderId),
    CONSTRAINT FK_Robotics_RobotMaintenanceLink_Robot FOREIGN KEY (RobotId) REFERENCES Robotics.Robot(RobotId),
    CONSTRAINT FK_Robotics_RobotMaintenanceLink_WorkOrder FOREIGN KEY (WorkOrderId) REFERENCES
        Maintenance.WorkOrder(WorkOrderId),
    CONSTRAINT FK_Robotics_RobotMaintenanceLink_LinkedBy FOREIGN KEY (LinkedByUserId) REFERENCES
        Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE Robotics.MissionOperationalEventLink (
    MissionOperationalEventLinkId BIGINT IDENTITY(1,1) NOT NULL,
    MissionId BIGINT NOT NULL,
    OperationalEventId BIGINT NOT NULL,
```

```

LinkedAtUtc          DATETIME2(7) NOT NULL CONSTRAINT
DF_Robotics_MissionOperationalEventLink_LinkedAtUtc DEFAULT SYSUTCDATETIME(),
LinkedByUserId        INT NULL,
RelationshipNote       NVARCHAR(500) NULL,

CONSTRAINT PK_Robotics_MissionOperationalEventLink PRIMARY KEY (MissionOperationalEventLinkId),
CONSTRAINT UQ_Robotics_MissionOperationalEventLink UNIQUE (MissionId, OperationalEventId),
CONSTRAINT FK_Robotics_MissionOperationalEventLink_Mission FOREIGN KEY (MissionId) REFERENCES
Robotics.Mission(MissionId),
CONSTRAINT FK_Robotics_MissionOperationalEventLink_Event FOREIGN KEY (OperationalEventId) REFERENCES
EventManagement.OperationalEvent(OperationalEventId),
CONSTRAINT FK_Robotics_MissionOperationalEventLink_LinkedBy FOREIGN KEY (LinkedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId)
);
GO

```

C.12.5 Indexes and seed examples

The most important access paths are robot-by-status, robot telemetry by time, mission-by-unit/status, mission timeline by time, route waypoint order and readiness failures.

```

CREATE INDEX IX_Robotics_Robot_Status ON Robotics.Robot(RobotStatusId, HomeUnitId);
CREATE INDEX IX_Robotics_Robot_Model ON Robotics.Robot(RobotModelId);
CREATE INDEX IX_Robotics_RobotCapabilityAssignment_Robot ON Robotics.RobotCapabilityAssignment(RobotId);
CREATE INDEX IX_Robotics_RobotPayload_Robot ON Robotics.RobotPayload(RobotId);
CREATE INDEX IX_Robotics_RobotSensorBinding_Signal ON Robotics.RobotSensorBinding(SignalId);
CREATE INDEX IX_Robotics_RobotTelemetry_Robot_Time ON Robotics.RobotTelemetry(RobotId, TimestampUtc DESC);
CREATE INDEX IX_Robotics_RobotTelemetry_Metric_Time ON Robotics.RobotTelemetry(MetricCode, TimestampUtc DESC);
CREATE INDEX IX_Robotics_RobotLocationHistory_Robot_Time ON Robotics.RobotLocationHistory(RobotId, TimestampUtc
DESC);
CREATE INDEX IX_Robotics_RobotHealthSnapshot_Robot_Time ON Robotics.RobotHealthSnapshot(RobotId, SnapshotAtUtc
DESC);
CREATE INDEX IX_Robotics_DockingStation_Unit ON Robotics.DockingStation(UnitId, DockingStationStatusId);
CREATE INDEX IX_Robotics_ChargingSession_Robot_Time ON Robotics.ChargingSession(RobotId, StartedAtUtc DESC);
CREATE INDEX IX_Robotics_Mission_Unit_Status ON Robotics.Mission(UnitId, MissionStatusId, RequestedAtUtc DESC);
CREATE INDEX IX_Robotics_MissionRobotAssignment_Robot ON Robotics.MissionRobotAssignment(RobotId, MissionId);
CREATE INDEX IX_Robotics_MissionTask_Mission_Order ON Robotics.MissionTask(MissionId, TaskOrder);
CREATE INDEX IX_Robotics_MissionRouteWaypoint_Route_Seq ON Robotics.MissionRouteWaypoint(MissionRouteId,
SequenceNo);
CREATE INDEX IX_Robotics_MissionEvent_Mission_Time ON Robotics.MissionEvent(MissionId, OccurredAtUtc DESC);
CREATE INDEX IX_Robotics_MissionReadinessAssessment_Mission_Time ON
Robotics.MissionReadinessAssessment(MissionId, AssessedAtUtc DESC);
GO

```

C.12.5.1 Seed examples

```

INSERT INTO Robotics.RobotType (Code, Name, DisplayOrder) VALUES
('CRAWLER', 'Tracked crawler', 10),
('DRONE', 'Aerial drone', 20),
('ROVER', 'Wheeled rover', 30),
('MANIPULATOR', 'Manipulator arm', 40),
('VEHICLE', 'Support vehicle', 50);

INSERT INTO Robotics.RobotStatus (Code, Name, DisplayOrder) VALUES
('AVAILABLE', 'Available', 10),
('CHARGING', 'Charging', 20),
('DEPLOYED', 'Deployed', 30),
('FAULTED', 'Faulted', 40),
('MAINTENANCE', 'In maintenance', 50),
('RETIRED', 'Retired', 60);

INSERT INTO Robotics.MissionStatus (Code, Name, DisplayOrder) VALUES
('PLANNED', 'Planned', 10),
('APPROVED', 'Approved', 20),
('ACTIVE', 'Active', 30),
('PAUSED', 'Paused', 40),
('COMPLETED', 'Completed', 50),
('ABORTED', 'Aborted', 60),
('CANCELLED', 'Cancelled', 70);

INSERT INTO Robotics.ReadinessStatus (Code, Name, DisplayOrder) VALUES
('READY', 'Ready', 10),
('CONDITIONAL', 'Conditional', 20),
('BLOCKED', 'Blocked', 30),
('EXPIRED', 'Expired', 40),
('UNKNOWN', 'Unknown', 50);
GO

```

C.12.5.2 Verification queries

```

-- Robots currently available by unit.
SELECT u.Code AS UnitCode, r.Code AS RobotCode, r.Name, rs.Code AS Status
FROM Robotics.Robot AS r
JOIN Robotics.RobotStatus AS rs ON rs.RobotStatusId = r.RobotStatusId
LEFT JOIN ReactorFleet.Unit AS u ON u.UnitId = r.HomeUnitId
WHERE rs.Code = 'AVAILABLE' AND r.IsDeleted = 0;

-- Latest health snapshot for each robot.
SELECT r.Code, h.SnapshotAtUtc, h.BatteryPercent, bs.Code AS Battery, cs.Code AS Communications
FROM Robotics.Robot AS r
OUTER APPLY (
    SELECT TOP (1) *
    FROM Robotics.RobotHealthSnapshot AS h
    WHERE h.RobotId = r.RobotId
    ORDER BY h.SnapshotAtUtc DESC
) AS h
LEFT JOIN Robotics.BatteryStatus AS bs ON bs.BatteryStatusId = h.BatteryStatusId
LEFT JOIN Robotics.CommunicationStatus AS cs ON cs.CommunicationStatusId = h.CommunicationStatusId;

-- Mission timeline for one mission.
SELECT m.Code AS MissionCode, e.OccurredAtUtc, e.EventCode, e.Title, r.Code AS RobotCode
FROM Robotics.Mission AS m
JOIN Robotics.MissionEvent AS e ON e.MissionId = m.MissionId
LEFT JOIN Robotics.Robot AS r ON r.RobotId = e.RobotId
WHERE m.Code = 'ROB-MIS-2026-0001'

```



```

ORDER BY e.OccurredAtUtc;

-- Readiness failures that block dispatch.
SELECT m.Code, ri.CheckName, rs.Code AS Readiness, ri.Detail
FROM Robotics.Mission AS m
JOIN Robotics.MissionReadinessAssessment AS ra ON ra.MissionId = m.MissionId
JOIN Robotics.MissionReadinessItem AS ri ON ri.MissionReadinessAssessmentId = ra.MissionReadinessAssessmentId
JOIN Robotics.ReadinessStatus AS rs ON rs.ReadinessStatusId = ri.ReadinessStatusId
WHERE ri.IsBlocking = 1 AND rs.Code IN ('BLOCKED', 'EXPIRED');

```

C.12.6 ER diagram

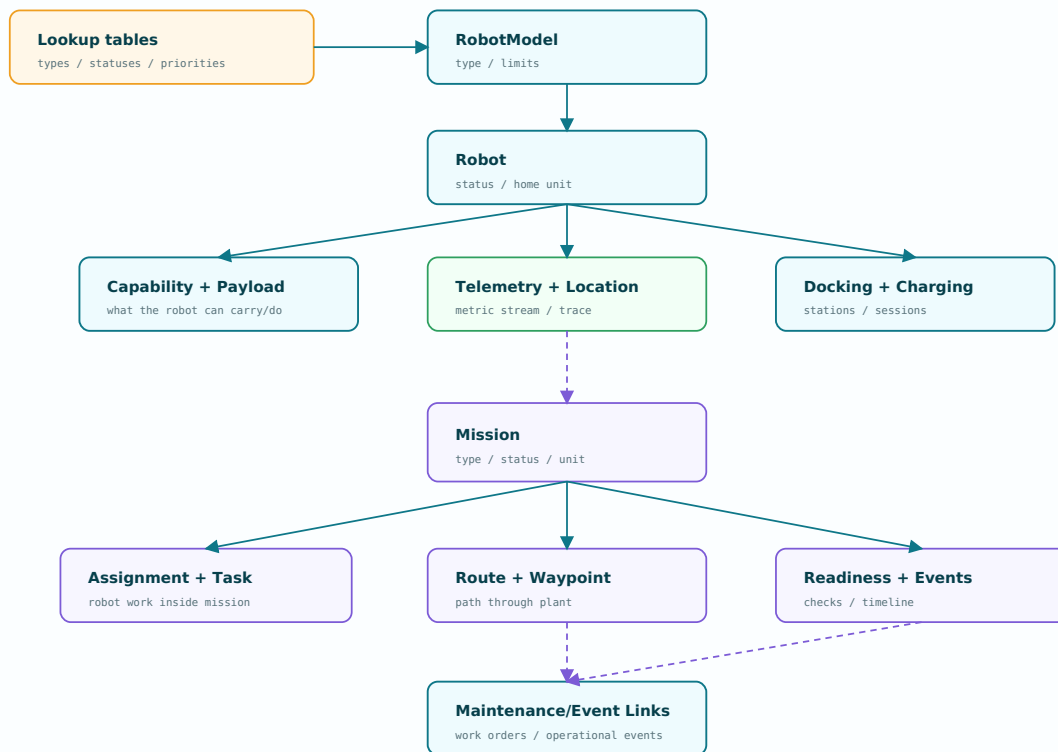


Figure C.12.1 - Robotics sector ER diagram. Robot identity, capability, telemetry, mission planning, readiness and integration links are stored in one bounded context.

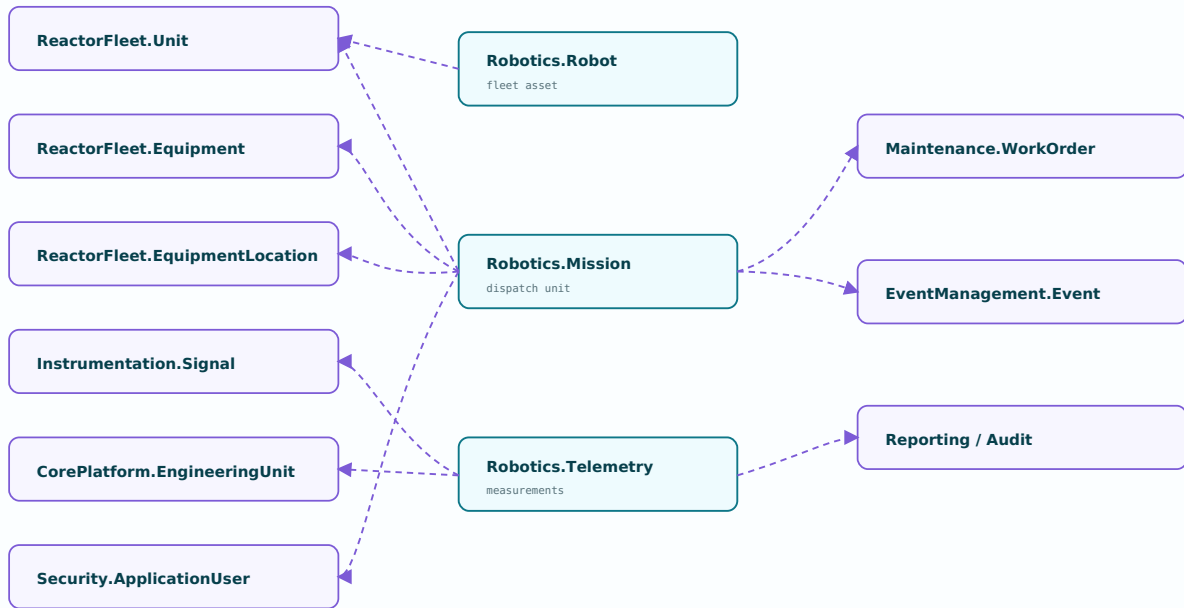


Figure C.12.2 - Robotics passport map. Robotics depends on ReactorFleet, Instrumentation, CorePlatform, Security, Maintenance and EventManagement through explicit foreign keys.

C.12.7 Foreign-key mapping

The table below lists the principal passports. Lookup-table foreign keys are present in the DDL but are not repeated line by line.

RobotModel FK RobotTypeId TO Robotics.RobotType Classifies robot model.	Robot FK RobotModelId TO Robotics.RobotModel Robot is an instance of a model.
Robot FK RobotStatusId TO Robotics.RobotStatus Robot lifecycle/status lookup.	Robot FK HomeUnitId TO ReactorFleet.Unit Robot belongs to, or is normally stationed at, a plant unit.
Robot FK HomeDockingStationId TO Robotics.DockingStation Default charging/docking point.	RobotCapability FK EngineeringUnitId TO CorePlatform.EngineeringUnit Capability rating may be expressed in a common unit.

RobotCapabilityAssignment

FK RobotId
TO Robotics.Robot
Robot has a validated capability.

RobotCapabilityAssignment

FK ValidatedByUserId
TO Security.ApplicationUser
Capability validation is attributable.

RobotPayload

FK PayloadTypeId
TO Robotics.PayloadType
Payload category.

RobotSensorBinding

FK SignalId
TO Instrumentation.Signal
Robot-mounted sensor is visible as a normal signal.

RobotTelemetry

FK RobotId
TO Robotics.Robot
High-volume robot metric.

RobotTelemetry

FK EngineeringUnitId
TO CorePlatform.EngineeringUnit
Telemetry metric unit.

RobotLocationHistory

FK UnitId
TO ReactorFleet.Unit
Location trace belongs to a unit.

RobotLocationHistory

FK EquipmentLocationId
TO ReactorFleet.EquipmentLocation
Location trace may resolve to a plant location.

DockingStation

FK UnitId
TO ReactorFleet.Unit
Docking point exists inside a unit.

ChargingSession

FK RobotId
TO Robotics.Robot
Charging interval for a robot.

Mission

FK UnitId
TO ReactorFleet.Unit
Mission is scoped to a unit.

Mission

FK RequestedByUserId
TO Security.ApplicationUser
Mission request is attributable.

MissionRobotAssignment

FK MissionId
TO Robotics.Mission
Robot assignment to a mission.

MissionTask

FK TargetEquipmentId
TO ReactorFleet.Equipment
Task can target physical equipment.

MissionRouteWaypoint

FK EquipmentLocationId
TO ReactorFleet.EquipmentLocation
Waypoint can be anchored to a plant location.

MissionReadinessAssessment

FK ReadinessStatusId
TO Robotics.ReadinessStatus
Overall readiness verdict.

RobotMaintenanceLink

FK WorkOrderId
TO Maintenance.WorkOrder
Robot fault or service connects to maintenance.

MissionOperationalEventLink

FK OperationalEventId
TO EventManagement.OperationalEvent
A mission may respond to or produce an operational event.

C.12.8 Boundary and next sector

Honest boundary. This schema models robotic assets for the NEXUS-1 demonstrator. It is not a certified robotics control system, not a navigation safety case and not an autonomy stack. It records mission planning, telemetry, readiness and accountability; it does not claim to command any real vehicle.

The next schema is [RadiationMonitoring](#). It will own radiation zones, dose instruments, dosimeters and dose readings. Robotics can later be extended with a cross-sector table that maps missions and robot telemetry to radiation exposure once that sector exists.

Appendix C.13

RadiationMonitoring Schema

The RadiationMonitoring sector stores the radiological layer of the NEXUS-1 data backbone: zones, fixed and portable monitors, monitor readings, personal dosimetry, dose limits, dose alerts, surveys, contamination samples, environmental samples and radiation work permits.

This schema does not own plant topology, signal identity, people, users, operational events or robot missions. Those records remain in their own sectors. RadiationMonitoring connects to them through explicit passports such as `UnitId`, `EquipmentLocationId`, `SignalId`, `PersonId`, `ApplicationUserId`, `OperationalEventId` and `MissionId`.

RadiationMonitoring answers: where the radiological boundary is, what was measured, how good the measurement was, who carried which dosimeter, what dose was recorded, what limit was approached, and which permit or event the evidence belongs to.

C.13.1

Sector purpose

C.13.2

Full table list

C.13.3

Lookup tables

C.13.4

SQL DDL

C.13.5

Indexes and seed examples

C.13.6

ER diagram

C.13.7

Foreign-key mapping

C.13.8

Boundary and next sector

C.13.1 Sector purpose

`RadiationMonitoring` is the controlled radiological record of the platform. It models radiation zones, dose-rate monitors, personal dose devices, survey work, sample results and permit evidence. The design is intentionally measurable and auditable: values carry units, measurements carry quality flags, and personal exposure is tied to an assignment window rather than a loose person ID.

- `RadiationZone` describes controlled and supervised areas.
- `RadiationMonitor`, `RadiationMonitorSignalBinding` and `RadiationReading` connect fixed or portable monitors to the common signal catalogue and to timestamped readings.

- `Dosimeter`, `PersonDosimeterAssignment` and `PersonDoseReading` store personal dose accountability.
- `DoseLimit`, `DoseLimitAssignment` and `DoseAlert` store the limit and alert layer.
- `RadiologicalSurvey`, `RadiologicalSurveyReading`, `ContaminationSample` and `ContaminationSampleResult` store survey evidence.
- `RadiationWorkPermit` connects planned radiological work to zones, people and dose records.

Design choice. Engineering units are still owned by `CorePlatform.EngineeringUnit`. The `RadiationMonitoring` schema owns the radiological meaning of the record, not the unit catalogue itself.

C.13.2 Full table list

The `RadiationMonitoring` sector contains forty-six tables: seventeen typed lookup tables and twenty-nine substantive tables. The tables fall into zones, monitors, dosimetry, surveys, environmental sampling, snapshots, permits and integration links.

LOOKUP RadiationZoneType PK <code>RadiationZoneTypeId</code> FK None Zone category: controlled area, supervised area, exclusion zone, hot cell, waste store or environmental station.	LOOKUP RadiationZoneStatus PK <code>RadiationZoneStatusId</code> FK None Operational status of a radiation zone: open, restricted, locked out, decontamination, retired.
LOOKUP RadiationAreaClassification PK <code>RadiationAreaClassificationId</code> FK None Classification label used by access rules and dose planning.	LOOKUP MonitorType PK <code>MonitorTypeId</code> FK None Fixed, portable, airborne, area gamma, neutron, contamination effluent monitor.
LOOKUP MonitorStatus PK <code>MonitorStatusId</code> FK None Monitor lifecycle: in service, calibration due, faulted, isolated, retired.	LOOKUP MeasurementType PK <code>MeasurementTypeId</code> FK None Measured quantity: dose rate, contamination, count rate, airborne activity, release concentration.

LOOKUP

MeasurementQuality

PK MeasurementQualityId

FK None

Quality flag for radiation readings: good, estimated, suspect, stale, simulated.

LOOKUP

DoseType

PK DoseTypeId

FK None

Dose category: whole body, extremity, lens, committed dose, operational dose.

LOOKUP

DosimeterType

PK DosimeterTypeId

FK None

Dosimeter category: electronic, TLD, OSL, neutron badge, area badge.

LOOKUP

DosimeterStatus

PK DosimeterStatusId

FK None

Dosimeter lifecycle: available, assigned, returned, reading pending, lost, retired.

LOOKUP

SurveyType

PK SurveyTypeId

FK None

Survey category: routine, post-work, release, contamination, emergency, drill.

LOOKUP

SurveyStatus

PK SurveyStatusId

FK None

Survey lifecycle: planned, active, completed, reviewed, rejected, cancelled.

LOOKUP

ContaminationType

PK ContaminationTypeId

FK None

Contamination family: alpha, beta, gamma, removable, fixed, airborne.

LOOKUP

SampleType

PK SampleTypeId

FK None

Sample category: smear, air filter, water, soil, effluent, surface.

LOOKUP

LimitType

PK LimitTypeId

FK None

Limit family: administrative, regulatory, job-specific, warning, alarm, stop-work.

LOOKUP

AlertStatus

PK AlertStatusId

FK None

Alert lifecycle: open, acknowledged, under review, closed, false positive.

LOOKUP**ExposureCategory**

PK ExposureCategoryId
FK None

Worker or task exposure category used by planning and reporting.

ZONE**RadiationMonitoring.RadiationZone**

PK RadiationZoneId
FK UnitId, EquipmentLocationId, lookups

One controlled or monitored radiological area in a unit, plant location or environmental station.

ZONE**RadiationMonitoring.RadiationZoneBoundaryPoint**

PK RadiationZoneBoundaryPointId
FK RadiationZoneId

Ordered geometry points for drawing a zone boundary in 2-D maps or 3-D plant views.

ZONE**RadiationMonitoring.RadiationZoneAccessRequirement**

PK RadiationZoneAccessRequirementId
FK RadiationZoneId, DoseTypeId, EngineeringUnitId

Dose or survey conditions that must be satisfied before entering zone.

MONITOR**RadiationMonitoring.RadiationMonitor**

PK RadiationMonitorId
FK UnitId, EquipmentId, RadiationZoneId, lookups

One installed or portable radiation monitor.

MONITOR**RadiationMonitoring.RadiationMonitorSignalBinding**

PK RadiationMonitorSignalBindingId
FK RadiationMonitorId, SignalId

Binds a radiation monitor channel to the common Instrumentation signal catalogue.

MONITOR**RadiationMonitoring.RadiationMonitorCalibration**

PK RadiationMonitorCalibrationId
FK RadiationMonitorId, ApplicationUserId

Calibration record, due date and pass/fail result for a monitor.

MONITOR**RadiationMonitoring.RadiationReading**

PK RadiationReadingId
FK RadiationMonitorId, EngineeringUnitId, lookups

High-volume timestamped radiation measurement from a monitor.

DOSIMETRY**RadiationMonitoring.Dosimeter**

PK DosimeterId
FK DosimeterTypeId, DosimeterStatusId

One dosimetry device or badge.

DOSIMETRY**RadiationMonitoring.PersonDosimeterAssignment**

PK PersonDosimeterAssignmentId
FK PersonId, DosimeterId

Assignment of a dosimeter to a person for a period.

DOSIMETRY**RadiationMonitoring.PersonDoseReading**

PK PersonDoseReadingId
FK PersonDosimeterAssignmentId, DoseTypeId, EngineeringUnitId

Dose reading collected from a person assignment.

DOSIMETRY**RadiationMonitoring.DoseLimit**

PK DoseLimitId
FK DoseTypeId, LimitTypeId, EngineeringUnitId

Administrative or regulatory dose threshold.

DOSIMETRY**RadiationMonitoring.DoseLimitAssignment**

PK DoseLimitAssignmentId
FK DoseLimitId, PersonId, ExposureCategoryId

Applies a dose limit to a person or exposure category over a validity window.

DOSIMETRY**RadiationMonitoring.DoseAlert**

PK DoseAlertId
FK DoseLimitId, PersonDoseReadingId, AlertStatusId

Alert raised when a personal dose reading approaches or exceeds a limit.

SURVEY**RadiationMonitoring.RadiologicalSurvey**

PK RadiologicalSurveyId
FK UnitId, RadiationZoneId, SurveyTypeId, SurveyStatusId

Survey header: purpose, location, status, assigned person and review status.

SURVEY**RadiationMonitoring.RadiologicalSurveyReading**

PK RadiologicalSurveyReadingId
FK RadiologicalSurveyId, EngineeringUnitId, lookups

Individual point or grid reading captured during a radiological survey.

SURVEY**RadiationMonitoring.ContaminationSample**

PK ContaminationSampleId
FK RadiologicalSurveyId, SampleTypeId, ContaminationTypeId

Sample collected during a survey.

SURVEY**RadiationMonitoring.ContaminationSampleResult**

PK ContaminationSampleResultId
FK ContaminationSampleId, EngineeringUnitId

Analytical result for a contamination sample.

ENVIRONMENT**RadiationMonitoring.EnvironmentalSamplingLocation**

PK EnvironmentalSamplingLocationId
FK PlantId, CountryId, RegionId

External or boundary location used for environmental radiological samples.

ENVIRONMENT**RadiationMonitoring.EnvironmentalSample**

PK EnvironmentalSampleId
FK EnvironmentalSamplingLocationId, SampleTypeId

Environmental sample header.

ENVIRONMENT**RadiationMonitoring.EnvironmentalSampleResult**

PK EnvironmentalSampleResultId
FK EnvironmentalSampleId, EngineeringUnitId

Measured environmental sample result.

INVENTORY**RadiationMonitoring.IsotopeInventory**

PK IsotopeInventoryId
FK UnitId, EquipmentId, EngineeringUnitId

Simple demonstrator-grade isotope inventory record for source waste or sample context.

SNAPSHOT RadiationMonitoring.ZoneDoseSnapshot PK ZoneDoseSnapshotId FK RadiationZoneId , EngineeringUnitId Periodic aggregate zone dose-rate snapshot for trends and dashboards.	PERMIT RadiationMonitoring.RadiationWorkPermit PK RadiationWorkPermitId FK UnitId , RequestedByUserId , ApprovedByUserId Radiation work permit for planned access or work in controlled areas.
PERMIT RadiationMonitoring.RadiationWorkPermitZone PK RadiationWorkPermitZoneId FK RadiationWorkPermitId , RadiationZoneId Zones covered by a radiation work permit.	PERMIT RadiationMonitoring.RadiationWorkPermitPerson PK RadiationWorkPermitPersonId FK RadiationWorkPermitId , PersonId People authorized on a radiation work permit.
PERMIT RadiationMonitoring.RadiationWorkPermitDoseRecord PK RadiationWorkPermitDoseRecordId FK RadiationWorkPermitPersonId , DoseTypeId , EngineeringUnitId Dose accumulated against a permit participant.	INTEGRATION RadiationMonitoring.RadiationEventLink PK RadiationEventLinkId FK OperationalEventId , optional radiation record FKs Links radiological data to an operational event or incident.
INTEGRATION RadiationMonitoring.RadiationRobotMissionLink PK RadiationRobotMissionLinkId FK MissionId , optional zone/reading/survey FKs Links a robotics mission to radiological monitoring records.	REPORTING RadiationMonitoring.RadiationTrendSummary PK RadiationTrendSummaryId FK UnitId , RadiationZoneId , EngineeringUnitId Precomputed trend bucket for dashboards and reporting.

C.13.3 Lookup tables

Lookup tables make zone types, monitor statuses, measurement quality, dose types, survey states and exposure categories explicit. A radiological record should never depend on an ungoverned free-text status.

LOOKUP TABLE	TYPICAL SEED CODES	MAIN CONSUMER
RadiationZoneType	CONTROLLED , SUPERVISED , EXCLUSION , HOT_CELL , WASTE_STORE	Referenced by RadiationZone .
RadiationZoneStatus	OPEN , RESTRICTED , LOCKED_OUT , DECON , RETIRED	Referenced by RadiationZone .
RadiationAreaClassification	LOW , MEDIUM , HIGH , VERY_HIGH , UNKNOWN	Referenced by zones and access planning.
MonitorType		Referenced by RadiationMonitor .

LOOKUP TABLE	TYPICAL SEED CODES	MAIN CONSUMER
	AREA_GAMMA , NEUTRON , AIRBORNE , CONTAMINATION , EFFLUENT	
MonitorStatus	IN_SERVICE , CAL_DUE , FAULTED , ISOLATED , RETIRED	Referenced by RadiationMonitor .
MeasurementType	DOSE_RATE , CONTAMINATION , COUNT_RATE , AIRBORNE_ACTIVITY	Referenced by readings and survey points.
MeasurementQuality	GOOD , ESTIMATED , SUSPECT , STALE , SIMULATED	Referenced by readings and results.
DoseType	WHOLE_BODY , EXTREMITY , LENS , COMMITTED , OPERATIONAL	Referenced by dose records and limits.
DosimeterType	ELECTRONIC , TLD , OSL , NEUTRON_BADGE , AREA_BADGE	Referenced by Dosimeter .
DosimeterStatus	AVAILABLE , ASSIGNED , RETURNED , READING_PENDING , LOST	Referenced by Dosimeter .
SurveyType	ROUTINE , POST_WORK , RELEASE , CONTAMINATION , EMERGENCY	Referenced by RadiologicalSurvey .
SurveyStatus	PLANNED , ACTIVE , COMPLETED , REVIEWED , REJECTED	Referenced by RadiologicalSurvey .
ContaminationType	ALPHA , BETA , GAMMA , REMOVABLE , FIXED , AIRBORNE	Referenced by samples.
SampleType	SMEAR , AIR_FILTER , WATER , SOIL , EFFLUENT , SURFACE	Referenced by samples.
LimitType	ADMIN , REGULATORY , JOB , WARNING , ALARM , STOP_WORK	Referenced by DoseLimit .
AlertStatus	OPEN , ACKNOWLEDGED , UNDER_REVIEW , CLOSED , FALSE_POSITIVE	Referenced by DoseAlert .
ExposureCategory	ROUTINE_WORKER , CONTRACTOR , VISITOR , EMERGENCY , TRAINING	Referenced by limit assignments and permits.

C.13.4 SQL DDL

C.13.4.1 Schema

```
IF SCHEMA ID(N'RadiationMonitoring') IS NULL
EXEC(N'CREATE SCHEMA RadiationMonitoring');
GO
```

C.13.4.2 RadiationMonitoring lookup tables

```
CREATE TABLE RadiationMonitoring.RadiationZoneType (
  RadiationZoneTypeId INT IDENTITY(1,1) NOT NULL,
  Code NVARCHAR(50) NOT NULL,
  Name NVARCHAR(150) NOT NULL,
  Description NVARCHAR(500) NULL,
  DisplayOrder INT NOT NULL
  CONSTRAINT DF_RadiationMonitoring_RadiationZoneType_DisplayOrder DEFAULT 0,
  IsActive BIT NOT NULL
  CONSTRAINT DF_RadiationMonitoring_RadiationZoneType_IsActive DEFAULT 1,
  CreatedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_RadiationMonitoring_RadiationZoneType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_RadiationMonitoring_RadiationZoneType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  RowVersion ROWVERSION NOT NULL,
```

```

CONSTRAINT PK_RadiationMonitoring_RadiationZoneType
PRIMARY KEY (RadiationZoneTypeId),
CONSTRAINT UQ_RadiationMonitoring_RadiationZoneType_Code
UNIQUE (Code)
);
GO

CREATE TABLE RadiationMonitoring.RadiationZoneStatus (
RadiationZoneStatusId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_RadiationMonitoring_RadiationZoneStatus_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_RadiationMonitoring_RadiationZoneStatus_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RadiationMonitoring_RadiationZoneStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RadiationMonitoring_RadiationZoneStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_RadiationMonitoring_RadiationZoneStatus
PRIMARY KEY (RadiationZoneStatusId),
CONSTRAINT UQ_RadiationMonitoring_RadiationZoneStatus_Code
UNIQUE (Code)
);
GO

CREATE TABLE RadiationMonitoring.RadiationAreaClassification (
RadiationAreaClassificationId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_RadiationMonitoring_RadiationAreaClassification_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_RadiationMonitoring_RadiationAreaClassification_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RadiationMonitoring_RadiationAreaClassification_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RadiationMonitoring_RadiationAreaClassification_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_RadiationMonitoring_RadiationAreaClassification
PRIMARY KEY (RadiationAreaClassificationId),
CONSTRAINT UQ_RadiationMonitoring_RadiationAreaClassification_Code
UNIQUE (Code)
);
GO

CREATE TABLE RadiationMonitoring.MonitorType (
MonitorTypeId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_RadiationMonitoring_MonitorType_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_RadiationMonitoring_MonitorType_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RadiationMonitoring_MonitorType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RadiationMonitoring_MonitorType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_RadiationMonitoring_MonitorType
PRIMARY KEY (MonitorTypeId),
CONSTRAINT UQ_RadiationMonitoring_MonitorType_Code
UNIQUE (Code)
);
GO

CREATE TABLE RadiationMonitoring.MonitorStatus (
MonitorStatusId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_RadiationMonitoring_MonitorStatus_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_RadiationMonitoring_MonitorStatus_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RadiationMonitoring_MonitorStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RadiationMonitoring_MonitorStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_RadiationMonitoring_MonitorStatus
PRIMARY KEY (MonitorStatusId),
CONSTRAINT UQ_RadiationMonitoring_MonitorStatus_Code
UNIQUE (Code)
);
GO

CREATE TABLE RadiationMonitoring.MeasurementType (
MeasurementTypeId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_RadiationMonitoring_MeasurementType_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_RadiationMonitoring_MeasurementType_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RadiationMonitoring_MeasurementType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RadiationMonitoring_MeasurementType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_RadiationMonitoring_MeasurementType
PRIMARY KEY (MeasurementTypeId),
CONSTRAINT UQ_RadiationMonitoring_MeasurementType_Code
UNIQUE (Code)
);
GO

CREATE TABLE RadiationMonitoring.MeasurementQuality (
MeasurementQualityId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_RadiationMonitoring_MeasurementQuality_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_RadiationMonitoring_MeasurementQuality_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RadiationMonitoring_MeasurementQuality_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RadiationMonitoring_MeasurementQuality_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_RadiationMonitoring_MeasurementQuality
PRIMARY KEY (MeasurementQualityId),
CONSTRAINT UQ_RadiationMonitoring_MeasurementQuality_Code
UNIQUE (Code)
);
GO

```

```

CREATE TABLE RadiationMonitoring.DoseType (
    DoseTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_RadiationMonitoring_DoseType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_RadiationMonitoring_DoseType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_RadiationMonitoring_DoseType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_RadiationMonitoring_DoseType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_RadiationMonitoring_DoseType
        PRIMARY KEY (DoseTypeId),
    CONSTRAINT UQ_RadiationMonitoring_DoseType_Code
        UNIQUE (Code)
);
GO

CREATE TABLE RadiationMonitoring.DosimeterType (
    DosimeterTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_RadiationMonitoring_DosimeterType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_RadiationMonitoring_DosimeterType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_RadiationMonitoring_DosimeterType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_RadiationMonitoring_DosimeterType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_RadiationMonitoring_DosimeterType
        PRIMARY KEY (DosimeterTypeId),
    CONSTRAINT UQ_RadiationMonitoring_DosimeterType_Code
        UNIQUE (Code)
);
GO

CREATE TABLE RadiationMonitoring.DosimeterStatus (
    DosimeterStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_RadiationMonitoring_DosimeterStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_RadiationMonitoring_DosimeterStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_RadiationMonitoring_DosimeterStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_RadiationMonitoring_DosimeterStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_RadiationMonitoring_DosimeterStatus
        PRIMARY KEY (DosimeterStatusId),
    CONSTRAINT UQ_RadiationMonitoring_DosimeterStatus_Code
        UNIQUE (Code)
);
GO

CREATE TABLE RadiationMonitoring.SurveyType (
    SurveyTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_RadiationMonitoring_SurveyType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_RadiationMonitoring_SurveyType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_RadiationMonitoring_SurveyType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_RadiationMonitoring_SurveyType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_RadiationMonitoring_SurveyType
        PRIMARY KEY (SurveyTypeId),
    CONSTRAINT UQ_RadiationMonitoring_SurveyType_Code
        UNIQUE (Code)
);
GO

CREATE TABLE RadiationMonitoring.SurveyStatus (
    SurveyStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_RadiationMonitoring_SurveyStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_RadiationMonitoring_SurveyStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_RadiationMonitoring_SurveyStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_RadiationMonitoring_SurveyStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_RadiationMonitoring_SurveyStatus
        PRIMARY KEY (SurveyStatusId),
    CONSTRAINT UQ_RadiationMonitoring_SurveyStatus_Code
        UNIQUE (Code)
);
GO

CREATE TABLE RadiationMonitoring.ContaminationType (
    ContaminationTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_RadiationMonitoring_ContaminationType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_RadiationMonitoring_ContaminationType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_RadiationMonitoring_ContaminationType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_RadiationMonitoring_ContaminationType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_RadiationMonitoring_ContaminationType
        PRIMARY KEY (ContaminationTypeId),
    CONSTRAINT UQ_RadiationMonitoring_ContaminationType_Code
        UNIQUE (Code)
);
GO

CREATE TABLE RadiationMonitoring.SampleType (
    SampleTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,

```

```

DisplayOrder INT NOT NULL
CONSTRAINT DF_RadiationMonitoring_SampleType_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_RadiationMonitoring_SampleType_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RadiationMonitoring_SampleType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RadiationMonitoring_SampleType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_RadiationMonitoring_SampleType
PRIMARY KEY (SampleTypeId),
CONSTRAINT UQ_RadiationMonitoring_SampleType_Code
UNIQUE (Code)
);
GO

CREATE TABLE RadiationMonitoring.LimitType (
LimitTypeId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_RadiationMonitoring_LimitType_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_RadiationMonitoring_LimitType_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RadiationMonitoring_LimitType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RadiationMonitoring_LimitType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_RadiationMonitoring_LimitType
PRIMARY KEY (LimitTypeId),
CONSTRAINT UQ_RadiationMonitoring_LimitType_Code
UNIQUE (Code)
);
GO

CREATE TABLE RadiationMonitoring.AlertStatus (
AlertStatusId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_RadiationMonitoring_AlertStatus_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_RadiationMonitoring_AlertStatus_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RadiationMonitoring_AlertStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RadiationMonitoring_AlertStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_RadiationMonitoring_AlertStatus
PRIMARY KEY (AlertStatusId),
CONSTRAINT UQ_RadiationMonitoring_AlertStatus_Code
UNIQUE (Code)
);
GO

CREATE TABLE RadiationMonitoring.ExposureCategory (
ExposureCategoryId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_RadiationMonitoring_ExposureCategory_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_RadiationMonitoring_ExposureCategory_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RadiationMonitoring_ExposureCategory_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_RadiationMonitoring_ExposureCategory_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_RadiationMonitoring_ExposureCategory
PRIMARY KEY (ExposureCategoryId),
CONSTRAINT UQ_RadiationMonitoring_ExposureCategory_Code
UNIQUE (Code)
);
GO

```

C.13.4.3 Zones and access requirements

```

CREATE TABLE RadiationMonitoring.RadiationZone (
RadiationZoneId INT IDENTITY(1,1) NOT NULL,
UnitId INT NOT NULL,
EquipmentLocationId INT NOT NULL,
RadiationZoneTypeId INT NOT NULL,
RadiationZoneStatusId INT NOT NULL,
RadiationAreaClassificationId INT NOT NULL,
Code NVARCHAR(80) NOT NULL,
Name NVARCHAR(200) NOT NULL,
Description NVARCHAR(1000) NULL,
IsEntryControlled BIT NOT NULL CONSTRAINT DF_RadiationZone_IsEntryControlled DEFAULT 1,
RequiresDosimeter BIT NOT NULL CONSTRAINT DF_RadiationZone_RequiresDosimeter DEFAULT 1,
PostedAtUtc DATETIME2(7) NULL,
CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RadiationZone_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
CreatedBy NVARCHAR(100) NOT NULL,
ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RadiationZone_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
ModifiedBy NVARCHAR(100) NOT NULL,
IsDeleted BIT NOT NULL CONSTRAINT DF_RadiationZone_IsDeleted DEFAULT 0,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_RadiationMonitoring_RadiationZone PRIMARY KEY (RadiationZoneId),
CONSTRAINT UQ_RadiationMonitoring_RadiationZone_Code UNIQUE (Code),
CONSTRAINT FK_RadiationZone_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
CONSTRAINT FK_RadiationZone_EquipmentLocation FOREIGN KEY (EquipmentLocationId) REFERENCES
ReactorFleet.EquipmentLocation(EquipmentLocationId),
CONSTRAINT FK_RadiationZone_ZoneType FOREIGN KEY (RadiationZoneTypeId) REFERENCES
RadiationMonitoring.RadiationZoneType(RadiationZoneTypeId),
CONSTRAINT FK_RadiationZone_ZoneStatus FOREIGN KEY (RadiationZoneStatusId) REFERENCES
RadiationMonitoring.RadiationZoneStatus(RadiationZoneStatusId),
CONSTRAINT FK_RadiationZone_AreaClassification FOREIGN KEY (RadiationAreaClassificationId) REFERENCES
RadiationMonitoring.RadiationAreaClassification(RadiationAreaClassificationId)
);
GO

CREATE TABLE RadiationMonitoring.RadiationZoneBoundaryPoint (
RadiationZoneBoundaryPointId INT IDENTITY(1,1) NOT NULL,
RadiationZoneId INT NOT NULL,
SequenceNo INT NOT NULL,
XCoordinate DECIMAL(18,6) NOT NULL,
YCoordinate DECIMAL(18,6) NOT NULL,
ZCoordinate DECIMAL(18,6) NULL,
CoordinateSystem NVARCHAR(50) NOT NULL CONSTRAINT DF_RadiationZoneBoundaryPoint_CoordinateSystem
DEFAULT N'PLANT_LOCAL',

CONSTRAINT PK_RadiationMonitoring_RadiationZoneBoundaryPoint PRIMARY KEY (RadiationZoneBoundaryPointId),
CONSTRAINT UQ_RadiationZoneBoundaryPoint_Zone_Seq UNIQUE (RadiationZoneId, SequenceNo),

```

```

CONSTRAINT FK_RadiationZoneBoundaryPoint_Zone FOREIGN KEY (RadiationZoneId) REFERENCES
RadiationMonitoring.RadiationZone(RadiationZoneId),
CONSTRAINT CK_RadiationZoneBoundaryPoint_Sequence CHECK (SequenceNo > 0)
);
GO

CREATE TABLE RadiationMonitoring.RadiationZoneAccessRequirement (
RadiationZoneAccessRequirementId INT IDENTITY(1,1) NOT NULL,
RadiationZoneId INT NOT NULL,
DoseTypeId INT NOT NULL,
EngineeringUnitId INT NOT NULL,
RequirementCode NVARCHAR(80) NOT NULL,
RequirementText NVARCHAR(1000) NOT NULL,
MaxPermittedValue DECIMAL(18,6) NULL,
RequiresSupervisorApproval BIT NOT NULL CONSTRAINT DF_RadiationZoneAccessRequirement_RequiresApproval
DEFAULT 0,
IsActive BIT NOT NULL CONSTRAINT DF_RadiationZoneAccessRequirement_IsActive DEFAULT
1,
CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT
DF_RadiationZoneAccessRequirement_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
CreatedBy NVARCHAR(100) NOT NULL,
ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT
DF_RadiationZoneAccessRequirement_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedBy NVARCHAR(100) NOT NULL,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_RadiationMonitoring_RadiationZoneAccessRequirement PRIMARY KEY
(RadiationZoneAccessRequirementId),
CONSTRAINT UQ_RadiationZoneAccessRequirement_Zone_Code UNIQUE (RadiationZoneId, RequirementCode),
CONSTRAINT FK_RadiationZoneAccessRequirement_Zone FOREIGN KEY (RadiationZoneId) REFERENCES
RadiationMonitoring.RadiationZone(RadiationZoneId),
CONSTRAINT FK_RadiationZoneAccessRequirement_DoseType FOREIGN KEY (DoseTypeId) REFERENCES
RadiationMonitoring.DoseType(DoseTypeId),
CONSTRAINT FK_RadiationZoneAccessRequirement_EngineeringUnit FOREIGN KEY (EngineeringUnitId) REFERENCES
CorePlatform.EngineeringUnit(EngineeringUnitId)
);
GO

```

C.13.4.4 Monitors, signal bindings, calibration and readings

```

CREATE TABLE RadiationMonitoring.RadiationMonitor (
RadiationMonitorId INT IDENTITY(1,1) NOT NULL,
UnitId INT NOT NULL,
EquipmentId INT NOT NULL,
RadiationZoneId INT NOT NULL,
MonitorTypeId INT NOT NULL,
MonitorStatusId INT NOT NULL,
Code NVARCHAR(80) NOT NULL,
Name NVARCHAR(200) NOT NULL,
SerialNumber NVARCHAR(120) NULL,
InstalledAtUtc DATETIME2(7) NULL,
CalibrationDueAtUtc DATETIME2(7) NULL,
CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RadiationMonitor_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
CreatedBy NVARCHAR(100) NOT NULL,
ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RadiationMonitor_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
ModifiedBy NVARCHAR(100) NOT NULL,
IsDeleted BIT NOT NULL CONSTRAINT DF_RadiationMonitor_IsDeleted DEFAULT 0,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_RadiationMonitoring_RadiationMonitor PRIMARY KEY (RadiationMonitorId),
CONSTRAINT UQ_RadiationMonitoring_RadiationMonitor_Code UNIQUE (Code),
CONSTRAINT FK_RadiationMonitor_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
CONSTRAINT FK_RadiationMonitor_Equipment FOREIGN KEY (EquipmentId) REFERENCES
ReactorFleet.Equipment(EquipmentId),
CONSTRAINT FK_RadiationMonitor_Zone FOREIGN KEY (RadiationZoneId) REFERENCES
RadiationMonitoring.RadiationZone(RadiationZoneId),
CONSTRAINT FK_RadiationMonitor_MonitorType FOREIGN KEY (MonitorTypeId) REFERENCES
RadiationMonitoring.MonitorType(MonitorTypeId),
CONSTRAINT FK_RadiationMonitor_MonitorStatus FOREIGN KEY (MonitorStatusId) REFERENCES
RadiationMonitoring.MonitorStatus(MonitorStatusId)
);
GO

CREATE TABLE RadiationMonitoring.RadiationMonitorSignalBinding (
RadiationMonitorSignalBindingId INT IDENTITY(1,1) NOT NULL,
RadiationMonitorId INT NOT NULL,
SignalId INT NOT NULL,
ChannelName NVARCHAR(100) NOT NULL,
IsPrimary BIT NOT NULL CONSTRAINT DF_RadiationMonitorSignalBinding_IsPrimary DEFAULT
0,
CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT
DF_RadiationMonitorSignalBinding_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
CreatedBy NVARCHAR(100) NOT NULL,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_RadiationMonitoring_RadiationMonitorSignalBinding PRIMARY KEY
(RadiationMonitorSignalBindingId),
CONSTRAINT UQ_RadiationMonitorSignalBinding_Monitor_Channel UNIQUE (RadiationMonitorId, ChannelName),
CONSTRAINT UQ_RadiationMonitorSignalBinding_Signal UNIQUE (SignalId),
CONSTRAINT FK_RadiationMonitorSignalBinding_Monitor FOREIGN KEY (RadiationMonitorId) REFERENCES
RadiationMonitoring.RadiationMonitor(RadiationMonitorId),
CONSTRAINT FK_RadiationMonitorSignalBinding_Signal FOREIGN KEY (SignalId) REFERENCES
Instrumentation.Signal(SignalId)
);
GO

CREATE TABLE RadiationMonitoring.RadiationMonitorCalibration (
RadiationMonitorCalibrationId INT IDENTITY(1,1) NOT NULL,
RadiationMonitorId INT NOT NULL,
PerformedByUserId INT NOT NULL,
CalibrationAtUtc DATETIME2(7) NOT NULL,
DueAtUtc DATETIME2(7) NOT NULL,
ReferenceSource NVARCHAR(200) NULL,
CalibrationFactor DECIMAL(18,8) NULL,
Passed BIT NOT NULL,
Notes NVARCHAR(1000) NULL,
CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RadiationMonitorCalibration_CreatedAtUtc
DEFAULT SYSUTCDATETIME(),
CreatedBy NVARCHAR(100) NOT NULL,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_RadiationMonitoring_RadiationMonitorCalibration PRIMARY KEY (RadiationMonitorCalibrationId),
CONSTRAINT FK_RadiationMonitorCalibration_Monitor FOREIGN KEY (RadiationMonitorId) REFERENCES
RadiationMonitoring.RadiationMonitor(RadiationMonitorId),
CONSTRAINT FK_RadiationMonitorCalibration_User FOREIGN KEY (PerformedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId),
CONSTRAINT CK_RadiationMonitorCalibration_DueAfterCalibration CHECK (DueAtUtc > CalibrationAtUtc)
);
GO

CREATE TABLE RadiationMonitoring.RadiationReading (
RadiationReadingId BIGINT IDENTITY(1,1) NOT NULL,
RadiationMonitorId INT NOT NULL,
MeasurementTypeId INT NOT NULL,
EngineeringUnitId INT NOT NULL,
MeasurementQualityId INT NOT NULL,
TimestampUtc DATETIME2(7) NOT NULL,
Value DECIMAL(18,6) NOT NULL,
IsAlarmRelevant BIT NOT NULL CONSTRAINT DF_RadiationReading_IsAlarmRelevant DEFAULT 0,
SourceTimestampUtc DATETIME2(7) NULL,
CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RadiationReading_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),

CONSTRAINT PK_RadiationMonitoring_RadiationReading PRIMARY KEY (RadiationReadingId),

```



```

CONSTRAINT FK_RadiationReading_Monitor FOREIGN KEY (RadiationMonitorId) REFERENCES
RadiationMonitoring.RadiationMonitor(RadiationMonitorId),
CONSTRAINT FK_RadiationReading_MeasurementType FOREIGN KEY (MeasurementTypeId) REFERENCES
RadiationMonitoring.MeasurementType(MeasurementTypeId),
CONSTRAINT FK_RadiationReading_EngineeringUnit FOREIGN KEY (EngineeringUnitId) REFERENCES
CorePlatform.EngineeringUnit(EngineeringUnitId),
CONSTRAINT FK_RadiationReading_MeasurementQuality FOREIGN KEY (MeasurementQualityId) REFERENCES
RadiationMonitoring.MeasurementQuality(MeasurementQualityId)
);
GO

```

C.13.4.5 Dosimetry, dose limits and dose alerts

```

CREATE TABLE RadiationMonitoring.Dosimeter (
    DosimeterId INT IDENTITY(1,1) NOT NULL,
    DosimeterTypeId INT NOT NULL,
    DosimeterStatusId INT NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    SerialNumber NVARCHAR(120) NULL,
    CalibrationDueAtUtc DATETIME2(7) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Dosimeter_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Dosimeter_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Dosimeter_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_RadiationMonitoring_Dosimeter PRIMARY KEY (DosimeterId),
    CONSTRAINT UQ_RadiationMonitoring_Dosimeter_Code UNIQUE (Code),
    CONSTRAINT FK_Dosimeter_DosimeterType FOREIGN KEY (DosimeterTypeId) REFERENCES
RadiationMonitoring.DosimeterType(DosimeterTypeId),
    CONSTRAINT FK_Dosimeter_DosimeterStatus FOREIGN KEY (DosimeterStatusId) REFERENCES
RadiationMonitoring.DosimeterStatus(DosimeterStatusId)
);
GO

CREATE TABLE RadiationMonitoring.PersonDosimeterAssignment (
    PersonDosimeterAssignmentId INT IDENTITY(1,1) NOT NULL,
    PersonId INT NOT NULL,
    DosimeterId INT NOT NULL,
    AssignedByUserId INT NULL,
    AssignedAtUtc DATETIME2(7) NOT NULL,
    ReturnedAtUtc DATETIME2(7) NULL,
    AssignmentPurpose NVARCHAR(300) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_PersonDosimeterAssignment_CreatedAtUtc
DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_RadiationMonitoring_PersonDosimeterAssignment PRIMARY KEY (PersonDosimeterAssignmentId),
    CONSTRAINT FK_PersonDosimeterAssignment_Person FOREIGN KEY (PersonId) REFERENCES
Organization.Person(PersonId),
    CONSTRAINT FK_PersonDosimeterAssignment_Dosimeter FOREIGN KEY (DosimeterId) REFERENCES
RadiationMonitoring.Dosimeter(DosimeterId),
    CONSTRAINT FK_PersonDosimeterAssignment_AssignedBy FOREIGN KEY (AssignedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_PersonDosimeterAssignment_ReturnAfterAssign CHECK (ReturnedAtUtc IS NULL OR ReturnedAtUtc >
AssignedAtUtc)
);
GO

CREATE TABLE RadiationMonitoring.PersonDoseReading (
    PersonDoseReadingId BIGINT IDENTITY(1,1) NOT NULL,
    PersonDosimeterAssignmentId INT NOT NULL,
    DoseTypeId INT NOT NULL,
    EngineeringUnitId INT NOT NULL,
    MeasurementQualityId INT NOT NULL,
    ReadingAtUtc DATETIME2(7) NOT NULL,
    DoseValue DECIMAL(18,6) NOT NULL,
    IsFinal BIT NOT NULL CONSTRAINT DF_PersonDoseReading_IsFinal DEFAULT 0,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_PersonDoseReading_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),

    CONSTRAINT PK_RadiationMonitoring_PersonDoseReading PRIMARY KEY (PersonDoseReadingId),
    CONSTRAINT FK_PersonDoseReading_Assignment FOREIGN KEY (PersonDosimeterAssignmentId) REFERENCES
RadiationMonitoring.PersonDosimeterAssignment(PersonDosimeterAssignmentId),
    CONSTRAINT FK_PersonDoseReading_DoseType FOREIGN KEY (DoseTypeId) REFERENCES
RadiationMonitoring.DoseType(DoseTypeId),
    CONSTRAINT FK_PersonDoseReading_EngineeringUnit FOREIGN KEY (EngineeringUnitId) REFERENCES
CorePlatform.EngineeringUnit(EngineeringUnitId),
    CONSTRAINT FK_PersonDoseReading_Quality FOREIGN KEY (MeasurementQualityId) REFERENCES
RadiationMonitoring.MeasurementQuality(MeasurementQualityId),
    CONSTRAINT CK_PersonDoseReading_DoseNonNegative CHECK (DoseValue >= 0)
);
GO

CREATE TABLE RadiationMonitoring.DoseLimit (
    DoseLimitId INT IDENTITY(1,1) NOT NULL,
    DoseTypeId INT NOT NULL,
    LimitTypeId INT NOT NULL,
    EngineeringUnitId INT NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    LimitValue DECIMAL(18,6) NOT NULL,
    PeriodDays INT NOT NULL,
    IsActive BIT NOT NULL CONSTRAINT DF_DoseLimit_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DoseLimit_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DoseLimit_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_RadiationMonitoring_DoseLimit PRIMARY KEY (DoseLimitId),
    CONSTRAINT UQ_RadiationMonitoring_DoseLimit_Code UNIQUE (Code),
    CONSTRAINT FK_DoseLimit_DoseType FOREIGN KEY (DoseTypeId) REFERENCES
RadiationMonitoring.DoseType(DoseTypeId),
    CONSTRAINT FK_DoseLimit_LimitType FOREIGN KEY (LimitTypeId) REFERENCES
RadiationMonitoring.LimitType(LimitTypeId),
    CONSTRAINT FK_DoseLimit_EngineeringUnit FOREIGN KEY (EngineeringUnitId) REFERENCES
CorePlatform.EngineeringUnit(EngineeringUnitId),
    CONSTRAINT CK_DoseLimit_Value CHECK (LimitValue >= 0),
    CONSTRAINT CK_DoseLimit_Period CHECK (PeriodDays > 0)
);
GO

CREATE TABLE RadiationMonitoring.DoseLimitAssignment (
    DoseLimitAssignmentId INT IDENTITY(1,1) NOT NULL,
    DoseLimitId INT NOT NULL,
    PersonId INT NOT NULL,
    ExposureCategoryId INT NOT NULL,
    ValidFromUtc DATETIME2(7) NOT NULL,
    ValidToUtc DATETIME2(7) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DoseLimitAssignment_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_RadiationMonitoring_DoseLimitAssignment PRIMARY KEY (DoseLimitAssignmentId),
    CONSTRAINT FK_DoseLimitAssignment_DoseLimit FOREIGN KEY (DoseLimitId) REFERENCES
RadiationMonitoring.DoseLimit(DoseLimitId),
    CONSTRAINT FK_DoseLimitAssignment_Person FOREIGN KEY (PersonId) REFERENCES Organization.Person(PersonId),
    CONSTRAINT FK_DoseLimitAssignment_ExposureCategory FOREIGN KEY (ExposureCategoryId) REFERENCES
RadiationMonitoring.ExposureCategory(ExposureCategoryId),
    CONSTRAINT CK_DoseLimitAssignment_Target CHECK (PersonId IS NOT NULL OR ExposureCategoryId IS NOT NULL),
    CONSTRAINT CK_DoseLimitAssignment_ValidWindow CHECK (ValidToUtc IS NULL OR ValidToUtc > ValidFromUtc)
);
GO

```



```
GO

CREATE TABLE RadiationMonitoring.DoseAlert (
    DoseAlertId BIGINT IDENTITY(1,1) NOT NULL,
    DoseLimitId INT NOT NULL,
    PersonDoseReadingId BIGINT NULL,
    AlertStatusId INT NOT NULL,
    AcknowledgedByUserId INT NULL,
    AlertAtUtc DATETIME2(7) NOT NULL,
    AcknowledgedAtUtc DATETIME2(7) NULL,
    Message NVARCHAR(1000) NOT NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DoseAlert_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_RadiationMonitoring_DoseAlert PRIMARY KEY (DoseAlertId),
    CONSTRAINT FK_DoseAlert_DoseLimit FOREIGN KEY (DoseLimitId) REFERENCES
        RadiationMonitoring.DoseLimit(DoseLimitId),
    CONSTRAINT FK_DoseAlert_PersonDoseReading FOREIGN KEY (PersonDoseReadingId) REFERENCES
        RadiationMonitoring.PersonDoseReading(PersonDoseReadingId),
    CONSTRAINT FK_DoseAlert_AlertStatus FOREIGN KEY (AlertStatusId) REFERENCES
        RadiationMonitoring.AlertStatus(AlertStatusId),
    CONSTRAINT FK_DoseAlert_AcknowledgedBy FOREIGN KEY (AcknowledgedByUserId) REFERENCES
        Security.ApplicationUser(ApplicationUserId)
);
GO
```

C.13.4.6 Radiological surveys and contamination samples

```
CREATE TABLE RadiationMonitoring.RadiologicalSurvey (
    RadiologicalSurveyId INT IDENTITY(1,1) NOT NULL,
    UnitId INT NULL,
    RadiationZoneId INT NULL,
    SurveyTypeId INT NOT NULL,
    SurveyStatusId INT NOT NULL,
    AssignedToPersonId INT NULL,
    ReviewedByUserId INT NULL,
    Code NVARCHAR(80) NOT NULL,
    Title NVARCHAR(250) NOT NULL,
    PlannedAtUtc DATETIME2(7) NULL,
    StartedAtUtc DATETIME2(7) NULL,
    CompletedAtUtc DATETIME2(7) NULL,
    ReviewedAtUtc DATETIME2(7) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RadiologicalSurvey_CreatedAtUtc DEFAULT
        SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RadiologicalSurvey_ModifiedAtUtc DEFAULT
        SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_RadiologicalSurvey_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_RadiationMonitoring_RadiologicalSurvey PRIMARY KEY (RadiologicalSurveyId),
    CONSTRAINT UQ_RadiationMonitoring_RadiologicalSurvey_Code UNIQUE (Code),
    CONSTRAINT FK_RadiologicalSurvey_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_RadiologicalSurvey_Zone FOREIGN KEY (RadiationZoneId) REFERENCES
        RadiationMonitoring.RadiationZone(RadiationZoneId),
    CONSTRAINT FK_RadiologicalSurvey_SurveyType FOREIGN KEY (SurveyTypeId) REFERENCES
        RadiationMonitoring.SurveyType(SurveyTypeId),
    CONSTRAINT FK_RadiologicalSurvey_SurveyStatus FOREIGN KEY (SurveyStatusId) REFERENCES
        RadiationMonitoring.SurveyStatus(SurveyStatusId),
    CONSTRAINT FK_RadiologicalSurvey_AssignedPerson FOREIGN KEY (AssignedToPersonId) REFERENCES
        Organization.Person(PersonId),
    CONSTRAINT FK_RadiologicalSurvey_ReviewedBy FOREIGN KEY (ReviewedByUserId) REFERENCES
        Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE RadiationMonitoring.RadiologicalSurveyReading (
    RadiologicalSurveyReadingId BIGINT IDENTITY(1,1) NOT NULL,
    RadiologicalSurveyId INT NOT NULL,
    MeasurementTypeId INT NOT NULL,
    MeasurementQualityId INT NOT NULL,
    EngineeringUnitId INT NOT NULL,
    PointCode NVARCHAR(80) NULL,
    ReadingAtUtc DATETIME2(7) NOT NULL,
    Value DECIMAL(18,6) NOT NULL,
    XCoordinate DECIMAL(18,6) NULL,
    YCoordinate DECIMAL(18,6) NULL,
    ZCoordinate DECIMAL(18,6) NULL,
    Notes NVARCHAR(1000) NULL,

    CONSTRAINT PK_RadiationMonitoring_RadiologicalSurveyReading PRIMARY KEY (RadiologicalSurveyReadingId),
    CONSTRAINT FK_RadiologicalSurveyReading_Survey FOREIGN KEY (RadiologicalSurveyId) REFERENCES
        RadiationMonitoring.RadiologicalSurvey(RadiologicalSurveyId),
    CONSTRAINT FK_RadiologicalSurveyReading_MeasurementType FOREIGN KEY (MeasurementTypeId) REFERENCES
        RadiationMonitoring.MeasurementType(MeasurementTypeId),
    CONSTRAINT FK_RadiologicalSurveyReading_Quality FOREIGN KEY (MeasurementQualityId) REFERENCES
        RadiationMonitoring.MeasurementQuality(MeasurementQualityId),
    CONSTRAINT FK_RadiologicalSurveyReading_EngineeringUnit FOREIGN KEY (EngineeringUnitId) REFERENCES
        CorePlatform.EngineeringUnit(EngineeringUnitId)
);
GO

CREATE TABLE RadiationMonitoring.ContaminationSample (
    ContaminationSampleId INT IDENTITY(1,1) NOT NULL,
    RadiologicalSurveyId INT NULL,
    SampleTypeId INT NOT NULL,
    ContaminationTypeId INT NOT NULL,
    SampleCode NVARCHAR(80) NOT NULL,
    CollectedAtUtc DATETIME2(7) NOT NULL,
    CollectedByPersonId INT NULL,
    SampleLocation NVARCHAR(250) NULL,
    Notes NVARCHAR(1000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ContaminationSample_CreatedAtUtc DEFAULT
        SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_RadiationMonitoring_ContaminationSample PRIMARY KEY (ContaminationSampleId),
    CONSTRAINT UQ_RadiationMonitoring_ContaminationSample_Code UNIQUE (SampleCode),
    CONSTRAINT FK_ContaminationSample_Survey FOREIGN KEY (RadiologicalSurveyId) REFERENCES
        RadiationMonitoring.RadiologicalSurvey(RadiologicalSurveyId),
    CONSTRAINT FK_ContaminationSample_SampleType FOREIGN KEY (SampleTypeId) REFERENCES
        RadiationMonitoring.SampleType(SampleTypeId),
    CONSTRAINT FK_ContaminationSample_ContaminationType FOREIGN KEY (ContaminationTypeId) REFERENCES
        RadiationMonitoring.ContaminationType(ContaminationTypeId),
    CONSTRAINT FK_ContaminationSample_CollectedBy FOREIGN KEY (CollectedByPersonId) REFERENCES
        Organization.Person(PersonId)
);
GO

CREATE TABLE RadiationMonitoring.ContaminationSampleResult (
    ContaminationSampleResultId BIGINT IDENTITY(1,1) NOT NULL,
    ContaminationSampleId INT NOT NULL,
    MeasurementQualityId INT NOT NULL,
    EngineeringUnitId INT NOT NULL,
    ResultAtUtc DATETIME2(7) NOT NULL,
    Value DECIMAL(18,6) NOT NULL,
    IsAboveLimit BIT NOT NULL CONSTRAINT DF_ContaminationSampleResult_IsAboveLimit DEFAULT 0,
    Notes NVARCHAR(1000) NULL,

    CONSTRAINT PK_RadiationMonitoring_ContaminationSampleResult PRIMARY KEY (ContaminationSampleResultId),
    CONSTRAINT FK_ContaminationSampleResult_Sample FOREIGN KEY (ContaminationSampleId) REFERENCES
        RadiationMonitoring.ContaminationSample(ContaminationSampleId),
    CONSTRAINT FK_ContaminationSampleResult_Quality FOREIGN KEY (MeasurementQualityId) REFERENCES
        RadiationMonitoring.MeasurementQuality(MeasurementQualityId),

```

```

CONSTRAINT FK ContaminationSampleResult_EngineeringUnit FOREIGN KEY (EngineeringUnitId) REFERENCES
CorePlatform.EngineeringUnit(EngineeringUnitId)
);
GO

```

C.13.4.7 Environmental sampling

```

CREATE TABLE RadiationMonitoring.EnvironmentalSamplingLocation (
    EnvironmentalSamplingLocationId INT IDENTITY(1,1) NOT NULL,
    PlantId INT NULL,
    CountryId INT NULL,
    RegionId INT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    Description NVARCHAR(1000) NULL,
    Latitude DECIMAL(18,7) NULL,
    Longitude DECIMAL(18,7) NULL,
    IsActive BIT NOT NULL CONSTRAINT DF_EnvironmentalSamplingLocation_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT
    DF_EnvironmentalSamplingLocation_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT
    DF_EnvironmentalSamplingLocation_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_RadiationMonitoring_EnvironmentalSamplingLocation PRIMARY KEY
    (EnvironmentalSamplingLocationId),
    CONSTRAINT UQ_RadiationMonitoring_EnvironmentalSamplingLocation_Code UNIQUE (Code),
    CONSTRAINT FK_EnvironmentalSamplingLocation_Plant FOREIGN KEY (PlantId) REFERENCES
    Organization.Plant(PlantId),
    CONSTRAINT FK_EnvironmentalSamplingLocation_Country FOREIGN KEY (CountryId) REFERENCES
    CorePlatform.Country(CountryId),
    CONSTRAINT FK_EnvironmentalSamplingLocation_Region FOREIGN KEY (RegionId) REFERENCES
    CorePlatform.Region(RegionId)
);
GO

CREATE TABLE RadiationMonitoring.EnvironmentalSample (
    EnvironmentalSampleId INT IDENTITY(1,1) NOT NULL,
    EnvironmentalSamplingLocationId INT NOT NULL,
    SampleTypeId INT NOT NULL,
    SampleCode NVARCHAR(80) NOT NULL,
    CollectedAtUtc DATETIME2(7) NOT NULL,
    CollectedByPersonId INT NULL,
    Notes NVARCHAR(1000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_EnvironmentalSample_CreatedAtUtc
    DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_RadiationMonitoring_EnvironmentalSample PRIMARY KEY (EnvironmentalSampleId),
    CONSTRAINT UQ_RadiationMonitoring_EnvironmentalSample_Code UNIQUE (SampleCode),
    CONSTRAINT FK_EnvironmentalSample_Location FOREIGN KEY (EnvironmentalSamplingLocationId) REFERENCES
    RadiationMonitoring.EnvironmentalSamplingLocation(EnvironmentalSamplingLocationId),
    CONSTRAINT FK_EnvironmentalSample_SampleType FOREIGN KEY (SampleTypeId) REFERENCES
    RadiationMonitoring.SampleType(SampleTypeId),
    CONSTRAINT FK_EnvironmentalSample_CollectedBy FOREIGN KEY (CollectedByPersonId) REFERENCES
    Organization.Person(PersonId)
);
GO

CREATE TABLE RadiationMonitoring.EnvironmentalSampleResult (
    EnvironmentalSampleResultId BIGINT IDENTITY(1,1) NOT NULL,
    EnvironmentalSampleId INT NOT NULL,
    MeasurementTypeId INT NOT NULL,
    MeasurementQualityId INT NOT NULL,
    EngineeringUnitId INT NOT NULL,
    ResultAtUtc DATETIME2(7) NOT NULL,
    Value DECIMAL(18,6) NOT NULL,
    IsAboveLimit BIT NOT NULL CONSTRAINT DF_EnvironmentalSampleResult_IsAboveLimit DEFAULT 0,
    Notes NVARCHAR(1000) NULL,

    CONSTRAINT PK_RadiationMonitoring_EnvironmentalSampleResult PRIMARY KEY (EnvironmentalSampleResultId),
    CONSTRAINT FK_EnvironmentalSampleResult_Sample FOREIGN KEY (EnvironmentalSampleId) REFERENCES
    RadiationMonitoring.EnvironmentalSample(EnvironmentalSampleId),
    CONSTRAINT FK_EnvironmentalSampleResult_MeasurementType FOREIGN KEY (MeasurementTypeId) REFERENCES
    RadiationMonitoring.MeasurementType(MeasurementTypeId),
    CONSTRAINT FK_EnvironmentalSampleResult_Quality FOREIGN KEY (MeasurementQualityId) REFERENCES
    RadiationMonitoring.MeasurementQuality(MeasurementQualityId),
    CONSTRAINT FK_EnvironmentalSampleResult_EngineeringUnit FOREIGN KEY (EngineeringUnitId) REFERENCES
    CorePlatform.EngineeringUnit(EngineeringUnitId)
);
GO

```

C.13.4.8 Isotope inventory, zone snapshots and radiation work permits

```

CREATE TABLE RadiationMonitoring.IsotopeInventory (
    IsotopeInventoryId INT IDENTITY(1,1) NOT NULL,
    UnitId INT NULL,
    EquipmentId INT NULL,
    EngineeringUnitId INT NOT NULL,
    IsotopeCode NVARCHAR(40) NOT NULL,
    Description NVARCHAR(500) NULL,
    ActivityValue DECIMAL(18,6) NOT NULL,
    ReferenceAtUtc DATETIME2(7) NOT NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_IsotopeInventory_CreatedAtUtc DEFAULT
    SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_IsotopeInventory_ModifiedAtUtc DEFAULT
    SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_IsotopeInventory_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_RadiationMonitoring_IsotopeInventory PRIMARY KEY (IsotopeInventoryId),
    CONSTRAINT FK_IsotopeInventory_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_IsotopeInventory_Equipment FOREIGN KEY (EquipmentId) REFERENCES
    ReactorFleet.Equipment(EquipmentId),
    CONSTRAINT FK_IsotopeInventory_EngineeringUnit FOREIGN KEY (EngineeringUnitId) REFERENCES
    CorePlatform.EngineeringUnit(EngineeringUnitId),
    CONSTRAINT CK_IsotopeInventory_Activity CHECK (ActivityValue >= 0)
);
GO

CREATE TABLE RadiationMonitoring.ZoneDoseSnapshot (
    ZoneDoseSnapshotId BIGINT IDENTITY(1,1) NOT NULL,
    RadiationZoneId INT NOT NULL,
    EngineeringUnitId INT NOT NULL,
    SnapshotAtUtc DATETIME2(7) NOT NULL,
    AverageValue DECIMAL(18,6) NOT NULL,
    MaximumValue DECIMAL(18,6) NOT NULL,
    MinimumValue DECIMAL(18,6) NOT NULL,
    SampleCount INT NOT NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ZoneDoseSnapshot_CreatedAtUtc DEFAULT
    SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL
);
GO

```

```

CONSTRAINT PK_RadiationMonitoring_ZoneDoseSnapshot PRIMARY KEY (ZoneDoseSnapshotId),
CONSTRAINT FK_ZoneDoseSnapshot_Zone FOREIGN KEY (RadiationZoneId) REFERENCES
RadiationMonitoring.RadiationZone(RadiationZoneId)
CONSTRAINT FK_ZoneDoseSnapshot_EngineeringUnit FOREIGN KEY (EngineeringUnitId) REFERENCES
CorePlatform.EngineeringUnit(EngineeringUnitId),
CONSTRAINT CK_ZoneDoseSnapshot_SampleCount CHECK (SampleCount >= 0)
);
GO

CREATE TABLE RadiationMonitoring.RadiationWorkPermit (
    RadiationWorkPermitId INT IDENTITY(1,1) NOT NULL,
    UnitId INT NULL,
    ExposureCategoryId INT NULL,
    RequestedByUserId INT NULL,
    ApprovedByUserId INT NULL,
    PermitNumber NVARCHAR(80) NOT NULL,
    Title NVARCHAR(250) NOT NULL,
    WorkDescription NVARCHAR(2000) NULL,
    ValidFromUtc DATETIME2(7) NOT NULL,
    ValidToUtc DATETIME2(7) NOT NULL,
    ApprovedAtUtc DATETIME2(7) NULL,
    IsClosed BIT NOT NULL CONSTRAINT DF_RadiationWorkPermit_IsClosed DEFAULT 0,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RadiationWorkPermit_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RadiationWorkPermit_ModifiedAtUtc DEFAULT
SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_RadiationWorkPermit_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_RadiationMonitoring_RadiationWorkPermit PRIMARY KEY (RadiationWorkPermitId),
    CONSTRAINT UQ_RadiationMonitoring_RadiationWorkPermit_Number UNIQUE (PermitNumber),
    CONSTRAINT FK_RadiationWorkPermit_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_RadiationWorkPermit_ExposureCategory FOREIGN KEY (ExposureCategoryId) REFERENCES
RadiationMonitoring.ExposureCategory(ExposureCategoryId),
    CONSTRAINT FK_RadiationWorkPermit_RequestedBy FOREIGN KEY (RequestedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT FK_RadiationWorkPermit_ApprovedBy FOREIGN KEY (ApprovedByUserId) REFERENCES
Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_RadiationWorkPermit_ValidWindow CHECK (ValidToUtc > ValidFromUtc)
);
GO

CREATE TABLE RadiationMonitoring.RadiationWorkPermitZone (
    RadiationWorkPermitZoneId INT IDENTITY(1,1) NOT NULL,
    RadiationWorkPermitId INT NOT NULL,
    RadiationZoneId INT NOT NULL,
    PlannedEntryAtUtc DATETIME2(7) NULL,
    PlannedExitAtUtc DATETIME2(7) NULL,
    Notes NVARCHAR(1000) NULL,

    CONSTRAINT PK_RadiationMonitoring_RadiationWorkPermitZone PRIMARY KEY (RadiationWorkPermitZoneId),
    CONSTRAINT UQ_RadiationWorkPermitZone_Permit_Zone UNIQUE (RadiationWorkPermitId, RadiationZoneId),
    CONSTRAINT FK_RadiationWorkPermitZone_Permit FOREIGN KEY (RadiationWorkPermitId) REFERENCES
RadiationMonitoring.RadiationWorkPermit(RadiationWorkPermitId),
    CONSTRAINT FK_RadiationWorkPermitZone_Zone FOREIGN KEY (RadiationZoneId) REFERENCES
RadiationMonitoring.RadiationZone(RadiationZoneId)
);
GO

CREATE TABLE RadiationMonitoring.RadiationWorkPermitPerson (
    RadiationWorkPermitPersonId INT IDENTITY(1,1) NOT NULL,
    RadiationWorkPermitId INT NOT NULL,
    PersonId INT NOT NULL,
    PersonDosimeterAssignmentId INT NULL,
    PlannedRole NVARCHAR(150) NULL,
    EntryAtUtc DATETIME2(7) NULL,
    ExitAtUtc DATETIME2(7) NULL,

    CONSTRAINT PK_RadiationMonitoring_RadiationWorkPermitPerson PRIMARY KEY (RadiationWorkPermitPersonId),
    CONSTRAINT UQ_RadiationWorkPermitPerson_Permit_Person UNIQUE (RadiationWorkPermitId, PersonId),
    CONSTRAINT FK_RadiationWorkPermitPerson_Permit FOREIGN KEY (RadiationWorkPermitId) REFERENCES
RadiationMonitoring.RadiationWorkPermit(RadiationWorkPermitId),
    CONSTRAINT FK_RadiationWorkPermitPerson_Person FOREIGN KEY (PersonId) REFERENCES
Organization.Person(PersonId),
    CONSTRAINT FK_RadiationWorkPermitPerson_Assignment FOREIGN KEY (PersonDosimeterAssignmentId) REFERENCES
RadiationMonitoring.PersonDosimeterAssignment(PersonDosimeterAssignmentId)
);
GO

CREATE TABLE RadiationMonitoring.RadiationWorkPermitDoseRecord (
    RadiationWorkPermitDoseRecordId BIGINT IDENTITY(1,1) NOT NULL,
    RadiationWorkPermitPersonId INT NOT NULL,
    DoseTypeId INT NOT NULL,
    EngineeringUnitId INT NOT NULL,
    DoseValue DECIMAL(18,6) NOT NULL,
    RecordedAtUtc DATETIME2(7) NOT NULL,
    Notes NVARCHAR(1000) NULL,

    CONSTRAINT PK_RadiationMonitoring_RadiationWorkPermitDoseRecord PRIMARY KEY
(RadiationWorkPermitDoseRecordId),
    CONSTRAINT FK_RadiationWorkPermitDoseRecord_PermitPerson FOREIGN KEY (RadiationWorkPermitPersonId)
REFERENCES RadiationMonitoring.RadiationWorkPermitPerson(RadiationWorkPermitPersonId),
    CONSTRAINT FK_RadiationWorkPermitDoseRecord_DoseType FOREIGN KEY (DoseTypeId) REFERENCES
RadiationMonitoring.DoseType(DoseTypeId),
    CONSTRAINT FK_RadiationWorkPermitDoseRecord_EngineeringUnit FOREIGN KEY (EngineeringUnitId) REFERENCES
CorePlatform.EngineeringUnit(EngineeringUnitId),
    CONSTRAINT CK_RadiationWorkPermitDoseRecord_Dose CHECK (DoseValue >= 0)
);
GO

```

C.13.4.9 Event, mission and trend integration

```

CREATE TABLE RadiationMonitoring.RadiationEventLink (
    RadiationEventLinkId INT IDENTITY(1,1) NOT NULL,
    OperationalEventId INT NOT NULL,
    RadiationZoneId INT NULL,
    RadiationReadingId BIGINT NULL,
    RadiologicalSurveyId INT NULL,
    PersonDoseReadingId BIGINT NULL,
    LinkReason NVARCHAR(500) NOT NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RadiationEventLink_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_RadiationMonitoring_RadiationEventLink PRIMARY KEY (RadiationEventLinkId),
    CONSTRAINT FK_RadiationEventLink_OperationalEvent FOREIGN KEY (OperationalEventId) REFERENCES
EventManagement.OperationalEvent(OperationalEventId),
    CONSTRAINT FK_RadiationEventLink_Zone FOREIGN KEY (RadiationZoneId) REFERENCES
RadiationMonitoring.RadiationZone(RadiationZoneId),
    CONSTRAINT FK_RadiationEventLink_Reading FOREIGN KEY (RadiationReadingId) REFERENCES
RadiationMonitoring.RadiationReading(RadiationReadingId),
    CONSTRAINT FK_RadiationEventLink_Survey FOREIGN KEY (RadiologicalSurveyId) REFERENCES
RadiationMonitoring.RadiologicalSurvey(RadiologicalSurveyId),
    CONSTRAINT FK_RadiationEventLink_PersonDoseReading FOREIGN KEY (PersonDoseReadingId) REFERENCES
RadiationMonitoring.PersonDoseReading(PersonDoseReadingId),
    CONSTRAINT CK_RadiationEventLink_Target CHECK (RadiationZoneId IS NOT NULL OR RadiationReadingId IS NOT NULL
OR RadiologicalSurveyId IS NOT NULL OR PersonDoseReadingId IS NOT NULL)
);
GO

CREATE TABLE RadiationMonitoring.RadiationRobotMissionLink (
    RadiationRobotMissionLinkId INT IDENTITY(1,1) NOT NULL,

```

```

MissionId                INT NOT NULL,
RadiationZoneId          INT NULL,
RadiationReadingId       BIGINT NULL,
RadiologicalSurveyId     INT NULL,
LinkReason               NVARCHAR(500) NOT NULL,
CreatedAtUtc             DATETIME2(7) NOT NULL CONSTRAINT DF_RadiationRobotMissionLink_CreatedAtUtc
DEFAULT SYSUTCDATETIME(),
CreatedBy                NVARCHAR(100) NOT NULL,
RowVersion               ROWVERSION NOT NULL,

CONSTRAINT PK_RadiationMonitoring_RadiationRobotMissionLink PRIMARY KEY (RadiationRobotMissionLinkId),
CONSTRAINT FK_RadiationRobotMissionLink_Mission FOREIGN KEY (MissionId) REFERENCES
Robotics.Mission(MissionId),
CONSTRAINT FK_RadiationRobotMissionLink_Zone FOREIGN KEY (RadiationZoneId) REFERENCES
RadiationMonitoring.RadiationZone(RadiationZoneId),
CONSTRAINT FK_RadiationRobotMissionLink_Reading FOREIGN KEY (RadiationReadingId) REFERENCES
RadiationMonitoring.RadiationReading(RadiationReadingId),
CONSTRAINT FK_RadiationRobotMissionLink_Survey FOREIGN KEY (RadiologicalSurveyId) REFERENCES
RadiationMonitoring.RadiologicalSurvey(RadiologicalSurveyId)
);
GO

CREATE TABLE RadiationMonitoring.RadiationTrendSummary (
    RadiationTrendSummaryId BIGINT IDENTITY(1,1) NOT NULL,
    UnitId                  INT NULL,
    RadiationZoneId         INT NULL,
    MeasurementTypeId       INT NOT NULL,
    EngineeringUnitId       INT NOT NULL,
    BucketStartUtc          DATETIME2(7) NOT NULL,
    BucketEndUtc            DATETIME2(7) NOT NULL,
    AverageValue            DECIMAL(18,6) NOT NULL,
    MaximumValue            DECIMAL(18,6) NOT NULL,
    MinimumValue            DECIMAL(18,6) NOT NULL,
    SampleCount             INT NOT NULL,
    CreatedAtUtc            DATETIME2(7) NOT NULL CONSTRAINT DF_RadiationTrendSummary_CreatedAtUtc DEFAULT
SYSUTCDATETIME(),

CONSTRAINT PK_RadiationMonitoring_RadiationTrendSummary PRIMARY KEY (RadiationTrendSummaryId),
CONSTRAINT FK_RadiationTrendSummary_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
CONSTRAINT FK_RadiationTrendSummary_Zone FOREIGN KEY (RadiationZoneId) REFERENCES
RadiationMonitoring.RadiationZone(RadiationZoneId),
CONSTRAINT FK_RadiationTrendSummary_MeasurementType FOREIGN KEY (MeasurementTypeId) REFERENCES
RadiationMonitoring.MeasurementType(MeasurementTypeId),
CONSTRAINT FK_RadiationTrendSummary_EngineeringUnit FOREIGN KEY (EngineeringUnitId) REFERENCES
CorePlatform.EngineeringUnit(EngineeringUnitId),
CONSTRAINT CK_RadiationTrendSummary_Window CHECK (BucketEndUtc > BucketStartUtc),
CONSTRAINT CK_RadiationTrendSummary_SampleCount CHECK (SampleCount >= 0)
);
GO

```

C.13.5 Indexes and seed examples

The most important access paths are zone-by-unit, monitor-by-zone/status, readings by monitor and time, personal dose by assignment and time, open dose alerts, survey readings by survey, environmental sample history and permit validity windows.

```

CREATE INDEX IX_RadiationZone_UnitId ON RadiationMonitoring.RadiationZone(UnitId);
CREATE INDEX IX_RadiationZone_EquipmentLocationId ON RadiationMonitoring.RadiationZone(EquipmentLocationId);
CREATE INDEX IX_RadiationZone_Status ON RadiationMonitoring.RadiationZone(RadiationZoneStatusId,
    RadiationAreaClassificationId);

CREATE INDEX IX_RadiationMonitor_UnitId ON RadiationMonitoring.RadiationMonitor(UnitId);
CREATE INDEX IX_RadiationMonitor_ZoneId ON RadiationMonitoring.RadiationMonitor(RadiationZoneId);
CREATE INDEX IX_RadiationMonitor_Status ON RadiationMonitoring.RadiationMonitor(MonitorStatusId,
    CalibrationDueAtUtc);

CREATE INDEX IX_RadiationReading_Monitor_Timestamp ON RadiationMonitoring.RadiationReading(RadiationMonitorId,
    TimestampUtc DESC);
CREATE INDEX IX_RadiationReading_Type_Timestamp ON RadiationMonitoring.RadiationReading(MeasurementTypeId,
    TimestampUtc DESC);
CREATE INDEX IX_RadiationReading_AlarmRelevant ON RadiationMonitoring.RadiationReading(IsAlarmRelevant,
    TimestampUtc DESC);

CREATE INDEX IX_PersonDoseAssignment_Person_Time ON RadiationMonitoring.PersonDoseAssignment(PersonId,
    AssignedAtUtc DESC);
CREATE INDEX IX_PersonDoseReading_Assignment_Time ON
    RadiationMonitoring.PersonDoseReading(PersonDoseAssignmentId, ReadingAtUtc DESC);
CREATE INDEX IX_DoseAlert_Status_Time ON RadiationMonitoring.DoseAlert(AlertStatusId, AlertAtUtc DESC);

CREATE INDEX IX_RadiologicalSurvey_Unit_Status ON RadiationMonitoring.RadiologicalSurvey(UnitId,
    SurveyStatusId);
CREATE INDEX IX_RadiologicalSurveyReading_Survey_Time ON
    RadiationMonitoring.RadiologicalSurveyReading(RadiologicalSurveyId, ReadingAtUtc DESC);
CREATE INDEX IX_ContaminationSample_Survey ON RadiationMonitoring.ContaminationSample(RadiologicalSurveyId,
    CollectedAtUtc DESC);

CREATE INDEX IX_EnvironmentalSample_Location_Time ON
    RadiationMonitoring.EnvironmentalSample(EnvironmentalSamplingLocationId, CollectedAtUtc DESC);
CREATE INDEX IX_ZoneDoseSnapshot_Zone_Time ON RadiationMonitoring.ZoneDoseSnapshot(RadiationZoneId,
    SnapshotAtUtc DESC);
CREATE INDEX IX_RadiationWorkPermit_Unit_Window ON RadiationMonitoring.RadiationWorkPermit(UnitId, ValidFromUtc,
    ValidToUtc);
CREATE INDEX IX_RadiationTrendSummary_Zone_Bucket ON RadiationMonitoring.RadiationTrendSummary(RadiationZoneId,
    BucketStartUtc DESC);
GO

```

C.13.5.1 Seed examples

```

INSERT INTO RadiationMonitoring.RadiationZoneType (Code, Name, Description, DisplayOrder) VALUES
(N'CONTROLLED', N'Controlled area', N'Area requiring controlled radiological access.', 10),
(N'SUPERVISED', N'Supervised area', N'Area subject to routine radiological monitoring.', 20),
(N'EXCLUSION', N'Exclusion zone', N'Area normally not entered during the active condition.', 30),
(N'HOT_CELL', N'Hot cell', N'Shielded work area for radioactive material.', 40);

INSERT INTO RadiationMonitoring.MonitorType (Code, Name, Description, DisplayOrder) VALUES
(N'AREA_GAMMA', N'Area gamma monitor', N'Fixed area dose-rate monitor.', 10),
(N'NEUTRON', N'Neutron monitor', N'Neutron dose or count-rate monitor.', 20),
(N'AIRBORNE', N'Airborne activity monitor', N'Airborne activity or particulate monitor.', 30),
(N'CONTAMINATION', N'Contamination monitor', N'Surface contamination monitor.', 40);

INSERT INTO RadiationMonitoring.MeasurementQuality (Code, Name, Description, DisplayOrder) VALUES
(N'GOOD', N'Good', N'Valid measurement.', 10),
(N'ESTIMATED', N'Estimated', N'Estimated or inferred value.', 20),
(N'SUSPECT', N'Suspect', N'Quality check failed or value needs review.', 30),
(N'SIMULATED', N'Simulated', N'Demonstrator-generated value.', 40);

INSERT INTO RadiationMonitoring.DoseType (Code, Name, Description, DisplayOrder) VALUES

```

```
(N'WHOLE BODY', N'Whole body', N'Whole-body operational dose.', 10),
(N'EXTREMITY', N'Extremity', N'Extremity dose.', 20),
(N'LENS', N'Eye lens', N'Lens of eye dose.', 30),
(N'OPERATIONAL', N'Operational', N'Operational dose used by the demonstrator.', 40);
GO
```

C.13.5.2 Verification queries

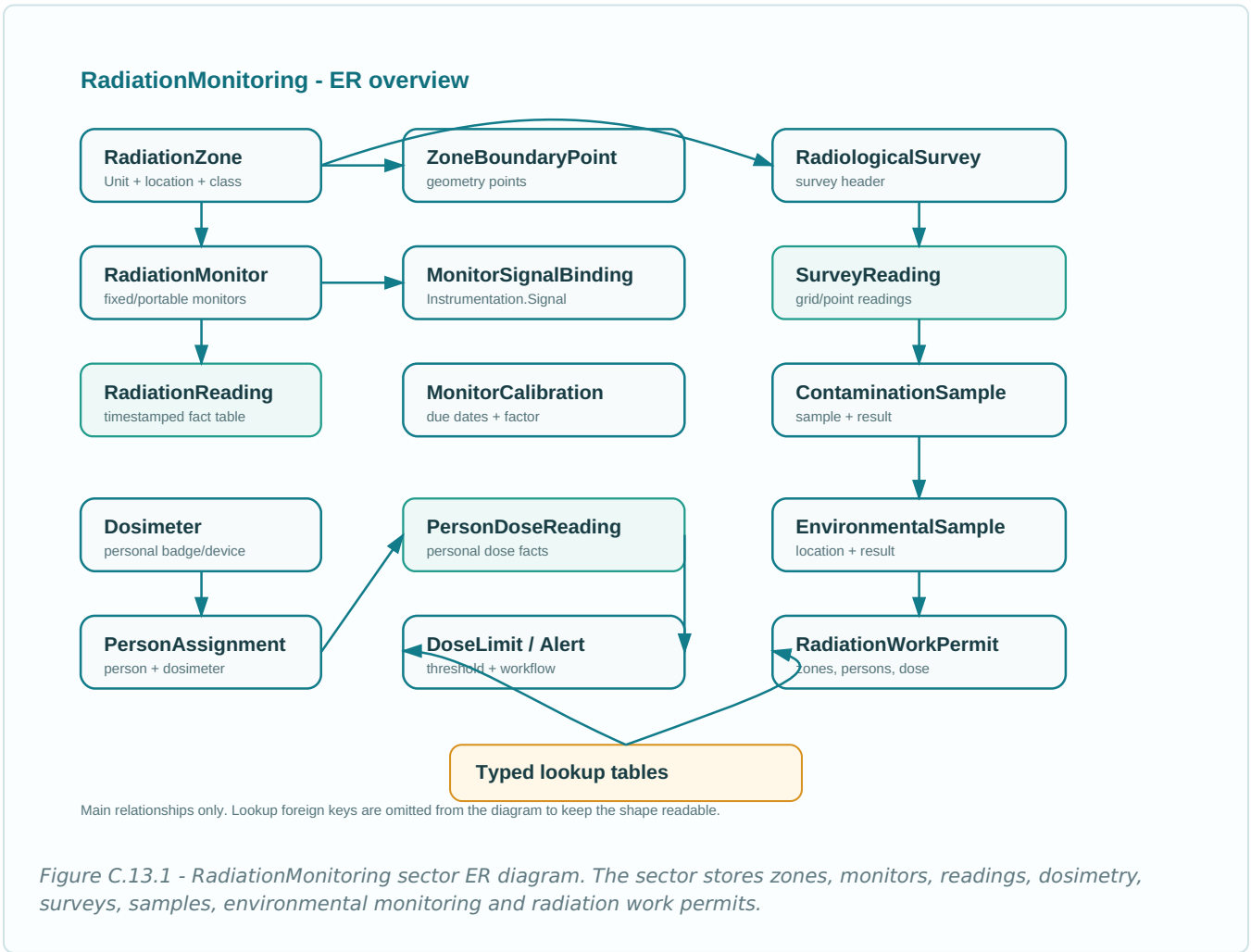
```
-- 1. List active radiation zones with their unit and classification.
SELECT z.Code, z.Name, u.Code AS UnitCode, cls.Code AS Classification, zs.Code AS Status
FROM RadiationMonitoring.RadiationZone AS z
LEFT JOIN ReactorFleet.Unit AS u ON u.UnitId = z.UnitId
JOIN RadiationMonitoring.RadiationAreaClassification AS cls ON cls.RadiationAreaClassificationId =
z.RadiationAreaClassificationId
JOIN RadiationMonitoring.RadiationZoneStatus AS zs ON zs.RadiationZoneStatusId = z.RadiationZoneStatusId
WHERE z.IsDeleted = 0;

-- 2. Find monitors whose calibration is due.
SELECT m.Code, m.Name, mt.Code AS MonitorType, m.CalibrationDueAtUtc
FROM RadiationMonitoring.RadiationMonitor AS m
JOIN RadiationMonitoring.MonitorType AS mt ON mt.MonitorTypeId = m.MonitorTypeId
WHERE m.CalibrationDueAtUtc IS NOT NULL
AND m.CalibrationDueAtUtc <= SYSUTCDATETIME();

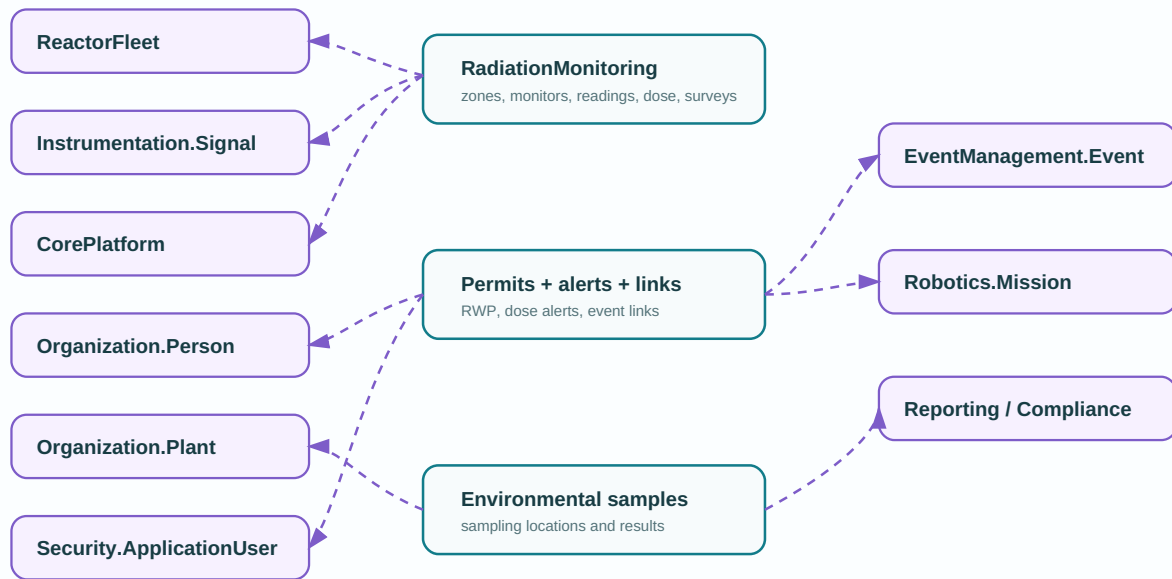
-- 3. Latest reading per monitor.
SELECT m.Code, r.TimestampUtc, r.Value, eu.Symbol, q.Code AS Quality
FROM RadiationMonitoring.RadiationMonitor AS m
CROSS APPLY (
    SELECT TOP (1) rr.*
    FROM RadiationMonitoring.RadiationReading AS rr
    WHERE rr.RadiationMonitorId = m.RadiationMonitorId
    ORDER BY rr.TimestampUtc DESC
) AS r
JOIN CorePlatform.EngineeringUnit AS eu ON eu.EngineeringUnitId = r.EngineeringUnitId
JOIN RadiationMonitoring.MeasurementQuality AS q ON q.MeasurementQualityId = r.MeasurementQualityId;

-- 4. Open dose alerts and the person assignment that produced them.
SELECT da.AlertAtUtc, da.Message, dl.Code AS LimitCode, p.DisplayName, pdr.DoseValue, eu.Symbol
FROM RadiationMonitoring.DoseAlert AS da
JOIN RadiationMonitoring.DoseLimit AS dl ON dl.DoseLimitId = da.DoseLimitId
LEFT JOIN RadiationMonitoring.PersonDoseReading AS pdr ON pdr.PersonDoseReadingId = da.PersonDoseReadingId
LEFT JOIN RadiationMonitoring.PersonDosimeterAssignment AS a ON a.PersonDosimeterAssignmentId =
pdr.PersonDosimeterAssignmentId
LEFT JOIN Organization.Person AS p ON p.PersonId = a.PersonId
LEFT JOIN CorePlatform.EngineeringUnit AS eu ON eu.EngineeringUnitId = pdr.EngineeringUnitId
JOIN RadiationMonitoring.AlertStatus AS s ON s.AlertStatusId = da.AlertStatusId
WHERE s.Code IN (N'OPEN', N'ACKNOWLEDGED');
GO
```

C.13.6 ER diagram



RadiationMonitoring - foreign-key passport map



RadiationMonitoring owns radiological facts. It references plant topology, signal tags, people, users, events and missions; it does not duplicate them.

Figure C.13.2 - RadiationMonitoring passport map. The sector references plant topology, instrumentation signals, people, users, events and robotic missions instead of duplicating them.

C.13.7 Foreign-key mapping

The table below lists the principal passports. Lookup-table foreign keys are present in the DDL but are not repeated line by line.

RadiationZone

FK UnitId
TO ReactorFleet.Unit
Places a radiation zone in a physical unit.

RadiationZone

FK EquipmentLocationId
TO ReactorFleet.EquipmentLocation
Places a zone in a plant location.

RadiationZoneAccessRequirement

FK EngineeringUnitId
TO CorePlatform.EngineeringUnit
Gives numeric access thresholds a controlled unit.

RadiationMonitor

FK UnitId
TO ReactorFleet.Unit
Associates a monitor with a unit.

RadiationMonitor

FK EquipmentId
TO ReactorFleet.Equipment
Associates a monitor with an installed equipment record.

RadiationMonitor

FK RadiationZoneld
TO RadiationMonitoring.RadiationZone
Locates a monitor in a controlled zone.

RadiationMonitorSignalBinding

FK SignalId
TO Instrumentation.Signal

Makes a monitor channel part of the common signal catalogue.

RadiationMonitorCalibration

FK PerformedByUserId
TO Security.ApplicationUser

Records who performed or entered calibration.

RadiationReading

FK EngineeringUnitId
TO CorePlatform.EngineeringUnit

Normalizes reading units.

PersonDosimeterAssignment

FK PersonId
TO Organization.Person

Assigns a dosimeter to a real person record.

PersonDosimeterAssignment

FK AssignedByUserId
TO Security.ApplicationUser

Records the account that assigned the dosimeter.

PersonDoseReading

FK PersonDosimeterAssignmentId
TO RadiationMonitoring.PersonDosimeterAssignment

Connects personal dose to an assignment period.

DoseLimitAssignment

FK PersonId
TO Organization.Person

Applies a dose limit to one person.

DoseLimitAssignment

FK ExposureCategoryId
TO RadiationMonitoring.ExposureCategory

Applies a limit to a class of exposure.

DoseAlert

FK AcknowledgedByUserId
TO Security.ApplicationUser

Records who acknowledged an alert.

RadiologicalSurvey

FK UnitId
TO ReactorFleet.Unit

Scopes a survey to a physical unit.

RadiologicalSurvey

FK RadiationZoneId
TO RadiationMonitoring.RadiationZone

Scopes a survey to a zone.

RadiologicalSurvey

FK AssignedToPersonId
TO Organization.Person

Assigns survey work to a person.

RadiologicalSurvey

FK ReviewedByUserId
TO Security.ApplicationUser

Records review responsibility.

ContaminationSample

FK RadiologicalSurveyId
TO RadiationMonitoring.RadiologicalSurvey

Connects a sample to a survey.

EnvironmentalSamplingLocation FK PlantId TO Organization.Plant Links environmental monitoring to the plant site.	EnvironmentalSamplingLocation FK CountryId / RegionId TO CorePlatform.Country / Region Normalizes geographic references.
EnvironmentalSample FK CollectedByPersonId TO Organization.Person Records the person who collected a sample.	IsotopeInventory FK UnitId / EquipmentId TO ReactorFleet.Unit / Equipment Places isotope inventory in the plant hierarchy.
RadiationWorkPermit FK RequestedByUserId / ApprovedByUserId TO Security.ApplicationUser Records permit accountability.	RadiationWorkPermitPerson FK PersonId TO Organization.Person Lists authorized workers on a radiation permit.
RadiationEventLink FK OperationalEventId TO EventManagement.OperationalEvent Binds radiological evidence to an operational event.	RadiationRobotMissionLink FK MissionId TO Robotics.Mission Binds robotic survey work to radiation monitoring records.

C.13.8 Boundary and next sector

Honest boundary. This schema is a demonstrator-grade radiation-monitoring data model for NEXUS-1. It is not a licensed radiological protection system, not a dosimetry legal record, and not a safety-class monitoring platform. It records structured radiological facts for an educational digital-twin database.

The next schema is **EmergencyPreparedness**. It will use plant units, zones, people, events and radiation information to model emergency plans, exercises, routes, muster points and drill evidence.

Appendix C.14 - EmergencyPreparedness Schema

EmergencyPreparedness turns plant structure, radiological zones, events, people, robots and communications into a controlled response model. It does not decide whether an emergency exists; it records the prepared plans, classifications, exercises, actions and resource readiness needed to respond in a way that can be queried and audited.

Position in the backbone. This sector depends on CorePlatform, Security, Organization, ReactorFleet, AlarmManagement, EventManagement, RadiationMonitoring and Robotics. Later sectors such as Compliance, Reporting and Audit will read from it.

C.14.1

Purpose and boundary

C.14.2

Full table list

C.14.3

Lookup tables

C.14.4

SQL DDL

C.14.5

ER diagram

C.14.6

Foreign-key mapping

C.14.7

Indexes, seeds and verification queries

C.14.8

Honest boundary

C.14.1 Purpose and boundary

The schema answers seven questions: which emergency plans are active, which revision is approved, who belongs to which response team, which scenarios and exercises test the plan, where people must assemble, which protective actions were ordered, and which resources were ready when the response depended on them.

The model separates *preparedness* from *cause*. Root cause analysis explains why an event happened; EmergencyPreparedness records how the organization prepared for it and responded to it. The connection is by foreign key, not by duplication.

EmergencyPreparedness is not a replacement for procedure documents. It is the structured backbone that lets the platform join procedures, exercises, roles, routes, muster records, resources and actual events.

C.14.2 Full table list

PLANNING

EmergencyPreparedness.EmergencyPlan

PK EmergencyPlanId
FK PlanStatusId, SiteId, PlantId, OwnerUserId

The named emergency plan for a site or plant, with status, owner and validity window.

PLANNING

EmergencyPreparedness.EmergencyPlanRevision

PK EmergencyPlanRevisionId
FK EmergencyPlanId, PlanStatusId, PreparedByUserId, ApprovedByUserId

Versioned plan text and approval metadata. The active plan points to the current revision number.

CLASSIFICATION

EmergencyPreparedness.EmergencyClassification

PK EmergencyClassificationId
FK EmergencyClassTypeld, UnitId, OperationalEventId, IncidentId

A declared emergency class for an actual event, unit, incident or exercise boundary.

TEAMS

EmergencyPreparedness.EmergencyRole

PK EmergencyRoleId
FK none

Role catalogue for command, technical, radiological, logistics and communications duties.

TEAMS

EmergencyPreparedness.ResponseTeam

PK ResponseTeamId
FK SiteId, PlantId, TeamId

Emergency response team aligned with organization team records and plant scope.

TEAMS

EmergencyPreparedness.ResponseTeamMember

PK ResponseTeamMemberId
FK ResponseTeamId, PersonId, EmergencyRoleId

Person-to-team assignment with emergency role and validity window.

SCENARIO

EmergencyPreparedness.EmergencyScenario

PK EmergencyScenarioId
FK EmergencyClassTypeld, ScenarioStatusId, PrimaryUnitId

Reusable scenario library entry for exercises, training and preparedness analysis.

EXERCISE

EmergencyPreparedness.Exercise

PK ExerciseId
FK ExerciseTypeld, ExerciseStatusId, EmergencyScenarioId, SiteId, PlantId

A planned or completed emergency exercise with schedule, coordinator and scope.

EXERCISE**EmergencyPreparedness.ExerciseParticipant**

PK ExerciseParticipantId
FK Exerciseld, PersonId, ResponseTeamId,
EmergencyRoleId, ParticipantStatusId

Who took part in the exercise, in what capacity, and whether they attended.

EXERCISE**EmergencyPreparedness.ExerciseInject**

PK ExerciseInjectId
FK Exerciseld, InjectTypeId, DeliveredByUserId

Scripted exercise injects: alarms, messages, field reports and simulated failures.

EXERCISE**EmergencyPreparedness.ExerciseObservation**

PK ExerciseObservationId
FK Exerciseld, ExerciseInjectId, ObservationSeverityId,
ObservedByUserId

Observed strengths, gaps and deficiencies captured during an exercise.

EXERCISE**EmergencyPreparedness.ExerciseEvaluation**

PK ExerciseEvaluationId
FK Exerciseld, EvaluationStatusId, EvaluatedByUserId

Final evaluation, lessons learned, recommendations and overall score.

ROUTES**EmergencyPreparedness.AssemblyPoint**

PK AssemblyPointId
FK Siteld, PlantId, RadiationZoneId

Muster or assembly location, optionally tied to a radiological zone.

ROUTES**EmergencyPreparedness.EvacuationRoute**

PK EvacuationRouteId
FK Siteld, PlantId, AssemblyPointId, RouteStatusId

Named evacuation route from an area to an assembly point.

ROUTES**EmergencyPreparedness.EvacuationRouteZone**

PK EvacuationRouteZoneId
FK EvacuationRouteId, RadiationZoneId

Radiological zones intersected by a route and whether the route should be avoided when alarmed.

MUSTER**EmergencyPreparedness.MusterRecord**

PK MusterRecordId
FK Exerciseld, OperationalEventId, AssemblyPointId,
PersonId, MusterStatusId

Accountability record for people during an exercise or real event.

ACTIONS**EmergencyPreparedness.ProtectiveAction**

PK ProtectiveActionId
FK ProtectiveActionTypeId, ProtectiveActionStatusId,
EmergencyClassificationId, UnitId

Ordered protective actions such as evacuation, sheltering or access restriction.

COMMS**EmergencyPreparedness.NotificationTemplate**

PK NotificationTemplateId
FK NotificationChannelId, LanguageId

Reusable notification wording by channel and language.

COMMS EmergencyPreparedness.EmergencyCommunication PK EmergencyCommunicationId FK EmergencyClassificationId, ExercisId, NotificationTemplateId, NotificationChannelId Outbound message instance, recipient, status and failure reason.	RESOURCES EmergencyPreparedness.EmergencyResource PK EmergencyResourceId FK ResourceTypeId, ResourceStatusId, SiteId, PlantId, OwnerTeamId, EngineeringUnitId Vehicles, communications equipment, PPE, monitoring kits and other emergency resources.
RESOURCES EmergencyPreparedness.ResourceReadinessCheck PK ResourceReadinessCheckId FK EmergencyResourceId, ReadinessStatusId, CheckedByUserId Periodic readiness check for a resource.	INTEGRATION EmergencyPreparedness.PlanIncidentLink PK PlanIncidentLinkId FK EmergencyPlanId, IncidentId, LinkedByUserId Connects an emergency plan to an incident that used, tested or changed it.
INTEGRATION EmergencyPreparedness.PlanAlarmLink PK PlanAlarmLinkId FK EmergencyPlanId, AlarmEventId, LinkedByUserId Connects an emergency plan to an alarm trigger or historical alarm event.	INTEGRATION EmergencyPreparedness.PlanRobotMissionLink PK PlanRobotMissionLinkId FK EmergencyPlanId, MissionId, LinkedByUserId Connects emergency plans to robotics missions used for survey or access support.

C.14.3 Lookup tables

Preparedness data is status-heavy. The lookup tables below keep statuses, classifications and channels governed and joinable rather than scattered through free-text columns.

LOOKUP TABLE	TYPICAL SEED CODES	MAIN CONSUMER
EmergencyPreparedness.PlanStatus	DRAFT, ACTIVE, UNDER_REVIEW, SUPERSEDED, RETIRED	Referenced by EmergencyPlan and PlanRevision.
EmergencyPreparedness.EmergencyClassificationType	ALERT, SITE_AREA, GENERAL, SECURITY, RADIOLOGICAL	Referenced by EmergencyClassification and scenarios.
EmergencyPreparedness.ScenarioStatus	DRAFT, VALIDATED, IN_TRAINING, RETIRED	Referenced by EmergencyScenario.
EmergencyPreparedness.ExerciseType	TABLETOP, FUNCTIONAL, FULL_SCALE, DRILL, COMMUNICATIONS	Referenced by Exercise.
EmergencyPreparedness.ExerciseStatus	PLANNED, ACTIVE, COMPLETED, EVALUATED, CANCELLED	Referenced by Exercise.
EmergencyPreparedness.ParticipantStatus	INVITED, CONFIRMED, PRESENT, ABSENT, EXCUSED	Referenced by ExerciseParticipant.
EmergencyPreparedness.InjectType	ALARM, MESSAGE, FIELD_REPORT, RAD_READING, RESOURCE_FAILURE	Referenced by ExerciseInject.

LOOKUP TABLE	TYPICAL SEED CODES	MAIN CONSUMER
EmergencyPreparedness.ObservationSeverity	INFO , MINOR , MAJOR , CRITICAL	Referenced by ExerciseObservation.
EmergencyPreparedness.EvaluationStatus	DRAFT , REVIEWED , APPROVED , REWORK_REQUIRED	Referenced by ExerciseEvaluation.
EmergencyPreparedness.ProtectiveActionType	EVACUATE , SHELTER , IODINE , RESTRICT_ACCESS , STOP_WORK	Referenced by ProtectiveAction.
EmergencyPreparedness.ProtectiveActionStatus	ORDERED , IN_PROGRESS , COMPLETED , CANCELLED , FAILED	Referenced by ProtectiveAction.
EmergencyPreparedness.ResourceType	VEHICLE , COMMUNICATION , PPE , MEDICAL , MONITORING , POWER	Referenced by EmergencyResource.
EmergencyPreparedness.ResourceStatus	AVAILABLE , ASSIGNED , OUT_OF_SERVICE , CALIBRATION_DUE , RETIRED	Referenced by EmergencyResource.
EmergencyPreparedness.ReadinessStatus	READY , DEGRADED , NOT_READY , UNKNOWN	Referenced by ResourceReadinessCheck.
EmergencyPreparedness.RouteStatus	OPEN , RESTRICTED , BLOCKED , UNDER_REVIEW , RETIRED	Referenced by EvacuationRoute.
EmergencyPreparedness.MusterStatus	EXPECTED , ACCOUNTED_FOR , MISSING , RELEASED , MEDICAL	Referenced by MusterRecord.
EmergencyPreparedness.NotificationChannel	EMAIL , SMS , RADIO , PA , PHONE , SECURE_APP	Referenced by templates and communications.
EmergencyPreparedness.CommunicationStatus	QUEUED , SENT , DELIVERED , FAILED , ACKNOWLEDGED	Referenced by EmergencyCommunication.

C.14.4 SQL DDL

C.14.4.1 Schema

```
IF SCHEMA_ID(N'EmergencyPreparedness') IS NULL
EXEC(N'CREATE SCHEMA EmergencyPreparedness');
GO
```

C.14.4.2 EmergencyPreparedness lookup tables

```
CREATE TABLE EmergencyPreparedness.PlanStatus (
    PlanStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_PlanStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_PlanStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_PlanStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_PlanStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EmergencyPreparedness_PlanStatus
    PRIMARY KEY (PlanStatusId),
    CONSTRAINT UQ_EmergencyPreparedness_PlanStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE EmergencyPreparedness.EmergencyClassType (
    EmergencyClassTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_EmergencyClassType_DisplayOrder DEFAULT 0,
```

```

IsActive BIT NOT NULL
CONSTRAINT DF_EmergencyPreparedness_EmergencyClassType_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EmergencyPreparedness_EmergencyClassType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EmergencyPreparedness_EmergencyClassType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_EmergencyPreparedness_EmergencyClassType
PRIMARY KEY (EmergencyClassTypeId),
CONSTRAINT UQ_EmergencyPreparedness_EmergencyClassType_Code
UNIQUE (Code)
);
GO

CREATE TABLE EmergencyPreparedness.ScenarioStatus (
ScenarioStatusId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_EmergencyPreparedness_ScenarioStatus_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_EmergencyPreparedness_ScenarioStatus_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EmergencyPreparedness_ScenarioStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EmergencyPreparedness_ScenarioStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_EmergencyPreparedness_ScenarioStatus
PRIMARY KEY (ScenarioStatusId),
CONSTRAINT UQ_EmergencyPreparedness_ScenarioStatus_Code
UNIQUE (Code)
);
GO

CREATE TABLE EmergencyPreparedness.ExerciseType (
ExerciseTypeId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_EmergencyPreparedness_ExerciseType_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_EmergencyPreparedness_ExerciseType_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EmergencyPreparedness_ExerciseType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EmergencyPreparedness_ExerciseType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_EmergencyPreparedness_ExerciseType
PRIMARY KEY (ExerciseTypeId),
CONSTRAINT UQ_EmergencyPreparedness_ExerciseType_Code
UNIQUE (Code)
);
GO

CREATE TABLE EmergencyPreparedness.ExerciseStatus (
ExerciseStatusId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_EmergencyPreparedness_ExerciseStatus_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_EmergencyPreparedness_ExerciseStatus_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EmergencyPreparedness_ExerciseStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EmergencyPreparedness_ExerciseStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_EmergencyPreparedness_ExerciseStatus
PRIMARY KEY (ExerciseStatusId),
CONSTRAINT UQ_EmergencyPreparedness_ExerciseStatus_Code
UNIQUE (Code)
);
GO

CREATE TABLE EmergencyPreparedness.ParticipantStatus (
ParticipantStatusId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_EmergencyPreparedness_ParticipantStatus_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_EmergencyPreparedness_ParticipantStatus_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EmergencyPreparedness_ParticipantStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EmergencyPreparedness_ParticipantStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_EmergencyPreparedness_ParticipantStatus
PRIMARY KEY (ParticipantStatusId),
CONSTRAINT UQ_EmergencyPreparedness_ParticipantStatus_Code
UNIQUE (Code)
);
GO

CREATE TABLE EmergencyPreparedness.InjectType (
InjectTypeId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_EmergencyPreparedness_InjectType_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_EmergencyPreparedness_InjectType_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EmergencyPreparedness_InjectType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EmergencyPreparedness_InjectType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_EmergencyPreparedness_InjectType
PRIMARY KEY (InjectTypeId),
CONSTRAINT UQ_EmergencyPreparedness_InjectType_Code
UNIQUE (Code)
);
GO

CREATE TABLE EmergencyPreparedness.ObservationSeverity (
ObservationSeverityId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_EmergencyPreparedness_ObservationSeverity_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_EmergencyPreparedness_ObservationSeverity_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EmergencyPreparedness_ObservationSeverity_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_EmergencyPreparedness_ObservationSeverity_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

```

```

CONSTRAINT PK_EmergencyPreparedness_ObservationSeverity
PRIMARY KEY (ObservationSeverityId),
CONSTRAINT UQ_EmergencyPreparedness_ObservationSeverity_Code
UNIQUE (Code)
);
GO

CREATE TABLE EmergencyPreparedness.EvaluationStatus (
    EvaluationStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_EvaluationStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_EvaluationStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_EvaluationStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_EvaluationStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EmergencyPreparedness_EvaluationStatus
    PRIMARY KEY (EvaluationStatusId),
    CONSTRAINT UQ_EmergencyPreparedness_EvaluationStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE EmergencyPreparedness.ProtectiveActionType (
    ProtectiveActionTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_ProtectiveActionType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_ProtectiveActionType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_ProtectiveActionType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_ProtectiveActionType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EmergencyPreparedness_ProtectiveActionType
    PRIMARY KEY (ProtectiveActionTypeId),
    CONSTRAINT UQ_EmergencyPreparedness_ProtectiveActionType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE EmergencyPreparedness.ProtectiveActionStatus (
    ProtectiveActionStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_ProtectiveActionStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_ProtectiveActionStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_ProtectiveActionStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_ProtectiveActionStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EmergencyPreparedness_ProtectiveActionStatus
    PRIMARY KEY (ProtectiveActionStatusId),
    CONSTRAINT UQ_EmergencyPreparedness_ProtectiveActionStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE EmergencyPreparedness.ResourceType (
    ResourceTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_ResourceType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_ResourceType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_ResourceType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_ResourceType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EmergencyPreparedness_ResourceType
    PRIMARY KEY (ResourceTypeId),
    CONSTRAINT UQ_EmergencyPreparedness_ResourceType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE EmergencyPreparedness.ResourceStatus (
    ResourceStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_ResourceStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_ResourceStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_ResourceStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_ResourceStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EmergencyPreparedness_ResourceStatus
    PRIMARY KEY (ResourceStatusId),
    CONSTRAINT UQ_EmergencyPreparedness_ResourceStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE EmergencyPreparedness.ReadinessStatus (
    ReadinessStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_ReadinessStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_ReadinessStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_ReadinessStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_ReadinessStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EmergencyPreparedness_ReadinessStatus
    PRIMARY KEY (ReadinessStatusId),
    CONSTRAINT UQ_EmergencyPreparedness_ReadinessStatus_Code
    UNIQUE (Code)
);
GO

```



```

CREATE TABLE EmergencyPreparedness.RouteStatus (
    RouteStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_RouteStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_RouteStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_RouteStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_RouteStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EmergencyPreparedness_RouteStatus
        PRIMARY KEY (RouteStatusId),
    CONSTRAINT UQ_EmergencyPreparedness_RouteStatus_Code
        UNIQUE (Code)
);
GO

CREATE TABLE EmergencyPreparedness.MusterStatus (
    MusterStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_MusterStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_MusterStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_MusterStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_MusterStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EmergencyPreparedness_MusterStatus
        PRIMARY KEY (MusterStatusId),
    CONSTRAINT UQ_EmergencyPreparedness_MusterStatus_Code
        UNIQUE (Code)
);
GO

CREATE TABLE EmergencyPreparedness.NotificationChannel (
    NotificationChannelId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_NotificationChannel_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_NotificationChannel_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_NotificationChannel_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_NotificationChannel_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EmergencyPreparedness_NotificationChannel
        PRIMARY KEY (NotificationChannelId),
    CONSTRAINT UQ_EmergencyPreparedness_NotificationChannel_Code
        UNIQUE (Code)
);
GO

CREATE TABLE EmergencyPreparedness.CommunicationStatus (
    CommunicationStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_CommunicationStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_CommunicationStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_CommunicationStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EmergencyPreparedness_CommunicationStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EmergencyPreparedness_CommunicationStatus
        PRIMARY KEY (CommunicationStatusId),
    CONSTRAINT UQ_EmergencyPreparedness_CommunicationStatus_Code
        UNIQUE (Code)
);
GO

```

C.14.4.3 EmergencyPreparedness core tables

```

CREATE TABLE EmergencyPreparedness.EmergencyPlan (
    EmergencyPlanId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    PlanStatusId INT NOT NULL,
    SiteId INT NOT NULL,
    PlantId INT NOT NULL,
    OwnerUserId INT NOT NULL,
    CurrentRevisionNumber INT NOT NULL
    CONSTRAINT DF_EmergencyPlan_CurrentRevisionNumber DEFAULT 0,
    EffectiveFromUtc DATETIME2(7) NULL,
    EffectiveToUtc DATETIME2(7) NULL,
    Description NVARCHAR(1000) NULL,
    IsActive BIT NOT NULL
    CONSTRAINT DF_EmergencyPlan_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EmergencyPlan_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_EmergencyPlan_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL
    CONSTRAINT DF_EmergencyPlan_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EmergencyPreparedness_EmergencyPlan PRIMARY KEY (EmergencyPlanId),
    CONSTRAINT UQ_EmergencyPreparedness_EmergencyPlan_Code UNIQUE (Code),
    CONSTRAINT FK_EmergencyPlan_PlanStatus FOREIGN KEY (PlanStatusId)
        REFERENCES EmergencyPreparedness.PlanStatus (PlanStatusId),
    CONSTRAINT FK_EmergencyPlan_Site FOREIGN KEY (SiteId)
        REFERENCES Organization.Site (SiteId),
    CONSTRAINT FK_EmergencyPlan_Plant FOREIGN KEY (PlantId)
        REFERENCES Organization.Plant (PlantId),
    CONSTRAINT FK_EmergencyPlan_OwnerUser FOREIGN KEY (OwnerUserId)
        REFERENCES Security.ApplicationUser (ApplicationUserId),
    CONSTRAINT CK_EmergencyPlan_EffectiveWindow CHECK (EffectiveToUtc IS NULL OR EffectiveFromUtc IS NULL OR EffectiveToUtc >= EffectiveFromUtc)
);
GO

CREATE TABLE EmergencyPreparedness.EmergencyPlanRevision (
    EmergencyPlanRevisionId INT IDENTITY(1,1) NOT NULL,
    EmergencyPlanId INT NOT NULL,
    RevisionNumber INT NOT NULL,
    Title NVARCHAR(200) NOT NULL,
    PlanStatusId INT NOT NULL,
    PreparedByUserId INT NOT NULL,

```

```

PreparedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_EmergencyPlanRevision_PreparedAtUtc DEFAULT SYSUTCDATETIME(),
ApprovedByUserId INT NULL,
ApprovedAtUtc DATETIME2(7) NULL,
DocumentUri NVARCHAR(1800) NULL,
ChangeSummary NVARCHAR(2000) NULL,
CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_EmergencyPlanRevision_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
CreatedBy NVARCHAR(100) NOT NULL,
ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_EmergencyPlanRevision_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedBy NVARCHAR(100) NOT NULL,
IsDeleted BIT NOT NULL CONSTRAINT DF_EmergencyPlanRevision_IsDeleted DEFAULT 0,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_EmergencyPreparedness_EmergencyPlanRevision PRIMARY KEY (EmergencyPlanRevisionId),
CONSTRAINT UQ_EmergencyPlanRevision_Plan_Revision UNIQUE (EmergencyPlanId, RevisionNumber),
CONSTRAINT FK_EmergencyPlanRevision_EmergencyPlan FOREIGN KEY (EmergencyPlanId)
REFERENCES EmergencyPreparedness.EmergencyPlan (EmergencyPlanId),
CONSTRAINT FK_EmergencyPlanRevision_PlanStatus FOREIGN KEY (PlanStatusId)
REFERENCES EmergencyPreparedness.PlanStatus (PlanStatusId),
CONSTRAINT FK_EmergencyPlanRevision_PreparedByUser FOREIGN KEY (PreparedByUserId)
REFERENCES Security.ApplicationUser (ApplicationUserId),
CONSTRAINT FK_EmergencyPlanRevision_ApprovedByUser FOREIGN KEY (ApprovedByUserId)
REFERENCES Security.ApplicationUser (ApplicationUserId)
);
GO

CREATE TABLE EmergencyPreparedness.EmergencyClassification (
EmergencyClassificationId BIGINT IDENTITY(1,1) NOT NULL,
EmergencyClassTypeId INT NOT NULL,
UnitId INT NULL,
OperationalEventId BIGINT NULL,
IncidentId BIGINT NULL,
DeclaredAtUtc DATETIME2(7) NOT NULL,
DeclaredByUserId INT NOT NULL,
EndedAtUtc DATETIME2(7) NULL,
EndedByUserId INT NULL,
Summary NVARCHAR(2000) NULL,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_EmergencyPreparedness_EmergencyClassification PRIMARY KEY (EmergencyClassificationId),
CONSTRAINT FK_EmergencyClassification_ClassType FOREIGN KEY (EmergencyClassTypeId)
REFERENCES EmergencyPreparedness.EmergencyClassType (EmergencyClassTypeId),
CONSTRAINT FK_EmergencyClassification_Unit FOREIGN KEY (UnitId)
REFERENCES ReactorFleet.Unit (UnitId),
CONSTRAINT FK_EmergencyClassification_OperationalEvent FOREIGN KEY (OperationalEventId)
REFERENCES EventManagement.OperationalEvent (OperationalEventId),
CONSTRAINT FK_EmergencyClassification_Incident FOREIGN KEY (IncidentId)
REFERENCES EventManagement.Incident (IncidentId),
CONSTRAINT FK_EmergencyClassification_DeclaredByUser FOREIGN KEY (DeclaredByUserId)
REFERENCES Security.ApplicationUser (ApplicationUserId),
CONSTRAINT FK_EmergencyClassification_EndedByUser FOREIGN KEY (EndedByUserId)
REFERENCES Security.ApplicationUser (ApplicationUserId),
CONSTRAINT CK_EmergencyClassification_TimeWindow CHECK (EndedAtUtc IS NULL OR EndedAtUtc >= DeclaredAtUtc)
);
GO

CREATE TABLE EmergencyPreparedness.EmergencyRole (
EmergencyRoleId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
IsCommandRole BIT NOT NULL CONSTRAINT DF_EmergencyRole_IsCommandRole DEFAULT 0,
IsActive BIT NOT NULL CONSTRAINT DF_EmergencyRole_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_EmergencyRole_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_EmergencyRole_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_EmergencyPreparedness_EmergencyRole PRIMARY KEY (EmergencyRoleId),
CONSTRAINT UQ_EmergencyPreparedness_EmergencyRole_Code UNIQUE (Code)
);
GO

CREATE TABLE EmergencyPreparedness.ResponseTeam (
ResponseTeamId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(200) NOT NULL,
SiteId INT NOT NULL,
PlantId INT NULL,
TeamId INT NULL,
IsPrimary BIT NOT NULL CONSTRAINT DF_ResponseTeam_IsPrimary DEFAULT 0,
Description NVARCHAR(1000) NULL,
CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ResponseTeam_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
CreatedBy NVARCHAR(100) NOT NULL,
ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ResponseTeam_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedBy NVARCHAR(100) NOT NULL,
IsDeleted BIT NOT NULL CONSTRAINT DF_ResponseTeam_IsDeleted DEFAULT 0,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_EmergencyPreparedness_ResponseTeam PRIMARY KEY (ResponseTeamId),
CONSTRAINT UQ_EmergencyPreparedness_ResponseTeam_Code UNIQUE (Code),
CONSTRAINT FK_ResponseTeam_Site FOREIGN KEY (SiteId) REFERENCES Organization.Site (SiteId),
CONSTRAINT FK_ResponseTeam_Plant FOREIGN KEY (PlantId) REFERENCES Organization.Plant (PlantId),
CONSTRAINT FK_ResponseTeam_Team FOREIGN KEY (TeamId) REFERENCES Organization.Team (TeamId)
);
GO

CREATE TABLE EmergencyPreparedness.ResponseTeamMember (
ResponseTeamMemberId INT IDENTITY(1,1) NOT NULL,
ResponseTeamId INT NOT NULL,
PersonId INT NOT NULL,
EmergencyRoleId INT NOT NULL,
IsDeputy BIT NOT NULL CONSTRAINT DF_ResponseTeamMember_IsDeputy DEFAULT 0,
EffectiveFromUtc DATETIME2(7) NOT NULL,
EffectiveToUtc DATETIME2(7) NULL,
CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ResponseTeamMember_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
CreatedBy NVARCHAR(100) NOT NULL,
ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ResponseTeamMember_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedBy NVARCHAR(100) NOT NULL,
IsDeleted BIT NOT NULL CONSTRAINT DF_ResponseTeamMember_IsDeleted DEFAULT 0,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_EmergencyPreparedness_ResponseTeamMember PRIMARY KEY (ResponseTeamMemberId),
CONSTRAINT FK_ResponseTeamMember_ResponseTeam FOREIGN KEY (ResponseTeamId)
REFERENCES EmergencyPreparedness.ResponseTeam (ResponseTeamId),
CONSTRAINT FK_ResponseTeamMember_Person FOREIGN KEY (PersonId)
REFERENCES Organization.Person (PersonId),
CONSTRAINT FK_ResponseTeamMember_EmergencyRole FOREIGN KEY (EmergencyRoleId)
REFERENCES EmergencyPreparedness.EmergencyRole (EmergencyRoleId),
CONSTRAINT CK_ResponseTeamMember_EffectiveWindow CHECK (EffectiveToUtc IS NULL OR EffectiveToUtc >= EffectiveFromUtc)
);
GO

CREATE TABLE EmergencyPreparedness.EmergencyScenario (
EmergencyScenarioId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(200) NOT NULL,
EmergencyClassTypeId INT NOT NULL,
ScenarioStatusId INT NOT NULL,
PrimaryUnitId INT NULL,
Description NVARCHAR(2000) NULL,
Assumptions NVARCHAR(2000) NULL,
CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_EmergencyScenario_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
CreatedBy NVARCHAR(100) NOT NULL,
ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_EmergencyScenario_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedBy NVARCHAR(100) NOT NULL,
IsDeleted BIT NOT NULL CONSTRAINT DF_EmergencyScenario_IsDeleted DEFAULT 0,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_EmergencyPreparedness_EmergencyScenario PRIMARY KEY (EmergencyScenarioId),
CONSTRAINT UQ_EmergencyPreparedness_EmergencyScenario_Code UNIQUE (Code),
CONSTRAINT FK_EmergencyScenario_ClassType FOREIGN KEY (EmergencyClassTypeId)
REFERENCES EmergencyPreparedness.EmergencyClassType (EmergencyClassTypeId),
CONSTRAINT FK_EmergencyScenario_Status FOREIGN KEY (ScenarioStatusId)
REFERENCES EmergencyPreparedness.ScenarioStatus (ScenarioStatusId),
CONSTRAINT FK_EmergencyScenario_PrimaryUnit FOREIGN KEY (PrimaryUnitId)
REFERENCES ReactorFleet.Unit (UnitId)
);

```

```

);
GO

CREATE TABLE EmergencyPreparedness.Exercise (
    ExerciseId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    ExerciseTypeId INT NOT NULL,
    ExerciseStatusId INT NOT NULL,
    EmergencyScenarioId INT NULL,
    SiteId INT NOT NULL,
    PlantId INT NULL,
    ScheduledStartUtc DATETIME2(7) NOT NULL,
    ScheduledEndUtc DATETIME2(7) NOT NULL,
    ActualStartUtc DATETIME2(7) NULL,
    ActualEndUtc DATETIME2(7) NULL,
    CoordinatorUserId INT NOT NULL,
    Summary NVARCHAR(2000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Exercise_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Exercise_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Exercise_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EmergencyPreparedness_Exercise PRIMARY KEY (ExerciseId),
    CONSTRAINT UQ_EmergencyPreparedness_Exercise_Code UNIQUE (Code),
    CONSTRAINT FK_Exercise_ExerciseType FOREIGN KEY (ExerciseTypeId) REFERENCES EmergencyPreparedness.ExerciseType(ExerciseTypeId),
    CONSTRAINT FK_Exercise_ExerciseStatus FOREIGN KEY (ExerciseStatusId) REFERENCES EmergencyPreparedness.ExerciseStatus(ExerciseStatusId),
    CONSTRAINT FK_Exercise_EmergencyScenario FOREIGN KEY (EmergencyScenarioId) REFERENCES EmergencyPreparedness.EmergencyScenario(EmergencyScenarioId),
    CONSTRAINT FK_Exercise_Site FOREIGN KEY (SiteId) REFERENCES Organization.Site(SiteId),
    CONSTRAINT FK_Exercise_Plant FOREIGN KEY (PlantId) REFERENCES Organization.Plant(PlantId),
    CONSTRAINT FK_Exercise_CoordinatorUser FOREIGN KEY (CoordinatorUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_Exercise_ScheduledWindow CHECK (ScheduledEndUtc >= ScheduledStartUtc),
    CONSTRAINT CK_Exercise_ActualWindow CHECK (ActualEndUtc IS NULL OR ActualStartUtc IS NULL OR ActualEndUtc >= ActualStartUtc)
);
GO

CREATE TABLE EmergencyPreparedness.ExerciseParticipant (
    ExerciseParticipantId INT IDENTITY(1,1) NOT NULL,
    ExerciseId INT NOT NULL,
    PersonId INT NULL,
    ResponseTeamId INT NULL,
    EmergencyRoleId INT NULL,
    ParticipantStatusId INT NOT NULL,
    Notes NVARCHAR(1000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ExerciseParticipant_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ExerciseParticipant_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_ExerciseParticipant_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EmergencyPreparedness_ExerciseParticipant PRIMARY KEY (ExerciseParticipantId),
    CONSTRAINT FK_ExerciseParticipant_Exercise FOREIGN KEY (ExerciseId) REFERENCES EmergencyPreparedness.Exercise(ExerciseId),
    CONSTRAINT FK_ExerciseParticipant_Person FOREIGN KEY (PersonId) REFERENCES Organization.Person(PersonId),
    CONSTRAINT FK_ExerciseParticipant_ResponseTeam FOREIGN KEY (ResponseTeamId) REFERENCES EmergencyPreparedness.ResponseTeam(ResponseTeamId),
    CONSTRAINT FK_ExerciseParticipant_EmergencyRole FOREIGN KEY (EmergencyRoleId) REFERENCES EmergencyPreparedness.EmergencyRole(EmergencyRoleId),
    CONSTRAINT FK_ExerciseParticipant_Status FOREIGN KEY (ParticipantStatusId) REFERENCES EmergencyPreparedness.ParticipantStatus(ParticipantStatusId)
);
GO

CREATE TABLE EmergencyPreparedness.ExerciseInject (
    ExerciseInjectId INT IDENTITY(1,1) NOT NULL,
    ExerciseId INT NOT NULL,
    InjectTypeId INT NOT NULL,
    SequenceNumber INT NOT NULL,
    PlannedAtMinute INT NOT NULL,
    DeliveredAtUtc DATETIME2(7) NULL,
    DeliveredByUserId INT NULL,
    Description NVARCHAR(2000) NOT NULL,
    ExpectedAction NVARCHAR(2000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ExerciseInject_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ExerciseInject_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_ExerciseInject_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EmergencyPreparedness_ExerciseInject PRIMARY KEY (ExerciseInjectId),
    CONSTRAINT UQ_ExerciseInject_Exercise_Sequence UNIQUE (ExerciseId, SequenceNumber),
    CONSTRAINT FK_ExerciseInject_Exercise FOREIGN KEY (ExerciseId) REFERENCES EmergencyPreparedness.Exercise(ExerciseId),
    CONSTRAINT FK_ExerciseInject_InjectType FOREIGN KEY (InjectTypeId) REFERENCES EmergencyPreparedness.InjectType(InjectTypeId),
    CONSTRAINT FK_ExerciseInject_DeliveredByUser FOREIGN KEY (DeliveredByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_ExerciseInject_PlannedAtMinute CHECK (PlannedAtMinute >= 0)
);
GO

CREATE TABLE EmergencyPreparedness.ExerciseObservation (
    ExerciseObservationId INT IDENTITY(1,1) NOT NULL,
    ExerciseId INT NOT NULL,
    ExerciseInjectId INT NULL,
    ObservationSeverityId INT NOT NULL,
    ObservedByUserId INT NOT NULL,
    ObservedAtUtc DATETIME2(7) NOT NULL,
    FindingText NVARCHAR(3000) NOT NULL,
    CorrectiveActionRequired BIT NOT NULL CONSTRAINT DF_ExerciseObservation_CorrectiveActionRequired DEFAULT 0,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ExerciseObservation_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ExerciseObservation_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_ExerciseObservation_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EmergencyPreparedness_ExerciseObservation PRIMARY KEY (ExerciseObservationId),
    CONSTRAINT FK_ExerciseObservation_Exercise FOREIGN KEY (ExerciseId) REFERENCES EmergencyPreparedness.Exercise(ExerciseId),
    CONSTRAINT FK_ExerciseObservation_ExerciseInject FOREIGN KEY (ExerciseInjectId) REFERENCES EmergencyPreparedness.ExerciseInject(ExerciseInjectId),
    CONSTRAINT FK_ExerciseObservation_Severity FOREIGN KEY (ObservationSeverityId) REFERENCES EmergencyPreparedness.ObservationSeverity(ObservationSeverityId),
    CONSTRAINT FK_ExerciseObservation_ObservedByUser FOREIGN KEY (ObservedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE EmergencyPreparedness.ExerciseEvaluation (
    ExerciseEvaluationId INT IDENTITY(1,1) NOT NULL,
    ExerciseId INT NOT NULL,
    EvaluationStatusId INT NOT NULL,
    EvaluatedByUserId INT NOT NULL,
    EvaluatedAtUtc DATETIME2(7) NOT NULL,
    OverallScore DECIMAL(5,2) NULL,
    LessonsLearned NVARCHAR(3000) NULL,
    Recommendations NVARCHAR(3000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ExerciseEvaluation_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ExerciseEvaluation_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_ExerciseEvaluation_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EmergencyPreparedness_ExerciseEvaluation PRIMARY KEY (ExerciseEvaluationId),
    CONSTRAINT UQ_ExerciseEvaluation_Exercise UNIQUE (ExerciseId),
    CONSTRAINT FK_ExerciseEvaluation_Exercise FOREIGN KEY (ExerciseId) REFERENCES EmergencyPreparedness.Exercise(ExerciseId),
    CONSTRAINT FK_ExerciseEvaluation_Status FOREIGN KEY (EvaluationStatusId) REFERENCES EmergencyPreparedness.EvaluationStatus(EvaluationStatusId),
    CONSTRAINT FK_ExerciseEvaluation_EvaluatedByUser FOREIGN KEY (EvaluatedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_ExerciseEvaluation_OverallScore CHECK (OverallScore IS NULL OR (OverallScore >= 0 AND OverallScore <= 100))
);
GO

CREATE TABLE EmergencyPreparedness.AssemblyPoint (
    AssemblyPointId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    SiteId INT NOT NULL,
    PlantId INT NULL,
    RadiationZoneId INT NULL,
    MaxOccupancy INT NULL,

```

```

IsIndoor BIT NOT NULL CONSTRAINT DF_AssemblyPoint_IsIndoor DEFAULT 0,
Latitude DECIMAL(9,6) NULL,
Longitude DECIMAL(9,6) NULL,
Description NVARCHAR(1000) NULL,
CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AssemblyPoint_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
CreatedBy NVARCHAR(100) NOT NULL,
ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AssemblyPoint_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedBy NVARCHAR(100) NOT NULL,
IsDeleted BIT NOT NULL CONSTRAINT DF_AssemblyPoint_IsDeleted DEFAULT 0,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_EmergencyPreparedness_AssemblyPoint PRIMARY KEY (AssemblyPointId),
CONSTRAINT UQ_EmergencyPreparedness_AssemblyPoint_Code UNIQUE (Code),
CONSTRAINT FK_AssemblyPoint_Site FOREIGN KEY (SiteId) REFERENCES Organization.Site(SiteId),
CONSTRAINT FK_AssemblyPoint_Plant FOREIGN KEY (PlantId) REFERENCES Organization.Plant(PlantId),
CONSTRAINT FK_AssemblyPoint_RadiationZone FOREIGN KEY (RadiationZoneId) REFERENCES RadiationMonitoring.RadiationZone(RadiationZoneId),
CONSTRAINT CK_AssemblyPoint_MaxOccupancy CHECK (MaxOccupancy IS NULL OR MaxOccupancy >= 0)
);
GO

CREATE TABLE EmergencyPreparedness.EvacuationRoute (
    EvacuationRouteId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    SiteId INT NOT NULL,
    PlantId INT NOT NULL,
    AssemblyPointId INT NOT NULL,
    RouteStatusId INT NOT NULL,
    FromLocation NVARCHAR(200) NOT NULL,
    EstimatedMinutes INT NOT NULL,
    RouteGeometryJson NVARCHAR(MAX) NULL,
    Notes NVARCHAR(1000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_EvacuationRoute_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_EvacuationRoute_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_EvacuationRoute_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EmergencyPreparedness_EvacuationRoute PRIMARY KEY (EvacuationRouteId),
    CONSTRAINT UQ_EmergencyPreparedness_EvacuationRoute_Code UNIQUE (Code),
    CONSTRAINT FK_EvacuationRoute_Site FOREIGN KEY (SiteId) REFERENCES Organization.Site(SiteId),
    CONSTRAINT FK_EvacuationRoute_Plant FOREIGN KEY (PlantId) REFERENCES Organization.Plant(PlantId),
    CONSTRAINT FK_EvacuationRoute_AssemblyPoint FOREIGN KEY (AssemblyPointId) REFERENCES EmergencyPreparedness.AssemblyPoint(AssemblyPointId),
    CONSTRAINT FK_EvacuationRoute_RouteStatus FOREIGN KEY (RouteStatusId) REFERENCES EmergencyPreparedness.RouteStatus(RouteStatusId),
    CONSTRAINT CK_EvacuationRoute_EstimatedMinutes CHECK (EstimatedMinutes IS NULL OR EstimatedMinutes >= 0)
);
GO

CREATE TABLE EmergencyPreparedness.EvacuationRouteZone (
    EvacuationRouteZoneId INT IDENTITY(1,1) NOT NULL,
    EvacuationRouteId INT NOT NULL,
    RadiationZoneId INT NOT NULL,
    IsAvoidIfAlarmed BIT NOT NULL CONSTRAINT DF_EvacuationRouteZone_IsAvoidIfAlarmed DEFAULT 1,
    Notes NVARCHAR(500) NULL,

    CONSTRAINT PK_EmergencyPreparedness_EvacuationRouteZone PRIMARY KEY (EvacuationRouteZoneId),
    CONSTRAINT UQ_EvacuationRouteZone_Route_Zone UNIQUE (EvacuationRouteId, RadiationZoneId),
    CONSTRAINT FK_EvacuationRouteZone_EvacuationRoute FOREIGN KEY (EvacuationRouteId) REFERENCES EmergencyPreparedness.EvacuationRoute(EvacuationRouteId),
    CONSTRAINT FK_EvacuationRouteZone_RadiationZone FOREIGN KEY (RadiationZoneId) REFERENCES RadiationMonitoring.RadiationZone(RadiationZoneId)
);
GO

CREATE TABLE EmergencyPreparedness.MusterRecord (
    MusterRecordId BIGINT IDENTITY(1,1) NOT NULL,
    ExerciseId INT NOT NULL,
    OperationalEventId BIGINT NULL,
    AssemblyPointId INT NOT NULL,
    PersonId INT NOT NULL,
    MusterStatusId INT NOT NULL,
    RecordedAtUtc DATETIME2(7) NOT NULL,
    RecordedByUserId INT NOT NULL,
    Remarks NVARCHAR(1000) NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EmergencyPreparedness_MusterRecord PRIMARY KEY (MusterRecordId),
    CONSTRAINT FK_MusterRecord_Exercise FOREIGN KEY (ExerciseId) REFERENCES EmergencyPreparedness.Exercise(ExerciseId),
    CONSTRAINT FK_MusterRecord_OperationalEvent FOREIGN KEY (OperationalEventId) REFERENCES EventManagement.OperationalEvent(OperationalEventId),
    CONSTRAINT FK_MusterRecord_AssemblyPoint FOREIGN KEY (AssemblyPointId) REFERENCES EmergencyPreparedness.AssemblyPoint(AssemblyPointId),
    CONSTRAINT FK_MusterRecord_Person FOREIGN KEY (PersonId) REFERENCES Organization.Person(PersonId),
    CONSTRAINT FK_MusterRecord_MusterStatus FOREIGN KEY (MusterStatusId) REFERENCES EmergencyPreparedness.MusterStatus(MusterStatusId),
    CONSTRAINT FK_MusterRecord_RecordedByUser FOREIGN KEY (RecordedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE EmergencyPreparedness.ProtectiveAction (
    ProtectiveActionId BIGINT IDENTITY(1,1) NOT NULL,
    ProtectiveActionTypeId INT NOT NULL,
    ProtectiveActionStatusId INT NOT NULL,
    EmergencyClassificationId BIGINT NULL,
    OperationalEventId BIGINT NULL,
    UnitId INT NOT NULL,
    OrderedAtUtc DATETIME2(7) NOT NULL,
    OrderedByUserId INT NOT NULL,
    CompletedAtUtc DATETIME2(7) NULL,
    CompletedByUserId INT NOT NULL,
    ActionText NVARCHAR(2000) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EmergencyPreparedness_ProtectiveAction PRIMARY KEY (ProtectiveActionId),
    CONSTRAINT FK_ProtectiveAction_Type FOREIGN KEY (ProtectiveActionTypeId) REFERENCES EmergencyPreparedness.ProtectiveActionType(ProtectiveActionTypeId),
    CONSTRAINT FK_ProtectiveAction_Status FOREIGN KEY (ProtectiveActionStatusId) REFERENCES EmergencyPreparedness.ProtectiveActionStatus(ProtectiveActionStatusId),
    CONSTRAINT FK_ProtectiveAction_EmergencyClassification FOREIGN KEY (EmergencyClassificationId) REFERENCES EmergencyPreparedness.EmergencyClassification(EmergencyClassificationId),
    CONSTRAINT FK_ProtectiveAction_OperationalEvent FOREIGN KEY (OperationalEventId) REFERENCES EventManagement.OperationalEvent(OperationalEventId),
    CONSTRAINT FK_ProtectiveAction_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_ProtectiveAction_OrderedByUser FOREIGN KEY (OrderedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT FK_ProtectiveAction_CompletedByUser FOREIGN KEY (CompletedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_ProtectiveAction_TimeWindow CHECK (CompletedAtUtc IS NULL OR CompletedAtUtc >= OrderedAtUtc)
);
GO

CREATE TABLE EmergencyPreparedness.NotificationTemplate (
    NotificationTemplateId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    NotificationChannelId INT NOT NULL,
    LanguageId INT NOT NULL,
    SubjectTemplate NVARCHAR(300) NULL,
    BodyTemplate NVARCHAR(MAX) NOT NULL,
    IsActive BIT NOT NULL CONSTRAINT DF_NotificationTemplate_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_NotificationTemplate_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_NotificationTemplate_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_NotificationTemplate_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EmergencyPreparedness_NotificationTemplate PRIMARY KEY (NotificationTemplateId),
    CONSTRAINT UQ_NotificationTemplate_Code_Channel_Language UNIQUE (Code, NotificationChannelId, LanguageId),
    CONSTRAINT FK_NotificationTemplate_Channel FOREIGN KEY (NotificationChannelId) REFERENCES EmergencyPreparedness.NotificationChannel(NotificationChannelId),
    CONSTRAINT FK_NotificationTemplate_Language FOREIGN KEY (LanguageId) REFERENCES CorePlatform.Language(LanguageId)
);
GO

CREATE TABLE EmergencyPreparedness.EmergencyCommunication (
    EmergencyCommunicationId BIGINT IDENTITY(1,1) NOT NULL,
    EmergencyClassificationId BIGINT NULL,
    ExerciseId INT NOT NULL,
    NotificationTemplateId INT NOT NULL,
    NotificationChannelId INT NOT NULL,
    CommunicationStatusId INT NOT NULL,
    SentAtUtc DATETIME2(7) NULL,
    SentByUserId INT NOT NULL,
    RecipientPersonId INT NOT NULL,

```

```

RecipientAddress NVARCHAR(300) NOT NULL,
Subject NVARCHAR(300) NULL,
Body NVARCHAR(MAX) NOT NULL,
FailureReason NVARCHAR(1000) NULL,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_EmergencyPreparedness_EmergencyCommunication PRIMARY KEY (EmergencyCommunicationId),
CONSTRAINT FK_EmergencyCommunication_EmergencyClassification FOREIGN KEY (EmergencyClassificationId) REFERENCES
EmergencyPreparedness.EmergencyClassification(EmergencyClassificationId),
CONSTRAINT FK_EmergencyCommunication_Exercise FOREIGN KEY (ExerciseId) REFERENCES EmergencyPreparedness.Exercise(ExerciseId),
CONSTRAINT FK_EmergencyCommunication_Template FOREIGN KEY (NotificationTemplateId) REFERENCES EmergencyPreparedness.NotificationTemplate(NotificationTemplateId),
CONSTRAINT FK_EmergencyCommunication_Channel FOREIGN KEY (NotificationChannelId) REFERENCES EmergencyPreparedness.NotificationChannel(NotificationChannelId),
CONSTRAINT FK_EmergencyCommunication_Status FOREIGN KEY (CommunicationStatusId) REFERENCES EmergencyPreparedness.CommunicationStatus(CommunicationStatusId),
CONSTRAINT FK_EmergencyCommunication_SentByUser FOREIGN KEY (SentByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
CONSTRAINT FK_EmergencyCommunication_RecipientPerson FOREIGN KEY (RecipientPersonId) REFERENCES Organization.Person(PersonId)
);
GO

CREATE TABLE EmergencyPreparedness.EmergencyResource (
    EmergencyResourceId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    ResourceType INT NOT NULL,
    ResourceTypeId INT NOT NULL,
    ResourceStatusId INT NOT NULL,
    SiteId INT NOT NULL,
    PlantId INT NOT NULL,
    OwnerTeamId INT NOT NULL,
    QuantityOnHand DECIMAL(18,3) NULL,
    EngineeringUnitId INT NOT NULL,
    LocationText NVARCHAR(300) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_EmergencyResource_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_EmergencyResource_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_EmergencyResource_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EmergencyPreparedness_EmergencyResource PRIMARY KEY (EmergencyResourceId),
    CONSTRAINT UQ_EmergencyPreparedness_EmergencyResource_Code UNIQUE (Code),
    CONSTRAINT FK_EmergencyResource_ResourceType FOREIGN KEY (ResourceTypeId) REFERENCES EmergencyPreparedness.ResourceType(ResourceTypeId),
    CONSTRAINT FK_EmergencyResource_ResourceStatus FOREIGN KEY (ResourceStatusId) REFERENCES EmergencyPreparedness.ResourceStatus(ResourceStatusId),
    CONSTRAINT FK_EmergencyResource_Site FOREIGN KEY (SiteId) REFERENCES Organization.Site(SiteId),
    CONSTRAINT FK_EmergencyResource_Plant FOREIGN KEY (PlantId) REFERENCES Organization.Plant(PlantId),
    CONSTRAINT FK_EmergencyResource_OwnerTeam FOREIGN KEY (OwnerTeamId) REFERENCES Organization.Team(TeamId),
    CONSTRAINT FK_EmergencyResource_EngineeringUnit FOREIGN KEY (EngineeringUnitId) REFERENCES CorePlatform.EngineeringUnit(EngineeringUnitId),
    CONSTRAINT CK_EmergencyResource_Quantity CHECK (QuantityOnHand IS NULL OR QuantityOnHand >= 0)
);
GO

CREATE TABLE EmergencyPreparedness.ResourceReadinessCheck (
    ResourceReadinessCheckId BIGINT IDENTITY(1,1) NOT NULL,
    EmergencyResourceId INT NOT NULL,
    ReadinessStatusId INT NOT NULL,
    CheckedAtUtc DATETIME2(7) NOT NULL,
    CheckedByUserId INT NOT NULL,
    ConditionSummary NVARCHAR(2000) NOT NULL,
    NextCheckDueUtc DATETIME2(7) NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_EmergencyPreparedness_ResourceReadinessCheck PRIMARY KEY (ResourceReadinessCheckId),
    CONSTRAINT FK_ResourceReadinessCheck_Resource FOREIGN KEY (EmergencyResourceId) REFERENCES EmergencyPreparedness.EmergencyResource(EmergencyResourceId),
    CONSTRAINT FK_ResourceReadinessCheck_Status FOREIGN KEY (ReadinessStatusId) REFERENCES EmergencyPreparedness.ReadinessStatus(ReadinessStatusId),
    CONSTRAINT FK_ResourceReadinessCheck_CheckedByUser FOREIGN KEY (CheckedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE EmergencyPreparedness.PlanIncidentLink (
    PlanIncidentLinkId BIGINT IDENTITY(1,1) NOT NULL,
    EmergencyPlanId INT NOT NULL,
    IncidentId BIGINT NOT NULL,
    LinkedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_PlanIncidentLink_LinkedAtUtc DEFAULT SYSUTCDATETIME(),
    LinkedByUserId INT NOT NULL,
    Purpose NVARCHAR(500) NULL,

    CONSTRAINT PK_EmergencyPreparedness_PlanIncidentLink PRIMARY KEY (PlanIncidentLinkId),
    CONSTRAINT UQ_PlanIncidentLink_Plan_Incident UNIQUE (EmergencyPlanId, IncidentId),
    CONSTRAINT FK_PlanIncidentLink_EmergencyPlan FOREIGN KEY (EmergencyPlanId) REFERENCES EmergencyPreparedness.EmergencyPlan(EmergencyPlanId),
    CONSTRAINT FK_PlanIncidentLink_Incident FOREIGN KEY (IncidentId) REFERENCES EventManagement.Incident(IncidentId),
    CONSTRAINT FK_PlanIncidentLink_LinkedByUser FOREIGN KEY (LinkedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE EmergencyPreparedness.PlanAlarmLink (
    PlanAlarmLinkId BIGINT IDENTITY(1,1) NOT NULL,
    EmergencyPlanId INT NOT NULL,
    AlarmEventId BIGINT NOT NULL,
    LinkedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_PlanAlarmLink_LinkedAtUtc DEFAULT SYSUTCDATETIME(),
    LinkedByUserId INT NOT NULL,
    Purpose NVARCHAR(500) NULL,

    CONSTRAINT PK_EmergencyPreparedness_PlanAlarmLink PRIMARY KEY (PlanAlarmLinkId),
    CONSTRAINT UQ_PlanAlarmLink_Plan_Alarm UNIQUE (EmergencyPlanId, AlarmEventId),
    CONSTRAINT FK_PlanAlarmLink_EmergencyPlan FOREIGN KEY (EmergencyPlanId) REFERENCES EmergencyPreparedness.EmergencyPlan(EmergencyPlanId),
    CONSTRAINT FK_PlanAlarmLink_AlarmEvent FOREIGN KEY (AlarmEventId) REFERENCES AlarmManagement.AlarmEvent(AlarmEventId),
    CONSTRAINT FK_PlanAlarmLink_LinkedByUser FOREIGN KEY (LinkedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE EmergencyPreparedness.PlanRobotMissionLink (
    PlanRobotMissionLinkId BIGINT IDENTITY(1,1) NOT NULL,
    EmergencyPlanId INT NOT NULL,
    MissionId BIGINT NOT NULL,
    LinkedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_PlanRobotMissionLink_LinkedAtUtc DEFAULT SYSUTCDATETIME(),
    LinkedByUserId INT NOT NULL,
    Purpose NVARCHAR(500) NULL,

    CONSTRAINT PK_EmergencyPreparedness_PlanRobotMissionLink PRIMARY KEY (PlanRobotMissionLinkId),
    CONSTRAINT UQ_PlanRobotMissionLink_Plan_Mission UNIQUE (EmergencyPlanId, MissionId),
    CONSTRAINT FK_PlanRobotMissionLink_EmergencyPlan FOREIGN KEY (EmergencyPlanId) REFERENCES EmergencyPreparedness.EmergencyPlan(EmergencyPlanId),
    CONSTRAINT FK_PlanRobotMissionLink_Mission FOREIGN KEY (MissionId) REFERENCES Robotics.Mission(MissionId),
    CONSTRAINT FK_PlanRobotMissionLink_LinkedByUser FOREIGN KEY (LinkedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId)
);
GO

```

C.14.5 ER diagram

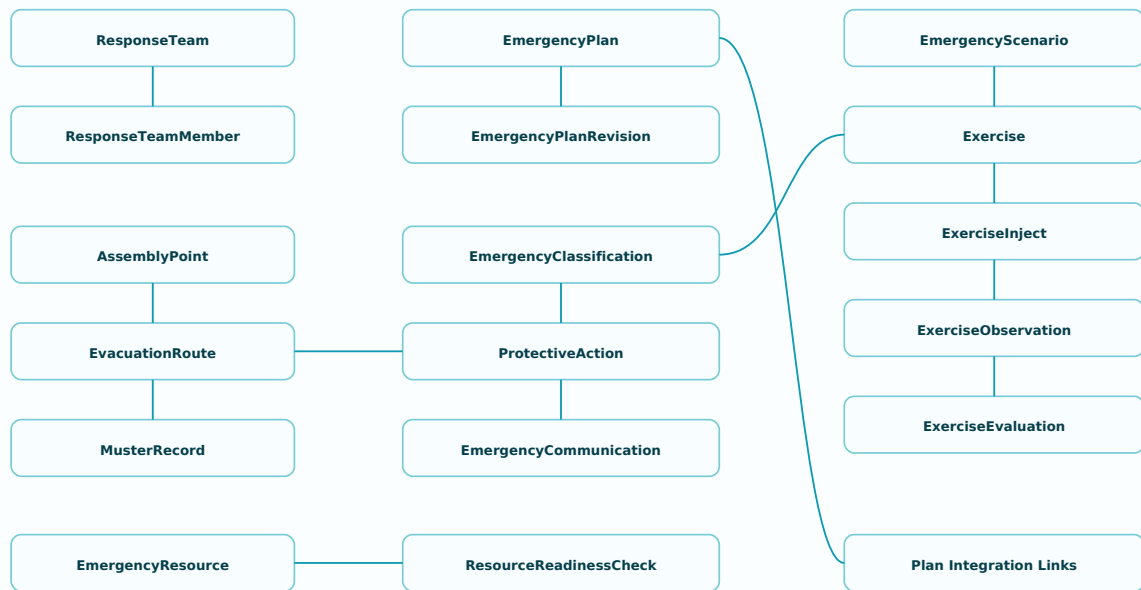


Figure C.14.1 - Main EmergencyPreparedness tables. Planning, exercise, classification, route, muster, communication and resource records all remain inside one schema, while cross-sector facts arrive by foreign key.

C.14.6 Foreign-key passport map

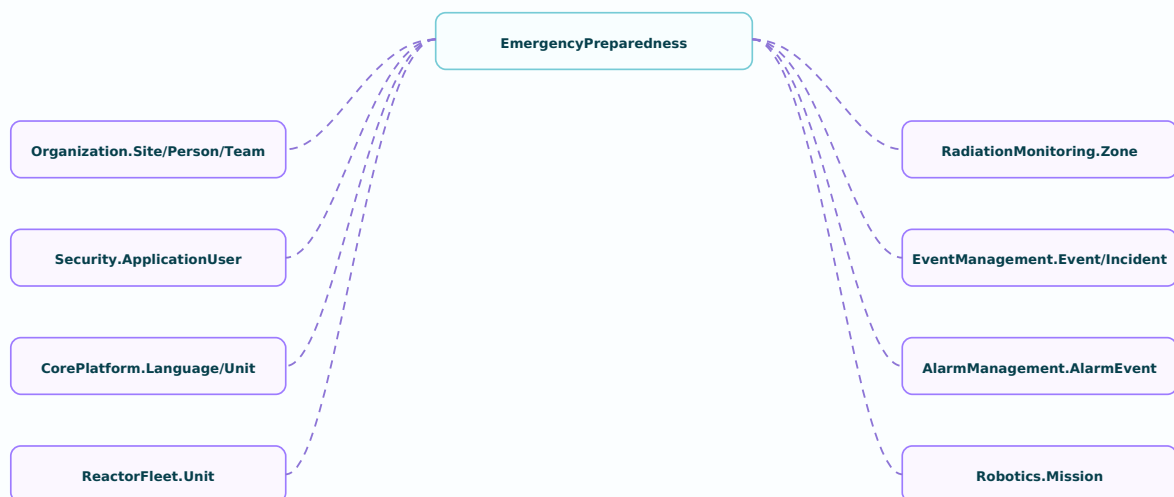


Figure C.14.2 - Passport dependencies. Dashed violet relationships cross schema boundaries into organization, security, platform, reactor fleet, radiation monitoring, event management, alarm management and robotics.

FROM TABLE	FK COLUMN	TO TABLE	RELATIONSHIP
EmergencyPreparedness.EmergencyPlan	PlanStatusId	PlanStatus	plan lifecycle state
EmergencyPreparedness.EmergencyPlan	SiteId / PlantId	Organization.Site / Organization.Plant	site and plant scope
EmergencyPreparedness.EmergencyPlan	OwnerUserId	Security.ApplicationUser	accountable owner
EmergencyPreparedness.EmergencyPlanRevision	EmergencyPlanId	EmergencyPlan	revision belongs to one plan
EmergencyPreparedness.EmergencyClassification	UnitId	ReactorFleet.Unit	declared emergency unit
EmergencyPreparedness.EmergencyClassification	OperationalEventId / IncidentId	EventManagement.OperationalEvent / Incident	event or incident context
EmergencyPreparedness.ResponseTeam	TeamId	Organization.Team	organizational team alignment
EmergencyPreparedness.ResponseTeamMember	PersonId	Organization.Person	emergency team member
EmergencyPreparedness.EmergencyScenario	PrimaryUnitId	ReactorFleet.Unit	scenario anchor unit
EmergencyPreparedness.Exercise	EmergencyScenarioId	EmergencyScenario	exercise scenario
EmergencyPreparedness.Exercise	SiteId / PlantId	Organization.Site / Organization.Plant	exercise scope
EmergencyPreparedness.ExerciseParticipant	ExerciseId	Exercise	participant belongs to exercise
EmergencyPreparedness.ExerciseInject	ExerciseId	Exercise	inject belongs to exercise
EmergencyPreparedness.ExerciseObservation	ExerciseInjectId	ExerciseInject	observation triggered by inject
EmergencyPreparedness.AssemblyPoint	RadiationZoneId	RadiationMonitoring.RadiationZone	radiological location context
EmergencyPreparedness.EvacuationRoute	AssemblyPointId	AssemblyPoint	route destination
EmergencyPreparedness.EvacuationRouteZone	RadiationZoneId	RadiationMonitoring.RadiationZone	route crosses or touches zone
EmergencyPreparedness.MusterRecord	OperationalEventId	EventManagement.OperationalEvent	real-event accountability
EmergencyPreparedness.MusterRecord	PersonId	Organization.Person	person accounted for
EmergencyPreparedness.ProtectiveAction	EmergencyClassificationId	EmergencyClassification	action ordered under classification
EmergencyPreparedness.NotificationTemplate	LanguageId	CorePlatform.Language	localized message template
EmergencyPreparedness.EmergencyCommunication	RecipientPersonId	Organization.Person	message recipient
EmergencyPreparedness.EmergencyResource	EngineeringUnitId	CorePlatform.EngineeringUnit	resource quantity unit
EmergencyPreparedness.EmergencyResource	OwnerTeamId	Organization.Team	owning team
EmergencyPreparedness.ResourceReadinessCheck	CheckedByUserId	Security.ApplicationUser	readiness inspector
EmergencyPreparedness.PlanIncidentLink	IncidentId	EventManagement.Incident	plan used or revised after incident
EmergencyPreparedness.PlanAlarmLink	AlarmEventId	AlarmManagement.AlarmEvent	alarm trigger or historical alarm context
	MissionId	Robotics.Mission	robotics support mission

FROM TABLE	FK COLUMN	TO TABLE	RELATIONSHIP
EmergencyPreparedness.PlanRobotMissionLink			

C.14.7 Indexes, seed examples and verification queries

C.14.7.1 Recommended indexes

```
CREATE INDEX IX_EmergencyPlan_SiteId ON EmergencyPreparedness.EmergencyPlan(SiteId);
CREATE INDEX IX_EmergencyPlan_PlantId ON EmergencyPreparedness.EmergencyPlan(PlantId);
CREATE INDEX IX_EmergencyPlanRevision_EmergencyPlanId ON EmergencyPreparedness.EmergencyPlanRevision(EmergencyPlanId);
CREATE INDEX IX_EmergencyClassification_UnitId_DeclaredAtUtc ON EmergencyPreparedness.EmergencyClassification(UnitId, DeclaredAtUtc DESC);
CREATE INDEX IX_EmergencyClassification_OperationalEventId ON EmergencyPreparedness.EmergencyClassification(OperationalEventId);
CREATE INDEX IX_ResponseTeam_SiteId ON EmergencyPreparedness.ResponseTeam(SiteId);
CREATE INDEX IX_ResponseTeamMember_PersonId ON EmergencyPreparedness.ResponseTeamMember(PersonId);
CREATE INDEX IX_Exercise_Site_Status ON EmergencyPreparedness.Exercise(SiteId, ExerciseStatusId, ScheduledStartUtc DESC);
CREATE INDEX IX_ExerciseParticipant_ExerciseId ON EmergencyPreparedness.ExerciseParticipant(ExerciseId);
CREATE INDEX IX_ExerciseInject_ExerciseId_SequenceNumber ON EmergencyPreparedness.ExerciseInject(ExerciseId, SequenceNumber);
CREATE INDEX IX_ExerciseObservation_ExerciseId ON EmergencyPreparedness.ExerciseObservation(ExerciseId);
CREATE INDEX IX_AssemblyPoint_SiteId ON EmergencyPreparedness.AssemblyPoint(SiteId);
CREATE INDEX IX_EvacuationRoute_Site_Status ON EmergencyPreparedness.EvacuationRoute(SiteId, RouteStatusId);
CREATE INDEX IX_MusterRecord_Event_Person ON EmergencyPreparedness.MusterRecord(OperationalEventId, PersonId);
CREATE INDEX IX_MusterRecord_Exercise_Person ON EmergencyPreparedness.MusterRecord(ExerciseId, PersonId);
CREATE INDEX IX_ProtectiveAction_Classification ON EmergencyPreparedness.ProtectiveAction(EmergencyClassificationId, OrderedAtUtc DESC);
CREATE INDEX IX_EmergencyCommunication_Status_SentAt ON EmergencyPreparedness.EmergencyCommunication(CommunicationStatusId, SentAtUtc DESC);
CREATE INDEX IX_EmergencyResource_Site_Type_Status ON EmergencyPreparedness.EmergencyResource(SiteId, ResourceTypeId, ResourceStatusId);
CREATE INDEX IX_ResourceReadinessCheck_Resource_CheckedAt ON EmergencyPreparedness.ResourceReadinessCheck(EmergencyResourceId, CheckedAtUtc DESC);
GO
```

C.14.7.2 Seed examples

```
INSERT INTO EmergencyPreparedness.PlanStatus (Code, Name, DisplayOrder) VALUES
(N'DRAFT', N'Draft', 10),
(N'ACTIVE', N'Active', 20),
(N'UNDER REVIEW', N'Under review', 30),
(N'SUPERSEDED', N'Superseded', 40),
(N'RETIRED', N'Retired', 50);

INSERT INTO EmergencyPreparedness.EmergencyClassType (Code, Name, DisplayOrder) VALUES
(N'ALERT', N'Alert', 10),
(N'SITE AREA', N'Site area emergency', 20),
(N'GENERAL', N'General emergency', 30),
(N'SECURITY', N'Security emergency', 40),
(N'RADIOLOGICAL', N'Radiological emergency', 50);

INSERT INTO EmergencyPreparedness.ExerciseType (Code, Name, DisplayOrder) VALUES
(N'TABLETOP', N'Tabletop exercise', 10),
(N'FUNCTIONAL', N'Functional exercise', 20),
(N'FULL SCALE', N'Full scale exercise', 30),
(N'DRILL', N'Drill', 40),
(N'COMMUNICATIONS', N'Communications drill', 50);

INSERT INTO EmergencyPreparedness.ProtectiveActionType (Code, Name, DisplayOrder) VALUES
(N'EACUATE', N'Evacuate', 10),
(N'SHELTER', N'Shelter in place', 20),
(N'IODINE', N'Iodine prophylaxis', 30),
(N'RESTRICT ACCESS', N'Restrict access', 40),
(N'STOP WORK', N'Stop work', 50);

INSERT INTO EmergencyPreparedness.RouteStatus (Code, Name, DisplayOrder) VALUES
(N'OPEN', N'Open', 10),
(N'RESTRICTED', N'Restricted', 20),
(N'BLOCKED', N'Blocked', 30),
(N'UNDER REVIEW', N'Under review', 40),
(N'RETIRED', N'Retired', 50);
```

C.14.7.3 Verification queries

```
-- One site, its active plans and their current revision count.
SELECT s.Code AS SiteCode,
       p.Code AS PlanCode,
       ps.Code AS PlanStatus,
       p.CurrentRevisionNumber,
       COUNT(r.EmergencyPlanRevisionId) AS RevisionRows
FROM EmergencyPreparedness.EmergencyPlan AS p
JOIN Organization.Site AS s ON s.SiteId = p.SiteId
JOIN EmergencyPreparedness.PlanStatus AS ps ON ps.PlanStatusId = p.PlanStatusId
LEFT JOIN EmergencyPreparedness.EmergencyPlanRevision AS r ON r.EmergencyPlanId = p.EmergencyPlanId
WHERE p.IsDeleted = 0
GROUP BY s.Code, p.Code, ps.Code, p.CurrentRevisionNumber;

-- Exercises with observations requiring corrective action.
SELECT e.Code AS ExerciseCode,
       e.Name AS ExerciseName,
       COUNT(o.ExerciseObservationId) AS CorrectiveObservations
FROM EmergencyPreparedness.Exercise AS e
JOIN EmergencyPreparedness.ExerciseObservation AS o ON o.ExerciseId = e.ExerciseId
WHERE o.CorrectiveActionRequired = 1
GROUP BY e.Code, e.Name;

-- Open or restricted evacuation routes crossing radiological zones.
SELECT r.Code AS RouteCode,
       rs.Code AS RouteStatus,
       z.Code AS RadiationZone,
       rz.IsAvoidIfAlarmed
FROM EmergencyPreparedness.EvacuationRoute AS r
JOIN EmergencyPreparedness.RouteStatus AS rs ON rs.RouteStatusId = r.RouteStatusId
JOIN EmergencyPreparedness.EvacuationRouteZone AS rz ON rz.EvacuationRouteId = r.EvacuationRouteId
JOIN RadiationMonitoring.RadiationZone AS z ON z.RadiationZoneId = rz.RadiationZoneId
WHERE rs.Code IN (N'OPEN', N'RESTRICTED');
```

```
-- Resource readiness dashboard by site.
SELECT s.Code AS SiteCode,
       rt.Code AS ResourceType,
       rds.Code AS ReadinessStatus,
```



```

COUNT(*) AS ResourceCount
FROM EmergencyPreparedness.EmergencyResource AS er
JOIN Organization.Site AS s ON s.SiteId = er.SiteId
JOIN EmergencyPreparedness.ResourceType AS rt ON rt.ResourceTypeId = er.ResourceTypeId
OUTER APPLY (
    SELECT TOP (1) c.ReadinessStatusId
    FROM EmergencyPreparedness.ResourceReadinessCheck AS c
    WHERE c.EmergencyResourceId = er.EmergencyResourceId
    ORDER BY c.CheckedAtUtc DESC
) AS last_check
LEFT JOIN EmergencyPreparedness.ReadinessStatus AS rds ON rds.ReadinessStatusId = last_check.ReadinessStatusId
GROUP BY s.Code, rt.Code, rds.Code;

```

C.14.8 Honest boundary

Boundary. This is a demonstrator-grade preparedness schema for NEXUS-1. It can model plans, drills, classifications, actions, communications and readiness checks, but it is not a certified emergency-planning system, not a regulatory filing system, not safety-class software and not connected to any real plant. Its value here is the data-modelling discipline: explicit plan revisions, typed response roles, queryable muster records, auditable protective actions, and foreign-key passports to the sectors that supply plant, radiation, alarm, event, robotics and personnel context.

Appendix C.15 - Compliance Schema

The **Compliance** schema is the assurance layer of the NEXUS-1 data backbone. It owns regulations, requirements, controls, audits, findings, corrective actions, waivers, non-conformances, attestations and evidence. Operational schemas record what happened; Compliance records how the platform proves that those facts are governed.

Boundary. This is a demonstrator-grade compliance model for the NEXUS-1 companion schema. It is not a licensed nuclear compliance system and it does not claim to encode any real regulator's mandatory reporting process.

C.15.1 Sector responsibility

The schema has five jobs: keep a regulatory library, map requirements to controls, run audits and reviews, track findings and corrective actions, and attach evidence to operational facts from other sectors.

- **Regulatory library:** regulatory bodies, regulations and individual requirement clauses.
- **Control framework:** controls, frequencies, owners and requirement coverage.
- **Assurance workflow:** audits, audit scope, findings, corrective actions and reviews.
- **Exceptions:** waivers and non-conformances with risk, status and approval.
- **Evidence:** documents, query results, screenshots, sign-offs and cross-sector evidence links.

C.15.2 Full table list

TABLE	PRIMARY KEY	IMPORTANT FOREIGN KEYS	PURPOSE
Compliance.RegulatoryBody	RegulatoryBodyId	JurisdictionId, CountryId	Authority, standard body or internal governance owner that publishes requirements.
Compliance.Regulation	RegulationId	RegulatoryBodyId, RegulationTypeId, JurisdictionId, CountryId	Named regulation, standard, license condition or internal governance source.
Compliance.Requirement	RequirementId	RegulationId, RequirementStatusId, RequirementPriorityId	Individual requirement clause that can be mapped to controls and obligations.
Compliance.Control	ControlId	ControlTypeId, ControlStatusId, ControlFrequencyId, OwnerUserId, OwnerTeamId	Control performed to satisfy one or more requirements.
Compliance.ControlRequirementMap	ControlRequirementMapId	ControlId, RequirementId	Many-to-many map between controls and requirements, with coverage rationale.

TABLE	PRIMARY KEY	IMPORTANT FOREIGN KEYS	PURPOSE
Compliance.ComplianceProgram	ComplianceProgramId	ComplianceProgramStatusId, SiteId, PlantId, OwnerUserId	Named compliance program for a site, plant, fleet scope or regulatory theme.
Compliance.ComplianceProgramControl	ComplianceProgramControlId	ComplianceProgramId, ControlId	Controls included in a program and their effective windows.
Compliance.Audit	AuditId	AuditTypeId, AuditStatusId, ComplianceProgramId, SiteId, PlantId, UnitId, LeadAuditorUserId	Audit or assessment instance with scope and lead auditor.
Compliance.AuditScope	AuditScopeId	AuditId, AuditScopeTypeId, SiteId, PlantId, UnitId, EquipmentId, RequirementId, ControlId	Explicit scope item for an audit.
Compliance.AuditTeamMember	AuditTeamMemberId	AuditId, ApplicationUserId, AuditRoleTypeId	Who participated in the audit and in what role.
Compliance.AuditFinding	AuditFindingId	AuditId, FindingSeverityId, FindingStatusId, RiskRatingId, RequirementId, ControlId, ResponsibleUserId	Finding raised during an audit, linked to the relevant requirement and/or control.
Compliance.CorrectiveAction	CorrectiveActionId	AuditFindingId, NonConformanceId, CorrectiveActionStatusId, AssignedToUserId, VerifiedByUserId	Action raised to correct or prevent recurrence of a finding or non-conformance.
Compliance.Evidence	EvidenceId	EvidenceTypeId, EvidenceStatusId, UploadedByUserId	Evidence artefact such as a procedure, screenshot, query result or signed record.
Compliance.EvidenceLink	EvidenceLinkId	EvidenceId, plus one optional target FK	Attaches evidence to a requirement, control, audit, finding, action, waiver, incident or exercise.
Compliance.Attestation	AttestationId	ComplianceProgramId, ControlId, AttestationStatusId, AttestedByUserId	Owner attestation that a control is performed, reviewed, or accepted.
Compliance.Waiver	WaiverId	WaiverStatusId, WaiverReasonId, RiskRatingId, RequirementId, ControlId, RequestedByUserId, ApprovedByUserId	Time-bounded exception to a requirement or control, with risk and approval.
Compliance.NonConformance	NonConformanceId	NonConformanceTypeId, FindingSeverityId, FindingStatusId, RiskRatingId, SiteId, PlantId, UnitId, ReportedByUserId	Non-conformance record not necessarily discovered through an audit.

TABLE	PRIMARY KEY	IMPORTANT FOREIGN KEYS	PURPOSE
Compliance.ComplianceReview	ComplianceReviewId	ReviewStatusId, ComplianceProgramId, SiteId, PlantId, UnitId, LedByUserId, ApprovedByUserId	Formal periodic review of a program, unit or domain.
Compliance.ReviewFinding	ReviewFindingId	ComplianceReviewId, RequirementId, ControlId, AuditFindingId, NonConformanceId	Finding or observation raised during a compliance review.
Compliance.ObligationCalendar	ObligationCalendarId	RequirementId, ObligationStatusId, ResponsibleUserId, SiteId, PlantId, UnitId	Scheduled regulatory obligation, submission, review or recurring check.

C.15.3 Lookup tables

Lookup tables follow the standard Appendix C pattern: typed tables with `Code`, `Name`, `Description`, `DisplayOrder`, `IsActive` and `RowVersion`.

LOOKUP TABLE	SEED CODES	USED FOR
Compliance.RegulationType	LAW, REGULATION, STANDARD, GUIDANCE, LICENSE_CONDITION	Classifies the source of a regulatory instrument.
Compliance.Jurisdiction	INTERNATIONAL, EU, NATIONAL, LOCAL, INTERNAL	Geographic or organizational jurisdiction of a requirement.
Compliance.RequirementStatus	DRAFT, ACTIVE, SUPERSEDED, RETIRED, UNDER_REVIEW	Lifecycle state for a requirement.
Compliance.RequirementPriority	LOW, MEDIUM, HIGH, CRITICAL	Practical priority for implementation and review.
Compliance.ControlType	PREVENTIVE, DETECTIVE, CORRECTIVE, COMPENSATING, GOVERNANCE	How a compliance control works.
Compliance.ControlStatus	DRAFT, ACTIVE, INEFFECTIVE, RETIRED, UNDER_REMEDIATION	Lifecycle state of a control.
Compliance.ControlFrequency	CONTINUOUS, DAILY, WEEKLY, MONTHLY, QUARTERLY, ANNUAL, EVENT_DRIVEN	How often the control is performed or checked.
Compliance.ComplianceProgramStatus	DRAFT, ACTIVE, PAUSED, CLOSED, ARCHIVED	Lifecycle state of a compliance program.
Compliance.AuditType	INTERNAL, EXTERNAL, REGULATORY, SUPPLIER, FOLLOW_UP	Kind of audit or assessment.
Compliance.AuditStatus	PLANNED, IN_PROGRESS, REPORTING, CLOSED, CANCELLED	Lifecycle state for an audit.
Compliance.AuditScopeType	SITE, PLANT, UNIT, SYSTEM, PROCESS, PROGRAM, SUPPLIER	Scope boundary for an audit.
Compliance.AuditRoleType		Role of a person on the audit team.

LOOKUP TABLE	SEED CODES	USED FOR
	LEAD , AUDITOR , OBSERVER , SME , REGULATOR	
Compliance.FindingSeverity	INFO , MINOR , MAJOR , CRITICAL	Severity of an audit finding or non-conformance.
Compliance.FindingStatus	OPEN , IN_REVIEW , ACTION_ASSIGNED , CLOSED , REJECTED	Lifecycle state for findings.
Compliance.CorrectiveActionStatus	OPEN , ASSIGNED , IN_PROGRESS , BLOCKED , VERIFIED , CLOSED , CANCELLED	Lifecycle state for corrective actions.
Compliance.EvidenceType	DOCUMENT , PHOTO , SCREENSHOT , QUERY_RESULT , LOG_EXTRACT , SIGN_OFF , PROCEDURE	Kind of evidence artefact.
Compliance.EvidenceStatus	DRAFT , SUBMITTED , ACCEPTED , REJECTED , EXPIRED	Evidence lifecycle state.
Compliance.AttestationStatus	REQUESTED , ATTESTED , REJECTED , EXPIRED , REVOKED	Status of a user or owner attestation.
Compliance.WaiverStatus	REQUESTED , APPROVED , REJECTED , EXPIRED , REVOKED	Lifecycle state of a compliance waiver.
Compliance.WaiverReason	TEMPORARY_CONSTRAINT , COMPENSATING_CONTROL , DESIGN_LIMITATION , AWAITING_OUTAGE , OTHER	Reason for requesting a waiver.
Compliance.NonConformanceType	PROCESS , EQUIPMENT , DOCUMENTATION , TRAINING , SECURITY , RADIOLOGICAL	Type of non-conformance.
Compliance.RiskRating	LOW , MEDIUM , HIGH , EXTREME	Risk rating used in findings, waivers and reviews.
Compliance.ReviewStatus	PLANNED , IN_PROGRESS , READY_FOR_APPROVAL , APPROVED , REWORK_REQUIRED , CLOSED	Lifecycle state for a compliance review.
Compliance.ObligationStatus	NOT_DUE , DUE_SOON , OVERDUE , SUBMITTED , ACCEPTED , MISSED	Status of a scheduled compliance obligation.

C.15.4 ER diagram

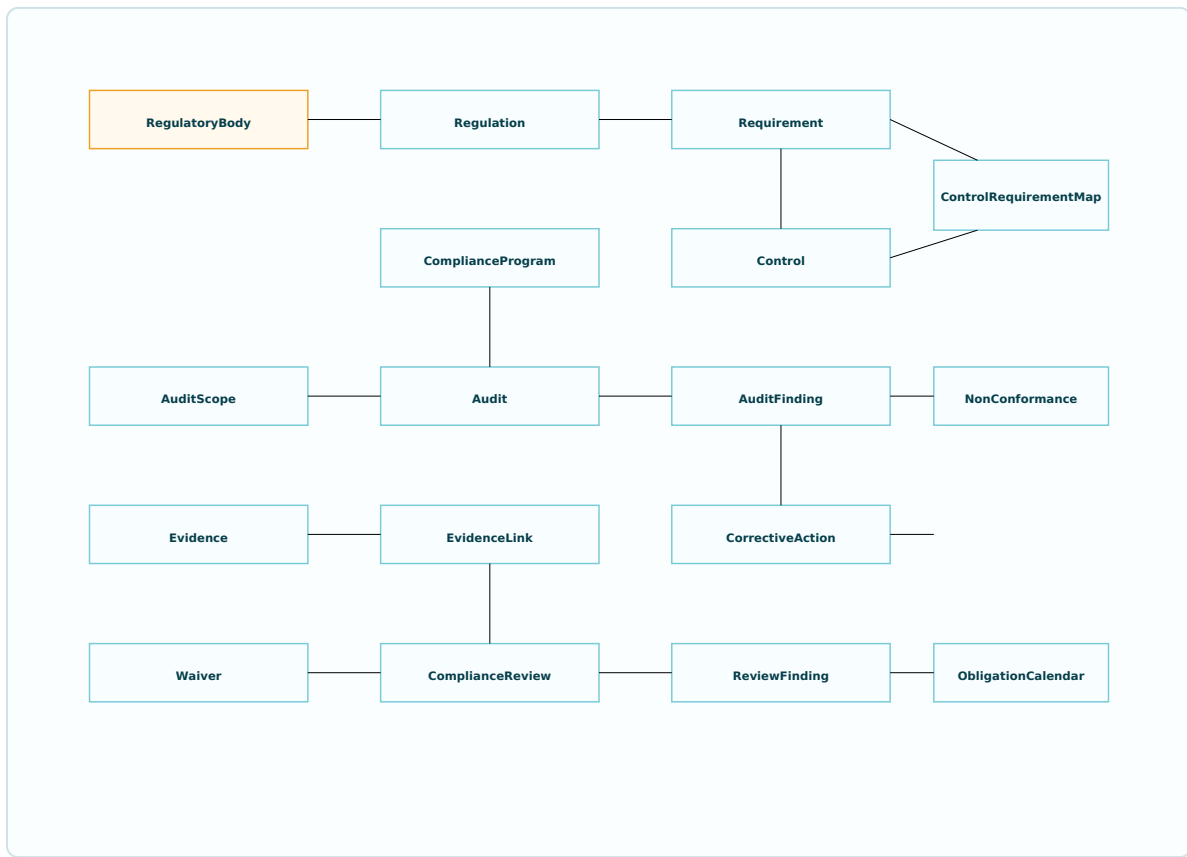


Figure C.15.1 - Compliance sector ER diagram. Regulations decompose into requirements; requirements are covered by controls; programs, audits, findings, actions, evidence, waivers and obligations form the governance workflow.

C.15.5 Foreign-key passport map

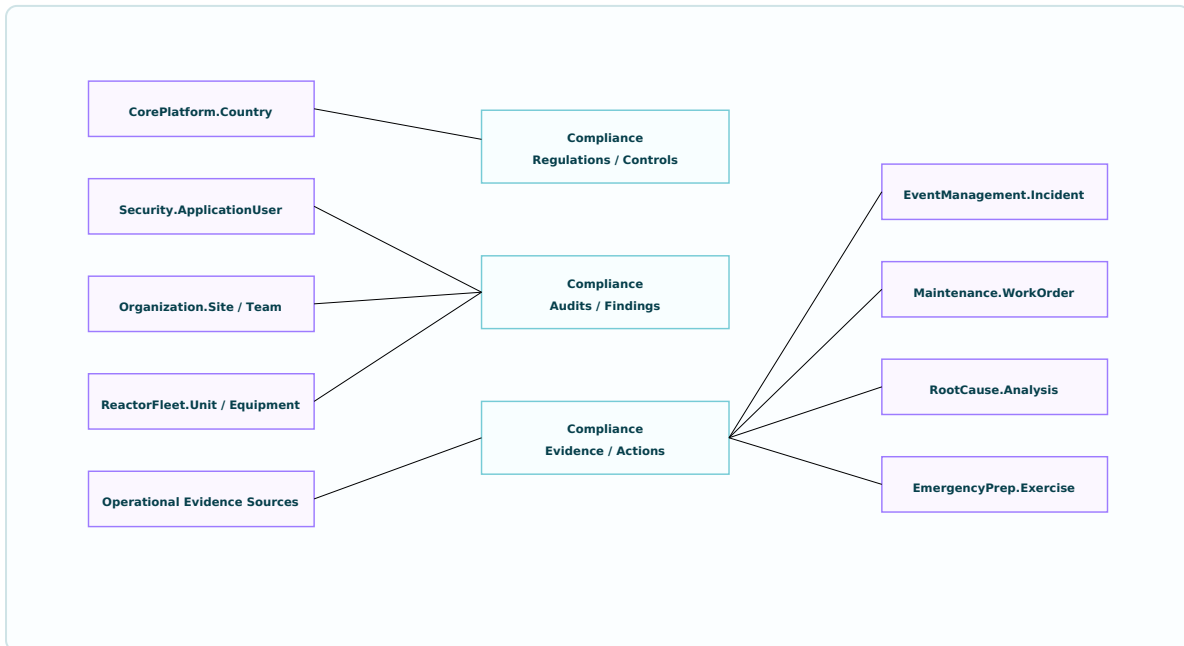


Figure C.15.2 - Cross-schema passport map. Compliance depends on CorePlatform, Security, Organization and ReactorFleet, and attaches evidence to EventManagement, Maintenance, RootCause and EmergencyPreparedness.

C.15.6 Lookup DDL

```
CREATE TABLE Compliance.RegulationType (
  RegulationTypeId INT IDENTITY(1,1) NOT NULL,
  Code NVARCHAR(50) NOT NULL,
  Name NVARCHAR(150) NOT NULL,
  Description NVARCHAR(500) NULL,
  DisplayOrder INT NOT NULL
  CONSTRAINT DF_Compliance_RegulationType_DisplayOrder DEFAULT 0,
  IsActive BIT NOT NULL
  CONSTRAINT DF_Compliance_RegulationType_IsActive DEFAULT 1,
  CreatedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_Compliance_RegulationType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_Compliance_RegulationType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  RowVersion ROWVERSION NOT NULL,

  CONSTRAINT PK_Compliance_RegulationType
    PRIMARY KEY (RegulationTypeId),
  CONSTRAINT UQ_Compliance_RegulationType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Compliance.Jurisdiction (
  JurisdictionId INT IDENTITY(1,1) NOT NULL,
  Code NVARCHAR(50) NOT NULL,
  Name NVARCHAR(150) NOT NULL,
  Description NVARCHAR(500) NULL,
  DisplayOrder INT NOT NULL
  CONSTRAINT DF_Compliance_Jurisdiction_DisplayOrder DEFAULT 0,
  IsActive BIT NOT NULL
  CONSTRAINT DF_Compliance_Jurisdiction_IsActive DEFAULT 1,
  CreatedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_Compliance_Jurisdiction_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_Compliance_Jurisdiction_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  RowVersion ROWVERSION NOT NULL,

  CONSTRAINT PK_Compliance_Jurisdiction
    PRIMARY KEY (JurisdictionId),
  CONSTRAINT UQ_Compliance_Jurisdiction_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Compliance.RequirementStatus (
  RequirementStatusId INT IDENTITY(1,1) NOT NULL,
  Code NVARCHAR(50) NOT NULL,
  Name NVARCHAR(150) NOT NULL,
  Description NVARCHAR(500) NULL,
  DisplayOrder INT NOT NULL
  CONSTRAINT DF_Compliance_RequirementStatus_DisplayOrder DEFAULT 0,
  IsActive BIT NOT NULL
  CONSTRAINT DF_Compliance_RequirementStatus_IsActive DEFAULT 1,
  CreatedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_Compliance_RequirementStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedAtUtc DATETIME2(7) NOT NULL
  CONSTRAINT DF_Compliance_RequirementStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  RowVersion ROWVERSION NOT NULL,
```

```

CONSTRAINT PK_Compliance_RequirementStatus
PRIMARY KEY (RequirementStatusId),
CONSTRAINT UQ_Compliance_RequirementStatus_Code
UNIQUE (Code)
);
GO

CREATE TABLE Compliance.RequirementPriority (
RequirementPriorityId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_Compliance_RequirementPriority_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_Compliance_RequirementPriority_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Compliance_RequirementPriority_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Compliance_RequirementPriority_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_Compliance_RequirementPriority
PRIMARY KEY (RequirementPriorityId),
CONSTRAINT UQ_Compliance_RequirementPriority_Code
UNIQUE (Code)
);
GO

CREATE TABLE Compliance.ControlType (
ControlTypeId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_Compliance_ControlType_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_Compliance_ControlType_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Compliance_ControlType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Compliance_ControlType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_Compliance_ControlType
PRIMARY KEY (ControlTypeId),
CONSTRAINT UQ_Compliance_ControlType_Code
UNIQUE (Code)
);
GO

CREATE TABLE Compliance.ControlStatus (
ControlStatusId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_Compliance_ControlStatus_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_Compliance_ControlStatus_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Compliance_ControlStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Compliance_ControlStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_Compliance_ControlStatus
PRIMARY KEY (ControlStatusId),
CONSTRAINT UQ_Compliance_ControlStatus_Code
UNIQUE (Code)
);
GO

CREATE TABLE Compliance.ControlFrequency (
ControlFrequencyId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_Compliance_ControlFrequency_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_Compliance_ControlFrequency_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Compliance_ControlFrequency_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Compliance_ControlFrequency_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_Compliance_ControlFrequency
PRIMARY KEY (ControlFrequencyId),
CONSTRAINT UQ_Compliance_ControlFrequency_Code
UNIQUE (Code)
);
GO

CREATE TABLE Compliance.ComplianceProgramStatus (
ComplianceProgramStatusId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_Compliance_ComplianceProgramStatus_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_Compliance_ComplianceProgramStatus_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Compliance_ComplianceProgramStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Compliance_ComplianceProgramStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_Compliance_ComplianceProgramStatus
PRIMARY KEY (ComplianceProgramStatusId),
CONSTRAINT UQ_Compliance_ComplianceProgramStatus_Code
UNIQUE (Code)
);
GO

CREATE TABLE Compliance.AuditType (
AuditTypeId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_Compliance_AuditType_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_Compliance_AuditType_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Compliance_AuditType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Compliance_AuditType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_Compliance_AuditType
PRIMARY KEY (AuditTypeId),
CONSTRAINT UQ_Compliance_AuditType_Code
UNIQUE (Code)
);
GO

```



```

CREATE TABLE Compliance.AuditStatus (
    AuditStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Compliance_AuditStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Compliance_AuditStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Compliance_AuditStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Compliance_AuditStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Compliance_AuditStatus
        PRIMARY KEY (AuditStatusId),
    CONSTRAINT UQ_Compliance_AuditStatus_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Compliance.AuditScopeType (
    AuditScopeTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Compliance_AuditScopeType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Compliance_AuditScopeType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Compliance_AuditScopeType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Compliance_AuditScopeType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Compliance_AuditScopeType
        PRIMARY KEY (AuditScopeTypeId),
    CONSTRAINT UQ_Compliance_AuditScopeType_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Compliance.AuditRoleType (
    AuditRoleTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Compliance_AuditRoleType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Compliance_AuditRoleType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Compliance_AuditRoleType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Compliance_AuditRoleType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Compliance_AuditRoleType
        PRIMARY KEY (AuditRoleTypeId),
    CONSTRAINT UQ_Compliance_AuditRoleType_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Compliance.FindingSeverity (
    FindingSeverityId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Compliance_FindingSeverity_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Compliance_FindingSeverity_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Compliance_FindingSeverity_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Compliance_FindingSeverity_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Compliance_FindingSeverity
        PRIMARY KEY (FindingSeverityId),
    CONSTRAINT UQ_Compliance_FindingSeverity_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Compliance.FindingStatus (
    FindingStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Compliance_FindingStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Compliance_FindingStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Compliance_FindingStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Compliance_FindingStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Compliance_FindingStatus
        PRIMARY KEY (FindingStatusId),
    CONSTRAINT UQ_Compliance_FindingStatus_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Compliance.CorrectiveActionStatus (
    CorrectiveActionStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
    CONSTRAINT DF_Compliance_CorrectiveActionStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
    CONSTRAINT DF_Compliance_CorrectiveActionStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Compliance_CorrectiveActionStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
    CONSTRAINT DF_Compliance_CorrectiveActionStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Compliance_CorrectiveActionStatus
        PRIMARY KEY (CorrectiveActionStatusId),
    CONSTRAINT UQ_Compliance_CorrectiveActionStatus_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Compliance.EvidenceType (
    EvidenceTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,

```

```

        DisplayOrder INT NOT NULL
        CONSTRAINT DF_Comppliance_EvidenceType_DisplayOrder DEFAULT 0,
        IsActive BIT NOT NULL
        CONSTRAINT DF_Comppliance_EvidenceType_IsActive DEFAULT 1,
        CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Comppliance_EvidenceType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
        ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Comppliance_EvidenceType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
        RowVersion ROWVERSION NOT NULL,

        CONSTRAINT PK_Comppliance_EvidenceType
        PRIMARY KEY (EvidenceTypeId),
        CONSTRAINT UQ_Comppliance_EvidenceType_Code
        UNIQUE (Code)
    );
GO

CREATE TABLE Compliance.EvidenceStatus (
    EvidenceStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_Comppliance_EvidenceStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_Comppliance_EvidenceStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Comppliance_EvidenceStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Comppliance_EvidenceStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Comppliance_EvidenceStatus
    PRIMARY KEY (EvidenceStatusId),
    CONSTRAINT UQ_Comppliance_EvidenceStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Compliance.AttestationStatus (
    AttestationStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_Comppliance_AttestationStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_Comppliance_AttestationStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Comppliance_AttestationStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Comppliance_AttestationStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Comppliance_AttestationStatus
    PRIMARY KEY (AttestationStatusId),
    CONSTRAINT UQ_Comppliance_AttestationStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Compliance.WaiverStatus (
    WaiverStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_Comppliance_WaiverStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_Comppliance_WaiverStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Comppliance_WaiverStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Comppliance_WaiverStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Comppliance_WaiverStatus
    PRIMARY KEY (WaiverStatusId),
    CONSTRAINT UQ_Comppliance_WaiverStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Compliance.WaiverReason (
    WaiverReasonId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_Comppliance_WaiverReason_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_Comppliance_WaiverReason_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Comppliance_WaiverReason_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Comppliance_WaiverReason_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Comppliance_WaiverReason
    PRIMARY KEY (WaiverReasonId),
    CONSTRAINT UQ_Comppliance_WaiverReason_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Compliance.NonConformanceType (
    NonConformanceTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_Comppliance_NonConformanceType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_Comppliance_NonConformanceType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Comppliance_NonConformanceType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Comppliance_NonConformanceType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Comppliance_NonConformanceType
    PRIMARY KEY (NonConformanceTypeId),
    CONSTRAINT UQ_Comppliance_NonConformanceType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Compliance.RiskRating (
    RiskRatingId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_Comppliance_RiskRating_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_Comppliance_RiskRating_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Comppliance_RiskRating_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL

```

```

        CONSTRAINT DF_Compliance_RiskRating_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
        RowVersion ROWVERSION NOT NULL,

        CONSTRAINT PK_Compliance_RiskRating
            PRIMARY KEY (RiskRatingId),
        CONSTRAINT UQ_Compliance_RiskRating_Code
            UNIQUE (Code)
    );
GO

CREATE TABLE Compliance.ReviewStatus (
    ReviewStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_Compliance_ReviewStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_Compliance_ReviewStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Compliance_ReviewStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Compliance_ReviewStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Compliance_ReviewStatus
        PRIMARY KEY (ReviewStatusId),
    CONSTRAINT UQ_Compliance_ReviewStatus_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Compliance.ObligationStatus (
    ObligationStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_Compliance_ObligationStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_Compliance_ObligationStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Compliance_ObligationStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Compliance_ObligationStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Compliance_ObligationStatus
        PRIMARY KEY (ObligationStatusId),
    CONSTRAINT UQ_Compliance_ObligationStatus_Code
        UNIQUE (Code)
);
GO

```

C.15.7 Main table DDL

The following DDL uses SQL Server schemas, explicit keys, named constraints, UTC timestamps and EF Core-friendly foreign keys.

```

CREATE TABLE Compliance.RegulatoryBody (
    RegulatoryBodyId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    JurisdictionId INT NOT NULL,
    CountryId INT NULL,
    WebsiteUrl NVARCHAR(1000) NULL,
    Description NVARCHAR(1000) NULL,
    IsActive BIT NOT NULL CONSTRAINT DF_RegulatoryBody_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RegulatoryBody_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RegulatoryBody_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_RegulatoryBody_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Compliance_RegulatoryBody PRIMARY KEY (RegulatoryBodyId),
    CONSTRAINT UQ_Compliance_RegulatoryBody_Code UNIQUE (Code),
    CONSTRAINT FK_RegulatoryBody_Jurisdiction FOREIGN KEY (JurisdictionId) REFERENCES Compliance.Jurisdiction(JurisdictionId),
    CONSTRAINT FK_RegulatoryBody_Country FOREIGN KEY (CountryId) REFERENCES CorePlatform.Country(CountryId)
);
GO

CREATE TABLE Compliance.Regulation (
    RegulationId INT IDENTITY(1,1) NOT NULL,
    RegulatoryBodyId INT NOT NULL,
    RegulationTypeId INT NOT NULL,
    JurisdictionId INT NOT NULL,
    CountryId INT NULL,
    Code NVARCHAR(80) NOT NULL,
    Title NVARCHAR(300) NOT NULL,
    VersionLabel NVARCHAR(80) NULL,
    PublishedDate DATE NULL,
    EffectiveFromUtc DATETIME2(7) NULL,
    EffectiveToUtc DATETIME2(7) NULL,
    SourceUri NVARCHAR(1000) NULL,
    Summary NVARCHAR(2000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Regulation_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Regulation_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Regulation_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Compliance_Regulation PRIMARY KEY (RegulationId),
    CONSTRAINT UQ_Compliance_Regulation_Code UNIQUE (Code),
    CONSTRAINT FK_Regulation_RegulatoryBody FOREIGN KEY (RegulatoryBodyId) REFERENCES Compliance.RegulatoryBody(RegulatoryBodyId),
    CONSTRAINT FK_Regulation_RegulationType FOREIGN KEY (RegulationTypeId) REFERENCES Compliance.RegulationType(RegulationTypeId),
    CONSTRAINT FK_Regulation_Jurisdiction FOREIGN KEY (JurisdictionId) REFERENCES Compliance.Jurisdiction(JurisdictionId),
    CONSTRAINT FK_Regulation_Country FOREIGN KEY (CountryId) REFERENCES CorePlatform.Country(CountryId),
    CONSTRAINT CK_RegulationEffectiveWindow CHECK (EffectiveToUtc IS NULL OR EffectiveFromUtc IS NULL OR EffectiveToUtc >= EffectiveFromUtc)
);
GO

CREATE TABLE Compliance.Requirement (
    RequirementId INT IDENTITY(1,1) NOT NULL,
    RegulationId INT NOT NULL,
    ParentRequirementId INT NULL,
    RequirementStatusId INT NOT NULL,
    RequirementPriorityId INT NOT NULL,
    ClauseReference NVARCHAR(100) NOT NULL,
    Title NVARCHAR(300) NOT NULL,
    RequirementText NVARCHAR(MAX) NOT NULL,
    EffectiveFromUtc DATETIME2(7) NULL,
    EffectiveToUtc DATETIME2(7) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Requirement_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,

```

```

ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Requirement_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedBy NVARCHAR(100) NOT NULL,
IsDeleted BIT NOT NULL CONSTRAINT DF_Requirement_IsDeleted DEFAULT 0,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_Compliance_Requirement PRIMARY KEY (RequirementId),
CONSTRAINT UQ_Compliance_Requirement_Regulation Clause UNIQUE (RegulationId, ClauseReference),
CONSTRAINT FK_Requirement_Regulation FOREIGN KEY (RegulationId) REFERENCES Compliance.Regulation(RegulationId),
CONSTRAINT FK_Requirement_Parent FOREIGN KEY (ParentRequirementId) REFERENCES Compliance.Requirement(RequirementId),
CONSTRAINT FK_Requirement_Status FOREIGN KEY (RequirementStatusId) REFERENCES Compliance.RequirementStatus(RequirementStatusId),
CONSTRAINT FK_Requirement_Priority FOREIGN KEY (RequirementPriorityId) REFERENCES Compliance.RequirementPriority(RequirementPriorityId),
CONSTRAINT CK_Requirement_EffectiveWindow CHECK (EffectiveToUtc IS NULL OR EffectiveFromUtc IS NULL OR EffectiveToUtc >= EffectiveFromUtc)
);
GO

```

```

CREATE TABLE Compliance.Control (
    ControlId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(250) NOT NULL,
    ControlTypeId INT NOT NULL,
    ControlStatusId INT NOT NULL,
    ControlFrequencyId INT NOT NULL,
    OwnerUserId INT NOT NULL,
    OwnerTeamId INT NOT NULL,
    Description NVARCHAR(2000) NULL,
    ProcedureUri NVARCHAR(1000) NULL,
    IsKeyControl BIT NOT NULL CONSTRAINT DF_Control_IsKeyControl DEFAULT 0,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Control_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Control_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Control_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Compliance_Control PRIMARY KEY (ControlId),
    CONSTRAINT UQ_Compliance_Control_Code UNIQUE (Code),
    CONSTRAINT FK_Control_ControlType FOREIGN KEY (ControlTypeId) REFERENCES Compliance.ControlType(ControlTypeId),
    CONSTRAINT FK_Control_ControlStatus FOREIGN KEY (ControlStatusId) REFERENCES Compliance.ControlStatus(ControlStatusId),
    CONSTRAINT FK_Control_ControlFrequency FOREIGN KEY (ControlFrequencyId) REFERENCES Compliance.ControlFrequency(ControlFrequencyId),
    CONSTRAINT FK_Control_OwnerUser FOREIGN KEY (OwnerUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT FK_Control_OwnerTeam FOREIGN KEY (OwnerTeamId) REFERENCES Organization.Team(TeamId)
);
GO

CREATE TABLE Compliance.ControlRequirementMap (
    ControlRequirementMapId INT IDENTITY(1,1) NOT NULL,
    ControlId INT NOT NULL,
    RequirementId INT NOT NULL,
    CoveragePercent DECIMAL(5,2) NULL,
    CoverageRationale NVARCHAR(1000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ControlRequirementMap_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Compliance_ControlRequirementMap PRIMARY KEY (ControlRequirementMapId),
    CONSTRAINT UQ_ControlRequirementMap_Control_Requirement UNIQUE (ControlId, RequirementId),
    CONSTRAINT FK_ControlRequirementMap_Control FOREIGN KEY (ControlId) REFERENCES Compliance.Control(ControlId),
    CONSTRAINT FK_ControlRequirementMap_Requirement FOREIGN KEY (RequirementId) REFERENCES Compliance.Requirement(RequirementId),
    CONSTRAINT CK_ControlRequirementMap_Coverage CHECK (CoveragePercent IS NULL OR (CoveragePercent >= 0 AND CoveragePercent <= 100))
);
GO

```

```

CREATE TABLE Compliance.ComplianceProgram (
    ComplianceProgramId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(250) NOT NULL,
    ComplianceProgramStatusId INT NOT NULL,
    SiteId INT NOT NULL,
    PlantId INT NOT NULL,
    OwnerUserId INT NOT NULL,
    StartDate DATE NOT NULL,
    EndDate DATE NULL,
    Description NVARCHAR(2000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ComplianceProgram_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ComplianceProgram_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_ComplianceProgram_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Compliance_ComplianceProgram PRIMARY KEY (ComplianceProgramId),
    CONSTRAINT UQ_Compliance_ComplianceProgram_Code UNIQUE (Code),
    CONSTRAINT FK_ComplianceProgram_Status FOREIGN KEY (ComplianceProgramStatusId) REFERENCES Compliance.ComplianceProgramStatus(ComplianceProgramStatusId),
    CONSTRAINT FK_ComplianceProgram_Site FOREIGN KEY (SiteId) REFERENCES Organization.Site(SiteId),
    CONSTRAINT FK_ComplianceProgram_Plant FOREIGN KEY (PlantId) REFERENCES Organization.Plant(PlantId),
    CONSTRAINT FK_ComplianceProgram_OwnerUser FOREIGN KEY (OwnerUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_ComplianceProgram_DateWindow CHECK (EndDate IS NULL OR StartDate IS NULL OR EndDate >= StartDate)
);
GO

CREATE TABLE Compliance.ComplianceProgramControl (
    ComplianceProgramControlId INT IDENTITY(1,1) NOT NULL,
    ComplianceProgramId INT NOT NULL,
    ControlId INT NOT NULL,
    EffectiveFromUtc DATETIME2(7) NULL,
    EffectiveToUtc DATETIME2(7) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ComplianceProgramControl_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Compliance_ComplianceProgramControl PRIMARY KEY (ComplianceProgramControlId),
    CONSTRAINT UQ_ComplianceProgramControl_Program_Control UNIQUE (ComplianceProgramId, ControlId),
    CONSTRAINT FK_ComplianceProgramControl_Program FOREIGN KEY (ComplianceProgramId) REFERENCES Compliance.ComplianceProgram(ComplianceProgramId),
    CONSTRAINT FK_ComplianceProgramControl_Control FOREIGN KEY (ControlId) REFERENCES Compliance.Control(ControlId),
    CONSTRAINT CK_ComplianceProgramControl_EffectiveWindow CHECK (EffectiveToUtc IS NULL OR EffectiveFromUtc IS NULL OR EffectiveToUtc >= EffectiveFromUtc)
);
GO

```

```

CREATE TABLE Compliance.Audit (
    AuditId BIGINT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Title NVARCHAR(250) NOT NULL,
    AuditTypeId INT NOT NULL,
    AuditStatusId INT NOT NULL,
    ComplianceProgramId INT NOT NULL,
    SiteId INT NOT NULL,
    PlantId INT NOT NULL,
    UnitId INT NOT NULL,
    LeadAuditorUserId INT NOT NULL,
    PlannedStartUtc DATETIME2(7) NULL,
    PlannedEndUtc DATETIME2(7) NULL,
    ActualStartUtc DATETIME2(7) NULL,
    ActualEndUtc DATETIME2(7) NULL,
    Summary NVARCHAR(3000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Audit_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Audit_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Audit_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Compliance_Audit PRIMARY KEY (AuditId),
    CONSTRAINT UQ_Compliance_Audit_Code UNIQUE (Code),

```

```

CONSTRAINT FK_Audit_AuditType FOREIGN KEY (AuditTypeId) REFERENCES Compliance.AuditType(AuditTypeId),
CONSTRAINT FK_Audit_AuditStatus FOREIGN KEY (AuditStatusId) REFERENCES Compliance.AuditStatus(AuditStatusId),
CONSTRAINT FK_Audit_ComplianceProgram FOREIGN KEY (ComplianceProgramId) REFERENCES Compliance.ComplianceProgram(ComplianceProgramId),
CONSTRAINT FK_Audit_Site FOREIGN KEY (SiteId) REFERENCES Organization.Site(SiteId),
CONSTRAINT FK_Audit_Plant FOREIGN KEY (PlantId) REFERENCES Organization.Plant(PlantId),
CONSTRAINT FK_Audit_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
CONSTRAINT FK_Audit_LeadAuditorUser FOREIGN KEY (LeadAuditorUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
CONSTRAINT CK_Audit_PlannedWindow CHECK (PlannedEndUtc IS NULL OR PlannedStartUtc IS NULL OR PlannedEndUtc >= PlannedStartUtc),
CONSTRAINT CK_Audit_ActualWindow CHECK (ActualEndUtc IS NULL OR ActualStartUtc IS NULL OR ActualEndUtc >= ActualStartUtc)
);
GO

CREATE TABLE Compliance.AuditScope (
    AuditScopeId BIGINT IDENTITY(1,1) NOT NULL,
    AuditId BIGINT NOT NULL,
    AuditScopeTypeId INT NOT NULL,
    SiteId INT NULL,
    PlantId INT NULL,
    UnitId INT NULL,
    EquipmentId INT NULL,
    RequirementId INT NULL,
    ControlId INT NULL,
    ScopeNote NVARCHAR(1000) NULL,

    CONSTRAINT PK_Compliance_AuditScope PRIMARY KEY (AuditScopeId),
    CONSTRAINT FK_AuditScope_Audit FOREIGN KEY (AuditId) REFERENCES Compliance.Audit(AuditId),
    CONSTRAINT FK_AuditScope_AuditScopeType FOREIGN KEY (AuditScopeTypeId) REFERENCES Compliance.AuditScopeType(AuditScopeTypeId),
    CONSTRAINT FK_AuditScope_Site FOREIGN KEY (SiteId) REFERENCES Organization.Site(SiteId),
    CONSTRAINT FK_AuditScope_Plant FOREIGN KEY (PlantId) REFERENCES Organization.Plant(PlantId),
    CONSTRAINT FK_AuditScope_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_AuditScope_Equipment FOREIGN KEY (EquipmentId) REFERENCES ReactorFleet.Equipment(EquipmentId),
    CONSTRAINT FK_AuditScope_Requirement FOREIGN KEY (RequirementId) REFERENCES Compliance.Requirement(RequirementId),
    CONSTRAINT FK_AuditScope_Control FOREIGN KEY (ControlId) REFERENCES Compliance.Control(ControlId)
);
GO

CREATE TABLE Compliance.AuditTeamMember (
    AuditTeamMemberId BIGINT IDENTITY(1,1) NOT NULL,
    AuditId BIGINT NOT NULL,
    ApplicationUserId INT NOT NULL,
    AuditRoleTypeId INT NOT NULL,
    IsExternal BIT NOT NULL CONSTRAINT DF_AuditTeamMember_IsExternal DEFAULT 0,
    Notes NVARCHAR(500) NULL,

    CONSTRAINT PK_Compliance_AuditTeamMember PRIMARY KEY (AuditTeamMemberId),
    CONSTRAINT UQ_AuditTeamMember_Audit User Role UNIQUE (AuditId, ApplicationUserId, AuditRoleTypeId),
    CONSTRAINT FK_AuditTeamMember_Audit FOREIGN KEY (AuditId) REFERENCES Compliance.Audit(AuditId),
    CONSTRAINT FK_AuditTeamMember_User FOREIGN KEY (ApplicationUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT FK_AuditTeamMember_AuditRoleType FOREIGN KEY (AuditRoleTypeId) REFERENCES Compliance.AuditRoleType(AuditRoleTypeId)
);
GO

CREATE TABLE Compliance.AuditFinding (
    AuditFindingId BIGINT IDENTITY(1,1) NOT NULL,
    AuditId BIGINT NOT NULL,
    FindingSeverityId INT NOT NULL,
    FindingStatusId INT NOT NULL,
    RiskRatingId INT NOT NULL,
    RequirementId INT NULL,
    ControlId INT NULL,
    ResponsibleUserId INT NULL,
    Code NVARCHAR(80) NOT NULL,
    Title NVARCHAR(250) NOT NULL,
    FindingText NVARCHAR(MAX) NOT NULL,
    RaisedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AuditFinding_RaisedAtUtc DEFAULT SYSUTCDATETIME(),
    DueAtUtc DATETIME2(7) NULL,
    ClosedAtUtc DATETIME2(7) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AuditFinding_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AuditFinding_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_AuditFinding_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Compliance_AuditFinding PRIMARY KEY (AuditFindingId),
    CONSTRAINT UQ_Compliance_AuditFinding Code UNIQUE (Code),
    CONSTRAINT FK_AuditFinding_Audit FOREIGN KEY (AuditId) REFERENCES Compliance.Audit(AuditId),
    CONSTRAINT FK_AuditFinding_Severity FOREIGN KEY (FindingSeverityId) REFERENCES Compliance.FindingSeverity(FindingSeverityId),
    CONSTRAINT FK_AuditFinding_Status FOREIGN KEY (FindingStatusId) REFERENCES Compliance.FindingStatus(FindingStatusId),
    CONSTRAINT FK_AuditFinding_RiskRating FOREIGN KEY (RiskRatingId) REFERENCES Compliance.RiskRating(RiskRatingId),
    CONSTRAINT FK_AuditFinding_Requirement FOREIGN KEY (RequirementId) REFERENCES Compliance.Requirement(RequirementId),
    CONSTRAINT FK_AuditFinding_Control FOREIGN KEY (ControlId) REFERENCES Compliance.Control(ControlId),
    CONSTRAINT FK_AuditFinding_ResponsibleUser FOREIGN KEY (ResponsibleUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_AuditFinding_CloseWindow CHECK (ClosedAtUtc IS NULL OR ClosedAtUtc >= RaisedAtUtc)
);
GO

```

```

CREATE TABLE Compliance.NonConformance (
    NonConformanceId BIGINT IDENTITY(1,1) NOT NULL,
    NonConformanceTypeId INT NOT NULL,
    FindingSeverityId INT NOT NULL,
    FindingStatusId INT NOT NULL,
    RiskRatingId INT NOT NULL,
    SiteId INT NULL,
    PlantId INT NULL,
    UnitId INT NULL,
    ReportedByUserId INT NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Title NVARCHAR(250) NOT NULL,
    Description NVARCHAR(MAX) NOT NULL,
    DetectedAtUtc DATETIME2(7) NOT NULL,
    ClosedAtUtc DATETIME2(7) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_NonConformance_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_NonConformance_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_NonConformance_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Compliance_NonConformance PRIMARY KEY (NonConformanceId),
    CONSTRAINT UQ_Compliance_NonConformance Code UNIQUE (Code),
    CONSTRAINT FK_NonConformance_Type FOREIGN KEY (NonConformanceTypeId) REFERENCES Compliance.NonConformanceType(NonConformanceTypeId),
    CONSTRAINT FK_NonConformance_Severity FOREIGN KEY (FindingSeverityId) REFERENCES Compliance.FindingSeverity(FindingSeverityId),
    CONSTRAINT FK_NonConformance_Status FOREIGN KEY (FindingStatusId) REFERENCES Compliance.FindingStatus(FindingStatusId),
    CONSTRAINT FK_NonConformance_RiskRating FOREIGN KEY (RiskRatingId) REFERENCES Compliance.RiskRating(RiskRatingId),
    CONSTRAINT FK_NonConformance_Site FOREIGN KEY (SiteId) REFERENCES Organization.Site(SiteId),
    CONSTRAINT FK_NonConformance_Plant FOREIGN KEY (PlantId) REFERENCES Organization.Plant(PlantId),
    CONSTRAINT FK_NonConformance_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_NonConformance_ReportedByUser FOREIGN KEY (ReportedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_NonConformance_CloseWindow CHECK (ClosedAtUtc IS NULL OR ClosedAtUtc >= DetectedAtUtc)
);
GO

CREATE TABLE Compliance.CorrectiveAction (
    CorrectiveActionId BIGINT IDENTITY(1,1) NOT NULL,
    AuditFindingId BIGINT NOT NULL,
    NonConformanceId BIGINT NOT NULL,
    CorrectiveActionStatusId INT NOT NULL,
    AssignedToUserId INT NOT NULL,
    VerifiedByUserId INT NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Title NVARCHAR(250) NOT NULL,
    ActionText NVARCHAR(MAX) NOT NULL,
    DueAtUtc DATETIME2(7) NULL,
    CompletedAtUtc DATETIME2(7) NULL,
    VerifiedAtUtc DATETIME2(7) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_CorrectiveAction_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_CorrectiveAction_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
);

```

```

ModifiedBy NVARCHAR(100) NOT NULL,
IsDeleted BIT NOT NULL CONSTRAINT DF_CorrectiveAction_IsDeleted DEFAULT 0,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_CorrectiveAction PRIMARY KEY (CorrectiveActionId),
CONSTRAINT UQ_CorrectiveAction UNIQUE (Code),
CONSTRAINT FK_CorrectiveAction_AuditFinding FOREIGN KEY (AuditFindingId) REFERENCES Compliance.AuditFinding(AuditFindingId),
CONSTRAINT FK_CorrectiveAction_NonConformance FOREIGN KEY (NonConformanceId) REFERENCES Compliance.NonConformance(NonConformanceId),
CONSTRAINT FK_CorrectiveAction_Status FOREIGN KEY (CorrectiveActionStatusId) REFERENCES Compliance.CorrectiveActionStatus(CorrectiveActionStatusId),
CONSTRAINT FK_CorrectiveAction_AssignedToUser FOREIGN KEY (AssignedToUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
CONSTRAINT FK_CorrectiveAction_VerifiedByUser FOREIGN KEY (VerifiedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
CONSTRAINT CK_CorrectiveAction_Source CHECK (AuditFindingId IS NOT NULL OR NonConformanceId IS NOT NULL)
);
GO

CREATE TABLE Compliance.Evidence (
    EvidenceId BIGINT IDENTITY(1,1) NOT NULL,
    EvidenceTypeId INT NOT NULL,
    EvidenceStatusId INT NOT NULL,
    UploadedByUserId INT NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Title NVARCHAR(250) NOT NULL,
    DocumentUri NVARCHAR(1000) NULL,
    HashSha256 CHAR(64) NULL,
    CapturedAtUtc DATETIME2(7) NULL,
    UploadedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Evidence_UploadedAtUtc DEFAULT SYSUTCDATETIME(),
    Description NVARCHAR(2000) NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_CorrectiveAction PRIMARY KEY (EvidenceId),
    CONSTRAINT UQ_CorrectiveAction UNIQUE (Code),
    CONSTRAINT FK_Evidence_Type FOREIGN KEY (EvidenceTypeId) REFERENCES Compliance.EvidenceType(EvidenceTypeId),
    CONSTRAINT FK_Evidence_Status FOREIGN KEY (EvidenceStatusId) REFERENCES Compliance.EvidenceStatus(EvidenceStatusId),
    CONSTRAINT FK_Evidence_UploadedByUser FOREIGN KEY (UploadedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId)
);
GO

```

```

CREATE TABLE Compliance.EvidenceLink (
    EvidenceLinkId BIGINT IDENTITY(1,1) NOT NULL,
    EvidenceId BIGINT NOT NULL,
    RequirementId INT NOT NULL,
    ControlId INT NOT NULL,
    AuditId BIGINT NOT NULL,
    AuditFindingId BIGINT NOT NULL,
    CorrectiveActionId BIGINT NOT NULL,
    WaiverId BIGINT NOT NULL,
    NonConformanceId BIGINT NOT NULL,
    IncidentId BIGINT NOT NULL,
    WorkOrderId BIGINT NOT NULL,
    RootCauseAnalysisId BIGINT NOT NULL,
    ExerciseId INT NOT NULL,
    LinkNote NVARCHAR(1000) NULL,
    LinkedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_EvidenceLink_LinkedAtUtc DEFAULT SYSUTCDATETIME(),
    LinkedByUserId INT NOT NULL,

    CONSTRAINT PK_CorrectiveAction PRIMARY KEY (EvidenceLinkId),
    CONSTRAINT FK_EvidenceLink_Evidence FOREIGN KEY (EvidenceId) REFERENCES Compliance.Evidence(EvidenceId),
    CONSTRAINT FK_EvidenceLink_Requirement FOREIGN KEY (RequirementId) REFERENCES Compliance.Requirement(RequirementId),
    CONSTRAINT FK_EvidenceLink_Control FOREIGN KEY (ControlId) REFERENCES Compliance.Control(ControlId),
    CONSTRAINT FK_EvidenceLink_Audit FOREIGN KEY (AuditId) REFERENCES Compliance.Audit(AuditId),
    CONSTRAINT FK_EvidenceLink_AuditFinding FOREIGN KEY (AuditFindingId) REFERENCES Compliance.AuditFinding(AuditFindingId),
    CONSTRAINT FK_EvidenceLink_CorrectiveAction FOREIGN KEY (CorrectiveActionId) REFERENCES Compliance.CorrectiveAction(CorrectiveActionId),
    CONSTRAINT FK_EvidenceLink_Waiver FOREIGN KEY (WaiverId) REFERENCES Compliance.Waiver(WaiverId),
    CONSTRAINT FK_EvidenceLink_NonConformance FOREIGN KEY (NonConformanceId) REFERENCES Compliance.NonConformance(NonConformanceId),
    CONSTRAINT FK_EvidenceLink_Incident FOREIGN KEY (IncidentId) REFERENCES EventManagement.Incident(IncidentId),
    CONSTRAINT FK_EvidenceLink_WorkOrder FOREIGN KEY (WorkOrderId) REFERENCES Maintenance.WorkOrder(WorkOrderId),
    CONSTRAINT FK_EvidenceLink_RootCauseAnalysis FOREIGN KEY (RootCauseAnalysisId) REFERENCES RootCause.RootCauseAnalysis(RootCauseAnalysisId),
    CONSTRAINT FK_EvidenceLink_Exercise FOREIGN KEY (ExerciseId) REFERENCES EmergencyPreparedness.Exercise(ExerciseId),
    CONSTRAINT FK_EvidenceLink_LinkedByUser FOREIGN KEY (LinkedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_EvidenceLink_Target CHECK (
        RequirementId IS NOT NULL OR ControlId IS NOT NULL OR AuditId IS NOT NULL OR
        AuditFindingId IS NOT NULL OR CorrectiveActionId IS NOT NULL OR WaiverId IS NOT NULL OR
        NonConformanceId IS NOT NULL OR IncidentId IS NOT NULL OR WorkOrderId IS NOT NULL OR
        RootCauseAnalysisId IS NOT NULL OR ExerciseId IS NOT NULL
    );
GO

CREATE TABLE Compliance.Attestation (
    AttestationId BIGINT IDENTITY(1,1) NOT NULL,
    ComplianceProgramId INT NOT NULL,
    ControlId INT NOT NULL,
    AttestationStatusId INT NOT NULL,
    AttestedByUserId INT NOT NULL,
    PeriodStartDate DATE NOT NULL,
    PeriodEndDate DATE NOT NULL,
    AttestedAtUtc DATETIME2(7) NULL,
    StatementText NVARCHAR(3000) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_CorrectiveAction PRIMARY KEY (AttestationId),
    CONSTRAINT FK_Attestation_Program FOREIGN KEY (ComplianceProgramId) REFERENCES Compliance.ComplianceProgram(ComplianceProgramId),
    CONSTRAINT FK_Attestation_Control FOREIGN KEY (ControlId) REFERENCES Compliance.Control(ControlId),
    CONSTRAINT FK_Attestation_Status FOREIGN KEY (AttestationStatusId) REFERENCES Compliance.AttestationStatus(AttestationStatusId),
    CONSTRAINT FK_Attestation_AttestedByUser FOREIGN KEY (AttestedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_Attestation_Period CHECK (PeriodEndDate >= PeriodStartDate)
);
GO

CREATE TABLE Compliance.Waiver (
    WaiverId BIGINT IDENTITY(1,1) NOT NULL,
    WaiverStatusId INT NOT NULL,
    WaiverReasonId INT NOT NULL,
    RiskRatingId INT NOT NULL,
    RequirementId INT NOT NULL,
    ControlId INT NOT NULL,
    SiteId INT NOT NULL,
    PlantId INT NOT NULL,
    UnitId INT NOT NULL,
    RequestedByUserId INT NOT NULL,
    ApprovedByUserId INT NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Title NVARCHAR(250) NOT NULL,
    Justification NVARCHAR(MAX) NOT NULL,
    CompensatingControlText NVARCHAR(MAX) NULL,
    EffectiveFromUtc DATETIME2(7) NOT NULL,
    EffectiveToUtc DATETIME2(7) NOT NULL,
    ApprovedAtUtc DATETIME2(7) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Waiver_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Waiver_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Waiver_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_CorrectiveAction PRIMARY KEY (WaiverId),
    CONSTRAINT UQ_CorrectiveAction UNIQUE (Code),
    CONSTRAINT FK_Waiver_Status FOREIGN KEY (WaiverStatusId) REFERENCES Compliance.WaiverStatus(WaiverStatusId),
    CONSTRAINT FK_Waiver_Reason FOREIGN KEY (WaiverReasonId) REFERENCES Compliance.WaiverReason(WaiverReasonId),
    CONSTRAINT FK_Waiver_RiskRating FOREIGN KEY (RiskRatingId) REFERENCES Compliance.RiskRating(RiskRatingId),
    CONSTRAINT FK_Waiver_Requirement FOREIGN KEY (RequirementId) REFERENCES Compliance.Requirement(RequirementId),
    CONSTRAINT FK_Waiver_Control FOREIGN KEY (ControlId) REFERENCES Compliance.Control(ControlId),
    CONSTRAINT FK_Waiver_Site FOREIGN KEY (SiteId) REFERENCES Organization.Site(SiteId),
    CONSTRAINT FK_Waiver_Plant FOREIGN KEY (PlantId) REFERENCES Organization.Plant(PlantId),
    CONSTRAINT FK_Waiver_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_Waiver_RequestedByUser FOREIGN KEY (RequestedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT FK_Waiver_ApprovedByUser FOREIGN KEY (ApprovedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_Waiver_Target CHECK (RequirementId IS NOT NULL OR ControlId IS NOT NULL),

```



```

);
GO
CONSTRAINT CK_Waiver_EffectiveWindow CHECK (EffectiveToUtc >= EffectiveFromUtc)
);
GO

```

```

CREATE TABLE Compliance.ComplianceReview (
    ComplianceReviewId BIGINT IDENTITY(1,1) NOT NULL,
    ReviewStatusId INT NOT NULL,
    ComplianceProgramId INT NULL,
    SiteId INT NULL,
    PlantId INT NULL,
    UnitId INT NULL,
    LedByUserId INT NOT NULL,
    ApprovedByUserId INT NULL,
    Code NVARCHAR(80) NOT NULL,
    Title NVARCHAR(250) NOT NULL,
    ReviewStartUtc DATETIME2(7) NOT NULL,
    ReviewEndUtc DATETIME2(7) NULL,
    Summary NVARCHAR(3000) NULL,
    ApprovedAtUtc DATETIME2(7) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ComplianceReview_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ComplianceReview_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_ComplianceReview_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Compliance_ComplianceReview PRIMARY KEY (ComplianceReviewId),
    CONSTRAINT UQ_Compliance_ComplianceReview Code UNIQUE (Code),
    CONSTRAINT FK_ComplianceReview_Status FOREIGN KEY (ReviewStatusId) REFERENCES Compliance.ReviewStatus(ReviewStatusId),
    CONSTRAINT FK_ComplianceReview_Program FOREIGN KEY (ComplianceProgramId) REFERENCES Compliance.ComplianceProgram(ComplianceProgramId),
    CONSTRAINT FK_ComplianceReview_Site FOREIGN KEY (SiteId) REFERENCES Organization.Site(SiteId),
    CONSTRAINT FK_ComplianceReview_Plant FOREIGN KEY (PlantId) REFERENCES Organization.Plant(PlantId),
    CONSTRAINT FK_ComplianceReview_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT FK_ComplianceReview_LedByUser FOREIGN KEY (LedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT FK_ComplianceReview_ApprovedByUser FOREIGN KEY (ApprovedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_ComplianceReview_TimeWindow CHECK (ReviewEndUtc IS NULL OR ReviewEndUtc >= ReviewStartUtc)
);
GO

CREATE TABLE Compliance.ReviewFinding (
    ReviewFindingId BIGINT IDENTITY(1,1) NOT NULL,
    ComplianceReviewId BIGINT NOT NULL,
    RequirementId INT NULL,
    ControlId INT NULL,
    AuditFindingId BIGINT NULL,
    NonConformanceId BIGINT NULL,
    FindingSeverityId INT NOT NULL,
    FindingStatusId INT NOT NULL,
    RiskRatingId INT NOT NULL,
    Title NVARCHAR(250) NOT NULL,
    FindingText NVARCHAR(MAX) NOT NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReviewFinding_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReviewFinding_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Compliance_ReviewFinding PRIMARY KEY (ReviewFindingId),
    CONSTRAINT FK_ReviewFinding_Review FOREIGN KEY (ComplianceReviewId) REFERENCES Compliance.ComplianceReview(ComplianceReviewId),
    CONSTRAINT FK_ReviewFinding_Requirement FOREIGN KEY (RequirementId) REFERENCES Compliance.Requirement(RequirementId),
    CONSTRAINT FK_ReviewFinding_Control FOREIGN KEY (ControlId) REFERENCES Compliance.Control(ControlId),
    CONSTRAINT FK_ReviewFinding_AuditFinding FOREIGN KEY (AuditFindingId) REFERENCES Compliance.AuditFinding(AuditFindingId),
    CONSTRAINT FK_ReviewFinding_NonConformance FOREIGN KEY (NonConformanceId) REFERENCES Compliance.NonConformance(NonConformanceId),
    CONSTRAINT FK_ReviewFinding_Severity FOREIGN KEY (FindingSeverityId) REFERENCES Compliance.FindingSeverity(FindingSeverityId),
    CONSTRAINT FK_ReviewFinding_Status FOREIGN KEY (FindingStatusId) REFERENCES Compliance.FindingStatus(FindingStatusId),
    CONSTRAINT FK_ReviewFinding_RiskRating FOREIGN KEY (RiskRatingId) REFERENCES Compliance.RiskRating(RiskRatingId)
);
GO

CREATE TABLE Compliance.ObligationCalendar (
    ObligationCalendarId BIGINT IDENTITY(1,1) NOT NULL,
    RequirementId INT NOT NULL,
    ObligationStatusId INT NOT NULL,
    ResponsibleUserId INT NOT NULL,
    SiteId INT NULL,
    PlantId INT NULL,
    UnitId INT NULL,
    DueAtUtc DATETIME2(7) NOT NULL,
    CompletedAtUtc DATETIME2(7) NULL,
    SubmissionReference NVARCHAR(200) NULL,
    Notes NVARCHAR(1000) NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Compliance_ObligationCalendar PRIMARY KEY (ObligationCalendarId),
    CONSTRAINT FK_ObligationCalendar_Requirement FOREIGN KEY (RequirementId) REFERENCES Compliance.Requirement(RequirementId),
    CONSTRAINT FK_ObligationCalendar_Status FOREIGN KEY (ObligationStatusId) REFERENCES Compliance.ObligationStatus(ObligationStatusId),
    CONSTRAINT FK_ObligationCalendar_ResponsibleUser FOREIGN KEY (ResponsibleUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT FK_ObligationCalendar_Site FOREIGN KEY (SiteId) REFERENCES Organization.Site(SiteId),
    CONSTRAINT FK_ObligationCalendar_Plant FOREIGN KEY (PlantId) REFERENCES Organization.Plant(PlantId),
    CONSTRAINT FK_ObligationCalendar_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT CK_ObligationCalendar_CompleteWindow CHECK (CompletedAtUtc IS NULL OR CompletedAtUtc >= DATEADD(day, -3650, DueAtUtc))
);
GO

```

C.15.8 Indexes

```

CREATE INDEX IX_Regulation_RegulatoryBodyId ON Compliance.Regulation(RegulatoryBodyId);
CREATE INDEX IX_Requirement_RegulationId ON Compliance.Requirement(RegulationId);
CREATE INDEX IX_Control_Status_Frequency ON Compliance.Control(ControlStatusId, ControlFrequencyId);
CREATE INDEX IX_ControlRequirementMap_RequirementId ON Compliance.ControlRequirementMap(RequirementId);
CREATE INDEX IX_ComplianceProgram_SiteId_PlantId ON Compliance.ComplianceProgram(SiteId, PlantId);
CREATE INDEX IX_Audit_Program_Status ON Compliance.Audit(ComplianceProgramId, AuditStatusId);
CREATE INDEX IX_Audit_UnitId ON Compliance.Audit(UnitId);
CREATE INDEX IX_AuditFinding_Audit_Status ON Compliance.AuditFinding(AuditId, FindingStatusId);
CREATE INDEX IX_AuditFinding_ResponsibleUser ON Compliance.AuditFinding(ResponsibleUserId, FindingStatusId);
CREATE INDEX IX_CorrectiveAction_Status_Due ON Compliance.CorrectiveAction(CorrectiveActionStatusId, DueAtUtc);
CREATE INDEX IX_Evidence_Status_Type ON Compliance.Evidence(EvidenceStatusId, EvidenceTypeId);
CREATE INDEX IX_EvidenceLink_EvidenceId ON Compliance.EvidenceLink(EvidenceId);
CREATE INDEX IX_Waiver_Status_Effective ON Compliance.Waiver(WaiverStatusId, EffectiveFromUtc, EffectiveToUtc);
CREATE INDEX IX_NonConformance_Unit_Status ON Compliance.NonConformance(UnitId, FindingStatusId);
CREATE INDEX IX_ComplianceReview_Program_Status ON Compliance.ComplianceReview(ComplianceProgramId, ReviewStatusId);
CREATE INDEX IX_ObligationCalendar_Status_Due ON Compliance.ObligationCalendar(ObligationStatusId, DueAtUtc);
GO

```

C.15.9 Seed examples

```
INSERT INTO Compliance.RegulationType (Code, Name, DisplayOrder) VALUES
('LAW','Law',1), ('REGULATION','Regulation',2), ('STANDARD','Standard',3),
('GUIDANCE','Guidance',4), ('LICENSE_CONDITION','License condition',5);

INSERT INTO Compliance.Jurisdiction (Code, Name, DisplayOrder) VALUES
('INTERNATIONAL','International',1), ('EU','European Union',2),
('NATIONAL','National',3), ('LOCAL','Local',4), ('INTERNAL','Internal',5);

INSERT INTO Compliance.FindingSeverity (Code, Name, DisplayOrder) VALUES
('INFO','Information',1), ('MINOR','Minor',2), ('MAJOR','Major',3), ('CRITICAL','Critical',4);

INSERT INTO Compliance.RiskRating (Code, Name, DisplayOrder) VALUES
('LOW','Low',1), ('MEDIUM','Medium',2), ('HIGH','High',3), ('EXTREME','Extreme',4);
```

C.15.10 Foreign-key mapping

FROM	TO	MEANING
RegulatoryBody.CountryId	CorePlatform.Country.CountryId	optional country for a regulator
Regulation.RegulatoryBodyId	Compliance.RegulatoryBody.RegulatoryBodyId	regulation owner
Requirement.RegulationId	Compliance.Regulation.RegulationId	clauses belong to a source
Control.OwnerUserId	Security.ApplicationUser.ApplicationUserId	control owner
Control.OwnerTeamId	Organization.Team.TeamId	control team owner
ComplianceProgram.SiteId	Organization.Site.SiteId	site-scoped program
Audit.UnitId	ReactorFleet.Unit.UnitId	unit-scoped audit
AuditScope.EquipmentId	ReactorFleet.Equipment.EquipmentId	equipment-scoped audit scope
AuditFinding.RequirementId	Compliance.Requirement.RequirementId	finding against requirement
CorrectiveAction.AuditFindingId	Compliance.AuditFinding.AuditFindingId	action from audit finding
EvidenceLink.IncidentId	EventManager.Incident.IncidentId	evidence attached to incident
EvidenceLink.WorkOrderId	Maintenance.WorkOrder.WorkOrderId	evidence attached to work order
EvidenceLink.RootCauseAnalysisId	RootCause.RootCauseAnalysis.RootCauseAnalysisId	evidence attached to RCA
EvidenceLink.ExerciseId	EmergencyPreparedness.Exercise.ExerciseId	evidence attached to exercise
ObligationCalendar.UnitId	ReactorFleet.Unit.UnitId	unit-scoped obligation

C.15.11 Verification queries

```
-- 1. Open findings with overdue corrective actions.
SELECT f.Code, f.Title, ca.Code AS action_code, ca.DueAtUtc
FROM Compliance.AuditFinding AS f
JOIN Compliance.CorrectiveAction AS ca ON ca.AuditFindingId = f.AuditFindingId
```



```

JOIN Compliance.CorrectiveActionStatus AS s ON s.CorrectiveActionStatusId = ca.CorrectiveActionStatusId
WHERE s.Code NOT IN ('VERIFIED','CLOSED','CANCELLED')
AND ca.DueAtUtc < SYSUTCDATETIME();

-- 2. Requirements without any mapped control.
SELECT r.RequirementId, r.ClauseReference, r.Title
FROM Compliance.Requirement AS r
LEFT JOIN Compliance.ControlRequirementMap AS m ON m.RequirementId = r.RequirementId
WHERE m.ControlRequirementMapId IS NULL
AND r.IsDeleted = 0;

-- 3. Evidence attached to a root-cause analysis or emergency exercise.
SELECT e.Code, e.Title, el.RootCauseAnalysisId, el.ExerciseId
FROM Compliance.Evidence AS e
JOIN Compliance.EvidenceLink AS el ON el.EvidenceId = e.EvidenceId
WHERE el.RootCauseAnalysisId IS NOT NULL OR el.ExerciseId IS NOT NULL;

```

C.15.12 Design notes

EvidenceLink is deliberately explicit. Instead of a generic string pair such as `ObjectType/ObjectId`, the table carries nullable foreign keys to the principal evidence targets. It is less compact but far more honest: the database can enforce the relationships that matter.

Corrective actions can come from two origins. A corrective action may answer an audit finding or a non-conformance. The check constraint requires at least one of those origins.

Waivers are time-bounded. A waiver without an effective window is not a waiver; it is an undocumented design change. The schema requires a start and end date.

C.15.13 Standing boundary

This schema models the assurance records required by the NEXUS-1 demonstration. It is structured enough for SQL Server, EF Core, joins, audits and reports, but it is not a certified regulatory implementation. A real deployment would require jurisdiction-specific reporting rules, document retention requirements, legal review, electronic-signature policy and regulator-specific evidence formats.

Appendix C.16 - Reporting Schema

The **Reporting** schema is the read-side presentation layer of the NEXUS-1 data backbone. It owns report definitions, dataset catalogues, dashboards, widgets, KPI definitions, KPI snapshots, generated report runs, export files, subscriptions and access grants. It reads from operational sectors; it does not become the owner of their facts.

Boundary. Reporting is an application reporting model for the NEXUS-1 demonstrator. It is not a regulatory reporting product, not a records-management authority, and not a substitute for the append-only Audit sector.

C.16.1 Sector responsibility

The schema has six jobs: describe reusable datasets, define reports, run and export reports, host dashboards, store KPI snapshots, and manage report access and delivery.

- **Datasets:** named read-side sources, columns, saved queries and source dependencies.
- **Reports:** definitions, parameters, schedules, runs, run parameters and sealed result archives.
- **Exports:** generated report packages and file metadata.
- **Dashboards:** dashboard definitions and widgets bound to reports, datasets or KPIs.
- **KPIs:** KPI definitions and point-in-time snapshots for site, plant or unit scope.
- **Delivery and access:** subscriptions, distribution lists and explicit access grants.

C.16.2 Full table list

DATASET Reporting.DatasetDefinition PK DatasetDefinitionId FK DatasetTypeId, OwnerUserId Named read-side data source used by reports, dashboards and KPIs.	DATASET Reporting.DatasetColumn PK DatasetColumnId FK DatasetDefinitionId, EngineeringUnitId Column catalogue for a dataset, including display metadata and unit hints.
DATASET Reporting.SavedQuery PK SavedQueryId FK DatasetDefinitionId, SavedByUserId Saved parameterized query over a dataset for repeatable analysis.	DATASET Reporting.ReportSourceDependency PK ReportSourceDependencyId FK DatasetDefinitionId, ReportDefinitionId Declared read dependency on a source table, view, procedure or external object.

REPORT DESIGN

Reporting.ReportDefinition

PK ReportDefinitionId

FK ReportTypeId, DatasetDefinitionId,
DefaultFormatId, OwnerUserId

Reusable report definition and template metadata.

REPORT DESIGN

Reporting.ReportParameter

PK ReportParameterId

FK ReportDefinitionId

Parameter declaration for a report definition; value type is enforced by a check constraint.

SCHEDULING

Reporting.ReportSchedule

PK ReportScheduleId

FK ReportDefinitionId, ScheduleFrequencyId,
TimeZonId, CreatedByUserId

Schedule that triggers report runs according to a local time rule.

EXECUTION

Reporting.ReportRun

PK ReportRunId

FK ReportDefinitionId, ReportScheduleId,
ReportRunStatusId, RequestedByUserId

One execution of a report, with scope, parameters and run status.

EXECUTION

Reporting.ReportRunParameter

PK ReportRunParameterId

FK ReportRunId

Concrete parameter values used by one report run.

EXPORT

Reporting.ReportExport

PK ReportExportId

FK ReportRunId, ReportFormatId, ExportStatusId,
RequestedByUserId

Generated export request and lifecycle state.

EXPORT

Reporting.ReportExportFile

PK ReportExportFileId

FK ReportExportId

Physical file metadata for generated report output.

DASHBOARD

Reporting.Dashboard

PK DashboardId

FK OwnerUserId, SiteId, PlantId, UnitId

Named dashboard with optional plant scope.

DASHBOARD

Reporting.DashboardWidget

PK DashboardWidgetId

FK DashboardId, WidgetTypeId, KpiDefinitionId,
ReportDefinitionId, DatasetDefinitionId

One widget placed on a dashboard canvas.

KPI

Reporting.KpiDefinition

PK KpiDefinitionId

FK KpiCategoryId, DatasetDefinitionId,
EngineeringUnitId, OwnerUserId

Definition of a KPI calculation, target and threshold logic.

KPI Reporting.KpiSnapshot PK KpiSnapshotId FK KpiDefinitionId, KpiStatusId, EngineeringUnitId, SiteId, PlantId, UnitId Point-in-time KPI value for a scope.	DELIVERY Reporting.DistributionList PK DistributionListId FK OwnerUserId Named recipient group for report delivery.
DELIVERY Reporting.DistributionListMember PK DistributionListMemberId FK DistributionListId, ApplicationUserId A user or external email member of a distribution list.	DELIVERY Reporting.ReportSubscription PK ReportSubscriptionId FK ReportDefinitionId, ApplicationUserId, DistributionListId, SubscriptionStatusId Subscription to receive or publish a report on a schedule.
SECURITY Reporting.ReportAccessGrant PK ReportAccessGrantId FK ReportDefinitionId, DashboardId, DatasetDefinitionId, ApplicationUserId, ApplicationRoleId Explicit access grant for report, dashboard or dataset objects.	ARCHIVE Reporting.ReportSnapshotArchive PK ReportSnapshotArchiveId FK ReportRunId Sealed JSON snapshot of a report result for reproducibility.

C.16.3 Lookup tables

LOOKUP TABLE	SEED CODES	PURPOSE
Reporting.ReportType	OPERATIONAL , COMPLIANCE , MAINTENANCE , RADIATION , EXECUTIVE , TRAINING , TECHNICAL	Classifies the business purpose of a report.
Reporting.ReportFormat	PDF , XLSX , CSV , JSON , HTML	Output format for generated reports and exports.
Reporting.ReportRunStatus	QUEUED , RUNNING , COMPLETED , FAILED , CANCELLED	Lifecycle state of a report run.
Reporting.ExportStatus	REQUESTED , GENERATING , READY , FAILED , EXPIRED	Lifecycle state of an export artefact.
Reporting.ScheduleFrequency	ON_DEMAND , HOURLY , DAILY , WEEKLY , MONTHLY , QUARTERLY , ANNUAL	Frequency used by schedules and subscriptions.
Reporting.WidgetType	KPI_CARD , TABLE , LINE_CHART , BAR_CHART , PIE_CHART , GAUGE , TIMELINE , LINK	Presentation kind of a dashboard widget.
Reporting.KpiCategory		Domain category for KPI definitions.

LOOKUP TABLE	SEED CODES	PURPOSE
	SAFETY , OPERATIONS , MAINTENANCE , COMPLIANCE , RADIATION , TRAINING , SECURITY	
Reporting.KpiStatus	GREEN , AMBER , RED , UNKNOWN	Traffic-light status assigned to a KPI snapshot.
Reporting.DatasetType	SQL_VIEW , STORED_QUERY , SNAPSHOT , EXTERNAL_FILE	Where a dataset obtains its data.
Reporting.AccessLevel	VIEW , EDIT , RUN , EXPORT , ADMIN	Permission level granted on reporting assets.
Reporting.SubscriptionStatus	ACTIVE , PAUSED , FAILED , CANCELLED	Lifecycle state of a report subscription.
Reporting.DeliveryChannel	EMAIL , DOWNLOAD , FOLDER , API	Delivery mechanism for scheduled report output.

C.16.4 ER diagram

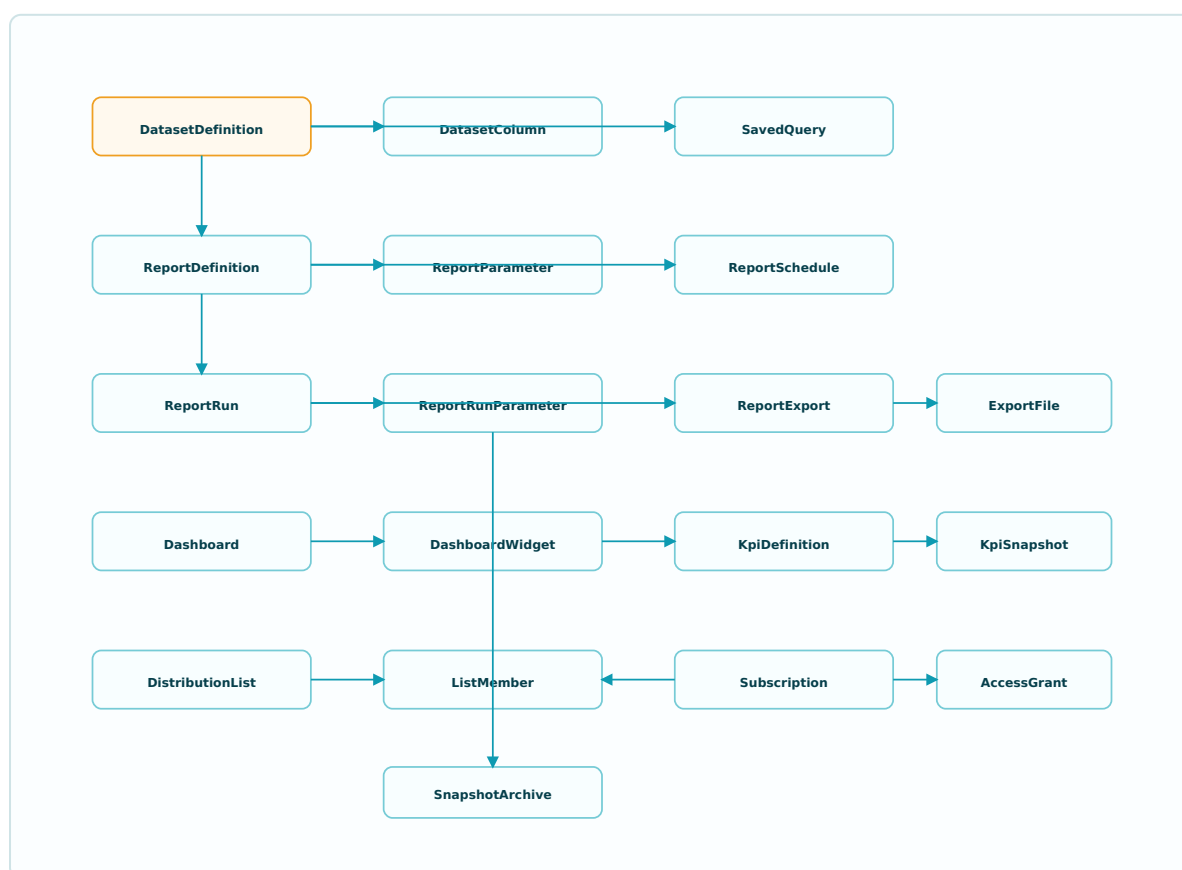


Figure C.16.1 - Reporting sector ER diagram. Datasets feed reports and KPIs; reports produce runs and exports; dashboards host widgets; subscriptions and access grants govern delivery and visibility.

C.16.5 Foreign-key passport map

CorePlatform

EngineeringUnit and TimeZone provide units of measure and schedule time-zone conversion.

Security

ApplicationUser and ApplicationRole own reports, schedule runs, receive subscriptions and receive access grants.

Organization

Site and Plant allow reports, dashboards and KPI snapshots to be scoped to an organizational level.

ReactorFleet

Unit provides the physical unit scope for report runs, dashboards and KPI snapshots.

Operational schemas

Datasets and source dependencies declare read-side links to AlarmManagement, Maintenance, Compliance, RootCause, RadiationMonitoring and others without owning their data.

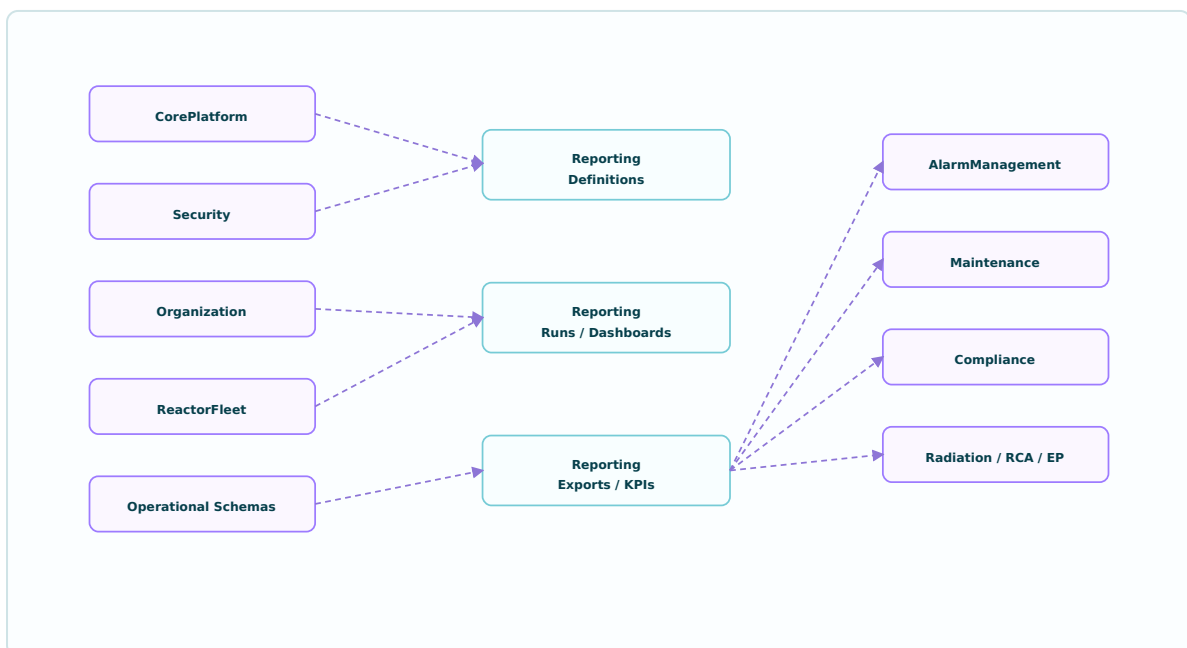


Figure C.16.2 - Cross-schema passport map. Reporting holds explicit foreign keys to platform identity and scope tables; operational data is read through datasets and declared source dependencies rather than copied as ownership.

C.16.6 Lookup DDL

```
CREATE TABLE Reporting.ReportType (  
  ReportTypeId INT IDENTITY(1,1) NOT NULL,  
  Code NVARCHAR(50) NOT NULL,  
  Name NVARCHAR(150) NOT NULL,
```

```

Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_Reporting_ReportType_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_Reporting_ReportType_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Reporting_ReportType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Reporting_ReportType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_Reporting_ReportType
PRIMARY KEY (ReportTypeId),
CONSTRAINT UQ_Reporting_ReportType_Code
UNIQUE (Code)
);
GO

CREATE TABLE Reporting.ReportFormat (
ReportFormatId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_Reporting_ReportFormat_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_Reporting_ReportFormat_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Reporting_ReportFormat_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Reporting_ReportFormat_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_Reporting_ReportFormat
PRIMARY KEY (ReportFormatId),
CONSTRAINT UQ_Reporting_ReportFormat_Code
UNIQUE (Code)
);
GO

CREATE TABLE Reporting.ReportRunStatus (
ReportRunStatusId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_Reporting_ReportRunStatus_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_Reporting_ReportRunStatus_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Reporting_ReportRunStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Reporting_ReportRunStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_Reporting_ReportRunStatus
PRIMARY KEY (ReportRunStatusId),
CONSTRAINT UQ_Reporting_ReportRunStatus_Code
UNIQUE (Code)
);
GO

CREATE TABLE Reporting.ExportStatus (
ExportStatusId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_Reporting_ExportStatus_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_Reporting_ExportStatus_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Reporting_ExportStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Reporting_ExportStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_Reporting_ExportStatus
PRIMARY KEY (ExportStatusId),
CONSTRAINT UQ_Reporting_ExportStatus_Code
UNIQUE (Code)
);
GO

CREATE TABLE Reporting.ScheduleFrequency (
ScheduleFrequencyId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_Reporting_ScheduleFrequency_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_Reporting_ScheduleFrequency_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Reporting_ScheduleFrequency_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Reporting_ScheduleFrequency_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_Reporting_ScheduleFrequency
PRIMARY KEY (ScheduleFrequencyId),
CONSTRAINT UQ_Reporting_ScheduleFrequency_Code
UNIQUE (Code)
);
GO

CREATE TABLE Reporting.WidgetType (
WidgetTypeId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_Reporting_WidgetType_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_Reporting_WidgetType_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Reporting_WidgetType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Reporting_WidgetType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_Reporting_WidgetType
PRIMARY KEY (WidgetTypeId),
CONSTRAINT UQ_Reporting_WidgetType_Code
UNIQUE (Code)
);
GO

CREATE TABLE Reporting.KpiCategory (
KpiCategoryId INT IDENTITY(1,1) NOT NULL,
Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_Reporting_KpiCategory_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_Reporting_KpiCategory_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Reporting_KpiCategory_CreatedAtUtc DEFAULT SYSUTCDATETIME(),

```

```

        ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Reporting_KpiCategory_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
        RowVersion ROWVERSION NOT NULL,

        CONSTRAINT PK_Reporting_KpiCategory
        PRIMARY KEY (KpiCategoryId),
        CONSTRAINT UQ_Reporting_KpiCategory_Code
        UNIQUE (Code)
    );
GO

CREATE TABLE Reporting.KpiStatus (
    KpiStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_Reporting_KpiStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_Reporting_KpiStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Reporting_KpiStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Reporting_KpiStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Reporting_KpiStatus
    PRIMARY KEY (KpiStatusId),
    CONSTRAINT UQ_Reporting_KpiStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Reporting.DatasetType (
    DatasetTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_Reporting_DatasetType_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_Reporting_DatasetType_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Reporting_DatasetType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Reporting_DatasetType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Reporting_DatasetType
    PRIMARY KEY (DatasetTypeId),
    CONSTRAINT UQ_Reporting_DatasetType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Reporting.AccessLevel (
    AccessLevelId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_Reporting_AccessLevel_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_Reporting_AccessLevel_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Reporting_AccessLevel_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Reporting_AccessLevel_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Reporting_AccessLevel
    PRIMARY KEY (AccessLevelId),
    CONSTRAINT UQ_Reporting_AccessLevel_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Reporting.SubscriptionStatus (
    SubscriptionStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_Reporting_SubscriptionStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_Reporting_SubscriptionStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Reporting_SubscriptionStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Reporting_SubscriptionStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Reporting_SubscriptionStatus
    PRIMARY KEY (SubscriptionStatusId),
    CONSTRAINT UQ_Reporting_SubscriptionStatus_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Reporting.DeliveryChannel (
    DeliveryChannelId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_Reporting_DeliveryChannel_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_Reporting_DeliveryChannel_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Reporting_DeliveryChannel_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Reporting_DeliveryChannel_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Reporting_DeliveryChannel
    PRIMARY KEY (DeliveryChannelId),
    CONSTRAINT UQ_Reporting_DeliveryChannel_Code
    UNIQUE (Code)
);
GO

```


C.16.7 Main table DDL

Datasets and report design

```
CREATE TABLE Reporting.DatasetDefinition (
    DatasetDefinitionId INT IDENTITY(1,1) NOT NULL,
    DatasetTypeId INT NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(250) NOT NULL,
    Description NVARCHAR(2000) NULL,
    SourceSchema NVARCHAR(128) NULL,
    SourceObject NVARCHAR(256) NULL,
    SqlText NVARCHAR(MAX) NULL,
    IsParameterized BIT NOT NULL CONSTRAINT DF_DatasetDefinition_IsParameterized DEFAULT 0,
    OwnerUserId INT NULL,
    IsActive BIT NOT NULL CONSTRAINT DF_DatasetDefinition_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DatasetDefinition_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DatasetDefinition_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_DatasetDefinition_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Reporting_DatasetDefinition PRIMARY KEY (DatasetDefinitionId),
    CONSTRAINT UQ_Reporting_DatasetDefinition Code UNIQUE (Code),
    CONSTRAINT FK_DatasetDefinition_DatasetType FOREIGN KEY (DatasetTypeId) REFERENCES Reporting.DatasetType(DatasetTypeId),
    CONSTRAINT FK_DatasetDefinition_OwnerUser FOREIGN KEY (OwnerUserId) REFERENCES Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE Reporting.DatasetColumn (
    DatasetColumnId INT IDENTITY(1,1) NOT NULL,
    DatasetDefinitionId INT NOT NULL,
    ColumnName NVARCHAR(128) NOT NULL,
    DisplayName NVARCHAR(200) NOT NULL,
    DataType NVARCHAR(50) NOT NULL,
    EngineeringUnitId INT NULL,
    IsNullable BIT NOT NULL CONSTRAINT DF_DatasetColumn_IsNullable DEFAULT 1,
    IsMeasure BIT NOT NULL CONSTRAINT DF_DatasetColumn_IsMeasure DEFAULT 0,
    DisplayOrder INT NOT NULL CONSTRAINT DF_DatasetColumn_DisplayOrder DEFAULT 0,
    FormatString NVARCHAR(100) NULL,
    Description NVARCHAR(500) NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Reporting_DatasetColumn PRIMARY KEY (DatasetColumnId),
    CONSTRAINT UQ_Reporting_DatasetColumn_DatasetColumn UNIQUE (DatasetDefinitionId, ColumnName),
    CONSTRAINT FK_DatasetColumn_DatasetDefinition FOREIGN KEY (DatasetDefinitionId) REFERENCES Reporting.DatasetDefinition(DatasetDefinitionId),
    CONSTRAINT FK_DatasetColumn_EngineeringUnit FOREIGN KEY (EngineeringUnitId) REFERENCES CorePlatform.EngineeringUnit(EngineeringUnitId)
);
GO

CREATE TABLE Reporting.SavedQuery (
    SavedQueryId INT IDENTITY(1,1) NOT NULL,
    DatasetDefinitionId INT NOT NULL,
    SavedByUserId INT NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    QueryText NVARCHAR(MAX) NOT NULL,
    ParameterJson NVARCHAR(MAX) NULL,
    IsShared BIT NOT NULL CONSTRAINT DF_SavedQuery_IsShared DEFAULT 0,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_SavedQuery_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_SavedQuery_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Reporting_SavedQuery PRIMARY KEY (SavedQueryId),
    CONSTRAINT FK_SavedQuery_DatasetDefinition FOREIGN KEY (DatasetDefinitionId) REFERENCES Reporting.DatasetDefinition(DatasetDefinitionId),
    CONSTRAINT FK_SavedQuery_SavedByUser FOREIGN KEY (SavedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_SavedQuery_ParameterJson CHECK (ParameterJson IS NULL OR ISJSON(ParameterJson) = 1)
);
GO

CREATE TABLE Reporting.ReportSourceDependency (
    ReportSourceDependencyId INT IDENTITY(1,1) NOT NULL,
    DatasetDefinitionId INT NOT NULL,
    ReportDefinitionId INT NOT NULL,
    SourceSchema NVARCHAR(128) NOT NULL,
    SourceObject NVARCHAR(256) NOT NULL,
    DependencyType NVARCHAR(40) NOT NULL,
    Notes NVARCHAR(1000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReportSourceDependency_CreatedAtUtc DEFAULT SYSUTCDATETIME(),

    CONSTRAINT PK_Reporting_ReportSourceDependency PRIMARY KEY (ReportSourceDependencyId),
    CONSTRAINT CK_ReportSourceDependency_Target CHECK (
        (CASE WHEN DatasetDefinitionId IS NULL THEN 0 ELSE 1 END) +
        (CASE WHEN ReportDefinitionId IS NULL THEN 0 ELSE 1 END) = 1
    ),
    CONSTRAINT CK_ReportSourceDependency_Type CHECK (DependencyType IN ('TABLE', 'VIEW', 'PROCEDURE', 'FUNCTION', 'EXTERNAL')),
    CONSTRAINT FK_ReportSourceDependency_DatasetDefinition FOREIGN KEY (DatasetDefinitionId) REFERENCES Reporting.DatasetDefinition(DatasetDefinitionId)
    -- FK to ReportDefinition is added after ReportDefinition is created.
);
GO
```

```
CREATE TABLE Reporting.ReportDefinition (
    ReportDefinitionId INT IDENTITY(1,1) NOT NULL,
    ReportTypeId INT NOT NULL,
    DatasetDefinitionId INT NOT NULL,
    DefaultFormatId INT NOT NULL,
    OwnerUserId INT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(250) NOT NULL,
    Description NVARCHAR(2000) NULL,
    TemplateJson NVARCHAR(MAX) NULL,
    VersionNumber INT NOT NULL CONSTRAINT DF_ReportDefinition_VersionNumber DEFAULT 1,
    IsActive BIT NOT NULL CONSTRAINT DF_ReportDefinition_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReportDefinition_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReportDefinition_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_ReportDefinition_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Reporting_ReportDefinition PRIMARY KEY (ReportDefinitionId),
    CONSTRAINT UQ_Reporting_ReportDefinition Code UNIQUE (Code),
    CONSTRAINT FK_ReportDefinition_ReportType FOREIGN KEY (ReportTypeId) REFERENCES Reporting.ReportType(ReportTypeId),
    CONSTRAINT FK_ReportDefinition_DatasetDefinition FOREIGN KEY (DatasetDefinitionId) REFERENCES Reporting.DatasetDefinition(DatasetDefinitionId),
    CONSTRAINT FK_ReportDefinition_DefaultFormat FOREIGN KEY (DefaultFormatId) REFERENCES Reporting.ReportFormat(ReportFormatId),
    CONSTRAINT FK_ReportDefinition_OwnerUser FOREIGN KEY (OwnerUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_ReportDefinition_TemplateJson CHECK (TemplateJson IS NULL OR ISJSON(TemplateJson) = 1)
);
GO

ALTER TABLE Reporting.ReportSourceDependency ADD CONSTRAINT FK_ReportSourceDependency_ReportDefinition
    FOREIGN KEY (ReportDefinitionId) REFERENCES Reporting.ReportDefinition(ReportDefinitionId);
GO

CREATE TABLE Reporting.ReportParameter (
    ReportParameterId INT IDENTITY(1,1) NOT NULL,
    ReportDefinitionId INT NOT NULL,
```

```

Code NVARCHAR(80) NOT NULL,
Name NVARCHAR(200) NOT NULL,
ValueType NVARCHAR(30) NOT NULL,
DefaultValue NVARCHAR(1000) NULL,
IsRequired BIT NOT NULL CONSTRAINT DF_ReportParameter_IsRequired DEFAULT 0,
DisplayOrder INT NOT NULL CONSTRAINT DF_ReportParameter_DisplayOrder DEFAULT 0,
HelpText NVARCHAR(500) NULL,
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_Reporting_ReportParameter PRIMARY KEY (ReportParameterId),
CONSTRAINT UQ_ReportParameter_Report_Code UNIQUE (ReportDefinitionId, Code),
CONSTRAINT FK_ReportParameter_ReportDefinition FOREIGN KEY (ReportDefinitionId) REFERENCES Reporting.ReportDefinition(ReportDefinitionId),
CONSTRAINT CK_ReportParameter_ValueType CHECK (ValueType IN ('String','Int','Decimal','Bool','Date','DateTime','Guid','Json'))
);
GO

CREATE TABLE Reporting.ReportSchedule (
    ReportScheduleId INT IDENTITY(1,1) NOT NULL,
    ReportDefinitionId INT NOT NULL,
    ScheduleFrequencyId INT NOT NULL,
    TimeZoneId INT NOT NULL,
    CreatedByUserId INT NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    CronExpression NVARCHAR(120) NULL,
    StartAtUtc DATETIME2(7) NULL,
    EndAtUtc DATETIME2(7) NULL,
    LastRunAtUtc DATETIME2(7) NULL,
    NextRunAtUtc DATETIME2(7) NULL,
    IsEnabled BIT NOT NULL CONSTRAINT DF_ReportSchedule_IsEnabled DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReportSchedule_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReportSchedule_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Reporting_ReportSchedule PRIMARY KEY (ReportScheduleId),
    CONSTRAINT FK_ReportSchedule_ReportDefinition FOREIGN KEY (ReportDefinitionId) REFERENCES Reporting.ReportDefinition(ReportDefinitionId),
    CONSTRAINT FK_ReportSchedule_ScheduleFrequency FOREIGN KEY (ScheduleFrequencyId) REFERENCES Reporting.ScheduleFrequency(ScheduleFrequencyId),
    CONSTRAINT FK_ReportSchedule_TimeZone FOREIGN KEY (TimeZoneId) REFERENCES CorePlatform.TimeZone(TimeZoneId),
    CONSTRAINT FK_ReportSchedule_CreatedByUser FOREIGN KEY (CreatedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_ReportSchedule_Window CHECK (EndAtUtc IS NULL OR StartAtUtc IS NULL OR EndAtUtc >= StartAtUtc)
);
GO

```

Runs, exports and archives

```

CREATE TABLE Reporting.ReportRun (
    ReportRunId BIGINT IDENTITY(1,1) NOT NULL,
    ReportDefinitionId INT NOT NULL,
    ReportScheduleId INT NOT NULL,
    ReportRunStatusId INT NOT NULL,
    RequestedByUserId INT NOT NULL,
    SiteId INT NOT NULL,
    PlantId INT NOT NULL,
    UnitId INT NOT NULL,
    RequestedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReportRun_RequestedAtUtc DEFAULT SYSUTCDATETIME(),
    StartedAtUtc DATETIME2(7) NULL,
    CompletedAtUtc DATETIME2(7) NULL,
    ParameterJson NVARCHAR(MAX) NULL,
    ResultSummaryJson NVARCHAR(MAX) NULL,
    ErrorMessage NVARCHAR(4000) NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Reporting_ReportRun PRIMARY KEY (ReportRunId),
    CONSTRAINT FK_ReportRun_ReportDefinition FOREIGN KEY (ReportDefinitionId) REFERENCES Reporting.ReportDefinition(ReportDefinitionId),
    CONSTRAINT FK_ReportRun_ReportSchedule FOREIGN KEY (ReportScheduleId) REFERENCES Reporting.ReportSchedule(ReportScheduleId),
    CONSTRAINT FK_ReportRun_ReportRunStatus FOREIGN KEY (ReportRunStatusId) REFERENCES Reporting.ReportRunStatus(ReportRunStatusId),
    CONSTRAINT FK_ReportRun_RequestedByUser FOREIGN KEY (RequestedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT FK_ReportRun_Site FOREIGN KEY (SiteId) REFERENCES Organization.Site(SiteId),
    CONSTRAINT FK_ReportRun_Plant FOREIGN KEY (PlantId) REFERENCES Organization.Plant(PlantId),
    CONSTRAINT FK_ReportRun_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT CK_ReportRun_ParameterJson CHECK (ParameterJson IS NULL OR ISJSON(ParameterJson) = 1),
    CONSTRAINT CK_ReportRun_ResultSummaryJson CHECK (ResultSummaryJson IS NULL OR ISJSON(ResultSummaryJson) = 1),
    CONSTRAINT CK_ReportRun_Window CHECK (CompletedAtUtc IS NULL OR StartedAtUtc IS NULL OR CompletedAtUtc >= StartedAtUtc)
);
GO

CREATE TABLE Reporting.ReportRunParameter (
    ReportRunParameterId BIGINT IDENTITY(1,1) NOT NULL,
    ReportRunId BIGINT NOT NULL,
    ParameterCode NVARCHAR(80) NOT NULL,
    ParameterValue NVARCHAR(MAX) NULL,

    CONSTRAINT PK_Reporting_ReportRunParameter PRIMARY KEY (ReportRunParameterId),
    CONSTRAINT UQ_ReportRunParameter_Run_Code UNIQUE (ReportRunId, ParameterCode),
    CONSTRAINT FK_ReportRunParameter_ReportRun FOREIGN KEY (ReportRunId) REFERENCES Reporting.ReportRun(ReportRunId)
);
GO

CREATE TABLE Reporting.ReportExport (
    ReportExportId BIGINT IDENTITY(1,1) NOT NULL,
    ReportRunId BIGINT NOT NULL,
    ReportFormatId INT NOT NULL,
    ExportStatusId INT NOT NULL,
    RequestedByUserId INT NOT NULL,
    RequestedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReportExport_RequestedAtUtc DEFAULT SYSUTCDATETIME(),
    CompletedAtUtc DATETIME2(7) NULL,
    ExpiresAtUtc DATETIME2(7) NULL,
    FileCount INT NOT NULL CONSTRAINT DF_ReportExport_FileCount DEFAULT 0,
    ErrorMessage NVARCHAR(4000) NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Reporting_ReportExport PRIMARY KEY (ReportExportId),
    CONSTRAINT FK_ReportExport_ReportRun FOREIGN KEY (ReportRunId) REFERENCES Reporting.ReportRun(ReportRunId),
    CONSTRAINT FK_ReportExport_ReportFormat FOREIGN KEY (ReportFormatId) REFERENCES Reporting.ReportFormat(ReportFormatId),
    CONSTRAINT FK_ReportExport_ExportStatus FOREIGN KEY (ExportStatusId) REFERENCES Reporting.ExportStatus(ExportStatusId),
    CONSTRAINT FK_ReportExport_RequestedByUser FOREIGN KEY (RequestedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_ReportExport_FileCount CHECK (FileCount >= 0)
);
GO

CREATE TABLE Reporting.ReportExportFile (
    ReportExportFileId BIGINT IDENTITY(1,1) NOT NULL,
    ReportExportId BIGINT NOT NULL,
    FileName NVARCHAR(260) NOT NULL,
    ContentType NVARCHAR(120) NOT NULL,
    StorageUri NVARCHAR(1000) NOT NULL,
    ByteSize BIGINT NOT NULL,
    ChecksumSha256 CHAR(64) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReportExportFile_CreatedAtUtc DEFAULT SYSUTCDATETIME(),

    CONSTRAINT PK_Reporting_ReportExportFile PRIMARY KEY (ReportExportFileId),
    CONSTRAINT FK_ReportExportFile_ReportExport FOREIGN KEY (ReportExportId) REFERENCES Reporting.ReportExport(ReportExportId),
    CONSTRAINT CK_ReportExportFile_ByteSize CHECK (ByteSize >= 0)
);
GO

CREATE TABLE Reporting.ReportSnapshotArchive (
    ReportSnapshotArchiveId BIGINT IDENTITY(1,1) NOT NULL,
    ReportRunId BIGINT NOT NULL,
    SnapshotJson NVARCHAR(MAX) NOT NULL,
    SnapshotHashSha256 CHAR(64) NOT NULL,
    CapturedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReportSnapshotArchive_CapturedAtUtc DEFAULT SYSUTCDATETIME(),

    CONSTRAINT PK_Reporting_ReportSnapshotArchive PRIMARY KEY (ReportSnapshotArchiveId),
    CONSTRAINT UQ_ReportSnapshotArchive_ReportRun UNIQUE (ReportRunId),
    CONSTRAINT FK_ReportSnapshotArchive_ReportRun FOREIGN KEY (ReportRunId) REFERENCES Reporting.ReportRun(ReportRunId),

```

```
);
GO
CONSTRAINT CK_ReportSnapshotArchive_Json CHECK (ISJSON(SnapshotJson) = 1)
```

Dashboards and KPIs

```
CREATE TABLE Reporting_Dashboard (
    DashboardId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(250) NOT NULL,
    Description NVARCHAR(2000) NULL,
    OwnerUserId INT NULL,
    SiteId INT NULL,
    PlantId INT NULL,
    UnitId INT NULL,
    IsDefault BIT NOT NULL CONSTRAINT DF_Dashboard_IsDefault DEFAULT 0,
    IsActive BIT NOT NULL CONSTRAINT DF_Dashboard_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Dashboard_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_Dashboard_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_Dashboard_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Reporting_Dashboard PRIMARY KEY (DashboardId),
    CONSTRAINT UQ_Reporting_Dashboard_Code UNIQUE (Code),
    CONSTRAINT FK_Dashboard_OwnerUser FOREIGN KEY (OwnerUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT FK_Dashboard_Site FOREIGN KEY (SiteId) REFERENCES Organization.Site(SiteId),
    CONSTRAINT FK_Dashboard_Plant FOREIGN KEY (PlantId) REFERENCES Organization.Plant(PlantId),
    CONSTRAINT FK_Dashboard_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId)
);
GO

CREATE TABLE Reporting_KpiDefinition (
    KpiDefinitionId INT IDENTITY(1,1) NOT NULL,
    KpiCategoryId INT NOT NULL,
    DatasetDefinitionId INT NULL,
    EngineeringUnitId INT NULL,
    OwnerUserId INT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(250) NOT NULL,
    Description NVARCHAR(2000) NULL,
    ExpressionSql NVARCHAR(MAX) NULL,
    TargetValue DECIMAL(18,6) NULL,
    WarningThreshold DECIMAL(18,6) NULL,
    CriticalThreshold DECIMAL(18,6) NULL,
    Direction NVARCHAR(50) NOT NULL CONSTRAINT DF_KpiDefinition_Direction DEFAULT 'HIGHER_IS_BETTER',
    IsActive BIT NOT NULL CONSTRAINT DF_KpiDefinition_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_KpiDefinition_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_KpiDefinition_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    IsDeleted BIT NOT NULL CONSTRAINT DF_KpiDefinition_IsDeleted DEFAULT 0,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Reporting_KpiDefinition PRIMARY KEY (KpiDefinitionId),
    CONSTRAINT UQ_Reporting_KpiDefinition_Code UNIQUE (Code),
    CONSTRAINT FK_KpiDefinition_KpiCategory FOREIGN KEY (KpiCategoryId) REFERENCES Reporting.KpiCategory(KpiCategoryId),
    CONSTRAINT FK_KpiDefinition_DatasetDefinition FOREIGN KEY (DatasetDefinitionId) REFERENCES Reporting.DatasetDefinition(DatasetDefinitionId),
    CONSTRAINT FK_KpiDefinition_EngineeringUnit FOREIGN KEY (EngineeringUnitId) REFERENCES CorePlatform.EngineeringUnit(EngineeringUnitId),
    CONSTRAINT FK_KpiDefinition_OwnerUser FOREIGN KEY (OwnerUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_KpiDefinition_Direction CHECK (Direction IN ('HIGHER_IS_BETTER','LOWER_IS_BETTER','RANGE'))
);
GO

CREATE TABLE Reporting_DashboardWidget (
    DashboardWidgetId INT IDENTITY(1,1) NOT NULL,
    DashboardId INT NOT NULL,
    WidgetTypeId INT NOT NULL,
    KpiDefinitionId INT NULL,
    ReportDefinitionId INT NULL,
    DatasetDefinitionId INT NULL,
    Title NVARCHAR(200) NOT NULL,
    ConfigJson NVARCHAR(MAX) NULL,
    PositionX INT NOT NULL CONSTRAINT DF_DashboardWidget_PositionX DEFAULT 0,
    PositionY INT NOT NULL CONSTRAINT DF_DashboardWidget_PositionY DEFAULT 0,
    WidthUnits INT NOT NULL CONSTRAINT DF_DashboardWidget_WidthUnits DEFAULT 4,
    HeightUnits INT NOT NULL CONSTRAINT DF_DashboardWidget_HeightUnits DEFAULT 3,
    RefreshSeconds INT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Reporting_DashboardWidget PRIMARY KEY (DashboardWidgetId),
    CONSTRAINT FK_DashboardWidget_Dashboard FOREIGN KEY (DashboardId) REFERENCES Reporting.Dashboard(DashboardId),
    CONSTRAINT FK_DashboardWidget_WidgetType FOREIGN KEY (WidgetTypeId) REFERENCES Reporting.WidgetType(WidgetTypeId),
    CONSTRAINT FK_DashboardWidget_KpiDefinition FOREIGN KEY (KpiDefinitionId) REFERENCES Reporting.KpiDefinition(KpiDefinitionId),
    CONSTRAINT FK_DashboardWidget_ReportDefinition FOREIGN KEY (ReportDefinitionId) REFERENCES Reporting.ReportDefinition(ReportDefinitionId),
    CONSTRAINT FK_DashboardWidget_DatasetDefinition FOREIGN KEY (DatasetDefinitionId) REFERENCES Reporting.DatasetDefinition(DatasetDefinitionId),
    CONSTRAINT CK_DashboardWidget_ConfigJson CHECK (ConfigJson IS NULL OR ISJSON(ConfigJson) = 1),
    CONSTRAINT CK_DashboardWidget_Size CHECK (WidthUnits > 0 AND HeightUnits > 0)
);
GO

CREATE TABLE Reporting_KpiSnapshot (
    KpiSnapshotId BIGINT IDENTITY(1,1) NOT NULL,
    KpiDefinitionId INT NOT NULL,
    KpiStatusId INT NOT NULL,
    EngineeringUnitId INT NULL,
    SiteId INT NULL,
    PlantId INT NULL,
    UnitId INT NULL,
    SnapshotAtUtc DATETIME2(7) NOT NULL,
    NumericValue DECIMAL(18,6) NULL,
    TextValue NVARCHAR(500) NULL,
    DetailJson NVARCHAR(MAX) NULL,

    CONSTRAINT PK_Reporting_KpiSnapshot PRIMARY KEY (KpiSnapshotId),
    CONSTRAINT FK_KpiSnapshot_KpiDefinition FOREIGN KEY (KpiDefinitionId) REFERENCES Reporting.KpiDefinition(KpiDefinitionId),
    CONSTRAINT FK_KpiSnapshot_KpiStatus FOREIGN KEY (KpiStatusId) REFERENCES Reporting.KpiStatus(KpiStatusId),
    CONSTRAINT FK_KpiSnapshot_EngineeringUnit FOREIGN KEY (EngineeringUnitId) REFERENCES CorePlatform.EngineeringUnit(EngineeringUnitId),
    CONSTRAINT FK_KpiSnapshot_Site FOREIGN KEY (SiteId) REFERENCES Organization.Site(SiteId),
    CONSTRAINT FK_KpiSnapshot_Plant FOREIGN KEY (PlantId) REFERENCES Organization.Plant(PlantId),
    CONSTRAINT FK_KpiSnapshot_Unit FOREIGN KEY (UnitId) REFERENCES ReactorFleet.Unit(UnitId),
    CONSTRAINT CK_KpiSnapshot_Value CHECK (NumericValue IS NOT NULL OR TextValue IS NOT NULL),
    CONSTRAINT CK_KpiSnapshot_DetailJson CHECK (DetailJson IS NULL OR ISJSON(DetailJson) = 1)
);
GO
```

Delivery and access

```
CREATE TABLE Reporting_DistributionList (
    DistributionListId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    OwnerUserId INT NULL,
    Description NVARCHAR(1000) NULL,
    IsActive BIT NOT NULL CONSTRAINT DF_DistributionList_IsActive DEFAULT 1,
```

```

    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DistributionList_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DistributionList_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Reporting_DistributionList PRIMARY KEY (DistributionListId),
    CONSTRAINT UQ_Reporting_DistributionList_Code UNIQUE (Code),
    CONSTRAINT FK_DistributionList_OwnerUser FOREIGN KEY (OwnerUserId) REFERENCES Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE Reporting.DistributionListMember (
    DistributionListMemberId INT IDENTITY(1,1) NOT NULL,
    DistributionListId INT NOT NULL,
    ApplicationUserId INT NULL,
    ExternalEmail NVARCHAR(320) NULL,
    DisplayName NVARCHAR(200) NULL,
    IsActive BIT NOT NULL CONSTRAINT DF_DistributionListMember_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DistributionListMember_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Reporting_DistributionListMember PRIMARY KEY (DistributionListMemberId),
    CONSTRAINT FK_DistributionListMember_DistributionList FOREIGN KEY (DistributionListId) REFERENCES Reporting.DistributionList(DistributionListId),
    CONSTRAINT FK_DistributionListMember_ApplicationUser FOREIGN KEY (ApplicationUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_DistributionListMember_Principal CHECK (ApplicationUserId IS NOT NULL OR ExternalEmail IS NOT NULL)
);
GO

CREATE TABLE Reporting.ReportSubscription (
    ReportSubscriptionId INT IDENTITY(1,1) NOT NULL,
    ReportDefinitionId INT NOT NULL,
    ApplicationUserId INT NULL,
    DistributionListId INT NULL,
    SubscriptionStatusId INT NOT NULL,
    DeliveryChannelId INT NOT NULL,
    ReportFormatId INT NOT NULL,
    ScheduleFrequencyId INT NOT NULL,
    TimeZoneId INT NULL,
    ParameterJson NVARCHAR(MAX) NULL,
    LastDeliveredAtUtc DATETIME2(7) NULL,
    IsActive BIT NOT NULL CONSTRAINT DF_ReportSubscription_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReportSubscription_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReportSubscription_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Reporting_ReportSubscription PRIMARY KEY (ReportSubscriptionId),
    CONSTRAINT FK_ReportSubscription_ReportDefinition FOREIGN KEY (ReportDefinitionId) REFERENCES Reporting.ReportDefinition(ReportDefinitionId),
    CONSTRAINT FK_ReportSubscription_ApplicationUser FOREIGN KEY (ApplicationUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT FK_ReportSubscription_DistributionList FOREIGN KEY (DistributionListId) REFERENCES Reporting.DistributionList(DistributionListId),
    CONSTRAINT FK_ReportSubscription_SubscriptionStatus FOREIGN KEY (SubscriptionStatusId) REFERENCES Reporting.SubscriptionStatus(SubscriptionStatusId),
    CONSTRAINT FK_ReportSubscription_DeliveryChannel FOREIGN KEY (DeliveryChannelId) REFERENCES Reporting.DeliveryChannel(DeliveryChannelId),
    CONSTRAINT FK_ReportSubscription_ReportFormat FOREIGN KEY (ReportFormatId) REFERENCES Reporting.ReportFormat(ReportFormatId),
    CONSTRAINT FK_ReportSubscription_ScheduleFrequency FOREIGN KEY (ScheduleFrequencyId) REFERENCES Reporting.ScheduleFrequency(ScheduleFrequencyId),
    CONSTRAINT FK_ReportSubscription_TimeZone FOREIGN KEY (TimeZoneId) REFERENCES CorePlatform.TimeZone(TimeZoneId),
    CONSTRAINT CK_ReportSubscription_Principal CHECK (ApplicationUserId IS NOT NULL OR DistributionListId IS NOT NULL),
    CONSTRAINT CK_ReportSubscription_ParameterJson CHECK (ParameterJson IS NULL OR ISJSON(ParameterJson) = 1)
);
GO

CREATE TABLE Reporting.ReportAccessGrant (
    ReportAccessGrantId INT IDENTITY(1,1) NOT NULL,
    ReportDefinitionId INT NULL,
    DashboardId INT NULL,
    DatasetDefinitionId INT NULL,
    AccessLevelId INT NOT NULL,
    ApplicationUserId INT NULL,
    ApplicationRoleId INT NULL,
    GrantedByUserId INT NOT NULL,
    ValidFromUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReportAccessGrant_ValidFromUtc DEFAULT SYSUTCDATETIME(),
    ValidToUtc DATETIME2(7) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ReportAccessGrant_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Reporting_ReportAccessGrant PRIMARY KEY (ReportAccessGrantId),
    CONSTRAINT FK_ReportAccessGrant_ReportDefinition FOREIGN KEY (ReportDefinitionId) REFERENCES Reporting.ReportDefinition(ReportDefinitionId),
    CONSTRAINT FK_ReportAccessGrant_Dashboard FOREIGN KEY (DashboardId) REFERENCES Reporting.Dashboard(DashboardId),
    CONSTRAINT FK_ReportAccessGrant_DatasetDefinition FOREIGN KEY (DatasetDefinitionId) REFERENCES Reporting.DatasetDefinition(DatasetDefinitionId),
    CONSTRAINT FK_ReportAccessGrant_AccessLevel FOREIGN KEY (AccessLevelId) REFERENCES Reporting.AccessLevel(AccessLevelId),
    CONSTRAINT FK_ReportAccessGrant_ApplicationUser FOREIGN KEY (ApplicationUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT FK_ReportAccessGrant_ApplicationRole FOREIGN KEY (ApplicationRoleId) REFERENCES Security.ApplicationRole(ApplicationRoleId),
    CONSTRAINT FK_ReportAccessGrant_GrantedByUser FOREIGN KEY (GrantedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_ReportAccessGrant_Target CHECK (
        (CASE WHEN ReportDefinitionId IS NULL THEN 0 ELSE 1 END) +
        (CASE WHEN DashboardId IS NULL THEN 0 ELSE 1 END) +
        (CASE WHEN DatasetDefinitionId IS NULL THEN 0 ELSE 1 END) = 1
    ),
    CONSTRAINT CK_ReportAccessGrant_Principal CHECK (
        (CASE WHEN ApplicationUserId IS NULL THEN 0 ELSE 1 END) +
        (CASE WHEN ApplicationRoleId IS NULL THEN 0 ELSE 1 END) = 1
    ),
    CONSTRAINT CK_ReportAccessGrant_Window CHECK (ValidToUtc IS NULL OR ValidToUtc >= ValidFromUtc)
);
GO

```

C.16.8 Foreign-key mapping

FROM TABLE	FK COLUMN	TO TABLE	RELATIONSHIP
Reporting.DatasetDefinition	DatasetTypeId	Reporting.DatasetType	Classifies dataset source kind.
Reporting.DatasetDefinition	OwnerUserId	Security.ApplicationUser	Owner of the reusable dataset.
Reporting.DatasetColumn	DatasetDefinitionId	Reporting.DatasetDefinition	Columns belong to a dataset.
Reporting.DatasetColumn	EngineeringUnitId	CorePlatform.EngineeringUnit	Optional unit hint for measured values.
Reporting.SavedQuery	DatasetDefinitionId	Reporting.DatasetDefinition	Saved query targets one dataset.
Reporting.SavedQuery	SavedByUserId	Security.ApplicationUser	Who saved the query.
Reporting.ReportDefinition	ReportTypeId	Reporting.ReportType	Business class of the report.
Reporting.ReportDefinition	DatasetDefinitionId	Reporting.DatasetDefinition	Default dataset used by the report.

FROM TABLE	FK COLUMN	TO TABLE	RELATIONSHIP
Reporting.ReportDefinition	DefaultFormatId	Reporting.ReportFormat	Default export format.
Reporting.ReportDefinition	OwnerUserId	Security.ApplicationUser	Report owner.
Reporting.ReportSchedule	ReportDefinitionId	Reporting.ReportDefinition	Schedule belongs to one report.
Reporting.ReportSchedule	ScheduleFrequencyId	Reporting.ScheduleFrequency	Schedule cadence.
Reporting.ReportSchedule	TimeZoneId	CorePlatform.TimeZone	Local wall-clock interpretation.
Reporting.ReportRun	ReportDefinitionId	Reporting.ReportDefinition	Run executes one report definition.
Reporting.ReportRun	ReportScheduleId	Reporting.ReportSchedule	Optional schedule that created the run.
Reporting.ReportRun	ReportRunStatusId	Reporting.ReportRunStatus	Execution state.
Reporting.ReportRun	RequestedByUserId	Security.ApplicationUser	Requesting user.
Reporting.ReportRun	SiteId / PlantId	Organization.Site / Organization.Plant	Optional organizational scope.
Reporting.ReportRun	UnitId	ReactorFleet.Unit	Optional physical unit scope.
Reporting.ReportExport	ReportRunId	Reporting.ReportRun	Export is generated from one run.
Reporting.ReportExport	ReportFormatId	Reporting.ReportFormat	Export output format.
Reporting.ReportExport	ExportStatusId	Reporting.ExportStatus	Export lifecycle state.
Reporting.Dashboard	OwnerUserId	Security.ApplicationUser	Dashboard owner.
Reporting.Dashboard	SiteId / PlantId / UnitId	Organization / ReactorFleet	Optional display scope.
Reporting.DashboardWidget	DashboardId	Reporting.Dashboard	Widget belongs to dashboard.
Reporting.DashboardWidget	WidgetTypeId	Reporting.WidgetType	Presentation type.
Reporting.KpiDefinition	KpiCategoryId	Reporting.KpiCategory	KPI domain category.
Reporting.KpiDefinition	DatasetDefinitionId	Reporting.DatasetDefinition	Dataset behind the calculation.
Reporting.KpiSnapshot	KpiDefinitionId	Reporting.KpiDefinition	Snapshot belongs to KPI.
Reporting.KpiSnapshot	KpiStatusId	Reporting.KpiStatus	Traffic-light result.
Reporting.ReportSubscription	ReportDefinitionId	Reporting.ReportDefinition	Subscribed report.
Reporting.ReportSubscription	ApplicationUserId	Security.ApplicationUser	Optional direct subscriber.
Reporting.ReportSubscription	DistributionListId	Reporting.DistributionList	Optional group subscriber.
Reporting.ReportAccessGrant	ApplicationUserId / ApplicationRoleId	Security.ApplicationUser / Security.ApplicationRole	User or role receiving access.

C.16.9 Recommended indexes

```

CREATE INDEX IX_DatasetDefinition_DatasetTypeId ON Reporting.DatasetDefinition(DatasetTypeId);
CREATE INDEX IX_DatasetColumn_DatasetDefinitionId ON Reporting.DatasetColumn(DatasetDefinitionId);
CREATE INDEX IX_ReportDefinition_ReportTypeId ON Reporting.ReportDefinition(ReportTypeId);
CREATE INDEX IX_ReportDefinition_DatasetDefinitionId ON Reporting.ReportDefinition(DatasetDefinitionId);
CREATE INDEX IX_ReportSchedule_ReportDefinitionId_NextRunAtUtc ON Reporting.ReportSchedule(ReportDefinitionId, NextRunAtUtc);
CREATE INDEX IX_ReportRun_ReportDefinitionId_RequestedAtUtc ON Reporting.ReportRun(ReportDefinitionId, RequestedAtUtc DESC);
CREATE INDEX IX_ReportRun_Status_RequestedAtUtc ON Reporting.ReportRun(ReportRunStatusId, RequestedAtUtc DESC);
CREATE INDEX IX_ReportExport_ReportRunId ON Reporting.ReportExport(ReportRunId);
CREATE INDEX IX_ReportExport_Status_ExpiresAtUtc ON Reporting.ReportExport(ExportStatusId, ExpiresAtUtc);
CREATE INDEX IX_Dashboard_OwnerUserId ON Reporting.Dashboard(OwnerUserId);
CREATE INDEX IX_DashboardWidget_DashboardId ON Reporting.DashboardWidget(DashboardId, PositionY, PositionX);
CREATE INDEX IX_KpiSnapshot_Kpi_Time ON Reporting.KpiSnapshot(KpiDefinitionId, SnapshotAtUtc DESC);
CREATE INDEX IX_KpiSnapshot_Unit_Time ON Reporting.KpiSnapshot(UnitId, SnapshotAtUtc DESC) WHERE UnitId IS NOT NULL;
CREATE INDEX IX_ReportSubscription_ReportDefinitionId ON Reporting.ReportSubscription(ReportDefinitionId);
CREATE INDEX IX_ReportAccessGrant_User ON Reporting.ReportAccessGrant(ApplicationUserId, AccessLevelId) WHERE ApplicationUserId IS NOT NULL;
CREATE INDEX IX_ReportAccessGrant_Role ON Reporting.ReportAccessGrant(ApplicationRoleId, AccessLevelId) WHERE ApplicationRoleId IS NOT NULL;
GO

```

C.16.10 Seed examples

```
INSERT INTO Reporting.ReportType (Code, Name, Description, DisplayOrder) VALUES
('OPERATIONAL','Operational','Plant and unit operating reports.',10),
('COMPLIANCE','Compliance','Audit, finding and regulatory reports.',20),
('MAINTENANCE','Maintenance','Work-order and equipment-health reports.',30),
('RADIATION','Radiation','Dose, survey and monitor reports.',40),
('EXECUTIVE','Executive','Summary dashboards for leadership.',50),
('TRAINING','Training','Training, drill and exercise reports.',60);

INSERT INTO Reporting.ReportFormat (Code, Name, Description, DisplayOrder) VALUES
('PDF','PDF','Paginated document export.',10),
('XLSX','Excel workbook','Spreadsheet export.',20),
('CSV','CSV','Delimited data export.',30),
('JSON','JSON','Structured machine-readable export.',40),
('HTML','HTML','Web-rendered output.',50);

INSERT INTO Reporting.ReportRunStatus (Code, Name, DisplayOrder) VALUES
('QUEUED','Queued',10),('RUNNING','Running',20),('COMPLETED','Completed',30),('FAILED','Failed',40),('CANCELLED','Cancelled',50);

INSERT INTO Reporting.ExportStatus (Code, Name, DisplayOrder) VALUES
('REQUESTED','Requested',10),('GENERATING','Generating',20),('READY','Ready',30),('FAILED','Failed',40),('EXPIRED','Expired',50);

INSERT INTO Reporting.ScheduleFrequency (Code, Name, DisplayOrder) VALUES
('ON_DEMAND','On demand',0),('HOURLY','Hourly',10),('DAILY','Daily',20),('WEEKLY','Weekly',30),('MONTHLY','Monthly',40),('QUARTERLY','Quarterly',50),('ANNUAL','Annual',60);

INSERT INTO Reporting.WidgetType (Code, Name, DisplayOrder) VALUES
('KPI_CARD','KPI card',10),('TABLE','Table',20),('LINE_CHART','Line chart',30),('BAR_CHART','Bar chart',40),('GAUGE','Gauge',50),('TIMELINE','Timeline',60);

INSERT INTO Reporting.KpiStatus (Code, Name, DisplayOrder) VALUES
('GREEN','Green',10),('AMBER','Amber',20),('RED','Red',30),('UNKNOWN','Unknown',40);
GO
```

C.16.11 Verification queries

```
-- 1. List report definitions and their default datasets.
SELECT r.Code, r.Name, rt.Code AS ReportType, d.Code AS DatasetCode, f.Code AS DefaultFormat
FROM Reporting.ReportDefinition r
JOIN Reporting.ReportType rt ON rt.ReportTypeId = r.ReportTypeId
LEFT JOIN Reporting.DatasetDefinition d ON d.DatasetDefinitionId = r.DatasetDefinitionId
JOIN Reporting.ReportFormat f ON f.ReportFormatId = r.DefaultFormatId
WHERE r.IsDeleted = 0;

-- 2. Find queued or running report work.
SELECT rr.ReportRunId, rd.Code, rs.Code AS RunStatus, rr.RequestedAtUtc
FROM Reporting.ReportRun rr
JOIN Reporting.ReportDefinition rd ON rd.ReportDefinitionId = rr.ReportDefinitionId
JOIN Reporting.ReportRunStatus rs ON rs.ReportRunStatusId = rr.ReportRunStatusId
WHERE rs.Code IN ('QUEUED','RUNNING')
ORDER BY rr.RequestedAtUtc;

-- 3. Read the latest KPI value for each unit-scoped KPI.
SELECT kd.Code, u.Code AS UnitCode, ks.SnapshotAtUtc, ks.NumericValue, st.Code AS StatusCode
FROM Reporting.KpiSnapshot ks
JOIN Reporting.KpiDefinition kd ON kd.KpiDefinitionId = ks.KpiDefinitionId
JOIN Reporting.KpiStatus st ON st.KpiStatusId = ks.KpiStatusId
LEFT JOIN ReactorFleet.Unit u ON u.UnitId = ks.UnitId
WHERE ks.SnapshotAtUtc = (
    SELECT MAX(ks2.SnapshotAtUtc)
    FROM Reporting.KpiSnapshot ks2
    WHERE ks2.KpiDefinitionId = ks.KpiDefinitionId
    AND ISNULL(ks2.UnitId, -1) = ISNULL(ks.UnitId, -1)
);

-- 4. Trace access granted to a specific report.
SELECT rd.Code AS ReportCode, al.Code AS AccessLevel,
       u.UserName AS GrantedToUser, r.Name AS GrantedToRole
FROM Reporting.ReportAccessGrant g
JOIN Reporting.ReportDefinition rd ON rd.ReportDefinitionId = g.ReportDefinitionId
JOIN Reporting.AccessLevel al ON al.AccessLevelId = g.AccessLevelId
LEFT JOIN Security.ApplicationUser u ON u.ApplicationUserId = g.ApplicationUserId
LEFT JOIN Security.ApplicationRole r ON r.ApplicationRoleId = g.ApplicationRoleId
WHERE rd.Code = 'UNIT_DAILY_SUMMARY';
GO
```

Honest boundary. Reporting is deliberately a read-side sector. It may cache generated outputs and KPI snapshots, but it does not become the authoritative source for alarms, work orders, dose records, audit findings or incident investigations. A report can preserve what it rendered; it must not rewrite what happened.

C.17.1 Purpose

The **Audit** schema is the final sector in the NEXUS-1 database backbone. Its job is to make every important state transition, read, command, export, signature and retention decision replayable. Earlier schemas hold operational facts. Audit records how those facts changed and how their derived artefacts can be trusted after the screen has moved on.

Design rule. Audit is append-first. It references stable actors and controlled lookup values with real foreign keys, but it does not create a foreign key to every possible target table. Instead, most targets are recorded with `EntitySchema`, `EntityType` and `EntityKeyText`. That keeps the audit trail universal without making every operational table depend back on Audit.

C.17.2 Full table list

AREA	TABLE	PRIMARY KEY	IMPORTANT FOREIGN KEYS	PURPOSE
Audit stream	<code>Audit.AuditBatch</code>	<code>AuditBatchId</code>	<code>CorrelationId</code> , <code>StartedByUserId</code> , <code>ChangeSourceTypeId</code>	Groups a set of entries produced by one request, job, migration or integration run.
Audit stream	<code>Audit.AuditEntry</code>	<code>AuditEntryId</code>	<code>AuditBatchId</code> , <code>ActionTypeId</code> , <code>EntityTypeId</code> , <code>SeverityId</code> , <code>PerformedByUserId</code>	Append-only record of a single audited action.
Change detail	<code>Audit.EntityChange</code>	<code>EntityChangeId</code>	<code>AuditEntryId</code>	Stores the target schema, table, key and high-level before/after hashes.
Change detail	<code>Audit.PropertyChange</code>	<code>PropertyChangeId</code>	<code>EntityChangeId</code>	Stores column-level before and after values where policy permits.
Command trail	<code>Audit.CommandLog</code>	<code>CommandLogId</code>	<code>CommandStatusId</code> , <code>SourceTypeId</code> , <code>RequestedByUserId</code>	Captures incoming write commands before and after execution.
Lineage	<code>Audit.DataLineage</code>	<code>DataLineageId</code>	<code>CreatedByUserId</code>	Names a lineage record for an import, transformation, report snapshot or analytical derivation.
Lineage	<code>Audit.LineageLink</code>	<code>LineageLinkId</code>	<code>DataLineageId</code>	Connects source and target entities in a traceable transformation.
Integration	<code>Audit.IntegrationMessage</code>	<code>IntegrationMessageId</code>	<code>IntegrationMessageStatusId</code> , <code>SourceTypeId</code>	

AREA	TABLE	PRIMARY KEY	IMPORTANT FOREIGN KEYS	PURPOSE
				Stores envelope metadata for inbound and outbound integration messages.
Integration	Audit.IntegrationMessageAttempt	IntegrationMessageAttemptId	IntegrationMessageId, IntegrationMessageStatusId	One delivery, replay or processing attempt for an integration message.
Access	Audit.AccessAudit	AccessAuditId	ApplicationUserId, AccessResultId, SourceTypeId	Records authentication, authorization and sensitive read decisions.
Export	Audit.ExportAudit	ExportAuditId	ReportExportId, ExportedByUserId, ExportPurposeId	Records data exported from reporting or operational screens.
Signature	Audit.SignatureRecord	SignatureRecordId	SignatureStatusId, SignedByUserId	Electronic signature or attestation over a target entity.
Retention	Audit.RetentionPolicy	RetentionPolicyId	RetentionActionTypeId, RetentionRuleStatusId	Defines retention and disposition rules for audited data classes.
Retention	Audit.RetentionExecution	RetentionExecutionId	RetentionPolicyId, RetentionActionTypeId, ExecutedByUserId	One run of a retention rule, including counts and outcome.
Registry	Audit.TemporalTableRegistry	TemporalTableRegistryId	-	Catalogue of temporal/history-enabled tables in the database.
Evidence	Audit.AuditEvidenceLink	AuditEvidenceLinkId	AuditEntryId, ComplianceEvidenceId	Binds audit entries to evidence artefacts, documents or external URIs.

C.17.3 Lookup tables

Audit lookup tables are deliberately typed. An action type, a command status and a signature status are different vocabularies and therefore different tables.

LOOKUP TABLE	SEED CODES	PURPOSE
Audit.AuditActionType	CREATE, UPDATE, DELETE, SOFT_DELETE, RESTORE, LOGIN, LOGOUT, READ, EXPORT, SIGN, APPROVE, REJECT, SYSTEM	Normalizes the kind of action recorded by an audit entry.
Audit.AuditEntityType	CONFIGURATION, SECURITY, ORGANIZATION, UNIT, EQUIPMENT, SIGNAL, ALARM, EVENT, WORK_ORDER, RCA, REPORT, DOCUMENT, UNKNOWN	Human-readable grouping of audited target entities.

LOOKUP TABLE	SEED CODES	PURPOSE
Audit.AuditSeverity	TRACE , INFO , NOTICE , WARNING , HIGH , CRITICAL	Severity level for audit review and retention prioritization.
Audit.ChangeSourceType	WEB_UI , API , BACKGROUND_JOB , IMPORT , MIGRATION , SYSTEM , SCRIPT , INTEGRATION	Where the audited change or command originated.
Audit.CommandStatus	RECEIVED , VALIDATED , EXECUTED , FAILED , REJECTED , ROLLED_BACK	Lifecycle state of a command or write request.
Audit.IntegrationMessageStatus	QUEUED , SENT , ACKNOWLEDGED , FAILED , DEAD_LETTER , REPLAYED	State of an inbound or outbound integration message.
Audit.RetentionActionType	ARCHIVE , PURGE , LEGAL_HOLD , ANONYMIZE , VERIFY_ONLY	Action taken by a retention execution.
Audit.RetentionRuleStatus	DRAFT , ACTIVE , SUSPENDED , RETIRED	Lifecycle state of an audit retention policy.
Audit.SignatureStatus	PENDING , SIGNED , COUNTERSIGNED , REVOKED , EXPIRED , INVALID	Lifecycle of an electronic signature record.
Audit.AccessResult	GRANTED , DENIED , CHALLENGED , EXPIRED , LOCKED_OUT	Outcome of an access decision or sensitive read.
Audit.ExportPurpose	REGULATORY , MAINTENANCE , INCIDENT_REVIEW , EXECUTIVE , TRAINING , SUPPORT , OTHER	Declared reason for exporting data outside the application screen.

C.17.4 ER diagram

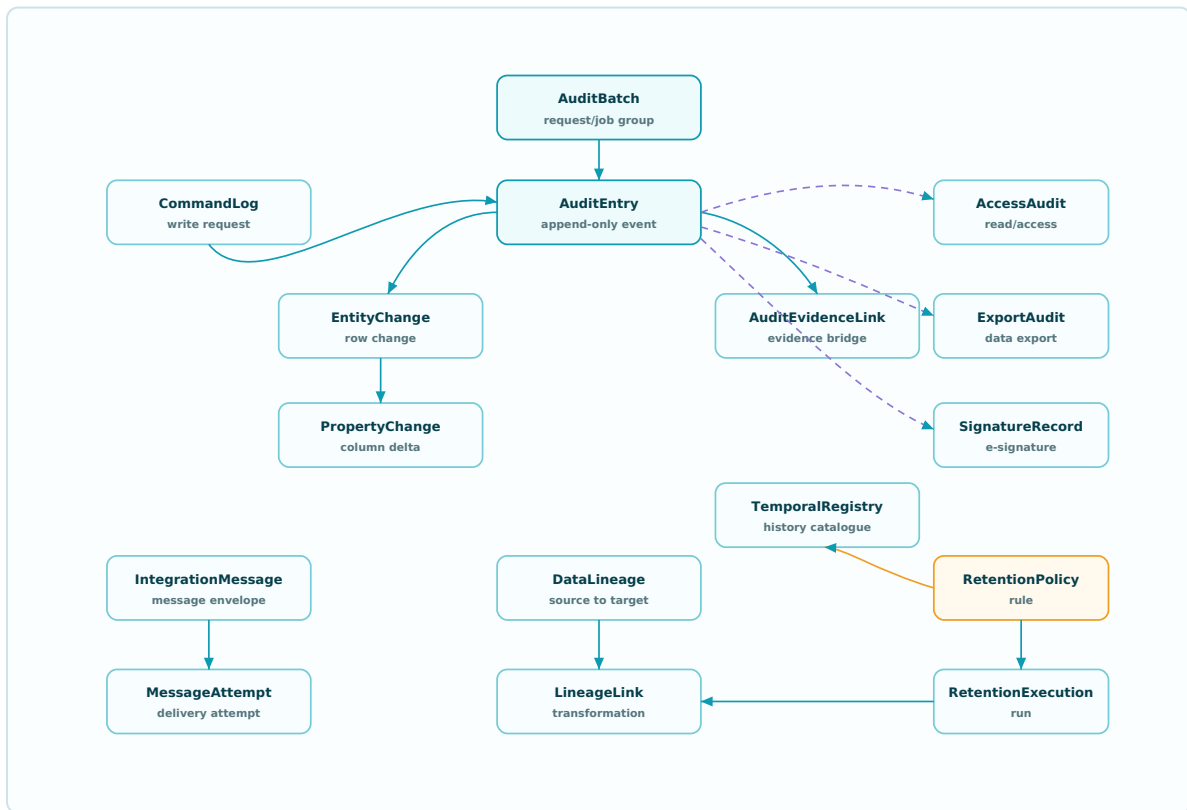


Figure C.17.1 - Audit sector ER diagram. AuditBatch groups entries; AuditEntry anchors changes, evidence, access, exports and signatures; lineage and retention tables make the trail replayable beyond one transaction.

C.17.5 Foreign-key passport map

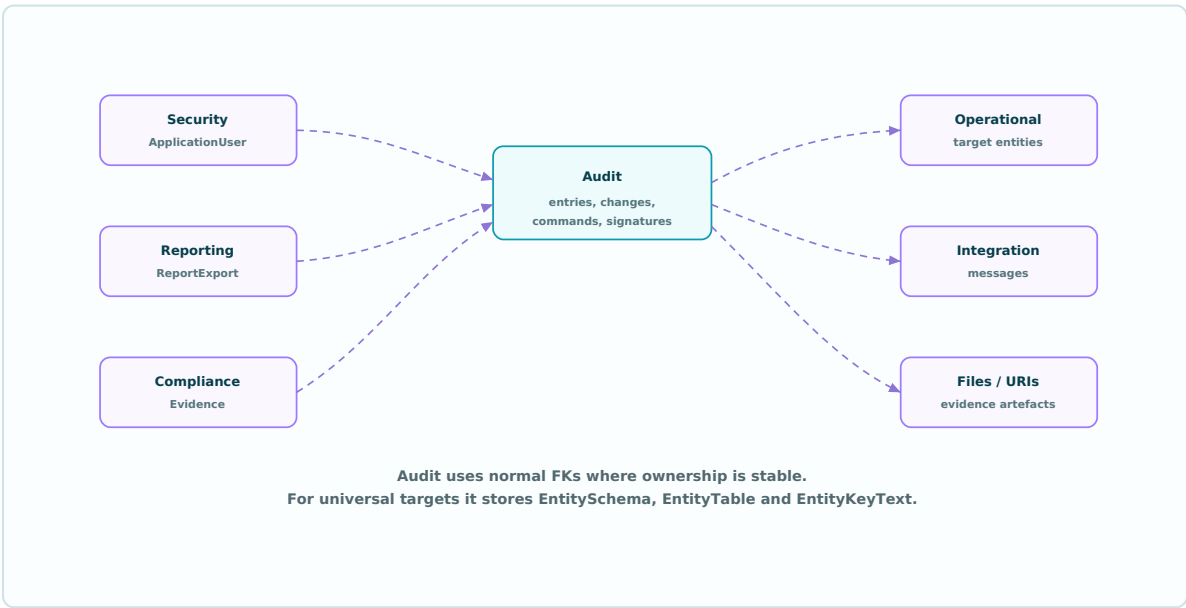


Figure C.17.2 - Audit passport map. Stable dependencies use foreign keys; audited targets across all sectors are captured by schema, table and key so Audit can observe every sector without owning them.

EXTERNAL SECTOR	HOW AUDIT USES IT
Security	ApplicationUser identifies the actor behind changes, commands, signatures, exports and access decisions.
Reporting	ReportExport is referenced when exported report output must be audited as a governed artefact.
Compliance	Evidence can be linked back to audit entries so findings and attestations have a replayable trail.
All operational sectors	Most audited targets are referenced by EntitySchema, EntityTable and EntityKeyText rather than a foreign key to every possible table.
CorePlatform	Retention, versioning and units remain owned by their own sectors; Audit records the trail, not the business meaning.

C.17.6 SQL DDL - lookup tables

```
CREATE TABLE Audit.AuditActionType (
  AuditActionTypeId INT IDENTITY(1,1) NOT NULL,
  Code NVARCHAR(50) NOT NULL,
  Name NVARCHAR(150) NOT NULL,
  Description NVARCHAR(500) NULL,
  DisplayOrder INT NOT NULL,
  CONSTRAINT DF_Audit_AuditActionType_DisplayOrder DEFAULT 0,
  IsActive BIT NOT NULL,
  CONSTRAINT DF_Audit_AuditActionType_IsActive DEFAULT 1,
  CreatedAtUtc DATETIME2(7) NOT NULL,
  CONSTRAINT DF_Audit_AuditActionType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
  ModifiedAtUtc DATETIME2(7) NOT NULL,
  CONSTRAINT DF_Audit_AuditActionType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
  RowVersion ROWVERSION NOT NULL,
  CONSTRAINT PK_Audit_AuditActionType
    PRIMARY KEY (AuditActionTypeId),
  CONSTRAINT UQ_Audit_AuditActionType_Code
    UNIQUE (Code)
);
GO

CREATE TABLE Audit.AuditEntityType (
  AuditEntityTypeId INT IDENTITY(1,1) NOT NULL,
```

```

Code NVARCHAR(50) NOT NULL,
Name NVARCHAR(150) NOT NULL,
Description NVARCHAR(500) NULL,
DisplayOrder INT NOT NULL
CONSTRAINT DF_Audit_AuditEntityType_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_Audit_AuditEntityType_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Audit_AuditEntityType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Audit_AuditEntityType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_Audit_AuditEntityType
PRIMARY KEY (AuditEntityTypeId),
CONSTRAINT UQ_Audit_AuditEntityType_Code
UNIQUE (Code)
);
GO

CREATE TABLE Audit.AuditSeverity (
    AuditSeverityId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
CONSTRAINT DF_Audit_AuditSeverity_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_Audit_AuditSeverity_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Audit_AuditSeverity_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Audit_AuditSeverity_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_Audit_AuditSeverity
PRIMARY KEY (AuditSeverityId),
CONSTRAINT UQ_Audit_AuditSeverity_Code
UNIQUE (Code)
);
GO

CREATE TABLE Audit.ChangeSourceType (
    ChangeSourceTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
CONSTRAINT DF_Audit_ChangeSourceType_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_Audit_ChangeSourceType_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Audit_ChangeSourceType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Audit_ChangeSourceType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_Audit_ChangeSourceTypes
PRIMARY KEY (ChangeSourceTypeId),
CONSTRAINT UQ_Audit_ChangeSourceType_Code
UNIQUE (Code)
);
GO

CREATE TABLE Audit.CommandStatus (
    CommandStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
CONSTRAINT DF_Audit_CommandStatus_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_Audit_CommandStatus_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Audit_CommandStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Audit_CommandStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_Audit_CommandStatus
PRIMARY KEY (CommandStatusId),
CONSTRAINT UQ_Audit_CommandStatus_Code
UNIQUE (Code)
);
GO

CREATE TABLE Audit.IntegrationMessageStatus (
    IntegrationMessageStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
CONSTRAINT DF_Audit_IntegrationMessageStatus_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_Audit_IntegrationMessageStatus_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Audit_IntegrationMessageStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Audit_IntegrationMessageStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_Audit_IntegrationMessageStatus
PRIMARY KEY (IntegrationMessageStatusId),
CONSTRAINT UQ_Audit_IntegrationMessageStatus_Code
UNIQUE (Code)
);
GO

CREATE TABLE Audit.RetentionActionType (
    RetentionActionTypeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
CONSTRAINT DF_Audit_RetentionActionType_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_Audit_RetentionActionType_IsActive DEFAULT 1,
CreatedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Audit_RetentionActionType_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
ModifiedAtUtc DATETIME2(7) NOT NULL
CONSTRAINT DF_Audit_RetentionActionType_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
RowVersion ROWVERSION NOT NULL,

CONSTRAINT PK_Audit_RetentionActionType
PRIMARY KEY (RetentionActionTypeId),
CONSTRAINT UQ_Audit_RetentionActionType_Code
UNIQUE (Code)
);
GO

CREATE TABLE Audit.RetentionRuleStatus (
    RetentionRuleStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
CONSTRAINT DF_Audit_RetentionRuleStatus_DisplayOrder DEFAULT 0,
IsActive BIT NOT NULL
CONSTRAINT DF_Audit_RetentionRuleStatus_IsActive DEFAULT 1,

```

```

        CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Audit_RetentionRuleStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
        ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Audit_RetentionRuleStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
        RowVersion ROWVERSION NOT NULL,

        CONSTRAINT PK_Audit_RetentionRuleStatus
            PRIMARY KEY (RetentionRuleStatusId),
        CONSTRAINT UQ_Audit_RetentionRuleStatus_Code
            UNIQUE (Code)
    );
GO

CREATE TABLE Audit.SignatureStatus (
    SignatureStatusId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_Audit_SignatureStatus_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_Audit_SignatureStatus_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Audit_SignatureStatus_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Audit_SignatureStatus_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Audit_SignatureStatus
        PRIMARY KEY (SignatureStatusId),
    CONSTRAINT UQ_Audit_SignatureStatus_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Audit.AccessResult (
    AccessResultId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_Audit_AccessResult_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_Audit_AccessResult_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Audit_AccessResult_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Audit_AccessResult_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Audit_AccessResult
        PRIMARY KEY (AccessResultId),
    CONSTRAINT UQ_Audit_AccessResult_Code
        UNIQUE (Code)
);
GO

CREATE TABLE Audit.ExportPurpose (
    ExportPurposeId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(50) NOT NULL,
    Name NVARCHAR(150) NOT NULL,
    Description NVARCHAR(500) NULL,
    DisplayOrder INT NOT NULL
        CONSTRAINT DF_Audit_ExportPurpose_DisplayOrder DEFAULT 0,
    IsActive BIT NOT NULL
        CONSTRAINT DF_Audit_ExportPurpose_IsActive DEFAULT 1,
    CreatedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Audit_ExportPurpose_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_Audit_ExportPurpose_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Audit_ExportPurpose
        PRIMARY KEY (ExportPurposeId),
    CONSTRAINT UQ_Audit_ExportPurpose_Code
        UNIQUE (Code)
);
GO

```

C.17.7 SQL DDL - main tables

```

CREATE TABLE Audit.AuditBatch (
    AuditBatchId BIGINT IDENTITY(1,1) NOT NULL,
    CorrelationId UNIQUEIDENTIFIER NOT NULL,
    TraceId NVARCHAR(100) NULL,
    OperationName NVARCHAR(200) NOT NULL,
    ChangeSourceTypeId INT NOT NULL,
    StartedByUserId INT NULL,
    StartedAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_AuditBatch_StartedAtUtc DEFAULT SYSUTCDATETIME(),
    CompletedAtUtc DATETIME2(7) NULL,
    ClientIpAddress NVARCHAR(64) NULL,
    UserAgent NVARCHAR(500) NULL,
    Notes NVARCHAR(1000) NULL,

    CONSTRAINT PK_Audit_AuditBatch PRIMARY KEY (AuditBatchId),
    CONSTRAINT UQ_Audit_AuditBatch_CorrelationId UNIQUE (CorrelationId),
    CONSTRAINT FK_AuditBatch_ChangeSourceType FOREIGN KEY (ChangeSourceTypeId) REFERENCES Audit.ChangeSourceType(ChangeSourceTypeId),
    CONSTRAINT FK_AuditBatch_StartedByUser FOREIGN KEY (StartedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE Audit.AuditEntry (
    AuditEntryId BIGINT IDENTITY(1,1) NOT NULL,
    AuditBatchId BIGINT NULL,
    OccurredAtUtc DATETIME2(7) NOT NULL
        CONSTRAINT DF_AuditEntry_OccurredAtUtc DEFAULT SYSUTCDATETIME(),
    ActionTypeId INT NOT NULL,
    EntityTypeId INT NOT NULL,
    SeverityId INT NOT NULL,
    ChangeSourceTypeId INT NOT NULL,
    PerformedByUserId INT NULL,
    EntitySchema NVARCHAR(128) NOT NULL,
    EntityTable NVARCHAR(128) NOT NULL,
    EntityKeyText NVARCHAR(380) NULL,
    Summary NVARCHAR(1000) NOT NULL,
    IsSensitive BIT NOT NULL
        CONSTRAINT DF_AuditEntry_IsSensitive DEFAULT 0,
    CorrelationId UNIQUEIDENTIFIER NULL,
    TraceId NVARCHAR(100) NULL,
    RequestPath NVARCHAR(500) NULL,
    ClientIpAddress NVARCHAR(64) NULL,
    HashAlgorithm NVARCHAR(40) NULL,
    PayloadHash VARBINARY(64) NULL,

    CONSTRAINT PK_Audit_AuditEntry PRIMARY KEY (AuditEntryId),
    CONSTRAINT FK_AuditEntry_AuditBatch FOREIGN KEY (AuditBatchId) REFERENCES Audit.AuditBatch(AuditBatchId),
    CONSTRAINT FK_AuditEntry_ActionType FOREIGN KEY (ActionTypeId) REFERENCES Audit.AuditActionType(AuditActionTypeId),
    CONSTRAINT FK_AuditEntry_EntityType FOREIGN KEY (EntityTypeId) REFERENCES Audit.AuditEntityType(AuditEntityTypeId),
    CONSTRAINT FK_AuditEntry_Severity FOREIGN KEY (SeverityId) REFERENCES Audit.AuditSeverity(AuditSeverityId),
    CONSTRAINT FK_AuditEntry_ChangeSourceType FOREIGN KEY (ChangeSourceTypeId) REFERENCES Audit.ChangeSourceType(ChangeSourceTypeId),
    CONSTRAINT FK_AuditEntry_PerformedByUser FOREIGN KEY (PerformedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId)
);
GO

```

```

);
GO

CREATE TABLE Audit.EntityChange (
    EntityChangeId BIGINT IDENTITY(1,1) NOT NULL,
    AuditEntryId BIGINT NOT NULL,
    EntitySchema NVARCHAR(128) NOT NULL,
    EntityTable NVARCHAR(128) NOT NULL,
    EntityKeyText NVARCHAR(300) NOT NULL,
    Operation NVARCHAR(20) NOT NULL,
    BeforeHash VARBINARY(64) NULL,
    AfterHash VARBINARY(64) NULL,
    ChangedColumnCount INT NOT NULL CONSTRAINT DF_EntityChange_ChangedColumnCount DEFAULT 0,
    BeforeJson NVARCHAR(MAX) NULL,
    AfterJson NVARCHAR(MAX) NULL,

    CONSTRAINT PK_Audit_EntityChange PRIMARY KEY (EntityChangeId),
    CONSTRAINT FK_EntityChange_AuditEntry FOREIGN KEY (AuditEntryId) REFERENCES Audit.AuditEntry(AuditEntryId),
    CONSTRAINT CK_EntityChange_Operation CHECK (Operation IN ('CREATE','UPDATE','DELETE','SOFT DELETE','RESTORE')),
    CONSTRAINT CK_EntityChange_BeforeJson CHECK (BeforeJson IS NULL OR ISJSON(BeforeJson) = 1),
    CONSTRAINT CK_EntityChange_AfterJson CHECK (AfterJson IS NULL OR ISJSON(AfterJson) = 1)
);
GO

CREATE TABLE Audit.PropertyChange (
    PropertyChangeId BIGINT IDENTITY(1,1) NOT NULL,
    EntityChangeId BIGINT NOT NULL,
    PropertyName NVARCHAR(128) NOT NULL,
    DataType NVARCHAR(60) NULL,
    OldValue NVARCHAR(MAX) NULL,
    NewValue NVARCHAR(MAX) NULL,
    OldValueHash VARBINARY(64) NULL,
    NewValueHash VARBINARY(64) NULL,
    IsSensitive BIT NOT NULL CONSTRAINT DF_PropertyChange_IsSensitive DEFAULT 0,

    CONSTRAINT PK_Audit_PropertyChange PRIMARY KEY (PropertyChangeId),
    CONSTRAINT FK_PropertyChange_EntityChange FOREIGN KEY (EntityChangeId) REFERENCES Audit.EntityChange(EntityChangeId)
);
GO

CREATE TABLE Audit.CommandLog (
    CommandLogId BIGINT IDENTITY(1,1) NOT NULL,
    CommandName NVARCHAR(200) NOT NULL,
    CommandStatusId INT NOT NULL,
    SourceTypeId INT NOT NULL,
    RequestedByUserId INT NOT NULL,
    RequestedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_CommandLog_RequestedAtUtc DEFAULT SYSUTCDATETIME(),
    CompletedAtUtc DATETIME2(7) NULL,
    CorrelationId UNIQUEIDENTIFIER NULL,
    RequestJson NVARCHAR(MAX) NULL,
    ResultJson NVARCHAR(MAX) NULL,
    ErrorMessage NVARCHAR(2000) NULL,

    CONSTRAINT PK_Audit_CommandLog PRIMARY KEY (CommandLogId),
    CONSTRAINT FK_CommandLog_CommandStatus FOREIGN KEY (CommandStatusId) REFERENCES Audit.CommandStatus(CommandStatusId),
    CONSTRAINT FK_CommandLog_SourceType FOREIGN KEY (SourceTypeId) REFERENCES Audit.ChangeSourceType(ChangeSourceTypeId),
    CONSTRAINT FK_CommandLog_RequestedByUser FOREIGN KEY (RequestedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT CK_CommandLog_RequestJson CHECK (RequestJson IS NULL OR ISJSON(RequestJson) = 1),
    CONSTRAINT CK_CommandLog_ResultJson CHECK (ResultJson IS NULL OR ISJSON(ResultJson) = 1)
);
GO

CREATE TABLE Audit.DataLineage (
    DataLineageId BIGINT IDENTITY(1,1) NOT NULL,
    LineageCode NVARCHAR(100) NOT NULL,
    Name NVARCHAR(250) NOT NULL,
    Description NVARCHAR(1000) NULL,
    CreatedByUserId INT NOT NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_DataLineage_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    SourceSummary NVARCHAR(1000) NULL,
    TargetSummary NVARCHAR(1000) NULL,
    IsSealed BIT NOT NULL CONSTRAINT DF_DataLineage_IsSealed DEFAULT 0,
    SealedAtUtc DATETIME2(7) NULL,

    CONSTRAINT PK_Audit_DataLineage PRIMARY KEY (DataLineageId),
    CONSTRAINT UQ_Audit_DataLineage_LineageCode UNIQUE (LineageCode),
    CONSTRAINT FK_DataLineage_CreatedByUser FOREIGN KEY (CreatedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE Audit.LineageLink (
    LineageLinkId BIGINT IDENTITY(1,1) NOT NULL,
    DataLineageId BIGINT NOT NULL,
    SourceSchema NVARCHAR(128) NOT NULL,
    SourceTable NVARCHAR(128) NOT NULL,
    SourceKeyText NVARCHAR(300) NULL,
    TargetSchema NVARCHAR(128) NOT NULL,
    TargetTable NVARCHAR(128) NOT NULL,
    TargetKeyText NVARCHAR(300) NULL,
    TransformationName NVARCHAR(200) NULL,
    TransformationHash VARBINARY(64) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_LineageLink_CreatedAtUtc DEFAULT SYSUTCDATETIME(),

    CONSTRAINT PK_Audit_LineageLink PRIMARY KEY (LineageLinkId),
    CONSTRAINT FK_LineageLink_DataLineage FOREIGN KEY (DataLineageId) REFERENCES Audit.DataLineage(DataLineageId)
);
GO

CREATE TABLE Audit.IntegrationMessage (
    IntegrationMessageId BIGINT IDENTITY(1,1) NOT NULL,
    MessageDirection NVARCHAR(20) NOT NULL,
    MessageName NVARCHAR(200) NOT NULL,
    ExternalMessageId NVARCHAR(200) NULL,
    IntegrationMessageStatusId INT NOT NULL,
    SourceTypeId INT NOT NULL,
    CorrelationId UNIQUEIDENTIFIER NULL,
    ReceivedAtUtc DATETIME2(7) NULL,
    SentAtUtc DATETIME2(7) NULL,
    PayloadHash VARBINARY(64) NULL,
    PayloadJson NVARCHAR(MAX) NULL,
    ErrorMessage NVARCHAR(2000) NULL,

    CONSTRAINT PK_Audit_IntegrationMessage PRIMARY KEY (IntegrationMessageId),
    CONSTRAINT FK_IntegrationMessage_Status FOREIGN KEY (IntegrationMessageStatusId) REFERENCES Audit.IntegrationMessageStatus(IntegrationMessageStatusId),
    CONSTRAINT FK_IntegrationMessage_SourceType FOREIGN KEY (SourceTypeId) REFERENCES Audit.ChangeSourceType(ChangeSourceTypeId),
    CONSTRAINT CK_IntegrationMessage_Direction CHECK (MessageDirection IN ('INBOUND','OUTBOUND')),
    CONSTRAINT CK_IntegrationMessage_PayloadJson CHECK (PayloadJson IS NULL OR ISJSON(PayloadJson) = 1)
);
GO

CREATE TABLE Audit.IntegrationMessageAttempt (
    IntegrationMessageAttemptId BIGINT IDENTITY(1,1) NOT NULL,
    IntegrationMessageId BIGINT NOT NULL,
    AttemptNumber INT NOT NULL,
    IntegrationMessageStatusId INT NOT NULL,
    AttemptedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_IntegrationMessageAttempt_AttemptedAtUtc DEFAULT SYSUTCDATETIME(),
    DurationMs INT NULL,
    ResponseCode NVARCHAR(50) NULL,
    ResponseMessage NVARCHAR(2000) NULL,

    CONSTRAINT PK_Audit_IntegrationMessageAttempt PRIMARY KEY (IntegrationMessageAttemptId),
    CONSTRAINT UQ_Audit_IntegrationMessageAttempt_MessageAttempt UNIQUE (IntegrationMessageId, AttemptNumber),
    CONSTRAINT FK_IntegrationMessageAttempt_Message FOREIGN KEY (IntegrationMessageId) REFERENCES Audit.IntegrationMessage(IntegrationMessageId),
    CONSTRAINT FK_IntegrationMessageAttempt_Status FOREIGN KEY (IntegrationMessageStatusId) REFERENCES Audit.IntegrationMessageStatus(IntegrationMessageStatusId)
);
GO

CREATE TABLE Audit.AccessAudit (
    AccessAuditId BIGINT IDENTITY(1,1) NOT NULL,
    ApplicationUserId INT NOT NULL,
    AccessResultId INT NOT NULL,
    SourceTypeId INT NOT NULL,
    OccurredAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AccessAudit_OccurredAtUtc DEFAULT SYSUTCDATETIME(),
    ResourceSchema NVARCHAR(128) NULL,
    ResourceName NVARCHAR(256) NULL,

```

```

PermissionCode NVARCHAR(100) NULL,
ClientIpAddress NVARCHAR(64) NULL,
UserAgent NVARCHAR(500) NULL,
Reason NVARCHAR(1000) NULL,

CONSTRAINT PK_Audit_AccessAudit PRIMARY KEY (AccessAuditId),
CONSTRAINT FK_AccessAudit_User FOREIGN KEY (ApplicationUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
CONSTRAINT FK_AccessAudit_Result FOREIGN KEY (AccessResultId) REFERENCES Audit.AccessResult(AccessResultId),
CONSTRAINT FK_AccessAudit_SourceType FOREIGN KEY (SourceTypeId) REFERENCES Audit.ChangeSourceType(ChangeSourceTypeId)
);
GO

CREATE TABLE Audit.ExportAudit (
    ExportAuditId BIGINT IDENTITY(1,1) NOT NULL,
    ReportExportId INT NULL,
    ExportedByUserId INT NULL,
    ExportPurposeId INT NOT NULL,
    ExportedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_ExportAudit_ExportedAtUtc DEFAULT SYSUTCDATETIME(),
    EntitySchema NVARCHAR(128) NULL,
    EntityName NVARCHAR(256) NULL,
    RowCount BIGINT NULL,
    FileName NVARCHAR(260) NULL,
    FileHash VARBINARY(64) NULL,
    Destination NVARCHAR(500) NULL,
    Justification NVARCHAR(1000) NULL,

    CONSTRAINT PK_Audit_ExportAudit PRIMARY KEY (ExportAuditId),
    CONSTRAINT FK_ExportAudit_ReportExport FOREIGN KEY (ReportExportId) REFERENCES Reporting.ReportExport(ReportExportId),
    CONSTRAINT FK_ExportAudit_ExportedByUser FOREIGN KEY (ExportedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId),
    CONSTRAINT FK_ExportAudit_ExportPurpose FOREIGN KEY (ExportPurposeId) REFERENCES Audit.ExportPurpose(ExportPurposeId)
);
GO

CREATE TABLE Audit.SignatureRecord (
    SignatureRecordId BIGINT IDENTITY(1,1) NOT NULL,
    SignatureStatusId INT NOT NULL,
    SignedByUserId INT NOT NULL,
    SignedAtUtc DATETIME2(7) NULL,
    TargetSchema NVARCHAR(128) NOT NULL,
    TargetTable NVARCHAR(128) NOT NULL,
    TargetKeyText NVARCHAR(300) NOT NULL,
    SignatureReason NVARCHAR(500) NOT NULL,
    SignatureHash VARBINARY(64) NULL,
    CertificateThumbprint NVARCHAR(200) NULL,
    RevokedAtUtc DATETIME2(7) NULL,
    RevocationReason NVARCHAR(1000) NULL,

    CONSTRAINT PK_Audit_SignatureRecord PRIMARY KEY (SignatureRecordId),
    CONSTRAINT FK_SignatureRecord_Status FOREIGN KEY (SignatureStatusId) REFERENCES Audit.SignatureStatus(SignatureStatusId),
    CONSTRAINT FK_SignatureRecord_SignedByUser FOREIGN KEY (SignedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE Audit.RetentionPolicy (
    RetentionPolicyId INT IDENTITY(1,1) NOT NULL,
    Code NVARCHAR(80) NOT NULL,
    Name NVARCHAR(250) NOT NULL,
    TargetSchema NVARCHAR(128) NOT NULL,
    TargetTable NVARCHAR(128) NOT NULL,
    RetentionDays INT NOT NULL,
    RetentionActionTypeId INT NOT NULL,
    RetentionRuleStatusId INT NOT NULL,
    LegalHoldExempt BIT NOT NULL CONSTRAINT DF_RetentionPolicy_LegalHoldExempt DEFAULT 0,
    Description NVARCHAR(1000) NULL,
    CreatedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RetentionPolicy_CreatedAtUtc DEFAULT SYSUTCDATETIME(),
    CreatedBy NVARCHAR(100) NOT NULL,
    ModifiedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RetentionPolicy_ModifiedAtUtc DEFAULT SYSUTCDATETIME(),
    ModifiedBy NVARCHAR(100) NOT NULL,
    RowVersion ROWVERSION NOT NULL,

    CONSTRAINT PK_Audit_RetentionPolicy PRIMARY KEY (RetentionPolicyId),
    CONSTRAINT UQ_Audit_RetentionPolicy_Code UNIQUE (Code),
    CONSTRAINT FK_RetentionPolicy_ActionType FOREIGN KEY (RetentionActionTypeId) REFERENCES Audit.RetentionActionType(RetentionActionTypeId),
    CONSTRAINT FK_RetentionPolicy_RuleStatus FOREIGN KEY (RetentionRuleStatusId) REFERENCES Audit.RetentionRuleStatus(RetentionRuleStatusId),
    CONSTRAINT CK_RetentionPolicy_RetentionDays CHECK (RetentionDays >= 0)
);
GO

CREATE TABLE Audit.RetentionExecution (
    RetentionExecutionId BIGINT IDENTITY(1,1) NOT NULL,
    RetentionPolicyId INT NOT NULL,
    RetentionActionTypeId INT NOT NULL,
    ExecutedByUserId INT NULL,
    StartedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_RetentionExecution_StartedAtUtc DEFAULT SYSUTCDATETIME(),
    CompletedAtUtc DATETIME2(7) NULL,
    CandidateRowCount BIGINT NOT NULL CONSTRAINT DF_RetentionExecution_CandidateRowCount DEFAULT 0,
    AffectedRowCount BIGINT NOT NULL CONSTRAINT DF_RetentionExecution_AffectedRowCount DEFAULT 0,
    WasDryRun BIT NOT NULL CONSTRAINT DF_RetentionExecution_WasDryRun DEFAULT 1,
    ResultSummary NVARCHAR(2000) NULL,

    CONSTRAINT PK_Audit_RetentionExecution PRIMARY KEY (RetentionExecutionId),
    CONSTRAINT FK_RetentionExecution_Policy FOREIGN KEY (RetentionPolicyId) REFERENCES Audit.RetentionPolicy(RetentionPolicyId),
    CONSTRAINT FK_RetentionExecution_ActionType FOREIGN KEY (RetentionActionTypeId) REFERENCES Audit.RetentionActionType(RetentionActionTypeId),
    CONSTRAINT FK_RetentionExecution_ExecutedByUser FOREIGN KEY (ExecutedByUserId) REFERENCES Security.ApplicationUser(ApplicationUserId)
);
GO

CREATE TABLE Audit.TemporalTableRegistry (
    TemporalTableRegistryId INT IDENTITY(1,1) NOT NULL,
    SchemaName NVARCHAR(128) NOT NULL,
    TableName NVARCHAR(128) NOT NULL,
    HistorySchemaName NVARCHAR(128) NULL,
    HistoryTableName NVARCHAR(128) NULL,
    IsSystemVersioned BIT NOT NULL CONSTRAINT DF_TemporalTableRegistry_IsSystemVersioned DEFAULT 0,
    RetentionPolicyId INT NULL,
    RegisteredAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_TemporalTableRegistry_RegisteredAtUtc DEFAULT SYSUTCDATETIME(),
    Notes NVARCHAR(1000) NULL,

    CONSTRAINT PK_Audit_TemporalTableRegistry PRIMARY KEY (TemporalTableRegistryId),
    CONSTRAINT UQ_Audit_TemporalTableRegistry_Table UNIQUE (SchemaName, TableName),
    CONSTRAINT FK_TemporalTableRegistry_RetentionPolicy FOREIGN KEY (RetentionPolicyId) REFERENCES Audit.RetentionPolicy(RetentionPolicyId)
);
GO

CREATE TABLE Audit.AuditEvidenceLink (
    AuditEvidenceLinkId BIGINT IDENTITY(1,1) NOT NULL,
    AuditEntryId BIGINT NOT NULL,
    ComplianceEvidenceId INT NULL,
    EvidenceUri NVARCHAR(1000) NULL,
    EvidenceHash VARBINARY(64) NULL,
    Description NVARCHAR(1000) NULL,
    LinkedAtUtc DATETIME2(7) NOT NULL CONSTRAINT DF_AuditEvidenceLink_LinkedAtUtc DEFAULT SYSUTCDATETIME(),

    CONSTRAINT PK_Audit_AuditEvidenceLink PRIMARY KEY (AuditEvidenceLinkId),
    CONSTRAINT FK_AuditEvidenceLink_AuditEntry FOREIGN KEY (AuditEntryId) REFERENCES Audit.AuditEntry(AuditEntryId),
    CONSTRAINT FK_AuditEvidenceLink_ComplianceEvidence FOREIGN KEY (ComplianceEvidenceId) REFERENCES Compliance.Evidence(EvidenceId),
    CONSTRAINT CK_AuditEvidenceLink_Target CHECK (ComplianceEvidenceId IS NOT NULL OR EvidenceUri IS NOT NULL)
);
GO

```

C.17.8 Indexes

The Audit sector is read mostly by time, actor, target entity and correlation. Indexes therefore favour replay and investigation over transactional writes. In a production implementation, partitioning by `OccurredAtUtc` and a retention-aware archive strategy would be added for high-volume environments.

```
CREATE INDEX IX_AuditEntry_OccurredAtUtc ON Audit.AuditEntry(OccurredAtUtc DESC);
CREATE INDEX IX_AuditEntry_Entity ON Audit.AuditEntry(EntitySchema, EntityTable, EntityKeyText, OccurredAtUtc DESC);
CREATE INDEX IX_AuditEntry_PerformedByUser ON Audit.AuditEntry(PerformedByUserId, OccurredAtUtc DESC);
CREATE INDEX IX_EntityChange_AuditEntry ON Audit.EntityChange(AuditEntryId);
CREATE INDEX IX_PropertyChange_EntityChange ON Audit.PropertyChange(EntityChangeId);
CREATE INDEX IX_CommandLog_Status_Time ON Audit.CommandLog(CommandStatusId, RequestedAtUtc DESC);
CREATE INDEX IX_DataLineage_Code ON Audit.DataLineage(LineageCode);
CREATE INDEX IX_LineageLink_Source ON Audit.LineageLink(SourceSchema, SourceTable, SourceKeyText);
CREATE INDEX IX_LineageLink_Target ON Audit.LineageLink(TargetSchema, TargetTable, TargetKeyText);
CREATE INDEX IX_IntegrationMessage_Status_ReceivedAtUtc ON Audit.IntegrationMessage(IntegrationMessageStatusId, ReceivedAtUtc DESC);
CREATE INDEX IX_IntegrationMessage_Status_SentAtUtc ON Audit.IntegrationMessage(IntegrationMessageStatusId, SentAtUtc DESC);
CREATE INDEX IX_AccessAudit_User_Time ON Audit.AccessAudit(ApplicationUserId, OccurredAtUtc DESC);
CREATE INDEX IX_ExportAudit_User_Time ON Audit.ExportAudit(ExportedByUserId, ExportedAtUtc DESC);
CREATE INDEX IX_SignatureRecord_Target ON Audit.SignatureRecord(TargetSchema, TargetTable, TargetKeyText);
CREATE INDEX IX_RetentionExecution_Policy_Time ON Audit.RetentionExecution(RetentionPolicyId, StartedAtUtc DESC);
```

C.17.9 Foreign-key mapping

FROM TABLE	FK COLUMN	TO TABLE	RELATIONSHIP
Audit.AuditBatch	ChangeSourceTypeId	Audit.ChangeSourceType	How the batch was initiated.
Audit.AuditBatch	StartedByUserId	Security.ApplicationUser	User who initiated the batch, if any.
Audit.AuditEntry	AuditBatchId	Audit.AuditBatch	Entry belongs to a request/job batch.
Audit.AuditEntry	ActionTypeId	Audit.AuditActionType	Normalized action kind.
Audit.AuditEntry	EntityTypeId	Audit.AuditEntityType	Normalized target entity group.
Audit.AuditEntry	SeverityId	Audit.AuditSeverity	Review severity.
Audit.AuditEntry	ChangeSourceTypeId	Audit.ChangeSourceType	Origin of the change.
Audit.AuditEntry	PerformedByUserId	Security.ApplicationUser	Actor who performed the action.
Audit.EntityChange	AuditEntryId	Audit.AuditEntry	Detailed entity change for one entry.
Audit.PropertyChange	EntityChangeId	Audit.EntityChange	Column-level change under one entity change.
Audit.CommandLog	CommandStatusId	Audit.CommandStatus	Execution state of the command.
Audit.CommandLog	SourceTypeId	Audit.ChangeSourceType	Where the command came from.
Audit.CommandLog	RequestedByUserId	Security.ApplicationUser	User or service principal that requested the command.
Audit.DataLineage	CreatedByUserId	Security.ApplicationUser	User who recorded or approved the lineage.
Audit.LineageLink	DataLineageId	Audit.DataLineage	Lineage link belongs to one lineage record.
Audit.IntegrationMessage	IntegrationMessageStatusId	Audit.IntegrationMessageStatus	Message lifecycle state.
Audit.IntegrationMessage	SourceTypeId	Audit.ChangeSourceType	Originating integration path.
Audit.IntegrationMessageAttempt	IntegrationMessageId	Audit.IntegrationMessage	Attempt belongs to one message envelope.
Audit.AccessAudit	ApplicationUserId	Security.ApplicationUser	User associated with the access decision.
Audit.AccessAudit	AccessResultId	Audit.AccessResult	Decision outcome.
Audit.ExportAudit	ReportExportId	Reporting.ReportExport	Optional link to generated export.
Audit.ExportAudit	ExportedByUserId	Security.ApplicationUser	Actor who exported the data.
Audit.ExportAudit	ExportPurposeId	Audit.ExportPurpose	Declared export purpose.

FROM TABLE	FK COLUMN	TO TABLE	RELATIONSHIP
Audit.SignatureRecord	SignatureStatusId	Audit.SignatureStatus	Signature lifecycle state.
Audit.SignatureRecord	SignedByUserId	Security.ApplicationUser	Signer or approver.
Audit.RetentionPolicy	RetentionActionTypeId	Audit.RetentionActionType	Action the policy performs.
Audit.RetentionPolicy	RetentionRuleStatusId	Audit.RetentionRuleStatus	Lifecycle state of the policy.
Audit.RetentionExecution	RetentionPolicyId	Audit.RetentionPolicy	Execution of one policy.
Audit.RetentionExecution	ExecutedByUserId	Security.ApplicationUser	Manual executor, if any.
Audit.AuditEvidenceLink	AuditEntryId	Audit.AuditEntry	Evidence supports one audit entry.
Audit.AuditEvidenceLink	ComplianceEvidenceId	Compliance.Evidence	Optional formal compliance evidence row.

C.17.10 Seed examples

```

INSERT INTO Audit.AuditActionType (Code, Name, DisplayOrder)
VALUES ('CREATE','Create',10), ('UPDATE','Update',20), ('DELETE','Delete',30),
('READ','Read',40), ('EXPORT','Export',50), ('SIGN','Sign',60), ('SYSTEM','System',90);

INSERT INTO Audit.AuditSeverity (Code, Name, DisplayOrder)
VALUES ('INFO','Info',10), ('NOTICE','Notice',20), ('WARNING','Warning',30),
('HIGH','High',40), ('CRITICAL','Critical',50);

INSERT INTO Audit.ChangeSourceType (Code, Name, DisplayOrder)
VALUES ('WEB UI','Web UI',10), ('API','API',20), ('BACKGROUND JOB','Background job',30),
('MIGRATION','Migration',40), ('INTEGRATION','Integration',50), ('SYSTEM','System',90);

INSERT INTO Audit.AccessResult (Code, Name, DisplayOrder)
VALUES ('GRANTED','Granted',10), ('DENIED','Denied',20), ('CHALLENGED','Challenged',30),
('LOCKED_OUT','Locked out',40);

```

C.17.11 Verification queries

```

-- Count audit entries by target table in the last 24 hours.
SELECT EntitySchema, EntityTable, COUNT(*) AS EntryCount
FROM Audit.AuditEntry
WHERE OccurredAtUtc >= DATEADD(day, -1, SYSUTCDATETIME())
GROUP BY EntitySchema, EntityTable
ORDER BY EntryCount DESC;

-- Replay one entity's audit trail.
SELECT e.OccurredAtUtc, a.Code AS ActionCode, e.Summary, u.UserName
FROM Audit.AuditEntry AS e
JOIN Audit.AuditActionType AS a ON a.AuditActionTypeId = e.ActionTypeId
LEFT JOIN Security.ApplicationUser AS u ON u.ApplicationUserId = e.PerformedByUserId
WHERE e.EntitySchema = 'Maintenance'
AND e.EntityTable = 'WorkOrder'
AND e.EntityKeyText = 'WO-2026-0418'
ORDER BY e.OccurredAtUtc;

-- Find failed commands and integration messages.
SELECT CommandName, RequestedAtUtc, ErrorMessage
FROM Audit.CommandLog AS c
JOIN Audit.CommandStatus AS s ON s.CommandStatusId = c.CommandStatusId
WHERE s.Code IN ('FAILED','REJECTED','ROLLED_BACK')
ORDER BY RequestedAtUtc DESC;

-- Show lineage for a report or exported artefact.
SELECT dl.LineageCode, ll.SourceSchema, ll.SourceTable, ll.TargetSchema, ll.TargetTable, ll.TransformationName
FROM Audit.DataLineage AS dl
JOIN Audit.LineageLink AS ll ON ll.DataLineageId = dl.DataLineageId
WHERE dl.LineageCode = 'RPT-DAILY-SAFETY-2026-06-29';

```

C.17.12 Honest boundary

Audit is not a substitute for business history. Temporal tables, incident histories and domain-specific state transitions still belong in their owning schemas. Audit is the cross-cutting trail that proves a change occurred and makes it replayable. It does not replace the meaning of the domain row.

The schema is written for the NEXUS-1 demonstrator and for SQL Server / EF Core friendly implementation. It is not safety-class software and it is not connected to any real plant. Its value is architectural: the entire platform can be explained, queried, exported and reviewed without losing the chain from actor to command to entity change to evidence.

Epilogue

The schema holds the twin

A digital twin is often imagined as a visual object: a model, a 3D plant, a dashboard, a simulation surface. But beneath every useful twin is a quieter thing: a schema that decides what exists, how it is named, where it belongs, and how its history is remembered.

The NEXUS-1 data backbone began with the smallest software object - a table - and grew outward until it could hold a plant, its signals, its alarms, its events, its maintenance history, its causal reasoning, its learned policy, its robots, its radiation records, its emergency plans, its compliance evidence, its reports and its audit trail. The point is not the table count. The point is that every sector knows what it owns, and every crossing between sectors is visible.

That is the discipline this book exists to show. The digital twin is not made trustworthy by a beautiful screen. It is made trustworthy when the screen can be traced back to rows, keys, constraints, measurements, evidence and audit entries that can be inspected after the fact.

The twin becomes serious when it can say not only what it believes, but which table, which row, which signal, which version, and which witness made it believe it.

This book closes the schema arc of the NEXUS-1 series, but it does not close the system. A schema this large is not a monument; it is a maintained artefact. New screens will ask for new tables. New evidence will require new links. New mistakes will reveal missing constraints. That is not failure. That is the work. A living model needs a living backbone, and the backbone is only honest if it can change without losing its memory.

References used to write this book

This volume was written from the NEXUS-1 companion material, the Phase-0 console, and standard software/database engineering references. The list below names the sources and bodies of documentation that shaped the schema decisions.

1. Agathangelidis, Grigorios. *From Grid to Core: How a Nuclear Power Plant Works, for the Engineer Who Has Never Seen One*. NEXUS-1 Companion Series, 2026.
2. Agathangelidis, Grigorios. *From Flood to Cause: Deterministic, Grounded, and Auditable Root-Cause Analysis for Alarm-Driven Systems*. NEXUS-1 Companion Series, 2026.
3. Agathangelidis, Grigorios. *From Trial to Policy: Reinforcement Learning from Scratch - Interpretable Q-Learning for Reactor Control*. NEXUS-1 Companion Series, 2026.
4. Agathangelidis, Grigorios. *From Table to Twin: Designing the Data Backbone of an Industrial Digital Twin*. NEXUS-1 Companion Series, 2026.
5. NEXUS-1 Phase-0 browser console, demonstration build NX1-094C79E0A366D34F. Local single-file application source: [index.html](#).
6. Microsoft Learn. SQL Server documentation: schemas, constraints, identity columns, datetime2, rowversion, indexes, temporal tables, transactions and query performance.
7. Microsoft Learn. Entity Framework Core documentation: reverse engineering, Fluent API configuration, relationships, indexes, migrations, concurrency tokens and value conversion.
8. Microsoft Learn. ASP.NET Core Identity documentation: users, roles, claims, tokens and policy-based authorization.
9. ISA-18.2 / IEC 62682 alarm-management principles, used as background for the alarm-management and alarm-flood portions of the schema.
10. EEMUA 191, alarm systems guidance, used as background for alarm rationalisation and alarm lifecycle thinking.
11. NUREG-0700, human-system interface design review guidance, used as background for control-room and operator-facing terminology.
12. General relational modelling practice: normalization, referential integrity, lookup tables, audit logging, soft delete, append-only facts, and bounded-context ownership.

The external standards above are named as design references and background influences. This book does not claim compliance, certification, licensing suitability, or operational authority.

Indexes

Index of schemas and chapters

SCHEMA	CHAPTER	PURPOSE
Schema-wide Conventions and Master Map	C.0	Schema-wide Conventions and Master Map
CorePlatform	C.1	CorePlatform Schema
Security	C.2	Security Schema
Organization	C.3	Organization Schema
ReactorFleet	C.4	ReactorFleet Schema
Instrumentation	C.5	Instrumentation Schema
DigitalTwin	C.6	DigitalTwin Schema
AlarmManagement	C.7	AlarmManagement Schema
EventManagerment	C.8	EventManagerment Schema
Maintenance	C.9	Maintenance Schema
RootCause	C.10	RootCause Schema
ReinforcementLearning	C.11	ReinforcementLearning Schema
Robotics	C.12	Robotics Schema
RadiationMonitoring	C.13	RadiationMonitoring Schema
EmergencyPreparedness	C.14	EmergencyPreparedness Schema
Compliance	C.15	Compliance Schema
Reporting	C.16	Reporting Schema
Audit	C.17	Audit Schema

Subject index

Access grants - C.2, C.16

Alarm definitions - C.7

Alarm floods - C.7, C.10

Append-only records - C.0, C.7, C.17

Application users and roles - C.2

Audit trail - C.17

Calibration - C.5

Causal graph - C.10

Compliance controls - C.15

Configuration - C.1

Control rods - C.4

Core platform reference data - C.1

Dashboards - C.16
Data lineage - C.17
Digital twin bindings - C.6
Divergence tracking - C.6
Dose records - C.13
Emergency drills - C.14
Engineering units - C.1
Equipment registry - C.4, C.9
Events and incidents - C.8
Evidence - C.10, C.15, C.17
Exports - C.16, C.17
Foreign-key passports - C.0
Historian readings - C.5
Identity and permissions - C.2
Instrumentation signals - C.5
KPI snapshots - C.16
Lookup tables - C.0-C.17
Maintenance work orders - C.9

Organizations and sites - C.3
Q-table - C.11
Radiation zones - C.13
Reactor fleet hierarchy - C.4
Reporting datasets - C.16
Root-cause hypotheses - C.10
Robotics missions - C.12
ROWVERSION - C.0
Security sessions - C.2
Soft delete - C.0
SQL Server DDL - C.0-C.17
Telemetry witnesses - C.10
Temporal table registry - C.17
Training episodes - C.11
UTC timestamps - C.0
Verification queries - C.1-C.17
Work-order materials - C.9

Table index by schema

The table index is generated from the `CREATE TABLE` statements in the schema chapters. It points to the chapter section rather than to a page number so the index remains stable as the book is revised.

AlarmManagement

AlarmManagement.AlarmAckMethod - C.7
AlarmManagement.AlarmAcknowledgement - C.7
AlarmManagement.AlarmCategory - C.7
AlarmManagement.AlarmClass - C.7
AlarmManagement.AlarmComment - C.7
AlarmManagement.AlarmCondition - C.7
AlarmManagement.AlarmConditionType - C.7
AlarmManagement.AlarmDeadband - C.7
AlarmManagement.AlarmDefinition - C.7
AlarmManagement.AlarmDirection - C.7
AlarmManagement.AlarmEscalation - C.7
AlarmManagement.AlarmEscalationAction - C.7

AlarmManagement.AlarmEvent - C.7
AlarmManagement.AlarmEventEvidence - C.7
AlarmManagement.AlarmEventStateHistory - C.7
AlarmManagement.AlarmFlood - C.7
AlarmManagement.AlarmFloodMember - C.7
AlarmManagement.AlarmFloodRanking - C.7
AlarmManagement.AlarmFloodStatus - C.7
AlarmManagement.AlarmFloodSummary - C.7
AlarmManagement.AlarmKpiSnapshot - C.7
AlarmManagement.AlarmLifecycleState - C.7
AlarmManagement.AlarmLimit - C.7
AlarmManagement.AlarmNotification - C.7
AlarmManagement.AlarmOperatorAction - C.7
AlarmManagement.AlarmPriority - C.7
AlarmManagement.AlarmRationalization - C.7
AlarmManagement.AlarmRationalizationReason - C.7
AlarmManagement.AlarmRoutingRecipient - C.7
AlarmManagement.AlarmRoutingRule - C.7
AlarmManagement.AlarmSeverity - C.7
AlarmManagement.AlarmShelving - C.7
AlarmManagement.AlarmShelvingReason - C.7
AlarmManagement.AlarmSourceType - C.7
AlarmManagement.AlarmStandingOrder - C.7
AlarmManagement.AlarmState - C.7
AlarmManagement.AlarmSuppression - C.7
AlarmManagement.AlarmSuppressionReason - C.7

AlarmManagement.AnnunciatorPanel - C.7
AlarmManagement.AnnunciatorTile - C.7
AlarmManagement.EscalationStatus - C.7
AlarmManagement.KpiMetricType - C.7
AlarmManagement.NotificationChannel - C.7
AlarmManagement.OperatorActionType - C.7
AlarmManagement.RationalizationStatus - C.7

Audit

Audit.AccessAudit - C.17
Audit.AccessResult - C.17
Audit.AuditActionType - C.17
Audit.AuditBatch - C.17
Audit.AuditEntityType - C.17
Audit.AuditEntry - C.17
Audit.AuditEvidenceLink - C.17
Audit.AuditSeverity - C.17
Audit.ChangeSourceType - C.17
Audit.CommandLog - C.17
Audit.CommandStatus - C.17
Audit.DataLineage - C.17
Audit.EntityChange - C.17
Audit.ExportAudit - C.17
Audit.ExportPurpose - C.17
Audit.IntegrationMessage - C.17
Audit.IntegrationMessageAttempt - C.17
Audit.IntegrationMessageStatus - C.17
Audit.LineageLink - C.17
Audit.PropertyChange - C.17
Audit.RetentionActionType - C.17
Audit.RetentionExecution - C.17
Audit.RetentionPolicy - C.17
Audit.RetentionRuleStatus - C.17
Audit.SignatureRecord - C.17
Audit.SignatureStatus - C.17
Audit.TemporalTableRegistry - C.17

Compliance

Compliance.Attestation - C.15
Compliance.AttestationStatus - C.15
Compliance.Audit - C.15

Compliance.AuditFinding - C.15
Compliance.AuditRoleType - C.15
Compliance.AuditScope - C.15
Compliance.AuditScopeType - C.15
Compliance.AuditStatus - C.15
Compliance.AuditTeamMember - C.15
Compliance.AuditType - C.15
Compliance.ComplianceProgram - C.15
Compliance.ComplianceProgramControl - C.15
Compliance.ComplianceProgramStatus - C.15
Compliance.ComplianceReview - C.15
Compliance.Control - C.15
Compliance.ControlFrequency - C.15
Compliance.ControlRequirementMap - C.15
Compliance.ControlStatus - C.15
Compliance.ControlType - C.15
Compliance.CorrectiveAction - C.15
Compliance.CorrectiveActionStatus - C.15
Compliance.Evidence - C.15
Compliance.EvidenceLink - C.15
Compliance.EvidenceStatus - C.15
Compliance.EvidenceType - C.15
Compliance.FindingSeverity - C.15
Compliance.FindingStatus - C.15
Compliance.Jurisdiction - C.15
Compliance.NonConformance - C.15
Compliance.NonConformanceType - C.15
Compliance.ObligationCalendar - C.15
Compliance.ObligationStatus - C.15
Compliance.Regulation - C.15
Compliance.RegulationType - C.15
Compliance.RegulatoryBody - C.15
Compliance.Requirement - C.15
Compliance.RequirementPriority - C.15
Compliance.RequirementStatus - C.15

Compliance.ReviewFinding - C.15
Compliance.ReviewStatus - C.15
Compliance.RiskRating - C.15
Compliance.Waiver - C.15
Compliance.WaiverReason - C.15
Compliance.WaiverStatus - C.15

CorePlatform

CorePlatform.AppSetting - C.1
CorePlatform.Calendar - C.1
CorePlatform.Country - C.1
CorePlatform.EngineeringUnit - C.1
CorePlatform.FeatureFlag - C.1
CorePlatform.Language - C.1
CorePlatform.Localization - C.1
CorePlatform.Region - C.1
CorePlatform.SystemConfiguration - C.1
CorePlatform.TimeZone - C.1
CorePlatform.Version - C.1

DigitalTwin

DigitalTwin.BindingRole - C.6
DigitalTwin.BindingStatus - C.6
DigitalTwin.DivergenceSeverity - C.6
DigitalTwin.DivergenceStatus - C.6
DigitalTwin.ModelAssumption - C.6
DigitalTwin.ModelParameterType - C.6
DigitalTwin.ModelVariableType - C.6
DigitalTwin.SignalBinding - C.6
DigitalTwin.SignalBindingCalibration - C.6
DigitalTwin.SimulationRun - C.6
DigitalTwin.SimulationRunOutput - C.6
DigitalTwin.SimulationRunStatus - C.6
DigitalTwin.SimulationRunStep - C.6
DigitalTwin.SimulationScenario - C.6
DigitalTwin.SimulationScenarioInput - C.6
DigitalTwin.SimulationScenarioType - C.6
DigitalTwin.SnapshotReason - C.6
DigitalTwin.SolverType - C.6

DigitalTwin.SynchronizationStatus - C.6
DigitalTwin.TwinAnnotation - C.6
DigitalTwin.TwinDivergence - C.6
DigitalTwin.TwinDivergenceReview - C.6
DigitalTwin.TwinFidelityLevel - C.6
DigitalTwin.TwinHealthCheck - C.6
DigitalTwin.TwinModel - C.6
DigitalTwin.TwinModelComponent - C.6
DigitalTwin.TwinModelStatus - C.6
DigitalTwin.TwinModelType - C.6
DigitalTwin.TwinModelValidation - C.6
DigitalTwin.TwinModelVersion - C.6
DigitalTwin.TwinParameter - C.6
DigitalTwin.TwinRuntimeSession - C.6
DigitalTwin.TwinSnapshot - C.6
DigitalTwin.TwinSnapshotValue - C.6
DigitalTwin.TwinStateVector - C.6
DigitalTwin.TwinSynchronization - C.6
DigitalTwin.TwinValidationMetric - C.6
DigitalTwin.TwinVariable - C.6
DigitalTwin.ValidationStatus - C.6
DigitalTwin.WhatIfCase - C.6
DigitalTwin.WhatIfCaseInput - C.6
DigitalTwin.WhatIfCaseResult - C.6

EmergencyPreparedness

EmergencyPreparedness.AssemblyPoint - C.14
EmergencyPreparedness.CommunicationStatus - C.14
EmergencyPreparedness.EmergencyClassType - C.14
EmergencyPreparedness.EmergencyClassification - C.14
EmergencyPreparedness.EmergencyCommunication - C.14
EmergencyPreparedness.EmergencyPlan - C.14
EmergencyPreparedness.EmergencyPlanRevision - C.14
EmergencyPreparedness.EmergencyResource - C.14

EmergencyPreparedness.EmergencyResource - C.14
EmergencyPreparedness.EmergencySeverity - C.14
EmergencyPreparedness.EvacuationRoute - C.14
EmergencyPreparedness.EvacuationRouteType - C.14
EmergencyPreparedness.EvaluationStatus - C.14
EmergencyPreparedness.Exercise - C.14
EmergencyPreparedness.ExerciseEvaluation - C.14
EmergencyPreparedness.ExerciseInject - C.14
EmergencyPreparedness.ExerciseObservation - C.14
EmergencyPreparedness.ExerciseParticipant - C.14
EmergencyPreparedness.ExerciseStatus - C.14
EmergencyPreparedness.ExerciseType - C.14
EmergencyPreparedness.InjectType - C.14
EmergencyPreparedness.MusterRecord - C.14
EmergencyPreparedness.MusterStatus - C.14
EmergencyPreparedness.NotificationChannel - C.14
EmergencyPreparedness.NotificationTemplate - C.14
EmergencyPreparedness.ObservationSeverity - C.14
EmergencyPreparedness.ParticipantStatus - C.14
EmergencyPreparedness.PlanAlarmLink - C.14
EmergencyPreparedness.PlanIncidentLink - C.14
EmergencyPreparedness.PlanRobotMissionLink - C.14
EmergencyPreparedness.PlanStatus - C.14
EmergencyPreparedness.ProtectiveAction - C.14
EmergencyPreparedness.ProtectiveActionStatus - C.14
EmergencyPreparedness.ProtectiveActionType - C.14
EmergencyPreparedness.ReadinessStatus - C.14
EmergencyPreparedness.ResourceReadiness - C.14
EmergencyPreparedness.ResourceStatus - C.14
EmergencyPreparedness.ResourceType - C.14
EmergencyPreparedness.ResponseTeam - C.14
EmergencyPreparedness.ResponseTeamMember - C.14
EmergencyPreparedness.RouteStatus - C.14
EmergencyPreparedness.ScenarioStatus - C.14

EventManagement

EventManagement.BarrierType - C.8
EventManagement.EventAlarmLink - C.8
EventManagement.EventAttachment - C.8
EventManagement.EventAttachmentType - C.8
EventManagement.EventEquipmentLink - C.8
EventManagement.EventFloodLink - C.8
EventManagement.EventImpactAssessment - C.8
EventManagement.EventImpactType - C.8
EventManagement.EventNote - C.8
EventManagement.EventNoteType - C.8
EventManagement.EventParticipant - C.8
EventManagement.EventParticipantRole - C.8
EventManagement.EventRelationship - C.8
EventManagement.EventRelationshipType - C.8
EventManagement.EventSeverity - C.8
EventManagement.EventSignalLink - C.8
EventManagement.EventSourceType - C.8
EventManagement.EventStatus - C.8
EventManagement.EventStatusHistory - C.8
EventManagement.EventTag - C.8
EventManagement.EventTagAssignment - C.8

EventManagement.EventTimelineEntry - C.8
EventManagement.EventTimelineEntryType - C.8
EventManagement.EventType - C.8
EventManagement.EventUnitStateSnapshot - C.8
EventManagement.Incident - C.8
EventManagement.IncidentAction - C.8
EventManagement.IncidentActionOwner - C.8
EventManagement.IncidentActionStatus - C.8
EventManagement.IncidentActionType - C.8
EventManagement.IncidentClassification - C.8
EventManagement.IncidentClassificationOnAssignment - C.8
EventManagement.IncidentReview - C.8
EventManagement.IncidentReviewFinding - C.8
EventManagement.IncidentReviewOutcome - C.8
EventManagement.IncidentStatus - C.8
EventManagement.IncidentType - C.8
EventManagement.NearMiss - C.8
EventManagement.NearMissBarrier - C.8
EventManagement.NearMissType - C.8
EventManagement.OperationalEvent - C.8

Instrumentation

Instrumentation.AcquisitionConnection - C.5
Instrumentation.AcquisitionPoint - C.5
Instrumentation.BackfillJobStatus - C.5
Instrumentation.CalibrationPlan - C.5
Instrumentation.CalibrationRecord - C.5
Instrumentation.CalibrationStatus - C.5
Instrumentation.ChannelStatus - C.5
Instrumentation.DataAcquisitionNode - C.5

Instrumentation.DataGap - C.5
Instrumentation.HistorianBackfillJob - C.5
Instrumentation.HistorianImportBatch - C.5
Instrumentation.HistorianImportBatchClass - C.5
Instrumentation.HistorianRetentionClass - C.5
Instrumentation.HistorianRetentionPolicy - C.5
Instrumentation.Instrument - C.5
Instrumentation.LineageRole - C.5
Instrumentation.Measurement - C.5
Instrumentation.MeasurementAggregate - C.5
Instrumentation.MeasurementAnnotation - C.5
Instrumentation.MeasurementAssignment - C.5
Instrumentation.MeasurementSource - C.5
Instrumentation.SamplingMode - C.5
Instrumentation.Sensor - C.5
Instrumentation.SensorChannel - C.5
Instrumentation.SensorStatus - C.5
Instrumentation.SensorType - C.5
Instrumentation.Signal - C.5
Instrumentation.SignalAlias - C.5
Instrumentation.SignalCategory - C.5
Instrumentation.SignalDeadband - C.5
Instrumentation.SignalDependency - C.5
Instrumentation.SignalGroup - C.5
Instrumentation.SignalGroupMember - C.5
Instrumentation.SignalLimit - C.5
Instrumentation.SignalLineage - C.5
Instrumentation.SignalMapping - C.5
Instrumentation.SignalQuality - C.5
Instrumentation.SignalQualityEvent - C.5
Instrumentation.SignalRole - C.5
Instrumentation.SignalType - C.5
Instrumentation.ThresholdType - C.5

Maintenance

Maintenance.ApprovalStatus - C.9
Maintenance.Asset - C.9
Maintenance.AssetCategory - C.9
Maintenance.AssetComponent - C.9
Maintenance.AssetCondition - C.9
Maintenance.AssetConditionMeasurement - C.9
Maintenance.AssetCriticality - C.9
Maintenance.AssetSparePart - C.9
Maintenance.AssetStatus - C.9
Maintenance.AssetStatusHistory - C.9
Maintenance.ConditionGrade - C.9
Maintenance.DegradationMechanism - C.9
Maintenance.DegradationRecord - C.9
Maintenance.DegradationTrendPoint - C.9
Maintenance.DocumentType - C.9
Maintenance.FindingSeverity - C.9
Maintenance.Inspection - C.9
Maintenance.InspectionFinding - C.9
Maintenance.InspectionType - C.9
Maintenance.LabourRole - C.9
Maintenance.MaintenanceApproval - C.9
Maintenance.MaintenanceDocument - C.9
Maintenance.MaintenancePlan - C.9
Maintenance.MaintenancePlanTask - C.9
Maintenance.MaintenancePlanType - C.9
Maintenance.MaintenanceSchedule - C.9
Maintenance.MaintenanceScheduleOccurrence - C.9
Maintenance.MaintenanceWindow - C.9
Maintenance.MaterialUsageType - C.9
Maintenance.ScheduleBasis - C.9
Maintenance.SparePart - C.9

Maintenance.SparePartStock - C.9
Maintenance.Supplier - C.9
Maintenance.SupplierPart - C.9
Maintenance.SupplierStatus - C.9
Maintenance.TaskStatus - C.9
Maintenance.WorkOrder - C.9
Maintenance.WorkOrderEventLink - C.9
Maintenance.WorkOrderLabour - C.9
Maintenance.WorkOrderMaterial - C.9
Maintenance.WorkOrderStatus - C.9
Maintenance.WorkOrderStatusHistory - C.9
Maintenance.WorkOrderTask - C.9
Maintenance.WorkOrderType - C.9
Maintenance.WorkPriority - C.9

Organization

Organization.Building - C.3
Organization.Certification - C.3
Organization.ContactMethodType - C.3
Organization.ContractorEngagement - C.3
Organization.Department - C.3
Organization.DepartmentAssignment - C.3
Organization.DepartmentType - C.3
Organization.Employment - C.3
Organization.EmploymentStatus - C.3
Organization.LegalEntity - C.3
Organization.LegalEntityType - C.3
Organization.OnCallRoster - C.3
Organization.Person - C.3
Organization.PersonAddress - C.3
Organization.PersonCertification - C.3
Organization.PersonContact - C.3
Organization.PersonQualification - C.3
Organization.PersonType - C.3
Organization.PersonnelRequirement - C.3
Organization.Plant - C.3
Organization.PlantType - C.3

Organization.Position - C.3
Organization.Qualification - C.3
Organization.QualificationStatus - C.3
Organization.Shift - C.3
Organization.ShiftAssignment - C.3
Organization.ShiftPattern - C.3
Organization.ShiftType - C.3
Organization.Site - C.3
Organization.SiteType - C.3
Organization.StaffingScenario - C.3
Organization.StaffingScenarioGap - C.3
Organization.StaffingScenarioRequirement - C.3
Organization.StaffingScenarioResult - C.3
Organization.Team - C.3
Organization.TeamMembership - C.3
Organization.TeamType - C.3

RadiationMonitoring

RadiationMonitoring.AlertStatus - C.13
RadiationMonitoring.ContaminationSample - C.13
RadiationMonitoring.ContaminationSampleResult - C.13
RadiationMonitoring.ContaminationType - C.13
RadiationMonitoring.DoseAlert - C.13
RadiationMonitoring.DoseLimit - C.13
RadiationMonitoring.DoseLimitAssignment - C.13
RadiationMonitoring.DoseType - C.13
RadiationMonitoring.Dosimeter - C.13
RadiationMonitoring.DosimeterStatus - C.13
RadiationMonitoring.DosimeterType - C.13
RadiationMonitoring.EnvironmentalSample - C.13
RadiationMonitoring.EnvironmentalSampleResult - C.13
RadiationMonitoring.EnvironmentalSamplingLocation - C.13
RadiationMonitoring.ExposureCategory - C.13
RadiationMonitoring.IsotopeInventory - C.13
RadiationMonitoring.LimitType - C.13
RadiationMonitoring.MeasurementQuality - C.13
RadiationMonitoring.MeasurementType - C.13
RadiationMonitoring.MonitorStatus - C.13
RadiationMonitoring.MonitorType - C.13
RadiationMonitoring.PersonDoseReading - C.13
RadiationMonitoring.PersonDosimeterAssignment - C.13
RadiationMonitoring.RadiationAreaClassification - C.13
RadiationMonitoring.RadiationEventLink - C.13
RadiationMonitoring.RadiationMonitor - C.13
RadiationMonitoring.RadiationMonitorCalibration - C.13
RadiationMonitoring.RadiationMonitorSignature - C.13
RadiationMonitoring.RadiationReading - C.13
RadiationMonitoring.RadiationRobotMission - C.13
RadiationMonitoring.RadiationTrendSummary - C.13
RadiationMonitoring.RadiationWorkPermit - C.13
RadiationMonitoring.RadiationWorkPermitDetails - C.13
RadiationMonitoring.RadiationWorkPermitForm - C.13
RadiationMonitoring.RadiationWorkPermitZone - C.13
RadiationMonitoring.RadiationZone - C.13
RadiationMonitoring.RadiationZoneAccess - C.13
RadiationMonitoring.RadiationZoneBoundary - C.13
RadiationMonitoring.RadiationZoneStatus - C.13
RadiationMonitoring.RadiationZoneType - C.13
RadiationMonitoring.RadiologicalSurvey - C.13
RadiationMonitoring.RadiologicalSurveyResults - C.13
RadiationMonitoring.SampleType - C.13

RadiationMonitoring.SurveyStatus - C.13
RadiationMonitoring.SurveyType - C.13
RadiationMonitoring.ZoneDoseSnapshot - C.13

ReactorFleet

ReactorFleet.BoronChemistrySnapshot - C.4
ReactorFleet.ControlRod - C.4
ReactorFleet.ControlRodBank - C.4
ReactorFleet.ControlRodMaterial - C.4
ReactorFleet.CoolantLoopType - C.4
ReactorFleet.CoreRegion - C.4
ReactorFleet.Equipment - C.4
ReactorFleet.EquipmentCriticality - C.4
ReactorFleet.EquipmentDependency - C.4
ReactorFleet.EquipmentExternalReference - C.4
ReactorFleet.EquipmentLocation - C.4
ReactorFleet.EquipmentStatus - C.4
ReactorFleet.EquipmentType - C.4
ReactorFleet.Fleet - C.4
ReactorFleet.FleetPlant - C.4
ReactorFleet.FleetSnapshot - C.4
ReactorFleet.FuelAssembly - C.4
ReactorFleet.FuelAssemblyType - C.4
ReactorFleet.FuelMaterialType - C.4
ReactorFleet.Generator - C.4
ReactorFleet.GridConnection - C.4
ReactorFleet.GridConnectionStatus - C.4
ReactorFleet.NeutronicsParameterSet - C.4
ReactorFleet.OperationalMode - C.4
ReactorFleet.PlantSystem - C.4
ReactorFleet.Pressurizer - C.4
ReactorFleet.PrimaryLoop - C.4
ReactorFleet.PumpType - C.4
ReactorFleet.Reactor - C.4

ReactorFleet.ReactorCoolantPump - C.4
ReactorFleet.ReactorCore - C.4
ReactorFleet.ReactorType - C.4
ReactorFleet.RodBankType - C.4
ReactorFleet.RodPositionSnapshot - C.4
ReactorFleet.SteamGenerator - C.4
ReactorFleet.SteamGeneratorType - C.4
ReactorFleet.SwitchyardBay - C.4
ReactorFleet.SystemDependency - C.4
ReactorFleet.SystemType - C.4
ReactorFleet.Transformer - C.4
ReactorFleet.TurbineStage - C.4
ReactorFleet.TurbineTrain - C.4
ReactorFleet.Unit - C.4
ReactorFleet.UnitDesignParameter - C.4
ReactorFleet.UnitOperationalState - C.4
ReactorFleet.UnitPowerSnapshot - C.4
ReactorFleet.UnitStatus - C.4
ReactorFleet.UnitType - C.4

ReinforcementLearning

ReinforcementLearning.ActionDefinition - C.11
ReinforcementLearning.ActionSpace - C.11
ReinforcementLearning.ActionSpaceType - C.11
ReinforcementLearning.AdvisoryMode - C.11
ReinforcementLearning.AdvisoryRecommendation - C.11
ReinforcementLearning.AdvisoryRecommendationReason - C.11
ReinforcementLearning.AdvisorySession - C.11
ReinforcementLearning.EnvironmentModel - C.11
ReinforcementLearning.EnvironmentModelType - C.11
ReinforcementLearning.Episode - C.11
ReinforcementLearning.EpisodeStatus - C.11
ReinforcementLearning.EpisodeStep - C.11
ReinforcementLearning.EvaluationMetric - C.11

ReinforcementLearning.Experiment - C.11
ReinforcementLearning.HyperparameterSet - C.11
ReinforcementLearning.LearningAlgorithm - C.11
ReinforcementLearning.ModelArtifact - C.11
ReinforcementLearning.Policy - C.11
ReinforcementLearning.PolicyDeployment - C.11
ReinforcementLearning.PolicyEntry - C.11
ReinforcementLearning.PolicyEvaluation - C.11
ReinforcementLearning.PolicyEvaluationMetric - C.11
ReinforcementLearning.PolicyStatus - C.11
ReinforcementLearning.QTable - C.11
ReinforcementLearning.QTableEntry - C.11
ReinforcementLearning.RecommendationSet - C.11
ReinforcementLearning.RewardFunction - C.11
ReinforcementLearning.RewardFunctionType - C.11
ReinforcementLearning.SafetyClamp - C.11
ReinforcementLearning.SafetyClampRule - C.11
ReinforcementLearning.SafetyClampType - C.11
ReinforcementLearning.StateDefinition - C.11
ReinforcementLearning.StateSpace - C.11
ReinforcementLearning.StateSpaceType - C.11
ReinforcementLearning.RecommendationReason - C.11
ReinforcementLearning.TrainingRun - C.11
ReinforcementLearning.TrainingRunAuditTrail - C.11
ReinforcementLearning.TrainingRunStatus - C.11

Reporting

Reporting.AccessLevel - C.16
Reporting.Dashboard - C.16
Reporting.DashboardWidget - C.16
Reporting.DatasetColumn - C.16

Reporting.DatasetDefinition - C.16
Reporting.DatasetType - C.16
Reporting.DeliveryChannel - C.16
Reporting.DistributionList - C.16
Reporting.DistributionListMember - C.16
Reporting.ExportStatus - C.16
Reporting.KpiCategory - C.16
Reporting.KpiDefinition - C.16
Reporting.KpiSnapshot - C.16
Reporting.KpiStatus - C.16
Reporting.ReportAccessGrant - C.16
Reporting.ReportDefinition - C.16
Reporting.ReportExport - C.16
Reporting.ReportExportFile - C.16
Reporting.ReportFormat - C.16
Reporting.ReportParameter - C.16
Reporting.ReportRun - C.16
Reporting.ReportRunParameter - C.16
Reporting.ReportRunStatus - C.16
Reporting.ReportSchedule - C.16
Reporting.ReportSnapshotArchive - C.16
Reporting.ReportSourceDependency - C.16
Reporting.ReportSubscription - C.16
Reporting.ReportType - C.16
Reporting.SavedQuery - C.16
Reporting.ScheduleFrequency - C.16
Reporting.SubscriptionStatus - C.16
Reporting.WidgetType - C.16

Robotics

Robotics.BatteryStatus - C.12
Robotics.ChargingSession - C.12
Robotics.CommunicationStatus - C.12
Robotics.DockingStation - C.12
Robotics.DockingStationStatus - C.12
Robotics.Mission - C.12
Robotics.MissionChecklist - C.12
Robotics.MissionChecklistItem - C.12
Robotics.MissionEvent - C.12

Robotics.MissionOperationalEventLink - C.12
Robotics.MissionPayloadUsage - C.12
Robotics.MissionPriority - C.12
Robotics.MissionReadinessAssessment - C.12
Robotics.MissionReadinessItem - C.12
Robotics.MissionRobotAssignment - C.12
Robotics.MissionRoute - C.12
Robotics.MissionRouteWaypoint - C.12
Robotics.MissionStatus - C.12
Robotics.MissionTask - C.12
Robotics.MissionTaskStatus - C.12
Robotics.MissionType - C.12
Robotics.PayloadType - C.12
Robotics.ReadinessStatus - C.12
Robotics.Robot - C.12
Robotics.RobotCapability - C.12
Robotics.RobotCapabilityAssignment - C.12
Robotics.RobotCapabilityType - C.12
Robotics.RobotCommunicationLink - C.12
Robotics.RobotHealthSnapshot - C.12
Robotics.RobotLocationHistory - C.12
Robotics.RobotMaintenanceLink - C.12
Robotics.RobotModel - C.12
Robotics.RobotPayload - C.12
Robotics.RobotSensorBinding - C.12
Robotics.RobotStatus - C.12
Robotics.RobotTelemetry - C.12
Robotics.RobotType - C.12
Robotics.RobotWorkEnvelope - C.12
Robotics.RouteType - C.12
Robotics.TelemetryQuality - C.12
Robotics.WaypointType - C.12

RootCause

RootCause.AbstentionReason - C.10
RootCause.AnalysisAlarm - C.10
RootCause.AnalysisAuditTrail - C.10

RootCause.AnalysisCandidate - C.10
RootCause.AnalysisComment - C.10
RootCause.AnalysisConclusion - C.10
RootCause.AnalysisEventLink - C.10
RootCause.AnalysisHypothesis - C.10
RootCause.AnalysisModelOutput - C.10
RootCause.AnalysisModelRun - C.10
RootCause.AnalysisStatus - C.10
RootCause.AnalysisTag - C.10
RootCause.AnalysisTagAssignment - C.10
RootCause.AnalysisValidationCheck - C.10
RootCause.CandidateRole - C.10
RootCause.CandidateStatus - C.10
RootCause.CausalEdge - C.10
RootCause.CausalEdgeDelayWindow - C.10
RootCause.CausalEdgeEvidence - C.10
RootCause.CausalEdgeType - C.10
RootCause.CausalGraph - C.10
RootCause.CausalGraphChangeRequest - C.10
RootCause.CausalGraphChangeReview - C.10
RootCause.CausalGraphVersion - C.10
RootCause.CausalNode - C.10
RootCause.CausalNodeEquipmentLink - C.10
RootCause.CausalNodeSignalLink - C.10
RootCause.CausalNodeType - C.10
RootCause.CausalPath - C.10
RootCause.CausalPathStep - C.10
RootCause.ConfidenceBand - C.10
RootCause.DocumentWitness - C.10
RootCause.EdgeProvenanceType - C.10
RootCause.EdgeStatus - C.10
RootCause.EvidenceType - C.10
RootCause.GroundingContext - C.10

RootCause.GroundingContextDocument - C.10

RootCause.GroundingContextSignalWindow - C.10

RootCause.GroundingSourceType - C.10

RootCause.HypothesisEvidence - C.10

RootCause.HypothesisStatus - C.10

RootCause.RegistryWitness - C.10

RootCause.RejectedCandidate - C.10

RootCause.RootCauseAnalysis - C.10

RootCause.TelemetryWitness - C.10

RootCause.ValidationResult - C.10

RootCause.WitnessType - C.10

Security

Security.AccessReview - C.2

Security.AccessReviewItem - C.2

Security.AccessReviewStatus - C.2

Security.ApiClient - C.2

Security.ApiClientSecret - C.2

Security.ApplicationRole - C.2

Security.ApplicationUser - C.2

Security.ExternalLogin - C.2

Security.LoginAttempt - C.2

Security.LoginResult - C.2

Security.Permission - C.2

Security.PermissionCategory - C.2

Security.Policy - C.2

Security.PolicyPermission - C.2

Security.PolicyType - C.2

Security.RoleClaim - C.2

Security.RolePermission - C.2

Security.RolePolicy - C.2

Security.RoleType - C.2

Security.SecurityToken - C.2

Security.SessionStatus - C.2

Security.TokenType - C.2

Security.UserClaim - C.2

Security.UserPermissionOverride - C.2

Security.UserPolicyAssignment - C.2

Security.UserPreference - C.2

Security.UserRole - C.2

Security.UserSession - C.2

Security.UserStatus - C.2